
**STM32H523/533xx, STM32H543/553xx, STM32H562xx,
and STM32H563/573xx Arm[®]-based 32-bit MCUs**

Introduction

This document is addressed to application developers. It provides complete information on how to use the STM32H523xx, STM32H533xx, STM32H543xx, STM32H553xx, STM32H562xx, STM32H563xx, and STM32H573xx microcontrollers memory and peripherals.

For ordering information, mechanical and electrical device characteristics, refer to the corresponding datasheets.

For information on the Arm[®] Cortex[®]-M33 core, refer to the corresponding Arm[®] Technical Reference Manuals available on <http://infocenter.arm.com>.

Related documents

- STM32H523/533xx datasheet
- STM32H543/553xx datasheet
- STM32H562/563xx datasheet
- STM32H573xx datasheet
- STM32H523/533xx errata sheet
- STM32H543/553xx errata sheet
- STM32H562/563/573xx errata sheets
- STM32 Cortex[®]-M33 MCUs programming manual

Contents

1	Documentation conventions	119
1.1	General information	119
1.2	List of abbreviations for registers	119
1.3	Register reset value	120
1.4	Glossary	120
2	Memory and bus architecture	121
2.1	System architecture	121
2.1.1	Fast C-bus	122
2.1.2	Slow C-bus	123
2.1.3	S-bus	123
2.1.4	DCache S-bus	123
2.1.5	GPDMA1 and GPDMA2 buses	123
2.1.6	SDMMC1 and SDMMC2 controllers DMA buses	123
2.1.7	BusMatrix	123
2.1.8	AHB/APB bridges	124
2.1.9	Ethernet MAC	124
2.2	TrustZone security architecture	124
2.2.1	Default TrustZone security state	124
2.2.2	TrustZone peripheral classification	125
2.3	Memory organization	130
2.3.1	Introduction	130
2.3.2	Memory map and register boundary addresses	131
2.3.3	Embedded SRAMs	147
2.3.4	Flash memory overview	147
2.3.5	Boot modes	147
3	System security	149
3.1	Key security features	149
3.2	Secure install	150
3.3	Secure boot	151
3.3.1	Unique boot entry	151
3.3.2	Immutable root of trust in system flash memory	152

3.4	Secure update	152
3.5	Resource isolation using hide protection levels	153
3.6	Resource isolation using TrustZone	153
3.6.1	TrustZone security architecture	154
3.6.2	Armv8-M security extension of Cortex-M33	154
3.6.3	Memory and peripheral allocation using IDAU/SAU	154
3.6.4	Memory and peripheral allocation using GTZC	156
3.6.5	Managing security in TrustZone-aware peripherals	160
3.6.6	Activating TrustZone security	166
3.6.7	Deactivating TrustZone security	167
3.7	Other resources isolation	168
3.7.1	Temporal isolation using secure hide protection (HDP)	168
3.7.2	Resource isolation using Cortex privileged mode	168
3.8	Secure execution	172
3.8.1	Memory protection unit (MPU)	172
3.8.2	Embedded flash memory write protection	173
3.8.3	Tamper detection and response	173
3.9	Secure storage	175
3.9.1	Hardware secret key management	176
3.9.2	Unique ID	177
3.10	Crypto engines	177
3.10.1	Crypto engines features	177
3.10.2	Secure AES co-processor (SAES)	178
3.10.3	On-the-fly decryption engine (OTFDEC)	179
3.11	Product life cycle	179
3.11.1	Product configurations and security services	181
3.11.2	Life cycle management	181
3.11.3	Recommended product settings	184
3.12	Software intellectual property protection and collaborative development	184
3.12.1	Software intellectual property protection	186
3.12.2	Software intellectual property protection with OTFDEC	186
3.12.3	Other software intellectual property protections	188
4	Boot modes	189
4.1	STM32H523/43/62/63xx boot modes	190
4.2	STM32H533/53/73xx boot modes	191

5	Global TrustZone® controller (GTZC)	192
5.1	GTZC introduction	192
5.2	GTZC main features	192
5.3	GTZC implementation	194
5.4	GTZC functional description	196
5.4.1	GTZC block diagram	196
5.4.2	Illegal access definition	197
5.4.3	TrustZone security controller (TZSC)	198
5.4.4	Memory protection controller - block based (MPCBB)	199
5.4.5	TrustZone illegal access controller (TZIC)	200
5.4.6	Power-on/reset state	200
5.5	GTZC interrupts	200
5.6	GTZC1 TZSC registers	201
5.6.1	GTZC1 TZSC control register (GTZC1_TZSC_CR)	201
5.6.2	GTZC1 TZSC secure configuration register 1 (GTZC1_TZSC_SECCFGR1)	201
5.6.3	GTZC1 TZSC secure configuration register 2 (GTZC1_TZSC_SECCFGR2)	204
5.6.4	GTZC1 TZSC secure configuration register 3 (GTZC1_TZSC_SECCFGR3)	206
5.6.5	GTZC1 TZSC privilege configuration register 1 (GTZC1_TZSC_PRIVCFGR1)	209
5.6.6	GTZC1 TZSC privilege configuration register 2 (GTZC1_TZSC_PRIVCFGR2)	211
5.6.7	GTZC1 TZSC privilege configuration register 3 (GTZC1_TZSC_PRIVCFGR3)	213
5.6.8	GTZC1 TZSC memory x subregion A watermark configuration register (GTZC1_TZSC_MPCWMxACFGR)	216
5.6.9	GTZC1 TZSC memory x subregion A watermark register (GTZC1_TZSC_MPCWMxAR)	217
5.6.10	GTZC1 TZSC memory x subregion B watermark configuration register (GTZC1_TZSC_MPCWMxBCFGR)	217
5.6.11	GTZC1 TZSC memory x subregion B watermark register (GTZC1_TZSC_MPCWMxBR)	218
5.6.12	GTZC1 TZSC register map	220
5.7	GTZC1 TZIC registers	222
5.7.1	GTZC1 TZIC interrupt enable register 1 (GTZC1_TZIC_IER1)	222
5.7.2	GTZC1 TZIC interrupt enable register 2 (GTZC1_TZIC_IER2)	224
5.7.3	GTZC1 TZIC interrupt enable register 3 (GTZC1_TZIC_IER3)	226

5.7.4	GTZC1 TZIC interrupt enable register 4 (GTZC1_TZIC_IER4)	229
5.7.5	GTZC1 TZIC status register 1 (GTZC1_TZIC_SR1)	231
5.7.6	GTZC1 TZIC status register 2 (GTZC1_TZIC_SR2)	233
5.7.7	GTZC1 TZIC status register 3 (GTZC1_TZIC_SR3)	236
5.7.8	GTZC1 TZIC status register 4 (GTZC1_TZIC_SR4)	238
5.7.9	GTZC1 TZIC flag clear register 1 (GTZC1_TZIC_FCR1)	240
5.7.10	GTZC1 TZIC flag clear register 2 (GTZC1_TZIC_FCR2)	242
5.7.11	GTZC1 TZIC flag clear register 3 (GTZC1_TZIC_FCR3)	244
5.7.12	GTZC1 TZIC flag clear register 4 (GTZC1_TZIC_FCR4)	246
5.7.13	GTZC1 TZIC register map	249
5.8	GTZC1 MPCBBz registers (z = 1 to 3)	251
5.8.1	GTZC1 SRAMz MPCBB control register (GTZC1_MPCBBz_CR) (z = 1 to 3)	251
5.8.2	GTZC1 SRAMz MPCBB configuration lock register 1 (GTZC1_MPCBBz_CFGLOCK1) (z = 1 to 3)	251
5.8.3	GTZC1 SRAMz MPCBB security configuration for super-block x register (GTZC1_MPCBBz_SECCFGRx) (z = 1 to 3)	252
5.8.4	GTZC1 SRAMz MPCBB privileged configuration for super-block x register (GTZC1_MPCBBz_PRIVCFGRx) (z = 1 to 3)	252
5.8.5	GTZC1 MPCBBz register map (z = 1 to 3)	253
6	RAMs configuration controller (RAMCFG)	254
6.1	RAMCFG introduction	254
6.2	RAMCFG main features	254
6.3	RAMCFG functional description	254
6.3.1	Internal SRAMs features	254
6.3.2	Error code correction (SRAM2, SRAM3, BKPSRAM)	255
6.3.3	Write protection (SRAM2)	258
6.3.4	Software erase	259
6.4	RAMCFG low-power modes	259
6.5	RAMCFG interrupts	259
6.6	RAMCFG registers	260
6.6.1	RAMCFG memory x control register (RAMCFG_MxCR)	260
6.6.2	RAMCFG memory x interrupt enable register (RAMCFG_MxIER)	260
6.6.3	RAMCFG memory interrupt status register (RAMCFG_MxISR)	261
6.6.4	RAMCFG memory x ECC single error address register (RAMCFG_MxSEAR)	262

6.6.5	RAMCFG memory x ECC double error address register (RAMCFG_MxDEAR)	262
6.6.6	RAMCFG memory x interrupt clear register x (RAMCFG_MxICR) ...	262
6.6.7	RAMCFG memory 2 write protection register 1 (RAMCFG_M2WPR1)	263
6.6.8	RAMCFG memory 2 write protection register 2 (RAMCFG_M2WPR2)	263
6.6.9	RAMCFG memory 2 write protection register 3 (RAMCFG_M2WPR3)	264
6.6.10	RAMCFG memory x ECC key register (RAMCFG_MxECCKEYR) ...	264
6.6.11	RAMCFG memory x erase key register (RAMCFG_MxERKEYR) ...	265
6.6.12	RAMCFG register map	266
7	Embedded flash memory (FLASH)	269
7.1	FLASH introduction	269
7.2	FLASH main features	269
7.3	FLASH functional description	270
7.3.1	FLASH block diagram	270
7.3.2	FLASH signals	271
7.3.3	Flash memory architecture and usage	272
7.3.4	FLASH read operations	274
7.3.5	FLASH program operations	276
7.3.6	FLASH erase operations	279
7.3.7	FLASH parallel operations	282
7.3.8	FLASH error protections	283
7.3.9	OTP and RO memory access	283
7.3.10	Flash high-cycle data	285
7.3.11	Flash bank swapping	287
7.3.12	FLASH reset and clocks	291
7.4	FLASH option bytes	292
7.4.1	About option bytes	292
7.4.2	Option bytes loading	292
7.4.3	Option bytes modification	293
7.4.4	Description of user and system option bytes	296
7.4.5	Description of data protection option bytes	297
7.4.6	Description of boot address option bytes	297
7.4.7	Specific rules for modifying option bytes	298
7.5	Option bytes key (OBK) management	300

7.5.1	OBK loading	300
7.5.2	OBK access per HDPL level	300
7.5.3	OBK programming sequence	301
7.5.4	OBK programming finite state machine	302
7.5.5	OBK swap sector	303
7.5.6	OBK alternate sector erase	305
7.6	FLASH security and protections	305
7.6.1	TrustZone security protection	306
7.6.2	Hide protection (HDP)	308
7.6.3	Block-based secure flash memory area protection	310
7.6.4	Block-based privileged flash memory area protection	311
7.6.5	Flash memory register privileged and unprivileged modes	312
7.6.6	Flash memory banks attributes in case of bank swap	313
7.6.7	Flash memory configuration protection	314
7.6.8	Write protection	315
7.6.9	Flash high-cycle data protections	315
7.6.10	Life cycle management	317
7.6.11	Product state transitions	318
7.6.12	OBK protection	320
7.6.13	One-time-programmable and read-only memory protections	321
7.7	System memory	322
7.7.1	Introduction	322
7.7.2	RSS user functions	322
7.8	FLASH low-power modes	329
7.9	FLASH error management	330
7.9.1	Introduction	330
7.9.2	Nonsecure write protection error (WRPERR)	330
7.9.3	Secure write protection error (WRPERR)	331
7.9.4	Non secure programming sequence error (PGSERR)	332
7.9.5	Secure programming sequence error (PGSERR)	333
7.9.6	Nonsecure strobe error (STRBERR)	334
7.9.7	Secure strobe error (STRBERR)	334
7.9.8	Nonsecure inconsistency error (INCERR)	334
7.9.9	Secure inconsistency error (INCERR)	335
7.9.10	Error correction code error (ECCC, ECCD)	336
7.9.11	Illegal access (ILAFM/ILAP)	337
7.9.12	Option byte change error (OPTCHANGEERR)	337

7.9.13	Miscellaneous HardFault errors	338
7.9.14	OBK error cases (OBKERR, OBKWERR)	338
7.10	FLASH interrupts	339
7.11	FLASH registers	341
7.11.1	FLASH access control register (FLASH_ACR)	341
7.11.2	FLASH nonsecure key register (FLASH_NSKEYR)	342
7.11.3	FLASH secure key register (FLASH_SECKEYR)	342
7.11.4	FLASH option key register (FLASH_OPTKEYR)	343
7.11.5	FLASH nonsecure OBK key register (FLASH_NSOBKKEYR)	343
7.11.6	FLASH secure OBK key register (FLASH_SECOBKKEYR)	343
7.11.7	FLASH operation status register (FLASH_OPSR)	344
7.11.8	FLASH option control register (FLASH_OPTCR)	345
7.11.9	FLASH nonsecure status register (FLASH_NSSR)	346
7.11.10	FLASH secure status register (FLASH_SECSR)	348
7.11.11	FLASH nonsecure control register (FLASH_NSCR)	350
7.11.12	FLASH secure control register (FLASH_SECCR)	352
7.11.13	FLASH nonsecure clear control register (FLASH_NSCCR)	355
7.11.14	FLASH secure clear control register (FLASH_SECCCR)	356
7.11.15	FLASH privilege configuration register (FLASH_PRIVCFGR)	357
7.11.16	FLASH nonsecure OBK configuration register (FLASH_NSOBKCFGR)	357
7.11.17	FLASH secure OBK configuration register (FLASH_SECOBKCFGR)	358
7.11.18	FLASH HDP extension register (FLASH_HDPEXTR)	359
7.11.19	FLASH option status register (FLASH_OPTSR_CUR)	360
7.11.20	FLASH option status register (FLASH_OPTSR_PRG)	362
7.11.21	FLASH nonsecure EPOCH register (FLASH_NSEPOCHR_CUR)	363
7.11.22	FLASH secure EPOCH register (FLASH_SECEPOCHR_CUR)	364
7.11.23	FLASH option status register 2 (FLASH_OPTSR2_CUR)	364
7.11.24	FLASH option status register 2 (FLASH_OPTSR2_PRG)	365
7.11.25	FLASH nonsecure boot register (FLASH_NSBOOTR_CUR)	367
7.11.26	FLASH nonsecure boot register (FLASH_NSBOOTR_PRG)	367
7.11.27	FLASH secure boot register (FLASH_SECBOOTR_CUR)	368
7.11.28	FLASH secure boot register (FLASH_BOOTR_PRG)	368
7.11.29	FLASH nonsecure OTP block lock (FLASH_OTPBLR_CUR)	369
7.11.30	FLASH nonsecure OTP block lock (FLASH_OTPBLR_PRG)	369
7.11.31	FLASH secure block based register for Bank1 (FLASH_SECBB1Rx)	370

7.11.32	FLASH privilege block based register for Bank1 (FLASH_PRIVBB1Rx)	370
7.11.33	FLASH security watermark for Bank1 (FLASH_SECWM1R_CUR) ..	371
7.11.34	FLASH security watermark for Bank1 (FLASH_SECWM1R_PRG) ..	371
7.11.35	FLASH write sector group protection for Bank1 (FLASH_WRP1R_CUR)	372
7.11.36	FLASH write sector group protection for Bank1 (FLASH_WRP1R_PRG)	373
7.11.37	FLASH data sector configuration Bank1 (FLASH_EDATA1R_CUR) ..	373
7.11.38	FLASH data sector configuration Bank1 (FLASH_EDATA1R_PRG) ..	374
7.11.39	FLASH HDP Bank1 configuration (FLASH_HDP1R_CUR)	375
7.11.40	FLASH HDP Bank1 configuration (FLASH_HDP1R_PRG)	375
7.11.41	FLASH ECC correction register (FLASH_ECCCORR)	376
7.11.42	FLASH ECC detection register (FLASH_ECCDETR)	377
7.11.43	FLASH ECC data (FLASH_ECCDR)	378
7.11.44	FLASH secure block-based register for Bank2 (FLASH_SECBB2Rx)	378
7.11.45	FLASH privilege block-based register for Bank2 (FLASH_PRIVBB2Rx)	379
7.11.46	FLASH security watermark for Bank2 (FLASH_SECWM2R_CUR) ..	379
7.11.47	FLASH security watermark for Bank2 (FLASH_SECWM2R_PRG) ..	380
7.11.48	FLASH write sector group protection for Bank2 (FLASH_WRP2R_CUR)	380
7.11.49	FLASH write sector group protection for Bank2 (FLASH_WRP2R_PRG)	381
7.11.50	FLASH data sectors configuration Bank2 (FLASH_EDATA2R_CUR) .	382
7.11.51	FLASH data sector configuration Bank2 (FLASH_EDATA2R_PRG) ..	382
7.11.52	FLASH HDP Bank2 configuration (FLASH_HDP2R_CUR)	383
7.11.53	FLASH HDP Bank2 configuration (FLASH_HDP2R_PRG)	383
7.12	FLASH register map and reset values	385
8	Instruction cache (ICACHE)	390
8.1	ICACHE introduction	390
8.2	ICACHE main features	390
8.3	ICACHE implementation	391
8.4	ICACHE functional description	391
8.4.1	ICACHE block diagram	392
8.4.2	ICACHE reset and clocks	392
8.4.3	ICACHE TAG memory	393

8.4.4	Direct-mapped ICACHE (1-way cache)	394
8.4.5	ICACHE enable	395
8.4.6	Cacheable and noncacheable traffic	395
8.4.7	Address remapping	396
8.4.8	Cacheable accesses	398
8.4.9	Dual-master cache	399
8.4.10	ICACHE security	399
8.4.11	ICACHE maintenance	399
8.4.12	ICACHE performance monitoring	400
8.4.13	ICACHE boot	400
8.5	ICACHE low-power modes	400
8.6	ICACHE error management and interrupts	401
8.7	ICACHE registers	401
8.7.1	ICACHE control register (ICACHE_CR)	401
8.7.2	ICACHE status register (ICACHE_SR)	402
8.7.3	ICACHE interrupt enable register (ICACHE_IER)	403
8.7.4	ICACHE flag clear register (ICACHE_FCR)	403
8.7.5	ICACHE hit monitor register (ICACHE_HMONR)	404
8.7.6	ICACHE miss monitor register (ICACHE_MMONR)	404
8.7.7	ICACHE region x configuration register (ICACHE_CRRx)	404
8.7.8	ICACHE register map	405
9	Data cache (DCACHE)	407
9.1	DCACHE introduction	407
9.2	DCACHE main features	407
9.3	DCACHE implementation	408
9.4	DCACHE functional description	408
9.4.1	DCACHE block diagram	409
9.4.2	DCACHE reset and clocks	409
9.4.3	DCACHE TAG memory	410
9.4.4	DCACHE enable	412
9.4.5	Cacheable and noncacheable traffic	412
9.4.6	Cacheable accesses	413
9.4.7	DCACHE security	415
9.4.8	DCACHE maintenance	415
9.4.9	DCACHE performance monitoring	417

9.4.10	DCACHE boot	417
9.5	DCACHE low-power modes	418
9.6	DCACHE error management and interrupts	418
9.7	DCACHE registers	419
9.7.1	DCACHE control register (DCACHE_CR)	419
9.7.2	DCACHE status register (DCACHE_SR)	420
9.7.3	DCACHE interrupt enable register (DCACHE_IER)	421
9.7.4	DCACHE flag clear register (DCACHE_FCR)	422
9.7.5	DCACHE read-hit monitor register (DCACHE_RHMONR)	422
9.7.6	DCACHE read-miss monitor register (DCACHE_RMMONR)	423
9.7.7	DCACHE write-hit monitor register (DCACHE_WHMONR)	423
9.7.8	DCACHE write-miss monitor register (DCACHE_WMMONR)	423
9.7.9	DCACHE command range start address register (DCACHE_CMDRSADDR)	424
9.7.10	DCACHE command range end address register (DCACHE_CMDREADDR)	424
9.7.11	DCACHE register map	424
10	Power control (PWR)	426
10.1	PWR introduction	426
10.2	PWR main features	426
10.3	PWR pins and internal signals	427
10.4	PWR power supplies and supply domains	428
10.4.1	External power supplies	430
10.4.2	Internal regulators	431
10.4.3	Power-up and power-down power sequences	433
10.4.4	Independent analog peripherals supply	433
10.4.5	Independent I/O supply rail	434
10.4.6	Independent USB transceivers supply	434
10.4.7	Backup domain	435
10.5	PWR system supply voltage regulation	436
10.5.1	SMPS and LDO embedded regulators	436
10.5.2	V _{CORE} supply versus reset, voltage scaling, and low-power modes	437
10.5.3	Embedded voltage regulator operating modes	437
10.6	PWR power supply and temperature supervision	437
10.6.1	Power-on reset (POR)/power-down reset (PDR)	438
10.6.2	Brownout reset (BOR)	438

10.6.3	Programmable voltage detector (PVD)	439
10.6.4	Analog voltage detector (AVD)	440
10.6.5	VDDIO2 voltage monitor (IO2VM)	441
10.6.6	Backup domain voltage monitoring	441
10.6.7	Temperature monitoring	442
10.7	PWR management	443
10.7.1	Voltage scaling	443
10.7.2	Power management examples	444
10.8	Power modes	445
10.8.1	Slowing down system clocks	449
10.8.2	Peripheral clock gating	449
10.8.3	Low-power modes	449
10.8.4	Sleep mode	450
10.8.5	Stop mode	451
10.8.6	Standby mode	453
10.8.7	Power modes output pins	456
10.9	PWR security and privileged protection	457
10.9.1	PWR security protection	457
10.9.2	PWR privileged protection	458
10.10	PWR interrupts	459
10.11	PWR registers	460
10.11.1	PWR power mode control register (PWR_PMCR)	460
10.11.2	PWR power mode control register [alternate] (PWR_PMCR)	462
10.11.3	PWR power mode control register [alternate] (PWR_PMCR)	464
10.11.4	PWR status register (PWR_PMSR)	466
10.11.5	PWR voltage scaling control register (PWR_VOSCR)	466
10.11.6	PWR voltage scaling status register (PWR_VOSSR)	467
10.11.7	PWR Backup domain control register (PWR_BDCR)	468
10.11.8	PWR Backup domain control register (PWR_DBPCR)	469
10.11.9	PWR Backup domain status register (PWR_BDSR)	469
10.11.10	PWR USB Type-C power delivery register (PWR_UCPDR)	470
10.11.11	PWR supply configuration control register (PWR_SCCR)	470
10.11.12	PWR voltage monitor control register (PWR_VMCR)	471
10.11.13	PWR USB supply control register (PWR_USBSCR)	472
10.11.14	PWR voltage monitor status register (PWR_VMSR)	473
10.11.15	PWR wake-up status clear register (PWR_WUSCR)	474

	10.11.16 PWR wake-up status register (PWR_WUSR)	474
	10.11.17 PWR wake-up configuration register (PWR_WUCR)	475
	10.11.18 PWR I/O retention register (PWR_IOPRETR)	475
	10.11.19 PWR security configuration register (PWR_SECCFGR)	476
	10.11.20 PWR privilege configuration register (PWR_PRIVCFGR)	477
	10.11.21 PWR register map	478
11	Reset and clock control (RCC)	480
11.1	RCC introduction	480
11.2	RCC pins and internal signals	480
11.3	RCC reset functional description	480
11.3.1	Power reset	480
11.3.2	System reset	481
11.3.3	Backup domain reset	482
11.3.4	Reset source identification	482
11.4	RCC clocks functional description	483
11.4.1	HSE clock	486
11.4.2	HSI clock	487
11.4.3	CSI oscillator	488
11.4.4	HSI48 clock	489
11.4.5	PLL description	489
11.4.6	LSE clock	493
11.4.7	LSI clock	494
11.4.8	System clock (SYSCLK) selection	494
11.4.9	Handling clock generators in stop and standby modes	495
11.4.10	Clock security system (CSS)	496
11.4.11	Clock output generation (MCO1/MCO2)	497
11.4.12	Kernel clock selection	498
11.4.13	RTC and TAMP clock	503
11.4.14	Timer clock	503
11.4.15	Watchdog clock	504
11.4.16	Peripherals clock gating and autonomous mode	504
11.5	RCC security and privilege functional description	504
11.5.1	RCC TrustZone security protection modes	504
11.5.2	RCC privilege protection modes	507
11.6	RCC low-power modes	508

11.7	RCC interrupts	509
11.8	RCC registers	511
11.8.1	RCC clock control register (RCC_CR)	511
11.8.2	RCC HSI calibration register (RCC_HSI CFGR)	514
11.8.3	RCC clock recovery RC register (RCC_CRR CR)	515
11.8.4	RCC CSI calibration register (RCC_CSICFGR)	515
11.8.5	RCC clock configuration register1 (RCC_CFGR1)	516
11.8.6	RCC CPU domain clock configuration register 2 (RCC_CFGR2)	518
11.8.7	RCC PLL clock source selection register (RCC_PLL1CFGR)	521
11.8.8	RCC PLL clock source selection register (RCC_PLL2CFGR)	523
11.8.9	RCC PLL clock source selection register (RCC_PLL3CFGR)	524
11.8.10	RCC PLL1 dividers register (RCC_PLL1DIVR)	526
11.8.11	RCC PLL1 fractional divider register (RCC_PLL1FRACR)	527
11.8.12	RCC PLL2 dividers register (RCC_PLL2DIVR)	528
11.8.13	RCC PLL2 fractional divider register (RCC_PLL2FRACR)	529
11.8.14	RCC PLL3 dividers register (RCC_PLL3DIVR)	530
11.8.15	RCC PLL3 fractional divider register (RCC_PLL3FRACR)	531
11.8.16	RCC clock source interrupt enable register (RCC_CIER)	532
11.8.17	RCC clock source interrupt flag register (RCC_CIFR)	533
11.8.18	RCC clock source interrupt clear register (RCC_CICR)	535
11.8.19	RCC AHB1 reset register (RCC_AHB1RSTR)	536
11.8.20	RCC AHB2 peripheral reset register (RCC_AHB2RSTR)	537
11.8.21	RCC AHB4 peripheral reset register (RCC_AHB4RSTR)	540
11.8.22	RCC APB1 peripheral low reset register (RCC_APB1LRSTR)	541
11.8.23	RCC APB1 peripheral high reset register (RCC_APB1HRSTR)	544
11.8.24	RCC APB2 peripheral reset register (RCC_APB2RSTR)	545
11.8.25	RCC APB3 peripheral reset register (RCC_APB3RSTR)	547
11.8.26	RCC AHB1 peripherals clock register (RCC_AHB1ENR)	549
11.8.27	RCC AHB2 peripheral clock register (RCC_AHB2ENR)	551
11.8.28	RCC AHB4 peripheral clock register (RCC_AHB4ENR)	554
11.8.29	RCC APB1 peripheral clock register (RCC_APB1LENR)	555
11.8.30	RCC APB1 peripheral clock register (RCC_APB1HENR)	558
11.8.31	RCC APB2 peripheral clock register (RCC_APB2ENR)	559
11.8.32	RCC APB3 peripheral clock register (RCC_APB3ENR)	561
11.8.33	RCC AHB1 sleep clock register (RCC_AHB1LPENR)	563
11.8.34	RCC AHB2 sleep clock register (RCC_AHB2LPENR)	565
11.8.35	RCC AHB4 sleep clock register (RCC_AHB4LPENR)	568

11.8.36	RCC APB1 sleep clock register (RCC_APB1LLPENR)	569
11.8.37	RCC APB1 sleep clock register (RCC_APB1HLPENR)	573
11.8.38	RCC APB2 sleep clock register (RCC_APB2LPENR)	574
11.8.39	RCC APB3 sleep clock register (RCC_APB3LPENR)	576
11.8.40	RCC kernel clock configuration register (RCC_CCIPR1)	578
11.8.41	RCC kernel clock configuration register (RCC_CCIPR2)	581
11.8.42	RCC kernel clock configuration register (RCC_CCIPR3)	583
11.8.43	RCC kernel clock configuration register (RCC_CCIPR4)	586
11.8.44	RCC kernel clock configuration register (RCC_CCIPR5)	588
11.8.45	RCC Backup domain control register (RCC_BDCR)	590
11.8.46	RCC reset status register (RCC_RSR)	592
11.8.47	RCC secure configuration register (RCC_SECCFGR)	593
11.8.48	RCC privilege configuration register (RCC_PRIVCFGR)	595
11.9	RCC register map	597
12	Clock recovery system (CRS)	603
12.1	CRS introduction	603
12.2	CRS main features	603
12.3	CRS implementation	603
12.4	CRS functional description	604
12.4.1	CRS block diagram	604
12.4.2	CRS internal signals	604
12.4.3	Synchronization input	605
12.4.4	Frequency error measurement	605
12.4.5	Frequency error evaluation and automatic trimming	606
12.4.6	CRS initialization and configuration	607
12.5	CRS in low-power modes	608
12.6	CRS interrupts	608
12.7	CRS registers	608
12.7.1	CRS control register (CRS_CR)	608
12.7.2	CRS configuration register (CRS_CFGR)	610
12.7.3	CRS interrupt and status register (CRS_ISR)	611
12.7.4	CRS interrupt flag clear register (CRS_ICR)	613
12.7.5	CRS register map	613
13	General-purpose I/Os (GPIO)	615

13.1	GPIO introduction	615
13.2	GPIO main features	615
13.3	GPIO functional description	615
13.3.1	General-purpose I/O (GPIO)	617
13.3.2	I/O pin alternate function multiplexer and mapping	618
13.3.3	I/O port control registers	619
13.3.4	I/O data bitwise handling	619
13.3.5	GPIO locking mechanism	619
13.3.6	I/O alternate function input/output	620
13.3.7	External interrupt/wake-up lines	620
13.3.8	Input configuration	620
13.3.9	Output configuration	621
13.3.10	Alternate function configuration	621
13.3.11	Analog configuration	622
13.3.12	Using the HSE or LSE oscillator pins as GPIOs	622
13.3.13	Using the GPIO pins in the RTC supply domain	623
13.3.14	I/Os state retention during standby mode	623
13.3.15	TrustZone security	623
13.3.16	Privileged and unprivileged modes	624
13.3.17	High-speed low-voltage mode (HSLV)	624
13.3.18	I/O compensation cell	625
13.4	GPIO registers	626
13.4.1	GPIO port mode register (GPIOx_MODER) (x = A to I)	626
13.4.2	GPIO port output type register (GPIOx_OTYPER) (x = A to I)	626
13.4.3	GPIO port output speed register (GPIOx_OSPEEDR) (x = A to I)	627
13.4.4	GPIO port pull-up/pull-down register (GPIOx_PUPDR) (x = A to I)	627
13.4.5	GPIO port input data register (GPIOx_IDR) (x = A to I)	628
13.4.6	GPIO port output data register (GPIOx_ODR) (x = A to I)	628
13.4.7	GPIO port bit set/reset register (GPIOx_BSRR) (x = A to I)	629
13.4.8	GPIO port configuration lock register (GPIOx_LCKR) (x = A to I)	629
13.4.9	GPIO alternate function low register (GPIOx_AFRL) (x = A to I)	630
13.4.10	GPIO alternate function high register (GPIOx_AFRH) (x = A to H)	631
13.4.11	GPIO port bit reset register (GPIOx_BRR) (x = A to I)	632
13.4.12	GPIO high-speed low-voltage register (GPIOx_HSLVR) (x = A to I)	632
13.4.13	GPIO secure configuration register (GPIOx_SECCFGR) (x = A to I)	633
13.4.14	GPIO register map	634

14	System configuration, boot, and security (SBS)	636
14.1	SBS introduction	636
14.2	SBS main features	636
14.3	SBS functional description	637
14.3.1	SBS block diagram	637
14.3.2	SBS signals	638
14.3.3	SBS reset and clocks	638
14.3.4	SBS system configuration	639
14.3.5	SBS boot control	640
14.3.6	SBS debug control	643
14.3.7	SBS hardware secure storage control	645
14.4	SBS interrupts	647
14.5	SBS registers	647
14.5.1	SBS temporal isolation control register (SBS_HDPLCR)	647
14.5.2	SBS temporal isolation status register (SBS_HDPLSR)	647
14.5.3	SBS next HDPL control register (SBS_NEXTHDPLCR)	648
14.5.4	SBS debug control register (SBS_DBGCR)	648
14.5.5	SBS debug lock register (SBS_DBGLOCKR)	649
14.5.6	SBS RSS command register (SBS_RSSCMDR)	650
14.5.7	SBS EPOCH selection control register (SBS_EPOCHSELCR)	650
14.5.8	SBS security mode configuration control register (SBS_SECCFGR)	651
14.5.9	SBS product mode and configuration register (SBS_PMCR)	651
14.5.10	SBS product mode and configuration register [alternate] (SBS_PMCR)	653
14.5.11	SBS FPU interrupt mask register (SBS_FPUIMR)	654
14.5.12	SBS memory erase status register (SBS_MESR)	655
14.5.13	SBS compensation cell for I/Os control and status register (SBS_CCCSR)	656
14.5.14	SBS compensation cell for I/Os value register (SBS_CCVALR)	657
14.5.15	SBS compensation cell for I/Os software code register (SBS_CCSWCR)	657
14.5.16	SBS Class B register (SBS_CFGR2)	658
14.5.17	SBS CPU nonsecure lock register (SBS_CNSLCKR)	659
14.5.18	SBS CPU secure lock register (SBS_CSLCKR)	660
14.5.19	SBS flitf ECC NMI mask register (SBS_ECCNMIR)	661
14.5.20	SBS register map	662

15	Peripherals interconnect matrix	664
15.1	Interconnect matrix introduction	664
15.2	Connection summary	664
15.3	Interconnection details	666
15.3.1	Master to slave interconnection for timers	666
15.3.2	Triggers to ADCs	666
15.3.3	ADC analog watchdogs as triggers to timers and PLAY	667
15.3.4	Triggers to DAC	668
15.3.5	Clock sources to timers	668
15.3.6	Triggers to low-power timers	669
15.3.7	RTC wake-up as inputs to timers	670
15.3.8	System errors as break signals to timers	670
15.3.9	Triggers for communication peripherals	671
15.3.10	Triggers to GPDMA1/2	671
15.3.11	Internal analog signals to analog peripherals	672
15.3.12	Clock source for the DAC sample and hold mode	672
15.3.13	Internal tamper sources	672
15.3.14	Output from tamper to RTC	672
15.3.15	Encryption keys to AES/SAES	673
15.3.16	Comparators as inputs, trigger, or break signals to timers1	673
15.3.17	Blanking sources to comparators	674
15.3.18	Play as input to timers	675
15.3.19	Clock source to PLAY	675
15.3.20	Inputs to PLAY	675
15.3.21	Triggers to DTS	676
16	General purpose direct memory access controller (GPDMA)	677
16.1	GPDMA introduction	677
16.2	GPDMA main features	677
16.3	GPDMA implementation	678
16.3.1	GPDMA instances	678
16.3.2	GPDMA channels	678
16.3.3	GPDMA in low-power modes	679
16.3.4	GPDMA requests	679
16.3.5	GPDMA block requests	684
16.3.6	GPDMA channels with peripheral early termination	684

16.3.7	GPDMA triggers	685
16.4	GPDMA functional description	687
16.4.1	GPDMA block diagram	687
16.4.2	GPDMA channel state and direct programming without any linked-list	688
16.4.3	GPDMA channel suspend and resume	689
16.4.4	GPDMA channel abort and restart	690
16.4.5	GPDMA linked-list data structure	691
16.4.6	Linked-list item transfer execution	694
16.4.7	GPDMA channel state and linked-list programming in run-to-completion mode	694
16.4.8	GPDMA channel state and linked-list programming in link step mode	698
16.4.9	GPDMA channel state and linked-list programming	705
16.4.10	GPDMA FIFO-based transfers	707
16.4.11	GPDMA transfer request and arbitration	714
16.4.12	GPDMA triggered transfer	718
16.4.13	GPDMA circular buffering with linked-list programming	719
16.4.14	GPDMA transfer in peripheral flow-control mode	721
16.4.15	GPDMA secure/nonsecure channel	722
16.4.16	GPDMA privileged/unprivileged channel	723
16.4.17	GPDMA error management	723
16.5	GPDMA in debug mode	725
16.6	GPDMA in low-power modes	725
16.7	GPDMA interrupts	726
16.8	GPDMA registers	727
16.8.1	GPDMA secure configuration register (GPDMA_SECCFGR)	727
16.8.2	GPDMA privileged configuration register (GPDMA_PRIVCFGR)	728
16.8.3	GPDMA configuration lock register (GPDMA_RCFGLOCKR)	728
16.8.4	GPDMA nonsecure masked interrupt status register (GPDMA_MISR)	729
16.8.5	GPDMA secure masked interrupt status register (GPDMA_SMISR) ..	730
16.8.6	GPDMA channel x linked-list base address register (GPDMA_CxLBAR)	730
16.8.7	GPDMA channel x flag clear register (GPDMA_CxFCR)	731
16.8.8	GPDMA channel x status register (GPDMA_CxSR)	732
16.8.9	GPDMA channel x control register (GPDMA_CxCR)	733
16.8.10	GPDMA channel x transfer register 1 (GPDMA_CxTR1)	735

16.8.11	GPDMA channel x transfer register 2 (GPDMA_CxTR2)	739
16.8.12	GPDMA channel x block register 1 (GPDMA_CxBR1)	743
16.8.13	GPDMA channel x alternate block register 1 (GPDMA_CxBR1)	744
16.8.14	GPDMA channel x source address register (GPDMA_CxSAR)	747
16.8.15	GPDMA channel x destination address register (GPDMA_CxDAR)	749
16.8.16	GPDMA channel x transfer register 3 (GPDMA_CxTR3)	750
16.8.17	GPDMA channel x block register 2 (GPDMA_CxBR2)	751
16.8.18	GPDMA channel x linked-list address register (GPDMA_CxLLR)	752
16.8.19	GPDMA channel x alternate linked-list address register (GPDMA_CxLLR)	754
16.8.20	GPDMA register map	755
17	Nested vectored interrupt controller (NVIC)	758
17.1	NVIC main features	758
17.2	SysTick calibration value register	758
17.3	Interrupt and exception vectors	759
18	Extended interrupts and event controller (EXTI)	775
18.1	EXTI introduction	775
18.2	EXTI main features	775
18.3	EXTI block diagram	776
18.3.1	EXTI connections between peripherals and CPU	777
18.3.2	EXTI interrupt/event mapping	777
18.4	EXTI functional description	779
18.4.1	EXTI configurable event input wake-up	780
18.4.2	EXTI direct event input wake-up	780
18.4.3	EXTI mux selection	781
18.5	EXTI functional behavior	781
18.6	EXTI event protection	782
18.6.1	EXTI security protection	783
18.6.2	EXTI privilege protection	783
18.7	EXTI registers	784
18.7.1	EXTI rising trigger selection register (EXTI_RTSTR1)	784
18.7.2	EXTI falling trigger selection register (EXTI_FTSTR1)	784
18.7.3	EXTI software interrupt event register (EXTI_SWIER1)	785
18.7.4	EXTI rising edge pending register (EXTI_RPR1)	786

18.7.5	EXTI falling edge pending register (EXTI_FPR1)	786
18.7.6	EXTI security configuration register (EXTI_SECCFGR1)	787
18.7.7	EXTI privilege configuration register (EXTI_PRIVCFGR1)	788
18.7.8	EXTI rising trigger selection register 2 (EXTI_RTSTR2)	788
18.7.9	EXTI falling trigger selection register 2 (EXTI_FTSR2)	790
18.7.10	EXTI software interrupt event register 2 (EXTI_SWIER2)	792
18.7.11	EXTI rising edge pending register 2 (EXTI_RPR2)	795
18.7.12	EXTI falling edge pending register 2 (EXTI_FPR2)	797
18.7.13	EXTI security configuration register 2 (EXTI_SECCFGR2)	799
18.7.14	EXTI privilege configuration register 2 (EXTI_PRIVCFGR2)	800
18.7.15	EXTI rising trigger selection register 3 (EXTI_RTSTR3)	801
18.7.16	EXTI falling trigger selection register 3 (EXTI_FTSR3)	802
18.7.17	EXTI software interrupt event register 3 (EXTI_SWIER3)	803
18.7.18	EXTI rising edge pending register 3 (EXTI_RPR3)	804
18.7.19	EXTI falling edge pending register 3 (EXTI_FPR3)	805
18.7.20	EXTI security configuration register 3 (EXTI_SECCFGR3)	806
18.7.21	EXTI privilege configuration register 3 (EXTI_PRIVCFGR3)	806
18.7.22	EXTI external interrupt selection register (EXTI_EXTICR1)	807
18.7.23	EXTI external interrupt selection register (EXTI_EXTICR2)	809
18.7.24	EXTI external interrupt selection register (EXTI_EXTICR3)	811
18.7.25	EXTI external interrupt selection register (EXTI_EXTICR4)	813
18.7.26	EXTI lock register (EXTI_LOCKR)	815
18.7.27	EXTI CPU wake-up with interrupt mask register (EXTI_IMR1)	816
18.7.28	EXTI CPU wake-up with event mask register (EXTI_EMR1)	816
18.7.29	EXTI CPU wake-up with interrupt mask register 2 (EXTI_IMR2)	817
18.7.30	EXTI CPU wake-up with event mask register 2 (EXTI_EMR2)	818
18.7.31	EXTI CPU wake-up with interrupt mask register 3 (EXTI_IMR3)	820
18.7.32	EXTI CPU wake-up with event mask register 3 (EXTI_EMR3)	821
18.7.33	EXTI register map	822
19	Cyclic redundancy check calculation unit (CRC)	825
19.1	CRC introduction	825
19.2	CRC main features	825
19.3	CRC implementation	825
19.4	CRC functional description	826
19.4.1	CRC block diagram	826
19.4.2	CRC internal signals	826

19.4.3	CRC operation	826
19.5	CRC registers	828
19.5.1	CRC data register (CRC_DR)	828
19.5.2	CRC independent data register (CRC_IDR)	828
19.5.3	CRC control register (CRC_CR)	829
19.5.4	CRC initial value (CRC_INIT)	830
19.5.5	CRC polynomial (CRC_POL)	830
19.5.6	CRC register map	831
20	CORDIC coprocessor (CORDIC)	832
20.1	CORDIC introduction	832
20.2	CORDIC main features	832
20.3	CORDIC functional description	832
20.3.1	General description	832
20.3.2	CORDIC functions	833
20.3.3	Fixed point representation	839
20.3.4	Scaling factor	840
20.3.5	Precision	840
20.3.6	Zero-overhead mode	843
20.3.7	Polling mode	844
20.3.8	Interrupt mode	845
20.3.9	DMA mode	845
20.4	CORDIC registers	846
20.4.1	CORDIC control/status register (CORDIC_CSR)	846
20.4.2	CORDIC argument register (CORDIC_WDATA)	848
20.4.3	CORDIC result register (CORDIC_RDATA)	849
20.4.4	CORDIC register map	849
21	Filter math accelerator (FMAC)	850
21.1	FMAC introduction	850
21.2	FMAC main features	850
21.3	FMAC functional description	851
21.3.1	General description	851
21.3.2	Local memory and buffers	852
21.3.3	Input buffers	852
21.3.4	Output buffer	855

21.3.5	Initialization functions	857
21.3.6	Filter functions	858
21.3.7	Fixed point representation	862
21.3.8	Implementing FIR filters with the FMAC	862
21.3.9	Implementing IIR filters with the FMAC	864
21.3.10	Examples of filter initialization	866
21.3.11	Examples of filter operation	867
21.3.12	Filter design tips	869
21.4	FMAC registers	870
21.4.1	FMAC X1 buffer configuration register (FMAC_X1BUFCFG)	870
21.4.2	FMAC X2 buffer configuration register (FMAC_X2BUFCFG)	870
21.4.3	FMAC Y buffer configuration register (FMAC_YBUFCFG)	871
21.4.4	FMAC parameter register (FMAC_PARAM)	872
21.4.5	FMAC control register (FMAC_CR)	873
21.4.6	FMAC status register (FMAC_SR)	874
21.4.7	FMAC write data register (FMAC_WDATA)	875
21.4.8	FMAC read data register (FMAC_RDATA)	876
21.4.9	FMAC register map	876
22	Programmable logic array (PLAY)	878
22.1	PLAY introduction	878
22.2	PLAY main features	878
22.3	PLAY functional description	879
22.3.1	PLAY block diagram	879
22.3.2	PLAY pins and internal signals	879
22.3.3	PLAY reset and clocks	882
22.3.4	Input selection	882
22.3.5	Input filtering and edge detection	883
22.3.6	Software programmable inputs	885
22.3.7	Logic elements	886
22.3.8	Input interconnect	889
22.3.9	Output interconnect	894
22.3.10	Flags	895
22.3.11	Miscellaneous status bits	896
22.4	PLAY in low-power modes	896
22.5	PLAY interrupts	896

22.6	PLAY registers	897
22.6.1	PLAY configuration control register (PLAY_CFGCR)	898
22.6.2	PLAY security attributes configuration register (PLAY_SECCFGR)	898
22.6.3	PLAY privilege attributes configuration register (PLAY_PRIVCFGR)	899
22.6.4	PLAY filter x configuration register (PLAY_FILTxCFG)	900
22.6.5	PLAY logic element x configuration register 1 (PLAY_LExCFG1)	901
22.6.6	PLAY logic element x configuration register 2 (PLAY_LExCFG2)	901
22.6.7	PLAY output x configuration register (PLAY_OUTxCFG)	902
22.6.8	PLAY software input status register (PLAY_SWIN)	903
22.6.9	PLAY software input set register (PLAY_SWINSET)	903
22.6.10	PLAY software input clear register (PLAY_SWINCLR)	904
22.6.11	PLAY output status register (PLAY_OSR)	905
22.6.12	PLAY flag status register (PLAY_FLSTAT)	905
22.6.13	PLAY flag set register (PLAY_FLSET)	906
22.6.14	PLAY flag clear register (PLAY_FLCLR)	907
22.6.15	PLAY flag interrupt enable register (PLAY_FLIER)	907
22.6.16	PLAY flag control register (PLAY_FLCTL)	908
22.6.17	PLAY miscellaneous status register (PLAY_MSR)	909
22.6.18	PLAY interrupt status register (PLAY_ISR)	909
22.6.19	PLAY interrupt clear register (PLAY_ICR)	910
22.6.20	PLAY interrupt enable register (PLAY_IER)	911
22.7	PLAY register map	911
23	Flexible memory controller (FMC)	914
23.1	FMC introduction	914
23.2	FMC main features	914
23.3	FMC block diagram	915
23.4	AHB interface	916
23.4.1	Supported memories and transactions	917
23.5	External device address mapping	918
23.5.1	NOR/PSRAM address mapping	919
23.5.2	NAND flash memory address mapping	919
23.5.3	SDRAM address mapping	920
23.6	NOR flash/PSRAM controller	922
23.6.1	External memory interface signals	924
23.6.2	Supported memories and transactions	926

23.6.3	General timing rules	927
23.6.4	NOR flash/PSRAM controller asynchronous transactions	927
23.6.5	Synchronous transactions	945
23.6.6	NOR/PSRAM controller registers	952
23.7	NAND flash controller	960
23.7.1	External memory interface signals	960
23.7.2	NAND flash supported memories and transactions	961
23.7.3	Timing diagrams for NAND flash memory	962
23.7.4	NAND flash operations	963
23.7.5	NAND flash prewait functionality	963
23.7.6	Computation of the error correction code (ECC) in NAND flash memory	964
23.7.7	NAND flash controller registers	965
23.8	SDRAM controller	971
23.8.1	SDRAM controller main features	971
23.8.2	SDRAM External memory interface signals	971
23.8.3	SDRAM controller functional description	972
23.8.4	Low-power modes	978
23.8.5	SDRAM controller registers	981
23.8.6	FMC register map	987
24	Octo-SPI interface (OCTOSPI)	990
24.1	OCTOSPI introduction	990
24.2	OCTOSPI main features	990
24.3	OCTOSPI implementation	991
24.4	OCTOSPI functional description	992
24.4.1	OCTOSPI block diagram	992
24.4.2	OCTOSPI pins and internal signals	993
24.4.3	OCTOSPI interface to memory modes	994
24.4.4	OCTOSPI regular-command protocol	994
24.4.5	OCTOSPI regular-command protocol signal interface	998
24.4.6	HyperBus protocol	1001
24.4.7	Specific features	1005
24.4.8	OCTOSPI operating mode introduction	1006
24.4.9	OCTOSPI indirect mode	1006
24.4.10	OCTOSPI automatic status-polling mode	1008
24.4.11	OCTOSPI memory-mapped mode	1009

24.4.12	OCTOSPI configuration introduction	1010
24.4.13	OCTOSPI system configuration	1010
24.4.14	OCTOSPI device configuration	1011
24.4.15	OCTOSPI regular-command mode configuration	1014
24.4.16	OCTOSPI HyperBus protocol configuration	1016
24.4.17	OCTOSPI error management	1017
24.4.18	OCTOSPI BUSY and ABORT	1017
24.4.19	OCTOSPI reconfiguration or deactivation	1018
24.4.20	NCS behavior	1018
24.5	Address alignment and data number	1020
24.6	OCTOSPI interrupts	1021
24.7	OCTOSPI registers	1021
24.7.1	OCTOSPI control register (OCTOSPI_CR)	1021
24.7.2	OCTOSPI device configuration register 1 (OCTOSPI_DCR1)	1024
24.7.3	OCTOSPI device configuration register 2 (OCTOSPI_DCR2)	1026
24.7.4	OCTOSPI device configuration register 3 (OCTOSPI_DCR3)	1027
24.7.5	OCTOSPI device configuration register 4 (OCTOSPI_DCR4)	1027
24.7.6	OCTOSPI status register (OCTOSPI_SR)	1028
24.7.7	OCTOSPI flag clear register (OCTOSPI_FCR)	1029
24.7.8	OCTOSPI data length register (OCTOSPI_DLR)	1030
24.7.9	OCTOSPI address register (OCTOSPI_AR)	1030
24.7.10	OCTOSPI data register (OCTOSPI_DR)	1031
24.7.11	OCTOSPI polling status mask register (OCTOSPI_PSMKR)	1031
24.7.12	OCTOSPI polling status match register (OCTOSPI_PSMAR)	1032
24.7.13	OCTOSPI polling interval register (OCTOSPI_PIR)	1032
24.7.14	OCTOSPI communication configuration register (OCTOSPI_CCR)	1032
24.7.15	OCTOSPI timing configuration register (OCTOSPI_TCR)	1034
24.7.16	OCTOSPI instruction register (OCTOSPI_IR)	1035
24.7.17	OCTOSPI alternate bytes register (OCTOSPI_ABR)	1035
24.7.18	OCTOSPI low-power timeout register (OCTOSPI_LPTR)	1036
24.7.19	OCTOSPI wrap communication configuration register (OCTOSPI_WPCCR)	1036
24.7.20	OCTOSPI wrap timing configuration register (OCTOSPI_WPTCR)	1038
24.7.21	OCTOSPI wrap instruction register (OCTOSPI_WPIR)	1039
24.7.22	OCTOSPI wrap alternate bytes register (OCTOSPI_WPABR)	1039
24.7.23	OCTOSPI write communication configuration register (OCTOSPI_WCCR)	1040

24.7.24	OCTOSPI write timing configuration register (OCTOSPI_WTCR) . . .	1042
24.7.25	OCTOSPI write instruction register (OCTOSPI_WIR)	1042
24.7.26	OCTOSPI write alternate bytes register (OCTOSPI_WABR)	1043
24.7.27	OCTOSPI HyperBus latency configuration register (OCTOSPI_HLCR)	1043
24.7.28	OCTOSPI register map	1044
25	Octo-SPI interface (OCTOSPI)	1047
25.1	OCTOSPI introduction	1047
25.2	OCTOSPI main features	1047
25.3	OCTOSPI implementation	1048
25.4	OCTOSPI functional description	1049
25.4.1	OCTOSPI block diagram	1049
25.4.2	OCTOSPI pins and internal signals	1050
25.4.3	OCTOSPI interface to memory modes	1051
25.4.4	OCTOSPI regular-command protocol	1051
25.4.5	OCTOSPI regular-command protocol signal interface	1055
25.4.6	HyperBus protocol	1058
25.4.7	Specific features	1062
25.4.8	OCTOSPI operating mode introduction	1063
25.4.9	OCTOSPI indirect mode	1063
25.4.10	OCTOSPI automatic status-polling mode	1065
25.4.11	OCTOSPI memory-mapped mode	1066
25.4.12	OCTOSPI configuration introduction	1067
25.4.13	OCTOSPI system configuration	1067
25.4.14	OCTOSPI device configuration	1068
25.4.15	OCTOSPI regular-command mode configuration	1071
25.4.16	OCTOSPI HyperBus protocol configuration	1073
25.4.17	OCTOSPI error management	1074
25.4.18	OCTOSPI BUSY and ABORT	1074
25.4.19	OCTOSPI reconfiguration or deactivation	1075
25.4.20	NCS behavior	1075
25.4.21	Software control of two external memories	1077
25.4.22	Hardware-controlled extended memory support	1077
25.5	Address alignment and data number	1078
25.6	OCTOSPI interrupts	1079
25.7	OCTOSPI registers	1080

25.7.1	OCTOSPI control register (OCTOSPI_CR)	1080
25.7.2	OCTOSPI device configuration register 1 (OCTOSPI_DCR1)	1083
25.7.3	OCTOSPI device configuration register 2 (OCTOSPI_DCR2)	1085
25.7.4	OCTOSPI device configuration register 3 (OCTOSPI_DCR3)	1086
25.7.5	OCTOSPI device configuration register 4 (OCTOSPI_DCR4)	1087
25.7.6	OCTOSPI status register (OCTOSPI_SR)	1087
25.7.7	OCTOSPI flag clear register (OCTOSPI_FCR)	1088
25.7.8	OCTOSPI data length register (OCTOSPI_DLR)	1089
25.7.9	OCTOSPI address register (OCTOSPI_AR)	1089
25.7.10	OCTOSPI data register (OCTOSPI_DR)	1090
25.7.11	OCTOSPI polling status mask register (OCTOSPI_PSMKR)	1090
25.7.12	OCTOSPI polling status match register (OCTOSPI_PSMAR)	1091
25.7.13	OCTOSPI polling interval register (OCTOSPI_PIR)	1091
25.7.14	OCTOSPI communication configuration register (OCTOSPI_CCR)	1092
25.7.15	OCTOSPI timing configuration register (OCTOSPI_TCR)	1094
25.7.16	OCTOSPI instruction register (OCTOSPI_IR)	1094
25.7.17	OCTOSPI alternate bytes register (OCTOSPI_ABR)	1095
25.7.18	OCTOSPI low-power timeout register (OCTOSPI_LPTR)	1095
25.7.19	OCTOSPI wrap communication configuration register (OCTOSPI_WPCCR)	1096
25.7.20	OCTOSPI wrap timing configuration register (OCTOSPI_WPTCR)	1098
25.7.21	OCTOSPI wrap instruction register (OCTOSPI_WPIR)	1098
25.7.22	OCTOSPI wrap alternate bytes register (OCTOSPI_WPABR)	1099
25.7.23	OCTOSPI write communication configuration register (OCTOSPI_WCCR)	1099
25.7.24	OCTOSPI write timing configuration register (OCTOSPI_WTCR)	1101
25.7.25	OCTOSPI write instruction register (OCTOSPI_WIR)	1102
25.7.26	OCTOSPI write alternate bytes register (OCTOSPI_WABR)	1102
25.7.27	OCTOSPI HyperBus latency configuration register (OCTOSPI_HLCR)	1103
25.7.28	OCTOSPI register map	1103

26 Secure digital input/output MultiMediaCard interface (SDMMC) . . 1107

26.1	SDMMC main features	1107
26.2	SDMMC implementation	1107
26.3	SDMMC bus topology	1108
26.4	SDMMC operation modes	1110

26.5	SDMMC functional description	1111
26.5.1	SDMMC block diagram	1111
26.5.2	SDMMC pins and internal signals	1111
26.5.3	General description	1112
26.5.4	SDMMC adapter	1114
26.5.5	SDMMC AHB slave interface	1136
26.5.6	SDMMC AHB master interface	1137
26.5.7	AHB and SDMMC_CK clock relation	1140
26.6	Card functional description	1140
26.6.1	SD I/O mode	1140
26.6.2	CMD12 send timing	1148
26.6.3	Sleep (CMD5)	1152
26.6.4	Interrupt mode (Wait-IRQ)	1153
26.6.5	Boot operation	1154
26.6.6	Response R1b handling	1157
26.6.7	Reset and card cycle power	1158
26.7	Hardware flow control	1159
26.8	Ultra-high-speed phase I (UHS-I) voltage switch	1159
26.9	SDMMC interrupts	1163
26.10	SDMMC registers	1164
26.10.1	SDMMC power control register (SDMMC_POWER)	1164
26.10.2	SDMMC clock control register (SDMMC_CLKCR)	1165
26.10.3	SDMMC argument register (SDMMC_ARGR)	1167
26.10.4	SDMMC command register (SDMMC_CMDR)	1167
26.10.5	SDMMC command response register (SDMMC_RESPCMDR)	1169
26.10.6	SDMMC response x register (SDMMC_RESPxR)	1170
26.10.7	SDMMC data timer register (SDMMC_DTIMER)	1170
26.10.8	SDMMC data length register (SDMMC_DLENR)	1171
26.10.9	SDMMC data control register (SDMMC_DCTRL)	1172
26.10.10	SDMMC data counter register (SDMMC_DCNTR)	1173
26.10.11	SDMMC status register (SDMMC_STAR)	1174
26.10.12	SDMMC interrupt clear register (SDMMC_ICR)	1177
26.10.13	SDMMC mask register (SDMMC_MASKR)	1179
26.10.14	SDMMC acknowledgment timer register (SDMMC_ACKTIMER)	1182
26.10.15	SDMMC DMA control register (SDMMC_IDMACTRLR)	1182
26.10.16	SDMMC IDMA buffer size register (SDMMC_IDMABSIZE)	1183

	26.10.17 SDMMC IDMA buffer base address register (SDMMC_IDMABASER)	1184
	26.10.18 SDMMC IDMA linked list address register (SDMMC_IDMALAR) ...	1184
	26.10.19 SDMMC IDMA linked list memory base register (SDMMC_IDMABAR)	1185
	26.10.20 SDMMC data FIFO registers x (SDMMC_FIFORx)	1186
	26.10.21 SDMMC register map	1186
27	Delay block (DLYB)	1189
	27.1 DLYB introduction	1189
	27.2 DLYB main features	1189
	27.3 DLYB implementation	1189
	27.4 DLYB functional description	1189
	27.4.1 DLYB diagram	1189
	27.4.2 DLYB pins and internal signals	1190
	27.4.3 General description	1190
	27.4.4 Delay line length configuration procedure	1191
	27.4.5 Output clock phase configuration procedure	1192
	27.5 DLYB registers	1192
	27.5.1 DLYB control register (DLYB_CR)	1192
	27.5.2 DLYB configuration register (DLYB_CFGR)	1193
	27.5.3 DLYB register map	1194
28	Analog-to-digital converters (ADC1/2/3)	1195
	28.1 ADC introduction	1195
	28.2 ADC main features	1195
	28.3 ADC implementation	1197
	28.4 ADC functional description	1198
	28.4.1 ADC block diagram	1198
	28.4.2 ADC pins and internal signals	1199
	28.4.3 ADC clocks	1202
	28.4.4 ADC connectivity	1204
	28.4.5 Slave AHB interface	1207
	28.4.6 ADC Deep-power-down mode (DEEPPWD) and ADC voltage regulator (ADVREGEN)	1207
	28.4.7 Single-ended and differential input channels	1208
	28.4.8 Calibration (ADCAL, ADCALDIF, ADC_CALFACT)	1208

28.4.9	ADC on-off control (ADEN, ADDIS, ADRDY)	1211
28.4.10	Constraints when writing the ADC control bits	1212
28.4.11	Channel selection (ADC_SQRy, ADC_JSQR)	1213
28.4.12	Channel-wise programmable sampling time (SMPR1, SMPR2)	1214
28.4.13	Single conversion mode (CONT = 0)	1216
28.4.14	Continuous conversion mode (CONT = 1)	1216
28.4.15	Starting conversions (ADSTART, JADSTART)	1217
28.4.16	ADC timing	1218
28.4.17	Stopping an ongoing conversion (ADSTP, JADSTP)	1219
28.4.18	Conversion on external trigger and trigger polarity (EXTSEL, EXTEN, JEXTSEL, JEXTEN)	1220
28.4.19	Injected channel management	1222
28.4.20	Discontinuous mode (DISCEN, DISCNUM, JDISCEN)	1223
28.4.21	Queue of context for injected conversions	1224
28.4.22	Programmable resolution (RES) - fast conversion mode	1232
28.4.23	End of conversion, end of sampling phase (EOC, JEOC, EOSMP) .	1233
28.4.24	End of conversion sequence (EOS, JEOS)	1233
28.4.25	Timing diagrams example (single/continuous modes, hardware/software triggers)	1234
28.4.26	Data management	1236
28.4.27	Dynamic low-power features	1242
28.4.28	Analog window watchdog (AWD1EN, JAWD1EN, AWD1SGL, AWD1CH, AWD2CH, AWD3CH, AWD_HTx, AWD_LTx, AWDx)	1247
28.4.29	Oversampler	1251
28.4.30	Dual ADC modes	1258
28.4.31	Temperature sensor	1271
28.4.32	VBAT supply monitoring	1273
28.4.33	Monitoring the internal voltage reference	1274
28.4.34	Monitoring the supply voltage	1275
28.5	ADC interrupts	1276
28.6	ADC registers (for each ADC)	1277
28.6.1	ADC interrupt and status register (ADC_ISR)	1277
28.6.2	ADC interrupt enable register (ADC_IER)	1279
28.6.3	ADC control register (ADC_CR)	1281
28.6.4	ADC configuration register (ADC_CFGR)	1284
28.6.5	ADC configuration register 2 (ADC_CFGR2)	1289
28.6.6	ADC sample time register 1 (ADC_SMPR1)	1291
28.6.7	ADC sample time register 2 (ADC_SMPR2)	1292

28.6.8	ADC watchdog threshold register 1 (ADC_TR1)	1293
28.6.9	ADC watchdog threshold register 2 (ADC_TR2)	1294
28.6.10	ADC watchdog threshold register 3 (ADC_TR3)	1295
28.6.11	ADC regular sequence register 1 (ADC_SQR1)	1295
28.6.12	ADC regular sequence register 2 (ADC_SQR2)	1296
28.6.13	ADC regular sequence register 3 (ADC_SQR3)	1297
28.6.14	ADC regular sequence register 4 (ADC_SQR4)	1298
28.6.15	ADC regular data register (ADC_DR)	1299
28.6.16	ADC injected sequence register (ADC_JSQR)	1299
28.6.17	ADC offset y register (ADC_OFRy)	1302
28.6.18	ADC injected channel y data register (ADC_JDRy)	1303
28.6.19	ADC analog watchdog 2 configuration register (ADC_AWD2CR) . . .	1304
28.6.20	ADC analog watchdog 3 configuration register (ADC_AWD3CR) . . .	1304
28.6.21	ADC Differential mode selection register (ADC_DIFSEL)	1305
28.6.22	ADC calibration factors (ADC_CALFACT)	1306
28.6.23	ADC option register (ADC_OR)	1306
28.7	ADC common registers	1307
28.7.1	ADC common status register (ADC_CSR)	1307
28.7.2	ADC common control register (ADC_CCR)	1309
28.7.3	ADC common regular data register for dual mode (ADC_CDR)	1312
28.8	ADC register map	1313
29	Digital temperature sensor (DTS)	1317
29.1	DTS introduction	1317
29.2	DTS main features	1317
29.3	DTS functional description	1318
29.3.1	DTS block diagram	1318
29.3.2	DTS internal signals	1318
29.3.3	DTS block operation	1319
29.3.4	Operating modes	1319
29.3.5	Calibration	1319
29.3.6	Prescaler	1319
29.3.7	Temperature measurement principles	1320
29.3.8	Sampling time	1321
29.3.9	Quick measurement mode	1321
29.3.10	Trigger input	1322
29.3.11	On-off control and ready flag	1322

	29.3.12	Temperature measurement sequence	1323
29.4		DTS low-power modes	1324
29.5		DTS interrupts	1324
	29.5.1	Temperature window comparator	1324
	29.5.2	Synchronous interrupt	1324
	29.5.3	Asynchronous wake-up	1324
29.6		DTS registers	1325
	29.6.1	Temperature sensor configuration register 1 (DTS_CFGR1)	1325
	29.6.2	Temperature sensor T0 value register 1 (DTS_T0VALR1)	1327
	29.6.3	Temperature sensor ramp value register (DTS_RAMPVALR)	1327
	29.6.4	Temperature sensor interrupt threshold register 1 (DTS_ITR1)	1328
	29.6.5	Temperature sensor data register (DTS_DR)	1328
	29.6.6	Temperature sensor status register (DTS_SR)	1329
	29.6.7	Temperature sensor interrupt enable register (DTS_ITENR)	1330
	29.6.8	Temperature sensor clear interrupt flag register (DTS_ICIFR)	1331
	29.6.9	Temperature sensor option register (DTS_OR)	1332
	29.6.10	DTS register map	1333
30		Digital-to-analog converter (DAC)	1335
30.1		DAC introduction	1335
30.2		DAC main features	1335
30.3		DAC implementation	1336
30.4		DAC functional description	1337
	30.4.1	DAC block diagram	1337
	30.4.2	DAC pins and internal signals	1338
	30.4.3	DAC clocks	1339
	30.4.4	DAC channel enable	1339
	30.4.5	DAC data format	1339
	30.4.6	DAC conversion	1341
	30.4.7	DAC output voltage	1342
	30.4.8	DAC trigger selection	1342
	30.4.9	DMA requests	1343
	30.4.10	Noise generation	1344
	30.4.11	Triangle-wave generation	1345
	30.4.12	DAC channel modes	1346
	30.4.13	DAC channel buffer calibration	1349

30.4.14	DAC channel conversion modes	1350
30.4.15	Dual DAC channel conversion modes (if dual channels are available)	1351
30.5	DAC in low-power modes	1355
30.6	DAC interrupts	1355
30.7	DAC registers	1356
30.7.1	DAC control register (DAC_CR)	1356
30.7.2	DAC software trigger register (DAC_SWTRGR)	1359
30.7.3	DAC channel1 12-bit right-aligned data holding register (DAC_DHR12R1)	1360
30.7.4	DAC channel1 12-bit left aligned data holding register (DAC_DHR12L1)	1361
30.7.5	DAC channel1 8-bit right aligned data holding register (DAC_DHR8R1)	1361
30.7.6	DAC channel2 12-bit right aligned data holding register (DAC_DHR12R2)	1362
30.7.7	DAC channel2 12-bit left aligned data holding register (DAC_DHR12L2)	1362
30.7.8	DAC channel2 8-bit right-aligned data holding register (DAC_DHR8R2)	1363
30.7.9	Dual DAC 12-bit right-aligned data holding register (DAC_DHR12RD)	1363
30.7.10	Dual DAC 12-bit left aligned data holding register (DAC_DHR12LD)	1364
30.7.11	Dual DAC 8-bit right aligned data holding register (DAC_DHR8RD)	1364
30.7.12	DAC channel1 data output register (DAC_DOR1)	1365
30.7.13	DAC channel2 data output register (DAC_DOR2)	1365
30.7.14	DAC status register (DAC_SR)	1366
30.7.15	DAC calibration control register (DAC_CCR)	1367
30.7.16	DAC mode control register (DAC_MCR)	1368
30.7.17	DAC channel1 sample and hold sample time register (DAC_SHSR1)	1369
30.7.18	DAC channel2 sample and hold sample time register (DAC_SHSR2)	1370
30.7.19	DAC sample and hold time register (DAC_SHHR)	1370
30.7.20	DAC sample and hold refresh time register (DAC_SHRR)	1371
30.7.21	DAC register map	1372
31	Voltage reference buffer (VREFBUF)	1374

31.1	VREFBUF introduction	1374
31.2	VREFBUF implementation	1374
31.3	VREFBUF functional description	1374
31.4	VREFBUF trimming	1375
31.5	VREFBUF registers	1376
31.5.1	VREFBUF control and status register (VREFBUF_CSR)	1376
31.5.2	VREFBUF calibration control register (VREFBUF_CCR)	1377
31.5.3	VREFBUF register map	1378
32	Comparator (COMP)	1379
32.1	COMP introduction	1379
32.2	COMP main features	1379
32.3	COMP functional description	1380
32.3.1	COMP block diagram	1380
32.3.2	COMP pins and internal signals	1380
32.3.3	COMP reset and clocks	1382
32.3.4	Comparator LOCK mechanism	1383
32.3.5	Window comparator	1383
32.3.6	Hysteresis	1383
32.3.7	Comparator output blanking function	1384
32.3.8	COMP startup stabilization time	1385
32.3.9	COMP power and speed modes	1385
32.4	COMP low-power modes	1385
32.5	COMP interrupts	1386
32.5.1	Wake-up interrupt through EXTI	1386
32.5.2	Interrupt through EXTI	1386
32.5.3	Interrupt through the NVIC of the CPU	1387
32.6	SCALER function	1387
32.7	COMP registers	1388
32.7.1	Comparator status register (COMP_SR)	1388
32.7.2	Comparator interrupt clear flag register (COMP_ICFR)	1389
32.7.3	Comparator configuration register 1 (COMP_CFGR1)	1389
32.7.4	Comparator configuration register 2 (COMP_CFGR2)	1391
32.7.5	COMP register map	1393
33	Digital camera interface (DCMI)	1394

33.1	DCMI introduction	1394
33.2	DCMI main features	1394
33.3	DCMI functional description	1394
33.3.1	DCMI block diagram	1395
33.3.2	DCMI pins and internal signals	1395
33.3.3	DCMI clocks	1396
33.3.4	DCMI DMA interface	1396
33.3.5	DCMI physical interface	1396
33.3.6	DCMI synchronization	1398
33.3.7	DCMI capture modes	1400
33.3.8	DCMI crop feature	1401
33.3.9	DCMI JPEG format	1402
33.3.10	DCMI FIFO	1402
33.3.11	DCMI data format description	1403
33.4	DCMI interrupts	1405
33.5	DCMI registers	1405
33.5.1	DCMI control register (DCMI_CR)	1405
33.5.2	DCMI status register (DCMI_SR)	1408
33.5.3	DCMI raw interrupt status register (DCMI_RIS)	1408
33.5.4	DCMI interrupt enable register (DCMI_IER)	1409
33.5.5	DCMI masked interrupt status register (DCMI_MIS)	1410
33.5.6	DCMI interrupt clear register (DCMI_ICR)	1411
33.5.7	DCMI embedded synchronization code register (DCMI_ESCR)	1412
33.5.8	DCMI embedded synchronization unmask register (DCMI_ESUR)	1412
33.5.9	DCMI crop window start (DCMI_CWSTRT)	1413
33.5.10	DCMI crop window size (DCMI_CWSIZE)	1414
33.5.11	DCMI data register (DCMI_DR)	1414
33.5.12	DCMI register map	1415
34	Parallel synchronous slave interface (PSSI)	1416
34.1	PSSI introduction	1416
34.2	PSSI main features	1416
34.3	PSSI functional description	1416
34.3.1	PSSI block diagram	1417
34.3.2	PSSI pins and internal signals	1417
34.3.3	PSSI clock	1418

34.3.4	PSSI data management	1418
34.3.5	PSSI optional control signals	1420
34.4	PSSI interrupts	1423
34.5	PSSI registers	1424
34.5.1	PSSI control register (PSSI_CR)	1424
34.5.2	PSSI status register (PSSI_SR)	1425
34.5.3	PSSI raw interrupt status register (PSSI_RIS)	1426
34.5.4	PSSI interrupt enable register (PSSI_IER)	1427
34.5.5	PSSI masked interrupt status register (PSSI_MIS)	1427
34.5.6	PSSI interrupt clear register (PSSI_ICR)	1428
34.5.7	PSSI data register (PSSI_DR)	1428
34.5.8	PSSI register map	1429
35	Coupling and chaining bridge (CCB)	1430
35.1	CCB introduction	1430
35.2	CCB main features	1430
35.3	CCB implementation	1431
35.4	CCB functional description	1431
35.4.1	CCB block diagram	1431
35.4.2	CCB internal signals	1432
35.4.3	CCB reset and clocks	1432
35.4.4	CCB coupling and chaining	1432
35.4.5	CCB error management	1435
35.5	CCB coupling and chaining operations	1435
35.5.1	Preliminary steps before any protected operation	1437
35.5.2	ECDSA signature	1438
35.5.3	ECC scalar multiplication	1445
35.5.4	RSA modular exponentiation	1448
35.6	CCB interrupts	1450
35.7	CCB registers	1451
35.7.1	CCB control register (CCB_CR)	1451
35.7.2	CCB status register (CCB_SR)	1452
35.7.3	CCB reference tag register (CCB_REFTAGRx)	1454
35.7.4	CCB register map	1455
36	True random number generator (RNG)	1456

36.1	RNG introduction	1456
36.2	RNG main features	1456
36.3	RNG functional description	1457
36.3.1	RNG block diagram	1457
36.3.2	RNG internal signals	1457
36.3.3	Random number generation	1457
36.3.4	RNG initialization	1460
36.3.5	RNG operation	1461
36.3.6	RNG clocking	1463
36.3.7	Error management	1463
36.3.8	RNG low-power use	1464
36.4	RNG interrupts	1465
36.5	RNG processing time	1465
36.6	RNG entropy source validation	1466
36.6.1	Introduction	1466
36.6.2	Validation conditions	1466
36.7	RNG registers	1467
36.7.1	RNG control register (RNG_CR)	1467
36.7.2	RNG status register (RNG_SR)	1469
36.7.3	RNG data register (RNG_DR)	1470
36.7.4	RNG noise source control register (RNG_NSCR)	1471
36.7.5	RNG health test control register (RNG_HTCR)	1472
36.7.6	RNG register map	1472
37	True random number generator (RNG)	1473
37.1	RNG introduction	1473
37.2	RNG main features	1473
37.3	RNG functional description	1474
37.3.1	RNG block diagram	1474
37.3.2	RNG internal signals	1474
37.3.3	Random number generation	1475
37.3.4	RNG initialization	1477
37.3.5	RNG operation	1478
37.3.6	RNG clocking	1480
37.3.7	Error management	1480
37.3.8	RNG low-power use	1482

37.4	RNG interrupts	1482
37.5	RNG processing time	1483
37.6	RNG entropy source validation	1483
37.6.1	Introduction	1483
37.6.2	Validation conditions	1483
37.7	RNG registers	1485
37.7.1	RNG control register (RNG_CR)	1485
37.7.2	RNG status register (RNG_SR)	1487
37.7.3	RNG data register (RNG_DR)	1488
37.7.4	RNG noise source control register (RNG_NSCR)	1489
37.7.5	RNG health test control register x (RNG_HTCRx)	1489
37.7.6	RNG health test status register 0 (RNG_HTSR0)	1490
37.7.7	RNG health test status register 1 (RNG_HTSR1)	1490
37.7.8	RNG noise source mask register (RNG_NSMR)	1491
37.7.9	RNG register map	1492
38	Secure AES coprocessor (SAES)	1493
38.1	SAES introduction	1493
38.2	SAES main features	1493
38.3	SAES implementation	1494
38.4	SAES functional description	1494
38.4.1	SAES block diagram	1494
38.4.2	SAES internal signals	1495
38.4.3	SAES reset and clocks	1496
38.4.4	SAES symmetric cipher implementation	1496
38.4.5	SAES encryption or decryption typical usage	1497
38.4.6	SAES authenticated encryption, decryption, and cipher-based message authentication	1499
38.4.7	SAES ciphertext stealing and data padding	1500
38.4.8	SAES suspend and resume operations	1500
38.4.9	SAES basic chaining modes (ECB, CBC)	1501
38.4.10	SAES counter (CTR) mode	1505
38.4.11	SAES Galois/counter mode (GCM)	1507
38.4.12	SAES Galois message authentication code (GMAC)	1511
38.4.13	SAES counter with CBC-MAC (CCM)	1513
38.4.14	SAES operation with wrapped keys	1518
38.4.15	SAES operation with shared keys	1522

38.4.16	SAES data registers and data swapping	1523
38.4.17	SAES key registers	1526
38.4.18	SAES initialization vector registers	1527
38.4.19	SAES error management	1528
38.5	SAES interrupts	1530
38.6	SAES DMA requests	1530
38.7	SAES processing latency	1531
38.8	SAES registers	1533
38.8.1	SAES control register (SAES_CR)	1533
38.8.2	SAES status register (SAES_SR)	1536
38.8.3	SAES data input register (SAES_DINR)	1537
38.8.4	SAES data output register (SAES_DOUTR)	1538
38.8.5	SAES key register 0 (SAES_KEYR0)	1538
38.8.6	SAES key register 1 (SAES_KEYR1)	1539
38.8.7	SAES key register 2 (SAES_KEYR2)	1539
38.8.8	SAES key register 3 (SAES_KEYR3)	1539
38.8.9	SAES initialization vector register 0 (SAES_IVR0)	1540
38.8.10	SAES initialization vector register 1 (SAES_IVR1)	1540
38.8.11	SAES initialization vector register 2 (SAES_IVR2)	1540
38.8.12	SAES initialization vector register 3 (SAES_IVR3)	1541
38.8.13	SAES key register 4 (SAES_KEYR4)	1541
38.8.14	SAES key register 5 (SAES_KEYR5)	1541
38.8.15	SAES key register 6 (SAES_KEYR6)	1542
38.8.16	SAES key register 7 (SAES_KEYR7)	1542
38.8.17	SAES suspend registers (SAES_SUSPRx)	1542
38.8.18	SAES interrupt enable register (SAES_IER)	1543
38.8.19	SAES interrupt status register (SAES_ISR)	1544
38.8.20	SAES interrupt clear register (SAES_ICR)	1545
38.8.21	SAES register map	1546
39	AES hardware accelerator (AES)	1548
39.1	AES introduction	1548
39.2	AES main features	1548
39.3	AES implementation	1549
39.4	AES functional description	1549
39.4.1	AES block diagram	1549

39.4.2	AES internal signals	1550
39.4.3	AES reset and clocks	1550
39.4.4	AES symmetric cipher implementation	1550
39.4.5	AES encryption or decryption typical usage	1551
39.4.6	AES authenticated encryption, decryption, and cipher-based message authentication	1553
39.4.7	AES ciphertext stealing and data padding	1553
39.4.8	AES suspend and resume operations	1554
39.4.9	AES basic chaining modes (ECB, CBC)	1555
39.4.10	AES counter (CTR) mode	1558
39.4.11	AES Galois/counter mode (GCM)	1561
39.4.12	AES Galois message authentication code (GMAC)	1565
39.4.13	AES counter with CBC-MAC (CCM)	1566
39.4.14	AES key sharing with secure AES co-processor	1571
39.4.15	AES data registers and data swapping	1572
39.4.16	AES key registers	1574
39.4.17	AES initialization vector registers	1574
39.4.18	AES error management	1575
39.5	AES interrupts	1576
39.6	AES DMA requests	1576
39.7	AES processing latency	1577
39.8	AES registers	1579
39.8.1	AES control register (AES_CR)	1579
39.8.2	AES status register (AES_SR)	1581
39.8.3	AES data input register (AES_DINR)	1582
39.8.4	AES data output register (AES_DOUTR)	1583
39.8.5	AES key register 0 (AES_KEYR0)	1583
39.8.6	AES key register 1 (AES_KEYR1)	1584
39.8.7	AES key register 2 (AES_KEYR2)	1584
39.8.8	AES key register 3 (AES_KEYR3)	1584
39.8.9	AES initialization vector register 0 (AES_IVR0)	1585
39.8.10	AES initialization vector register 1 (AES_IVR1)	1585
39.8.11	AES initialization vector register 2 (AES_IVR2)	1585
39.8.12	AES initialization vector register 3 (AES_IVR3)	1586
39.8.13	AES key register 4 (AES_KEYR4)	1586
39.8.14	AES key register 5 (AES_KEYR5)	1586
39.8.15	AES key register 6 (AES_KEYR6)	1587

	39.8.16	AES key register 7 (AES_KEYR7)	1587
	39.8.17	AES suspend registers (AES_SUSPRx)	1587
	39.8.18	AES interrupt enable register (AES_IER)	1588
	39.8.19	AES interrupt status register (AES_ISR)	1589
	39.8.20	AES interrupt clear register (AES_ICR)	1590
	39.8.21	AES register map	1590
40		Hash processor (HASH)	1593
	40.1	HASH introduction	1593
	40.2	HASH main features	1593
	40.3	HASH implementation	1594
	40.4	HASH functional description	1594
	40.4.1	HASH block diagram	1594
	40.4.2	HASH internal signals	1595
	40.4.3	About secure hash algorithms	1595
	40.4.4	Message data feeding	1596
	40.4.5	Message digest computing	1597
	40.4.6	Message padding	1600
	40.4.7	HMAC operation	1601
	40.4.8	HASH suspend/resume operations	1603
	40.4.9	HASH DMA interface	1605
	40.4.10	HASH error management	1606
	40.4.11	HASH processing time	1606
	40.5	HASH interrupts	1606
	40.6	HASH registers	1607
	40.6.1	HASH control register (HASH_CR)	1607
	40.6.2	HASH data input register (HASH_DIN)	1609
	40.6.3	HASH start register (HASH_STR)	1610
	40.6.4	HASH digest registers	1611
	40.6.5	HASH interrupt enable register (HASH_IMR)	1613
	40.6.6	HASH status register (HASH_SR)	1613
	40.6.7	HASH context swap registers	1615
	40.6.8	HASH register map	1615
41		Public key accelerator (PKA)	1617
	41.1	PKA introduction	1617

41.2	PKA main features	1617
41.3	PKA implementation	1617
41.4	PKA functional description	1618
41.4.1	PKA block diagram	1618
41.4.2	PKA internal signals	1618
41.4.3	PKA reset and clocks	1619
41.4.4	PKA public key acceleration	1619
41.4.5	Typical applications for PKA	1621
41.4.6	PKA procedure to perform an operation	1623
41.4.7	Private keys protection using CCB	1624
41.4.8	PKA error management	1628
41.5	PKA operating modes	1628
41.5.1	Introduction	1628
41.5.2	Montgomery parameter computation	1630
41.5.3	Modular addition	1630
41.5.4	Modular subtraction	1631
41.5.5	Modular and Montgomery multiplication	1631
41.5.6	Modular exponentiation	1632
41.5.7	Modular inversion	1634
41.5.8	Modular reduction	1634
41.5.9	Arithmetic addition	1635
41.5.10	Arithmetic subtraction	1635
41.5.11	Arithmetic multiplication	1636
41.5.12	Arithmetic comparison	1636
41.5.13	RSA CRT exponentiation	1636
41.5.14	Point on elliptic curve Fp check	1637
41.5.15	ECC Fp scalar multiplication	1638
41.5.16	ECDSA sign	1639
41.5.17	ECDSA verification	1641
41.5.18	ECC complete addition	1642
41.5.19	ECC double base ladder	1642
41.5.20	ECC projective to affine	1643
41.6	Example of configurations and processing times	1644
41.6.1	Supported elliptic curves	1644
41.6.2	Computation times	1646
41.7	PKA interrupts	1648

41.8	PKA registers	1649
41.8.1	PKA control register (PKA_CR)	1649
41.8.2	PKA status register (PKA_SR)	1651
41.8.3	PKA clear flag register (PKA_CLRFR)	1653
41.8.4	PKA RAM	1654
41.8.5	PKA register map	1655
42	On-the-fly decryption engine (OTFDEC)	1656
42.1	OTFDEC introduction	1656
42.2	OTFDEC main features	1656
42.3	OTFDEC functional description	1657
42.3.1	OTFDEC block diagram	1657
42.3.2	OTFDEC internal signals	1657
42.3.3	OTFDEC on-the-fly decryption	1658
42.3.4	OTFDEC usage of AES in counter mode decryption	1659
42.3.5	Flow control management	1660
42.3.6	OTFDEC error management	1660
42.4	OTFDEC interrupts	1661
42.5	OTFDEC application information	1661
42.5.1	OTFDEC initialization process	1661
42.5.2	OTFDEC and power management	1663
42.5.3	Encrypting for OTFDEC	1663
42.5.4	OTFDEC key CRC source code	1664
42.6	OTFDEC registers	1665
42.6.1	OTFDEC control register (OTFDEC_CR)	1665
42.6.2	OTFDEC privileged access control configuration register (OTFDEC_PRIVCFGR)	1666
42.6.3	OTFDEC region x configuration register (OTFDEC_RxCFGR)	1666
42.6.4	OTFDEC region x start address register (OTFDEC_RxSTARTADDR)	1668
42.6.5	OTFDEC region x end address register (OTFDEC_RxENDADDR)	1668
42.6.6	OTFDEC region x nonce register 0 (OTFDEC_RxNONCER0)	1669
42.6.7	OTFDEC region x nonce register 1 (OTFDEC_RxNONCER1)	1670
42.6.8	OTFDEC region x key register 0 (OTFDEC_RxKEYR0)	1670
42.6.9	OTFDEC region x key register 1 (OTFDEC_RxKEYR1)	1671
42.6.10	OTFDEC region x key register 2 (OTFDEC_RxKEYR2)	1671
42.6.11	OTFDEC region x key register 3 (OTFDEC_RxKEYR3)	1672

42.6.12	OTFDEC interrupt status register (OTFDEC_ISR)	1672
42.6.13	OTFDEC interrupt clear register (OTFDEC_ICR)	1673
42.6.14	OTFDEC interrupt enable register (OTFDEC_IER)	1674
42.6.15	OTFDEC register map	1675
43	Advanced-control timers (TIM1/TIM8)	1679
43.1	TIM1/TIM8 introduction	1679
43.2	TIM1/TIM8 main features	1679
43.3	TIM1/TIM8 functional description	1680
43.3.1	Block diagram	1680
43.3.2	TIM1/TIM8 pins and internal signals	1681
43.3.3	Time-base unit	1686
43.3.4	Counter modes	1688
43.3.5	Repetition counter	1700
43.3.6	External trigger input	1701
43.3.7	Clock selection	1702
43.3.8	Capture/compare channels	1706
43.3.9	Input capture mode	1709
43.3.10	PWM input mode	1710
43.3.11	Forced output mode	1711
43.3.12	Output compare mode	1711
43.3.13	PWM mode	1713
43.3.14	Asymmetric PWM mode	1721
43.3.15	Combined PWM mode	1722
43.3.16	Combined 3-phase PWM mode	1723
43.3.17	Complementary outputs and dead-time insertion	1724
43.3.18	Using the break function	1727
43.3.19	Bidirectional break inputs	1733
43.3.20	Clearing the tim_ocxref signal on an external event	1734
43.3.21	6-step PWM generation	1736
43.3.22	One-pulse mode	1737
43.3.23	Retriggerable One-pulse mode	1739
43.3.24	Pulse on compare mode	1740
43.3.25	Encoder interface mode	1742
43.3.26	Direction bit output	1759
43.3.27	UIF bit remapping	1760
43.3.28	Timer input XOR function	1760

43.3.29	Interfacing with Hall sensors	1760
43.3.30	Timer synchronization	1762
43.3.31	ADC triggers	1767
43.3.32	DMA burst mode	1767
43.3.33	TIM1/TIM8 DMA requests	1768
43.3.34	Debug mode	1768
43.4	TIM1/TIM8 low-power modes	1769
43.5	TIM1/TIM8 interrupts	1769
43.6	TIM1/TIM8 registers	1770
43.6.1	TIMx control register 1 (TIMx_CR1)(x = 1, 8)	1770
43.6.2	TIMx control register 2 (TIMx_CR2)(x = 1, 8)	1771
43.6.3	TIMx slave mode control register (TIMx_SMCR)(x = 1, 8)	1775
43.6.4	TIMx DMA/interrupt enable register (TIMx_DIER)(x = 1, 8)	1779
43.6.5	TIMx status register (TIMx_SR)(x = 1, 8)	1780
43.6.6	TIMx event generation register (TIMx_EGR)(x = 1, 8)	1783
43.6.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)(x = 1, 8)	1784
43.6.8	TIMx capture/compare mode register 1 [alternate] (TIMx_CCMR1)(x = 1, 8)	1786
43.6.9	TIMx capture/compare mode register 2 (TIMx_CCMR2)(x = 1, 8)	1789
43.6.10	TIMx capture/compare mode register 2 [alternate] (TIMx_CCMR2)(x = 1, 8)	1790
43.6.11	TIMx capture/compare enable register (TIMx_CCER)(x = 1, 8)	1793
43.6.12	TIMx counter (TIMx_CNT)(x = 1, 8)	1797
43.6.13	TIMx prescaler (TIMx_PSC)(x = 1, 8)	1797
43.6.14	TIMx autoreload register (TIMx_ARR)(x = 1, 8)	1798
43.6.15	TIMx repetition counter register (TIMx_RCR)(x = 1, 8)	1798
43.6.16	TIMx capture/compare register 1 (TIMx_CCR1)(x = 1, 8)	1799
43.6.17	TIMx capture/compare register 2 (TIMx_CCR2)(x = 1, 8)	1799
43.6.18	TIMx capture/compare register 3 (TIMx_CCR3)(x = 1, 8)	1800
43.6.19	TIMx capture/compare register 4 (TIMx_CCR4)(x = 1, 8)	1801
43.6.20	TIMx break and dead-time register (TIMx_BDTR)(x = 1, 8)	1802
43.6.21	TIMx capture/compare register 5 (TIMx_CCR5)(x = 1, 8)	1806
43.6.22	TIMx capture/compare register 6 (TIMx_CCR6)(x = 1, 8)	1807
43.6.23	TIMx capture/compare mode register 3 (TIMx_CCMR3)(x = 1, 8)	1808
43.6.24	TIMx timer deadtime register 2 (TIMx_DTR2)(x = 1, 8)	1809

	43.6.25	TIMx timer encoder control register (TIMx_ECR)(x = 1, 8)	1810
	43.6.26	TIMx timer input selection register (TIMx_TISEL)(x = 1, 8)	1811
	43.6.27	TIMx alternate function option register 1 (TIMx_AF1)(x = 1, 8)	1812
	43.6.28	TIMx alternate function register 2 (TIMx_AF2)(x = 1, 8)	1815
	43.6.29	TIMx DMA control register (TIMx_DCR)(x = 1, 8)	1817
	43.6.30	TIMx DMA address for full transfer (TIMx_DMAR)(x = 1, 8)	1819
	43.6.31	TIMx register map	1819
44		General-purpose timers (TIM2/TIM3/TIM4/TIM5)	1822
	44.1	TIM2/TIM3/TIM4/TIM5 introduction	1822
	44.2	TIM2/TIM3/TIM4/TIM5 main features	1822
	44.3	TIM2/TIM3/TIM4/TIM5 implementation	1823
	44.4	TIM2/TIM3/TIM4/TIM5 functional description	1824
	44.4.1	Block diagram	1824
	44.4.2	TIM2/TIM3/TIM4/TIM5 pins and internal signals	1825
	44.4.3	Time-base unit	1829
	44.4.4	Counter modes	1831
	44.4.5	Clock selection	1842
	44.4.6	Capture/compare channels	1846
	44.4.7	Input capture mode	1848
	44.4.8	PWM input mode	1849
	44.4.9	Forced output mode	1850
	44.4.10	Output compare mode	1850
	44.4.11	PWM mode	1852
	44.4.12	Asymmetric PWM mode	1860
	44.4.13	Combined PWM mode	1861
	44.4.14	Clearing the tim_ocxref signal on an external event	1862
	44.4.15	One-pulse mode	1864
	44.4.16	Retriggerable one-pulse mode	1865
	44.4.17	Pulse on compare mode	1866
	44.4.18	Encoder interface mode	1868
	44.4.19	Direction bit output	1886
	44.4.20	UIF bit remapping	1887
	44.4.21	Timer input XOR function	1887
	44.4.22	Timers and external trigger synchronization	1887
	44.4.23	Timer synchronization	1891
	44.4.24	ADC triggers	1896

44.4.25	DMA burst mode	1897
44.4.26	TIM2/TIM3/TIM4/TIM5 DMA requests	1898
44.4.27	Debug mode	1898
44.4.28	TIM2/TIM3/TIM4/TIM5 low-power modes	1898
44.4.29	TIM2/TIM3/TIM4/TIM5 interrupts	1899
44.5	TIM2/TIM3/TIM4/TIM5 registers	1900
44.5.1	TIMx control register 1 (TIMx_CR1)(x = 2 to 5)	1900
44.5.2	TIMx control register 2 (TIMx_CR2)(x = 2 to 5)	1901
44.5.3	TIMx slave mode control register (TIMx_SMCR)(x = 2 to 5)	1903
44.5.4	TIMx DMA/Interrupt enable register (TIMx_DIER)(x = 2 to 5)	1907
44.5.5	TIMx status register (TIMx_SR)(x = 2 to 5)	1908
44.5.6	TIMx event generation register (TIMx_EGR)(x = 2 to 5)	1910
44.5.7	TIMx capture/compare mode register 1 (TIMx_CCMR1)(x = 2 to 5)	1911
44.5.8	TIMx capture/compare mode register 1 [alternate] (TIMx_CCMR1)(x = 2 to 5)	1913
44.5.9	TIMx capture/compare mode register 2 (TIMx_CCMR2)(x = 2 to 5)	1915
44.5.10	TIMx capture/compare mode register 2 [alternate] (TIMx_CCMR2)(x = 2 to 5)	1916
44.5.11	TIMx capture/compare enable register (TIMx_CCER)(x = 2 to 5)	1919
44.5.12	TIMx counter (TIMx_CNT)(x = 3, 4)	1920
44.5.13	TIMx counter (TIMx_CNT)(x = 2, 5)	1921
44.5.14	TIMx prescaler (TIMx_PSC)(x = 2 to 5)	1921
44.5.15	TIMx autoreload register (TIMx_ARR)(x = 3, 4)	1922
44.5.16	TIMx autoreload register (TIMx_ARR)(x = 2, 5)	1922
44.5.17	TIMx capture/compare register 1 (TIMx_CCR1)(x = 3, 4)	1923
44.5.18	TIMx capture/compare register 1 (TIMx_CCR1)(x = 2, 5)	1924
44.5.19	TIMx capture/compare register 2 (TIMx_CCR2)(x = 3, 4)	1924
44.5.20	TIMx capture/compare register 2 (TIMx_CCR2)(x = 2, 5)	1925
44.5.21	TIMx capture/compare register 3 (TIMx_CCR3)(x = 3, 4)	1926
44.5.22	TIMx capture/compare register 3 (TIMx_CCR3)(x = 2, 5)	1927
44.5.23	TIMx capture/compare register 4 (TIMx_CCR4)(x = 3, 4)	1928
44.5.24	TIMx capture/compare register 4 (TIMx_CCR4)(x = 2, 5)	1929
44.5.25	TIMx timer encoder control register (TIMx_ECR)(x = 2 to 5)	1930
44.5.26	TIMx timer input selection register (TIMx_TISEL)(x = 2 to 5)	1931
44.5.27	TIMx alternate function register 1 (TIMx_AF1)(x = 2 to 5)	1932
44.5.28	TIMx alternate function register 2 (TIMx_AF2)(x = 2 to 5)	1933
44.5.29	TIMx DMA control register (TIMx_DCR)(x = 2 to 5)	1934

	44.5.30	TIMx DMA address for full transfer (TIMx_DMAR)(x = 2 to 5)	1935
	44.5.31	TIMx register map	1936
45		Basic timers (TIM6/TIM7)	1939
	45.1	TIM6/TIM7 introduction	1939
	45.2	TIM6/TIM7 main features	1939
	45.3	TIM6/TIM7 functional description	1940
	45.3.1	TIM6/TIM7 block diagram	1940
	45.3.2	TIM6/TIM7 internal signals	1940
	45.3.3	TIM6/TIM7 clocks	1941
	45.3.4	Time-base unit	1941
	45.3.5	Counting mode	1943
	45.3.6	UIF bit remapping	1950
	45.3.7	ADC triggers	1951
	45.3.8	TIM6/TIM7 DMA requests	1951
	45.3.9	Debug mode	1951
	45.3.10	TIM6/TIM7 low-power modes	1951
	45.3.11	TIM6/TIM7 interrupts	1951
	45.4	TIM6/TIM7 registers	1952
	45.4.1	TIMx control register 1 (TIMx_CR1)(x = 6 to 7)	1952
	45.4.2	TIMx control register 2 (TIMx_CR2)(x = 6 to 7)	1954
	45.4.3	TIMx DMA/Interrupt enable register (TIMx_DIER)(x = 6 to 7)	1954
	45.4.4	TIMx status register (TIMx_SR)(x = 6 to 7)	1955
	45.4.5	TIMx event generation register (TIMx_EGR)(x = 6 to 7)	1955
	45.4.6	TIMx counter (TIMx_CNT)(x = 6 to 7)	1955
	45.4.7	TIMx prescaler (TIMx_PSC)(x = 6 to 7)	1956
	45.4.8	TIMx autoreload register (TIMx_ARR)(x = 6 to 7)	1956
	45.4.9	TIMx register map	1957
46		General-purpose timers (TIM12/TIM13/TIM14)	1958
	46.1	TIM12/TIM13/TIM14 introduction	1958
	46.2	TIM12 main features	1958
	46.3	TIM13/TIM14 main features	1959
	46.4	TIM12/TIM13/TIM14 functional description	1960
	46.4.1	Block diagram	1960
	46.4.2	TIM12/TIM13/TIM14 pins and internal signals	1961

46.4.3	Time-base unit	1964
46.4.4	Counter modes	1966
46.4.5	Clock selection	1969
46.4.6	Capture/compare channels	1971
46.4.7	Input capture mode	1973
46.4.8	PWM input mode (TIM12 only)	1974
46.4.9	Forced output mode	1975
46.4.10	Output compare mode	1976
46.4.11	PWM mode	1977
46.4.12	Combined PWM mode (TIM12 only)	1982
46.4.13	One-pulse mode	1983
46.4.14	Retriggerable one pulse mode (TIM12 only)	1985
46.4.15	UIF bit remapping	1986
46.4.16	Timer input XOR function	1986
46.4.17	TIM12 external trigger synchronization	1986
46.4.18	Slave mode – combined reset + trigger mode	1989
46.4.19	Slave mode – combined reset + gated mode	1989
46.4.20	Timer synchronization (TIM12 only)	1989
46.4.21	Using timer output as trigger for other timers (TIM13/TIM14 only)	1989
46.4.22	ADC triggers (TIM12 only)	1989
46.4.23	Debug mode	1989
46.5	TIM12/TIM13/TIM14 low-power modes	1990
46.6	TIM12/TIM13/TIM14 interrupts	1990
46.7	TIM12 registers	1990
46.7.1	TIM12 control register 1 (TIM12_CR1)	1990
46.7.2	TIM12 control register 2 (TIM12_CR2)	1992
46.7.3	TIM12 slave mode control register (TIM12_SMCR)	1992
46.7.4	TIM12 interrupt enable register (TIM12_DIER)	1994
46.7.5	TIM12 status register (TIM12_SR)	1995
46.7.6	TIM12 event generation register (TIM12_EGR)	1996
46.7.7	TIM12 capture/compare mode register 1 (TIM12_CCMR1)	1997
46.7.8	TIM12 capture/compare mode register 1 [alternate] (TIM12_CCMR1)	1998
46.7.9	TIM12 capture/compare enable register (TIM12_CCER)	2001
46.7.10	TIM12 counter (TIM12_CNT)	2002
46.7.11	TIM12 prescaler (TIM12_PSC)	2003

46.7.12	TIM12 autoreload register (TIM12_ARR)	2003
46.7.13	TIM12 capture/compare register 1 (TIM12_CCR1)	2004
46.7.14	TIM12 capture/compare register 2 (TIM12_CCR2)	2004
46.7.15	TIM12 timer input selection register (TIM12_TISEL)	2005
46.7.16	TIM12 register map	2006
46.8	TIM13/TIM14 registers	2008
46.8.1	TIMx control register 1 (TIMx_CR1)(x = 13, 14)	2008
46.8.2	TIMx interrupt enable register (TIMx_DIER)(x = 13, 14)	2009
46.8.3	TIMx status register (TIMx_SR)(x = 13, 14)	2009
46.8.4	TIMx event generation register (TIMx_EGR)(x = 13, 14)	2010
46.8.5	TIMx capture/compare mode register 1 (TIMx_CCMR1)(x = 13, 14)	2011
46.8.6	TIMx capture/compare mode register 1 [alternate] (TIMx_CCMR1)(x = 13, 14)	2012
46.8.7	TIMx capture/compare enable register (TIMx_CCER)(x = 13, 14)	2014
46.8.8	TIMx counter (TIMx_CNT)(x = 13, 14)	2015
46.8.9	TIMx prescaler (TIMx_PSC)(x = 13, 14)	2016
46.8.10	TIMx autoreload register (TIMx_ARR)(x = 13, 14)	2016
46.8.11	TIMx capture/compare register 1 (TIMx_CCR1)(x = 13, 14)	2017
46.8.12	TIMx timer input selection register (TIMx_TISEL)(x = 13, 14)	2017
46.8.13	TIM13/TIM14 register map	2018
47	General purpose timers (TIM15/TIM16/TIM17)	2020
47.1	TIM15/TIM16/TIM17 introduction	2020
47.2	TIM15 main features	2020
47.3	TIM16/TIM17 main features	2021
47.4	TIM15/TIM16/TIM17 functional description	2022
47.4.1	Block diagram	2022
47.4.2	TIM15/TIM16/TIM17 pins and internal signals	2023
47.4.3	Time-base unit	2027
47.4.4	Counter modes	2029
47.4.5	Repetition counter	2033
47.4.6	Clock selection	2034
47.4.7	Capture/compare channels	2036
47.4.8	Input capture mode	2038
47.4.9	PWM input mode (only for TIM15)	2040

47.4.10	Forced output mode	2041
47.4.11	Output compare mode	2041
47.4.12	PWM mode	2043
47.4.13	Combined PWM mode (TIM15 only)	2048
47.4.14	Complementary outputs and dead-time insertion	2049
47.4.15	Using the break function	2052
47.4.16	Bidirectional break input	2056
47.4.17	Clearing the tim_ocxref signal on an external event	2057
47.4.18	6-step PWM generation	2058
47.4.19	One-pulse mode	2060
47.4.20	Retriggerable one pulse mode (TIM15 only)	2061
47.4.21	UIF bit remapping	2062
47.4.22	Timer input XOR function (TIM15 only)	2062
47.4.23	External trigger synchronization (TIM15 only)	2062
47.4.24	Slave mode – combined reset + trigger mode (TIM15 only)	2065
47.4.25	Slave mode – combined reset + gated mode (TIM15 only)	2065
47.4.26	Timer synchronization (TIM15 only)	2066
47.4.27	Using timer output as trigger for other timers (TIM16/TIM17 only)	2066
47.4.28	ADC triggers (TIM15 only)	2066
47.4.29	DMA burst mode	2066
47.4.30	TIM15/TIM16/TIM17 DMA requests	2067
47.4.31	Debug mode	2067
47.5	TIM15/TIM16/TIM17 low-power modes	2068
47.6	TIM15/TIM16/TIM17 interrupts	2068
47.7	TIM15 registers	2068
47.7.1	TIM15 control register 1 (TIM15_CR1)	2069
47.7.2	TIM15 control register 2 (TIM15_CR2)	2070
47.7.3	TIM15 slave mode control register (TIM15_SMCR)	2072
47.7.4	TIM15 DMA/interrupt enable register (TIM15_DIER)	2074
47.7.5	TIM15 status register (TIM15_SR)	2075
47.7.6	TIM15 event generation register (TIM15_EGR)	2077
47.7.7	TIM15 capture/compare mode register 1 (TIM15_CCMR1)	2078
47.7.8	TIM15 capture/compare mode register 1 [alternate] (TIM15_CCMR1)	2079
47.7.9	TIM15 capture/compare enable register (TIM15_CCER)	2082
47.7.10	TIM15 counter (TIM15_CNT)	2085
47.7.11	TIM15 prescaler (TIM15_PSC)	2085

47.7.12	TIM15 autoreload register (TIM15_ARR)	2086
47.7.13	TIM15 repetition counter register (TIM15_RCR)	2086
47.7.14	TIM15 capture/compare register 1 (TIM15_CCR1)	2087
47.7.15	TIM15 capture/compare register 2 (TIM15_CCR2)	2088
47.7.16	TIM15 break and dead-time register (TIM15_BDTR)	2088
47.7.17	TIM15 timer deadtime register 2 (TIM15_DTR2)	2091
47.7.18	TIM15 input selection register (TIM15_TISEL)	2092
47.7.19	TIM15 alternate function register 1 (TIM15_AF1)	2093
47.7.20	TIM15 alternate function register 2 (TIM15_AF2)	2095
47.7.21	TIM15 DMA control register (TIM15_DCR)	2096
47.7.22	TIM15 DMA address for full transfer (TIM15_DMAR)	2097
47.7.23	TIM15 register map	2097
47.8	TIM16/TIM17 registers	2100
47.8.1	TIMx control register 1 (TIMx_CR1)(x = 16 to 17)	2100
47.8.2	TIMx control register 2 (TIMx_CR2)(x = 16 to 17)	2101
47.8.3	TIMx DMA/interrupt enable register (TIMx_DIER)(x = 16 to 17)	2102
47.8.4	TIMx status register (TIMx_SR)(x = 16 to 17)	2103
47.8.5	TIMx event generation register (TIMx_EGR)(x = 16 to 17)	2104
47.8.6	TIMx capture/compare mode register 1 (TIMx_CCMR1) (x = 16 to 17)	2105
47.8.7	TIMx capture/compare mode register 1 [alternate] (TIMx_CCMR1)(x = 16 to 17)	2106
47.8.8	TIMx capture/compare enable register (TIMx_CCER)(x = 16 to 17) .	2108
47.8.9	TIMx counter (TIMx_CNT)(x = 16 to 17)	2111
47.8.10	TIMx prescaler (TIMx_PSC)(x = 16 to 17)	2111
47.8.11	TIMx auto-reautoreload register (TIMx_ARR)(x = 16 to 17)	2112
47.8.12	TIMx repetition counter register (TIMx_RCR)(x = 16 to 17)	2112
47.8.13	TIMx capture/compare register 1 (TIMx_CCR1)(x = 16 to 17)	2113
47.8.14	TIMx break and dead-time register (TIMx_BDTR)(x = 16 to 17)	2114
47.8.15	TIMx timer deadtime register 2 (TIMx_DTR2)(x = 16 to 17)	2117
47.8.16	TIMx input selection register (TIMx_TISEL)(x = 16 to 17)	2118
47.8.17	TIMx alternate function register 1 (TIMx_AF1)(x = 16 to 17)	2118
47.8.18	TIMx alternate function register 2 (TIMx_AF2)(x = 16 to 17)	2121
47.8.19	TIM17 option register 1 (TIM17_OR1)	2121
47.8.20	TIMx DMA control register (TIMx_DCR)(x = 16 to 17)	2122
47.8.21	TIM16/TIM17 DMA address for full transfer (TIMx_DMAR)(x = 16 to 17)	2123
47.8.22	TIM16/TIM17 register map	2124

48	Low-power timer (LPTIM)	2127
48.1	LPTIM introduction	2127
48.2	LPTIM main features	2127
48.3	LPTIM implementation	2127
48.4	LPTIM functional description	2129
48.4.1	LPTIM block diagram	2129
48.4.2	LPTIM pins and internal signals	2130
48.4.3	LPTIM input and trigger mapping	2132
48.4.4	LPTIM reset and clocks	2133
48.4.5	Glitch filter	2134
48.4.6	Prescaler	2135
48.4.7	Trigger multiplexer	2135
48.4.8	Operating mode	2136
48.4.9	Timeout function	2138
48.4.10	Waveform generation	2138
48.4.11	Register update	2139
48.4.12	Counter mode	2140
48.4.13	Timer enable	2140
48.4.14	Timer counter reset	2141
48.4.15	Encoder mode	2141
48.4.16	Repetition counter	2143
48.4.17	Capture/compare channels	2144
48.4.18	Input capture mode	2145
48.4.19	PWM mode	2147
48.4.20	DMA requests	2149
48.4.21	Debug mode	2150
48.5	LPTIM low-power modes	2150
48.6	LPTIM interrupts	2150
48.7	LPTIM registers	2151
48.7.1	LPTIM4 interrupt and status register (LPTIM4_ISR)	2152
48.7.2	LPTIMx interrupt and status register [alternate] (LPTIMx_ISR) (x = 1 to 3, 5, 6)	2153
48.7.3	LPTIMx interrupt and status register [alternate] (LPTIMx_ISR) (x = 1 to 3, 5, 6)	2155
48.7.4	LPTIM4 interrupt clear register (LPTIM4_ICR)	2157
48.7.5	LPTIMx interrupt clear register [alternate] (LPTIMx_ICR) (x = 1 to 3, 5, 6)	2158

48.7.6	LPTIMx interrupt clear register [alternate] (LPTIMx_ICR) (x = 1 to 3, 5, 6)	2159
48.7.7	LPTIM4 interrupt enable register (LPTIM4_DIER)	2160
48.7.8	LPTIMx interrupt enable register [alternate] (LPTIMx_DIER) (x = 1 to 3, 5, 6)	2162
48.7.9	LPTIMx interrupt enable register [alternate] (LPTIMx_DIER) (x = 1 to 3, 5, 6)	2163
48.7.10	LPTIM configuration register (LPTIM_CFGR)	2165
48.7.11	LPTIM control register (LPTIM_CR)	2168
48.7.12	LPTIM compare register 1 (LPTIM_CCR1)	2169
48.7.13	LPTIM autoreload register (LPTIM_ARR)	2170
48.7.14	LPTIM counter register (LPTIM_CNT)	2170
48.7.15	LPTIM configuration register 2 (LPTIM_CFGR2)	2171
48.7.16	LPTIM repetition register (LPTIM_RCR)	2172
48.7.17	LPTIM capture/compare mode register 1 (LPTIM_CCMR1)	2172
48.7.18	LPTIM compare register 2 (LPTIM_CCR2)	2175
48.7.19	LPTIM register map	2175
49	Independent watchdog (IWDG)	2178
49.1	IWDG introduction	2178
49.2	IWDG main features	2178
49.3	IWDG implementation	2178
49.4	IWDG functional description	2179
49.4.1	IWDG block diagram	2179
49.4.2	IWDG internal signals	2180
49.4.3	Software and hardware watchdog modes	2180
49.4.4	Window option	2181
49.4.5	Debug	2184
49.4.6	Register access protection	2184
49.5	IWDG low power modes	2185
49.6	IWDG interrupts	2185
49.7	IWDG registers	2187
49.7.1	IWDG key register (IWDG_KR)	2188
49.7.2	IWDG prescaler register (IWDG_PR)	2188
49.7.3	IWDG reload register (IWDG_RLR)	2189
49.7.4	IWDG status register (IWDG_SR)	2189
49.7.5	IWDG window register (IWDG_WINR)	2191

	49.7.6	IWDG early wake-up interrupt register (IWDG_EWCR)	2191
	49.7.7	IWDG register map	2193
50		System window watchdog (WWDG)	2194
	50.1	WWDG introduction	2194
	50.2	WWDG main features	2194
	50.3	WWDG implementation	2194
	50.4	WWDG functional description	2195
	50.4.1	WWDG block diagram	2195
	50.4.2	WWDG internal signals	2195
	50.4.3	Enabling the watchdog	2196
	50.4.4	Controlling the down-counter	2196
	50.4.5	How to program the watchdog timeout	2196
	50.4.6	Debug mode	2197
	50.5	WWDG interrupts	2198
	50.6	WWDG registers	2198
	50.6.1	WWDG control register (WWDG_CR)	2198
	50.6.2	WWDG configuration register (WWDG_CFR)	2199
	50.6.3	WWDG status register (WWDG_SR)	2200
	50.6.4	WWDG register map	2200
51		Real-time clock (RTC)	2201
	51.1	RTC introduction	2201
	51.2	RTC main features	2201
	51.3	RTC functional description	2202
	51.3.1	RTC block diagram	2202
	51.3.2	RTC pins and internal signals	2204
	51.3.3	GPIOs controlled by the RTC and TAMP	2205
	51.3.4	RTC secure protection modes	2209
	51.3.5	RTC privilege protection modes	2211
	51.3.6	Clock and prescalers	2212
	51.3.7	Real-time clock and calendar	2213
	51.3.8	Calendar ultra-low power mode	2213
	51.3.9	Programmable alarms	2214
	51.3.10	Periodic auto-wake-up	2214
	51.3.11	RTC initialization and configuration	2215

51.3.12	Reading the calendar	2218
51.3.13	Resetting the RTC	2219
51.3.14	RTC synchronization	2219
51.3.15	RTC reference clock detection	2219
51.3.16	RTC smooth digital calibration	2220
51.3.17	Timestamp function	2222
51.3.18	Calibration clock output	2223
51.3.19	Tamper and alarm output	2224
51.4	RTC low-power modes	2224
51.5	RTC interrupts	2225
51.6	RTC registers	2226
51.6.1	RTC time register (RTC_TR)	2226
51.6.2	RTC date register (RTC_DR)	2227
51.6.3	RTC subsecond register (RTC_SSR)	2228
51.6.4	RTC initialization control and status register (RTC_ICSR)	2229
51.6.5	RTC prescaler register (RTC_PRER)	2231
51.6.6	RTC wake-up timer register (RTC_WUTR)	2232
51.6.7	RTC control register (RTC_CR)	2232
51.6.8	RTC privilege mode control register (RTC_PRIVCFGR)	2236
51.6.9	RTC secure configuration register (RTC_SECCFGR)	2238
51.6.10	RTC write protection register (RTC_WPR)	2239
51.6.11	RTC calibration register (RTC_CALR)	2240
51.6.12	RTC shift control register (RTC_SHIFTR)	2241
51.6.13	RTC timestamp time register (RTC_TSTR)	2242
51.6.14	RTC timestamp date register (RTC_TSDR)	2243
51.6.15	RTC timestamp subsecond register (RTC_TSSSR)	2244
51.6.16	RTC alarm A register (RTC_ALRMAR)	2244
51.6.17	RTC alarm A subsecond register (RTC_ALRMASR)	2246
51.6.18	RTC alarm B register (RTC_ALRMBR)	2247
51.6.19	RTC alarm B subsecond register (RTC_ALRMBSSR)	2248
51.6.20	RTC status register (RTC_SR)	2249
51.6.21	RTC nonsecure masked interrupt status register (RTC_MISR)	2251
51.6.22	RTC secure masked interrupt status register (RTC_SMISR)	2252
51.6.23	RTC status clear register (RTC_SCR)	2253
51.6.24	RTC option register (RTC_OR)	2254
51.6.25	RTC alarm A binary mode register (RTC_ALRABINR)	2254
51.6.26	RTC alarm B binary mode register (RTC_ALRBBINR)	2255

51.6.27	RTC register map	2256
52	Tamper and backup registers (TAMP)	2259
52.1	TAMP introduction	2259
52.2	TAMP main features	2260
52.3	TAMP implementation	2260
52.4	TAMP functional description	2261
52.4.1	TAMP block diagram	2261
52.4.2	TAMP pins and internal signals	2262
52.4.3	GPIOs controlled by the RTC and TAMP	2265
52.4.4	TAMP register write protection	2265
52.4.5	TAMP secure protection modes	2265
52.4.6	Backup registers protection zones	2266
52.4.7	TAMP privilege protection modes	2266
52.4.8	Boot hardware key (BHK)	2267
52.4.9	Tamper detection	2267
52.4.10	TAMP backup registers and other device secrets erase	2267
52.4.11	Tamper detection configuration and initialization	2269
52.5	TAMP low-power modes	2276
52.6	TAMP interrupts	2277
52.7	TAMP registers	2277
52.7.1	TAMP control register 1 (TAMP_CR1)	2277
52.7.2	TAMP control register 2 (TAMP_CR2)	2279
52.7.3	TAMP control register 3 (TAMP_CR3)	2282
52.7.4	TAMP filter control register (TAMP_FLTCR)	2283
52.7.5	TAMP active tamper control register 1 (TAMP_ATCR1)	2284
52.7.6	TAMP active tamper seed register (TAMP_ATSEEDR)	2287
52.7.7	TAMP active tamper output register (TAMP_ATOR)	2287
52.7.8	TAMP active tamper control register 2 (TAMP_ATCR2)	2288
52.7.9	TAMP secure configuration register (TAMP_SECCFGR)	2291
52.7.10	TAMP privilege configuration register (TAMP_PRIVCFGR)	2293
52.7.11	TAMP interrupt enable register (TAMP_IER)	2294
52.7.12	TAMP status register (TAMP_SR)	2296
52.7.13	TAMP nonsecure masked interrupt status register (TAMP_MISR)	2298
52.7.14	TAMP secure masked interrupt status register (TAMP_SMISR)	2299
52.7.15	TAMP status clear register (TAMP_SCR)	2301

52.7.16	TAMP monotonic counter 1 register (TAMP_COUNT1R)	2303
52.7.17	TAMP option register (TAMP_OR)	2303
52.7.18	TAMP resources protection configuration register (TAMP_RPCFGR)	2304
52.7.19	TAMP backup x register (TAMP_BKPxR)	2305
52.7.20	TAMP register map	2306
53	Inter-integrated circuit interface (I2C)	2308
53.1	I2C introduction	2308
53.2	I2C main features	2308
53.3	I2C implementation	2309
53.4	I2C functional description	2309
53.4.1	I2C block diagram	2310
53.4.2	I2C pins and internal signals	2310
53.4.3	I2C clock requirements	2311
53.4.4	I2C mode selection	2311
53.4.5	I2C initialization	2312
53.4.6	I2C reset	2316
53.4.7	I2C data transfer	2317
53.4.8	I2C target mode	2319
53.4.9	I2C controller mode	2328
53.4.10	I2C_TIMINGR register configuration examples	2339
53.4.11	SMBus specific features	2341
53.4.12	SMBus initialization	2343
53.4.13	SMBus I2C_TIMEOUTR register configuration examples	2345
53.4.14	SMBus target mode	2346
53.4.15	SMBus controller mode	2349
53.4.16	Wake-up from Stop mode on address match	2352
53.4.17	Error conditions	2353
53.5	I2C in low-power modes	2355
53.6	I2C interrupts	2355
53.7	I2C DMA requests	2356
53.7.1	Transmission using DMA	2356
53.7.2	Reception using DMA	2356
53.8	I2C debug modes	2356
53.9	I2C registers	2357
53.9.1	I2C control register 1 (I2C_CR1)	2357

53.9.2	I2C control register 2 (I2C_CR2)	2360
53.9.3	I2C own address 1 register (I2C_OAR1)	2362
53.9.4	I2C own address 2 register (I2C_OAR2)	2362
53.9.5	I2C timing register (I2C_TIMINGR)	2363
53.9.6	I2C timeout register (I2C_TIMEOUTR)	2364
53.9.7	I2C interrupt and status register (I2C_ISR)	2365
53.9.8	I2C interrupt clear register (I2C_ICR)	2368
53.9.9	I2C PEC register (I2C_PECR)	2369
53.9.10	I2C receive data register (I2C_RXDR)	2369
53.9.11	I2C transmit data register (I2C_TXDR)	2370
53.9.12	I2C register map	2371
54	Improved inter-integrated circuit (I3C)	2372
54.1	I3C introduction	2372
54.2	I3C main features	2372
54.3	I3C implementation	2374
54.3.1	I3C instantiation	2374
54.3.2	I3C wake-up from low-power mode(s)	2374
54.3.3	I3C FIFOs	2374
54.3.4	I3C triggers	2374
54.3.5	I3C interrupt(s)	2374
54.3.6	I3C MIPI® support	2375
54.4	I3C block diagram	2376
54.5	I3C pins and internal signals	2376
54.6	I3C reset and clocks	2377
54.6.1	I3C reset	2377
54.6.2	I3C clocks and requirements	2377
54.7	I3C peripheral state and programming	2379
54.7.1	I3C peripheral state	2379
54.7.2	I3C controller state and programming sequence	2380
54.7.3	I3C target state and programming sequence	2385
54.8	I3C registers and programming	2392
54.8.1	I3C register set, as controller/target	2392
54.8.2	I3C registers and fields use versus peripheral state, as controller	2393
54.8.3	I3C registers and fields usage versus peripheral state, as target	2395
54.9	I3C bus transfers and programming	2397

54.9.1	I3C command set (CCCs), as controller/target	2397
54.9.2	I3C broadcast/direct CCC transfer (except ENTDA, RSTACT), as controller	2401
54.9.3	I3C broadcast ENTDA CCC transfer, as controller	2403
54.9.4	I3C broadcast/direct RSTACT CCC transfer, as controller	2403
54.9.5	I3C broadcast/direct CCC transfer (except ENTDA, DEFTGTS, DEFGRPA), as target	2405
54.9.6	I3C broadcast ENTDA CCC transfer, as target	2407
54.9.7	I3C broadcast DEFTGTS CCC transfer, as target	2408
54.9.8	I3C broadcast DEFGRPA CCC transfer, as target	2409
54.9.9	I3C direct GETSTATUS CCC response, as target	2410
54.9.10	I3C private read/write transfer, as controller	2411
54.9.11	I3C private read/write transfer, as target	2412
54.9.12	Legacy I2C read/write transfer, as controller	2413
54.9.13	Legacy I2C read/write transfer, as target	2414
54.9.14	I3C IBI transfer, as controller/target	2414
54.9.15	I3C hot-join request transfer, as controller/target	2416
54.9.16	I3C controller-role request transfer, as controller/target	2417
54.10	I3C FIFOs management, as controller	2418
54.10.1	C-FIFO management, as controller	2418
54.10.2	TX-FIFO management, as controller	2419
54.10.3	RX-FIFO management, as controller	2422
54.10.4	S-FIFO management, as controller	2424
54.11	I3C FIFOs management, as target	2426
54.11.1	RX-FIFO management, as target	2426
54.11.2	TX-FIFO management, as target	2429
54.12	I3C error management	2433
54.12.1	Controller error management	2433
54.12.2	Target error management	2435
54.13	I3C in low-power modes	2436
54.14	I3C interrupts	2438
54.15	I3C registers	2438
54.15.1	I3C message control register (I3C_CR)	2438
54.15.2	I3C message control register [alternate] (I3C_CR)	2440
54.15.3	I3C configuration register (I3C_CFGR)	2442
54.15.4	I3C receive data byte register (I3C_RDR)	2447
54.15.5	I3C receive data word register (I3C_RDWR)	2448

54.15.6	I3C transmit data byte register (I3C_TDR)	2449
54.15.7	I3C transmit data word register (I3C_TDWR)	2450
54.15.8	I3C IBI payload data register (I3C_IBIDR)	2451
54.15.9	I3C target transmit configuration register (I3C_TGTTDR)	2452
54.15.10	I3C status register (I3C_SR)	2453
54.15.11	I3C status error register (I3C_SER)	2454
54.15.12	I3C received message register (I3C_RMR)	2456
54.15.13	I3C event register (I3C_EVR)	2457
54.15.14	I3C interrupt enable register (I3C_IER)	2461
54.15.15	I3C clear event register (I3C_CEVr)	2463
54.15.16	I3C masked interrupt status register (I3C_MISR)	2465
54.15.17	I3C own device characteristics register (I3C_DEVR0)	2466
54.15.18	I3C device x characteristics register (I3C_DEVRx)	2468
54.15.19	I3C maximum read length register (I3C_MAXRLR)	2470
54.15.20	I3C maximum write length register (I3C_MAXWLR)	2471
54.15.21	I3C I2C target configuration register (I3C_I2CCFGR)	2472
54.15.22	I3C timing register 0 (I3C_TIMINGR0)	2473
54.15.23	I3C timing register 1 (I3C_TIMINGR1)	2474
54.15.24	I3C timing register 2 (I3C_TIMINGR2)	2476
54.15.25	I3C bus characteristics register (I3C_BCR)	2478
54.15.26	I3C device characteristics register (I3C_DCR)	2479
54.15.27	I3C get capability register (I3C_GETCAPR)	2480
54.15.28	I3C controller-role capability register (I3C_CRCAPR)	2481
54.15.29	I3C get max data speed register (I3C_GETMXDSR)	2482
54.15.30	I3C extended provisioned ID register (I3C_EPIDR)	2484
54.15.31	I3C register map	2485
55	Improved inter-integrated circuit (I3C)	2488
55.1	I3C introduction	2488
55.2	I3C main features	2488
55.3	I3C implementation	2490
55.3.1	I3C instantiation	2490
55.3.2	I3C wake-up from low-power mode(s)	2490
55.3.3	I3C autonomous mode with DMA in Stop mode(s)	2490
55.3.4	I3C FIFOs	2490
55.3.5	I3C triggers	2490
55.3.6	I3C interrupt(s)	2490

55.3.7	I3C MIPI® support	2491
55.4	I3C block diagram	2492
55.5	I3C pins and internal signals	2492
55.6	I3C reset and clocks	2493
55.6.1	I3C reset	2493
55.6.2	I3C clocks and requirements	2493
55.7	I3C peripheral state and programming	2495
55.7.1	I3C peripheral state	2495
55.7.2	I3C controller state and programming sequence	2496
55.7.3	I3C target state and programming sequence	2501
55.8	I3C registers and programming	2505
55.8.1	I3C register set, as controller/target	2505
55.8.2	I3C registers and fields use versus peripheral state, as controller ..	2506
55.8.3	I3C registers and fields usage versus peripheral state, as target ...	2509
55.9	I3C bus transfers and programming	2511
55.9.1	I3C command set (CCCs), as controller/target	2511
55.9.2	I3C broadcast/direct CCC transfer (except ENTDA, RSTACT), as controller	2515
55.9.3	I3C broadcast ENTDA CCC transfer, as controller	2517
55.9.4	I3C broadcast/direct RSTACT CCC transfer, as controller	2517
55.9.5	I3C broadcast/direct CCC transfer (except ENTDA, DEFTGTS, DEFGRPA), as target	2519
55.9.6	I3C broadcast ENTDA CCC transfer, as target	2521
55.9.7	I3C broadcast DEFTGTS CCC transfer, as target	2522
55.9.8	I3C broadcast DEFGRPA CCC transfer, as target	2523
55.9.9	I3C direct GETSTATUS CCC response, as target	2524
55.9.10	I3C private read/write transfer, as controller	2525
55.9.11	I3C private read/write transfer, as target	2526
55.9.12	Legacy I2C read/write transfer, as controller	2527
55.9.13	I3C IBI transfer, as controller/target	2528
55.9.14	I3C hot-join request transfer, as controller/target	2529
55.9.15	I3C controller-role request transfer, as controller/target	2530
55.10	I3C FIFOs management, as controller	2531
55.10.1	C-FIFO management, as controller	2531
55.10.2	TX-FIFO management, as controller	2532
55.10.3	RX-FIFO management, as controller	2535
55.10.4	S-FIFO management, as controller	2537

55.11	I3C FIFOs management, as target	2539
55.11.1	RX-FIFO management, as target	2539
55.11.2	TX-FIFO management, as target	2540
55.12	I3C error management	2543
55.12.1	Controller error management	2543
55.12.2	Target error management	2545
55.13	I3C in low-power modes	2546
55.14	I3C interrupts	2547
55.15	I3C registers	2547
55.15.1	I3C message control register (I3C_CR)	2547
55.15.2	I3C message control register [alternate] (I3C_CR)	2550
55.15.3	I3C configuration register (I3C_CFGR)	2552
55.15.4	I3C receive data byte register (I3C_RDR)	2557
55.15.5	I3C receive data word register (I3C_RDWR)	2558
55.15.6	I3C transmit data byte register (I3C_TDR)	2559
55.15.7	I3C transmit data word register (I3C_TDWR)	2560
55.15.8	I3C IBI payload data register (I3C_IBIDR)	2561
55.15.9	I3C target transmit configuration register (I3C_TGTTDR)	2562
55.15.10	I3C status register (I3C_SR)	2563
55.15.11	I3C status error register (I3C_SER)	2564
55.15.12	I3C received message register (I3C_RMR)	2566
55.15.13	I3C event register (I3C_EVR)	2567
55.15.14	I3C interrupt enable register (I3C_IER)	2571
55.15.15	I3C clear event register (I3C_CEVr)	2573
55.15.16	I3C masked interrupt status register (I3C_MISR)	2575
55.15.17	I3C own device characteristics register (I3C_DEVR0)	2576
55.15.18	I3C device x characteristics register (I3C_DEVRx)	2578
55.15.19	I3C maximum read length register (I3C_MAXRLR)	2580
55.15.20	I3C maximum write length register (I3C_MAXWLR)	2581
55.15.21	I3C timing register 0 (I3C_TIMINGR0)	2582
55.15.22	I3C timing register 1 (I3C_TIMINGR1)	2583
55.15.23	I3C timing register 2 (I3C_TIMINGR2)	2585
55.15.24	I3C bus characteristics register (I3C_BCR)	2586
55.15.25	I3C device characteristics register (I3C_DCR)	2587
55.15.26	I3C get capability register (I3C_GETCAPR)	2588
55.15.27	I3C controller-role capability register (I3C_CRCAPR)	2589

	55.15.28 I3C get max data speed register (I3C_GETMXDSR)	2590
	55.15.29 I3C extended provisioned ID register (I3C_EPIDR)	2592
	55.15.30 I3C register map	2593
56	Universal synchronous/asynchronous receiver transmitter (USART/UART)	2596
56.1	USART introduction	2596
56.2	USART main features	2596
56.3	USART extended features	2597
56.4	USART implementation	2597
56.5	USART functional description	2599
56.5.1	USART block diagram	2599
56.5.2	USART pins and internal signals	2599
56.5.3	USART clocks	2601
56.5.4	USART character description	2601
56.5.5	USART FIFOs and thresholds	2604
56.5.6	USART transmitter	2604
56.5.7	USART receiver	2607
56.5.8	USART baud rate generation	2614
56.5.9	Tolerance of the USART receiver to clock deviation	2616
56.5.10	USART auto baud rate detection	2617
56.5.11	USART multiprocessor communication	2619
56.5.12	USART Modbus communication	2621
56.5.13	USART parity control	2622
56.5.14	USART LIN (local interconnection network) mode	2623
56.5.15	USART synchronous mode	2625
56.5.16	USART single-wire half-duplex communication	2629
56.5.17	USART receiver timeout	2629
56.5.18	USART smartcard mode	2630
56.5.19	USART IrDA SIR ENDEC block	2634
56.5.20	Continuous communication using USART and DMA	2637
56.5.21	RS232 hardware flow control and RS485 driver enable	2639
56.5.22	USART low-power management	2642
56.6	USART in low-power modes	2645
56.7	USART interrupts	2645
56.8	USART registers	2648

56.8.1	USART control register 1 (USART_CR1)	2648
56.8.2	USART control register 1 [alternate] (USART_CR1)	2652
56.8.3	USART control register 2 (USART_CR2)	2655
56.8.4	USART control register 3 (USART_CR3)	2659
56.8.5	USART control register 3 [alternate] (USART_CR3)	2663
56.8.6	USART baud rate register (USART_BRR)	2666
56.8.7	USART guard time and prescaler register (USART_GTPR)	2667
56.8.8	USART receiver timeout register (USART_RTOR)	2668
56.8.9	USART request register (USART_RQR)	2669
56.8.10	USART interrupt and status register (USART_ISR)	2670
56.8.11	USART interrupt and status register [alternate] (USART_ISR)	2676
56.8.12	USART interrupt flag clear register (USART_ICR)	2681
56.8.13	USART receive data register (USART_RDR)	2682
56.8.14	USART transmit data register (USART_TDR)	2683
56.8.15	USART prescaler register (USART_PRESC)	2683
56.8.16	USART register map	2684

57	Low-power universal asynchronous receiver transmitter (LPUART)	2686
57.1	LPUART introduction	2686
57.2	LPUART main features	2686
57.3	LPUART implementation	2687
57.4	LPUART functional description	2689
57.4.1	LPUART block diagram	2689
57.4.2	LPUART pins and internal signals	2690
57.4.3	LPUART clocks	2691
57.4.4	LPUART character description	2691
57.4.5	LPUART FIFOs and thresholds	2693
57.4.6	LPUART transmitter	2693
57.4.7	LPUART receiver	2697
57.4.8	LPUART baud rate generation	2701
57.4.9	Tolerance of the LPUART receiver to clock deviation	2702
57.4.10	LPUART multiprocessor communication	2703
57.4.11	LPUART parity control	2705
57.4.12	LPUART single-wire half-duplex communication	2706
57.4.13	Continuous communication using DMA and LPUART	2706
57.4.14	RS232 hardware flow control and RS485 driver enable	2709

	57.4.15 LPUART low-power management	2711
57.5	LPUART in low-power modes	2714
57.6	LPUART interrupts	2715
57.7	LPUART registers	2716
	57.7.1 LPUART control register 1 (LPUART_CR1)	2716
	57.7.2 LPUART control register 1 [alternate] (LPUART_CR1)	2719
	57.7.3 LPUART control register 2 (LPUART_CR2)	2722
	57.7.4 LPUART control register 3 (LPUART_CR3)	2724
	57.7.5 LPUART control register 3 [alternate] (LPUART_CR3)	2727
	57.7.6 LPUART baud rate register (LPUART_BRR)	2729
	57.7.7 LPUART request register (LPUART_RQR)	2729
	57.7.8 LPUART interrupt and status register (LPUART_ISR)	2730
	57.7.9 LPUART interrupt and status register [alternate] (LPUART_ISR) ...	2735
	57.7.10 LPUART interrupt flag clear register (LPUART_ICR)	2738
	57.7.11 LPUART receive data register (LPUART_RDR)	2739
	57.7.12 LPUART transmit data register (LPUART_TDR)	2739
	57.7.13 LPUART prescaler register (LPUART_PRESC)	2740
	57.7.14 LPUART register map	2741
58	Serial peripheral interface (SPI)	2743
58.1	SPI introduction	2743
58.2	SPI main features	2743
58.3	SPI implementation	2744
58.4	SPI functional description	2745
	58.4.1 SPI block diagram	2745
	58.4.2 SPI pins and internal signals	2746
	58.4.3 SPI communication general aspects	2747
	58.4.4 Communications between one master and one slave	2747
	58.4.5 Standard multislave communication	2750
	58.4.6 Multimaster communication	2753
	58.4.7 Slave select (NSS pin) management	2754
	58.4.8 Ready pin (RDY) management	2758
	58.4.9 Communication formats	2758
	58.4.10 Configuring the SPI	2760
	58.4.11 Enabling the SPI	2761
	58.4.12 SPI data transmission and reception procedures	2762

58.4.13	Disabling the SPI	2766
58.4.14	Communication using DMA (direct memory addressing)	2767
58.5	SPI specific modes and control	2769
58.5.1	TI mode	2769
58.5.2	SPI error flags	2769
58.5.3	CRC computation	2773
58.6	SPI in low-power modes	2774
58.7	SPI interrupts	2774
58.8	I2S main features	2776
58.9	I2S functional description	2776
58.9.1	I2S general description	2776
58.9.2	Pin sharing with SPI function	2777
58.9.3	Bitfields usable in I2S/PCM mode	2777
58.9.4	Slave and master modes	2778
58.9.5	Supported audio protocols	2778
58.9.6	Additional serial interface flexibility	2784
58.9.7	Startup sequence	2786
58.9.8	Stop sequence	2788
58.9.9	Clock generator	2789
58.9.10	Internal FIFOs	2791
58.9.11	FIFO status flags	2792
58.9.12	Handling of underrun situation	2792
58.9.13	Handling of overrun situation	2793
58.9.14	Frame error detection	2794
58.9.15	DMA interface	2796
58.9.16	Programing examples	2796
58.10	I2S interrupts	2798
58.11	SPI/I2S registers	2799
58.11.1	SPI/I2S control register 1 (SPI_CR1)	2799
58.11.2	SPI/I2S control register 2 (SPI_CR2)	2801
58.11.3	SPI/I2S configuration register 1 (SPI_CFG1)	2801
58.11.4	SPI/I2S configuration register 2 (SPI_CFG2)	2805
58.11.5	SPI/I2S interrupt enable register (SPI_IER)	2807
58.11.6	SPI/I2S status register (SPI_SR)	2808
58.11.7	SPI/I2S interrupt/status flags clear register (SPI_IFCR)	2811
58.11.8	SPI/I2S transmit data register (SPI_TXDR)	2812

	58.11.9 SPI/I2S receive data register (SPI_RXDR)	2812
	58.11.10 SPI/I2S polynomial register (SPI_CRCPOLY)	2813
	58.11.11 SPI/I2S transmitter CRC register (SPI_TXCRC)	2813
	58.11.12 SPI/I2S receiver CRC register (SPI_RXCRC)	2814
	58.11.13 SPI/I2S underrun data register (SPI_UDRDR)	2815
	58.11.14 SPI/I2S configuration register (SPI_I2SCFGR)	2815
	58.11.15 SPI/I2S register map	2817
59	Serial audio interface (SAI)	2819
59.1	SAI introduction	2819
59.2	SAI main features	2819
59.3	SAI implementation	2820
59.4	SAI functional description	2820
59.4.1	SAI block diagram	2820
59.4.2	SAI pins and internal signals	2822
59.4.3	Main SAI modes	2823
59.4.4	SAI synchronization mode	2824
59.4.5	Audio data size	2825
59.4.6	Frame synchronization	2825
59.4.7	Slot configuration	2828
59.4.8	SAI clock generator	2830
59.4.9	Internal FIFOs	2833
59.4.10	PDM interface	2835
59.4.11	AC'97 link controller	2843
59.4.12	SPDIF output	2845
59.4.13	Specific features	2848
59.4.14	Error flags	2852
59.4.15	Disabling the SAI	2855
59.4.16	SAI DMA interface	2855
59.5	SAI interrupts	2856
59.6	SAI registers	2858
59.6.1	SAI global configuration register (SAI_GCR)	2858
59.6.2	SAI configuration register 1 (SAI_ACR1)	2858
59.6.3	SAI configuration register 2 (SAI_ACR2)	2861
59.6.4	SAI frame configuration register (SAI_AFRCR)	2863
59.6.5	SAI slot register (SAI_ASLOTR)	2864

59.6.6	SAI interrupt mask register (SAI_AIM)	2865
59.6.7	SAI status register (SAI_ASR)	2867
59.6.8	SAI clear flag register (SAI_ACLRFR)	2869
59.6.9	SAI data register (SAI_ADR)	2870
59.6.10	SAI configuration register 1 (SAI_BCR1)	2870
59.6.11	SAI configuration register 2 (SAI_BCR2)	2873
59.6.12	SAI frame configuration register (SAI_BFRCR)	2875
59.6.13	SAI slot register (SAI_BSLOTR)	2876
59.6.14	SAI interrupt mask register (SAI_BIM)	2877
59.6.15	SAI status register (SAI_BSR)	2878
59.6.16	SAI clear flag register (SAI_BCLRFR)	2880
59.6.17	SAI data register (SAI_BDR)	2881
59.6.18	SAI PDM control register (SAI_PDMCR)	2882
59.6.19	SAI PDM delay register (SAI_PDMDLY)	2883
59.6.20	SAI register map	2885
60	FD controller area network (FDCAN)	2887
60.1	FDCAN introduction	2887
60.2	FDCAN main features	2889
60.3	FDCAN functional description	2890
60.3.1	FDCAN block diagram	2890
60.3.2	FDCAN pins and internal signals	2891
60.3.3	Bit timing	2892
60.3.4	Operating modes	2893
60.3.5	Error management	2902
60.3.6	Message RAM	2903
60.3.7	FIFO acknowledge handling	2912
60.3.8	FDCAN Rx FIFO element	2912
60.3.9	FDCAN Tx buffer element	2914
60.3.10	FDCAN Tx event FIFO element	2916
60.3.11	FDCAN standard message ID filter element	2917
60.3.12	FDCAN extended message ID filter element	2918
60.4	FDCAN registers	2920
60.4.1	FDCAN core release register (FDCAN_CREL)	2920
60.4.2	FDCAN endian register (FDCAN_ENDN)	2920
60.4.3	FDCAN data bit timing and prescaler register (FDCAN_DBTP)	2920
60.4.4	FDCAN test register (FDCAN_TEST)	2921

60.4.5	FDCAN RAM watchdog register (FDCAN_RWD)	2922
60.4.6	FDCAN CC control register (FDCAN_CCCR)	2923
60.4.7	FDCAN nominal bit timing and prescaler register (FDCAN_NBTP)	2924
60.4.8	FDCAN timestamp counter configuration register (FDCAN_TSCC)	2926
60.4.9	FDCAN timestamp counter value register (FDCAN_TSCV)	2926
60.4.10	FDCAN timeout counter configuration register (FDCAN_TOCC)	2927
60.4.11	FDCAN timeout counter value register (FDCAN_TOCV)	2928
60.4.12	FDCAN error counter register (FDCAN_ECR)	2928
60.4.13	FDCAN protocol status register (FDCAN_PSR)	2929
60.4.14	FDCAN transmitter delay compensation register (FDCAN_TDCR)	2931
60.4.15	FDCAN interrupt register (FDCAN_IR)	2931
60.4.16	FDCAN interrupt enable register (FDCAN_IE)	2934
60.4.17	FDCAN interrupt line select register (FDCAN_ILS)	2936
60.4.18	FDCAN interrupt line enable register (FDCAN_ILE)	2937
60.4.19	FDCAN global filter configuration register (FDCAN_RXGFC)	2937
60.4.20	FDCAN extended ID and mask register (FDCAN_XIDAM)	2939
60.4.21	FDCAN high-priority message status register (FDCAN_HPMS)	2939
60.4.22	FDCAN Rx FIFO 0 status register (FDCAN_RXF0S)	2940
60.4.23	CAN Rx FIFO 0 acknowledge register (FDCAN_RXF0A)	2941
60.4.24	FDCAN Rx FIFO 1 status register (FDCAN_RXF1S)	2941
60.4.25	FDCAN Rx FIFO 1 acknowledge register (FDCAN_RXF1A)	2942
60.4.26	FDCAN Tx buffer configuration register (FDCAN_TXBC)	2942
60.4.27	FDCAN Tx FIFO/queue status register (FDCAN_TXFQS)	2943
60.4.28	FDCAN Tx buffer request pending register (FDCAN_TXBRP)	2943
60.4.29	FDCAN Tx buffer add request register (FDCAN_TXBAR)	2944
60.4.30	FDCAN Tx buffer cancellation request register (FDCAN_TXBCR)	2945
60.4.31	FDCAN Tx buffer transmission occurred register (FDCAN_TXBTO)	2945
60.4.32	FDCAN Tx buffer cancellation finished register (FDCAN_TXBCF)	2946
60.4.33	FDCAN Tx buffer transmission interrupt enable register (FDCAN_TXBTIE)	2946
60.4.34	FDCAN Tx buffer cancellation finished interrupt enable register (FDCAN_TXBCIE)	2947
60.4.35	FDCAN Tx event FIFO status register (FDCAN_TXEFS)	2947
60.4.36	FDCAN Tx event FIFO acknowledge register (FDCAN_TXEFA)	2948
60.4.37	FDCAN CFG clock divider register (FDCAN_CKDIV)	2948
60.4.38	FDCAN register map	2949

61 Universal serial bus full-speed host/device interface (USB) 2953

61.1	USB introduction	2953
61.2	USB main features	2953
61.3	USB implementation	2953
61.4	USB functional description	2954
61.4.1	USB block diagram	2954
61.4.2	USB pins and internal signals	2954
61.4.3	USB reset and clocks	2955
61.4.4	General description and Device mode functionality	2955
61.4.5	Description of USB blocks used in both Device and Host modes	2956
61.4.6	Description of host frame scheduler (HFS) specific to Host mode	2957
61.5	Programming considerations for Device and Host modes	2958
61.5.1	Generic USB Device programming	2958
61.5.2	System and power-on reset	2958
61.5.3	Double-buffered endpoints and usage in Device mode	2965
61.5.4	Double buffered channels: usage in Host mode	2967
61.5.5	Isochronous transfers in Device mode	2968
61.5.6	Isochronous transfers in Host mode	2969
61.5.7	Suspend/resume events	2970
61.6	USB registers	2973
61.6.1	USB control register (USB_CNTR)	2974
61.6.2	USB interrupt status register (USB_ISTR)	2977
61.6.3	USB frame number register (USB_FNR)	2981
61.6.4	USB Device address (USB_DADDR)	2981
61.6.5	USB LPM control and status register (USB_LPMCSR)	2982
61.6.6	USB battery charging detector (USB_BCDR)	2983
61.6.7	USB endpoint/channel n register (USB_CHEPnR)	2984
61.7	USBSRAM registers	2993
61.7.1	Channel/endpoint transmit buffer descriptor n (USB_CHEP_TXRXBD_n)	2994
61.7.2	Channel/endpoint receive buffer descriptor n [alternate] (USB_CHEP_TXRXBD_n)	2994
61.7.3	Channel/endpoint receive buffer descriptor n (USB_CHEP_RXTXBD_n)	2996
61.7.4	Channel/endpoint transmit buffer descriptor n [alternate] (USB_CHEP_RXTXBD_n)	2997
61.7.5	USBSRAM register map	2998
62	USB Type-C®/USB Power Delivery interface (UCPD)	2999

62.1	UCPD introduction	2999
62.2	UCPD main features	2999
62.3	UCPD implementation	3000
62.4	UCPD functional description	3000
62.4.1	UCPD block diagram	3001
62.4.2	UCPD reset and clocks	3002
62.4.3	Physical layer protocol	3003
62.4.4	UCPD BMC transmitter	3010
62.4.5	UCPD BMC receiver	3011
62.4.6	UCPD Type-C pull-ups (Rp) and pull-downs (Rd)	3013
62.4.7	UCPD Type-C voltage monitoring and de-bouncing	3013
62.4.8	UCPD fast role swap (FRS)	3014
62.4.9	UCPD DMA Interface	3014
62.4.10	Wake-up from Stop mode	3014
62.5	UCPD programming sequences	3014
62.5.1	Initialization phase	3015
62.5.2	Type-C state machine handling	3015
62.5.3	USB PD transmit	3017
62.5.4	USB PD receive	3018
62.5.5	UCPD software trimming	3019
62.6	UCPD low-power modes	3019
62.7	UCPD interrupts	3020
62.8	UCPD registers	3021
62.8.1	UCPD configuration register 1 (UCPD_CFGR1)	3021
62.8.2	UCPD configuration register 2 (UCPD_CFGR2)	3023
62.8.3	UCPD configuration register 3 (UCPD_CFGR3)	3023
62.8.4	UCPD control register (UCPD_CR)	3024
62.8.5	UCPD interrupt mask register (UCPD_IMR)	3026
62.8.6	UCPD status register (UCPD_SR)	3028
62.8.7	UCPD interrupt clear register (UCPD_ICR)	3031
62.8.8	UCPD Tx ordered set type register (UCPD_TX_ORDSETR)	3032
62.8.9	UCPD Tx payload size register (UCPD_TX_PAYSZR)	3032
62.8.10	UCPD Tx data register (UCPD_TXDR)	3033
62.8.11	UCPD Rx ordered set register (UCPD_RX_ORDSETR)	3033
62.8.12	UCPD Rx payload size register (UCPD_RX_PAYSZR)	3034
62.8.13	UCPD receive data register (UCPD_RXDR)	3035

62.8.14	UCPD Rx ordered set extension register 1 (UCPD_RX_ORDEXTR1)	3035
62.8.15	UCPD Rx ordered set extension register 2 (UCPD_RX_ORDEXTR2)	3036
62.8.16	UCPD register map	3036

63	Ethernet (ETH): media access control (MAC) with DMA controller	3039
63.1	Ethernet introduction	3039
63.2	Ethernet main features	3039
63.2.1	Standard compliance	3039
63.2.2	MAC features	3039
63.2.3	Transaction layer (MTL) features	3041
63.2.4	DMA block features	3042
63.2.5	Bus interface features	3042
63.3	Ethernet pins and internal signals	3043
63.4	Ethernet architecture	3045
63.4.1	DMA controller	3046
63.4.2	MTL	3054
63.4.3	MAC	3054
63.5	Ethernet functional description: MAC	3059
63.5.1	Double VLAN processing	3059
63.5.2	Source address and VLAN insertion, replacement, or deletion	3060
63.5.3	Packet filtering	3062
63.5.4	IEEE 1588 timestamp support	3069
63.5.5	Checksum offload engine	3093
63.5.6	TCP segmentation offload	3099
63.5.7	IPv4 ARP offload	3105
63.5.8	Loopback	3106
63.5.9	Flow control	3107
63.5.10	MAC management counters	3109
63.5.11	Interrupts generated by the MAC	3111
63.5.12	MAC and MMC register descriptions	3111
63.6	Ethernet functional description: PHY interfaces	3112
63.6.1	Station management agent (SMA)	3112
63.6.2	Media independent interface (MII)	3119
63.6.3	Reduced media independent interface (RMII)	3120

63.7	Ethernet low-power modes	3123
63.7.1	Low-power management	3123
63.7.2	Energy-efficient Ethernet (EEE)	3129
63.8	Ethernet interrupts	3135
63.8.1	DMA interrupts	3135
63.8.2	MTL interrupts	3137
63.8.3	MAC interrupts	3137
63.9	Ethernet programming model	3138
63.9.1	DMA initialization	3138
63.9.2	MTL initialization	3139
63.9.3	MAC initialization	3139
63.9.4	Performing normal receive and transmit operation	3140
63.9.5	Stopping and starting transmission	3141
63.9.6	Programming guidelines for switching to new descriptor list in RxDMA	3141
63.9.7	Programming guidelines for switching the AHB clock frequency	3141
63.9.8	Programming guidelines for MII link state transitions	3142
63.9.9	Programming guidelines for IEEE 1588 timestamping	3143
63.9.10	Programming guidelines for PTP offload feature	3144
63.9.11	Programming guidelines for Energy Efficient Ethernet (EEE)	3148
63.9.12	Programming guidelines for flexible pulse-per-second (PPS) output	3150
63.9.13	Programming guidelines for IEEE 1588 auxiliary snapshot	3152
63.9.14	Programming guidelines for TSO	3152
63.9.15	Programming guidelines to perform VLAN filtering on the receiver . .	3153
63.10	Descriptors	3154
63.10.1	Descriptor overview	3154
63.10.2	Descriptor structure	3154
63.10.3	Transmit descriptor	3158
63.10.4	Receive descriptor	3170
63.11	Ethernet registers	3182
63.11.1	Ethernet register maps	3182
63.11.2	Ethernet DMA registers	3182
63.11.3	Ethernet MTL registers	3203
63.11.4	Ethernet MAC and MMC registers	3213

64 Ethernet (ETH): media access control (MAC) with DMA controller 3307

64.1	Ethernet introduction	3307
64.2	Ethernet main features	3307
64.2.1	Standard compliance	3307
64.2.2	MAC features	3307
64.2.3	Transaction layer (MTL) features	3309
64.2.4	DMA block features	3310
64.2.5	Bus interface features	3310
64.3	Ethernet pins and internal signals	3311
64.4	Ethernet architecture	3313
64.4.1	DMA controller	3314
64.4.2	MTL	3322
64.4.3	MAC	3322
64.5	Ethernet functional description: MAC	3327
64.5.1	Double VLAN processing	3327
64.5.2	Source address and VLAN insertion, replacement, or deletion	3328
64.5.3	Packet filtering	3330
64.5.4	IEEE 1588 timestamp support	3337
64.5.5	Checksum offload engine	3361
64.5.6	TCP segmentation offload	3367
64.5.7	IPv4 ARP offload	3373
64.5.8	Loopback	3374
64.5.9	Flow control	3375
64.5.10	MAC management counters	3377
64.5.11	Interrupts generated by the MAC	3379
64.5.12	MAC and MMC register descriptions	3379
64.6	Ethernet functional description: PHY interfaces	3380
64.6.1	Station management agent (SMA)	3380
64.6.2	Media independent interface (MII)	3387
64.6.3	Reduced media independent interface (RMII)	3388
64.7	Ethernet low-power modes	3391
64.7.1	Low-power management	3391
64.7.2	Energy-efficient Ethernet (EEE)	3397
64.8	Ethernet interrupts	3403
64.8.1	DMA interrupts	3403
64.8.2	MTL interrupts	3406
64.8.3	MAC interrupts	3406

64.9	Ethernet programming model	3407
64.9.1	DMA initialization	3408
64.9.2	MTL initialization	3409
64.9.3	MAC initialization	3409
64.9.4	Performing normal receive and transmit operation	3410
64.9.5	Stopping and starting transmission	3411
64.9.6	Programming guidelines for switching to new descriptor list in RxDMA	3411
64.9.7	Programming guidelines for switching the AHB clock frequency	3411
64.9.8	Programming guidelines for MII link state transitions	3412
64.9.9	Programming guidelines for IEEE 1588 timestamping	3413
64.9.10	Programming guidelines for PTP offload feature	3414
64.9.11	Programming guidelines for Energy Efficient Ethernet (EEE)	3418
64.9.12	Programming guidelines for flexible pulse-per-second (PPS) output	3420
64.9.13	Programming guidelines for IEEE 1588 auxiliary snapshot	3422
64.9.14	Programming guidelines for TSO	3422
64.9.15	Programming guidelines to perform VLAN filtering on the receiver	3423
64.9.16	Programming guidelines for receive idle and receive interrupt watchdog timers	3424
64.10	Descriptors	3425
64.10.1	Descriptor overview	3425
64.10.2	Descriptor structure	3425
64.10.3	Transmit descriptor	3429
64.10.4	Receive descriptor	3441
64.11	Ethernet registers	3454
64.11.1	Ethernet register maps	3454
64.11.2	Ethernet DMA registers	3454
64.11.3	Ethernet MTL registers	3476
64.11.4	Ethernet MAC and MMC registers	3485
65	HDMI-CEC controller (CEC)	3587
65.1	HDMI-CEC introduction	3587
65.2	HDMI-CEC controller main features	3587
65.3	HDMI-CEC functional description	3588
65.3.1	HDMI-CEC pin and internal signals	3588
65.3.2	HDMI-CEC block diagram	3589
65.3.3	Message description	3589

65.3.4	Bit timing	3590
65.4	Arbitration	3590
65.4.1	SFT option bit	3592
65.5	Error handling	3592
65.5.1	Bit error	3592
65.5.2	Message error	3593
65.5.3	Bit rising error (BRE)	3593
65.5.4	Short bit period error (SBPE)	3593
65.5.5	Long bit period error (LBPE)	3593
65.5.6	Transmission error detection (TXERR)	3595
65.6	HDMI-CEC interrupts	3596
65.7	HDMI-CEC registers	3597
65.7.1	CEC control register (CEC_CR)	3597
65.7.2	CEC configuration register (CEC_CFGR)	3598
65.7.3	CEC Tx data register (CEC_TXDR)	3600
65.7.4	CEC Rx data register (CEC_RXDR)	3600
65.7.5	CEC interrupt and status register (CEC_ISR)	3600
65.7.6	CEC interrupt enable register (CEC_IER)	3602
65.7.7	HDMI-CEC register map	3604
66	Debug support (DBG)	3605
66.1	DBG introduction	3605
66.2	DBG functional description	3606
66.2.1	DBG block diagram	3606
66.2.2	DBG pins and internal signals	3606
66.2.3	DBG reset and clocks	3607
66.2.4	DBG power domains	3607
66.2.5	Debug and low-power modes	3607
66.2.6	Security	3608
66.2.7	Debug authentication	3610
66.3	Serial-wire and JTAG debug port (SWJ-DP)	3613
66.3.1	JTAG debug port	3614
66.3.2	Serial-wire debug port	3616
66.3.3	Debug port registers	3618
66.3.4	Debug port register map and reset values	3624
66.4	Access ports	3625

66.4.1	Access port registers	3626
66.4.2	Access port register map	3632
66.5	ROM tables	3633
66.5.1	System ROM table registers	3636
66.5.2	System ROM table register map	3640
66.5.3	MCU ROM table registers	3641
66.5.4	MCU ROM table register map	3645
66.5.5	Processor ROM table registers	3646
66.5.6	Processor ROM table register map	3651
66.6	Data watchpoint and trace unit (DWT)	3652
66.6.1	DWT registers	3652
66.6.2	DWT register map	3666
66.7	Instrumentation trace macrocell (ITM)	3669
66.7.1	ITM registers	3669
66.7.2	ITM register map	3677
66.8	Breakpoint unit (BPU)	3679
66.8.1	BPU registers	3679
66.8.2	BPU register map	3685
66.9	Embedded Trace Macrocell (ETM)	3686
66.9.1	ETM registers	3686
66.9.2	ETM register map	3710
66.10	Trace port interface unit (TPIU)	3714
66.10.1	TPIU registers	3715
66.10.2	TPIU register map	3724
66.11	Cross-trigger interface (CTI)	3726
66.11.1	CTI registers	3727
66.11.2	CTI register map	3737
66.12	Microcontroller debug unit (DBGMCU)	3739
66.12.1	Device ID	3739
66.12.2	Low-power mode emulation	3739
66.12.3	Peripheral clock freeze	3740
66.12.4	DBGMCU registers	3741
66.12.5	DBGMCU register map	3757
66.13	References	3759
67	Device electronic signature	3760

67.1	Unique device ID register (96 bits)	3760
67.2	Flash size data register	3761
67.3	Package data register	3761
68	Important security notice	3763
69	Revision history	3764

List of tables

Table 1.	Implementation of masters	121
Table 2.	Implementation of slaves	121
Table 3.	Securable peripherals by TZSC	125
Table 4.	TrustZone-aware peripherals	129
Table 5.	Memory map and peripheral register addresses (STM32H562/563/573xx)	134
Table 6.	Memory map and peripheral register addresses (STM32H523/533xx)	139
Table 7.	Memory map and peripheral register addresses (STM32H543/553xx)	143
Table 8.	Configuring security attributes with IDAU and SAU	156
Table 9.	MPCWMx resources	158
Table 10.	MPCBBx resources (STM32H562/563/573xx devices)	158
Table 11.	MPCBBx resources (STM32H523/533xx devices)	158
Table 12.	MPCBBx resources (STM32H543/553xx devices)	159
Table 13.	DMA channel use (security)	162
Table 14.	Secure alternate function between peripherals and allocated I/Os	164
Table 15.	Nonsecure peripheral functions that cannot be connected to secure I/Os	165
Table 16.	Nonsecure peripheral functions that can be connected to secure I/Os	165
Table 17.	TrustZone-aware DBGMCU nonsecure accesses management	166
Table 18.	DMA channel use (privilege)	170
Table 19.	Internal tamperers in TAMP	174
Table 20.	Effect of low-power modes on TAMP	175
Table 21.	Accelerated cryptographic operations	178
Table 22.	Main product life cycle transitions	180
Table 23.	Typical product life cycle phases	181
Table 24.	Software intellectual property protection with PRODUCT_STAT	186
Table 25.	Boot mode when TrustZone is disabled (TZEN = 0xC3) - STM32H523/43/62/63xx devices	190
Table 26.	Boot mode when TrustZone is enabled (TZEN = 0xB4) - STM32H523/43/62/63xx devices	190
Table 27.	Boot mode when TrustZone is disabled (TZEN = 0xC3) - STM32H533/53/73xx devices	191
Table 28.	Boot mode when TrustZone is enabled (TZEN = 0xB4) - STM32H533/53/73xx devices	191
Table 29.	GTZC features	194
Table 30.	GTZC1 subblock address offset	195
Table 31.	MPCWM resource assignment	195
Table 32.	MPCWM3 and MPCWM4 (subregions A and B)	195
Table 33.	MPCBB resource assignment (STM32H562/63/73xx devices)	195
Table 34.	MPCBB resource assignment (STM32H543/53xx devices)	196
Table 35.	MPCBB resource assignment (STM32H523/33xx devices)	196
Table 36.	Secure properties of subregions A and B	199
Table 37.	Privileged properties of subregions A and B	199
Table 38.	GTZC interrupt request	200
Table 39.	GTZC1 TZSC register map and reset values	220
Table 40.	GTZC1 TZIC register map and reset values	249
Table 41.	GTZC1 MPCBBz register map and reset values (z = 1 to 3)	253
Table 42.	SRAMs density (Kbytes)	254
Table 43.	Internal SRAMs features	255
Table 44.	SRAM2 features	258

Table 45.	Effect of low-power modes on RAMCFG	259
Table 46.	RAMCFG interrupt requests	259
Table 47.	RAMCFG register map and reset values	266
Table 48.	Recommended number of wait states and programming delay	275
Table 49.	Flash memory OTP organization	284
Table 50.	Read-only public data organization	285
Table 51.	Memory map and swapping options (STM32H562/563/573xx devices)	288
Table 52.	Memory map and swapping options (STM32H543/553xx devices)	288
Table 53.	Memory map and swapping options (STM32H523/533xx devices)	289
Table 54.	Recommended reactions to FLASH_OPSR contents	291
Table 55.	Option bytes organization	294
Table 56.	Specific modifying rules	298
Table 57.	OB modifiable in closed product	299
Table 58.	Option bytes key area	300
Table 59.	Default secure watermark	306
Table 60.	Flash memory TZ protection summary	306
Table 61.	TZ protection and bank or mass erase summary	307
Table 62.	Secure watermark-based area	308
Table 63.	Secure hide protection	309
Table 64.	HDP protections summary	310
Table 65.	Secure configuration block-based registers access conditions	310
Table 66.	Privilege protection summary	311
Table 67.	Privilege and mass or bank erase	311
Table 68.	Privilege configuration register access conditions (TZ enabled)	311
Table 69.	Privilege configuration register access conditions (TZ disabled)	312
Table 70.	Flash register accesses	312
Table 71.	Flash interface register protection summary	315
Table 72.	High-cycle area protection summary: access to data area address range	316
Table 73.	HDP protected definition	316
Table 74.	Privileged sectors and data area - Access to data area address range	316
Table 75.	Product states, debug states and debug policy	317
Table 76.	PRODUCT_STATE transitions	319
Table 77.	TZ OBK protection summary	320
Table 78.	OBK protection summary with TZ disabled	321
Table 79.	Access conditions to secure control register	321
Table 80.	Access conditions to nonsecure control register	321
Table 81.	OTP/RO access constraints	322
Table 82.	RSSLIB/NSSLIB accesses	322
Table 83.	RSSLIB/NSSLIB entry point access	323
Table 84.	RSS lib interface functions	323
Table 85.	OBKeys section boundaries	325
Table 86.	NSS lib interface functions	327
Table 87.	Effect of low-power modes on the embedded flash memory	329
Table 88.	Locating ECC failure	336
Table 89.	Flash interrupt request	339
Table 90.	Register map and reset value table	385
Table 91.	ICACHE features	391
Table 92.	TAG memory dimensioning parameters for n-way set associative operating mode (default)	393
Table 93.	TAG memory dimensioning parameters for direct-mapped cache mode	394
Table 94.	ICACHE cacheability for AHB transaction	396
Table 95.	Memory configurations	396

Table 96.	ICACHE remap region size, base address, and remap address	397
Table 97.	ICACHE interrupts	401
Table 98.	ICACHE register map and reset values	405
Table 99.	DCACHE features	408
Table 100.	TAG memory dimensioning parameters	411
Table 101.	DCACHE cacheability for AHB transaction	413
Table 102.	DCACHE interrupts	418
Table 103.	DCACHE register map and reset values	424
Table 104.	PWR input/output pins	427
Table 105.	PWR internal input/output signals	427
Table 106.	Analog switched recommended configuration	434
Table 107.	Low-power mode summary	446
Table 108.	Functionalities depending on the working mode	446
Table 109.	Sleep mode	451
Table 110.	Memory shut-off block selection	451
Table 111.	Stop mode	453
Table 112.	Standby mode	456
Table 113.	Power modes output states versus MCU power modes	456
Table 114.	PWR security configuration summary	457
Table 115.	PWR interrupt requests	459
Table 116.	PWR register map and reset values	478
Table 117.	RCC input/output signals connected to package pins or balls	480
Table 118.	Reset source identification (RCC_RSR)	482
Table 119.	STOPWUCK and STOPKERWUCK description	495
Table 120.	HSIKERON and CSIKERON behavior	496
Table 121.	Kernel clock distribution overview	498
Table 122.	RCC security configuration summary	505
Table 123.	Interrupt sources and control	509
Table 124.	RCC register map and reset values	597
Table 125.	CRS features	603
Table 126.	CRS internal input/output signals	604
Table 127.	CRS interconnection	605
Table 128.	Effect of low-power modes on CRS	608
Table 129.	Interrupt control bits	608
Table 130.	CRS register map and reset values	613
Table 131.	Port bit configuration	616
Table 132.	GPIO secured bits	624
Table 133.	GPIO register map and reset values	634
Table 134.	SBS internal input/output signals	638
Table 135.	HDPL encoded values	642
Table 136.	SBS boot logic	642
Table 137.	OBK-HDPL logic	647
Table 138.	SBS register map and reset values	662
Table 139.	Peripherals interconnect matrix	664
Table 140.	GPDMA1/2 channel implementation	679
Table 141.	GPDMA1/2 wake-up in low-power modes	679
Table 142.	Programmed GPDMA1/2 request	679
Table 143.	Programmed GPDMA1/2 request as a block request	684
Table 144.	GPDMA1/2 channel with peripheral early termination	684
Table 145.	Programmed GPDMA1/2 request with peripheral early termination	684
Table 146.	Programmed GPDMA1/2 trigger	685
Table 147.	Programmed GPDMA source/destination burst	707

Table 148.	Programmed data handling	712
Table 149.	Effect of low-power modes on GPDMA	725
Table 150.	GPDMA interrupt requests	726
Table 151.	GPDMA register map and reset values	755
Table 152.	STM32H562/563/573xx vector table	759
Table 153.	STM32H523/533xx vector table	764
Table 154.	STM32H543/553xx vector table	768
Table 155.	EXTI signals	776
Table 156.	EVG signals	777
Table 157.	EXTI line connections	777
Table 158.	Masking functionality	782
Table 159.	Register protection overview	782
Table 160.	EXTI register map sections	784
Table 161.	EXTI register map and reset values	822
Table 162.	CRC internal input/output signals	826
Table 163.	CRC register map and reset values	831
Table 164.	CORDIC functions	833
Table 165.	Cosine parameters	834
Table 166.	Sine parameters	834
Table 167.	Phase parameters	835
Table 168.	Modulus parameters	835
Table 169.	Arctangent parameters	836
Table 170.	Hyperbolic cosine parameters	836
Table 171.	Hyperbolic sine parameters	837
Table 172.	Hyperbolic arctangent parameters	837
Table 173.	Natural logarithm parameters	838
Table 174.	Natural log scaling factors and corresponding ranges	838
Table 175.	Square root parameters	839
Table 176.	Square root scaling factors and corresponding ranges	839
Table 177.	Precision vs. number of iterations	842
Table 178.	CORDIC register map and reset value	849
Table 179.	Valid combinations for read and write methods	863
Table 180.	FMAC register map and reset values	876
Table 181.	PLAY pins	879
Table 182.	PLAY internal signals	879
Table 183.	PLAY logic array input n signal selection	880
Table 184.	PLAY output connections	881
Table 185.	Truth table example 1	887
Table 186.	Truth table example 2	888
Table 187.	Logic element n LUT input selection	890
Table 188.	Logic element n register clock selection	891
Table 189.	Output n source selection	894
Table 190.	PLAY register map and reset values	911
Table 191.	NOR/PSRAM bank selection	919
Table 192.	NOR/PSRAM External memory address	919
Table 193.	NAND memory mapping and timing registers	919
Table 194.	NAND bank selection	920
Table 195.	SDRAM bank selection	920
Table 196.	SDRAM address mapping	920
Table 197.	SDRAM address mapping with 8-bit data bus width	921
Table 198.	SDRAM address mapping with 16-bit data bus width	922
Table 199.	Programmable NOR/PSRAM access parameters	923

Table 200.	Non-multiplexed I/O NOR flash memory	924
Table 201.	16-bit multiplexed I/O NOR flash memory	924
Table 202.	Non-multiplexed I/Os PSRAM/SRAM	925
Table 203.	16-Bit multiplexed I/O PSRAM	925
Table 204.	NOR flash/PSRAM: example of supported memories and transactions	926
Table 205.	FMC_BCRx bitfields (mode 1)	929
Table 206.	FMC_BTRx bitfields (mode 1)	929
Table 207.	FMC_BCRx bitfields (mode A)	931
Table 208.	FMC_BTRx bitfields (mode A)	931
Table 209.	FMC_BWTRx bitfields (mode A)	932
Table 210.	FMC_BCRx bitfields (mode 2/B)	934
Table 211.	FMC_BTRx bitfields (mode 2/B)	934
Table 212.	FMC_BWTRx bitfields (mode 2/B)	935
Table 213.	FMC_BCRx bitfields (mode C)	936
Table 214.	FMC_BTRx bitfields (mode C)	937
Table 215.	FMC_BWTRx bitfields (mode C)	937
Table 216.	FMC_BCRx bitfields (mode D)	939
Table 217.	FMC_BTRx bitfields (mode D)	940
Table 218.	FMC_BWTRx bitfields (mode D)	940
Table 219.	FMC_BCRx bitfields (Muxed mode)	942
Table 220.	FMC_BTRx bitfields (Muxed mode)	943
Table 221.	FMC_BCRx bitfields (Synchronous multiplexed read mode)	948
Table 222.	FMC_BTRx bitfields (Synchronous multiplexed read mode)	949
Table 223.	FMC_BCRx bitfields (Synchronous multiplexed write mode)	950
Table 224.	FMC_BTRx bitfields (Synchronous multiplexed write mode)	951
Table 225.	Programmable NAND flash access parameters	960
Table 226.	8-bit NAND flash	960
Table 227.	16-bit NAND flash	961
Table 228.	Supported memories and transactions	961
Table 229.	ECC result relevant bits	970
Table 230.	SDRAM signals	971
Table 231.	FMC register map and reset values	987
Table 232.	OCTOSPI implementation	991
Table 233.	OCTOSPI input/output pins	993
Table 234.	OCTOSPI internal signals	993
Table 235.	Command/address phase description	1002
Table 236.	OctaRAM command address bit assignment (based on 64-Mbyte OctaRAM)	1013
Table 237.	Address alignment cases	1020
Table 238.	OCTOSPI interrupt requests	1021
Table 239.	OCTOSPI register map and reset values	1044
Table 240.	OCTOSPI implementation	1048
Table 241.	OCTOSPI input/output pins	1050
Table 242.	OCTOSPI internal signals	1050
Table 243.	Command/address phase description	1059
Table 244.	OctaRAM command address bit assignment (based on 64-Mbyte OctaRAM)	1070
Table 245.	Address alignment cases	1078
Table 246.	OCTOSPI interrupt requests	1080
Table 247.	OCTOSPI register map and reset values	1103
Table 248.	SDMMC instances on device	1107

Table 249.	SDMMC features	1108
Table 250.	SDMMC operation modes SD and SDIO	1110
Table 251.	SDMMC operation modes eMMC	1110
Table 252.	SDMMC internal input/output signals	1111
Table 253.	SDMMC pins	1112
Table 254.	SDMMC Command and data phase selection	1113
Table 255.	Command token format	1119
Table 256.	Short response with CRC token format	1120
Table 257.	Short response without CRC token format	1120
Table 258.	Long response with CRC token format	1120
Table 259.	Specific Commands overview	1121
Table 260.	Command path status flags	1122
Table 261.	Command path error handling	1122
Table 262.	Data token format	1130
Table 263.	Data path status flags and clear bits	1130
Table 264.	Data path error handling	1132
Table 265.	Data FIFO access	1133
Table 266.	Transmit FIFO status flags	1134
Table 267.	Receive FIFO status flags	1135
Table 268.	AHB and SDMMC_CK clock frequency relation	1140
Table 269.	SDIO special operation control	1140
Table 270.	4-bit mode Start, interrupt, and CRC-status Signaling detection	1144
Table 271.	CMD12 use cases	1149
Table 272.	SDMMC interrupts	1163
Table 273.	Response type and SDMMC_RESPxR registers	1170
Table 274.	SDMMC register map	1186
Table 275.	STM32H5 features	1189
Table 276.	DLYB internal input/output signals	1190
Table 277.	Delay block control	1191
Table 278.	DLYB register map and reset values	1194
Table 279.	ADC features	1197
Table 280.	ADC input/output pins	1199
Table 281.	ADC internal input/output signals	1199
Table 282.	ADC interconnection	1199
Table 283.	Configuring the trigger polarity for regular external triggers	1220
Table 284.	Configuring the trigger polarity for injected external triggers	1221
Table 285.	TSAR timings depending on resolution	1233
Table 286.	Offset computation versus data resolution	1236
Table 287.	Analog watchdog channel selection	1247
Table 288.	Analog watchdog 1 comparison	1248
Table 289.	Analog watchdog 2 and 3 comparison	1248
Table 290.	Maximum output results versus N and M (gray cells indicate truncation)	1252
Table 291.	Oversampler operating modes summary	1257
Table 292.	ADC interrupts	1276
Table 293.	DELAY bits versus ADC resolution	1311
Table 294.	ADC global register map	1313
Table 295.	ADC register map and reset values for each ADC (offset = 0x000 for master ADC, 0x100 for slave ADC)	1313
Table 296.	ADC register map and reset values (master and slave ADC common registers)	1315
Table 297.	DTS internal input/output signals	1318
Table 298.	Sampling time configuration	1321

Table 299.	Trigger configuration	1322
Table 300.	Temperature sensor behavior in low-power modes	1324
Table 301.	Interrupt control bits	1325
Table 302.	DTS register map and reset values	1333
Table 303.	DAC features	1336
Table 304.	DAC input/output pins	1338
Table 305.	DAC internal input/output signals	1338
Table 306.	DAC interconnection	1338
Table 307.	Data format (case of 12-bit data)	1341
Table 308.	HFSEL description	1342
Table 309.	Sample and refresh timings	1347
Table 310.	Channel output modes summary	1349
Table 311.	Effect of low-power modes on DAC	1355
Table 312.	DAC interrupts	1355
Table 313.	DAC register map and reset values	1372
Table 314.	VREFBUF typical values	1374
Table 315.	VREF buffer modes	1375
Table 316.	VREFBUF register map and reset values	1378
Table 317.	COMP1/2 input/output internal signals	1380
Table 318.	COMP1 noninverting input assignment	1381
Table 319.	COMP1 inverting input assignment	1381
Table 320.	COMP2 noninverting input assignment	1381
Table 321.	COMP2 inverting input assignment	1381
Table 322.	COMP1 blanking source assignment	1382
Table 323.	COMP2 blanking source assignment	1382
Table 324.	Comparator behavior in low-power modes	1385
Table 325.	Interrupt through EXTI control bits	1386
Table 326.	Interrupt through NVIC control bits	1386
Table 327.	Interrupt through NVIC control bits	1387
Table 328.	COMP register map and reset values	1393
Table 329.	DCMI input/output pins	1395
Table 330.	DCMI internal input/output signals	1395
Table 331.	Positioning of captured data bytes in 32-bit words (8-bit width)	1397
Table 332.	Positioning of captured data bytes in 32-bit words (10-bit width)	1397
Table 333.	Positioning of captured data bytes in 32-bit words (12-bit width)	1397
Table 334.	Positioning of captured data bytes in 32-bit words (14-bit width)	1398
Table 335.	Data storage in monochrome progressive video format	1403
Table 336.	Data storage in RGB progressive video format	1404
Table 337.	Data storage in YCbCr progressive video format	1404
Table 338.	Data storage in YCbCr progressive video format - Y extraction mode	1404
Table 339.	DCMI interrupts	1405
Table 340.	DCMI register map and reset values	1415
Table 341.	PSSI input/output pins	1418
Table 342.	PSSI internal input/output signals	1418
Table 343.	Positioning of captured data bytes in 32-bit words (8-bit width)	1419
Table 344.	Positioning of captured data bytes in 32-bit words (16-bit width)	1420
Table 345.	PSSI interrupt requests	1423
Table 346.	PSSI register map and reset values	1429
Table 347.	Peripherals connected by CCB interconnect	1431
Table 348.	CCB internal input/output signals	1432
Table 349.	Blob creation steps for ECDSA signature (CCOP = 0b110000<i>i>0)	1439
Table 350.	Blob use steps for ECDSA signature (CCOP = 0b11000011)	1442

Table 351.	Blob use steps for signing public key computation (CCOP = 0b10000001)	1444
Table 352.	Blob creation steps for scalar multiplication (CCOP = 0b100000<i>i>0)	1446
Table 353.	Blob use steps for scalar multiplication (CCOP = 0b10000001)	1447
Table 354.	Blob creation steps for modular exponentiation (CCOP = 0b01000100)	1448
Table 355.	Blob use steps for modular exponentiation (CCOP = 0b01000101)	1449
Table 356.	CCB register map and reset values	1455
Table 357.	RNG internal input/output signals	1457
Table 358.	RNG interrupt requests	1465
Table 359.	RNG initialization times	1466
Table 360.	RNG configurations	1466
Table 361.	Configuration selection	1467
Table 362.	RNG register map and reset map	1472
Table 363.	RNG internal input/output signals	1474
Table 364.	RNG interrupt requests	1483
Table 365.	RNG configurations	1484
Table 366.	Additional health test configurations	1484
Table 367.	Configuration selection	1484
Table 368.	RNG register map and reset map	1492
Table 369.	AES versus SAES features	1494
Table 370.	SAES internal input/output signals	1495
Table 371.	SAES approved symmetric key functions	1496
Table 372.	Counter mode initialization vector definition	1506
Table 373.	Initialization of IV registers in GCM mode	1509
Table 374.	GCM last block definition	1509
Table 375.	Initialization of IV registers in CCM mode	1515
Table 376.	AES data swapping example	1524
Table 377.	Key endianness in SAES_KEYRx registers (128/256-bit keys)	1526
Table 378.	IVI bitfield spread over SAES_IVRx registers	1528
Table 379.	SAES interrupt requests	1530
Table 380.	Processing latency for ECB, CBC and CTR	1531
Table 381.	Processing latency for GCM and CCM (in SAES kernel clock cycles)	1531
Table 382.	SAES register map and reset values	1546
Table 383.	AES versus SAES features	1549
Table 384.	AES internal input/output signals	1550
Table 385.	AES approved symmetric key functions	1550
Table 386.	Counter mode initialization vector definition	1560
Table 387.	Initialization of IV registers in GCM mode	1562
Table 388.	GCM last block definition	1562
Table 389.	Initialization of IV registers in CCM mode	1569
Table 390.	AES data swapping example	1572
Table 391.	Key endianness in AES_KEYRx registers (128/256-bit keys)	1574
Table 392.	IVI bitfield spread over AES_IVRx registers	1575
Table 393.	AES interrupt requests	1576
Table 394.	Processing latency for ECB, CBC and CTR	1577
Table 395.	Processing latency for GCM and CCM (in clock cycles)	1578
Table 396.	AES register map and reset values	1590
Table 397.	HASH implementation on different devices	1594
Table 398.	HASH internal input/output signals	1595
Table 399.	Information on supported hash algorithms	1595
Table 400.	Hash processor outputs	1599
Table 401.	Processing time (in clock cycle)	1606
Table 402.	HASH interrupt requests	1607

Table 403.	HASH1 register map and reset values	1615
Table 404.	Internal input/output signals	1618
Table 405.	PKA integer arithmetic functions list	1619
Table 406.	PKA prime field (Fp) elliptic curve functions list	1620
Table 407.	Chaining mode to/from PKA	1625
Table 408.	Integrity protected parameters list	1627
Table 409.	Confidentiality protected parameter list	1627
Table 410.	Example of 'a' curve coefficient for ECC Fp scalar	1629
Table 411.	Montgomery parameter computation	1630
Table 412.	Modular addition	1630
Table 413.	Modular subtraction	1631
Table 414.	Montgomery multiplication	1632
Table 415.	Modular exponentiation (normal mode)	1633
Table 416.	Modular exponentiation (fast mode)	1633
Table 417.	Modular exponentiation (protected mode)	1634
Table 418.	Modular inversion	1634
Table 419.	Modular reduction	1635
Table 420.	Arithmetic addition	1635
Table 421.	Arithmetic subtraction	1635
Table 422.	Arithmetic multiplication	1636
Table 423.	Arithmetic comparison	1636
Table 424.	CRT exponentiation	1637
Table 425.	Point on elliptic curve Fp check	1638
Table 426.	ECC Fp scalar multiplication	1638
Table 427.	ECDSA sign - Inputs	1640
Table 428.	ECDSA sign - Outputs	1640
Table 429.	Extended ECDSA sign - additional outputs	1641
Table 430.	ECDSA verification - inputs	1641
Table 431.	ECDSA verification - outputs	1642
Table 432.	ECC complete addition	1642
Table 433.	ECC double base ladder	1643
Table 434.	ECC projective to affine	1644
Table 435.	Family of supported curves for ECC operations	1645
Table 436.	Modular exponentiation	1646
Table 437.	ECC scalar multiplication	1646
Table 438.	ECDSA signature average computation time	1647
Table 439.	ECDSA verification average computation times	1647
Table 440.	ECC double base ladder average computation times	1647
Table 441.	ECC projective to affine average computation times	1647
Table 442.	ECC complete addition average computation times	1647
Table 443.	Point on elliptic curve Fp check average computation times	1647
Table 444.	Montgomery parameters average computation times	1648
Table 445.	PKA interrupt requests	1648
Table 446.	PKA register map and reset values	1655
Table 447.	OTFDEC internal input/output signals	1657
Table 448.	OTFDEC interrupt requests	1661
Table 449.	OTFDEC register map and reset values	1675
Table 450.	TIM input/output pins	1681
Table 451.	TIM internal input/output signals	1681
Table 452.	Interconnect to the tim_ti1 input multiplexer	1683
Table 453.	Interconnect to the tim_ti2 input multiplexer	1683
Table 454.	Interconnect to the tim_ti3 input multiplexer	1683

Table 455.	Interconnect to the tim_ti4 input multiplexer	1683
Table 456.	Internal trigger connection	1683
Table 457.	Interconnect to the tim_etr input multiplexer	1684
Table 458.	Timer break interconnect	1685
Table 459.	Timer break2 interconnect	1685
Table 460.	System break interconnect	1685
Table 461.	Interconnect to the tim_ocref_clr input multiplexer	1685
Table 462.	CCR and ARR register change dithering pattern	1719
Table 463.	CCR register change dithering pattern in center-aligned PWM mode	1720
Table 464.	Behavior of timer outputs versus tim_brk/tim_brk2 inputs	1732
Table 465.	Break protection disarming conditions	1734
Table 466.	Counting direction versus encoder signals (CC1P = CC2P = 0)	1743
Table 467.	Counting direction versus encoder signals and polarity settings	1747
Table 468.	DMA request	1768
Table 469.	Effect of low-power modes on TIM1/TIM8	1769
Table 470.	Interrupt requests	1769
Table 471.	Output control bits for complementary tim_ocx and tim_ocxn channels with break feature	1796
Table 472.	TIMx register map and reset values	1819
Table 473.	STM32H5 general purpose timers	1823
Table 474.	TIM input/output pins	1825
Table 475.	TIM internal input/output signals	1825
Table 476.	Interconnect to the tim_ti1 input multiplexer	1826
Table 477.	Interconnect to the tim_ti2 input multiplexer	1826
Table 478.	Interconnect to the tim_ti3 input multiplexer	1827
Table 479.	Interconnect to the tim_ti4 input multiplexer	1827
Table 480.	TIMx internal trigger connection	1827
Table 481.	Interconnect to the tim_etr input multiplexer	1828
Table 482.	Interconnect to the tim_ocref_clr input multiplexer	1828
Table 483.	CCR and ARR register change dithering pattern	1859
Table 484.	CCR register change dithering pattern in center-aligned PWM mode	1860
Table 485.	Counting direction versus encoder signals(CC1P = CC2P = 0)	1869
Table 486.	Counting direction versus encoder signals and polarity settings	1874
Table 487.	DMA request	1898
Table 488.	Effect of low-power modes on TIM2/TIM3/TIM4/TIM5	1898
Table 489.	Interrupt requests	1899
Table 490.	Output control bit for standard tim_ocx channels	1920
Table 491.	TIM2/TIM3/TIM4/TIM5 register map and reset values	1936
Table 492.	TIM internal input/output signals	1940
Table 493.	TIMx_ARR register change dithering pattern	1950
Table 494.	DMA request	1951
Table 495.	Effect of low-power modes on TIM6/TIM7	1951
Table 496.	Interrupt request	1951
Table 497.	TIMx register map and reset values	1957
Table 498.	TIM input/output pins	1961
Table 499.	TIM internal input/output signals	1961
Table 500.	Interconnect to the tim_ti1 input multiplexer	1963
Table 501.	Interconnect to the tim_ti2 input multiplexer	1963
Table 502.	TIMx internal trigger connection	1963
Table 503.	CCR and ARR register change dithering pattern	1981
Table 504.	Effect of low-power modes on TIM12/TIM13/TIM14	1990
Table 505.	Interrupt requests	1990

Table 506.	Output control bit for standard tim_ocx channels	2002
Table 507.	TIM12 register map and reset values	2006
Table 508.	Output control bit for standard tim_ocx channels	2015
Table 509.	TIM13/TIM14 register map and reset values	2018
Table 510.	TIM input/output pins	2023
Table 511.	TIM internal input/output signals	2024
Table 512.	Interconnect to the tim_ti1 input multiplexer	2025
Table 513.	Interconnect to the tim_ti2 input multiplexer	2025
Table 514.	TIMx internal trigger connection	2025
Table 515.	Timer break interconnect	2026
Table 516.	System break interconnect	2026
Table 517.	Interconnect to the tim_ocref_clr input multiplexer	2027
Table 518.	CCR and ARR register change dithering pattern	2047
Table 519.	Break protection disarming conditions	2056
Table 520.	DMA request	2067
Table 521.	Effect of low-power modes on TIM15/TIM16/TIM17	2068
Table 522.	Interrupt requests	2068
Table 523.	Output control bits for complementary tim_ocx and tim_ocxn channels with break feature (TIM15)	2084
Table 524.	TIM15 register map and reset values	2097
Table 525.	Output control bits for complementary tim_oc1 and tim_oc1n channels with break feature (TIM16/TIM17)	2110
Table 526.	TIM16/TIM17 register map and reset values	2124
Table 527.	STM32H5 LPTIM features	2128
Table 528.	LPTIM1/2/3/5/6 input/output pins	2130
Table 529.	LPTIM4 input/output pins	2130
Table 530.	LPTIM1/2/3/5/6 internal signals	2131
Table 531.	LPTIM4 internal signals	2131
Table 532.	LPTIM1/2/3/4/5/6 external trigger connections	2132
Table 533.	LPTIM1/2/3/5/6 input 1 connections	2132
Table 534.	LPTIM1/2/3/5/6 input 2 connections	2132
Table 535.	LPTIM1/2/3/5/6 input capture 1 connections	2132
Table 536.	LPTIM1 input capture 2 connections	2133
Table 537.	LPTIM2 input capture 2 connections	2133
Table 538.	LPTIM3/5/6 input capture 2 connections	2133
Table 539.	Prescaler division ratios	2135
Table 540.	Encoder counting scenarios	2142
Table 541.	Input capture Glitch filter latency (in counter step unit)	2146
Table 542.	Effect of low-power modes on the LPTIM	2150
Table 543.	Interrupt events	2151
Table 544.	LPTIM register map and reset values	2175
Table 545.	IWDG features	2178
Table 546.	IWDG delays versus actions	2179
Table 547.	IWDG internal input/output signals	2180
Table 548.	Effect of low power modes on IWDG	2185
Table 549.	IWDG interrupt request	2187
Table 550.	IWDG register map and reset values	2193
Table 551.	WWDG features	2194
Table 552.	WWDG internal input/output signals	2195
Table 553.	WWDG interrupt requests	2198
Table 554.	WWDG register map and reset values	2200
Table 555.	RTC input/output pins	2204

Table 556.	RTC internal input/output signals	2204
Table 557.	RTC interconnection	2205
Table 558.	RTC pin PC13 configuration	2206
Table 559.	PI8 configuration	2208
Table 560.	RTC_OUT mapping	2209
Table 561.	Effect of low-power modes on RTC	2224
Table 562.	RTC pins functionality over modes	2224
Table 563.	Nonsecure interrupt requests	2225
Table 564.	Secure interrupt requests	2225
Table 565.	RTC register map and reset values	2256
Table 566.	TAMP implementation	2260
Table 567.	TAMP input/output pins	2262
Table 568.	TAMP internal input/output signals	2262
Table 569.	TAMP interconnection	2263
Table 570.	Device resource x tamper protection	2269
Table 571.	Active tamper output change period	2272
Table 572.	Minimum ATPER value	2273
Table 573.	Active tamper filtered pulse duration	2274
Table 574.	Effect of low-power modes on TAMP	2276
Table 575.	TAMP pins functionality over modes	2276
Table 576.	Interrupt requests	2277
Table 577.	TAMP register map and reset values	2306
Table 578.	I2C implementation	2309
Table 579.	I2C input/output pins	2310
Table 580.	I2C internal input/output signals	2311
Table 581.	Comparison of analog and digital filters	2313
Table 582.	I ² C-bus and SMBus specification data setup and hold times	2315
Table 583.	I2C configuration	2319
Table 584.	I ² C-bus and SMBus specification clock timings	2330
Table 585.	Timing settings for f_{I2CCLK} of 8 MHz	2340
Table 586.	Timing settings for f_{I2CCLK} of 16 MHz	2340
Table 587.	SMBus timeout specifications	2342
Table 588.	SMBus with PEC configuration	2344
Table 589.	TIMEOUTA[11:0] for maximum $t_{TIMEOUT}$ of 25 ms	2345
Table 590.	TIMEOUTB[11:0] for maximum $t_{LOW:SEXT}$ and $t_{LOW:MEXT}$ of 8 ms	2345
Table 591.	TIMEOUTA[11:0] for maximum t_{IDLE} of 50 μ s	2345
Table 592.	Effect of low-power modes to I2C	2355
Table 593.	I2C interrupt requests	2355
Table 594.	I2C register map and reset values	2371
Table 595.	I3C instantiation	2374
Table 596.	I3C FIFOs implementation	2374
Table 597.	I3C interrupt(s)	2374
Table 598.	I3C peripheral controller/target features versus MIPI v1.1	2375
Table 599.	I3C input/output pins	2376
Table 600.	I3C internal input/output signals	2376
Table 601.	I3C register usage	2392
Table 602.	I3C registers/fields usage versus controller state	2393
Table 603.	I3C registers/fields usage versus target state	2395
Table 604.	List of supported I3C CCCs, as controller/target	2398
Table 605.	I3C controller error management	2433
Table 606.	I3C target error management	2435
Table 607.	Effect of low-power modes	2436

Table 608.	I3C register map and reset values	2485
Table 609.	I3C instantiation	2490
Table 610.	I3C autonomous mode with DMA in Stop mode(s).	2490
Table 611.	I3C FIFOs implementation	2490
Table 612.	I3C interrupt(s)	2491
Table 613.	I3C peripheral controller/target features versus MIPI v1.1	2491
Table 614.	I3C input/output pins	2492
Table 615.	I3C internal input/output signals	2492
Table 616.	I3C register usage	2505
Table 617.	I3C registers/fields usage versus controller state	2506
Table 618.	I3C registers/fields usage versus target state	2509
Table 619.	List of supported I3C CCCs, as controller/target	2512
Table 620.	I3C controller error management	2543
Table 621.	I3C target error management	2545
Table 622.	Effect of low-power modes	2546
Table 623.	I3C register map and reset values	2593
Table 624.	Instance implementation	2597
Table 625.	USART/LPUART features	2598
Table 626.	USART/UART input/output pins	2600
Table 627.	USART internal input/output signals	2601
Table 628.	Noise detection from sampled data	2613
Table 629.	Tolerance of the USART receiver when BRR [3:0] = 0000.	2617
Table 630.	Tolerance of the USART receiver when BRR[3:0] is different from 0000.	2617
Table 631.	USART frame formats	2622
Table 632.	Effect of low-power modes on the USART	2645
Table 633.	USART interrupt requests.	2646
Table 634.	USART register map and reset values	2684
Table 635.	Instance implementation on STM32H5	2687
Table 636.	USART/LPUART features	2687
Table 637.	LPUART input/output pins	2690
Table 638.	LPUART internal input/output signals	2690
Table 639.	Error calculation for programmed baud rates at lpuart_ker_ck_pres= 32.768 kHz	2701
Table 640.	Tolerance of the LPUART receiver	2702
Table 642.	Effect of low-power modes on the LPUART	2714
Table 643.	LPUART interrupt requests.	2715
Table 644.	LPUART register map and reset values	2741
Table 645.	SPI features	2744
Table 646.	SPI/I2S input/output pins	2747
Table 647.	SPI internal input/output signals	2747
Table 648.	Effect of low-power modes on the SPI	2774
Table 649.	SPI wake-up and interrupt requests	2775
Table 650.	Bitfields usable in PCM/I2S mode	2777
Table 651.	WS and CK level before SPI/I2S is enabled when AFCNTR = 1	2786
Table 652.	Serial data line swapping	2786
Table 653.	CLKGEN programming examples for usual I2S frequencies	2790
Table 654.	I2S interrupt requests	2799
Table 655.	SPI register map and reset values	2817
Table 656.	STM32H5 SAI features	2820
Table 657.	SAI internal input/output signals	2822
Table 658.	SAI input/output pins	2822
Table 659.	External synchronization selection	2825
Table 660.	MCLK_x activation conditions.	2830

Table 661.	Clock generator programming examples	2833
Table 662.	SAI_A configuration for TDM mode	2840
Table 663.	TDM frame configuration examples	2842
Table 664.	SOPD pattern	2846
Table 665.	Parity bit calculation	2846
Table 666.	Audio sampling frequency versus symbol rates	2847
Table 667.	SAI interrupt sources	2856
Table 668.	SAI register map and reset values	2885
Table 669.	CAN subsystem I/O signals	2891
Table 670.	CAN subsystem I/O pins	2891
Table 671.	DLC coding in FDCAN	2895
Table 672.	Possible configurations for frame transmission	2909
Table 673.	Rx FIFO element	2912
Table 674.	Rx FIFO element description	2912
Table 675.	Tx buffer and FIFO element	2914
Table 676.	Tx buffer element description	2914
Table 677.	Tx event FIFO element	2916
Table 678.	Tx event FIFO element description	2916
Table 679.	Standard message ID filter element	2917
Table 680.	Standard message ID filter element field description	2918
Table 681.	Extended message ID filter element	2918
Table 682.	Extended message ID filter element field description	2919
Table 683.	FDCAN register map and reset values	2949
Table 684.	STM32H5 USB implementation	2953
Table 685.	USB input/output pins	2954
Table 686.	Double-buffering buffer flag definition	2966
Table 687.	Bulk double-buffering memory buffers usage (Device mode)	2966
Table 688.	Bulk double-buffering memory buffers usage (Host mode)	2968
Table 689.	Isochronous memory buffers usage	2969
Table 690.	Isochronous memory buffers usage	2970
Table 691.	Resume event detection	2972
Table 692.	Resume event detection for host	2973
Table 693.	Reception status encoding	2991
Table 694.	Endpoint/channel type encoding	2991
Table 695.	Endpoint/channel kind meaning	2991
Table 696.	Transmission status encoding	2991
Table 697.	USB register map and reset values	2992
Table 698.	Definition of allocated buffer memory	2995
Table 699.	USBSRAM register map and reset values	2998
Table 700.	UCPD implementation	3000
Table 701.	UCPD software trim data	3000
Table 702.	UCPD signals on pins	3002
Table 703.	UCPD internal signals	3002
Table 704.	4b5b symbol encoding table	3004
Table 705.	Ordered sets	3005
Table 706.	Validation of ordered sets	3006
Table 707.	Data size	3006
Table 708.	Coding for ANAMODE, ANASUBMODE and link with TYPEC_VSTATE_CCx	3015
Table 709.	Type-C sequence (source: 3A); cable/sink connected (Rd on CC1; Ra on CC2)	3017
Table 710.	Effect of low power modes on the UCPD	3019
Table 711.	UCPD interrupt requests	3020
Table 712.	UCPD register map and reset values	3036

Table 713.	Ethernet peripheral pins	3043
Table 714.	Internal Ethernet input/output signals	3044
Table 715.	Ethernet trigger connections	3044
Table 716.	Priority scheme for Tx DMA and Rx DMA	3053
Table 717.	Double VLAN processing features in Tx path	3059
Table 718.	Double VLAN processing in Rx path	3060
Table 719.	VLAN insertion or replacement based on VLTi bit	3061
Table 720.	Destination address filtering	3064
Table 721.	Source address filtering	3065
Table 722.	VLAN match status	3066
Table 723.	Ordinary clock: PTP messages for snapshot	3070
Table 724.	End-to-end transparent clock: PTP messages for snapshot	3070
Table 725.	Peer-to-peer transparent clock: PTP messages for snapshot	3071
Table 726.	Egress and ingress latency for PHY interfaces	3075
Table 727.	Minimum PTP clock frequency example	3075
Table 728.	Message format defined in IEEE 1588-2008	3076
Table 729.	Message format defined in IEEE 1588-2008	3076
Table 730.	IPv6-UDP PTP packet fields required for control and status	3077
Table 731.	Ethernet PTP packet fields required for control and status	3078
Table 732.	Timestamp Snapshot Dependency on ETH_MACTSCR bits	3080
Table 733.	PTP message generation criteria	3086
Table 734.	Common PTP message header fields	3088
Table 735.	MAC transmit PTP mode and one-step timestamping operation	3091
Table 736.	Transmit checksum offload engine functions for different packet types	3095
Table 737.	Receive checksum offload engine functions for different packet types	3098
Table 738.	TSO: TCP and IP header fields	3102
Table 739.	Pause packet fields	3107
Table 740.	Tx MAC flow control	3108
Table 741.	Rx MAC flow control	3108
Table 742.	Size of the maximum receive packet	3111
Table 743.	MCD clock selection	3113
Table 744.	MDIO Clause 45 frame structure	3114
Table 745.	MDIO Clause 22 frame structure	3115
Table 746.	Remote wake-up packet filter register	3126
Table 747.	Description of the remote wake-up filter fields	3126
Table 748.	Remote wake-up packet and PMT interrupt generation	3128
Table 749.	Transfer complete interrupt behavior	3136
Table 750.	TDES0 normal descriptor (read format)	3158
Table 751.	TDES1 normal descriptor (read format)	3159
Table 752.	TDES2 normal descriptor (read format)	3159
Table 753.	TDES3 normal descriptor (read format)	3160
Table 754.	TDES0 normal descriptor (write-back format)	3163
Table 755.	TDES1 normal descriptor (write-back format)	3163
Table 756.	TDES2 normal descriptor (write-back format)	3164
Table 757.	TDES3 normal descriptor (write-back format)	3164
Table 758.	TDES0 context descriptor	3167
Table 759.	TDES1 context descriptor	3167
Table 760.	TDES2 context descriptor	3168
Table 761.	TDES3 context descriptor	3168
Table 762.	RDES0 normal descriptor (read format)	3171
Table 763.	RDES1 normal descriptor (read format)	3171
Table 764.	RDES2 normal descriptor (read format)	3171

Table 765.	RDES3 normal descriptor (read format)	3171
Table 766.	RDES0 normal descriptor (write-back format)	3173
Table 767.	RDES1 normal descriptor (write-back format)	3173
Table 768.	RDES2 normal descriptor (write-back format)	3175
Table 769.	RDES3 normal descriptor (write-back format)	3177
Table 770.	RDES0 context descriptor	3180
Table 771.	RDES1 context descriptor	3181
Table 772.	RDES2 context descriptor	3181
Table 773.	RDES3 context descriptor	3181
Table 774.	ETH_DMA common register map and reset values	3201
Table 775.	ETH_DMA_CH register map and reset values	3202
Table 776.	ETH_MTL register map and reset values	3212
Table 777.	Giant Packet Status based on S2KP and JE Bits	3218
Table 778.	Packet Length based on the CST and ACS bits	3218
Table 779.	Ethernet MAC register map and reset values	3297
Table 780.	Ethernet peripheral pins	3311
Table 781.	Internal Ethernet input/output signals	3312
Table 782.	Ethernet trigger connections	3312
Table 783.	Priority scheme for Tx DMA and Rx DMA	3321
Table 784.	Double VLAN processing features in Tx path	3327
Table 785.	Double VLAN processing in Rx path	3328
Table 786.	VLAN insertion or replacement based on VLTi bit	3329
Table 787.	Destination address filtering	3332
Table 788.	Source address filtering	3333
Table 789.	VLAN match status	3334
Table 790.	Ordinary clock: PTP messages for snapshot	3338
Table 791.	End-to-end transparent clock: PTP messages for snapshot	3338
Table 792.	Peer-to-peer transparent clock: PTP messages for snapshot	3339
Table 793.	Egress and ingress latency for PHY interfaces	3343
Table 794.	Minimum PTP clock frequency example	3343
Table 795.	Message format defined in IEEE 1588-2008	3344
Table 796.	Message format defined in IEEE 1588-2008	3345
Table 797.	IPv6-UDP PTP packet fields required for control and status	3346
Table 798.	Ethernet PTP packet fields required for control and status	3347
Table 799.	Timestamp Snapshot Dependency on ETH_MACTSCR bits	3348
Table 800.	PTP message generation criteria	3354
Table 801.	Common PTP message header fields	3356
Table 802.	MAC transmit PTP mode and one-step timestamping operation	3359
Table 803.	Transmit checksum offload engine functions for different packet types	3363
Table 804.	Receive checksum offload engine functions for different packet types	3366
Table 805.	TSO: TCP and IP header fields	3370
Table 806.	Pause packet fields	3375
Table 807.	Tx MAC flow control	3376
Table 808.	Rx MAC flow control	3376
Table 809.	Size of the maximum receive packet	3379
Table 810.	MCD clock selection	3381
Table 811.	MDIO Clause 45 frame structure	3382
Table 812.	MDIO Clause 22 frame structure	3383
Table 813.	Remote wake-up packet filter register	3394
Table 814.	Description of the remote wake-up filter fields	3394
Table 815.	Remote wake-up packet and PMT interrupt generation	3396
Table 816.	Transfer complete interrupt behavior	3405

Table 817.	TDES0 normal descriptor (read format)	3429
Table 818.	TDES1 normal descriptor (read format)	3430
Table 819.	TDES2 normal descriptor (read format)	3430
Table 820.	TDES3 normal descriptor (read format)	3431
Table 821.	TDES0 normal descriptor (write-back format)	3434
Table 822.	TDES1 normal descriptor (write-back format)	3434
Table 823.	TDES2 normal descriptor (write-back format)	3435
Table 824.	TDES3 normal descriptor (write-back format)	3435
Table 825.	TDES0 context descriptor	3439
Table 826.	TDES1 context descriptor	3439
Table 827.	TDES2 context descriptor	3439
Table 828.	TDES3 context descriptor	3440
Table 829.	RDES0 normal descriptor (read format)	3443
Table 830.	RDES1 normal descriptor (read format)	3443
Table 831.	RDES2 normal descriptor (read format)	3443
Table 832.	RDES3 normal descriptor (read format)	3444
Table 833.	RDES0 normal descriptor (write-back format)	3445
Table 834.	RDES1 normal descriptor (write-back format)	3446
Table 835.	RDES2 normal descriptor (write-back format)	3448
Table 836.	RDES3 normal descriptor (write-back format)	3449
Table 837.	RDES0 context descriptor	3452
Table 838.	RDES1 context descriptor	3453
Table 839.	RDES2 context descriptor	3453
Table 840.	RDES3 context descriptor	3453
Table 841.	ETH_DMA common register map and reset values	3474
Table 842.	ETH_DMA_CH register map and reset values	3474
Table 843.	ETH_MTL register map and reset values	3484
Table 844.	Giant Packet Status based on S2KP and JE Bits	3490
Table 845.	Packet Length based on the CST and ACS bits	3490
Table 846.	Ethernet MAC register map and reset values	3577
Table 847.	HDMI pin	3588
Table 848.	HDMI-CEC internal input/output signals	3588
Table 849.	Error handling timing parameters	3594
Table 850.	TXERR timing parameters	3595
Table 851.	HDMI-CEC interrupts	3596
Table 852.	HDMI-CEC register map and reset values	3604
Table 853.	JTAG/Serial-wire debug port pins	3606
Table 854.	Trace port pins	3606
Table 855.	Single-wire trace port pins	3607
Table 856.	Authentication signal states with TrustZone enabled (TZEN = 0xB4)	3609
Table 857.	Authentication signal states with TrustZone disabled (TZEN = 0xC3)	3609
Table 858.	Life cycle state and debug states	3609
Table 859.	Definition of data to provision	3612
Table 860.	Permission mask (Endianness: Little Endian)	3612
Table 861.	JTAG-DP data registers	3615
Table 862.	Packet request	3617
Table 863.	ACK response	3617
Table 864.	Data transfer	3617
Table 865.	Debug port registers	3618
Table 866.	Debug port register map and reset values	3624
Table 867.	MEM-AP registers	3626
Table 868.	Access port register map and reset values	3632

Table 869.	System ROM table	3633
Table 870.	MCU ROM table	3634
Table 871.	Processor ROM table	3634
Table 872.	System ROM table register map and reset values	3640
Table 873.	MCU ROM table register map and reset values	3645
Table 874.	CPU ROM table register map and reset values	3651
Table 875.	DWT register map and reset values	3666
Table 876.	ITM register map and reset values	3677
Table 877.	BPU register map and reset values	3685
Table 878.	ETM register map and reset values	3710
Table 879.	TPIU register map and reset values	3724
Table 880.	CTI inputs	3726
Table 881.	CTI outputs	3726
Table 882.	CTI register map and reset values	3737
Table 883.	Peripheral clock freeze control bits	3740
Table 884.	Peripheral behaviour in debug mode	3741
Table 885.	DBGMCU register map and reset values	3757
Table 886.	Document revision history	3764

List of figures

Figure 1.	System architecture	122
Figure 2.	Memory map based on IDAU mapping (STM32H562/563/573xx devices)	131
Figure 3.	Memory map based on IDAU mapping (STM32H523/533xx devices)	132
Figure 4.	Memory map based on IDAU mapping (STM32H543/553xx devices)	133
Figure 5.	Secure/nonsecure partitioning using TrustZone technology	153
Figure 6.	Sharing memory map between CPU in secure and nonsecure state	155
Figure 7.	Secure world transition and memory partitioning	156
Figure 8.	Global TrustZone framework and TrustZone awareness	157
Figure 9.	Flash memory TrustZone protections	161
Figure 10.	Flash memory secure HDP area	168
Figure 11.	Key management principle	176
Figure 12.	Device life cycle security	180
Figure 13.	PRODUCT_STATES (simplified TrustZone activated view)	182
Figure 14.	PRODUCT_STATES (full TrustZone activated view)	183
Figure 15.	Collaborative development principle	185
Figure 16.	External flash memory protection using SFI	187
Figure 17.	GTZC in Armv8-M subsystem block diagram	194
Figure 18.	GTZC block diagram	197
Figure 19.	Watermark memory protection controller (region x/subregions A and B)	198
Figure 20.	MPCBB block diagram	199
Figure 21.	Memory map: SRAM1, SRAM2/3 with ECC (STM32H562/63/73xx devices)	257
Figure 22.	Memory map: SRAM1/3, SRAM2 with ECC (STM32H523/33xx devices)	257
Figure 23.	Memory map: SRAM1, SRAM2/3 with ECC (STM32H543/53xx devices)	258
Figure 24.	FLASH block diagram (simplified)	270
Figure 25.	Embedded flash memory organization (2-Mbyte devices)	272
Figure 26.	Embedded flash memory usage	273
Figure 27.	Flash high-cycle data memory map on 2-Mbyte devices	286
Figure 28.	Flash high-cycle data memory map on 1-Mbyte devices	286
Figure 29.	Flash high-cycle data memory map on 512-Kbyte devices	287
Figure 30.	Flash high-cycle data memory map on 256-Kbyte devices	287
Figure 31.	Flash bank swapping sequence	290
Figure 32.	OBK protection checks	302
Figure 33.	Key writing flow	303
Figure 34.	Swap workflow	304
Figure 35.	HDP in user flash memory (STM32H562/563/573xx devices)	309
Figure 36.	Protection attributes in case of bank swap illustration	314
Figure 37.	ICACHE block diagram	392
Figure 38.	ICACHE TAG and data memories functional view	394
Figure 39.	ICACHE remapping address mechanism	397
Figure 40.	DCACHE block diagram	409
Figure 41.	DCACHE TAG and data memories functional view	412
Figure 42.	Power supply with SMPS (STM32H563/573xx devices only)	428
Figure 43.	Power supply with LDO	429
Figure 44.	Power supply with LDO (STM32H543/553xx devices only)	430
Figure 45.	System supply configurations	432

Figure 46.	Power-on (POR) / power-down (PDR) reset waveform	438
Figure 47.	BOR thresholds	439
Figure 48.	PVD thresholds	440
Figure 49.	AVD thresholds	441
Figure 50.	VBAT thresholds	442
Figure 51.	Temperature thresholds	443
Figure 52.	Dynamic voltage scaling in Run mode	444
Figure 53.	I/O states in Standby mode	454
Figure 54.	Simplified diagram of the reset circuit	481
Figure 55.	Clock tree (STM32H523/533/56x/573xx devices)	484
Figure 56.	Clock tree (STM32H543/553xx devices)	485
Figure 57.	HSE/LSE clock sources	486
Figure 58.	CSI calibration flow	489
Figure 59.	PLL block diagram	490
Figure 60.	PLLs initialization flow	493
Figure 61.	Clock distribution for CCB subsystem	501
Figure 62.	CRS block diagram	604
Figure 63.	CRS counter behavior	606
Figure 64.	Structure of 3- or 5-V tolerant GPIO (TT or FT)	616
Figure 65.	Input floating/pull-up/pull-down configurations	620
Figure 66.	Output configuration	621
Figure 67.	Alternate function configuration	622
Figure 68.	High-impedance analog configuration	622
Figure 69.	SBS block diagram	637
Figure 70.	Compensation cell management	639
Figure 71.	Compensation cell usage	640
Figure 72.	SBS boot control	641
Figure 73.	SBS debug control	644
Figure 74.	SBS hardware secure storage control	646
Figure 75.	GPDMA block diagram	687
Figure 76.	GPDMA channel direct programming without linked-list (GPDMA_CxLLR = 0)	689
Figure 77.	GPDMA channel suspend and resume sequence	690
Figure 78.	GPDMA channel abort and restart sequence	691
Figure 79.	Static linked-list data structure (all Uxx = 1) of a linear addressing channel x	692
Figure 80.	Static linked-list data structure (all Uxx = 1) of a 2D addressing channel x	693
Figure 81.	GPDMA dynamic linked-list data structure of a linear addressing channel x	694
Figure 82.	GPDMA dynamic linked-list data structure of a 2D addressing channel x	694
Figure 83.	GPDMA channel execution and linked-list programming in run-to-completion mode (GPDMA_CxCR.LSM = 0)	696
Figure 84.	Inserting a LLIn with an auxiliary GPDMA channel y	698
Figure 85.	GPDMA channel execution and linked-list programming in link step mode (GPDMA_CxCR.LSM = 1)	700
Figure 86.	Building LLIn+1: GPDMA dynamic linked-lists in link step mode	701
Figure 87.	Replace with a new LLIn' in register file in link step mode	702
Figure 88.	Replace with a new LLIn' and LLIn+1' in memory in link step mode (option 1)	703
Figure 89.	Replace with a new LLIn' and LLIn+1' in memory in link step mode (option 2)	704
Figure 90.	GPDMA channel execution and linked-list programming	706
Figure 91.	Programmed 2D addressing	709

Figure 92.	GPDMA arbitration policy	716
Figure 93.	Trigger hit, memorization, and overrun waveform	719
Figure 94.	GPDMA circular buffer programming: update of the memory start address with a linear addressing channel	720
Figure 95.	Shared GPDMA channel with circular buffering: update of the memory start address with a linear addressing channel.	721
Figure 96.	EXTI block diagram	776
Figure 97.	Configurable event trigger logic CPU wake-up.	780
Figure 98.	EXTI direct events	781
Figure 99.	EXTI mux GPIO selection.	781
Figure 100.	CRC calculation unit block diagram	826
Figure 101.	CORDIC convergence for trigonometric functions	840
Figure 102.	CORDIC convergence for hyperbolic functions	841
Figure 103.	CORDIC convergence for square root	842
Figure 104.	Block diagram	851
Figure 105.	Input buffer areas	853
Figure 106.	Circular input buffer	854
Figure 107.	Circular input buffer operation	855
Figure 108.	Circular output buffer	856
Figure 109.	Circular output buffer operation	857
Figure 110.	FIR filter structure	859
Figure 111.	IIR filter structure (direct form 1)	861
Figure 112.	X1 buffer initialization	866
Figure 113.	Filtering example 1	867
Figure 114.	Filtering example 2	868
Figure 115.	PLAY block diagram	879
Figure 116.	Input multiplexing	883
Figure 117.	Input filter and edge detection	883
Figure 118.	Filter operation example	884
Figure 119.	Edge detector operation (rising and falling edge) with filter enabled	884
Figure 120.	Edge detector operation (rising and falling edge) with filter bypassed	885
Figure 121.	swin_we pulse timing	886
Figure 122.	Logic element	886
Figure 123.	Example logic expression 1	887
Figure 124.	Example logic expression 2	888
Figure 125.	Input interconnect	889
Figure 126.	Logic element cascading	892
Figure 127.	Combining logic elements in parallel	892
Figure 128.	Splitting logic expressions between logic elements	893
Figure 129.	Output interconnect	894
Figure 130.	FMC block diagram.	916
Figure 131.	FMC memory banks	918
Figure 132.	Mode 1 read access waveforms	928
Figure 133.	Mode 1 write access waveforms.	928
Figure 134.	Mode A read access waveforms.	930
Figure 135.	Mode A write access waveforms	930
Figure 136.	Mode 2 and mode B read access waveforms.	932
Figure 137.	Mode 2 write access waveforms.	933
Figure 138.	Mode B write access waveforms	933
Figure 139.	Mode C read access waveforms	935
Figure 140.	Mode C write access waveforms	936
Figure 141.	Mode D read access waveforms	938

Figure 142. Mode D write access waveforms	939
Figure 143. Muxed read access waveforms	941
Figure 144. Muxed write access waveforms	942
Figure 145. Asynchronous wait during a read access waveforms	944
Figure 146. Asynchronous wait during a write access waveforms	945
Figure 147. Wait configuration waveforms	947
Figure 148. Synchronous multiplexed read mode waveforms - NOR, PSRAM (CRAM)	948
Figure 149. Synchronous multiplexed write mode waveforms - PSRAM (CRAM)	950
Figure 150. NAND flash controller waveforms for common memory access	962
Figure 151. Access to non 'CE don't care' NAND-flash	964
Figure 152. Burst write SDRAM access waveforms	973
Figure 153. Burst read SDRAM access	974
Figure 154. Logic diagram of Read access with RBURST bit set (CAS=1, RPIPE=0)	975
Figure 155. Read access crossing row boundary	977
Figure 156. Write access crossing row boundary	977
Figure 157. Self-refresh mode	979
Figure 158. Power-down mode	980
Figure 159. OCTOSPI block diagram in octal configuration	992
Figure 160. OCTOSPI block diagram in quad configuration	992
Figure 161. OCTOSPI block diagram in dual-quad configuration	993
Figure 162. SDR read command in octal configuration	994
Figure 163. DTR read in octal-SPI mode with DQS (Macronix mode) example	997
Figure 164. SDR write command in octo-SPI mode example	999
Figure 165. DTR write in octal-SPI mode (Macronix mode) example	1000
Figure 166. Example of HyperBus read operation	1002
Figure 167. HyperBus write operation with initial latency	1003
Figure 168. HyperBus read operation with additional latency	1003
Figure 169. HyperBus write operation with additional latency	1004
Figure 170. HyperBus write operation with no latency (register write)	1004
Figure 171. HyperBus read operation page crossing with latency	1005
Figure 172. D0/D1 data ordering in octal-SPI DTR mode (Micron) - Read access	1012
Figure 173. D0/D1 data ordering in octal-SPI DTR mode with DQS and CLK polarity aligned during Read access	1012
Figure 174. OctaRAM read operation with reverse data ordering D1/D0	1013
Figure 175. NCS when CKMODE = 0 (T = CLK period)	1018
Figure 176. NCS when CKMODE = 1 in SDR mode (T = CLK period)	1019
Figure 177. NCS when CKMODE = 1 in DTR mode (T = CLK period)	1019
Figure 178. NCS when CKMODE = 1 with an abort (T = CLK period)	1019
Figure 179. OCTOSPI block diagram in octal configuration (for STM32H543/553xx devices)	1049
Figure 180. OCTOSPI block diagram in quad configuration (for STM32H543/553xx devices)	1049
Figure 181. OCTOSPI block diagram in dual-quad configuration (for STM32H543/553xx devices)	1050
Figure 182. SDR read command in octal configuration	1051
Figure 183. DTR read in octal-SPI mode with DQS (Macronix mode) example	1054
Figure 184. SDR write command in octo-SPI mode example	1056
Figure 185. DTR write in octal-SPI mode (Macronix mode) example	1057
Figure 186. Example of HyperBus read operation	1059
Figure 187. HyperBus write operation with initial latency	1060
Figure 188. HyperBus read operation with additional latency	1060
Figure 189. HyperBus write operation with additional latency	1061

Figure 190.	HyperBus write operation with no latency (register write).	1061
Figure 191.	HyperBus read operation page crossing with latency.	1062
Figure 192.	D0/D1 data ordering in octal-SPI DTR mode (Micron) - Read access	1069
Figure 193.	D0/D1 data ordering in octal-SPI DTR mode with DQS and CLK polarity aligned during Read access	1069
Figure 194.	OctaRAM read operation with reverse data ordering D1/D0	1070
Figure 195.	NCS when CKMODE = 0 (T = CLK period)	1075
Figure 196.	NCS when CKMODE = 1 in SDR mode (T = CLK period)	1076
Figure 197.	NCS when CKMODE = 1 in DTR mode (T = CLK period)	1076
Figure 198.	NCS when CKMODE = 1 with an abort (T = CLK period).	1076
Figure 199.	Example of software control of two external memories	1077
Figure 200.	Example of hardware-controller extended memory support.	1078
Figure 201.	SDMMC "no response" and "no data" operations.	1108
Figure 202.	SDMMC (multiple) block read operation.	1108
Figure 203.	SDMMC (multiple) block write operation.	1109
Figure 204.	SDMMC (sequential) stream read operation	1109
Figure 205.	SDMMC (sequential) stream write operation	1109
Figure 206.	SDMMC block diagram.	1111
Figure 207.	SDMMC Command and data phase relation	1113
Figure 208.	Control unit	1115
Figure 209.	Command/response path	1116
Figure 210.	Command path state machine (CPSM)	1117
Figure 211.	Data path	1123
Figure 212.	DDR mode data packet clocking	1124
Figure 213.	DDR mode CRC status / boot acknowledgment clocking.	1124
Figure 214.	Data path state machine (DPSM).	1125
Figure 215.	CLKMUX unit	1136
Figure 216.	Linked list structures.	1138
Figure 217.	Asynchronous interrupt generation.	1141
Figure 218.	Synchronous interrupt period data read	1142
Figure 219.	Synchronous interrupt period data write	1142
Figure 220.	Asynchronous interrupt period data read	1143
Figure 221.	Asynchronous interrupt period data write	1144
Figure 222.	Clock stop with SDMMC_CLK for DS, HS, SDR12, SDR25.	1147
Figure 223.	Clock stop with SDMMC_CLK for DDR50, SDR50, SDR104.	1147
Figure 224.	Read Wait with SDMMC_CLK < 50 MHz	1148
Figure 225.	Read Wait with SDMMC_CLK > 50 MHz	1148
Figure 226.	CMD12 stream timing	1151
Figure 227.	CMD5 Sleep Awake procedure	1153
Figure 228.	Normal boot mode operation	1155
Figure 229.	Alternative boot mode operation.	1156
Figure 230.	Command response R1b busy signaling	1157
Figure 231.	SDMMC state control	1158
Figure 232.	Card cycle power / power up diagram	1159
Figure 233.	CMD11 signal voltage switch sequence.	1160
Figure 234.	Voltage switch transceiver typical application.	1162
Figure 235.	DLYB block diagram.	1190
Figure 236.	ADC block diagram	1198
Figure 237.	ADC clock scheme	1203
Figure 238.	ADC1 connectivity	1204
Figure 239.	ADC2 connectivity	1205
Figure 240.	ADC3 connectivity	1206

Figure 241. ADC calibration	1209
Figure 242. Updating the ADC calibration factor	1210
Figure 243. Mixing single-ended and differential channels	1211
Figure 244. Enabling / disabling the ADC	1212
Figure 245. Bulb mode timing diagram	1215
Figure 246. Analog-to-digital conversion time	1218
Figure 247. Stopping ongoing regular conversions	1219
Figure 248. Stopping ongoing regular and injected conversions	1220
Figure 249. Triggers shared between ADC master and slave	1221
Figure 250. Injected conversion latency	1223
Figure 251. Example of ADC_JSQR queue of context (sequence change)	1226
Figure 252. Example of ADC_JSQR queue of context (trigger change)	1226
Figure 253. Example of ADC_JSQR queue of context with overflow before conversion	1227
Figure 254. Example of ADC_JSQR queue of context with overflow during conversion	1227
Figure 255. Example of ADC_JSQR queue of context with empty queue (case JQM = 0)	1228
Figure 256. Example of ADC_JSQR queue of context with empty queue (JQM = 1)	1229
Figure 257. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs during an ongoing conversion	1229
Figure 258. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs during an ongoing conversion and a new trigger occurs	1230
Figure 259. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 0) - JADSTP occurs outside an ongoing conversion	1230
Figure 260. Flushing ADC_JSQR queue of context by setting JADSTP = 1 (JQM = 1)	1231
Figure 261. Flushing ADC_JSQR queue of context by setting ADDIS = 1 (JQM = 0)	1231
Figure 262. Flushing ADC_JSQR queue of context by setting ADDIS = 1 (JQM = 1)	1232
Figure 263. Single conversions of a sequence, software trigger	1234
Figure 264. Continuous conversion of a sequence, software trigger	1234
Figure 265. Single conversions of a sequence, hardware trigger	1235
Figure 266. Continuous conversions of a sequence, hardware trigger	1235
Figure 267. Right alignment (offset disabled, unsigned value)	1237
Figure 268. Right alignment (offset enabled, signed value)	1238
Figure 269. Left alignment (offset disabled, unsigned value)	1238
Figure 270. Left alignment (offset enabled, signed value)	1239
Figure 271. Example of overrun (OVRMOD = 0)	1240
Figure 272. Example of overrun (OVRMOD = 1)	1241
Figure 273. AUTODLY = 1, regular conversion in continuous mode, software trigger	1244
Figure 274. AUTODLY = 1, regular HW conversions interrupted by injected conversions (DISCEN = 0; JDISCEN = 0)	1244
Figure 275. AUTODLY = 1, regular HW conversions interrupted by injected conversions (DISCEN = 1, JDISCEN = 1)	1245
Figure 276. AUTODLY = 1, regular continuous conversions interrupted by injected conversions	1246
Figure 277. AUTODLY = 1 in auto- injected mode (JAUTO = 1)	1246
Figure 278. Analog watchdog guarded area	1247
Figure 279. ADCy_AWDx_OUT signal generation (on all regular channels)	1249
Figure 280. ADCy_AWDx_OUT signal generation (AWDx flag not cleared by software)	1250
Figure 281. ADCy_AWDx_OUT signal generation (on a single regular channel)	1250
Figure 282. ADCy_AWDx_OUT signal generation (on all injected channels)	1250
Figure 283. 20-bit to 16-bit result truncation	1252
Figure 284. Numerical example with 5-bit shift and rounding	1252
Figure 285. Triggered regular oversampling mode (TROVS bit = 1)	1254
Figure 286. Regular oversampling modes (4x ratio)	1255

Figure 287. Regular and injected oversampling modes used simultaneously	1256
Figure 288. Triggered regular oversampling with injection	1256
Figure 289. Oversampling in auto-injected mode	1257
Figure 290. Dual ADC block diagram ⁽¹⁾	1259
Figure 291. Injected simultaneous mode on 4 channels: dual ADC mode	1260
Figure 292. Regular simultaneous mode on 16 channels: dual ADC mode	1262
Figure 293. Interleaved mode on one channel in continuous conversion mode: dual ADC mode. . .	1263
Figure 294. Interleaved mode on 1 channel in single conversion mode: dual ADC mode.	1264
Figure 295. Interleaved conversion with injection	1264
Figure 296. Alternate trigger: injected group of each ADC	1265
Figure 297. Alternate trigger: 4 injected channels (each ADC) in discontinuous mode.	1266
Figure 298. Alternate + regular simultaneous	1267
Figure 299. Case of trigger occurring during injected conversion	1267
Figure 300. Interleaved single channel CH0 with injected sequence CH11, CH12	1268
Figure 301. Two Interleaved channels (CH1, CH2) with injected sequence CH11, CH12 - case 1: Master interrupted first	1268
Figure 302. Two Interleaved channels (CH1, CH2) with injected sequence CH11, CH12 - case 2: Slave interrupted first	1268
Figure 303. DMA Requests in regular simultaneous mode when MDMA = 0b00	1269
Figure 304. DMA requests in regular simultaneous mode when MDMA = 0b10	1270
Figure 305. DMA requests in interleaved mode when MDMA = 0b10	1270
Figure 306. Temperature sensor channel block diagram	1272
Figure 307. VBAT channel block diagram	1274
Figure 308. VREFINT channel block diagram	1274
Figure 309. Temperature sensor functional block diagram	1318
Figure 310. Method for low REF_CLK frequencies	1320
Figure 311. Method for high REF_CLK frequencies	1320
Figure 312. Temperature sensor sequence	1323
Figure 313. Dual-channel DAC block diagram	1337
Figure 314. Data registers in single DAC channel mode	1340
Figure 315. Data registers in dual DAC channel mode	1340
Figure 316. Timing diagram for conversion with trigger disabled TEN = 0	1342
Figure 317. DAC LFSR register calculation algorithm	1344
Figure 318. DAC conversion (SW trigger enabled) with LFSR wave generation.	1345
Figure 319. DAC triangle wave generation	1345
Figure 320. DAC conversion (SW trigger enabled) with triangle wave generation	1346
Figure 321. DAC sample and hold mode phase diagram	1348
Figure 322. VREFBUF block diagram	1374
Figure 323. Comparator functional block diagram	1380
Figure 324. Window comparators functional block diagram	1383
Figure 325. Comparator hysteresis	1384
Figure 326. Comparator output blanking	1384
Figure 327. Scaler block diagram	1388
Figure 328. DCMI block diagram	1395
Figure 329. DCMI signal waveforms	1396
Figure 330. Timing diagram	1398
Figure 331. Frame capture waveforms in snapshot mode.	1400
Figure 332. Frame capture waveforms in continuous grab mode	1401
Figure 333. Coordinates and size of the window after cropping	1401
Figure 334. Data capture waveforms.	1402
Figure 335. Pixel raster scan order	1403
Figure 336. PSSI block diagram	1417

Figure 337. Top-level block diagram	1417
Figure 338. Data enable in receive mode waveform diagram (CKPOL=0)	1421
Figure 339. Data enable waveform diagram in transmit mode (CKPOL=1)	1421
Figure 340. Ready in receive mode waveform diagram (CKPOL=0)	1422
Figure 341. Bidirectional PSSI_DE/PSSI_RDY waveform	1423
Figure 342. Bidirectional PSSI_DE/PSSI_RDY connection diagram	1423
Figure 343. CCB block diagram	1431
Figure 344. CCB operation CPU → PKA + SAES (write)	1433
Figure 345. CCB operation PKA → SAES	1433
Figure 346. CCB operation SAES → PKA	1433
Figure 347. CCB operation SAES → CCB	1434
Figure 348. CCB operation RNG → PKA (write)	1434
Figure 349. CCB operation RNG → SAES (write)	1435
Figure 350. Blob creation scenarios	1436
Figure 351. RNG block diagram	1457
Figure 352. NIST SP800-90B entropy source model	1458
Figure 353. RNG initialization overview	1461
Figure 354. RNG block diagram	1474
Figure 355. NIST SP800-90B entropy source model	1475
Figure 356. RNG initialization overview	1478
Figure 357. SAES block diagram	1495
Figure 358. Encryption/ decryption typical usage	1497
Figure 359. Typical operation with authentication	1499
Figure 360. Example of suspend mode management	1500
Figure 361. ECB encryption	1501
Figure 362. ECB decryption	1501
Figure 363. CBC encryption	1502
Figure 364. CBC decryption	1502
Figure 365. Message construction in CTR mode	1505
Figure 366. CTR encryption	1506
Figure 367. Message construction in GCM	1507
Figure 368. GCM authenticated encryption	1509
Figure 369. Message construction in GMAC mode	1512
Figure 370. GMAC authentication mode	1512
Figure 371. Message construction in CCM mode	1513
Figure 372. CCM mode authenticated encryption	1515
Figure 373. Operation with wrapped keys for SAES in ECB and CBC modes	1518
Figure 374. Operation with wrapped keys for SAES in CTR mode	1521
Figure 375. Usage of Shared-key mode	1522
Figure 376. 128-bit block construction according to the data type	1525
Figure 377. Key protection mechanisms	1527
Figure 378. AES block diagram	1549
Figure 379. Encryption/ decryption typical usage	1551
Figure 380. Typical operation with authentication	1553
Figure 381. Example of suspend mode management	1554
Figure 382. ECB encryption	1555
Figure 383. ECB decryption	1555
Figure 384. CBC encryption	1556
Figure 385. CBC decryption	1556
Figure 386. Message construction in CTR mode	1559
Figure 387. CTR encryption	1559
Figure 388. Message construction in GCM	1561

Figure 389. GCM authenticated encryption	1562
Figure 390. Message construction in GMAC mode	1565
Figure 391. GMAC authentication mode	1566
Figure 392. Message construction in CCM mode	1567
Figure 393. CCM mode authenticated encryption	1568
Figure 394. 128-bit block construction according to the data type.	1573
Figure 395. HASH block diagram	1594
Figure 396. Message data swapping feature	1597
Figure 397. HASH suspend/resume mechanism	1603
Figure 398. PKA block diagram	1618
Figure 399. Chaining of PKA with SAES or RNG	1625
Figure 400. OTFDEC block diagram	1657
Figure 401. Typical OTFDEC use in a SoC	1658
Figure 402. AES CTR decryption flow	1659
Figure 403. OTFDEC flow control overview (dual burst read request)	1660
Figure 404. Advanced-control timer block diagram	1680
Figure 405. Counter timing diagram with prescaler division change from 1 to 2	1687
Figure 406. Counter timing diagram with prescaler division change from 1 to 4	1687
Figure 407. Counter timing diagram, internal clock divided by 1	1689
Figure 408. Counter timing diagram, internal clock divided by 2	1689
Figure 409. Counter timing diagram, internal clock divided by 4	1690
Figure 410. Counter timing diagram, internal clock divided by N	1690
Figure 411. Counter timing diagram, update event when ARPE = 0 (TIMx_ARR not preloaded).	1691
Figure 412. Counter timing diagram, update event when ARPE = 1 (TIMx_ARR preloaded).	1692
Figure 413. Counter timing diagram, internal clock divided by 1	1693
Figure 414. Counter timing diagram, internal clock divided by 2	1694
Figure 415. Counter timing diagram, internal clock divided by 4	1694
Figure 416. Counter timing diagram, internal clock divided by N	1695
Figure 417. Counter timing diagram, update event when repetition counter is not used.	1695
Figure 418. Counter timing diagram, internal clock divided by 1, TIMx_ARR = 0x6	1697
Figure 419. Counter timing diagram, internal clock divided by 2	1697
Figure 420. Counter timing diagram, internal clock divided by 4, TIMx_ARR = 0x36	1698
Figure 421. Counter timing diagram, internal clock divided by N	1698
Figure 422. Counter timing diagram, update event with ARPE = 1 (counter underflow)	1699
Figure 423. Counter timing diagram, Update event with ARPE = 1 (counter overflow)	1700
Figure 424. Update rate examples depending on mode and TIMx_RCR register settings	1701
Figure 426. Control circuit in normal mode, internal clock divided by 1	1703
Figure 427. tim_ti2 external clock connection example	1703
Figure 428. Control circuit in external clock mode 1	1704
Figure 429. External trigger input block	1705
Figure 430. Control circuit in external clock mode 2	1706
Figure 431. Capture/compare channel (example: channel 1 input stage)	1706
Figure 432. Capture/compare channel 1 main circuit	1707
Figure 433. Output stage of capture/compare channel (channel 1, idem ch. 2, 3 and 4)	1708
Figure 434. Output stage of capture/compare channel (channel 5, idem ch. 6)	1708
Figure 435. PWM input mode timing	1711
Figure 436. Output compare mode, toggle on tim_oc1	1713
Figure 437. Edge-aligned PWM waveforms (ARR = 8)	1714
Figure 438. Center-aligned PWM waveforms (ARR = 8)	1715
Figure 439. Dithering principle	1716

Figure 440. Data format and register coding in dithering mode	1717
Figure 441. PWM resolution vs frequency	1718
Figure 442. PWM dithering pattern	1719
Figure 443. Dithering effect on duty cycle in center-aligned PWM mode	1720
Figure 444. Generation of 2 phase-shifted PWM signals with 50% duty cycle	1722
Figure 445. Combined PWM mode on channel 1 and 3	1723
Figure 446. 3-phase combined PWM signals with multiple trigger pulses per period	1724
Figure 447. Complementary output with symmetrical dead-time insertion	1725
Figure 448. Asymmetrical deadtime	1726
Figure 449. Dead-time waveforms with delay greater than the negative pulse	1726
Figure 450. Dead-time waveforms with delay greater than the positive pulse	1726
Figure 451. Break and Break2 circuitry overview	1729
Figure 452. Various output behavior in response to a break event on tim_brk (OSSI = 1)	1731
Figure 453. PWM output state following tim_brk and tim_brk2 assertion (OSSI = 1)	1732
Figure 454. PWM output state following tim_brk assertion (OSSI = 0)	1733
Figure 455. Output redirection (tim_brk2 request not represented)	1734
Figure 456. tim_ocref_clr input selection multiplexer	1735
Figure 457. Clearing TIMx tim_ocxref	1736
Figure 458. 6-step generation, COM example (OSSR = 1)	1737
Figure 459. Example of one pulse mode	1738
Figure 460. Retriggerable one-pulse mode	1739
Figure 461. Pulse generator circuitry	1740
Figure 462. Pulse generation on compare event, for edge-aligned and encoder modes	1741
Figure 463. Extended pulsewidth in case of concurrent triggers	1742
Figure 464. Example of counter operation in encoder interface mode	1744
Figure 465. Example of encoder interface mode with tim_ti1fp1 polarity inverted	1744
Figure 466. Quadrature encoder counting modes	1745
Figure 467. Direction plus clock encoder mode	1746
Figure 468. Directional clock encoder mode (CC1P = CC2P = 0)	1746
Figure 469. Directional clock encoder mode (CC1P = CC2P = 1)	1747
Figure 470. Index gating options	1748
Figure 471. Jittered Index signals	1748
Figure 472. Index generation for IPOS[1:0] = 11	1749
Figure 473. Counter reading with index gated on channel A (IPOS[1:0] = 11)	1749
Figure 474. Counter reading with index ungated (IPOS[1:0] = 00)	1750
Figure 475. Counter reading with index gated on channel A and B	1750
Figure 476. Encoder mode behavior in case of narrow index pulse (IPOS[1:0] = 11)	1751
Figure 477. Counter reset Narrow index pulse (closer view, ARR = 0x07)	1752
Figure 478. Index behavior in x1 and x2 mode (IPOS[1:0] = 01)	1753
Figure 479. Directional index sensitivity	1753
Figure 480. Counter reset as function of FIDX bit setting	1754
Figure 481. Index blanking	1754
Figure 482. Index behavior in clock + direction mode, IPOS[0] = 1	1755
Figure 483. Index behavior in directional clock mode, IPOS[0] = 1	1755
Figure 484. State diagram for quadrature encoded signals	1756
Figure 485. Up-counting encoder error detection	1757
Figure 486. Down-counting encode error detection	1758
Figure 487. Encoder mode change with preload transferred on update (SMSPS = 0)	1759
Figure 488. Measuring time interval between edges on three signals	1760
Figure 489. Example of Hall sensor interface	1762
Figure 490. Control circuit in reset mode	1763
Figure 491. Control circuit in Gated mode	1764

Figure 492.	Control circuit in trigger mode	1765
Figure 493.	Control circuit in external clock mode 2 + trigger mode	1766
Figure 494.	General-purpose timer block diagram	1824
Figure 495.	Counter timing diagram with prescaler division change from 1 to 2	1830
Figure 496.	Counter timing diagram with prescaler division change from 1 to 4	1830
Figure 497.	Counter timing diagram, internal clock divided by 1	1831
Figure 498.	Counter timing diagram, internal clock divided by 2	1832
Figure 499.	Counter timing diagram, internal clock divided by 4	1832
Figure 500.	Counter timing diagram, internal clock divided by N	1833
Figure 501.	Counter timing diagram, Update event when ARPE = 0 (TIMx_ARR not preloaded). . .	1833
Figure 502.	Counter timing diagram, Update event when ARPE = 1 (TIMx_ARR preloaded).	1834
Figure 503.	Counter timing diagram, internal clock divided by 1	1835
Figure 504.	Counter timing diagram, internal clock divided by 2	1836
Figure 505.	Counter timing diagram, internal clock divided by 4	1836
Figure 506.	Counter timing diagram, internal clock divided by N	1837
Figure 507.	Counter timing diagram, Update event	1837
Figure 508.	Counter timing diagram, internal clock divided by 1, TIMx_ARR = 0x6	1839
Figure 509.	Counter timing diagram, internal clock divided by 2	1839
Figure 510.	Counter timing diagram, internal clock divided by 4, TIMx_ARR = 0x36	1840
Figure 511.	Counter timing diagram, internal clock divided by N	1840
Figure 512.	Counter timing diagram, Update event with ARPE = 1 (counter underflow).	1841
Figure 513.	Counter timing diagram, Update event with ARPE = 1 (counter overflow).	1842
Figure 514.	Control circuit in normal mode, internal clock divided by 1.	1843
Figure 515.	tim_ti2 external clock connection example	1843
Figure 516.	Control circuit in external clock mode 1	1844
Figure 517.	External trigger input block	1845
Figure 518.	Control circuit in external clock mode 2	1846
Figure 519.	Capture/compare channel (example: channel 1 input stage).	1846
Figure 520.	Capture/compare channel 1 main circuit	1847
Figure 521.	Output stage of capture/compare channel (channel 1, idem ch.2, 3 and 4).	1847
Figure 522.	PWM input mode timing	1850
Figure 523.	Output compare mode, toggle on tim_oc1	1852
Figure 524.	Edge-aligned PWM waveforms (ARR = 8)	1853
Figure 525.	Center-aligned PWM waveforms (ARR = 8).	1854
Figure 526.	Dithering principle	1855
Figure 527.	Data format and register coding in dithering mode.	1856
Figure 528.	PWM resolution vs frequency (16-bit mode).	1857
Figure 529.	PWM resolution vs frequency (32-bit mode).	1857
Figure 530.	PWM dithering pattern	1858
Figure 531.	Dithering effect on duty cycle in center-aligned PWM mode	1859
Figure 532.	Generation of two phase-shifted PWM signals with 50% duty cycle	1861
Figure 533.	Combined PWM mode on channels 1 and 3	1862
Figure 534.	OCREF_CLR input selection multiplexer	1863
Figure 535.	Clearing TIMx tim_ocxref	1863
Figure 536.	Example of One-pulse mode	1864
Figure 537.	Retriggerable one-pulse mode	1866
Figure 538.	Pulse generator circuitry	1867
Figure 539.	Pulse generation on compare event, for edge-aligned and encoder modes	1867
Figure 540.	Extended pulse width in case of concurrent triggers	1868
Figure 541.	Example of counter operation in encoder interface mode	1870
Figure 542.	Example of encoder interface mode with tim_ti1fp1 polarity inverted	1870
Figure 543.	Quadrature encoder counting modes	1871

Figure 544. Direction plus clock encoder mode	1872
Figure 545. Directional clock encoder mode (CC1P = CC2P = 0)	1873
Figure 546. Directional clock encoder mode (CC1P = CC2P = 1)	1873
Figure 547. Index gating options	1875
Figure 548. Jittered Index signals	1875
Figure 549. Index generation for IPOS[1:0] = 11	1876
Figure 550. Counter reading with index gated on channel A (IPOS[1:0] = 11)	1876
Figure 551. Counter reading with index ungated (IPOS[1:0] = 00)	1877
Figure 552. Counter reading with index gated on channel A and B	1877
Figure 553. Encoder mode behavior in case of narrow index pulse (IPOS[1:0] = 11)	1878
Figure 554. Counter reset Narrow index pulse (closer view, ARR = 0x07)	1879
Figure 555. Index behavior in x1 and x2 mode (IPOS[1:0] = 01)	1880
Figure 556. Directional index sensitivity.	1880
Figure 557. Counter reset as function of FIDX bit setting	1881
Figure 558. Index blanking.	1881
Figure 559. Index behavior in clock + direction mode, IPOS[0] = 1	1882
Figure 560. Index behavior in directional clock mode, IPOS[0] = 1	1882
Figure 561. State diagram for quadrature encoded signals.	1883
Figure 562. Up-counting encoder error detection	1884
Figure 563. Down-counting encode error detection.	1885
Figure 564. Encoder mode change with preload transferred on update (SMSPS = 0)	1886
Figure 565. Control circuit in reset mode	1888
Figure 566. Control circuit in gated mode	1889
Figure 567. Control circuit in trigger mode	1889
Figure 568. Control circuit in external clock mode 2 + trigger mode	1891
Figure 569. Master/Slave timer example	1891
Figure 570. Master/slave connection example with 1 channel only timers	1892
Figure 571. Gating TIM_slv with tim_oc1ref of TIM_mstr	1893
Figure 572. Gating TIM_slv with Enable of TIM_mstr	1894
Figure 573. Triggering TIM_slv with update of TIM_mstr.	1895
Figure 574. Triggering TIM_slv with Enable of TIM_mstr	1895
Figure 575. Triggering TIM_mstr and TIM_slv with TIM_mstr tim_ti1 input.	1896
Figure 576. Basic timer block diagram.	1940
Figure 577. Control circuit in normal mode, internal clock divided by 1.	1941
Figure 578. Counter timing diagram with prescaler division change from 1 to 2	1942
Figure 579. Counter timing diagram with prescaler division change from 1 to 4	1943
Figure 580. Counter timing diagram, internal clock divided by 1	1944
Figure 581. Counter timing diagram, internal clock divided by 2	1944
Figure 582. Counter timing diagram, internal clock divided by 4	1945
Figure 583. Counter timing diagram, internal clock divided by N.	1945
Figure 584. Counter timing diagram, update event when ARPE = 0 (TIMx_ARR not preloaded).	1946
Figure 585. Counter timing diagram, update event when ARPE = 1 (TIMx_ARR preloaded).	1947
Figure 586. Dithering principle	1948
Figure 587. Data format and register coding in dithering mode	1948
Figure 588. FCnt resolution vs frequency	1949
Figure 589. PWM dithering pattern	1949
Figure 590. General-purpose timer block diagram (TIM12).	1960
Figure 591. General-purpose timer block diagram (TIM13/TIM14) ⁽²⁾	1961
Figure 592. Counter timing diagram with prescaler division change from 1 to 2	1965
Figure 593. Counter timing diagram with prescaler division change from 1 to 4	1965

Figure 594. Counter timing diagram, internal clock divided by 1	1966
Figure 595. Counter timing diagram, internal clock divided by 2	1967
Figure 596. Counter timing diagram, internal clock divided by 4	1967
Figure 597. Counter timing diagram, internal clock divided by N	1968
Figure 598. Counter timing diagram, update event when ARPE = 0 (TIMx_ARR not preloaded).	1968
Figure 599. Counter timing diagram, update event when ARPE = 1 (TIMx_ARR preloaded).	1969
Figure 600. Control circuit in normal mode, internal clock divided by 1	1970
Figure 601. tim_ti2 external clock connection example	1970
Figure 602. Control circuit in external clock mode 1	1971
Figure 603. Capture/compare channel 1 input stage (TIM13/TIM14)	1972
Figure 604. Capture/compare channel 1 input stage (TIM12)	1972
Figure 605. Capture/compare channel 1 main circuit	1973
Figure 606. Output stage of capture/compare channel 1	1973
Figure 607. PWM input mode timing	1975
Figure 608. Output compare mode, toggle on tim_oc1	1977
Figure 609. Edge-aligned PWM waveforms (ARR = 8)	1978
Figure 610. Dithering principle	1979
Figure 611. Data format and register coding in dithering mode	1979
Figure 612. PWM resolution vs frequency	1980
Figure 613. PWM dithering pattern	1981
Figure 614. Combined PWM mode on channel 1 and 2	1983
Figure 615. Example of one pulse mode	1984
Figure 616. Retriggerable one pulse mode	1985
Figure 617. Measuring time interval between edges on 2 signals	1986
Figure 618. Control circuit in reset mode	1987
Figure 619. Control circuit in gated mode	1988
Figure 620. Control circuit in trigger mode	1988
Figure 621. TIM15 block diagram	2022
Figure 622. TIM16/TIM17 block diagram ⁽³⁾	2023
Figure 623. Counter timing diagram with prescaler division change from 1 to 2	2028
Figure 624. Counter timing diagram with prescaler division change from 1 to 4	2028
Figure 625. Counter timing diagram, internal clock divided by 1	2030
Figure 626. Counter timing diagram, internal clock divided by 2	2030
Figure 627. Counter timing diagram, internal clock divided by 4	2031
Figure 628. Counter timing diagram, internal clock divided by N	2031
Figure 629. Counter timing diagram, update event when ARPE = 0 (TIMx_ARR not preloaded).	2032
Figure 630. Counter timing diagram, update event when ARPE = 1 (TIMx_ARR preloaded).	2033
Figure 631. Update rate examples depending on mode and TIMx_RCR register settings	2034
Figure 632. Control circuit in normal mode, internal clock divided by 1	2035
Figure 633. tim_ti2 external clock connection example	2035
Figure 634. Control circuit in external clock mode 1	2036
Figure 635. Capture/compare channel (example: channel 1 input stage)	2037
Figure 636. Capture/compare channel 1 main circuit	2037
Figure 637. Output stage of capture/compare channel (channel 1)	2038
Figure 638. Output stage of capture/compare channel (channel 2 for TIM15)	2038
Figure 639. PWM input mode timing	2041
Figure 640. Output compare mode, toggle on tim_oc1	2043
Figure 641. Edge-aligned PWM waveforms (ARR = 8)	2044

Figure 642. Dithering principle	2045
Figure 643. Data format and register coding in dithering mode	2045
Figure 644. PWM resolution vs frequency	2046
Figure 645. PWM dithering pattern	2047
Figure 646. Combined PWM mode on channel 1 and 2	2049
Figure 647. Complementary output with symmetrical dead-time insertion.	2050
Figure 648. Asymmetrical deadtime	2051
Figure 649. Dead-time waveforms with delay greater than the negative pulse.	2051
Figure 650. Dead-time waveforms with delay greater than the positive pulse.	2051
Figure 651. Break circuitry overview	2053
Figure 652. Output behavior in response to a break event on tim_brk	2055
Figure 653. Output redirection	2057
Figure 654. tim_ocref_clr input selection multiplexer	2058
Figure 655. 6-step generation, COM example (OSSR = 1)	2059
Figure 656. Example of one pulse mode.	2060
Figure 657. Retriggerable one pulse mode	2062
Figure 658. Measuring time interval between edges on 2 signals	2062
Figure 659. Control circuit in reset mode	2063
Figure 660. Control circuit in gated mode	2064
Figure 661. Control circuit in trigger mode	2065
Figure 662. LPTIM1/2/3/5/6 block diagram ⁽¹⁾	2129
Figure 663. LPTIM4 block diagram ⁽¹⁾	2130
Figure 664. Glitch filter timing diagram	2134
Figure 665. LPTIM output waveform, single-counting mode configuration when repetition register content is different than zero (with PRELOAD = 1)	2136
Figure 666. LPTIM output waveform, single-counting mode configuration and Set-once mode activated (WAVE bit is set)	2137
Figure 667. LPTIM output waveform, Continuous counting mode configuration	2137
Figure 668. Waveform generation	2139
Figure 669. Encoder mode counting sequence	2143
Figure 670. Continuous counting mode when repetition register LPTIM_RCR different from zero (with PRELOAD = 1)	2144
Figure 671. Capture/compare input stage (channel 1)	2145
Figure 672. Capture/compare output stage (channel 1)	2145
Figure 673. Edge-aligned PWM mode (PRELOAD = 1)	2147
Figure 674. Edge-aligned PWM waveforms (ARR=8 and CCxP = 0)	2148
Figure 675. PWM mode with immediate update versus preloaded update	2149
Figure 676. Independent watchdog block diagram	2179
Figure 677. Reset timing due to timeout	2181
Figure 678. Reset timing due to refresh in the not allowed area	2182
Figure 679. Changing PR, RL, and performing a refresh ⁽¹⁾	2183
Figure 680. Window comparator update ⁽¹⁾	2184
Figure 681. Independent watchdog interrupt timing diagram.	2186
Figure 682. Early wake-up comparator update ⁽¹⁾	2187
Figure 683. Watchdog block diagram	2195
Figure 684. Window watchdog timing diagram	2197
Figure 685. RTC block diagram	2203
Figure 686. TAMP block diagram	2261
Figure 687. Backup registers protection zones	2266
Figure 688. Tamper sampling with precharge pulse	2271
Figure 689. Low level detection with precharge and filtering	2271
Figure 690. Active tamper filtering	2274

Figure 691. Block diagram	2310
Figure 692. I ² C-bus protocol	2312
Figure 693. Setup and hold timings	2314
Figure 694. I2C initialization flow	2316
Figure 695. Data reception	2317
Figure 696. Data transmission	2318
Figure 697. Target initialization flow	2321
Figure 698. Transfer sequence flow for I2C target transmitter, NOSTRETCH = 0	2323
Figure 699. Transfer sequence flow for I2C target transmitter, NOSTRETCH = 1	2324
Figure 700. Transfer bus diagrams for I2C target transmitter (mandatory events only)	2325
Figure 701. Transfer sequence flow for I2C target receiver, NOSTRETCH = 0	2326
Figure 702. Transfer sequence flow for I2C target receiver, NOSTRETCH = 1	2327
Figure 703. Transfer bus diagrams for I2C target receiver (mandatory events only)	2327
Figure 704. Controller clock generation	2329
Figure 705. Controller initialization flow	2331
Figure 706. 10-bit address read access with HEAD10R = 0	2331
Figure 707. 10-bit address read access with HEAD10R = 1	2332
Figure 708. Transfer sequence flow for I2C controller transmitter, $N \leq 255$ bytes	2333
Figure 709. Transfer sequence flow for I2C controller transmitter, $N > 255$ bytes	2334
Figure 710. Transfer bus diagrams for I2C controller transmitter (mandatory events only)	2335
Figure 711. Transfer sequence flow for I2C controller receiver, $N \leq 255$ bytes	2337
Figure 712. Transfer sequence flow for I2C controller receiver, $N > 255$ bytes	2338
Figure 713. Transfer bus diagrams for I2C controller receiver (mandatory events only)	2339
Figure 714. Timeout intervals for $t_{\text{LOW:SEXT}}$, $t_{\text{LOW:MEXT}}$	2343
Figure 715. Transfer sequence flow for SMBus target transmitter N bytes + PEC	2346
Figure 716. Transfer bus diagram for SMBus target transmitter (SBC = 1)	2347
Figure 717. Transfer sequence flow for SMBus target receiver N bytes + PEC	2348
Figure 718. Bus transfer diagrams for SMBus target receiver (SBC = 1)	2349
Figure 719. Bus transfer diagrams for SMBus controller transmitter	2350
Figure 720. Bus transfer diagrams for SMBus controller receiver	2352
Figure 721. I3C block diagram	2376
Figure 722. I3C (primary) controller state and programming sequence diagram	2381
Figure 723. I3C target state and programing sequence diagram, when initialized on an I3C bus without a static address	2386
Figure 724. I3C target state and programing sequence diagram when initialized on an I3C bus with a static address,	2387
Figure 725. I3C target state and programing sequence diagram, when initialized on an I2C bus with a static address	2388
Figure 726. I3C CCC messages, as controller	2402
Figure 727. I3C broadcast ENTDAACCC, as controller	2403
Figure 728. I3C broadcast, direct read and direct write RSTACT CCC, as controller	2404
Figure 729. I3C CCC messages, as target	2406
Figure 730. I3C broadcast ENTDAACCC, as target	2407
Figure 731. I3C broadcast DEFTGTS CCC, as target	2408
Figure 732. I3C broadcast DEFGRPA CCC, as target	2409
Figure 733. I3C private read/write messages, as controller	2411
Figure 734. I3C private read/write messages, as target	2412
Figure 735. Legacy I2C read/write messages, as controller	2413
Figure 736. Legacy I2C read/write messages, as target	2414

Figure 737. IBI transfer, as controller/target	2415
Figure 738. Hot-join request transfer, as controller/target	2416
Figure 739. Controller-role request transfer, as controller/target	2417
Figure 740. C-FIFO management, as controller	2418
Figure 741. TX-FIFO management, as controller	2420
Figure 742. RX-FIFO management, as controller	2422
Figure 743. S-FIFO management, as controller	2425
Figure 744. RX-FIFO management, as target on the I3C bus	2427
Figure 745. RX-FIFO management, as target on the I2C bus	2428
Figure 746. TX-FIFO management with I3C_TGTTDR, as target on the I3C bus	2430
Figure 747. TX-FIFO management with I3C_TGTTDR, as target on the I2C bus	2431
Figure 748. TX-FIFO management by software without I3C_TGTTDR if reading less bytes than TX-FIFO size, as target	2433
Figure 749. I3C block diagram	2492
Figure 750. I3C (primary) controller state and programming sequence diagram	2497
Figure 751. I3C target state and programming sequence diagram	2502
Figure 752. I3C CCC messages, as controller	2516
Figure 753. I3C broadcast ENTDAACCC, as controller	2517
Figure 754. I3C broadcast, direct read and direct write RSTACT CCC, as controller	2518
Figure 755. I3C CCC messages, as target	2520
Figure 756. I3C broadcast ENTDAACCC, as target	2521
Figure 757. I3C broadcast DEFTGTS CCC, as target	2522
Figure 758. I3C broadcast DEFGRPA CCC, as target	2523
Figure 759. I3C private read/write messages, as controller	2525
Figure 760. I3C private read/write messages, as target	2526
Figure 761. Legacy I2C read/write messages, as controller	2527
Figure 762. IBI transfer, as controller/target	2528
Figure 763. Hot-join request transfer, as controller/target	2529
Figure 764. Controller-role request transfer, as controller/target	2530
Figure 765. C-FIFO management, as controller	2531
Figure 766. TX-FIFO management, as controller	2533
Figure 767. RX-FIFO management, as controller	2535
Figure 768. S-FIFO management, as controller	2538
Figure 769. RX-FIFO management, as target on the I3C bus	2539
Figure 770. TX-FIFO management with I3C_TGTTDR, as target on the I3C bus	2541
Figure 771. TX-FIFO management by software without I3C_TGTTDR if reading less bytes than TX-FIFO size, as target	2543
Figure 772. USART block diagram	2599
Figure 773. Word length programming	2603
Figure 774. Configurable stop bits	2605
Figure 775. TC/TXE behavior when transmitting	2607
Figure 776. Start bit detection when oversampling by 16 or 8	2608
Figure 777. usart_ker_ck clock divider block diagram	2611
Figure 778. Data sampling when oversampling by 16	2612
Figure 779. Data sampling when oversampling by 8	2613
Figure 780. Mute mode using Idle line detection	2620
Figure 781. Mute mode using address mark detection	2621
Figure 782. Break detection in LIN mode (11-bit break length - LBDL bit is set)	2624
Figure 783. Break detection in LIN mode vs. Framing error detection	2625
Figure 784. USART example of synchronous master transmission	2626
Figure 785. USART data clock timing diagram in synchronous master mode (M bits = 00)	2626

Figure 786. USART data clock timing diagram in synchronous master mode (M bits = 01)	2627
Figure 787. USART data clock timing diagram in synchronous slave mode (M bits = 00)	2628
Figure 788. ISO 7816-3 asynchronous protocol	2630
Figure 789. Parity error detection using the 1.5 stop bits	2632
Figure 790. IrDA SIR ENDEC block diagram	2636
Figure 791. IrDA data modulation (3/16) - normal mode	2636
Figure 792. Transmission using DMA	2638
Figure 793. Reception using DMA	2639
Figure 794. Hardware flow control between two USARTs	2639
Figure 795. RS232 RTS flow control	2640
Figure 796. RS232 CTS flow control	2641
Figure 797. Wake-up event verified (wake-up event = address match, FIFO disabled)	2644
Figure 798. Wake-up event not verified (wake-up event = address match, FIFO disabled)	2644
Figure 799. LPUART block diagram	2689
Figure 800. LPUART word length programming	2692
Figure 801. Configurable stop bits	2694
Figure 802. TC/TXE behavior when transmitting	2696
Figure 803. lpuart_ker_ck clock divider block diagram	2700
Figure 804. Mute mode using Idle line detection	2704
Figure 805. Mute mode using address mark detection	2705
Figure 806. Transmission using DMA	2707
Figure 807. Reception using DMA	2708
Figure 808. Hardware flow control between two LPUARTs	2709
Figure 809. RS232 RTS flow control	2709
Figure 810. RS232 CTS flow control	2710
Figure 811. Wake-up event verified (wake-up event = address match, FIFO disabled)	2713
Figure 812. Wake-up event not verified (wake-up event = address match, FIFO disabled)	2713
Figure 813. SPI/I2S block diagram	2745
Figure 814. Full-duplex single master/ single slave application	2748
Figure 815. Half-duplex single master/ single slave application	2749
Figure 816. Simplex single master / single slave application (master in transmit-only / slave in receive-only mode)	2750
Figure 817. Master and three independent slaves connected in star topology	2751
Figure 818. Master and three slaves connected in circular (daisy chain) topology	2753
Figure 819. Multimaster application	2754
Figure 820. Scheme of NSS control logic	2756
Figure 821. Data flow timing control (SSOE = 1, SSOM = 0, SSM = 0)	2756
Figure 822. NSS interleaving pulses between data (SSOE = 1, SSOM = 1, SSM = 0)	2757
Figure 823. Data clock timing diagram	2759
Figure 824. Data alignment when data size is not equal to 8, 16 or 32 bits	2760
Figure 825. TI mode transfer	2769
Figure 826. Optional configurations of the slave behavior when an underrun condition is detected	2771
Figure 827. Waveform examples	2779
Figure 828. Master I2S Philips protocol waveforms (16/32-bit full accuracy)	2780
Figure 829. I2S Philips standard waveforms	2780
Figure 830. Master MSB-justified 16- or 32-bit full-accuracy length	2781
Figure 831. Master MSB-justified 16- or 24-bit data length	2781

Figure 832. Slave MSB-justified 16-, 24- or 32-bit data length	2782
Figure 833. LSB-justified 16 or 24-bit data length	2782
Figure 834. Master PCM when the frame length is equal the data length	2783
Figure 835. Master PCM standard waveforms (16 or 24-bit data length)	2783
Figure 836. Slave PCM waveforms	2784
Figure 837. Startup sequence, I2S Philips standard, master	2787
Figure 838. Startup sequence, I2S Philips standard, slave	2788
Figure 839. Stop sequence, I2S Philips standard, master	2788
Figure 840. I ² S clock generator architecture	2789
Figure 841. Data Format	2791
Figure 842. Handling of underrun situation	2793
Figure 843. Handling of overrun situation	2794
Figure 844. Frame error detection, with FIXCH = 0	2795
Figure 845. Frame error detection, with FIXCH = 1	2795
Figure 846. SAI functional block diagram	2821
Figure 847. Audio frame	2825
Figure 848. FS role is start of frame + channel side identification (FSDEF = TRIS = 1)	2827
Figure 849. FS role is start of frame (FSDEF = 0)	2828
Figure 850. Slot size configuration with FBOFF = 0 in SAI_xSLOTR	2829
Figure 851. First bit offset	2829
Figure 852. Audio block clock generator overview	2831
Figure 853. PDM typical connection and timing	2835
Figure 854. Detailed PDM interface block diagram	2836
Figure 855. Start-up sequence	2837
Figure 856. SAI_ADR format in TDM mode, 32-bit slot width	2838
Figure 857. SAI_ADR format in TDM mode, 16-bit slot width	2839
Figure 858. SAI_ADR format in TDM mode, 8-bit slot width	2840
Figure 859. AC'97 audio frame	2843
Figure 860. Example of typical AC'97 configuration on devices featuring at least two embedded SAIs (three external AC'97 decoders)	2844
Figure 861. SPDIF format	2845
Figure 862. SAI_xDR register ordering	2846
Figure 863. Data companding hardware in an audio block in the SAI	2849
Figure 864. Tristate strategy on SD output line on an inactive slot	2851
Figure 865. Tristate on output data line in a protocol like I2S	2852
Figure 866. Overrun detection error	2853
Figure 867. FIFO underrun event	2853
Figure 868. CAN subsystem	2888
Figure 869. FDCAN block diagram	2890
Figure 870. Bit timing	2892
Figure 871. Transceiver delay measurement	2897
Figure 872. Pin control in bus monitoring mode	2898
Figure 873. Pin control in loop-back mode	2901
Figure 874. CAN error state diagram	2902
Figure 875. Message RAM configuration	2903
Figure 876. Standard message ID filter path	2906
Figure 877. Extended message ID filter path	2907
Figure 878. USB peripheral block diagram	2954
Figure 879. Packet buffer areas with examples of buffer description table locations	2960
Figure 880. UCPD block diagram	3001
Figure 881. Clock division and timing elements	3003
Figure 882. K-code transmission	3005

Figure 883. Transmit order for various sizes of data	3007
Figure 884. Packet format	3007
Figure 885. Line format of Hard Reset.	3008
Figure 886. Line format of Cable Reset.	3008
Figure 887. BIST test data frame.	3009
Figure 888. BIST Carrier Mode 2 frame.	3010
Figure 889. UCPD BMC transmitter architecture.	3010
Figure 890. UCPD BMC receiver architecture	3012
Figure 891. High-level Ethernet block diagram	3045
Figure 892. DMA transmission flow (standard mode)	3048
Figure 893. DMA transmission flow (OSP mode)	3050
Figure 894. Receive DMA flow	3052
Figure 895. Overview of MAC transmission flow	3056
Figure 896. MAC reception flow	3058
Figure 897. Packet filtering sequence	3062
Figure 898. Networked time synchronization	3071
Figure 899. Propagation delay calculation in clocks supporting peer-to-peer path correction	3072
Figure 900. System time update using fine correction method	3082
Figure 901. TCP segmentation offload overview	3099
Figure 902. TCP segmentation offload flow	3100
Figure 903. Header and payload fields of segmented packets	3103
Figure 904. Supported PHY interfaces	3112
Figure 905. SMA Interface block	3112
Figure 906. MDIO packet structure (Clause 45)	3113
Figure 907. MDIO packet structure (Clause 22)	3114
Figure 908. SMA write operation flow	3116
Figure 909. Write data packet	3117
Figure 910. Read data packet	3117
Figure 911. Media independent interface (MII) signals	3119
Figure 912. RMII block diagram.	3121
Figure 913. Transmission bit order	3122
Figure 914. Receive bit order.	3123
Figure 915. LPI transitions (Transmit, 100 Mbds)	3131
Figure 916. LPI Tx clock gating (when LPITCSE = 1)	3132
Figure 917. LPI transitions (receive, 100 Mbit/s)	3133
Figure 918. Descriptor ring structure	3154
Figure 919. DMA descriptor ring	3156
Figure 920. Descriptor tail pointer example 1	3156
Figure 921. Descriptor tail pointer example 2	3157
Figure 922. Transmit descriptor (read format)	3158
Figure 923. Transmit descriptor write-back format.	3163
Figure 924. Transmit context descriptor format	3167
Figure 925. Receive normal descriptor (read format)	3170
Figure 926. Receive normal descriptor (write-back format)	3172
Figure 927. Receive context descriptor	3180
Figure 928. Generation of ETH_DMAISR flags	3195
Figure 929. High-level Ethernet block diagram	3313
Figure 930. DMA transmission flow (standard mode)	3316
Figure 931. DMA transmission flow (OSP mode)	3318
Figure 932. Receive DMA flow	3320
Figure 933. Overview of MAC transmission flow	3324

Figure 934. MAC reception flow	3326
Figure 935. Packet filtering sequence	3330
Figure 936. Networked time synchronization	3339
Figure 937. Propagation delay calculation in clocks supporting peer-to-peer path correction	3340
Figure 938. System time update using fine correction method	3350
Figure 939. TCP segmentation offload overview	3367
Figure 940. TCP segmentation offload flow	3368
Figure 941. Header and payload fields of segmented packets	3371
Figure 942. Supported PHY interfaces	3380
Figure 943. SMA Interface block	3380
Figure 944. MDIO packet structure (Clause 45)	3381
Figure 945. MDIO packet structure (Clause 22)	3382
Figure 946. SMA write operation flow	3384
Figure 947. Write data packet	3385
Figure 948. Read data packet	3385
Figure 949. Media independent interface (MII) signals	3387
Figure 950. RMII block diagram	3389
Figure 951. Transmission bit order	3390
Figure 952. Receive bit order	3391
Figure 953. LPI transitions (Transmit, 100 Mbds)	3399
Figure 954. LPI Tx clock gating (when LPITCSE = 1)	3400
Figure 955. LPI transitions (receive, 100 Mbit/s)	3401
Figure 956. Descriptor ring structure	3425
Figure 957. DMA descriptor ring	3427
Figure 958. Descriptor tail pointer example 1	3427
Figure 959. Descriptor tail pointer example 2	3428
Figure 960. Transmit descriptor (read format)	3429
Figure 961. Transmit descriptor write-back format	3434
Figure 962. Transmit context descriptor format	3438
Figure 963. Receive normal descriptor (read format)	3442
Figure 964. Receive normal descriptor (write-back format)	3445
Figure 965. Receive context descriptor	3452
Figure 966. Generation of ETH_DMAISR flags	3467
Figure 967. HDMI-CEC block diagram	3589
Figure 968. Message structure	3589
Figure 969. Blocks	3590
Figure 970. Bit timings	3590
Figure 971. Signal free time	3591
Figure 972. Arbitration phase	3591
Figure 973. SFT of three nominal bit periods	3591
Figure 974. Error bit timing	3592
Figure 975. Error handling	3594
Figure 976. TXERR detection	3595
Figure 977. Block diagram of debug support infrastructure	3606
Figure 978. Product life cycle states and debug authentication	3610
Figure 979. JTAG TAP state machine	3614
Figure 980. CoreSight topology	3635
Figure 981. Trace port interface unit (TPIU)	3714
Figure 982. Embedded cross trigger	3726

1 Documentation conventions

1.1 General information

The STM32H5 devices have an Arm® Cortex®-M33 core.



Note: Arm and Cortex are registered trademarks of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere.

The Arm word and logo are trademarks of Arm Limited (or its subsidiaries) in the US and/or elsewhere. All rights reserved.

1.2 List of abbreviations for registers

The following abbreviations^(a) are used in register descriptions:

read/write (rw)	Software can read and write to this bit.
read-only (r)	Software can only read this bit.
write-only (w)	Software can only write to this bit. Reading this bit returns the reset value.
read/clear write0 (rc_w0)	Software can read as well as clear this bit by writing 0. Writing 1 has no effect on the bit value.
read/clear write1 (rc_w1)	Software can read as well as clear this bit by writing 1. Writing 0 has no effect on the bit value.
read/clear write (rc_w)	Software can read as well as clear this bit by writing to the register. The value written to this bit is not important.
read/clear by read (rc_r)	Software can read this bit. Reading this bit automatically clears it to 0. Writing this bit has no effect on the bit value.
read/set by read (rs_r)	Software can read this bit. Reading this bit automatically sets it to 1. Writing this bit has no effect on the bit value.
read/set (rs)	Software can read as well as set this bit. Writing 0 has no effect on the bit value.
read/write once (rwo)	Software can only write once to this bit and can also read it at any time. Only a reset can return the bit to its reset value.
toggle (t)	The software can toggle this bit by writing 1. Writing 0 has no effect.
read-only write trigger (rt_w1)	Software can read this bit. Writing 1 triggers an event but has no effect on the bit value.
Reserved (Res.)	Reserved bit, must be kept at reset value.

a. This is an exhaustive list of all abbreviations applicable to STMicroelectronics microcontrollers, some of them may not be used in the current document.

1.3 Register reset value

Bits (binary notation) or bits nibbles (hexadecimal notation) of which the reset value is undefined are marked as X.

Bits (binary notation) or bits nibbles (hexadecimal notation) of which the reset value is unmodified are marked as U.

1.4 Glossary

This section gives a brief definition of acronyms and abbreviations used in this document:

- **Word**: data of 32-bit length.
- **Half-word**: data of 16-bit length.
- **Byte**: data of 8-bit length.
- **AHB**: advanced high-performance bus.

2 Memory and bus architecture

2.1 System architecture

The architecture relies on a Arm Cortex-M33 core optimized for execution thanks to an instruction cache having a direct access to the embedded flash memory.

This architecture also features a 32-bit multilayer AHB bus matrix with the interconnections detailed in [Table 1](#) and [Table 2](#).

Table 1. Implementation of masters

Master	Comments	STM32H523/ 533xx	STM32H562/563/ 573xx	STM32H543/ 553xx
Cortex-M33 Fast C-bus	Connecting Cortex-M33 (with Arm TrustZone® mainline and FPU) to the internal SRAMs and flash memory through ICACHE	X	X	X
Cortex-M33 Slow C-bus	Connecting Cortex-M33 (with Arm TrustZone mainline and FPU) to the external memories through ICACHE	X	X	X
Cortex-M33 S-bus	Connecting the Cortex-M33 (with Arm TrustZone mainline and FPU) to internal SRAMs without latency	3 masters		
	Connecting Cortex-M33 (with Arm TrustZone mainline and FPU) to the external memories through DCACHE	X	X	X
GPDMA1	-	2 masters		
GPDMA2	-	2 masters		
SDMMC1	-	X	X	X
SDMMC2	-	-	X	-
Ethernet MAC	-	-	X ⁽¹⁾	X

1. Not available on STM32H562xx devices.

Table 2. Implementation of slaves

Slave	Comments	STM32H523/ 533xx	STM32H562/563/ 573xx	STM32H543/ 553xx
Flash memory	-	X	X	X
SRAM1	-	X	X	X
SRAM2	-	X	X	X
SRAM3	-	X	X	X

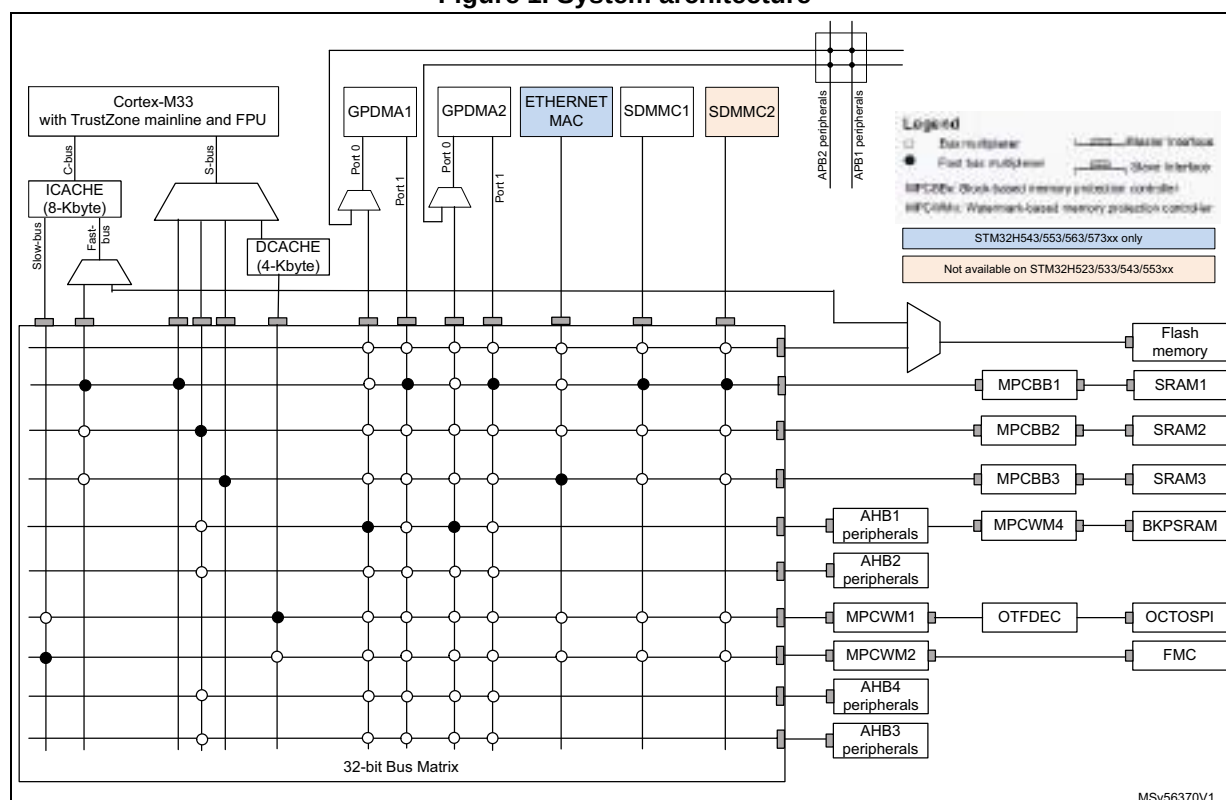
Table 2. Implementation of slaves (continued)

Slave	Comments	STM32H523/ 533xx	STM32H562/563/ 573xx	STM32H543/ 553xx
AHB1	Peripherals and BKPSRAM including AHB to APB bridge, and APB peripherals (connected to APB1 and APB2)	X	X	X
AHB2	Peripherals	X	X	X
FMC	-	X ⁽¹⁾	X	X
OCTOSPI1	-	X	X	X
AHB3	Including AHB to APB bridge, and APB peripherals (connected to APB3)	X	X	X
AHB4	Peripherals	X	X	X

1. STM32H523/533xx devices do not support SDRAM.

The bus matrix provides access from a master to a slave, enabling concurrent accesses and efficient operation even when several high-speed peripherals work simultaneously. This architecture is shown in [Figure 1](#).

Figure 1. System architecture



2.1.1 Fast C-bus

This bus connects the C-bus of the Cortex-M33 core to the internal flash memory and to the BusMatrix via the instruction cache. This bus is used for instruction fetch and data access to

the internal memories mapped in code region. This bus targets the internal flash memory and the internal SRAMs (SRAM1, SRAM2 and SRAM3).

SRAM1, SRAM2 and SRAM3 are accessible on this bus with a continuous mapping.

2.1.2 Slow C-bus

This bus connects the C-bus of the Cortex-M33 core to the BusMatrix via the instruction cache. This bus is used for instruction fetch and data access to the external memories mapped in code region. This bus targets the external memories (FMC and OCTOSPI).

2.1.3 S-bus

This bus connects the system bus of the Cortex-M33 core to the BusMatrix. This bus is used by the core to access data located in a peripheral or SRAM area. This bus targets the internal SRAMs (SRAM1, SRAM2, SRAM3, and BKPSRAM), the AHB1 peripherals including the APB1, APB2, AHB2, AHB3 and AHB4 peripherals.

SRAM1, SRAM2 and SRAM3 are accessible on this bus with a continuous mapping.

Note: The Bus Matrix has a zero latency when accessing SRAM1, SRAM2 and SRAM3.

2.1.4 DCache S-bus

This bus connects the system bus of the Cortex-M33 core to the BusMatrix via the data cache. This bus is used for instruction fetch and data access to the external memories mapped in data region. This bus targets the external memories (FMC and OCTOSPI).

Note: Fetching instructions through this bus is less efficient than fetching instructions through the slow C-bus.

2.1.5 GPDMA1 and GPDMA2 buses

These buses connect the four AHB master interfaces of the GPDMA1 and GPDMA2 to the BusMatrix. These buses target the internal flash memory, the internal SRAMs (SRAM1, SRAM2, SRAM3, and BKPSRAM), the AHB1 peripherals including the APB1 and APB2 peripherals, the AHB2 peripherals, the AHB3 peripherals, AHB4 peripherals and the external memories through FMC or OCTOSPI.

2.1.6 SDMMC1 and SDMMC2^(a) controllers DMA buses

These buses connect the SDMMC1 and SDMMC2 DMA master interfaces to the BusMatrix. They are used only by the SDMMC1 and SDMMC2 DMA to load/store data from/to the memory. These buses target the data memories: internal flash memory, internal SRAMs (SRAM1, SRAM2 and SRAM3), and external memories through FMC or OCTOSPI.

2.1.7 BusMatrix

The BusMatrix manages the access arbitration between masters. The arbitration uses a round-robin algorithm. This BusMatrix features a fast bus multiplexer used to connect each

a. SDMMC2 is not available on STM32H523/533xx and STM32H543/553xx devices.

master to a given slave without latency (see [Figure 1](#)). For the same master, other slaves undergo a latency of at least one cycle at each new access.

2.1.8 AHB/APB bridges

The three AHB/APB bridges provide full synchronous connections between the AHB and the APB buses, allowing flexible selection of the peripheral frequency.

Refer to [Section 2.3.2: Memory map and register boundary addresses](#) for the address mapping of the peripherals connected to these bridges.

After each device reset, all peripheral clocks are disabled (except for the internal SRAMs and flash memory interfaces). Before using a peripheral, its clock must be enabled in the RCC_AHBxENR and RCC_APBxENR registers.

Note: When a 8- or 16-bit access is performed on an APB register, the access is transformed into a 32-bit access: the bridge duplicates the 8- or 16-bit data to feed the 32-bit vector.

2.1.9 Ethernet MAC^(a)

The Ethernet MAC uses a 32-bit bus, connected to the AHB bus matrix. Through the system bus matrix, it can access the internal flash memory, the internal memories, and the external memories through the OCTOSPI and the FMC.

2.2 TrustZone security architecture

The security architecture is based on Arm TrustZone with the Armv8-M mainline extension.

The TrustZone security is activated by the TZEN option bit in the FLASH_OTPR register.

When the TrustZone is enabled, the SAU (security-attribution unit) and IDAU (implementation-defined-attribution unit) defines the access permissions based on secure and nonsecure states.

- SAU: Up to eight SAU configurable regions are available for security attribution.
- IDAU: provides a first memory partition as nonsecure or nonsecure callable attributes. The IDAU memory map partition is not configurable, it is fixed by hardware implementation (refer to [Figure 2: Memory map based on IDAU mapping \(STM32H562/563/573xx devices\)](#), [Figure 3: Memory map based on IDAU mapping \(STM32H523/533xx devices\)](#), and [Figure 4: Memory map based on IDAU mapping \(STM32H543/553xx devices\)](#)). It is then combined with the results from the SAU security attribution, and the higher security state is selected.

Based on IDAU security attribution, the flash memory, system SRAMs and peripherals memory spaces are aliased twice for secure and nonsecure states. However, the external memories space is not aliased.

2.2.1 Default TrustZone security state

When the TrustZone security is activated by the TZEN option bit in the FLASH_OTPR, the default system security state is detailed below:

a. Not available on STM32H523/533xx and STM32H562xx devices.

- CPU:
 - Cortex-M33 is in secure state after reset. The boot address must be at a secure address.
- Memory map:
 - SAU is fully secure after reset. Consequently, all memory map is fully secure. Up to eight SAU configurable regions are available for security attribution.
- Flash memory:
 - The flash memory security area is defined by watermark user options.
 - Block-based security attributions are nonsecure after reset.
- SRAMs:
 - All SRAMs are secure after reset. MPCBBx (block-based memory protection controller) are secure.
- External memories:
 - FMC and OCTOSPI banks are secure after reset. MPCWMx (watermark-based memory protection controller) are secure.
- Peripherals (see [Table 3](#) and [Table 4](#) for a list of securable and TrustZone-aware peripherals)
 - Securable peripherals are nonsecure after reset.
 - TrustZone-aware peripherals are nonsecure after reset. Their secure configuration registers are secure.
- All GPIO are secure after reset.
- Interrupts:
 - NVIC: All interrupts are secure after reset. NVIC is banked for secure and nonsecure state.
 - TZIC: All illegal access interrupts are disabled after reset (see [GTZC TrustZone system architecture](#)).

2.2.2 TrustZone peripheral classification

When the TrustZone security is active, a peripheral can be either securable or TrustZone-aware type as follows:

- Securable: peripheral protected by an AHB/APB firewall gate that is controlled from TZSC controller to define security properties
- TrustZone-aware: peripheral connected directly to AHB or APB bus and implementing a specific TrustZone behavior such as a subset of registers being secure.

Refer to [GTZC TrustZone system architecture](#) for more details.

The following tables list the securable and TrustZone-aware peripherals within the system.

Table 3. Securable peripherals by TZSC

Bus	Peripheral	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
AHB4	OCTOSPI	X	X	X
	FMC	X	X	X
	SDMMC2	-	X ⁽¹⁾	-
	SDMMC1	X	X	X

Table 3. Securable peripherals by TZSC (continued)

Bus	Peripheral	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
AHB2	CCB	-	-	X
	PKA	X	X	X
	SAES ⁽²⁾	X	X	X
	RNG	X	X	X
	HASH	X	X	X
	AES ⁽²⁾	X	X	X
	DCMI/PSSI	X	X	-
	ADC1/ADC2	X	X	X
	ADC3	-	-	X
	DAC1/DAC2	X	X	X
AHB1	DCACHE registers	X	X	X
	ICACHE registers	X	X	X
	ETHERNET	-	X ⁽¹⁾	X
	RAMCFG	X	X	X
	FMAC	-	X	-
	CORDIC	-	X	X
	CRC	X	X	X
APB3	VREFBUF	X	X	X
	LPTIM1	X	X	X
	LPTIM3	-	X	-
	LPTIM4	-	X	-
	LPTIM5	-	X	-
	LPTIM6	-	X	-
	I2C4	-	X	-
	I2C3	X	X	X
	LPUART1	X	X	X
	SPI5	-	X	-
	I3C2	X	-	X

Table 3. Securable peripherals by TZSC (continued)

Bus	Peripheral	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
APB2	USB FS	X	X	X
	SAI2	-	X	-
	SAI1	-	X	-
	SPI6	-	X	-
	SPI4	X	X	X
	TIM17	-	X	-
	TIM16	-	X	-
	TIM15	X	X	X
	USART1	X	X	X
	TIM8	X	X	X
	SPI1	X	X	X
	TIM1	X	X	X

Table 3. Securable peripherals by TZSC (continued)

Bus	Peripheral	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
APB1	COMP	-	-	X
	UCPD	X	X	X
	FDCAN2	X	X ⁽¹⁾	X
	FDCAN1	X	X	X
	LPTIM2	X	X	X
	DTS	X	X	X
	UART12	-	X	-
	UART9	-	X	-
	UART8	-	X	X
	UART7	-	X	X
	HDMI-CEC	X	X	X
	USART11	-	X	-
	USART10	-	X	-
	USART6	X	X	X
	CRS	X	X	X
	I3C1	X	X	X
	I2C2	X	X	X
	I2C1	X	X	X
	UART5	X	X	X
	UART4	X	X	X
	USART3	X	X	X
	USART2	X	X	X
	SPI3/I2S3	X	X	X
	SPI2/I2S2	X	X	X
	IWDG	X	X	X
	WWDG	X	X	X
	TIM14	-	X	-
	TIM13	-	X	-
	TIM12	X	X	X
	TIM7	X	X	X
	TIM6	X	X	X
	TIM5	X	X	X
	TIM4	X	X	X
	TIM3	X	X	X
	TIM2	X	X	X

1. Not available on STM32H562xx devices.
2. Available only on devices with cryptography (STM32H533/553/573xx).

Table 4. TrustZone-aware peripherals

Bus	Peripheral	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
AHB4	OTFDEC1	X	X	X
AHB3	EXTI	X	X	X
	RCC	X	X	X
	PWR	X	X	X
AHB2	GPIOI	-	X	-
	GPIOH	X	X	X
	GPIOG	X	X	X
	GPIOF	X	X	X
	GPIOE	X	X	X
	GIPOD	X	X	X
	GPIOC	X	X	X
	GPIOB	X	X	X
	GPIOA	X	X	X
AHB1	GTZC	X	X	X
	DCACHE	X	X	X
	ICACHE	X	X	X
	FLASH	X	X	X
	GPDMA2	X	X	X
	GPDMA1	X	X	X
APB3	TAMP	X	X	X
	RTC	X	X	X
	SBS	X	X	X
	PLAY	-	-	X

2.3 Memory organization

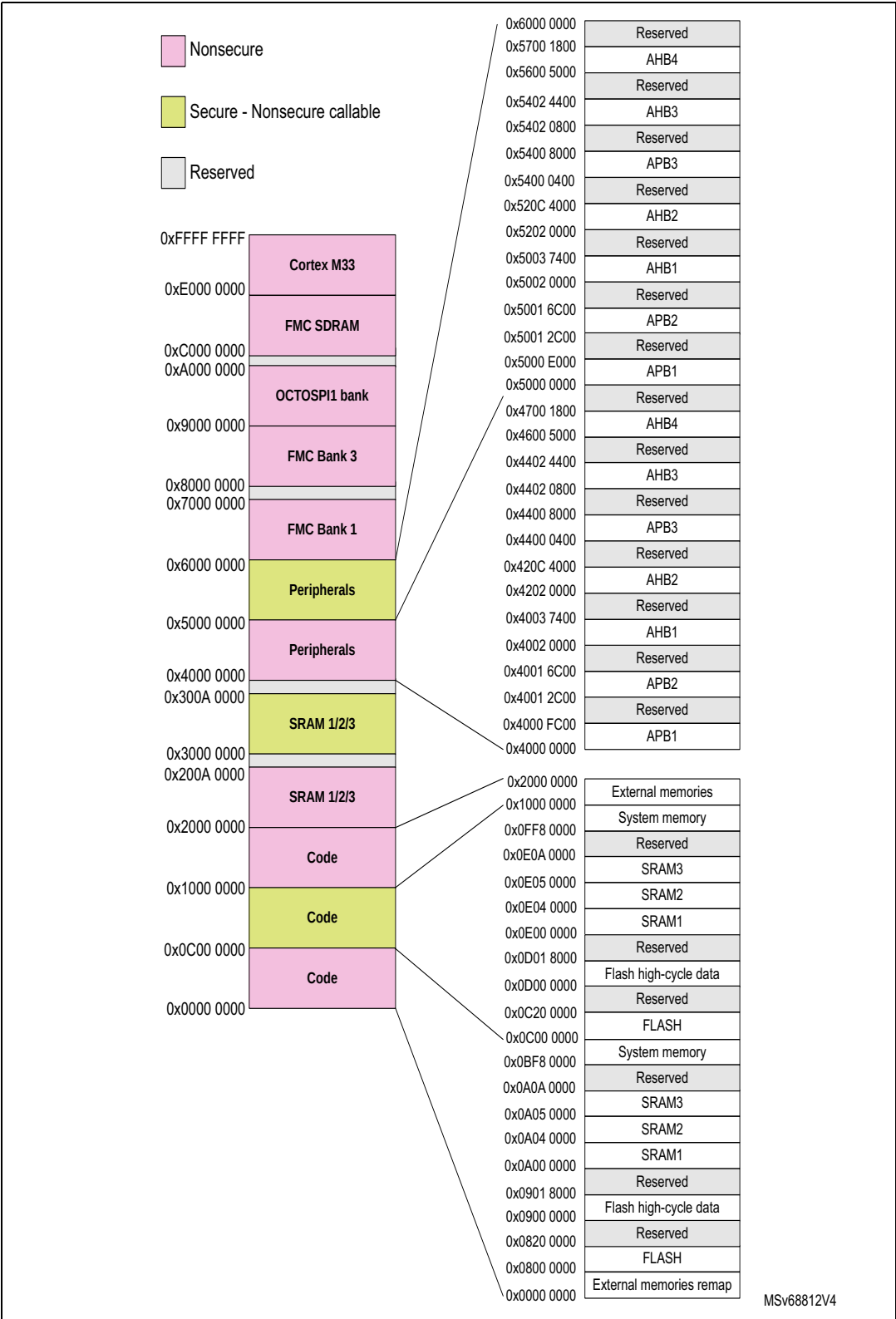
2.3.1 Introduction

Program memory, data memory, registers and I/O ports are organized within the same linear address space.

The bytes are coded in memory in Little Endian format. The lowest numbered byte in a word is considered the word's least significant byte and the highest numbered byte the most significant.

2.3.2 Memory map and register boundary addresses

Figure 2. Memory map based on IDAU mapping (STM32H562/563/573xx devices)



MSv68812V4

Figure 3. Memory map based on IDAU mapping (STM32H523/533xx devices)

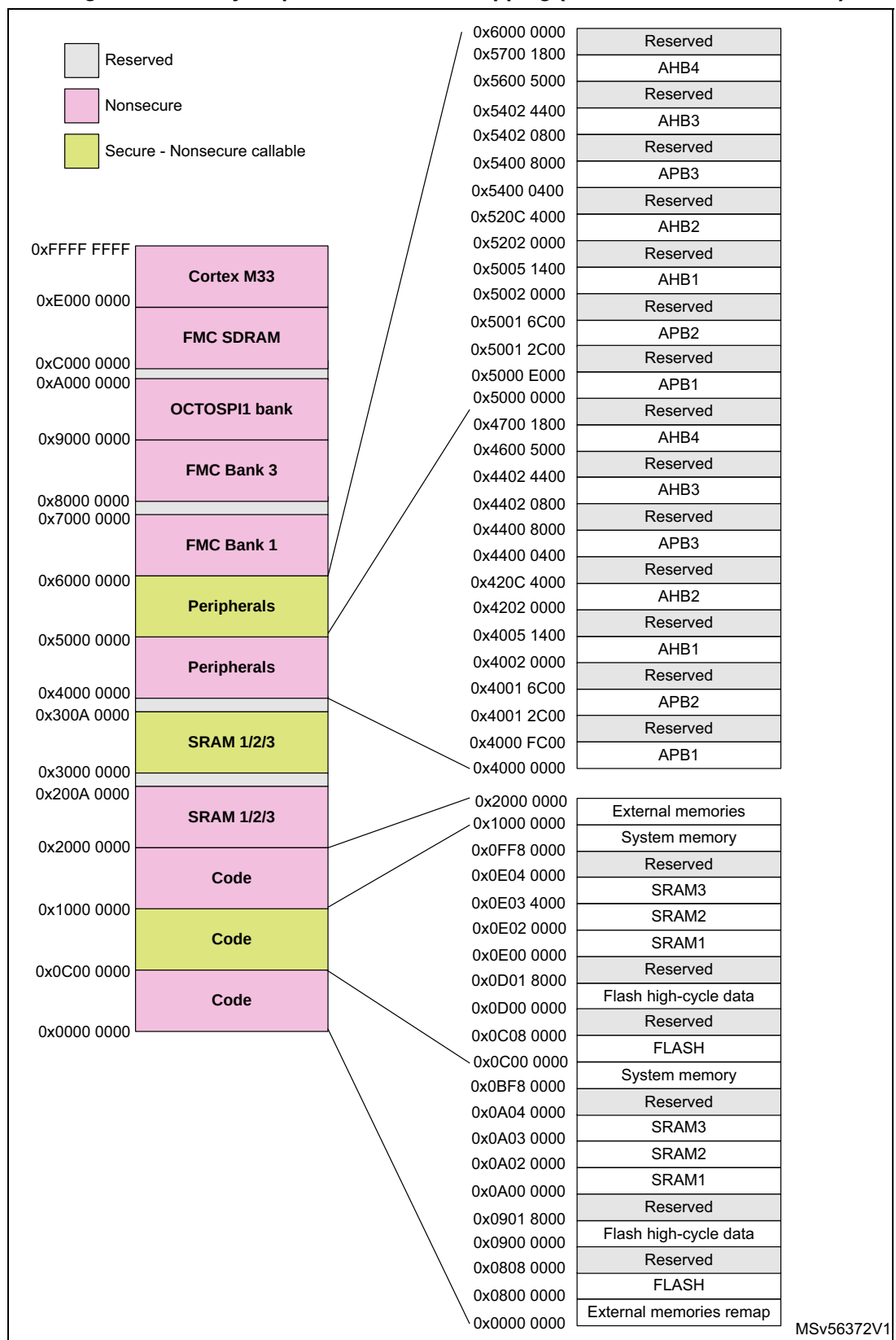
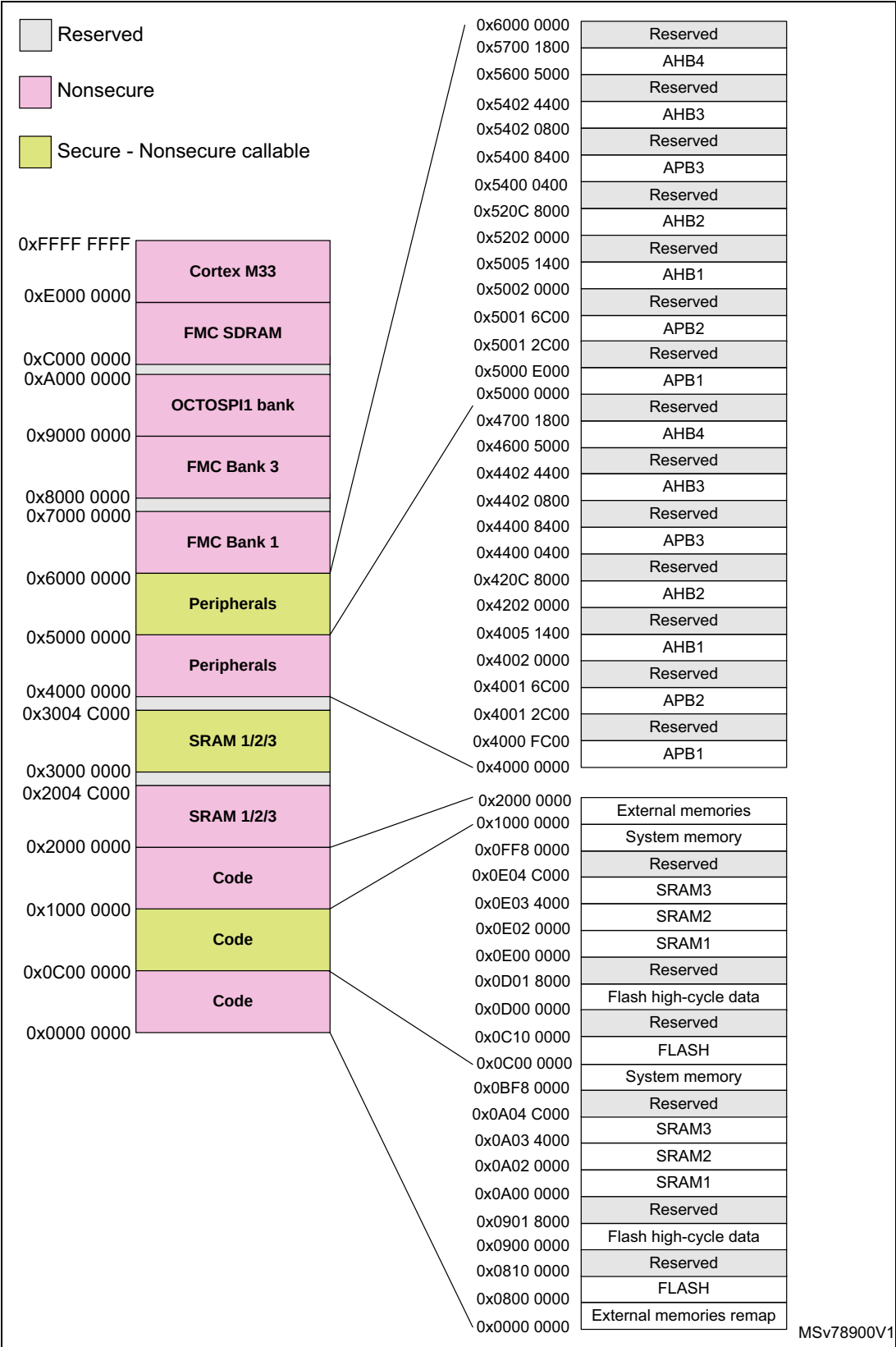


Figure 4. Memory map based on IDAU mapping (STM32H543/553xx devices)



MSv78900V1

All the memory map areas not allocated to on-chip memories and peripherals are reserved.
[Table 5](#) and [Table 6](#) give the boundary addresses of the available peripherals.

Table 5. Memory map and peripheral register addresses (STM32H562/563/573xx)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB4	0x5700 1800 - 0x57FF FFFF	0x4700 1800 - 0x47FF FFFF	Reserved	-
	0x5700 1400 - 0x5700 17FF	0x4700 1400 - 0x4700 17FF	OCTOSPI1	OCTOSPI
	0x5700 0800 - 0x5700 13FF	0x4700 0800 - 0x4700 13FF	Reserved	-
	0x5700 0400 - 0x5700 07FF	0x4700 0400 - 0x4700 07FF	FMC	FMC
	0x5600 F800 - 0x5700 03FF	0x4600 F800 - 0x4700 03FF	Reserved	-
	0x5600 F000 - 0x5600 F3FF	0x4600 F000 - 0x4600 F3FF	DLYBOS1	DLYB
	0x5600 9000 - 0x5600 EFFF	0x4600 9000 - 0x4600 EFFF	Reserved	-
	0x5600 8C00 - 0x5600 8FFF	0x4600 8C00 - 0x4600 8FFF	SDMMC2 ⁽¹⁾	SDMMC
	0x5600 8800 - 0x5600 8BFF	0x4600 8800 - 0x4600 8BFF	DLYBSD2	DLYB
	0x5600 8400 - 0x5600 87FF	0x4600 8400 - 0x4600 87FF	DLYBSD1	
	0x5600 8000 - 0x5600 83FF	0x4600 8000 - 0x4600 83FF	SDMMC1	SDMMC
	0x5600 5400 - 0x5600 7FFF	0x4600 5400 - 0x4600 7FFF	Reserved	-
	0x5600 5000 - 0x5600 53FF	0x4600 5000 - 0x4600 53FF	OTFDEC1	OTFDEC
	0x5600 0000 - 0x5600 4FFF	0x4600 0000 - 0x4600 4FFF	Reserved	-
AHB3	0x5402 4400 - 0x54FF FFFF	0x4402 4400 - 0x44FF FFFF	Reserved	-
	0x5402 4000 - 0x5402 43FF	0x4402 4000 - 0x4402 43FF	DEBUG	DEBUG
	0x5402 3000 - 0x5402 3FFF	0x4402 3000 - 0x4402 3FFF	Reserved	-
	0x5402 2000 - 0x5402 23FF	0x4402 2000 - 0x4402 23FF	EXTI	EXTI
	0x5402 1000 - 0x5402 1FFF	0x4402 1000 - 0x4402 1FFF	Reserved	-
	0x5402 0C00 - 0x5402 0FFF	0x4402 0C00 - 0x4402 0FFF	RCC	RCC
	0x5402 0800 - 0x5402 0BFF	0x4402 0800 - 0x4402 0BFF	PWR	PWR
	0x5402 0000 - 0x5402 07FF	0x4402 0000 - 0x4402 07FF	Reserved	-
APB3	0x5400 8000 - 0x5400 FFFF	0x4400 8000 - 0x4400 FFFF	Reserved	-
	0x5400 7C00 - 0x5400 7FFF	0x4400 7C00 - 0x4400 7FFF	TAMP	TAMP
	0x5400 7800 - 0x5400 7BFF	0x4400 7800 - 0x4400 7BFF	RTC	RTC
	0x5400 7400 - 0x5400 77FF	0x4400 7400 - 0x4400 77FF	VREFBUF	VREFBUF
	0x5400 5800 - 0x5400 73FF	0x4400 5800 - 0x4400 73FF	Reserved	-

Table 5. Memory map and peripheral register addresses (STM32H562/563/573xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
APB3 (continued)	0x5400 5400 - 0x5400 57FF	0x4400 5400 - 0x4400 57FF	LPTIM6	LPTIM
	0x5400 5000 - 0x5400 53FF	0x4400 5000 - 0x4400 53FF	LPTIM5	
	0x5400 4C00 - 0x5400 4FFF	0x4400 4C00 - 0x4400 4FFF	LPTIM4	
	0x5400 4800 - 0x5400 4BFF	0x4400 4800 - 0x4400 4BFF	LPTIM3	
	0x5400 4400 - 0x5400 47FF	0x4400 4400 - 0x4400 47FF	LPTIM1	
	0x5400 3000 - 0x5400 43FF	0x4400 3000 - 0x4400 43FF	Reserved	-
	0x5400 2C00 - 0x5400 2FFF	0x4400 2C00 - 0x4400 2FFF	I2C4	I2C
	0x5400 2800 - 0x5400 2BFF	0x4400 2800 - 0x4400 2BFF	I2C3	
	0x5400 2400 - 0x5400 27FF	0x4400 2400 - 0x4400 27FF	LPUART1	LPUART
	0x5400 2000 - 0x5400 23FF	0x4400 2000 - 0x4400 23FF	SPI5	SPI
	0x5400 0800 - 0x5400 1FFF	0x4400 0800 - 0x4400 1FFF	Reserved	-
	0x5400 0400 - 0x5400 07FF	0x4400 0400 - 0x4400 07FF	SBS	SBS
	0x5400 0000 - 0x5400 03FF	0x4400 0000 - 0x4400 03FF	Reserved	-
	0x520C 4000 - 0x53FF FFFF	0x420C 4000 - 0x43FF FFFF	Reserved	-
AHB2	0x520C 2000 - 0x520C 3FFF	0x420C 2000 - 0x420C 3FFF	PKA +RAM	PKA
	0x520C 1000 - 0x520C 1FFF	0x420C 1000 - 0x420C 1FFF	Reserved	-
	0x520C 0C00 - 0x520C 0FFF	0x420C 0C00 - 0x420C 0FFF	SAES	SAES
	0x520C 0800 - 0x520C 0BFF	0x420C 0800 - 0x420C 0BFF	RNG	RNG
	0x520C 0400 - 0x520C 07FF	0x420C 0400 - 0x420C 07FF	HASH	HASH
	0x520C 0000 - 0x520C 03FF	0x420C 0000 - 0x420C 03FF	AES	AES
	0x5202 C800 - 0x520B FFFF	0x4202 C800 - 0x420B FFFF	Reserved	-
	0x5202 C400 - 0x5202 C7FF	0x4202 C400 - 0x4202 C7FF	PSSI	PSSI
	0x5202 C000 - 0x5202 C3FF	0x4202 C000 - 0x4202 C3FF	DCMI	DCMI
	0x5202 8800 - 0x5202 BFFF	0x4202 8800 - 0x4202 BFFF	Reserved	-
	0x5202 8400 - 0x5202 87FF	0x4202 8400 - 0x4202 87FF	DAC1	DAC
	0x5202 8000 - 0x5202 83FF	0x4202 8000 - 0x4202 83FF	ADC1/ADC2	ADC
	0x5202 2400 - 0x5202 7FFF	0x4202 2400 - 0x4202 7FFF	Reserved	-

Table 5. Memory map and peripheral register addresses (STM32H562/563/573xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB2 (continued)	0x5202 2000 - 0x5202 23FF	0x4202 2000 - 0x4202 23FF	GPIOI	GPIO
	0x5202 1C00 - 0x5202 1FFF	0x4202 1C00 - 0x4202 1FFF	GPIOH	
	0x5202 1800 - 0x5202 1BFF	0x4202 1800 - 0x4202 1BFF	GPIOG	
	0x5202 1400 - 0x5202 17FF	0x4202 1400 - 0x4202 17FF	GPIOF	
	0x5202 1000 - 0x5202 13FF	0x4202 1000 - 0x4202 13FF	GPIOE	
	0x5202 0C00 - 0x5202 0FFF	0x4202 0C00 - 0x4202 0FFF	GPIOD	
	0x5202 0800 - 0x5202 0BFF	0x4202 0800 - 0x4202 0BFF	GPIOC	
	0x5202 0400 - 0x5202 07FF	0x4202 0400 - 0x4202 07FF	GPIOB	
	0x5202 0000 - 0x5202 03FF	0x4202 0000 - 0x4202 03FF	GPIOA	
AHB1	0x5003 7400 - 0x51FF FFFF	0x4003 7400 - 0x41FF FFFF	Reserved	-
	0x5003 6400 - 0x5003 73FF	0x4003 6400 - 0x4003 73FF	MPC_WM_BKPRAM	GTZC1 TZSC
	0x5003 3800 - 0x5003 63FF	0x4003 3C00 - 0x4003 63FF	Reserved	-
	0x5003 2400 - 0x5003 37FF	0x4003 2400 - 0x4003 37FF	GTZC1	GTZC1
	0x5003 1800 - 0x5003 23FF	0x4003 1800 - 0x4003 23FF	Reserved	-
	0x5003 1400 - 0x5003 17FF	0x4003 1400 - 0x4003 17FF	DCACHE	DCACHE
	0x5003 0800 - 0x5003 13FF	0x4003 0800 - 0x4003 13FF	Reserved	-
	0x5003 0400 - 0x5003 07FF	0x4003 0400 - 0x4003 07FF	ICACHE	ICACHE
	0x5002 9400 - 0x5003 03FF	0x4002 9400 - 0x4003 03FF	Reserved	-
	0x5002 8000 - 0x5002 93FF	0x4002 8000 - 0x4002 93FF	ETHERNET MAC ⁽¹⁾	ETHERNET
	0x5002 7000 - 0x5002 7FFF	0x4002 7000 - 0x4002 7FFF	Reserved	-
	0x5002 6000 - 0x5002 6FFF	0x4002 6000 - 0x4002 6FFF	RAMCFG	RAMCFG
	0x5002 4000 - 0x5002 5FFF	0x4002 4000 - 0x4002 5FFF	Reserved	-
	0x5002 3C00 - 0x5002 3FFF	0x4002 3C00 - 0x4002 3FFF	FMAC	FMAC
	0x5002 3800 - 0x5002 3BFF	0x4002 3800 - 0x4002 3BFF	CORDIC	CORDIC
	0x5002 3400 - 0x5002 37FF	0x4002 3400 - 0x4002 37FF	Reserved	-
	0x5002 3000 - 0x5002 33FF	0x4002 3000 - 0x4002 33FF	CRC	CRC
	0x5002 2400 - 0x5002 2FFF	0x4002 2400 - 0x4002 2FFF	Reserved	-
	0x5002 2000 - 0x5002 23FF	0x4002 2000 - 0x4002 23FF	FLASH	FLASH
	0x5002 1000 - 0x5002 1FFF	0x4002 1000 - 0x4002 1FFF	GDMA2	DMA
	0x5002 0000 - 0x5002 0FFF	0x4002 0000 - 0x4002 0FFF	GDMA1	

Table 5. Memory map and peripheral register addresses (STM32H562/563/573xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
APB2	0x5001 6C00 - 0x5001 FFFF	0x4001 6C00 - 0x4001 FFFF	Reserved	-
	0x5001 6400 - 0x5001 6BFF	0x4001 6400 - 0x4001 6BFF	USB_FS RAM	USB
	0x5001 6000 - 0x5001 63FF	0x4001 6000 - 0x4001 63FF	USB_FS	
	0x5001 5C00 - 0x5001 5FFF	0x4001 5C00 - 0x4001 5FFF	Reserved	-
	0x5001 5800 - 0x5001 5BFF	0x4001 5800 - 0x4001 5BFF	SAI2	SAI
	0x5001 5400 - 0x5001 57FF	0x4001 5400 - 0x4001 57FF	SAI1	
	0x5001 5000 - 0x5001 53FF	0x4001 5000 - 0x4001 53FF	SPI6	SPI
	0x5001 4C00 - 0x5001 4FFF	0x4001 4C00 - 0x4001 4FFF	SPI4	
	0x5001 4800 - 0x5001 4BFF	0x4001 4800 - 0x4001 4BFF	TIM17	TIM16/TIM17
	0x5001 4400 - 0x5001 47FF	0x4001 4400 - 0x4001 47FF	TIM16	
	0x5001 4000 - 0x5001 43FF	0x4001 4000 - 0x4001 43FF	TIM15	TIM15
	0x5001 3C00 - 0x5001 3FFF	0x4001 3C00 - 0x4001 3FFF	Reserved	-
	0x5001 3800 - 0x5001 3BFF	0x4001 3800 - 0x4001 3BFF	USART1	USART
	0x5001 3400 - 0x5001 37FF	0x4001 3400 - 0x4001 37FF	TIM8	TIMx
	0x5001 3000 - 0x5001 33FF	0x4001 3000 - 0x4001 33FF	SPI1/I2S1	SPI
	0x5001 2C00 - 0x5001 2FFF	0x4001 2C00 - 0x4001 2FFF	TIM1	TIMx
	0x5001 0000 - 0x5001 2BFF	0x4001 0000 - 0x4001 2BFF	Reserved	-
APB1	0x5000 E000 - 0x5000 FFFF	0x4000 E000 - 0x4000 FFFF	Reserved	-
	0x5000 DC00 - 0x5000 DFFF	0x4000 DC00 - 0x4000 DFFF	UCPD1	UCPD
	0x5000 B400 - 0x5000 DBFF	0x4000 B400 - 0x4000 DBFF	Reserved	-
	0x5000 AC00 - 0x5000 B3FF	0x4000 AC00 - 0x4000 B3FF	FDCAN SRAM	FDCAN
	0x5000 A800 - 0x5000 ABFF	0x4000 A800 - 0x4000 ABFF	FDCAN2 ⁽¹⁾	
	0x5000 A400 - 0x5000 A7FF	0x4000 A400 - 0x4000 A7FF	FDCAN1	
	0x5000 9800 - 0x5000 A3FF	0x4000 9800 - 0x4000 A3FF	Reserved	-
	0x5000 9400 - 0x5000 97FF	0x4000 9400 - 0x4000 97FF	LPTIM2	LPTIM
	0x5000 9000 - 0x5000 93FF	0x4000 9000 - 0x4000 93FF	Reserved	-
	0x5000 8C00 - 0x5000 8FFF	0x4000 8C00 - 0x4000 8FFF	DTS	DTS
	0x5000 8800 - 0x5000 8BFF	0x4000 8800 - 0x4000 8BFF	Reserved	-
	0x5000 8400 - 0x5000 87FF	0x4000 8400 - 0x4000 87FF	UART12	USART
	0x5000 8000 - 0x5000 83FF	0x4000 8000 - 0x4000 83FF	UART9	
	0x5000 7C00 - 0x5000 7FFF	0x4000 7C00 - 0x4000 7FFF	UART8	
	0x5000 7800 - 0x5000 7BFF	0x4000 7800 - 0x4000 7BFF	UART7	
	0x5000 7400 - 0x5000 77FF	0x4000 7400 - 0x4000 77FF	Reserved	-

Table 5. Memory map and peripheral register addresses (STM32H562/563/573xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
APB1 (continued)	0x5000 7000 - 0x5000 73FF	0x4000 7000 - 0x4000 73FF	HDMI-CEC	HDMI
	0x5000 6C00 - 0x5000 6FFF	0x4000 6C00 - 0x4000 6FFF	USART11	USART
	0x5000 6800 - 0x5000 6BFF	0x4000 6800 - 0x4000 6BFF	USART10	
	0x5000 6400 - 0x5000 67FF	0x4000 6400 - 0x4000 67FF	USART6	
	0x5000 6000 - 0x5000 63FF	0x4000 6000 - 0x4000 63FF	CRS	CRS
	0x5000 5C00 - 0x5000 5FFF	0x4000 5C00 - 0x4000 5FFF	I3C1	I3C
	0x5000 5800 - 0x5000 5BFF	0x4000 5800 - 0x4000 5BFF	I2C2	I2C
	0x5000 5400 - 0x5000 57FF	0x4000 5400 - 0x4000 57FF	I2C1	
	0x5000 5000 - 0x5000 53FF	0x4000 5000 - 0x4000 53FF	UART5	USART
	0x5000 4C00 - 0x5000 4FFF	0x4000 4C00 - 0x4000 4FFF	UART4	
	0x5000 4800 - 0x5000 4BFF	0x4000 4800 - 0x4000 4BFF	USART3	
	0x5000 4400 - 0x5000 47FF	0x4000 4400 - 0x4000 47FF	USART2	
	0x5000 4000 - 0x5000 43FF	0x4000 4000 - 0x4000 43FF	Reserved	-
	0x5000 3C00 - 0x5000 3FFF	0x4000 3C00 - 0x4000 3FFF	SPI3/I2S3	SPI
	0x5000 3800 - 0x5000 3BFF	0x4000 3800 - 0x4000 3BFF	SPI2/I2S2	
	0x5000 3400 - 0x5000 37FF	0x4000 3400 - 0x4000 37FF	Reserved	-
	0x5000 3000 - 0x5000 33FF	0x4000 3000 - 0x4000 33FF	IWDG	IWDG
	0x5000 2C00 - 0x5000 2FFF	0x4000 2C00 - 0x4000 2FFF	WWDG	WWDG
	0x5000 2400 - 0x5000 2BFF	0x4000 2400 - 0x4000 2BFF	Reserved	-
	0x5000 2000 - 0x5000 23FF	0x4000 2000 - 0x4000 23FF	TIM14	TIMx
	0x5000 1C00 - 0x5000 1FFF	0x4000 1C00 - 0x4000 1FFF	TIM13	
	0x5000 1800 - 0x5000 1BFF	0x4000 1800 - 0x4000 1BFF	TIM12	
	0x5000 1400 - 0x5000 17FF	0x4000 1400 - 0x4000 17FF	TIM7	
	0x5000 1000 - 0x5000 13FF	0x4000 1000 - 0x4000 13FF	TIM6	
	0x5000 0C00 - 0x5000 0FFF	0x4000 0C00 - 0x4000 0FFF	TIM5	
	0x5000 0800 - 0x5000 0BFF	0x4000 0800 - 0x4000 0BFF	TIM4	
	0x5000 0400 - 0x5000 07FF	0x4000 0400 - 0x4000 07FF	TIM3	
	0x5000 0000 - 0x5000 03FF	0x4000 0000 - 0x4000 03FF	TIM2	

1. Not available on STM32H562xx devices.

Table 6. Memory map and peripheral register addresses (STM32H523/533xx)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB4	0x5700 1800 - 0x57FF FFFF	0x4700 1800 - 0x47FF FFFF	Reserved	-
	0x5700 1400 - 0x5700 17FF	0x4700 1400 - 0x4700 17FF	OCTOSPI1	OCTOSPI
	0x5700 0800 - 0x5700 13FF	0x4700 0800 - 0x4700 13FF	Reserved	-
	0x5700 0400 - 0x5700 07FF	0x4700 0400 - 0x4700 07FF	FMC	FMC
	0x5600 F800 - 0x5700 03FF	0x4600 F800 - 0x4700 03FF	Reserved	-
	0x5600 F000 - 0x5600 F3FF	0x4600 F000 - 0x4600 F3FF	DLYBOS1	DLYB
	0x5600 8800 - 0x5600 EFFF	0x4600 8800 - 0x4600 EFFF	Reserved	-
	0x5600 8400 - 0x5600 87FF	0x4600 8400 - 0x4600 87FF	DLYBSD1	DLYB
	0x5600 8000 - 0x5600 83FF	0x4600 8000 - 0x4600 83FF	SDMMC1	SDMMC
	0x5600 5400 - 0x5600 7FFF	0x4600 5400 - 0x4600 7FFF	Reserved	-
	0x5600 5000 - 0x5600 53FF	0x4600 5000 - 0x4600 53FF	OTFDEC1	OTFDEC
	0x5600 0000 - 0x5600 4FFF	0x4600 0000 - 0x4600 4FFF	Reserved	-
AHB3	0x5402 4400 - 0x54FF FFFF	0x4402 4400 - 0x44FF FFFF	Reserved	-
	0x5402 4000 - 0x5402 43FF	0x4402 4000 - 0x4402 43FF	DEBUG	DEBUG
	0x5402 3000 - 0x5402 3FFF	0x4402 3000 - 0x4402 3FFF	Reserved	-
	0x5402 2000 - 0x5402 23FF	0x4402 2000 - 0x4402 23FF	EXTI	EXTI
	0x5402 1000 - 0x5402 1FFF	0x4402 1000 - 0x4402 1FFF	Reserved	-
	0x5402 0C00 - 0x5402 0FFF	0x4402 0C00 - 0x4402 0FFF	RCC	RCC
	0x5402 0800 - 0x5402 0BFF	0x4402 0800 - 0x4402 0BFF	PWR	PWR
	0x5402 0000 - 0x5402 07FF	0x4402 0000 - 0x4402 07FF	Reserved	-
APB3	0x5400 8000 - 0x5400 FFFF	0x4400 8000 - 0x4400 FFFF	Reserved	-
	0x5400 7C00 - 0x5400 7FFF	0x4400 7C00 - 0x4400 7FFF	TAMP	TAMP
	0x5400 7800 - 0x5400 7BFF	0x4400 7800 - 0x4400 7BFF	RTC	RTC
	0x5400 7400 - 0x5400 77FF	0x4400 7400 - 0x4400 77FF	VREFBUF	VREFBUF
	0x5400 4800 - 0x5400 73FF	0x4400 4800 - 0x4400 73FF	Reserved	-
	0x5400 4400 - 0x5400 47FF	0x4400 4400 - 0x4400 47FF	LPTIM1	LPTIM
	0x5400 3400 - 0x5400 43FF	0x4400 3400 - 0x4400 43FF	Reserved	-
	0x5400 3000 - 0x5400 33FF	0x4400 3000 - 0x4400 33FF	I3C2	I3C
	0x5400 2C00 - 0x5400 2FFF	0x4400 2C00 - 0x4400 2FFF	Reserved	-
	0x5400 2800 - 0x5400 2BFF	0x4400 2800 - 0x4400 2BFF	I2C3	I2C
	0x5400 2400 - 0x5400 27FF	0x4400 2400 - 0x4400 27FF	LPUART1	LPUART
	0x5400 0800 - 0x5400 23FF	0x4400 0800 - 0x4400 23FF	Reserved	-
	0x5400 0400 - 0x5400 07FF	0x4400 0400 - 0x4400 07FF	SBS	SBS
	0x5400 0000 - 0x5400 03FF	0x4400 0000 - 0x4400 03FF	Reserved	-

Table 6. Memory map and peripheral register addresses (STM32H523/533xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB2	0x520C 4000 - 0x53FF FFFF	0x420C 4000 - 0x43FF FFFF	Reserved	-
	0x520C 2000 - 0x520C 3FFF	0x420C 2000 - 0x420C 3FFF	PKA + RAM	PKA
	0x520C 1000 - 0x520C 1FFF	0x420C 1000 - 0x420C 1FFF	Reserved	-
	0x520C 0C00 - 0x520C 0FFF	0x420C 0C00 - 0x420C 0FFF	SAES	SAES
	0x520C 0800 - 0x520C 0BFF	0x420C 0800 - 0x420C 0BFF	RNG	RNG
	0x520C 0400 - 0x520C 07FF	0x420C 0400 - 0x420C 07FF	HASH	HASH
	0x520C 0000 - 0x520C 03FF	0x420C 0000 - 0x420C 03FF	AES	AES
	0x5202 C800 - 0x520B FFFF	0x4202 C800 - 0x420B FFFF	Reserved	-
	0x5202 C400 - 0x5202 C7FF	0x4202 C400 - 0x4202 C7FF	PSSI	PSSI
	0x5202 C000 - 0x5202 C3FF	0x4202 C000 - 0x4202 C3FF	DCMI	DCMI
	0x5202 8800 - 0x5202 BFFF	0x4202 8800 - 0x4202 BFFF	Reserved	-
	0x5202 8400 - 0x5202 87FF	0x4202 8400 - 0x4202 87FF	DAC1	DAC
	0x5202 8000 - 0x5202 83FF	0x4202 8000 - 0x4202 83FF	ADC1/ADC2	ADC
	0x5202 2000 - 0x5202 7FFF	0x4202 2000 - 0x4202 7FFF	Reserved	-
	0x5202 1C00 - 0x5202 1FFF	0x4202 1C00 - 0x4202 1FFF	GPIOH	GPIO
	0x5202 1800 - 0x5202 1BFF	0x4202 1800 - 0x4202 1BFF	GPIOG	
	0x5202 1400 - 0x5202 17FF	0x4202 1400 - 0x4202 17FF	GPIOF	
	0x5202 1000 - 0x5202 13FF	0x4202 1000 - 0x4202 13FF	GPIOE	
	0x5202 0C00 - 0x5202 0FFF	0x4202 0C00 - 0x4202 0FFF	GPIOD	
	0x5202 0800 - 0x5202 0BFF	0x4202 0800 - 0x4202 0BFF	GPIOC	
	0x5202 0400 - 0x5202 07FF	0x4202 0400 - 0x4202 07FF	GPIOB	
	0x5202 0000 - 0x5202 03FF	0x4202 0000 - 0x4202 03FF	GPIOA	

Table 6. Memory map and peripheral register addresses (STM32H523/533xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB1	0x5003 6C00 - 0x51FF FFFF	0x4003 6C00 - 0x41FF FFFF	Reserved	-
	0x5003 6400 - 0x5003 6BFF	0x4003 6400 - 0x4003 6BFF	MPC_WM_BKPRAM	GTZC1 TZSC
	0x5003 3800 - 0x5003 63FF	0x4003 3C00 - 0x4003 63FF	Reserved	-
	0x5003 2400 - 0x5003 37FF	0x4003 2400 - 0x4003 37FF	GTZC1	GTZC1
	0x5003 1800 - 0x5003 23FF	0x4003 1800 - 0x4003 23FF	Reserved	-
	0x5003 1400 - 0x5003 17FF	0x4003 1400 - 0x4003 17FF	DCACHE	DCACHE
	0x5003 0800 - 0x5003 13FF	0x4003 0800 - 0x4003 13FF	Reserved	-
	0x5003 0400 - 0x5003 07FF	0x4003 0400 - 0x4003 07FF	ICACHE	ICACHE
	0x5002 7000 - 0x5003 03FF	0x4002 7000 - 0x4003 03FF	Reserved	-
	0x5002 6000 - 0x5002 6FFF	0x4002 6000 - 0x4002 6FFF	RAMCFG	RAMCFG
	0x5002 3400 - 0x5002 5FFF	0x4002 3400 - 0x4002 5FFF	Reserved	-
	0x5002 3000 - 0x5002 33FF	0x4002 3000 - 0x4002 33FF	CRC	CRC
	0x5002 2400 - 0x5002 2FFF	0x4002 2400 - 0x4002 2FFF	Reserved	-
	0x5002 2000 - 0x5002 23FF	0x4002 2000 - 0x4002 23FF	FLASH	FLASH
	0x5002 1000 - 0x5002 1FFF	0x4002 1000 - 0x4002 1FFF	GDMA2	DMA
	0x5002 0000 - 0x5002 0FFF	0x4002 0000 - 0x4002 0FFF	GDMA1	
APB2	0x5001 6C00 - 0x5001 FFFF	0x4001 6C00 - 0x4001 FFFF	Reserved	-
	0x5001 6400 - 0x5001 6BFF	0x4001 6400 - 0x4001 6BFF	USB_FS RAM	USB
	0x5001 6000 - 0x5001 63FF	0x4001 6000 - 0x4001 63FF	USB_FS	
	0x5001 5000 - 0x5001 5FFF	0x4001 5000 - 0x4001 5FFF	Reserved	-
	0x5001 4C00 - 0x5001 4FFF	0x4001 4C00 - 0x4001 4FFF	SPI4	SPI
	0x5001 4400 - 0x5001 4BFF	0x4001 4400 - 0x4001 4BFF	Reserved	-
	0x5001 4000 - 0x5001 43FF	0x4001 4000 - 0x4001 43FF	TIM15	TIM15
	0x5001 3C00 - 0x5001 3FFF	0x4001 3C00 - 0x4001 3FFF	Reserved	-
	0x5001 3800 - 0x5001 3BFF	0x4001 3800 - 0x4001 3BFF	USART1	USART
	0x5001 3400 - 0x5001 37FF	0x4001 3400 - 0x4001 37FF	TIM8	TIMx
	0x5001 3000 - 0x5001 33FF	0x4001 3000 - 0x4001 33FF	SPI1/I2S1	SPI
	0x5001 2C00 - 0x5001 2FFF	0x4001 2C00 - 0x4001 2FFF	TIM1	TIMx
	0x5001 0000 - 0x5001 2BFF	0x4001 0000 - 0x4001 2BFF	Reserved	-

Table 6. Memory map and peripheral register addresses (STM32H523/533xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
APB1	0x5000 E000 - 0x5000 FFFF	0x4000 E000 - 0x4000 FFFF	Reserved	-
	0x5000 DC00 - 0x5000 DFFF	0x4000 DC00 - 0x4000 DFFF	UCPD1	UCPD
	0x5000 B400 - 0x5000 DBFF	0x4000 B400 - 0x4000 DBFF	Reserved	-
	0x5000 AC00 - 0x5000 B3FF	0x4000 AC00 - 0x4000 B3FF	FDCAN SRAM	FDCAN
	0x5000 A800 - 0x5000 ABFF	0x4000 A800 - 0x4000 ABFF	FDCAN2	
	0x5000 A400 - 0x5000 A7FF	0x4000 A400 - 0x4000 A7FF	FDCAN1	
	0x5000 9800 - 0x5000 A3FF	0x4000 9800 - 0x4000 A3FF	Reserved	-
	0x5000 9400 - 0x5000 97FF	0x4000 9400 - 0x4000 97FF	LPTIM2	LPTIM
	0x5000 9000 - 0x5000 93FF	0x4000 9000 - 0x4000 93FF	Reserved	-
	0x5000 8C00 - 0x5000 8FFF	0x4000 8C00 - 0x4000 8FFF	DTS	DTS
	0x5000 7400 - 0x5000 8BFF	0x4000 7400 - 0x4000 8BFF	Reserved	-
	0x5000 7000 - 0x5000 73FF	0x4000 7000 - 0x4000 73FF	HDMI-CEC	HDMI
	0x5000 6800 - 0x5000 6FFF	0x4000 6800 - 0x4000 6FFF	Reserved	-
	0x5000 6400 - 0x5000 67FF	0x4000 6400 - 0x4000 67FF	USART6	USART
	0x5000 6000 - 0x5000 63FF	0x4000 6000 - 0x4000 63FF	CRS	CRS
	0x5000 5C00 - 0x5000 5FFF	0x4000 5C00 - 0x4000 5FFF	I3C1	I3C
	0x5000 5800 - 0x5000 5BFF	0x4000 5800 - 0x4000 5BFF	I2C2	I2C
	0x5000 5400 - 0x5000 57FF	0x4000 5400 - 0x4000 57FF	I2C1	
	0x5000 5000 - 0x5000 53FF	0x4000 5000 - 0x4000 53FF	UART5	USART
	0x5000 4C00 - 0x5000 4FFF	0x4000 4C00 - 0x4000 4FFF	UART4	
	0x5000 4800 - 0x5000 4BFF	0x4000 4800 - 0x4000 4BFF	USART3	
	0x5000 4400 - 0x5000 47FF	0x4000 4400 - 0x4000 47FF	USART2	
	0x5000 4000 - 0x5000 43FF	0x4000 4000 - 0x4000 43FF	Reserved	-
	0x5000 3C00 - 0x5000 3FFF	0x4000 3C00 - 0x4000 3FFF	SPI3/I2S3	SPI
	0x5000 3800 - 0x5000 3BFF	0x4000 3800 - 0x4000 3BFF	SPI2/I2S2	
	0x5000 3400 - 0x5000 37FF	0x4000 3400 - 0x4000 37FF	Reserved	-
	0x5000 3000 - 0x5000 33FF	0x4000 3000 - 0x4000 33FF	IWDG	IWDG
	0x5000 2C00 - 0x5000 2FFF	0x4000 2C00 - 0x4000 2FFF	WWDG	WWDG
	0x5000 1C00 - 0x5000 2BFF	0x4000 1C00 - 0x4000 2BFF	Reserved	-
	0x5000 1800 - 0x5000 1BFF	0x4000 1800 - 0x4000 1BFF	TIM12	TIMx
	0x5000 1400 - 0x5000 17FF	0x4000 1400 - 0x4000 17FF	TIM7	
	0x5000 1000 - 0x5000 13FF	0x4000 1000 - 0x4000 13FF	TIM6	
	0x5000 0C00 - 0x5000 0FFF	0x4000 0C00 - 0x4000 0FFF	TIM5	
	0x5000 0800 - 0x5000 0BFF	0x4000 0800 - 0x4000 0BFF	TIM4	
	0x5000 0400 - 0x5000 07FF	0x4000 0400 - 0x4000 07FF	TIM3	
	0x5000 0000 - 0x5000 03FF	0x4000 0000 - 0x4000 03FF	TIM2	

Table 7. Memory map and peripheral register addresses (STM32H543/553xx)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB4	0x5700 1800 - 0x5FFF FFFF	0x4700 1800 - 0x4FFF FFFF	Reserved	-
	0x5700 1400 - 0x5700 17FF	0x4700 1400 - 0x4700 17FF	OCTOSPI1	OCTOSPI
	0x5700 0800 - 0x5700 13FF	0x4700 0800 - 0x4700 13FF	Reserved	-
	0x5700 0400 - 0x5700 07FF	0x4700 0400 - 0x4700 07FF	FMC	FMC
	0x5600 F400 - 0x5700 03FF	0x4600 F400 - 0x4700 03FF	Reserved	-
	0x5600 F000 - 0x5600 F3FF	0x4600 F000 - 0x4600 F3FF	DLYBOS1	DLYB
	0x5600 8800 - 0x5600 EFFF	0x4600 8800 - 0x4600 EFFF	Reserved	-
	0x5600 8400 - 0x5600 87FF	0x4600 8400 - 0x4600 87FF	DLYBSD1	DLYB
	0x5600 8000 - 0x5600 83FF	0x4600 8000 - 0x4600 83FF	SDMMC1	SDMMC
	0x5600 5400 - 0x5600 7FFF	0x4600 5400 - 0x4600 7FFF	Reserved	-
	0x5600 5000 - 0x5600 53FF	0x4600 5000 - 0x4600 53FF	OTFDEC1	OTFDEC
	0x5600 0000 - 0x5600 4FFF	0x4600 0000 - 0x4600 4FFF	Reserved	-
	0x5402 4400 - 0x54FF FFFF	0x4402 4400 - 0x44FF FFFF	Reserved	-
AHB3	0x5402 4000 - 0x5402 43FF	0x4402 4000 - 0x4402 43FF	DEBUG	DEBUG
	0x5402 2400 - 0x5402 3FFF	0x4402 2400 - 0x4402 3FFF	Reserved	-
	0x5402 2000 - 0x5402 23FF	0x4402 2000 - 0x4402 23FF	EXTI	EXTI
	0x5402 1000 - 0x5402 1FFF	0x4402 1000 - 0x4402 1FFF	Reserved	-
	0x5402 0C00 - 0x5402 0FFF	0x4402 0C00 - 0x4402 0FFF	RCC	RCC
	0x5402 0800 - 0x5402 0BFF	0x4402 0800 - 0x4402 0BFF	PWR	PWR
	0x5402 0000 - 0x5402 07FF	0x4402 0000 - 0x4402 07FF	Reserved	-
	0x5400 8400 - 0x5400 FFFF	0x4400 8400 - 0x4400 FFFF	Reserved	-
APB3	0x5400 8000 - 0x5400 83FF	0x4400 8000 - 0x4400 83FF	PLAY	PLAY
	0x5400 7C00 - 0x5400 7FFF	0x4400 7C00 - 0x4400 7FFF	TAMP	TAMP
	0x5400 7800 - 0x5400 7BFF	0x4400 7800 - 0x4400 7BFF	RTC	RTC
	0x5400 7400 - 0x5400 77FF	0x4400 7400 - 0x4400 77FF	VREFBUF	VREFBUF
	0x5400 4800 - 0x5400 73FF	0x4400 4800 - 0x4400 73FF	Reserved	-
	0x5400 4400 - 0x5400 47FF	0x4400 4400 - 0x4400 47FF	LPTIM1	LPTIM1
	0x5400 3400 - 0x5400 43FF	0x4400 3400 - 0x4400 43FF	Reserved	-
	0x5400 3000 - 0x5400 33FF	0x4400 3000 - 0x4400 33FF	I3C2	I3C
	0x5400 2C00 - 0x5400 2FFF	0x4400 2C00 - 0x4400 2FFF	Reserved	-
	0x5400 2800 - 0x5400 2BFF	0x4400 2800 - 0x4400 2BFF	I2C3	I2C
	0x5400 2400 - 0x5400 27FF	0x4400 2400 - 0x4400 27FF	LPUART1	LPUART
	0x5400 0800 - 0x5400 23FF	0x4400 0800 - 0x4400 23FF	Reserved	-
	0x5400 0400 - 0x5400 07FF	0x4400 0400 - 0x4400 07FF	SBS	SBS
	0x5400 0000 - 0x5400 03FF	0x4400 0000 - 0x4400 03FF	Reserved	-

Table 7. Memory map and peripheral register addresses (STM32H543/553xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB2	0x520C 8000 - 0x53FF FFFF	0x420C 8000 - 0x43FF FFFF	Reserved	-
	0x520C 7C00 - 0x520C 7FFF	0x420C 7C00 - 0x420C 7FFF	CCB	CCB
	0x520C 4000 - 0x520C 7BFF	0x420C 4000 - 0x420C 7BFF	Reserved	-
	0x520C 2000 - 0x520C 3FFF	0x420C 2000 - 0x420C 3FFF	PKA + RAM	PKA
	0x520C 1000 - 0x520C 1FFF	0x420C 1000 - 0x420C 1FFF	Reserved	-
	0x520C 0C00 - 0x520C 0FFF	0x420C 0C00 - 0x420C 0FFF	SAES	SAES
	0x520C 0800 - 0x520C 0BFF	0x420C 0800 - 0x420C 0BFF	RNG	RNG
	0x520C 0400 - 0x520C 07FF	0x420C 0400 - 0x420C 07FF	HASH	HASH
	0x520C 0000 - 0x520C 03FF	0x420C 0000 - 0x420C 03FF	AES	AES
	0x5202 DC00 - 0x520B FFFF	0x4202 DC00 - 0x420B FFFF	Reserved	-
	0x5202 D800 - 0x5202 DBFF	0x4202 D800 - 0x4202 DBFF	ADC3	ADC
	0x5202 8800 - 0x5202 D7FF	0x4202 8800 - 0x4202 D7FF	Reserved	-
	0x5202 8400 - 0x5202 87FF	0x4202 8400 - 0x4202 87FF	DAC1	DAC
	0x5202 8000 - 0x5202 83FF	0x4202 8000 - 0x4202 83FF	ADC1/ADC2	ADC
	0x5202 2000 - 0x5202 7FFF	0x4202 2000 - 0x4202 7FFF	Reserved	-
	0x5202 1C00 - 0x5202 1FFF	0x4202 1C00 - 0x4202 1FFF	GPIOH	GPIO
	0x5202 1800 - 0x5202 1BFF	0x4202 1800 - 0x4202 1BFF	GPIOG	
	0x5202 1400 - 0x5202 17FF	0x4202 1400 - 0x4202 17FF	GPIOF	
	0x5202 1000 - 0x5202 13FF	0x4202 1000 - 0x4202 13FF	GPIOE	
	0x5202 0C00 - 0x5202 0FFF	0x4202 0C00 - 0x4202 0FFF	GIOD	
	0x5202 0800 - 0x5202 0BFF	0x4202 0800 - 0x4202 0BFF	GPIOC	
	0x5202 0400 - 0x5202 07FF	0x4202 0400 - 0x4202 07FF	GPIOB	
	0x5202 0000 - 0x5202 03FF	0x4202 0000 - 0x4202 03FF	GPIOA	
	0x5003 6C00 - 0x51FF FFFF	0x4003 6C00 - 0x41FF FFFF	Reserved	-
AHB1	0x5003 6400 - 0x4003 6BFF	0x4003 6400 - 0x4003 6BFF	BKPSRAM	BKPSRAM
	0x5003 3800 - 0x5003 63FF	0x4002 3800 - 0x4003 63FF	Reserved	-
	0x5003 2400 - 0x5003 37FF	0x4003 2400 - 0x4003 37FF	GTZC1	GTZC1
	0x5003 1800 - 0x5003 23FF	0x4003 1800 - 0x4003 23FF	Reserved	-
	0x5003 1400 - 0x5003 17FF	0x4003 1400 - 0x4003 17FF	DCACHE	DCACHE
	0x5003 0800 - 0x5003 13FF	0x4003 0800 - 0x4003 13FF	Reserved	-
	0x5003 0400 - 0x5003 07FF	0x4003 0400 - 0x4003 07FF	ICACHE	ICACHE
	0x5002 9400 - 0x5003 03FF	0x4002 9400 - 0x4003 03FF	Reserved	-
	0x5002 8000 - 0x5002 93FF	0x4002 8000 - 0x4002 93FF	MAC ETHERNET	ETHERNET

Table 7. Memory map and peripheral register addresses (STM32H543/553xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
AHB1 (continued)	0x5002 7000 - 0x5002 7FFF	0x4002 7000 - 0x4002 7FFF	Reserved	-
	0x5002 6000 - 0x5002 6FFF	0x4002 6000 - 0x4002 6FFF	RAMCFG	RAMCFG
	0x5002 3C00 - 0x5002 5FFF	0x4002 3C00 - 0x4002 5FFF	Reserved	-
	0x5002 3800 - 0x5002 3BFF	0x4002 3800 - 0x4002 3BFF	CORDIC	CORDIC
	0x5002 3400 - 0x5002 37FF	0x4002 3400 - 0x4002 37FF	Reserved	-
	0x5002 3000 - 0x5002 33FF	0x4002 3000 - 0x4002 33FF	CRC	CRC
	0x5002 2400 - 0x5002 2FFF	0x4002 2400 - 0x4002 2FFF	Reserved	-
	0x5002 2000 - 0x5002 23FF	0x4002 2000 - 0x4002 23FF	FLASH	FLASH
	0x5002 1000 - 0x5002 1FFF	0x4002 1000 - 0x4002 1FFF	GDMA2	DMA
	0x5002 0000 - 0x5002 0FFF	0x4002 0000 - 0x4002 0FFF	GDMA1	
APB2	0x5001 6C00 - 0x5001 FFFF	0x4001 6C00 - 0x4001 FFFF	Reserved	-
	0x5001 6400 - 0x5001 6BFF	0x4001 6400 - 0x4001 6BFF	USB_FS RAM	USB
	0x5001 6000 - 0x5001 63FF	0x4001 6000 - 0x4001 63FF	USB_FS	
	0x5001 5000 - 0x5001 5FFF	0x4001 5000 - 0x4001 5FFF	Reserved	-
	0x5001 4C00 - 0x5001 4FFF	0x4001 4C00 - 0x4001 4FFF	SPI4	SPI
	0x5001 4400 - 0x5001 4BFF	0x4001 4400 - 0x4001 4BFF	Reserved	-
	0x5001 4000 - 0x5001 43FF	0x4001 4000 - 0x4001 43FF	TIM15	TIMx
	0x5001 3C00 - 0x5001 3FFF	0x4001 3C00 - 0x4001 3FFF	Reserved	-
	0x5001 3800 - 0x5001 3BFF	0x4001 3800 - 0x4001 3BFF	USART1	USART
	0x5001 3400 - 0x5001 37FF	0x4001 3400 - 0x4001 37FF	TIM8	TIMx
	0x5001 3000 - 0x5001 33FF	0x4001 3000 - 0x4001 33FF	SPI1	SPI
	0x5001 2C00 - 0x5001 2FFF	0x4001 2C00 - 0x4001 2FFF	TIM1	TIMx
	0x5001 0000 - 0x5001 2BFF	0x4001 0000 - 0x4001 2BFF	Reserved	-
	0x5000 E000 - 0x5000 FFFF	0x4000 E000 - 0x4000 FFFF	Reserved	-
APB1	0x5000 DC00 - 0x5000 DFFF	0x4000 DC00 - 0x4000 DFFF	UCPD1	UCPD
	0x5000 B400 - 0x5000 DBFF	0x4000 B400 - 0x4000 DBFF	Reserved	-
	0x5000 AC00 - 0x5000 B3FF	0x4000 AC00 - 0x4000 B3FF	FDCAN SRAM	FDCAN
	0x5000 A800 - 0x5000 ABFF	0x4000 A800 - 0x4000 ABFF	FDCAN2	
	0x5000 A400 - 0x5000 A7FF	0x4000 A400 - 0x4000 A7FF	FDCAN1	
	0x5000 9800 - 0x5000 A3FF	0x4000 9800 - 0x4000 A3FF	Reserved	-
	0x5000 9400 - 0x5000 97FF	0x4000 9400 - 0x4000 97FF	LPTIM2	LPTIM
	0x5000 9000 - 0x5000 93FF	0x4000 9000 - 0x4000 93FF	Reserved	-
	0x5000 8C00 - 0x5000 8FFF	0x4000 8C00 - 0x4000 8FFF	DTS	DTS

Table 7. Memory map and peripheral register addresses (STM32H543/553xx) (continued)

Bus	Secure boundary address	Nonsecure boundary address	Peripheral	Register map
APB1 (continued)	0x5000 8000 - 0x5000 8BFF	0x4000 8000 - 0x4000 8BFF	Reserved	-
	0x5000 7C00 - 0x5000 7FFF	0x4000 7C00 - 0x4000 7FFF	UART8	USART
	0x5000 7800 - 0x5000 7BFF	0x4000 7800 - 0x4000 7BFF	UART7	
	0x5000 7400 - 0x5000 77FF	0x4000 7400 - 0x4000 77FF	Reserved	-
	0x5000 7000 - 0x5000 73FF	0x4000 7000 - 0x4000 73FF	HDMI-CEC	HDMI
	0x5000 6800 - 0x5000 6FFF	0x4000 6800 - 0x4000 6FFF	Reserved	-
	0x5000 6400 - 0x5000 67FF	0x4000 6400 - 0x4000 67FF	USART6	USART
	0x5000 6000 - 0x5000 63FF	0x4000 6000 - 0x4000 63FF	CRS	CRS
	0x5000 5C00 - 0x5000 5FFF	0x4000 5C00 - 0x4000 5FFF	I3C1	I3C
	0x5000 5800 - 0x5000 5BFF	0x4000 5800 - 0x4000 5BFF	I2C2	I2C
	0x5000 5400 - 0x5000 57FF	0x4000 5400 - 0x4000 57FF	I2C1	
	0x5000 5000 - 0x5000 53FF	0x4000 5000 - 0x4000 53FF	UART5	USART
	0x5000 4C00 - 0x5000 4FFF	0x4000 4C00 - 0x4000 4FFF	UART4	
	0x5000 4800 - 0x5000 4BFF	0x4000 4800 - 0x4000 4BFF	USART3	
	0x5000 4400 - 0x5000 47FF	0x4000 4400 - 0x4000 47FF	USART2	
	0x5000 4000 - 0x5000 43FF	0x4000 4000 - 0x4000 43FF	COMP1/COMP2	COMP
	0x5000 3C00 - 0x5000 3FFF	0x4000 3C00 - 0x4000 3FFF	SPI3	SPI
	0x5000 3800 - 0x5000 3BFF	0x4000 3800 - 0x4000 3BFF	SPI2	
	0x5000 3400 - 0x5000 37FF	0x4000 3400 - 0x4000 37FF	Reserved	-
	0x5000 3000 - 0x5000 33FF	0x4000 3000 - 0x4000 33FF	IWDG	IWDG
	0x5000 2C00 - 0x5000 2FFF	0x4000 2C00 - 0x4000 2FFF	WWDG	WWDG
	0x5000 1C00 - 0x5000 2BFF	0x4000 1C00 - 0x4000 2BFF	Reserved	-
	0x5000 1800 - 0x5000 1BFF	0x4000 1800 - 0x4000 1BFF	TIM12	TIMx
	0x5000 1400 - 0x5000 17FF	0x4000 1400 - 0x4000 17FF	TIM7	
	0x5000 1000 - 0x5000 13FF	0x4000 1000 - 0x4000 13FF	TIM6	
	0x5000 0C00 - 0x5000 0FFF	0x4000 0C00 - 0x4000 0FFF	TIM5	
	0x5000 0800 - 0x5000 0BFF	0x4000 0800 - 0x4000 0BFF	TIM4	
	0x5000 0400 - 0x5000 07FF	0x4000 0400 - 0x4000 07FF	TIM3	
	0x5000 0000 - 0x5000 03FF	0x4000 0000 - 0x4000 03FF	TIM2	

2.3.3 Embedded SRAMs

The STM32H562/563/573xx devices feature up to 644-Kbyte SRAMs:

- 256-Kbyte SRAM1
- 64-Kbyte SRAM2 with ECC
- 320-Kbyte SRAM3 with optional ECC
- 4-Kbyte BKPSRAM with ECC

The STM32H543/553xx devices feature up to 306-Kbyte SRAMs:

- 128-Kbyte SRAM1
- 80-Kbyte SRAM2 with ECC
- 96-Kbyte SRAM3 with optional ECC
- 2-Kbyte BKPSRAM with ECC

The STM32H523/533xx devices feature up to 274-Kbyte SRAMs:

- 128-Kbyte SRAM1
- 80-Kbyte SRAM2 with ECC
- 64-Kbyte SRAM3
- 2-Kbyte BKPSRAM with ECC

The SRAMs can be accessed by byte, half-word (16 bits), or full word (32 bits).

The memories can be addressed both by CPU and DMAs. The CPU can access the SRAM1, SRAM2, and SRAM3 through the system bus or through the C-bus, depending on the selected address. The CPU can access the BKPSRAM only through the system bus.

When the TrustZone security is enabled, all SRAMs are secure after reset. The SRAMs can be programmed as nonsecure with a block granularity. For more details, refer to [Section 5: Global TrustZone® controller \(GTZC\)](#).

SRAM features are detailed in [Section 6.3.1: Internal SRAMs features](#).

2.3.4 Flash memory overview

The flash memory is composed of two distinct physical areas:

- The main memory block, which contains the application program and user data.
- The information block, composed of the following parts:
 - option bytes for hardware and memory protection user configuration
 - system memory, with the ST proprietary code
 - OTP (one-time programmable) area

The flash interface implements instruction and data access based on the AHB protocol. It also implements the logic necessary to carry out the program/erase operations controlled through the FLASH registers plus security access control features. Refer to [Section 7: Embedded flash memory \(FLASH\)](#) for more details.

2.3.5 Boot modes

At startup, the core jumps to the boot address configured through the BOOT0 pin, the BOOT_ADDR option bytes, the TZEN option bit, debug request and the product state.

If BOOT_ADDR is not yet configured, dedicated libraries (programmed during ST production), can be used for secure boot. They are located in system flash memory:

- ST libraries in system flash memory assist the application software boot with special features such as secure boot and secure firmware install (SFI-RSS).
- STiRoT (immutable root of trust) secure software in user flash memory are used for secure firmware update and provisioning (SFU).

If a debugger is attached to the product, the entry point is a debug authentication policy, used to unlock the device when attached to debugger. Digital signature must be provided to perform a regression to product state, where debug is allowed.

Embedded bootloader

The embedded bootloader is located in the system memory, programmed by ST during production. Refer to AN2606 *“Introduction to system memory boot mode on STM32 MCUs”*.

3 System security

The STM32H5 devices are designed with a comprehensive set of security features, some of them based on the standard Arm TrustZone® technology.

These security features simplify the process of evaluating IoT devices against security standards. They also significantly reduce the cost and complexity of software development for OEM and third-party developers, by facilitating the reuse, improving the interoperability, and minimizing the API fragmentation.

This section explains the different security features available on the devices.

3.1 Key security features

- Resource isolation using privilege mode and Armv8-M mainline security extension of Cortex-M33, extended to securable I/Os, memories, and peripherals
- Secure firmware installation (SFI) with device unique cryptographic key pair
 - Leveraging the on-chip immutable bootloader that supports the image download through USART, USB, I²C, I³C, SPI, FDCAN, and JTAG
- Boot entry: the platform makes it possible to select between native immutable root of trust or proprietary boot entry (in user flash memory).
- Security services (in system flash): the platform comes with native security services, embedded in the system memory to manage the root of trust services. Native root of trust services takes care of: platform security, including secure boot, secure updates of next boot level (uROT: updatable root of trust), secure debug control (debug reopening, regression control). The services can be personalized for each OEM, using the provisioning tools.
- Temporal isolation: boot levels are isolated thanks to HDPL (hide protect level) monotonic counter.
- Secure storage, featuring:
 - Five nonvolatile areas dedicated to secure storage, protected with HDPL and TrustZone.
 - Battery-powered volatile secure storage, automatically erased in case of tamper
 - Write-only key registers in the AES engines
 - Device 96-bit unique ID and JTAG 32-bit device-specific ID
 - Secure storage can rely upon the SAES engine to encrypt the stored data, to the benefit of DHUK properties. All data encrypted with the DHUK benefit of temporal and runtime isolation (HDPL and TrustZone), RHUK, EPOCH (version counter) properties.
- a) Data are isolated: all data encrypted/decrypted relying upon SAES+DHUK benefit of the variation of the DHUK for each different isolated area (HDPL0, HDPL1, HDPL2, HDPL3S, HDPL3NS).
- b) RHUK: hardware secret nonvolatile, unique per device keys (copy protection).
- c) EPOCH: counter incremented each time a regression is done (antirollback).
- General purpose cryptographic acceleration
 - AES 256-bit engine, supporting ECB, CBC, CTR, GCM, and CCM chaining modes

- Secure AES 256-bit security coprocessor, supporting ECB, CBC, CTR, GCM, and CCM chaining modes with side-channel countermeasures and mitigations
- HASH processor, supporting SHA-1 checksums and SHA-2 secure hash (SHA2, SHA2-384, SHA2-512, and SHA-3^(a))
- Public key accelerator (PKA) for RSA/DH (up to 4096 bits) and ECC (up to 640 bits), implementing side-channel counter measures, and mitigations when manipulating secrets
- True random number generator (RNG), NIST SP800-90B precertified
- Coupling and chaining bridge (CCB)^(a) for special coupling and chaining operations (involving PKA, SAES, RNG peripherals) to protect private keys used in PKA protected operations
- On-the-fly decryption of encrypted image stored on external flash memory connected through the OCTOSPI
 - Almost-zero latency with standard NOR flash memories
 - Can be used to encrypt the image using the device unique secret keys
 - Automatic key-erase in case of tamper
- New flexible life cycle scheme
 - Allows product maintenance in-the-field (debug reopening or regressions)
 - Allows protected firmware distribution in up to three steps: immutable root of trust, secure and nonsecure application
- Active tamper and protection against temperature, voltage, and frequency attacks
 - Up to eight active inputs, eight active output tamper pins, available in different power modes (refer to TAMP low-power modes)

3.2 Secure install

The secure firmware install (SFI) is an STMicroelectronics secure service authenticated and decrypted by the immutable RSS code stored in the device. The SFI allows secure and counted installation of OEM firmware in an untrusted production environment (such as OEM contract manufacturer).

The confidentiality of the installed images written in the internal flash memory or encrypted in an external flash memory, is also protected, using the AES.

The SFI native service leverages the following hardware security features:

- secure boot (see [Section 3.3](#))
- resource isolation using TrustZone (see [Section 3.6](#))
- temporal isolation using hide protection (see [Section 3.7.1](#))
- secure execution (see [Section 3.8](#))
- secure storage, with associated cryptographic engines (see [Section 3.9](#) and [Section 3.10](#)) and flash memory dedicated areas

Further information can be found in AN4992 “*Introduction to secure firmware install (SFI) for STM32 MCUs*”.

a. Available only on STM32H543/553xx devices.

3.3 Secure boot

Secure boot is an immutable code that is always executed after a system reset. As a root of trust, this code checks the device static protections and activates available device runtime protections, reducing the risk of invalid or malicious code running on the platform. As root of trust, the secure boot also checks the integrity and authenticity of the next level firmware before executing it.

The STM32H533/553/573xx offer two options (using BOOT_UBE) to execute the immutable code after a reset:

- Security services when STiRoT (code natively present in the system memory) is selected managing the secure boot of the next boot level. STiRoT takes care of the next level integrity, authenticity using configuration done by OEMs. The next stage can be an ST updatable service (STuRoT), or a proprietary one (uRoT)
- Proprietary boot entry when the OEM manages the full chain (OEMiRoT), to be installed in user flash memory with proper security activation.

The actual functions of the secure boot depend on the availability of TrustZone features, and on the firmware stored in the device. The secure boot typically initializes the secure storage, and installs on-the-fly decryption keys in the OTFDEC, to be able to use encrypted firmware stored in an external flash memory.

TrustZone application, supported by the STM32 ecosystem, provides a root of trust solution, including secure boot functions. For more information, refer to the user manual UM3065 “*Getting started with STM32CubeH5 for STM32H5 series*”.

In the devices, the secure boot benefits of hardware security features:

- resource isolation using TrustZone (see [Section 3.6](#))
- temporal isolation using hide protection levels (HDPL) (see [Section 3.7.1](#))
- secure execution (see [Section 3.8](#))
- secure install and update (see [Section 3.2](#) and [Section 3.4](#))
- per domain secure storage, with associated cryptographic engines if available (see [Section 3.9](#) and [Section 3.10](#))

This section describes the features specifically designed for secure boot.

3.3.1 Unique boot entry

Thanks to the unique boot entry (BOOT_UBE), it is possible to select the boot entry point between security services in system flash (STiRoT) and proprietary boot entry (OEMiRoT). This selection is possible only for products embedding cryptographic acceleration (STM32H533/553/573xx), when TrustZone is enabled (TZEN = 0xB4).

When TrustZone is enabled and SECBOOT_LOCK secure option bit is cleared, the application selects a boot entry point located either in the system flash memory (see the next section), or in the secure user flash memory, at the address defined by SECBOOTADD option bytes.

When TrustZone is enabled and SECBOOT_LOCK secure option bit is set, the device unique boot entry is the unmodifiable secure address defined by SECBOOTADD option bytes. The application cannot modify these option bytes when SECBOOT_LOCK is set.

When TrustZone is disabled (TZEN = 0xC3) and NSBOOT_LOCK option bit is cleared, the application selects a boot entry point located in the nonsecure user flash memory, at the address defined by NSBOOTADD option bytes.

When TrustZone is disabled and NSBOOT_LOCK option bit is set, the device unique boot entry is the unmodifiable nonsecure address defined by NSBOOTADD option bytes. The application cannot modify these option bytes when NSBOOT_LOCK is set.

Note: SECBOOT_LOCK and NSBOOT_LOCK can be changed only when PRODUCT_STATE = Open, Provisioning, or Regression.

For more information on the boot mechanisms, refer to [Section 4: Boot modes](#).

3.3.2 Immutable root of trust in system flash memory

The immutable root-of-trust code stored in the system flash memory is first used to perform SFI, allowing secure and counted installation of OEM firmware in untrusted production environments (such as OEM contract manufacturer).

The immutable code also includes secure runtime services that can be called at runtime, when a secure application sets the SBS_RSSCMDR register to a non-null value, before triggering a system reset. This runtime feature is deactivated when the BOOT_LOCK secure option bit is set or when the PRODUCT_STATE is different from "Open".

The STM32H5 products feature ST-DA (debug authentication), to manage the debug authentication control feature, allowing to control the debug reopening and regressions of the product for after sales (field return) of the product.

The products embedding cryptographic acceleration feature STiRoT (immutable root of trust), to handle the secure provisioning, secure boot, secure updates of the first updatable level of the platform.

For more information, refer to AN6007 "Getting started with STiRoT (ST immutable Root of Trust) for STM32H5 MCUs".

3.4 Secure update

The secure firmware update is a secure service that runs after a secure boot. Its actual functions depend on the availability of the TrustZone features, and on the firmware stored in the device.

Cortex-M based application processors is commonly used to run trusted boot and a trusted OS to create a Trusted Execution Environment (TEE). The trusted boot and TEE application, supported by the STM32 ecosystem, allow the update of the microcontroller built-in program with new firmware versions, adding new features and correcting potential issues. The update process is performed in a secure way, to prevent unauthorized updates and access to confidential on-device data.

A firmware update can be done either on a single firmware image including both secure and nonsecure parts, or on the secure (respectively nonsecure) part of the firmware image, independently.

In the devices, the secure update application leverages the same hardware security as the firmware install described in [Section 3.2](#). For more information, refer to the user manual UM3065 "Getting started with STM32CubeH5 for STM32H5 series".

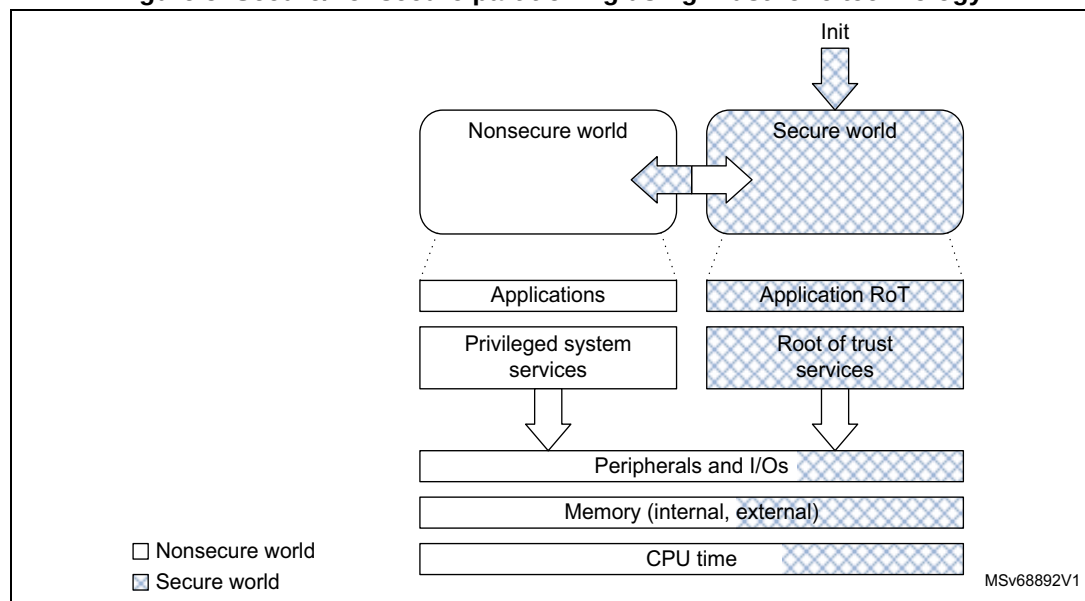
3.5 Resource isolation using hide protection levels

The hardware and software resources used to boot can be isolated. This is called temporal isolation, based on a monotonic counter, taking care of incrementing the levels. When the counter is incremented, the resources (code and data) of the previous levels are hidden.

3.6 Resource isolation using TrustZone

The hardware and software resources can be partitioned so that they exist in the secure or in the nonsecure world, as shown in [Figure 5](#).

Figure 5. Secure/nonsecure partitioning using TrustZone technology



Note: The initial partitioning of the platform is under the responsibility of the secure firmware executed after device reset.

Thanks to this resource isolation technology, the secure world can be used to protect critical code against intentional or unintentional tampering from the more exposed code running in the nonsecure world.

Note: The secure code is typically small and rarely modified, while the nonsecure code is more exposed, and prone to firmware updates.

3.6.1 TrustZone security architecture

The Armv8-M TrustZone technology is a comprehensive hardware architecture that proposes to developers a comprehensive, holistic protection across the entire processor and system. The device TrustZone hardware features include:

- The Armv8-M mainline security extension of Cortex-M33, enabling a new processor secure state, with its associated secure interrupts
- The dynamic allocation of memory and peripherals to TrustZone using eight security attribution unit (SAU) regions of Cortex-M33
- A global TrustZone framework (GTZC), extending the TrustZone protection against transactions coming from masters in the system different from Cortex-M33
- TrustZone-aware embedded flash memory and peripherals

Note: The TZEN option bit in the FLASH_OPTSR2_PRG register activates TrustZone security.

3.6.2 Armv8-M security extension of Cortex-M33

The Arm security extension of the Cortex-M33 is an evolution, not a revolution. It uses the programmer model from earlier Cortex-M subfamilies like Cortex-M4. Indeed, Armv8-M is architecturally similar to Armv7-M, using the same 32-bit architecture, the same memory mapped resources protected with an MPU. Armv8-M also uses the nested vectored interrupt controller (NVIC).

The implementation on STM32H5 devices is composed of the following features:

- A new processor state, with almost no additional code/cycle overhead (as opposed to Armv8-A TrustZone) that uses a dedicated exception routine to trigger a secure/nonsecure world change
- Two memory map views of a shared 4-Gbyte address space
- A low interrupt latency for both secure and nonsecure domains, and a new interrupt configuration for security grouping and priority setting
- Separated exception vector tables for the secure and nonsecure exceptions
- Micro-coded context preservation
- Banking of specific registers across secure/nonsecure states, including stack pointers with stack-limit checkers
- Banking of the following Cortex-M33 programmable components (two separate units for secure and nonsecure):
 - SysTick timer
 - MPU configuration registers (12 MPU regions in secure, eight in nonsecure)
 - some of the system control block (SCB) registers
- New system exception (SecureFault) for handling of security violations
- Configurable debug support, as defined in [Section 3.12](#)

For more information, refer to the Cortex-M33 programming manual (PM0264).

3.6.3 Memory and peripheral allocation using IDAU/SAU

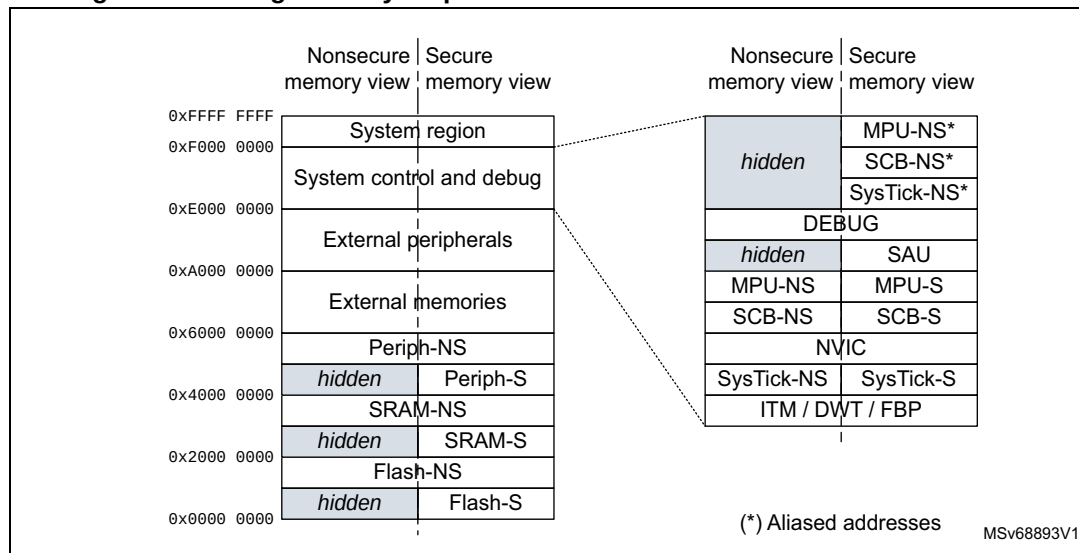
Security attributes

As illustrated in [Figure 6](#), the Armv8-M nonsecure memory view is similar to Armv7-M (found in Cortex-M4), with the difference that the secure memory is hidden. The secure

memory view shows the flash memory, SRAM and peripherals that are accessible only while the Cortex processor executes in Secure state.

Note: [Figure 6](#) shows the 32-bit address space after SAU configuration, as viewed by the secure code.

Figure 6. Sharing memory map between CPU in secure and nonsecure state

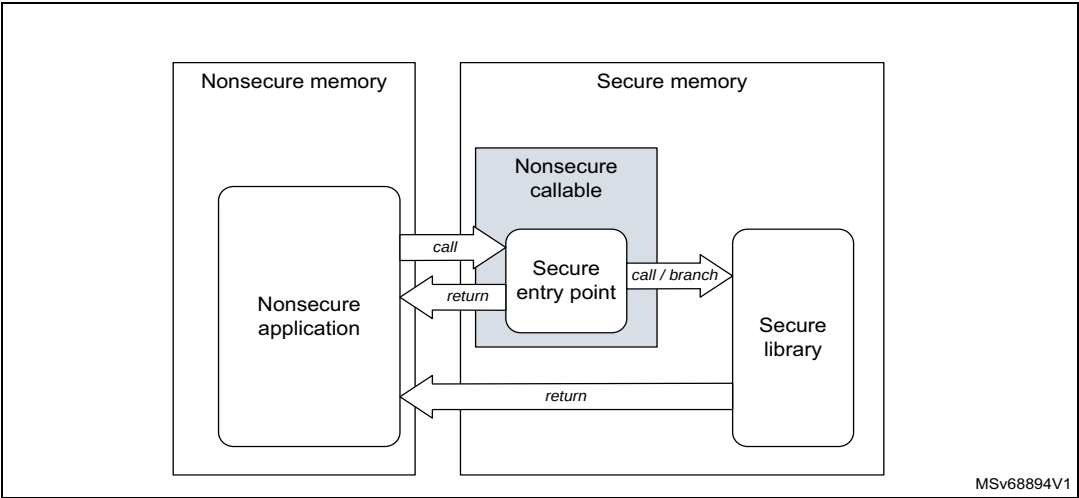


The Cortex processor state (and the associated rights) depends upon the security attribute assigned to the memory region where it is executed:

- A processor in a nonsecure/secure state executes only from nonsecure (NS)/secure (S) program memory.
- When running in secure state, the processor can access data from both S and NS memories. When running in nonsecure state, the CPU is limited to nonsecure memories.

To manage transitions to the secure world, developers must create nonsecure callable (NSC) regions containing valid entry points to the secure libraries. The first instruction in these entry points must be the new secure gate (SG) instruction, used by the nonsecure code to call a secure function (see [Figure 7](#)).

Figure 7. Secure world transition and memory partitioning



Programming security attributes

In Cortex-M33, the static implementation defined attribution unit (IDAU) works with the programmable security attribution unit (SAU) to assign a specific security attribute (S, NS, or NSC) to a specific address, as shown in [Table 8](#).

Table 8. Configuring security attributes with IDAU and SAU

IDAU security attribution	SAU security attribution ⁽¹⁾	Final security attribution
Nonsecure	Secure	Secure
	Secure-NSC	Secure-NSC
	Nonsecure	Nonsecure
Secure-NSC	Secure	Secure
	Nonsecure	Secure-NSC

1. Defined regions are aligned to 32-byte boundaries.

The the Cortex-M33 can configure the SAU only in the secure-privilege state. When the TrustZone is enabled, the SAU defaults all addresses as secure (S). A secure boot application can then program the SAU to create NSC or NS regions, as shown in [Table 8](#).

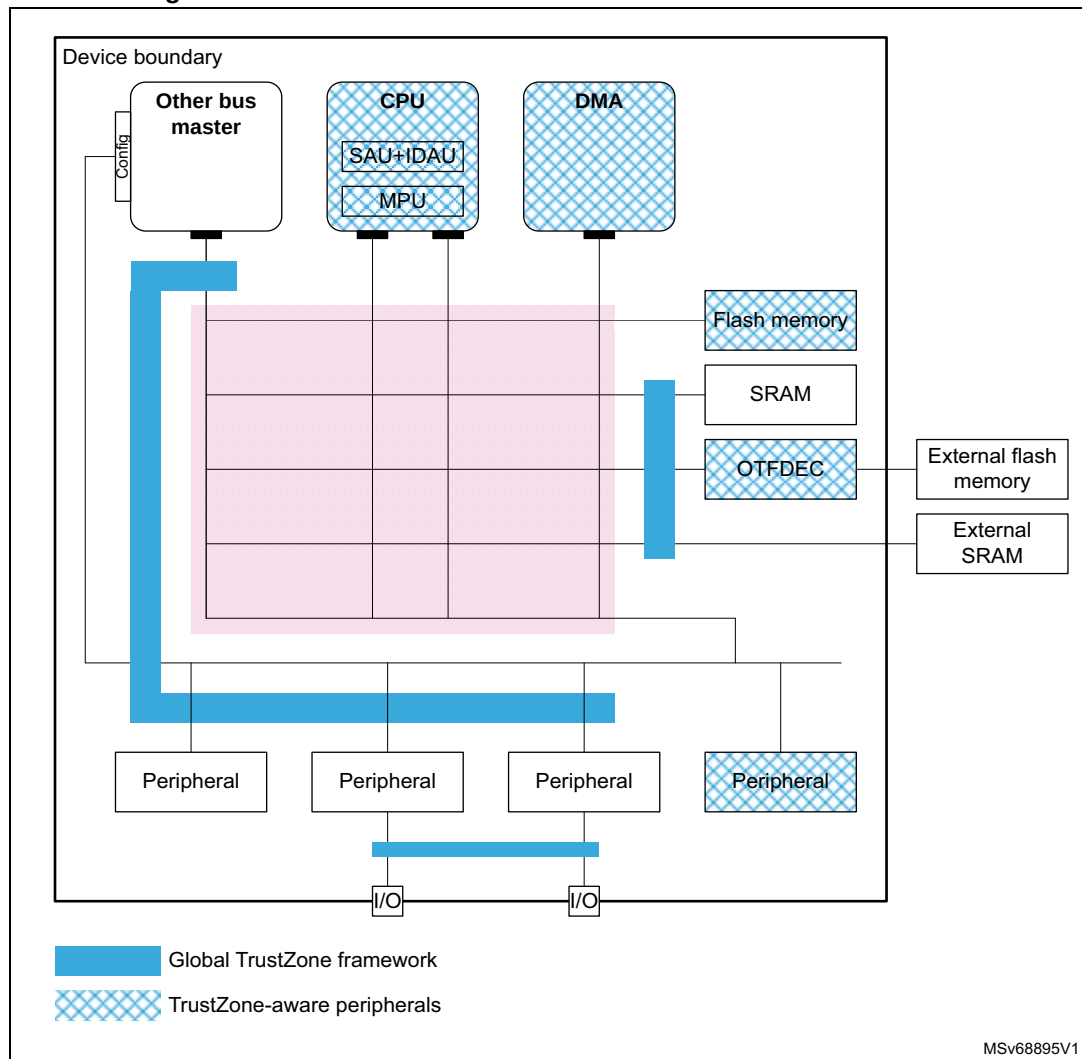
Note: The SAU/IDAU settings are applicable only to the Cortex-M33. The other masters, like DMA, are not affected by these policies.

3.6.4 Memory and peripheral allocation using GTZC

Global TrustZone framework architecture

On top of the Armv8-M TrustZone security extension in Cortex-M33, the devices embed complementary security features that reinforce, in a flexible way, the isolation between secure and nonsecure worlds. Unlike the SAU/IDAU, the GTZC can protect legacy memories and peripherals against nonsecure transactions coming from other masters than the Cortex-M33.

Figure 8. Global TrustZone framework and TrustZone awareness



Securing peripherals with TZSC

When the TrustZone security is active, a peripheral is either securable through the GTZC, or is natively TrustZone-aware, as shown in the previous figure:

- A securable peripheral or memory is protected by an AHB/APB firewall gate, controlled by the TrustZone security controller (TZSC).
- A TrustZone-aware peripheral or memory is connected directly to AHB or APB interconnect, implementing a specific TrustZone behavior, such as a subset of secure registers or a secure memory area.

When a securable peripheral is made secure-only with the GTZC, if this peripheral is master on the interconnect, it automatically issues secure transactions. The SDMMC is an example of securable master. TrustZone-aware AHB masters like Cortex-M33 or DMAs, drive a secure signal in the AHB interconnect, according to their security mode, independently to the GTZC.

Note: Like with TrustZone, a peripheral can be made privileged-only with TZSC (see [Section 3.7.2](#)). In this case, if this peripheral is master on the interconnect, it automatically issues privileged transactions.

Securing memories with TZSC and MPCBB

The TZSC block in GTZC provides the capability to manage the security and privilege for all securable external memories, programming the MPCWM resources as defined in [Table 9](#).

Table 9. MPCWMx resources

Memory	MPC resource	Type of filtering	Number of regions	Default security	On-the-fly decryption ⁽¹⁾
OCTOSPI1	MPCWM1	Nonsecure privileged or unprivileged region (watermarks)	2	Secure privileged ⁽²⁾	Yes
FMC_NOR bank	MPCWM2		2		No
FMC_NAND / FMC_SDRAM ⁽³⁾ bank	MPCWM3		1		
Backup SRAM (BKPSRAM)/FMC_SDRAM ⁽³⁾	MPCWM4		1		

1. Using the OTFDEC.
2. Assuming TrustZone is activated on the device, nonsecure unprivileged otherwise.
3. FMC_SDRAM is not available on STM32H523/533xx devices.

The MPCBB resources in GTZC provide the capability to configure the security and privilege of embedded SRAM blocks, as defined in [Table 10](#), [Table 11](#), and [Table 12](#).

Table 10. MPCBBx resources (STM32H562/563/573xx devices)

Memory	MPC resource	Type of filtering	Memory size	Block size	Number of super-blocks	Default security
SRAM1	GTZC1_MPCBB1	Block based, managing security and privilege	256 KB	512 ⁽¹⁾ bytes	16	Secure privileged ⁽²⁾
SRAM2	GTZC1_MPCBB2		64 KB		4	
SRAM3	GTZC1_MPCBB3		320 KB		20	

1. Blocks are grouped in super-blocks of 32 consecutive blocks, to manage the configuration locking.
2. Assuming TrustZone is activated on the device, nonsecure unprivileged otherwise.

Table 11. MPCBBx resources (STM32H523/533xx devices)

Memory	MPC resource	Type of filtering	Memory size	Block size	Number of super-blocks	Default security
SRAM1	GTZC1_MPCBB1	Block based, managing security and privilege	128 KB	512 ⁽¹⁾ bytes	8	Secure privileged ⁽²⁾
SRAM2	GTZC1_MPCBB2		80 KB		5	
SRAM3	GTZC1_MPCBB3		64 KB		4	

1. Blocks are grouped in super-blocks of 32 consecutive blocks, to manage the configuration locking.
2. Assuming TrustZone is activated on the device, nonsecure unprivileged otherwise.

Table 12. MPCBBx resources (STM32H543/553xx devices)

Memory	MPC resource	Type of filtering	Memory size	Block size	Number of super-blocks	Default security
SRAM1	GTZC1_MPCBB1	Block based, managing security and privilege	128 KB	512 bytes ⁽¹⁾	8	Secure privileged ⁽²⁾
SRAM2	GTZC1_MPCBB2		80 KB		5	-
SRAM3	GTZC1_MPCBB3		96 KB		6	-

1. Blocks are grouped in super-blocks of 32 consecutive blocks, to manage the configuration locking.

2. Assuming TrustZone is activated on the device, nonsecure unprivileged otherwise.

Applying GTZC configurations

The TZSC and MPCBB blocks can be used in one of the following ways:

- statically programmed during the secure boot, locked and not changed afterwards
- dynamically reprogrammed using a specific application code or real-time kernel

When the dynamic option is selected and the configuration is not locked:

- MPCBB secure blocks or MPCWM nonsecure region size can be changed by a secure software. This software must be privileged for MPCWM, can be unprivileged if the particular block is not privileged-only.
- The secure (respectively privilege) state of each peripheral can be changed writing to GTZC_TZSC_SECCFRGx (respectively GTZC_TZSC_PRIVCFGRx) registers.

Securing peripherals with TZSC

The TZSC block in GTZC provides the capability to manage the security and the privilege for all securable peripherals. The list of these peripherals can be found in [Section 5: Global TrustZone® controller \(GTZC\)](#).

Note: When the TrustZone is deactivated, the resource isolation hardware GTZC can still be used to isolate peripherals to privileged code only (see [Section 3.7.2](#)).

When the TrustZone is activated, peripherals are set as nonsecure and unprivileged after reset.

TrustZone-aware peripherals

The devices include the following TrustZone-aware peripherals:

- GPIOA to GPIOI
- GTZCx_MPCBB, GTZCx_TZIC and GTZCx_TZSC (GTZC blocks)
- OTFDEC1, writable only in secure if TZEN is enabled
- EXTI
- Flash memory
- RCC and PWR
- GPDMA1 and GPDMA2
- SBS registers
- RTC and TAMP
- MCU debug unit DBGMCU
- PLAY1^(a)

The way illegal accesses to those peripherals are monitored through the TZIC registers is described in [Section 5: Global TrustZone® controller \(GTZC\)](#).

For more details, refer to [Section 3.6.5](#).

TrustZone illegal access controller (TZIC)

The TZIC block in GTZC gathers all illegal access events originated from sources either protected by GTZC or TrustZone-aware, generating one global secure interrupt towards the NVIC.

TZIC is available only when the system is TrustZone enabled (TZEN = 0xB4). All accesses to TZIC registers must be secured.

For each illegal event source, a status flag and a clear bit exist. Each illegal event can be masked, not generating an interrupt toward the NVIC.

Note: By default, all events are masked.

3.6.5 Managing security in TrustZone-aware peripherals

This section gives more details on how the security is implemented in the TrustZone-aware peripherals listed in the previous section.

Embedded flash memory

When the TrustZone security is enabled through option bytes (TZEN = 0xB4), the whole memory is secure after reset. The following protections, shown in [Figure 9](#), are available to the application:

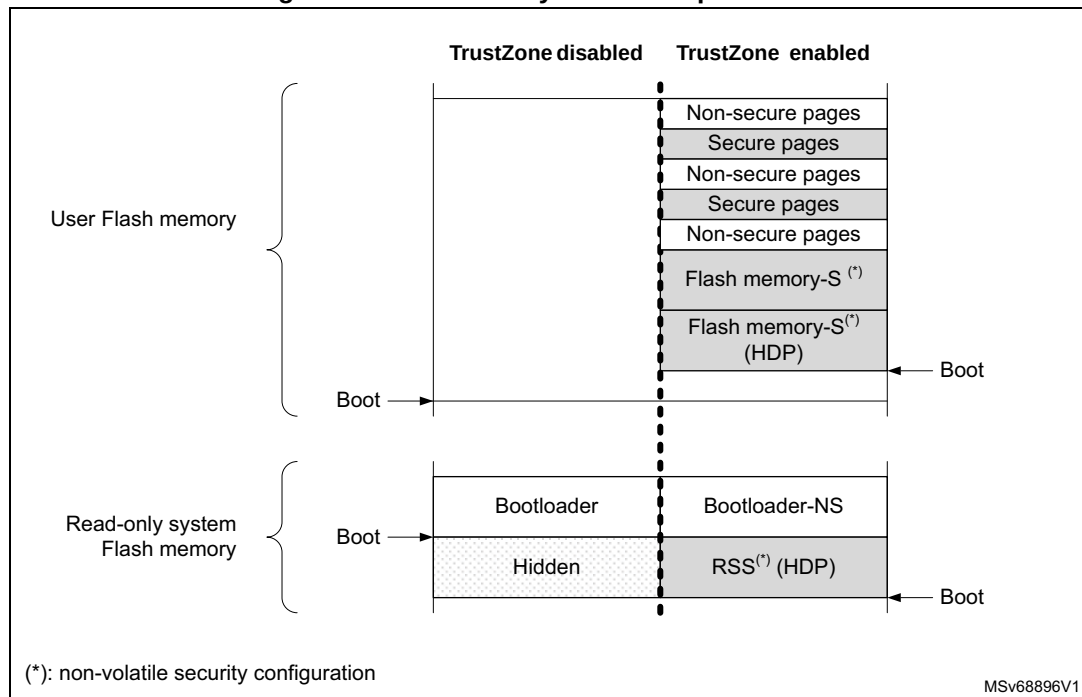
- nonvolatile user secure areas, defined with nonvolatile secure user option bytes
 - watermark-based secure only area (x2)
 - secure hide protection (HDP) area, stickily hidden after boot (x2)
- volatile user secure pages, defined with volatile secure registers (lost after reset)
 - Any page set as nonsecure (example: outside watermark-based secure only area), can be set as secure on-the-fly using the block-based configuration registers.

Note: All areas are aligned on the page granularity of the flash memory, which can be configured as secure while it is tagged as nonsecure in Cortex-M33 IDAU/SAU. In this case, nonsecure accesses by the CPU to the flash memory are denied.

Erase or program operations can be available to secure/nonsecure code only for secure/nonsecure pages or memory. A flash memory is considered secure if at least one page is secure.

a. Available only on STM32H543/553xx devices.

Figure 9. Flash memory TrustZone protections



As shown above, when TrustZone is enabled (TZEN = 0xB4), the application code can use the HDP area that is part of the flash memory watermark-based secure area. Indeed, when the application sets the HDPx_ACCDIS bit, data read, write, and instruction fetch on this HDP area is denied until the next system reset.

For example, the software code in the secure flash memory HDP area can be executed only once, with any further access to this area denied until the next system reset. Additionally, any flash memory page belonging to an active HDP area cannot be erased anymore.

When the TrustZone is disabled (TZEN = 0xC3), the volatile/nonvolatile secure area features are deactivated and all secure registers are RAZ/WI.

See [Section 7: Embedded flash memory \(FLASH\)](#) for more details.

On-the-fly encryption/decryption (OTFDEC)

When the TrustZone security is enabled (TZEN = 0xB4), the OTFDEC can be initialized only by secure applications. Each of the four encrypted regions, once the configuration is confirmed, can be write-locked until the next power-on-reset.

Note: Any application (secure or nonsecure) can verify the initialization context of each OTFDEC region (including CRC of the keys), by reading the peripheral registers.

Key registers in each OTFDEC are write-only.

See [Section 3.10.3](#) for more details on this cryptographic engine.

Direct memory access controllers (GPDMAx)

When a DMA channel x is defined as secure (SECx = 1 in GPDMA_SECCFGR), the source and destination transfers can be independently set as secure or nonsecure by a secure application using SSEC and DSEC bits in GPDMA_CxTR1. [Table 13](#) summarizes these security options available in each DMA channel.

Table 13. DMA channel use (security)⁽¹⁾

Destination type	Secure DMA channel x (SECx = 1)		Nonsecure DMA channel y (SECy = 0)	
	Secure source	Nonsecure source	Secure source	Nonsecure source
Secure destination	OK	OK ⁽²⁾	Transfer blocked	
Nonsecure destination	OK ⁽³⁾	OK ⁽⁴⁾	Transfer blocked	OK

1. When a transfer is blocked, the transfer completes but the corresponding writes are ignored, and reads return 0s. An illegal access event to TZIC is automatically triggered by the memory/peripheral used as source or destination.
2. If the source is a memory, the transfer is possible only if SSEC = 0, otherwise the transfer is blocked.
3. If the destination is a memory, the transfer is possible only if DSEC = 0, otherwise the transfer is blocked.
4. If the transfer is memory-to-memory, it is possible only if SSEC = 0 and DSEC = 0, otherwise it is blocked.

When a channel is configured as secure:

- Registers allocated to this channel (excluding GPDMA_SECCFGR, GPDMA_PRIVCFGR and GPDMA_RCFGLOCKR) are read as 0. Writes are ignored for nonsecure accesses. A secure illegal access event may also be triggered towards the TZIC peripheral.
 - Writes to GPDMA_SECCFGR and GPDMA_RCFGLOCKR must be secure. For each bit in GPDMA_PRIVCFGR, write must be secure if the corresponding bit in GPDMA_SECCFGR is set.
- In linked-list mode, the loading of the next linked-list data structure from memory is performed with secure transfers.
- When switching to a nonsecure state, the secure application must abort the channel or wait until the secure channel is completed before doing the switch.

Note: DMA secure channels are not available when TrustZone is deactivated. When a channel is configured as nonsecure, in linked-list mode, the loading of the next linked-list data structure from memory is performed with nonsecure transfers.

See [Section 16: General purpose direct memory access controller \(GPDMA\)](#) for more details.

Power control (PWR)

When the TrustZone security is enabled (TZEN = 0xB4), the selected PWR registers can be secured through PWR_SECCFGR, protecting the following PWR features:

- low power mode setup
- wake-up (WKUP) pins definition
- voltage detection and monitoring
- backup domain control

Other PWR configuration bits become secure:

- when the system clock selection is secure in the RCC: the voltage scaling (VOS) becomes secure.
- when a GPIO is configured as secure: its corresponding bit for pull-up/pull-down configuration in Standby mode becomes secure.
- when the USB Type-C/USB power delivery interface (UCPD) is configured as secure in TZSC: PWR_UCPDR register becomes secure.

See [Section 10: Power control \(PWR\)](#) for details.

Secure clock and reset (RCC)

When the TrustZone security is enabled (TZEN = 0xB4) and security is enabled in the RCC, the bits controlling the peripheral clocks and resets become TrustZone-aware:

- If the peripheral is securable and programmed as secure in the TZSC, the peripheral clock and reset bits become secure.
- If the peripheral is TrustZone-aware, the peripheral clock and reset bits become secure when at least one function is configured as secure inside the peripheral.

Note: Refer to [Section 3.6.4](#) for the list of securable and TrustZone-aware peripherals.

Additionally, the following configurations can be made secure-only using RCC_SECCFGR:

- external clock (such as HSE or LSE), internal oscillator (such as HSI, CSI, or LSI)
- main PLL and AHB prescaler
- system clock source selection
- MCO clock output
- reset flag
- automatic internal oscillator waking up configuration

See [Section 11: Reset and clock control \(RCC\)](#) for details.

Real-time clock (RTC)

Like all TrustZone-aware peripherals, a nonsecure read/write access to a secured RTC register is RAZ/WI. It also generates an illegal access event that triggers a secure illegal access interrupt if the RTC illegal access event is enabled in the TZIC.

After a backup domain power-on reset, all RTC registers can be read or written in both secure and nonsecure modes. The secure boot code can then change the security setup, making registers Alarm A, Alarm B, wakeup timer, and timestamp secure or not, using RTC_SECCFGR.

When the SEC bit is set in secure-only RTC_SECCFGR:

- Writing the RTC registers is possible only in secure mode.
- Reading RTC_SECCFGR, RTC_PRIVCFGR, RTC_MISR, RTC_TR, RTC_DR, RTC_SSR, RTC_PRER, and RTC_CALR is always possible in secure and nonsecure modes. All the other RTC registers can be read only in secure mode.

When SEC is cleared in secure-only RTC_SECCFGR, it is still possible to restrict access in secure mode to some RTC registers by setting dedicated control bits: INITSEC, CALSEC, TSSEC, WUTSEC, ALRASEC, and ALRBSEC.

Note: The RTC security configuration is not affected by a system reset.

See [Section 51: Real-time clock \(RTC\)](#) for more details.

Tamper and backup registers (TAMP)

Like all TrustZone-aware peripherals, a nonsecure read/write access to a secured TAMP register is RAZ/WI. It also generates an illegal access event that triggers a secure illegal access interrupt if the TAMP illegal access event is enabled in the TZIC.

After a backup domain power-on reset, all TAMP registers can be read or written in both secure and nonsecure modes. The secure boot code can change this security setup, making some registers secure or not as needed, using TAMP_SECCFGR register.

When TAMPSEC is set in TAMP_SECCFGR:

- Writing the TAMP registers is possible only in secure mode. Backup registers have their own write protection (see below).
- Reading the TAMP registers (exception TAMP_SECCFGR, TAMP_PRIVCFGR, and TAMP_MISR) returns 0 if the access is nonsecure. Backup registers have their own read protection (see below).

The application can also:

- make TAMP_COUNTR register read and write secure-only by setting the CNT1SEC bit in TAMP_SECCFGR secure register
- increase security in backup registers for two of the three protection zones configured using BKPRWSEC[7:0] and BKPWSEC[7:0] bitfields in TAMP_SECCFGR:
 - protection zone 1 is read nonsecure, write nonsecure
 - protection zone 2 is read nonsecure, write secure
 - protection zone 3 is read secure, write secure

Note: The TAMP security configuration is not affected by a system reset.

See [Section 52: Tamper and backup registers \(TAMP\)](#) for more details.

General-purpose I/Os (GPIO)

When the TrustZone security is enabled (TZEN = 0xB4), each I/O pin of the GPIO ports can be individually configured as secure through the GPIOx_SECCFGR registers. Only a secure application can write to GPIOx_SECCFGR registers. After boot, each I/O pin is set as secure.

When an I/O pin is configured as secure, its corresponding configuration bits for alternate function (AF), mode selection (MODE), and I/O data are RAZ/WI in case of nonsecure access.

When a digital alternate function is used (input/output mode) to protect data transiting from/to the I/O managed by a secure peripheral, the devices add a secure alternate function gate on the path between the peripheral and its allocated I/Os:

- if the peripheral is secure, the I/O pin must also be secure to allow input/output of data
- if the peripheral is not secure, the connection is allowed regardless of the I/O pin state.

The TrustZone-aware logic around GPIO ports, used as alternate functions, is summarized in [Table 14](#).

Table 14. Secure alternate function between peripherals and allocated I/Os

Security configuration		Alternate function logic		Comment
Peripheral	Allocated I/O pin	Input	Output	
Secure	Secure	I/O data	Peripheral data	-
Nonsecure				Out of reset configuration
Secure	Nonsecure	0	0	-
Nonsecure		I/O data	Peripheral data	

When an analog function with an analog switch is used, the connection to the peripherals listed in [Table 15](#) is blocked by hardware when the peripheral is nonsecure and the I/O is secure.

Table 15. Nonsecure peripheral functions that cannot be connected to secure I/Os

Peripheral	Analog function ⁽¹⁾	Input	Output
ADC12	ADC12_INy (y = 1 to 17)	X	-
ADC3 ⁽²⁾	ADC3_INP	X	-
COMP1/2 ⁽²⁾	COMP1/2_INP/INM	X	-

1. Used to find the I/O corresponding to the signal/function on the package (refer to the product datasheet).
2. Available only on STM32H543/553xx devices.

Finally, regarding GPIO and security, [Table 16](#) summarizes the list of I/Os without any hardware protection linked to TrustZone. The listed signals (input and/or outputs) are not blocked when the I/O is set as secure, and the associated peripheral is non secure.

For example, when a secure application sets PA4 as secure to be used as LPTIM2_CH1, if the DAC is nonsecure, it can be programmed to output data to PA4, potentially causing malfunction to the secure application.

Similarly, when a secure application sets PA0 as secure to be used as UART4_TX, if TAMP is nonsecure, it can be programmed to capture the USART input traffic through the TAMP_IN signal.

It is important that, for each case described in [Table 16](#), the secure application decides if a potential effect on data integrity or confidentiality is critical or not. For example, if the USART situation described above is not acceptable (data transiting on secure USART is confidential), then the secure application must configure the TAMP as secure even if not used by the secure application.

Table 16. Nonsecure peripheral functions that can be connected to secure I/Os

Peripheral	Signal ⁽¹⁾	Input	Output	How to set the peripheral or function as secure
DAC	DAC1_OUTx (x = 1, 2)	-	X	Set DAC1SEC in GTZC1_TZSC_SECCFGR1.
UCPD	UCPD1_CCx (x = 1, 2)	X	X	Set UCPD1SEC in GTZC1_TZSC_SECCFGR1.
	UCPD1_DBx (x = 1, 2)	X	-	
TAMP	TAMP_INx (x = 1 to 8)	X	-	Set TAMPSEC in TAMP_SECCFGR
	TAMP_OUTx (x = 1 to 8)	-	X	
RTC	RTC_OUTx (x = 1, 2)	-	X	Set SEC in RTC_SECCFGR.
	RTC_TS	X	-	Set TSSEC in RTC_SECCFGR.
PWR	WKUPx (x = 1 to 8)	X	-	Set WUPxSEC in PWR_SECCFGR.
RCC	LSCO	-	X	Set LSESEC in RCC_SECCFGR.
EXTI	EXTIx (x = 0 to 66)	X	-	Set SECx bit in EXTI_SECCFGR.

1. To find the I/O corresponding to the signal/function on the package, refer to the product datasheet.

Refer to [Section 13: General-purpose I/Os \(GPIO\)](#) for more details.

Extended interrupts and event controller (EXTI)

When the TrustZone security is enabled, the EXTI is able to protect event register bits from being modified by nonsecure accesses. The protection can individually be activated per input event via the register bits in EXTI_SECCFGRx. When an input event is configured as secure, only a secure application can change the configuration (including security), change the masking or clear the status of this input event.

The security configuration in EXTI_SECCFGR1 and EXTI_SECCFGR2 can be globally locked after reset in EXTI_LOCKR.

See [Section 18: Extended interrupts and event controller \(EXTI\)](#) for more details.

System configuration, boot, and security (SBS)

Like all TrustZone-aware peripherals, when the TrustZone security is enabled, a nonsecure read/write access to a secured SBS register is RAZ/WI. Such access also generates an illegal access event that triggers a secure illegal access interrupt if the SBS illegal access event is not masked in the TZIC.

See [Section 14: System configuration, boot, and security \(SBS\)](#) for more details.

Microcontroller debug unit (DBGMCU)

The MCU debug component (DBGMCU) helps the debugger, providing support for:

- low-power modes behavior during debug
- peripheral freeze during debug, applicable to I2Cs, I3Cs, IWDG, WWDG, timers, low-power timers, and GPDMA channels

The DBGCMU is a TrustZone-aware peripheral, managing accesses to its control registers as described in [Table 17](#).

Table 17. TrustZone-aware DBGMCU nonsecure accesses management

Debug profile	Peripheral status ⁽¹⁾	DBG_xx_STOP control bits	
		Write access	Read access
Nonsecure invasive (SPIDEN = 0)	NS	Yes (S ⁽²⁾ or NS)	Yes (S or NS)
	S	None (S or NS)	
Secure invasive (SPIDEN =1)	NS	Yes (S or NS)	
	S	Yes (S only)	

1. As reported by the GTZC, the TrustZone-aware peripheral or the DMA channel.

2. Secure access from debugger is converted to nonsecure access in the device.

Refer to [Section 66.12: Microcontroller debug unit \(DBGMCU\)](#) for more details.

3.6.6 Activating TrustZone security

The TrustZone is deactivated by default in all devices. It can be activated by setting the TZEN option bit in FLASH_OPTSR2_PRG when PRODUCT_STATE is Open. Once TZEN

has changed from disabled to enabled (0xC3 to 0xB4), the default security state, after reset, is always the following:

- CPU subsystem
 - Cortex-M33 exits reset in secure state, hence the boot address must point toward a secure memory area.
 - All interrupt sources are secure (in NVIC).
 - The memory mapped viewed by the CPU through IDAU/SAU is fully secure.
- Embedded flash memory
 - Flash memory nonvolatile secure areas (with their HDP zone), are defined with nonvolatile registers FLASH_SECWMxR (x = 1, 2). Default secure option bytes setup is all user flash secure, without HDP area defined.
 - Volatile block-based security attributions of the flash memory are nonsecure.
- Embedded SRAMs
 - All SRAMs are secure, as defined in GTZC/MPCBB (see [Section 3.6.4](#)). The secure boot code can change this security setup, making blocks secure or not.
- External memories
 - All memory devices connected to the FMC and OCTOSPI are secure, as defined in GTZC/MPCWM (see [Section 3.6.4](#)). The secure-boot code can change this security setup, making components secure or not.
- All GPIOs are secure.
- All GPDMA channels are nonsecure.
- Backup registers are nonsecure.
- Peripherals and GTZC:
 - Securable peripherals are nonsecure and unprivileged.
 - TrustZone-aware peripherals are nonsecure, with their secure configuration registers being secure.
 - All illegal access interrupts in GTZC/TZIC are disabled.

Note: Refer to [Section 3.6.4](#) for the list of securable and TrustZone-aware peripherals.

3.6.7 Deactivating TrustZone security

Once TrustZone is activated, it can only be deactivated during a PRODUCT_STATE regression to Open.

Note: Such PRODUCT_STATE regression triggers the erasing of embedded memories (SRAM2, flash), and the reset of all peripherals, including the OTFDEC and all crypto engines.

After the TrustZone deactivation, most of the features mentioned in [Section 3.6](#) are no longer available:

- The nonvolatile secure area of the embedded flash memory is deactivated, including the HDP area.
- Only the NVIC manages nonsecure interrupts.
- All secure registers in TrustZone-aware peripherals are RAZ/WI.

Note: When the TrustZone is deactivated, the resource isolation using privilege remains available (see [Section 3.7.2](#) for details).

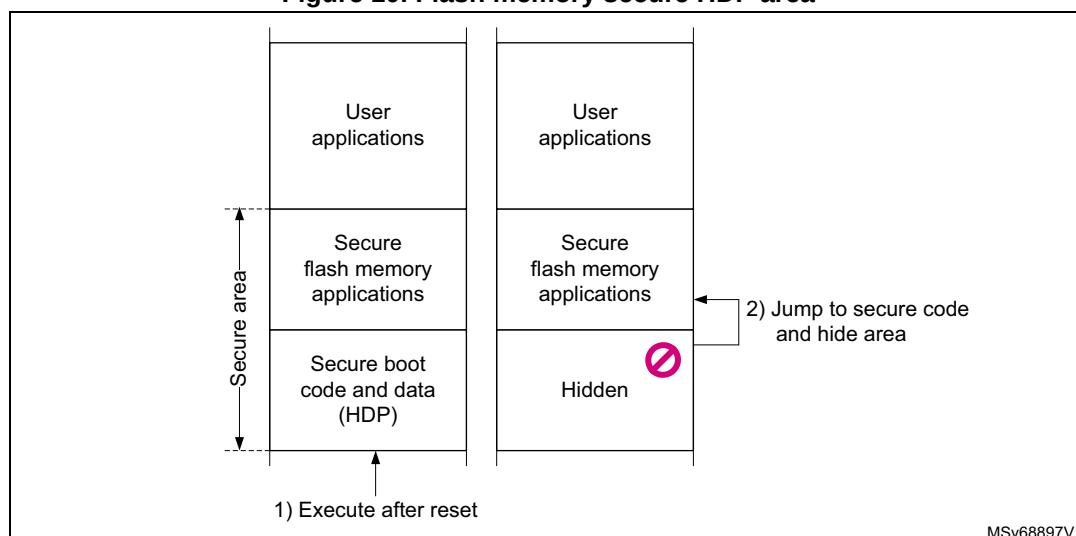
3.7 Other resources isolation

These are hardware mechanisms offering an additional level of isolation on top of the TrustZone technology.

3.7.1 Temporal isolation using secure hide protection (HDP)

When the TrustZone security is enabled, the embedded flash memory allows to define an HDP area per watermarked-secure area of each bank (8-Kbyte page granularity). The code executed in this HDP area, with its related data and keys, can be hidden after boot, until the next system reset. The hide protection principle is shown in [Figure 10](#).

Figure 10. Flash memory secure HDP area



Activation of HDP area in user flash memory is related to HDPL1: as soon as $HDPL \geq 2$, data read, write, and instruction fetch (on the area defined by `HDPx_PSTRT` and `HDPx_PEND` in `FLASH_HDPxR_PRG` option bytes), are denied until the next device reset.

The END of the `HDPx` areas can be extended (dynamically by the application) thanks to `FLASH_HDPEXTR` flash memory register.

Note: Bank erase aborts when it contains a write-protected area (WRP or HDP area).

3.7.2 Resource isolation using Cortex privileged mode

In parallel to the TrustZone isolation described in [Section 3.6](#), the hardware and software resources can be partitioned so that they are restricted to software running in Cortex privileged mode.

Thanks to this hardware isolation technology, available even if TrustZone is disabled, critical code or data can be protected against intentional or unintentional tampering from the more exposed unprivileged code.

Memory and peripheral privileged allocation using MPU

The Cortex-M33 MPU divides the unified memory into eight regions in nonsecure and twelve in TrustZone, each aligned to a multiple of 32 bytes. Each memory region can be programmed to generate faults when accessed inappropriately by unprivileged software.

Memory and peripheral privileged allocation using GTZC

For the Cortex-M33 master, to complement the coarse isolation provided by the MPU, the GTZC reinforces, in a flexible way, the isolation between privileged and unprivileged tasks, for peripherals and selected memories.

For masters other than the Cortex-M33, the GTZC can assign them as unprivileged initiators, automatically protecting resources defined as privileged against this master.

- Securing peripherals with TZSC (privileged-only)

In the devices, a peripheral is either securable privileged-only through GTZC, or is natively privileged-aware:

- A securable privileged-only peripheral or memory is protected by an AHB/APB firewall gate controlled by the TZSC.
- A privileged-aware peripheral or memory is connected directly to AHB or APB interconnect, implementing a specific behavior (for example, a subset of registers or a memory area is privilege-only).

When such a peripheral is made privileged-only with GTZC, if it is master on the interconnect (SDMMC), it automatically issues privileged transactions. Privilege-aware masters like GPDMA1 and GPDMA2, drive privileged signal in the AHB interconnect according to their internal privileged mode, independently to the GTZC.

The list of securable peripherals can be found in [Section 5: Global TrustZone® controller \(GTZC\)](#).

- Securing memories with TZSC and MPCBB (privileged-only)

The TZSC logic in GTZC provides the capability to manage the privilege level for all securable external memories, programming the MPCWM resources defined in [Section 3.6.4](#).

Similarly, the TZSC logic in GTZC provides the capability to configure the privilege level of embedded SRAM blocks, programming the MPCBB resources defined in [Section 3.6.4](#).

- Error management (privileged-only)

- Any unprivileged transaction trying to access a privileged resource is considered as illegal. There is no illegal access event generated for illegal unprivileged read and write accesses.
- The addressed resource follows a silent-fail behavior, returning all-0 data for read and ignoring any write.
- When an illegal unprivileged access occurs, no bus error is generated, except when this is an instruction fetch, accessing a privileged memory or a peripheral register.

Managing security in privileged-aware peripherals

TrustZone-aware peripherals also implement privileged-only access mode. The privileged protection is valid even if TrustZone is disabled (TZEN = 0xC3):

- *Embedded flash memory*

By default, all registers can be read or programmed in both privileged and unprivileged modes.

When secure privileged bit SPRIV is set in FLASH_PRIVCFGR, reading and writing the secure registers is possible only in privileged mode. Write access to this bit is ignored if TrustZone is disabled (TZEN = 0xC3).

When nonsecure privileged bit NSPRIV is set in FLASH_PRIVCFGR, reading and writing the flash memory nonsecure registers is possible only in privileged mode.

Regarding privileged protection, the devices offer the following features:

- The system flash memory can be accessed in both privileged and unprivileged modes.
- Each watermark-based secure area, including its secure HDP area, is accessible in secure-privileged and secure-unprivileged mode, if applicable.
- Each 8-Kbyte page of the embedded flash memory can be programmed on-the-fly as privileged only, using the block-based privileged configuration registers FLASH_PRIV1BBRx and FLASH_PRIV2BBRx. An unprivileged page is accessible by privileged or unprivileged access.

Note: Switching a page from privileged to unprivileged does not erase its content.

When applicable, an erase or program operation is always available to privileged code, and is available to unprivileged code only for unprivileged pages or unprivileged memory.

- *On-the-fly encryption/decryption (OTFDEC)*

When privileged bit PRIV is set in OTFDEC_PRIVCFGR, only a privileged application can initialize the OTFDEC.

Note: OTFDEC_PRIVCFGR can be read by both privileged and unprivileged code.

- *Direct memory access controllers (GPDMAx)*

When a DMA channel x is defined as privileged (PRIVx = 1 in GPDMA_PRIVCFGR), special rules (see [Table 18](#)) apply when accessing privileged/unprivileged source or destination.

Table 18. DMA channel use (privilege)

Destination	Privileged DMA channel x (PRIVx = 1)		Unprivileged DMA channel y (PRIVy = 0)	
	Privileged source	Unprivileged source	Privileged source	Unprivileged source
Privileged	OK		Transfer blocked ⁽¹⁾	
Unprivileged			Transfer blocked	OK

1. When a transfer is blocked, the transfer completes but the corresponding writes are ignored, and reads return 0s.

- *Power control (PWR)*

By default, after a power-on or a system reset, all PWR registers except PWR_PRIVCFGR, can be read or written in both privileged and unprivileged modes.

When secure privileged bit SPRIV is set in PWR_PRIVCFGR, reading and writing the PWR securable registers are possible only in privileged mode. Write access to this bit is ignored if TrustZone is disabled (TZEN = 0xC3).

When nonsecure privileged bit NSPRIV is set in PWR_PRIVCFGR, reading and writing the PWR nonsecure registers are possible only in privileged mode.

See [Section 10: Power control \(PWR\)](#) for details.

- *Secure clock and reset (RCC)*

By default, after a power-on or a system reset, all RCC registers except RCC_PRIVCFGR can be read or written in both privileged and unprivileged modes.

When secure privileged bit SPRIV is set in RCC_PRIVCFGR, reading and writing the RCC securable bits are possible only in privileged mode. Write access to this bit is ignored if TrustZone is disabled (TZEN = 0xC3).

When nonsecure privileged bit NSPRIV is set in RCC_PRIVCFGR, reading and writing the RCC nonsecure bits are possible only in privileged mode.

See [Section 11: Reset and clock control \(RCC\)](#) for details.

- *Real time clock (RTC)*

By default after any backup domain reset, all RTC registers except RTC_PRIVCFGR, can be read or written in both privileged and unprivileged modes.

When PRIV bit is set in privileged-only RTC_PRIVCFGR:

- Writing the RTC registers is possible only in privileged mode.
- Reading the RTC_SECCFGR, RTC_PRIVCFGR, RTC_TR, RTC_DR, RTC_SSR, RTC_PRER and RTC_CALR is always possible in privileged and unprivileged modes.

All the other RTC registers can be read only in privileged mode.

When PRIV bit is cleared in privileged-only RTC_PRIVCFGR register, it is still possible to restrict access to privileged mode to some RTC registers by setting dedicated control bits: INITPRIV, CALPRIV, TSPRIV, WUTPRIV, ALRAPRV, or ALRBPRIV.

See [Section 51: Real-time clock \(RTC\)](#) for details.

- *Tamper and backup registers (TAMP)*

By default after any backup domain reset, all TAMP registers except TAMP_PRIVCFGR can be read or written in both privileged and unprivileged modes.

When PRIV bit is set in privileged-only TAMP_PRIVCFGR:

- Writing the TAMP registers is possible only in privileged mode, except for the backup registers and the monotonic counters that have their own protection setting.
- Reading the TAMP_SECCFGR or TAMP_PRIVCFGR is always possible in privilege and unprivilege modes. All the other TAMP registers can be read only in privilege mode, except for the backup registers and the monotonic counters that have their own protection setting.

The application can also:

- make TAMP_COUNT1R register read and write privileged-only by setting the CNTPRIV bit in TAMP_PRIVCFGR
- increase security for two of the three protection zones in backup registers, using BKPRWPRIV and BKPWPRIV bits in TAMP_PRIVCFGR:
 - Make protection zone 1 read privileged, write privileged.
 - Make protection zone 2 read privileged or unprivileged, write privileged.
 - Protection zone 3 is always read and write privileged or unprivileged.
- *General-purpose I/Os (GPIO)*

All GPIO registers can be read and written by privileged and unprivileged accesses, whatever the security state (secure or nonsecure).
- *Extended interrupts and event controller (EXTI)*

The EXTI peripheral is able to protect event register bits from being modified by unprivileged accesses. The protection is individually activated per input event via the register bits in the privileged-only EXTI_PRIVCFGRx registers. When an input event is configured as privileged, only a privileged application can change the configuration (including security if applicable), change the masking or clear the status of this input event.

The security configuration in EXTI_PRIVCFGR1 and EXTI_PRIVCFGR2 can be globally locked after reset in EXTI_LOCKR.

See [Section 18: Extended interrupts and event controller \(EXTI\)](#) for more details.
- *System configuration boot and security (SBS)*

All SBS registers can be read and written in both privileged and unprivileged modes, except:

 - FPUSEC bit in SBS_SECCFGR registers (privileged only)
 - SBS registers for CPU configuration: SBS_CSLOCKR, SBS_FPUIMR, and SBS_CNSLCKR

See [Section 14: System configuration, boot, and security \(SBS\)](#) for more details.

3.8 Secure execution

Through a mix of special software and hardware features, the devices ensure the correct operation of their functions against abnormal situations caused by programmer errors, software attacks through network access or local attempt for tampering code execution.

This section describes the hardware features specifically designed for secure execution.

3.8.1 Memory protection unit (MPU)

The Cortex-M33 includes a memory protection unit (MPU) that can restrict the read and write accesses to Each memory region (including those mapped to peripherals), based on one or more of the following parameters

- Cortex-M33 operating mode (privileged, unprivileged)
- data/instruction fetch

The memory map and the programming of the nonsecure and secure MPUs split memory into regions (up to eight for the nonsecure, and up to twelve for the TrustZone). Secure MPU is only available when TrustZone is activated.

3.8.2 Embedded flash memory write protection

The embedded flash memory write protection (WRP) prevents illegal or unwanted write/erase to special sections of the embedded flash memory user area (system area is permanently write protected).

Write protected area is defined through the option bytes, writing the start and end addresses: two write-protected areas can be defined in each bank, with the granularity of a 32-Kbyte page.

WRP areas can be modified through option byte changes unless the corresponding FLASH_WRPSTGNxR has its UNLOCK option bit cleared (meaning ROM emulation). UNLOCK can be set only when regressing to PRODUCT_STATE = Open.

Note: Bank erase aborts when it contains a write-protected area (WRP or HDP area).

3.8.3 Tamper detection and response

Principle

The devices include active protection of critical security assets against temperature, voltage and frequency attacks, with the following features:

- erasure of device secrets upon tamper detection
- improved guarantee of safe execution for the CPU and its associated security peripherals, including:
 - out-of-range voltage (example: V_{BAT} , V_{DDA}), temperature and clocking (LSE) detection
 - security watchdog IWDG clocked by the internal oscillator LSI
 - possible selection of internal oscillator HSI as system clock
- power supply protection
 - RTC/TAMP domain powered automatically with V_{DD} or V_{BAT}

See [Section 52: Tamper and backup registers \(TAMP\)](#) for more details.

Tamper detection sources

The devices support eight active input/output pins, allowing four independent active-tamper meshes, or up to seven meshes if the same output pin is shared by several input pins (for a total of eight active-tamper I/Os). The active-tamper balls are mapped in the center of packages that can be used in the POS market (such as WLCSP80).

The active pins are clocked by the LSE, and are functional in different system operating modes (Run, Sleep, Stop, or Standby), and in VBAT mode. Refer to TAMP pins functionality over modes for list of tamper pins and their availability across power modes

Detection time is programmable, and a digital filtering is available (tamper triggered after two false comparisons in four consecutive comparison samples).

Note: Timestamps are automatically generated when a tamper event occurs.

The internal tamper sources are listed in [Table 19](#).

Table 19. Internal tamperers in TAMP

Tamper input	NOER bit number in TAMP_CR3	Tamper source
itamp1	0	Backup domain voltage continuous monitoring, functional in VBAT mode
itamp2	1	Temperature monitoring, functional in VBAT mode
itamp3	2	LSE monitoring ⁽¹⁾ , functional in VBAT mode
itamp4	3	HSE monitoring
itamp5	4	RTC calendar overflow (rtc_calovf)
itamp6	5	JTAG/SWD access
itamp7, 12, 13	6, 11, 12	Voltage monitoring (V_{CORE} , V_{REF+}), through ADC analog watchdog
itamp8	7	Monotonic counter overflow (generated internally)
itamp9	8	Fault generation for cryptographic peripherals (SAES, PKA, AES, RNG)
itamp11	10	IWDG timeout and potential tamper (IWDG reset when at least one enabled tamper flag is set)
itamp15	14	System fault detection

1. LSE missing or over frequency detection (> 2 MHz), Glitch filter (> 2 MHz).

Response to tamperers

Each source of tamper in the device can be configured to trigger the following events:

- Generate an interrupt, capable of waking up the device from Stop and Standby modes (see TAMPxMSK bits in TAMP_CR2 register).
- Generate a hardware trigger for the low-power timers.
- Erase device secrets if the corresponding TAMPxNOER bit is cleared in TAMP_CR2 (for tamper pins) or TAMP_CR3 (for internal tamper). These erasable secrets are:
 - symmetric keys stored in backup registers (x32), in SAES, AES, HASH, CCB^(a), and OTFDEC (encrypted flash memory regions are read as 0)
 - asymmetric keys stored in PKA SRAM, erased when V_{DD} is present
 - other secrets stored in SRAM2 and CPU instruction cache memory (SRAM2 erased when V_{DD} is present)
 - nonvolatile information used to derive the DHUK in SAES is 0-ed until complete SRAM2 erase
 - Backup SRAM (depending on configuration bit), erased when V_{DD} is present

Read/write accesses by software to all these secrets can be blocked, by setting the BKBLOCK bit in TAMP_CR2. The device secrets access is possible only when BKBLOCK is cleared, and no tamper flag is set for any enabled tamper source.

If V_{DD} is not present, the secrets that are erased when V_{DD} is present are erased only at the next V_{DD} power on.

a. Available only on STM32H543/553xx devices.

Note: Device secret erase is also triggered by setting the *BKERASE* bit in *TAMP_CR2*, or by performing a *PRODUCT_STATE* regression as defined in [Section 3.11.2](#).

Device secrets are not reset by a system reset or when the device wakes up from Standby mode.

Software filtering mechanism

Each tamper source can be configured not to launch an immediate erase, by setting the corresponding *TAMPxNOER* bit in *TAMP_CR2* (for external tamper pin) or *TAMP_CR3* (for internal tamper).

In such situation, when the tamper flag is raised, access to below secrets is blocked until all tamper flags are cleared:

- DHUK in AES: fixed to a dummy value
- Backup registers, backup SRAM, SRAM2: read as 0, write-ignored
- AES, SAES, CCB^(a), and HASH peripherals: automatically reset by RCC
- PKA peripheral: reset, with memory use blocked (meaning PKA not usable)

Once the application, notified by the tamper event, analyzes the situation, there are two possible cases:

- The application launches secrets erase with a software command (confirmed tamper).
- The application clears the flags to release secrets blocking (false tamper).

Note: If the tamper software fails to react to such a tamper flag, an IWDG reset triggers an automatic erasing of secrets.

Tamper detection and low-power modes

The effect of low-power modes on a tamper detection are summarized in [Table 20](#).

Table 20. Effect of low-power modes on TAMP

Mode	Description
Sleep	No effect on tamper detection features. TAMP interrupts cause the device to exit the Sleep mode.
Stop	No effect on tamper detection features, except for level detection with filtering and active tamper modes that remain active only when the clock source is LSE or LSI. Tamper events cause the device to exit the Stop mode.
Standby	No effect on tamper detection features, except for level detection with filtering and active tamper modes which remain active only when the clock source is LSE or LSI. Tamper events cause the device to exit the Standby mode.

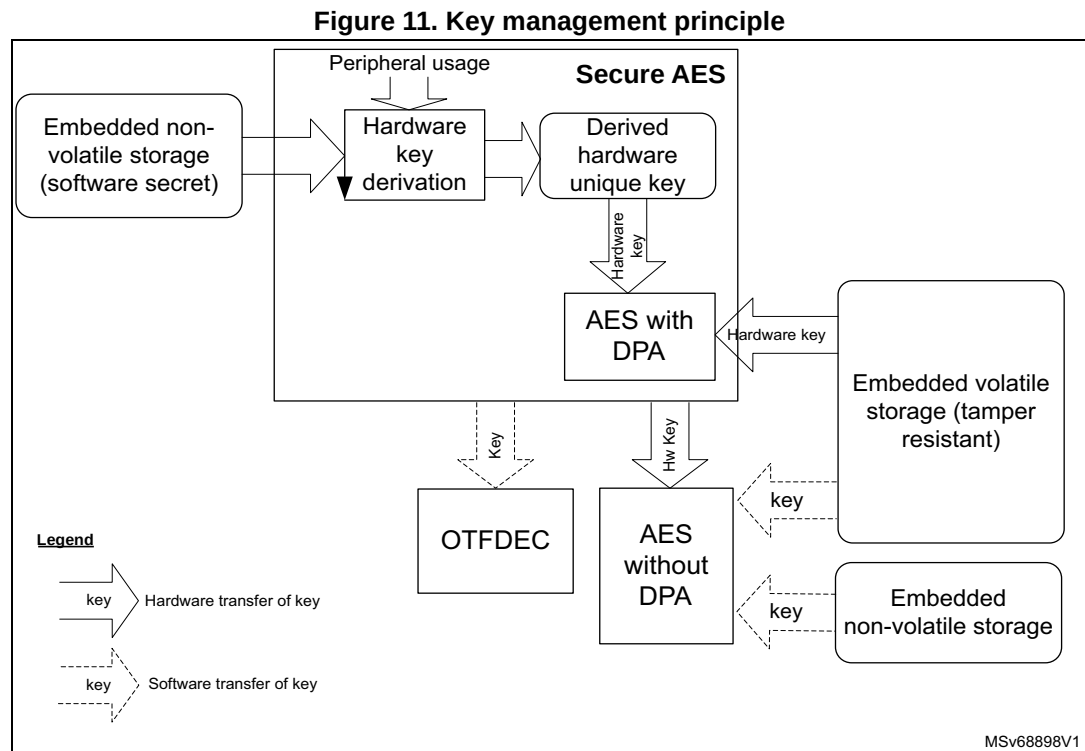
3.9 Secure storage

A critical feature of security systems is how long-term keys are stored, protected, and provisioned. Such keys are typically used for loading a boot image, or handling of critical user data.

a. Available only on STM32H543/553xx devices.

Figure 11 shows how key management service application can use the AES engine, for example, to compute external image decryption keys. Nonvolatile keys are stored in the flash memory in a dedicated area including access control (see Section 3.7.1), while volatile key storage consists in the battery-powered, tamper-protected SRAM or registers in TrustZone-aware TAMP.

Figure 11 also shows keys that are manipulated by software (like OTFDEC keys), or keys that are managed only by hardware (like DHUK). More information on those hardware keys can be found in Section 3.9.1.



Details on tamper protection can be found in Section 3.8.3, while TAMP TrustZone features are briefly described in Section 3.6.5.

3.9.1 Hardware secret key management

As shown in the previous figure, the devices propose a better protection for application keys, using hardware secret keys. These AES keys can be made usable to the application, without exposing them in clear-text (unencrypted). Such keys also become immediately unusable in case of tamper.

There are three different sources of hardware secret keys:

- DHUK: derived keys based on 256-bit nonvolatile device unique secret in flash memory. The flash memory provides value provisioned during product manufacturing (called RHUK). The generation of DHUK key takes into account the TrustZone state,

the OBK-HDPLx (temporal isolation counter), the EPOCH (regression counter allowing antireplay protection) and key use state (KMOD).

- BHK: 256-bit application key stored in tamper-resistant volatile storage in TAMP
This key is written at boot time, then read/write locked to the application until the next reset.
- XORK: result of an XOR of BHK and DHUK

Those keys can be used in the following modes:

- as a normal key, loading in write-only key registers (software key mode)
- as an encryption/decryption key for another key, to be used in the DPA-resistant SAES (wrapped key mode)
- as an encryption/decryption key for another key, to be used in a faster AES engine (shared key mode)

3.9.2 Unique ID

The devices store a 96-bit ID that is unique to each device (see [Section 67.1: Unique device ID register \(96 bits\)](#)).

Application services can use this unique identity key to identify the product in the cloud network, or make it difficult for counterfeit devices or clones to inject untrusted data into the network.

Alternatively, the 256-bit device unique key (DHUK) can be used (see [Section 3.9.1](#)).

3.10 Crypto engines

The devices implement state-of-the-art cryptographic algorithms featuring key sizes and computing protection as recommended by national security agencies such as NIST for the U.S.A, BSI for Germany or ANSSI for France. Those algorithms are used to support privacy, authentication, integrity, entropy and identity attestation.

The embedded crypto engines reduce weaknesses on the implementation of critical cryptographic functions, preventing, for example, the use of weak cryptographic algorithms and key sizes. They also enable lower processing times and lower power consumption when performing cryptographic operations, offloading those computations from Cortex-M33. This is especially true for asymmetric cryptography.

For product certification purposes, ST can provide certified device information on how these security functions are implemented and validated.

For more information on crypto engine processing times, refer to their respective sections in the reference manual.

3.10.1 Crypto engines features

[Table 21](#) lists the accelerated cryptographic operations available in the devices. Two AES accelerators are available (both can be reserved to secure application only).

Note: Additional operations can be added using firmware. The PKA can accelerate asymmetric crypto operations (like key pair generation, ECC scalar multiplication, point on curve check). See [Section 41: Public key accelerator \(PKA\)](#) for more details.

Table 21. Accelerated cryptographic operations

Operations	Algorithm	Specification	Length of keys (in bit)	Modes
Get entropy	RNG	NIST SP800-90B ⁽¹⁾	N/A	Software and hardware ⁽²⁾ modes running in parallel
Encryption, decryption	AES	FIPS PUB 197 NIST SP800-38A	128, 256	ECB, CBC, CTR ⁽³⁾
Authenticated encryption or decryption		NIST SP800-38C NIST SP800-38D	128, 256	GCM, CCM
Cipher-based message authentication code		NIST SP800-38D	128, 256	GMAC
Checksum	SHA-1	FIPS PUB 180-4	N/A	Digest 160-bit
Cryptographic hash	SHA-2		-	SHA-224, SHA-256 SHA2-384, SHA2-512
	SHA-3 ⁽⁴⁾			SHA3-224, SHA3-256 SHA3-384, SHA3-512
Keyed-hashing for message authentication	HMAC	FIPS PUB 198-1 IETF RFC 2104	Short, long (>64 bytes)	-
Encryption/decryption key-pair generation ⁽⁵⁾	RSA	IETF RFC 8017 NIST SP800-56B	Up to 4160	RSAS-OAEP
Signature ⁽⁵⁾ with hashing	RSA	IETF RFC 8017 FIPS PUB 186-4	Up to 4160	PKCS1-v1_5, PSS
Signature verification	ECDSA	ANSI X9.62 IETF RFC 7027 FIPS PUB 186-4	Up to 640	Refer to table 'Family of supported curves for ECC operations' in PKA section for details
Key agreement	ECDH	ANSI X9.42		

1. Certifiable using STMicroelectronics reviewed documents.
2. Random numbers distribution to SAES and PKA using a dedicated hardware bus.
3. ECB and CBC chaining modes protected against side-channel and timing attacks in SAES (see [Section 3.10.2](#)).
4. Available only on STM32H543/553xx devices.
5. Private key cryptography protected against side-channel and timing attacks.

Note: Binary curves, Edwards curves and Curve25519 are not supported by the PKA.

3.10.2 Secure AES co-processor (SAES)

The devices provide an additional on-chip hardware AES encryption and decryption engine, implementing countermeasures and mitigations against power and electromagnetic side-channel attacks.

Clocked by the system clock the SAES results in very good performance for a dpa resistant hardware accelerator. The SAES engine supports 128-bit or 256-bit key in electronic code book (ECB), cipher block chaining (CBC), (CTR), (GCM), (CCM), (GMAC) modes.

As shown in [Section 3.9](#), the SAES can be used for extra-secure on-chip storage for sensitive information. It can also be made secure-only.

For more information, refer to [Section 38: Secure AES coprocessor \(SAES\)](#).

3.10.3 On-the-fly decryption engine (OTFDEC)

The OTFDEC TrustZone-aware peripheral proposes on-the-fly decryption of encrypted images stored on external flash memory, connected through the OCTOSPI. This decryption process introduces almost no additional cycle overhead when the standard NOR flash memory is used. The OTFDEC can also be used to encrypt flash memory images on the device (for example to encrypt with a device unique secret key).

When a tamper event is confirmed in TAMP, all OTFDEC keys are erased and encrypted regions are read as 0 until the OTFDEC is properly initialized again.

An OTFDEC typical use is detailed in [Section 3.12.2](#). For more details on the peripheral programming, refer to [Section 42: On-the-fly decryption engine \(OTFDEC\)](#).

3.11 Product life cycle

A typical IoT device life cycle is summarized in [Figure 12](#). For each step, the devices propose secure life cycle management mechanisms embedded in the hardware.

Figure 12. Device life cycle security



Additional details on the various phases and associated transitions, found either at the vendor or end-user premises, are summarized in [Table 22](#).

Table 22. Main product life cycle transitions

Transitions	Description
Device manufacturing	ST creates new STM32 devices, always checking for manufacturing defects. During this process, the device is provisioned with ROM firmware, secure firmware install (SFI) unique key pair, and a public ID.
Vendor manufacturing	One (or more) vendor is responsible for the platform assembly, initialization, and provisioning before delivery to the end user. This end user can use the final product ("productization" transition) or he/she can use the platform for software development ("user provisioning" transition).
Productization	The end user gets a product ready for use. All security functions of the platform are enabled, the debugging/testing features are restricted/disabled, and unique boot entry to immutable code is enforced.
User provisioning	Platform vendor prepares an individual platform for development, not to be connected to a production cloud network.

Table 22. Main product life cycle transitions (continued)

Transitions	Description
Full progression	The product is returned for analysis to field return centers. Analysis is possible by opening it partially, or by launching a partial or a full regression. Such accesses require provisioning, including certificates provided by the vendor.
Field return	These are one-way transitions, with devices kept in user premises or returned to the manufacturer. In both cases, all data including user data is destroyed, therefore the devices lose the ability to operate securely (like connecting to a managed IoT network). This Field return is valid including when PRODUCT_STATE is Locked.

The features described hereafter contribute to secure the device life cycle.

3.11.1 Product configurations and security services

The product is provisioned with ST security services in system flash, but can be configured to let the full control of the boot chain to OEMs. This is done thanks to the unique boot entry (FLASH_OPTSR: BOOT_UBE) option byte, allowing to select between the security services (STiRoT: ST immutable root of trust), or OEM boot implementation (OEMiRoT: OEM immutable root of trust) to be installed in the user part of the flash memory.

Security services are provisioned by ST in system flash (immutable). They provide the root of trust of the platform managing the verification and the update of the first updatable code (uRoT).

3.11.2 Life cycle management

The product life cycle allows to control access to different assets (code and data) of the product, including during development, manufacturing, and after sales. It allows to provision the product with different distribution models, taking care of the code and data provisioned.

Table 23. Typical product life cycle phases

PRODUCT_STATE + DebugState		Debug (default configuration)	Comments
Open	Device open	Secure ⁽¹⁾ and nonsecure	Allows to develop the product, as it provides the debug of the code. Boot address must target a secure area when TrustZone is enabled. Using the boot pin allows to launch the bootloader. Using boot pin and BOOT_UBE allows to launch user flash code through STiRoT.
Provisioning	Debug partially opened (only nonsecure)	HDPL3 + Nonsecure only	Allows to manage the provisioning of the product (partial or full). It allows to launch secure firmware Install, or bootloader to provision the product. Boot address must target a secure area when TrustZone is enabled. Boot on SRAM is not permitted.
iRoT-Provisioned	Debug partially opened (only nonsecure)	HDPL3 + Nonsecure only	Assumes that immutable root of trust is installed, including its configuration (code, option bytes, secure storage). Boot address must target a secure area when TrustZone is enabled. Boot on SRAM is not permitted.

Table 23. Typical product life cycle (continued)phases (continued)

PRODUCT_STATE + DebugState		Debug (default configuration)	Comments
TZ-Closed	Debug partially opened (only nonsecure)	HDPL3 + Nonsecure only	Assumes that the Secure OS is installed in TrustZone (this state exist only if TZEN = 0xB4). Debug is opened for nonsecure applications
Closed	Debug closed	NoDebug + debug authentication control	Assumes the product configuration as finalized. It allows supporting debug authentication for in-the-field repair (read the dedicated application note).
Locked	Debug locked	None	Assumes the product configuration as finalized. Debug authentication is not permitted. The product is definitively in this state. Nevertheless, the field return procedure allows launching RMA state.
NS-Regression	Debug closed	NoDebug	This is a temporal (but nonvolatile) state to manage the partial regression to the TZ-Closed, removing all nonsecure code and data including in secure storage.
Regression	Debug closed	NoDebug	This is a temporal state (but nonvolatile) to manage the full regression to Open state, removing all user flash code and data including in secure storage (HDPL1 to HDPL3).
Debug Constrained	Debug opened following debug authentication permissions	Depends upon permissions	This is a temporal state (until power-on-reset) to manage debug based on the permissions acquired by the debug authentication protocol.

1. Debug is not available when executing RSS code.

STMicroelectronics does not perform analysis on defective parts in closed or locked states.

The supported transitions, summarized in [Figure 13](#), can be requested (when available) through the debug interface or via the system bootloader.

Figure 13. PRODUCT_STATES (simplified TrustZone activated view)

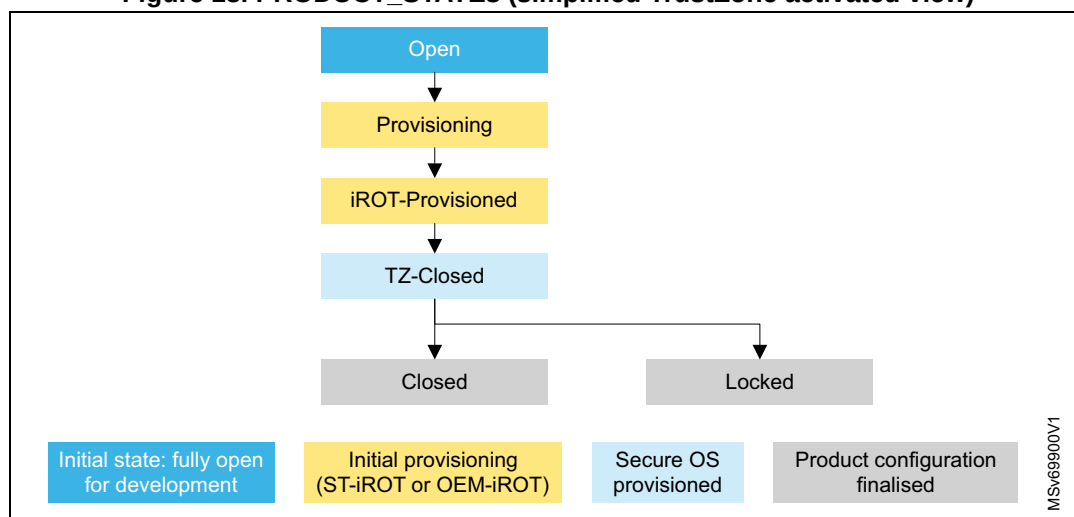
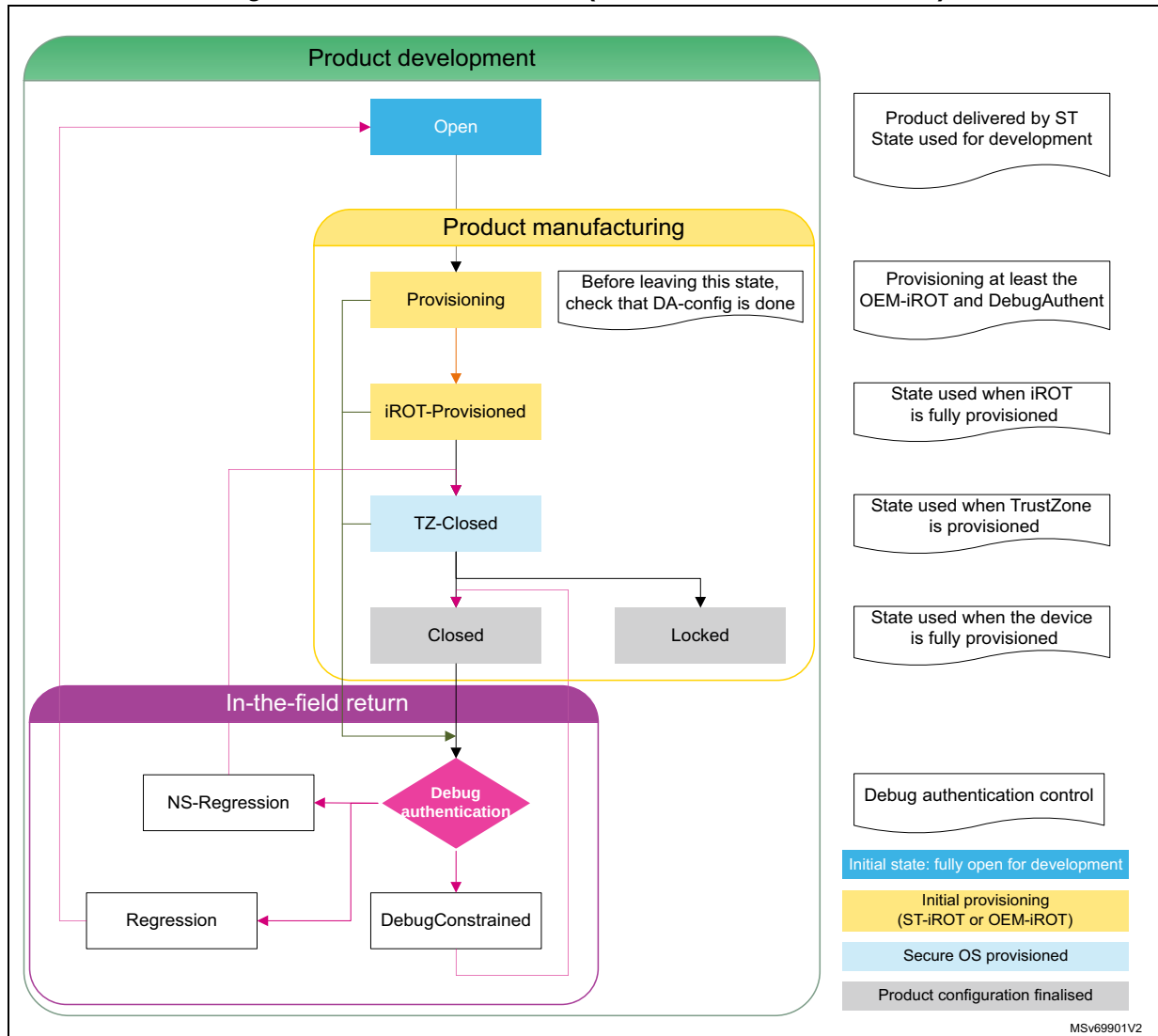


Figure 14. PRODUCT_STATES (full TrustZone activated view)



The full view of the life cycle includes the debug authentication part, to help product maintenance to manage field returns (in-the-field centers). The debug authentication control is ensured thanks to a protocol based on Arm PSA ADAC specification. It allows controlling that the host has a trusted certificate with permissions. Permissions definition allows a lot of flexibility, in our default model we consider: Full or Partial regression, or to open the debug for nonsecure.

When TrustZone is disabled, the state TZ-Closed does not exist, and the debug authentication is based on a password authentication method. For details on the debug authentication feature, refer to the corresponding application note.

The protocol includes a "DISCOVER" command allowing to get basic informations of the target: manufacturer, product name, device identifier, protocol version supported and crypto scheme for authentication (this help selecting between certificates and password).

3.11.3 Recommended product settings

To ease the product maintenance (in-the-field), we recommend to take benefit of the feature called debug authentication control. This enables the maintenance of product activating the debug and makes possible to manage regressions while considering the security of the sensible information.

This implies the following actions:

- set the PRODUCT_STATE in Closed state when the product has been configured
- When debug authentication control is based on certificates (when TrustZone enabled), **provision the OEM-PublicKey and the SOC-Mask**. Setting the SOC-Mask to propose only the nonsecure-debug and partial and full regression guaranty to be compliant with the Arm PSA security model.
- When debug authentication control is based on password (when TrustZone is disabled), only the HASH of the password must be provisioned.

A dedicated application note is available to help configuring the platform, the host tools and to generate certificates.

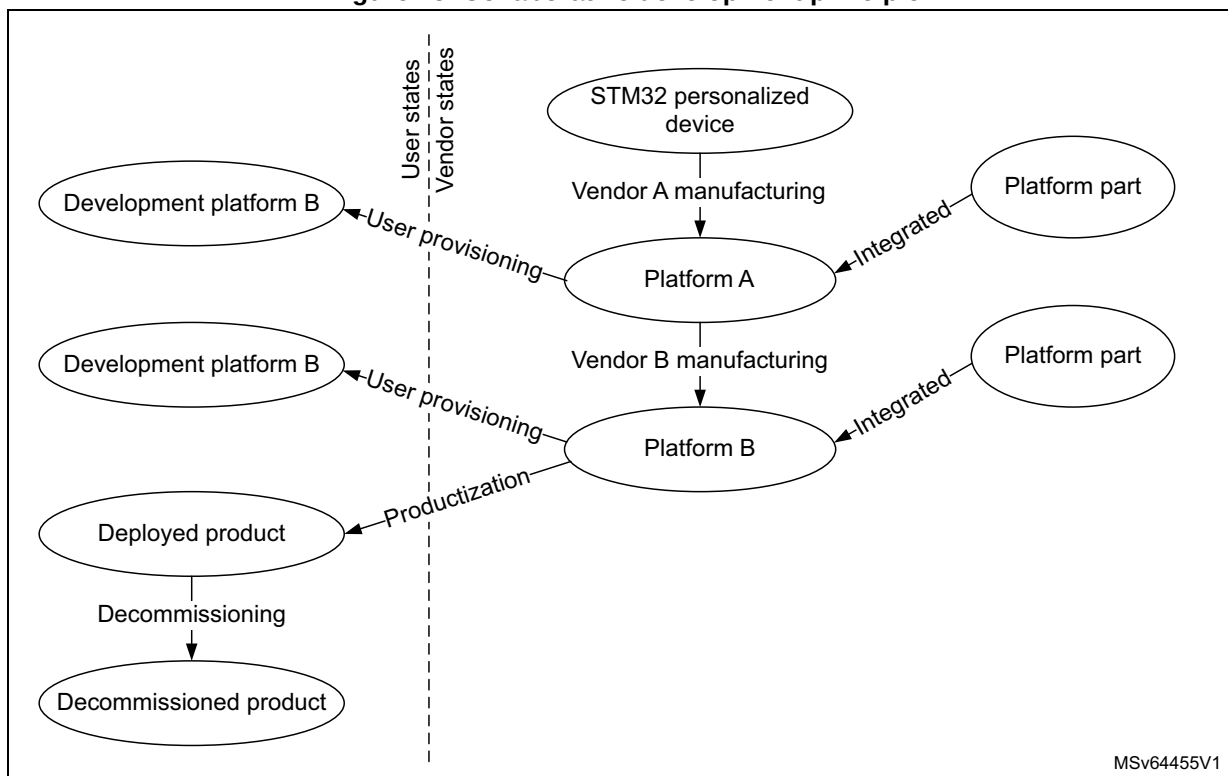
3.12 Software intellectual property protection and collaborative development

Thanks to software intellectual property protection and collaborative model, the devices allow the design of solutions integrating innovative third-party libraries.

Collaborative development is summarized in [Figure 15](#). Starting from a personalized device sold by STMicroelectronics, a vendor A can integrate a portion of hardware and software on a platform A, which can then be used by a vendor B, who does the same before deploying a final product to the end users.

Note: Each platform vendor can provision individual platforms for development not to be connected to a production cloud network ("Development Platform X").

Figure 15. Collaborative development principle



The features described hereafter contribute to securing the software intellectual property within such a collaborative development.

3.12.1 Software intellectual property protection

As described in [Section 3.11.2](#), the hardware PRODUCT_STATE mechanism automatically controls the accesses to secrets provisioned in the device. The protection of these secrets are defined in [Table 24](#).

Table 24. Software intellectual property protection with PRODUCT_STAT

PRODUCT_STATE protection level		Secrets protection
Open	Device open	No special protections.
Provisioning	Debug partially opened (only nonsecure)	All areas protected with HDPL1 cannot be dumped, debugged or traced. The iRoT can setup a higher level of protection.
iRoT-Provisioned	Debug partially opened (only nonsecure)	All areas protected with HDPL1 cannot be dumped, debugged or traced. The iRoT can setup a higher level of protection.
TZ-Closed	Debug partially opened (only nonsecure)	All peripherals and memories mapped as secure during secure boot cannot be dumped, debugged or traced
Closed	No debug	No debug possible except through debug authentication.
Locked	No debug	All data and code stored in the device or encrypted in external flash memory cannot be dumped clear-text, debugged or traced.

3.12.2 Software intellectual property protection with OTFDEC

As described in [Section 3.10.3](#), the OTFDEC associated with the OCTOSPI is able to decrypt on the fly, the encrypted images stored in external SPI flash memories.

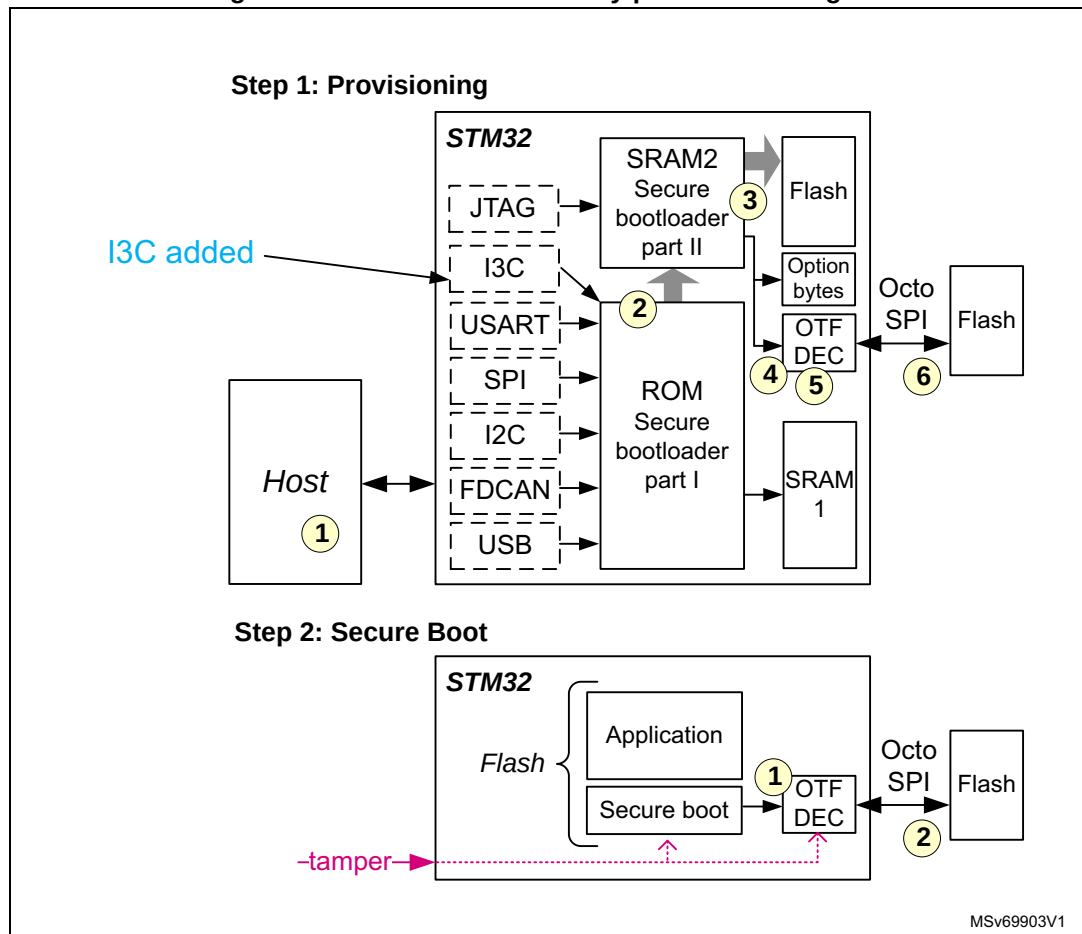
Thanks to this feature, the devices allow the installation of intellectual properties, in one of the following ways:

- over the air, with the image already encrypted with a key provisioned in the device
- through a provisioning host located in a trusted or a non-trusted environment/facility

[Figure 16](#) illustrates this last case, with the provisioning, in a non-trusted environment, of software intellectual properties both in the embedded flash memory and in an external SPI flash memory (encrypted).

Note: Since the OTFDEC uses the AES in counter mode (CTR) to achieve the lowest possible latency, each time the content of one encrypted region is changed, the corresponding cryptographic context (key or initialization vector) must be changed. This constraint makes OTFDEC suitable to decrypt read-only data or code, stored in external NOR flash memory.

Figure 16. External flash memory protection using SFI



Provisioning

Assuming the device is virgin, the first step is to provision both flash memories, as detailed below:

1. The user creates a SFI image, composed of:
 - encrypted internal firmware and data (including external flash memory drivers)
 - encrypted external firmware and data AES key (up to 4)
 - encrypted external firmware and data image
2. The secure bootloader stored in the system memory, loads the second part of the secure bootloader in SRAM2, through the supported communication ports (USART, SPI, I2C, FDCAN, USB, I3C, and JTAG). This second part runs in the secure SRAM2 and is responsible for executing the SFI process, applying the SFI protocol thanks to the commands received through the above mentioned supported communication ports.
3. The internal flash memory is programmed with decrypted option bytes, internal firmware and data, and external firmware and data AES keys. Alternatively, device unique external firmware AES keys can be used instead of such global keys.
4. The OTFDEC is properly initialized with encrypted region information, including the corresponding external firmware and data AES key.
5. Running the SFI process, chunks of encrypted external firmware and data image are decrypted in the device, then re-encrypted in the OTFDEC.

6. After a chunk OTFDEC re-encryption, the user external flash memory programmer is responsible for programming the last encrypted chunks to the external SPI flash memories through the OCTOSPI.

Secure boot

After provisioning, each time the device initializes on a trusted firmware, the following actions are required:

1. Secure-boot firmware executes, programming the external firmware and data AES keys to the OTFDEC write-only key registers, along with the other needed information.
2. The application reads or executes the encrypted external firmware and data through the OCTOSPI in memory mapped mode, unless a tamper event is detected. In this case, all OTFDEC keys are erased and encrypted regions are read as 0 until the OTFDEC is properly initialized.

For more information on SFI solutions for the devices, refer to the AN4992 “*STM32 MCUs secure firmware install (SFI) overview*”.

3.12.3 Other software intellectual property protections

The device additional protections to software intellectual property are:

- Invasive attacks such as physical tampering or perturbation are countered by detection then decommissioning of the device before the detected attack succeeds.
- Noninvasive attacks, such as side channel attacks, are countered by not leaking secret information via side channels (such as timing, power, and EM emissions).

4 Boot modes

At startup, a BOOT0 pin and NSBOOTADD[31:8]/SECBOOTADD[31:8] option bytes are used to select the boot memory address that includes:

- Boot from any address in user flash memory
- Boot from system memory:
 - Bootloader
 - ST immutable root of trust (STiRoT)
 - Root security service (RSS)
 - Debug authentication library (RSS-DA)

Embedded bootloader

The embedded bootloader is located in the system memory, programmed by ST during production. It is used to reprogram the flash memory by using USART, I2C, I3C, SPI, FDCAN, or USB_FS in device mode through the DFU (device firmware upgrade).

Refer to AN2606 “*Introduction to system memory boot mode on STM32 MCUs*”.

Embedded root security services (RSS)

The embedded RSS is located in the secure information block, programmed by ST during production.

Refer to AN4992 “*Introduction to secure firmware install (SFI) for STM32 MCUs*”.

Embedded immutable root of trust (STiRoT)

The embedded STiRoT in the system memory, is programmed by ST during production. STiRoT is the immutable root of trust managing the secure boot and secure install of the first updatable level to execute in a boot sequence.

Refer to AN6007 “*Getting started with STiRoT (ST immutable Root of Trust) for STM32H5 MCUs*”.

Embedded debug authentication (ST-DA)

The embedded ST-DA in the system memory, is programmed by ST during production. ST-DA is the library that manages the debug authentication protocol by allowing to securely reopen the debug or to launch regressions on secured products in the field.

Refer to AN6008 “*Getting started with debug authentication (DA) for STM32 MCUs*”.

4.1 STM32H523/43/62/63xx boot modes

[Table 25](#) and [Table 26](#) provide the detail of the boot mode when the TrustZone is, respectively, disabled (TZEN = 0xC3) and enabled (TZEN = 0xB4).

Table 25. Boot mode when TrustZone is disabled (TZEN = 0xC3) - STM32H523/43/62/63xx devices

PRODUCT_STATE	BOOT0 pin	Boot address option-byte selection	Boot area	ST programmed default value
Open	0	NSBOOTADD[31:8]	Boot address defined by user option byte NSBOOTADD[31:8]	Flash: 0x0800 0000
	1	NA	Bootloader	Bootloader
Provisioning	x	NA	RSS	RSS
Provisioned, Closed, Locked	x	NSBOOTADD[31:8]	Boot address defined by user option byte NSBOOTADD[31:8]	Flash: 0x0800 0000

Table 26. Boot mode when TrustZone is enabled (TZEN = 0xB4) - STM32H523/43/62/63xx devices

PRODUCT_STATE	BOOT0 pin	Boot address option-byte selection	Boot area	ST programmed default value
Open	0	SECBOOTADD[31:8]	Boot address defined by user option byte SECBOOTADD[31:8]	Flash: 0x0C00 0000
	1	NA	Bootloader	Bootloader
Provisioning	x	NA	RSS	RSS
Provisioned, TZ_Closed, Closed, Locked	x	SECBOOTADD[31:8]	Boot address defined by user option byte SECBOOTADD[31:8]	Flash: 0x0C00 0000

Note: The *BOOT_UBE* is available only on STM32H533/53/73xx devices. Refer to [Section 4.2](#).

When TrustZone is enabled (TZEN = 0xB4), the boot space must be in secure area. The SECBOOTADD0[24:0] option bytes are used to select the boot secure memory address. A unique boot entry option can be selected by setting the SECBOOT_LOCK option bit.

4.2 STM32H533/53/73xx boot modes

[Table 27](#) and [Table 28](#) provide the detail of the boot mode products when the TrustZone is, respectively, disabled (TZEN = 0xC3) and enabled (TZEN = 0xB4).

Table 27. Boot mode when TrustZone is disabled (TZEN = 0xC3) - STM32H533/53/73xx devices

PRODUCT_STATE	BOOT0 pin	BOOT_UBE FLASH_OPTSR [29:22]	Boot address option-byte selection	Boot area	ST programmed default value
Open	0	NA	NSBOOTADD[31:8]	Boot address defined by user option byte NSBOOTADD[31:8]	Flash: 0x0800 0000
	1	NA	NA	Bootloader	Bootloader
Provisioning	x	NA	NA	RSS	RSS
Provisioned, Closed, Locked	x	NA	NSBOOTADD[31:8]	Boot address defined by user option byte NSBOOTADD[31:8]	Flash: 0x0800 0000

Table 28. Boot mode when TrustZone is enabled (TZEN = 0xB4) - STM32H533/53/73xx devices

PRODUCT_STATE	BOOT0 pin	BOOT_UBE FLASH_OPTSR [29:22]	Boot address option-byte selection	Boot area	ST programmed default value
Open	0	x	SECBOOTADD [31:8]	Boot address defined by user option byte SECBOOTADD[31:8]	Flash: 0x0C00 0000
	1	0xB4	NA	Bootloader	Bootloader
	1	0xC3	NA	STiRoT	STiRoT
Provisioning	x	NA	NA	RSS	RSS
Provisioned, TZ_Closed, Closed, Locked	x	0xC3	STiRoT	STiRoT	STiRoT
		0xB4	SECBOOTADD [31:8]	Boot address defined by user option byte SECBOOTADD[31:8]	Flash: 0x0C00 0000

When TrustZone is enabled (TZEN = 0xB4), the boot space must be in a secure area. The SECBOOTADD[24:0] option bytes are used to select the boot secure memory address. A unique boot entry option can be selected by setting the SECBOOT_LOCK option bit.

5 Global TrustZone® controller (GTZC)

5.1 GTZC introduction

The global TrustZone controller (GTZC) block contains the following subblocks:

- **TZSC:** TrustZone security controller
Defines the secure/privileged state of slave peripherals, and controls the subregion area size and properties for the watermark memory peripheral controller (MPCWM). The TZSC informs some peripherals (such as RCC or GPIOs) about the secure status of each securable peripheral, by sharing with RCC and I/O logic.
- **MPCBB:** memory protection controller - block based
Configures the internal RAM in a TrustZone-system product having segmented SRAM (pages of 512 bytes) with programmable-security and privileged attributes.
- **TZIC:** TrustZone illegal access controller
Gathers all illegal access events in the system, and generates secure interrupts towards NVIC.

These subblocks are used to configure TrustZone system security in a product having bus agents with programmable-security and privileged attributes such as:

- on-chip RAM with programmable secure and/or privileged blocks (pages)
- AHB and APB peripherals with programmable security and/or privileged access
- off-chip memories with secure and/or privileged areas

5.2 GTZC main features

- Three independent 32-bit AHB interfaces for TZSC, TZIC, and MPCBB
- TZIC accessible only with secure transactions
- Secure and nonsecure access supported for privileged and unprivileged parts of TZSC and MPCBB
- Set of registers to define product security settings:
 - Secure/privileged blocks for internal SRAMs
 - Secure/privileged regions for external memories and internal backup SRAM
 - Secure/privileged access mode for securable peripherals
 - Secure/privileged access mode for securable masters

GTZC TrustZone system architecture

The Armv8-M supports security per TrustZone-M model with isolation between:

- a secure world, where usually security sensitive applications are run and critical resources are located
- a nonsecure or public world (such as the usual non secure operating system and user space)

The TrustZone architecture is extended beyond AHB and Armv8-M with:

- AHB/APB bridge used as secure gate to block or propagate secure/nonsecure and privileged/unprivileged transaction towards APB agents

- PPC (peripheral protection controller) used as secure gate to block or propagate secure/nonsecure and privileged/unprivileged transaction towards AHB agents
- TrustZone block-based MPC firewalls used as secure gate to filter secure/nonsecure, privileged/unprivileged access towards internal SRAMs
- TrustZone watermark MPC firewalls used as secure gate to filter secure/nonsecure, privileged/unprivileged access towards external memories

AHB and APB peripherals can be categorized as:

- **privileged:** peripherals protected by an AHB/APB firewall stub controlled from TZSC to define privilege properties
- **secure:** peripherals always protected by an AHB/APB firewall stub, always secure (such as TZIC)
- **securable:** peripherals protected by an AHB/APB firewall stub that is controlled from TZSC to define security properties (optional)
- **nonsecure and unprivileged:** peripherals connected directly to AHB/APB interconnect without any secure gate
- **TrustZone-aware:** peripherals connected directly to the AHB or APB bus and implementing a specific TrustZone behavior (such as a subset of registers being secure). TrustZone-aware AHB masters always drive HNONSEC signal according to their security mode (such as Armv8-M core or DMA)

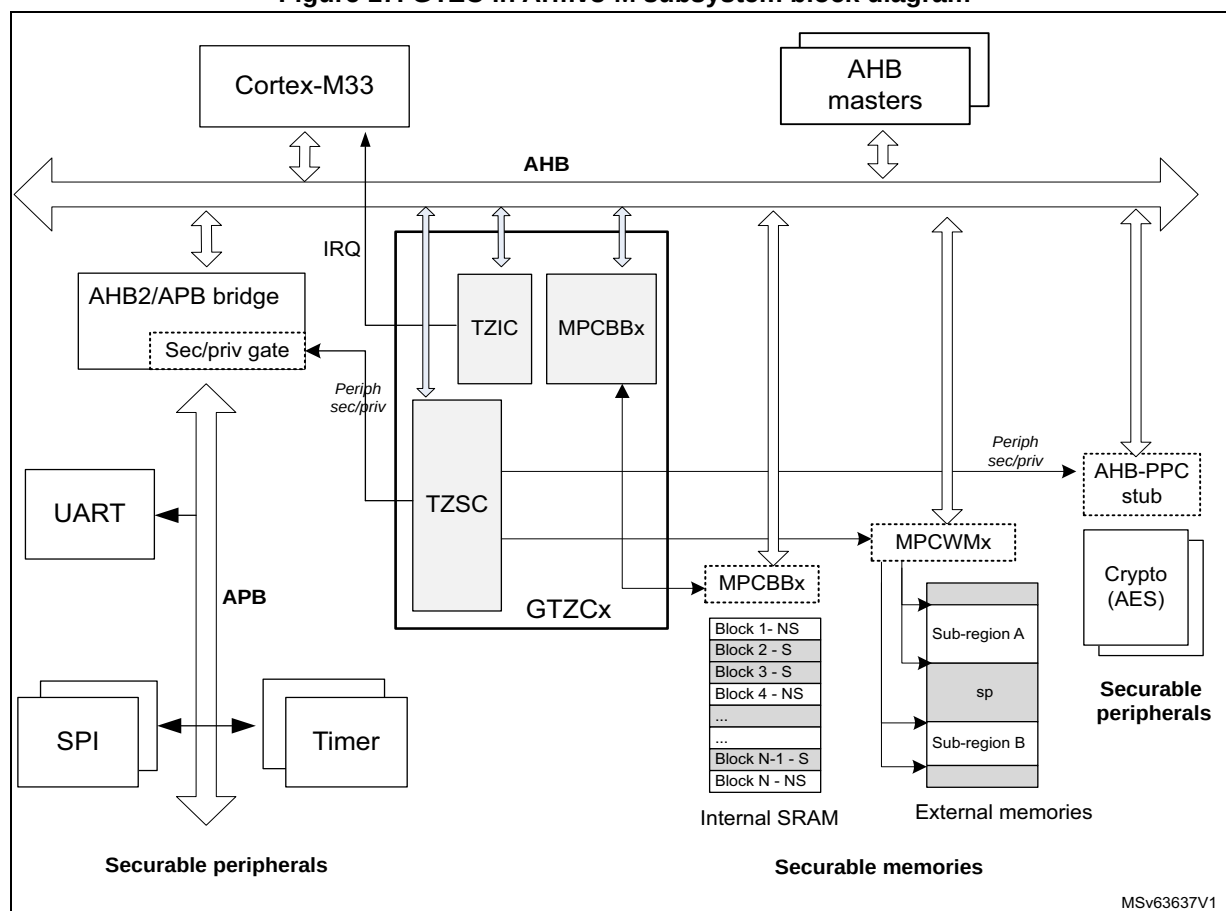
AHB securable masters can be configured in the TZSC to be secure/nonsecure and/or privileged/unprivileged.

Application information

The TZSC, MPCBB, and TZIC can be used in one of the following ways:

- programmed only during secure boot, locked and not changed afterwards
- dynamically re-programmed when using specific application code or secure kernel (microvisor). When not locked, MPC secure blocks or region size can be changed by secure software executing from the secure FLASH region or secure SRAM. The same applies to the GTZC1_TZSC_SECCFGRx/PRIVCFGRx registers that define the secure/privileged state of each peripheral.

Figure 17. GTZC in Armv8-M subsystem block diagram



The STM32H5 devices embed one instance of GTZC.

Table 29. GTZC features

GTZC subblocks	GTZC1
TZSC	X
TZIC	X
MPCBB subblock (number of MPCBBs)	X (3)

[Table 30](#) shows the address offset of GTZC subblocks versus GTZC base address (refer to [Section 2.3](#) for GTZC1 base address).

Table 30. GTZC1 subblock address offset

GTZC1 subblock	Address offset
GTZC1_TZSC	0x0
GTZC1_TZIC	0x400
GTZC1_MPCBB1	0x800
GTZC1_MPCBB2	0xC00
GTZC1_MPCBB3	0x1000

[Table 31](#) and [Table 32](#) describe the characteristics of the available MPCWMs.

Table 31. MPCWM resource assignment

GTZC	MPC	Target memory interface	Number of sec/nonsec and priv/unpriv regions	Watermark granularity (bytes)
GTZC1	MPCWM1	OCTOSPI1	2	128 K
	MPCWM2	FMC_NOR bank	2	128 K
	MPCWM3	FMC_NAND bank	1	128 K
		FMC_SDRAM bank 1 ⁽¹⁾	1	128 K
	MPCWM4	BKPSRAM	1	32
		FMC_SDRAM_bank 2 ⁽¹⁾	1	128 K

1. Not available on STM32H523/33xx devices.

Table 32. MPCWM3 and MPCWM4 (subregions A and B)

GTZ	MPC	Target memory interface	Subregion
GTZC1	MPCWM3	FMC_NAND bank	A
		FMC_SDRAM bank 1 ⁽¹⁾	B
	MPCWM4	BKPSRAM	A
		FMC_SDRAM_bank 2 ⁽¹⁾	B

1. Not available on STM32H523/33xx devices.

[Table 33](#) and [Table 35](#) describe the characteristics of the available MPCBBs.

Table 33. MPCBB resource assignment (STM32H562/63/73xx devices)

GTZC	MPC	Resource	Memory size (Kbytes)	Block size (bytes)	Number of blocks	Number of super-blocks
GTZC1	MPCBB1	SRAM1	256	512	512	16
	MPCBB2	SRAM2	64		128	4
	MPCBB3	SRAM3	320		640	20

Table 34. MPCBB resource assignment (STM32H543/53xx devices)

GTZC	MPC	Resource	Memory size (Kbytes)	Block size (bytes)	Number of blocks	Number of super-blocks
GTZC1	MPCBB1	SRAM1	128	512	256	8
	MPCBB2	SRAM2	80		160	5
	MPCBB3	SRAM3	96		192	6

Table 35. MPCBB resource assignment (STM32H523/33xx devices)

GTZC	MPC	Resource	Memory size (Kbytes)	Block size (bytes)	Number of blocks	Number of super-blocks
GTZC1	MPCBB1	SRAM1	128	512	256	8
	MPCBB2	SRAM2	80		160	5
	MPCBB3	SRAM3	64		128	4

5.4 GTZC functional description

5.4.1 GTZC block diagram

[Figure 18](#) describes the combined feature of TZSC, MPCBB, and TZIC. Each subblock is controlled by its own AHB configuration port.

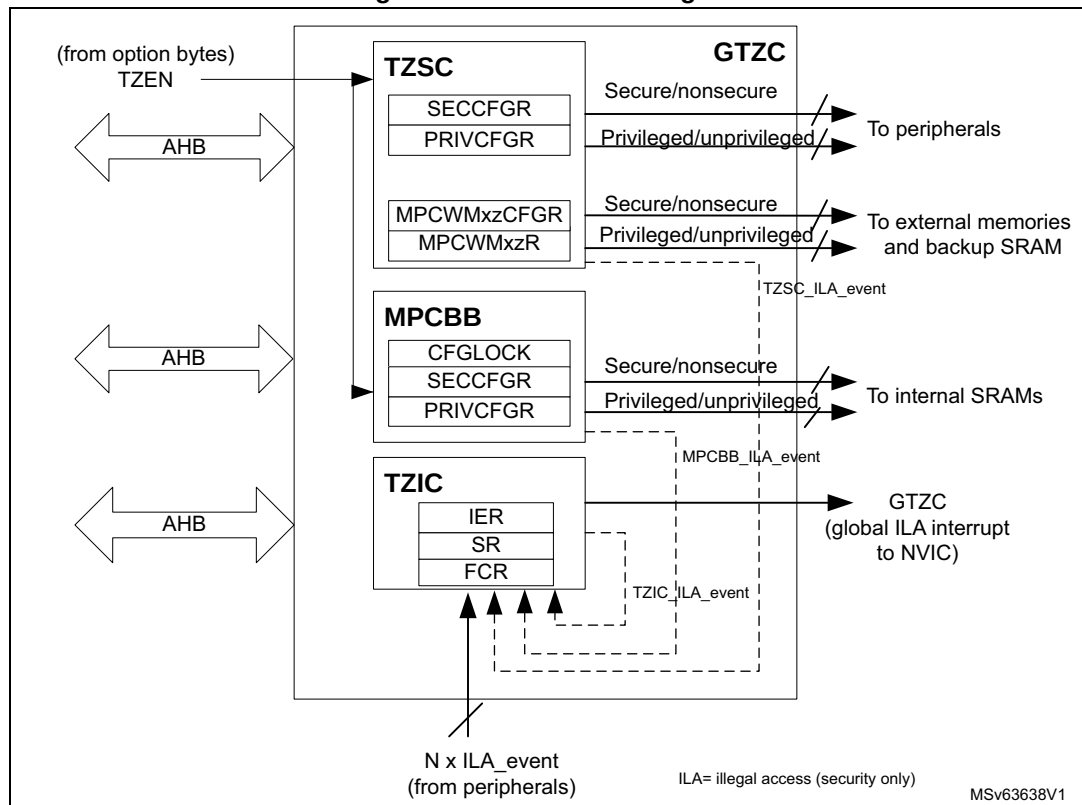
The TZSC defines which peripheral is secure and/or privileged. The privileged configuration bit of a peripheral can be modified by a secure privileged transaction when the peripheral is configured as secure. Otherwise, a privileged transaction (nonsecure) is sufficient.

On the opposite, the secure configuration bit of a peripheral can be modified only with a secure privileged transaction if the peripheral is configured as privileged. Otherwise, a secure transaction (unprivileged) is sufficient.

The secure configuration bit of a given RAM block can be modified only with a secure privileged transaction if the same block is configured as privileged. Otherwise, a secure transaction (unprivileged) is sufficient.

The TZIC gathers illegal events generated within the system when an illegal access is detected. TZIC can then generate a secure interrupt towards the CPU if needed.

Figure 18. GTZC block diagram



5.4.2 Illegal access definition

Three different types of illegal access exist:

- Illegal nonsecure access

Any nonsecure transaction trying to write a secure resource is considered as illegal, hence the addressed resource generates an illegal access interrupt for illegal write access and a bus error for illegal fetch access. There are some exceptions on secure and privileged configuration registers: the latter ones authorize non secure read access to secure registers (see GTZC1_TZSC_SECCFGRx/GTZC1_TZSC_PRIVCFGRx).

- Illegal secure access

Any secure transaction trying to access nonsecure block in internal block-based SRAM or watermarked memory is considered as illegal.

A correct TZIC setting allows the capture of the associated event and then generates the GTZC_IRQn interrupt to the NVIC. This applies for read, write, and execute access.

Concerning the MPCBB controller, there is an option to ignore secure data read/write access on nonsecure SRAM blocks, by setting the SRWILADIS bit in the GTZC1_MPCBBz_CR register. Secure read and write data transactions are then

allowed on nonsecure SRAM blocks, while secure execution access remains not allowed.

Any secure execute transaction trying to access a nonsecure peripheral register is considered as illegal and generates a bus error.

- Illegal unprivileged access

Any unprivileged transaction trying to access a privileged resource is considered as illegal. There is no illegal access event generated for illegal read and write accesses. The addressed resource follows a silent-fail behavior, returning all 0 data for read and ignoring any write. No bus error is generated. A bus error is generated when an unprivileged execute transaction tries to access a privileged memory.

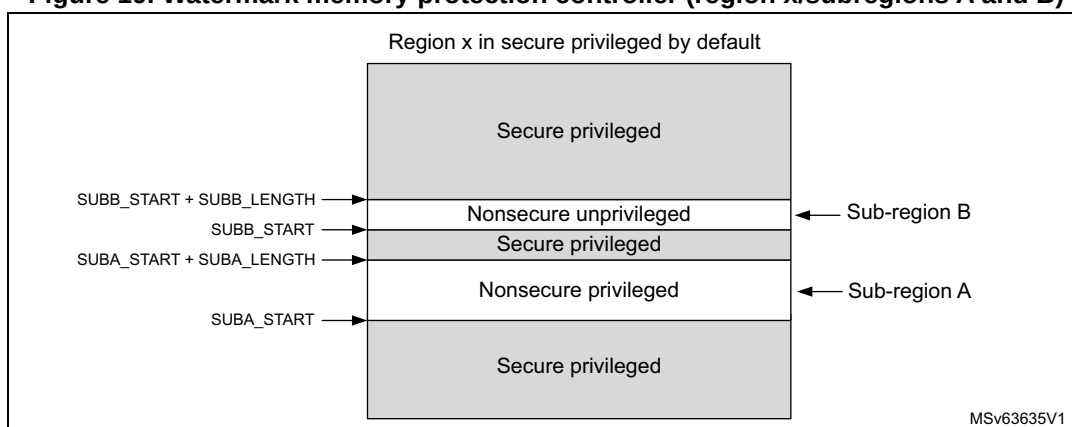
5.4.3 TrustZone security controller (TZSC)

The TZSC is composed of a configurable set of registers, providing the following features:

- Control of secure and privileged state for all peripherals, done through:
 - GTZC1_TZSC_SECCFGRx registers to control AHB/APB firewall stubs for the securable peripherals
 - GTZC1_TZSC_PRIVCFGRx registers to control AHB/APB firewall stubs for the privileged peripherals
- For watermark memory protection controller (external memories and backup SRAM), two independent regions can be defined and the following fields are used to program:
 - the start of the first protected subregion on the external memory/backup SRAM: SUBA_START[10:0]
 - the length of the first protected subregion on the external memory/backup SRAM: SUBA_LENGTH[11:0]
 - the start of the second protected subregion on the external memory/backup SRAM: SUBB_START[10:0]
 - the length of the second protected subregion on the external memory/backup SRAM: SUBB_LENGTH[11:0]

A control register for each subregion can be used to enable/disable the watermark memory protection controller, and to define the right attributes of each subregion.

Figure 19. Watermark memory protection controller (region x/subregions A and B)



In [Figure 19](#), region x represents the external memory or backup SRAM region (such as FMC bank, OCTOSPI1, or BKPSRAM). Secure and privileged attributes of subregions A

and B are independently configurable. When no subregions are defined or enabled on the region x, the default attribute of the region x is set as secure-privileged.

The following tables describe the secure/privileged properties of the common area of subregions A and B, when an overlap exists.

Table 36. Secure properties of subregions A and B

Subregion A	Subregion B	Properties of overlapped region A and B
Nonsecure	Nonsecure	Nonsecure
Nonsecure	Secure	Nonsecure
Secure	Nonsecure	Nonsecure
Secure	Secure	Secure

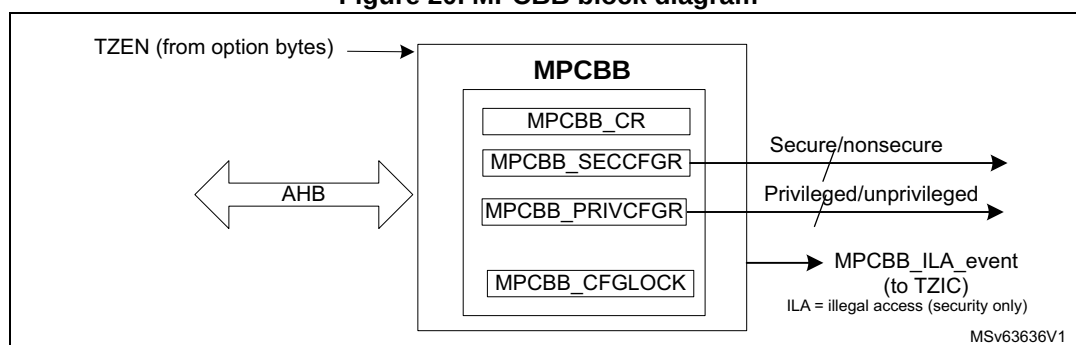
Table 37. Privileged properties of subregions A and B

Subregion A	Subregion B	Properties of overlapped region A and B
Unprivileged	Unprivileged	Unprivileged
Unprivileged	Privileged	Unprivileged
Privileged	Unprivileged	Unprivileged
Privileged	Privileged	Privileged

5.4.4 Memory protection controller - block based (MPCBB)

The MPCBB is composed of a configurable set of registers allowing to define security and privileged policy for internal SRAMs. The security and privileged policy can be individually configured per each 512-byte block.

Figure 20. MPCBB block diagram



To set up the MPCBB, the following actions are needed (for example at boot time):

- Secure firmware must define which memory blocks are secure by setting the correct bits in GTZC1_MPCBBz_SECCFGRx.
- Privileged firmware must define which memory blocks are privileged by setting the correct bits in GTZC1_MPCBBz_PRIVCFGRx.

An MPCBB super-block is made of 32 consecutive blocks. For each super-block, secure application can lock all related security/privileged bits using the correct bits in GTZC1_MPCBBz_CFGLOCK. This lock remains active until the next system reset.

Note: The block size is 512 bytes. The super-block size is $512 * 32 = 16$ Kbytes.

5.4.5 TrustZone illegal access controller (TZIC)

The TZIC concentrates all illegal access source events. It is used only when the system is TrustZone enabled (TZEN = 0xB4).

TZIC allows to trace (flag) the event that triggered the secure illegal access interrupt. Register masks (GTZC1_TZIC_IERx) are available to filter unwanted event. On an unmasked illegal event, TZIC generates the GTZC_IRQn interrupt to the NVIC.

For each illegal event source, a status flag and a clear bit exist (respectively within GTZC1_TZIC_SRx and GTZC1_TZIC_FCRx). The reset value of mask registers (GTZC1_TZIC_IERx) is such that all events are masked.

5.4.6 Power-on/reset state

The power-on and reset state of the TZSC clear to 0 all bits of GTZC1_TZSC_SECCFGRx and GTZC1_TZSC_PRIVCFGRx, meaning that all securable peripherals are respectively set to nonsecure and unprivileged.

For internal SRAMx (x = 1 to 3), all GTZC1_MPCBBz_SECCFGRx and GTZC1_MPCBBz_PRIVCFGRx are set:

- to 0xFFFF FFFF, making these internal memories block secure and privileged by default when TrustZone security is enabled at system level (TZEN = 0xB4)
- to 0x0000 0000, making these internal memories block nonsecure and unprivileged by default when TrustZone security is disabled at system level (TZEN = 0xC3)

For external memories and backup SRAM, GTZC1_TZSC_MPCWMxR registers are set:

- to 0x0000 0000, making these memories secure and privileged by default when TrustZone security is enabled at system level (TZEN = 0xB4).
- to 0x0800 0000, making these memories nonsecure and unprivileged by default when TrustZone security is disabled at system level (TZEN = 0xC3)

Secure boot code can then program the security settings, making components secure or not as needed.

5.5 GTZC interrupts

TZIC is a secure peripheral, thus it systematically generates an illegal access event when accessed by a nonsecure access. The MPCBB and TZSC are TrustZone-aware peripherals, meaning that secure and nonsecure registers coexist within the peripheral.

Table 38. GTZC interrupt request

Interrupt acronym	Interrupt event	Event flag	Enable control bit	Interrupt clear method	Exit Sleep mode	Exit Stop mode	Exit Standby mode
GTZC	Illegal access	All flags in GTZC1_TZIC_SRx	All bits in GTZC1_TZIC_IERx	Write 1 in the bit GTZC1_TZIC_FCRx	Yes	Yes	No

5.6 GTZC1 TZSC registers

All registers are accessed only by words (32-bit).

5.6.1 GTZC1 TZSC control register (GTZC1_TZSC_CR)

Address offset: 0x000

Reset value: 0x0000 0000

Secure privileged access only.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	LCK
															rs

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **LCK**: lock the configuration of GTZC1_TZSC_SECCFGRx and GTZC1_TZSC_PRIVCFGRx until next reset

This bit is cleared by default, and once set, it cannot be reset until system reset.

0: configuration of all GTZC1_TZSC_SECCFGRx and GTZC1_TZSC_PRIVCFGRx not locked

1: configuration of all GTZC1_TZSC_SECCFGRx and GTZC1_TZSC_PRIVCFGRx locked

5.6.2 GTZC1 TZSC secure configuration register 1 (GTZC1_TZSC_SECCFGR1)

Address offset: 0x010

Reset value: 0x0000 0000

Write-secure access only.

This register can be written only by a secure privileged transaction when the corresponding GTZC1_TZSC_PRIVCFGR signal is set to 1. If a given PRIV bit is not set, the equivalent SEC bit can be written by secure unprivileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM2 SEC	DTSSEC	UART1 2SEC	UART9 SEC	UART8 SEC	UART7 SEC	DAC1S EC	HDMIC ECSEC	USART 11SEC	USART 10SEC	USART 6SEC	CRSSEC	I3C1SEC	I2C2SEC	I2C1SEC	UART5 SEC
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UART4 SEC	USART 3SEC	USART 2SEC	SPI3SEC	SPI2SEC	IWDGSEC	WWDG SEC	TIM14S EC	TIM13S EC	TIM12S EC	TIM7S EC	TIM6S EC	TIM5S EC	TIM4S EC	TIM3S EC	TIM2S EC
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Bit 31 **LPTIM2SEC**: secure access mode for LPTIM2
0: nonsecure
1: secure
- Bit 30 **DTSSEC**: secure access mode for DTS
0: nonsecure
1: secure
- Bit 29 **UART12SEC**: secure access mode for UART12
0: nonsecure
1: secure
- Bit 28 **UART9SEC**: secure access mode for UART9
0: nonsecure
1: secure
- Bit 27 **UART8SEC**: secure access mode for UART8
0: nonsecure
1: secure
- Bit 26 **UART7SEC**: secure access mode for UART7
0: nonsecure
1: secure
- Bit 25 **DAC1SEC**: secure access mode for DAC1
0: nonsecure
1: secure
- Bit 24 **HDMICECSEC**: secure access mode for HDMICEC
0: nonsecure
1: secure
- Bit 23 **USART11SEC**: secure access mode for USART11
0: nonsecure
1: secure
- Bit 22 **USART10SEC**: secure access mode for USART10
0: nonsecure
1: secure
- Bit 21 **USART6SEC**: secure access mode for USART6
0: nonsecure
1: secure
- Bit 20 **CRSSEC**: secure access mode for CRS
0: nonsecure
1: secure
- Bit 19 **I3C1SEC**: secure access mode for I3C1
0: nonsecure
1: secure
- Bit 18 **I2C2SEC**: secure access mode for I2C2
0: nonsecure
1: secure

- Bit 17 **I2C1SEC**: secure access mode for I2C1
0: nonsecure
1: secure
- Bit 16 **UART5SEC**: secure access mode for UART5
0: nonsecure
1: secure
- Bit 15 **UART4SEC**: secure access mode for UART4
0: nonsecure
1: secure
- Bit 14 **USART3SEC**: secure access mode for USART3
0: nonsecure
1: secure
- Bit 13 **USART2SEC**: secure access mode for USART2
0: nonsecure
1: secure
- Bit 12 **SPI3SEC**: secure access mode for SPI3
0: nonsecure
1: secure
- Bit 11 **SPI2SEC**: secure access mode for SPI2
0: nonsecure
1: secure
- Bit 10 **IWDGSEC**: secure access mode for IWDG
0: nonsecure
1: secure
- Bit 9 **WWDGSEC**: secure access mode for WWDG
0: nonsecure
1: secure
- Bit 8 **TIM14SEC**: secure access mode for TIM14
0: nonsecure
1: secure
- Bit 7 **TIM13SEC**: secure access mode for TIM13
0: nonsecure
1: secure
- Bit 6 **TIM12SEC**: secure access mode for TIM12
0: nonsecure
1: secure
- Bit 5 **TIM7SEC**: secure access mode for TIM7
0: nonsecure
1: secure
- Bit 4 **TIM6SEC**: secure access mode for TIM6
0: nonsecure
1: secure
- Bit 3 **TIM5SEC**: secure access mode for TIM5
0: nonsecure
1: secure

Bit 2 **TIM4SEC**: secure access mode for TIM4

0: nonsecure

1: secure

Bit 1 **TIM3SEC**: secure access mode for TIM3

0: nonsecure

1: secure

Bit 0 **TIM2SEC**: secure access mode for TIM2

0: nonsecure

1: secure

5.6.3 GTZC1 TZSC secure configuration register 2 (GTZC1_TZSC_SECCFGR2)

Address offset: 0x014

Reset value: 0x0000 0000

Write-secure access only.

This register can be written only by a secure privileged transaction when the corresponding GTZC1_TZSC_PRIVCFGR signal is set to 1. If a given PRIV is not set, the equivalent SEC bit can be written by a secure unprivileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM5 SEC	LPTIM4 SEC	LPTIM3 SEC	LPTIM1 SEC	I2C4SEC	I2C3SEC	LPUART1SEC	SPI5SEC	Res.	Res.	Res.	Res.	USBSEC	SAI2SEC	SAI1SEC	SPI6SEC
rw	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI4SEC	TIM17SEC	TIM16SEC	TIM15SEC	USART1SEC	TIM8SEC	SPI1SEC	TIM1SEC	Res.	Res.	Res.	COMP SEC	Res.	UCPDSEC	FDCCAN2SEC	FDCCAN1SEC
rw	rw	rw	rw	rw	rw	rw	rw				rw		rw	rw	rw

Bit 31 **LPTIM5SEC**: secure access mode for LPTIM5

0: nonsecure

1: secure

Bit 30 **LPTIM4SEC**: secure access mode for LPTIM4

0: nonsecure

1: secure

Bit 29 **LPTIM3SEC**: secure access mode for LPTIM3

0: nonsecure

1: secure

Bit 28 **LPTIM1SEC**: secure access mode for LPTIM1

0: nonsecure

1: secure

- Bit 27 **I2C4SEC**: secure access mode for I2C4
0: nonsecure
1: secure
- Bit 26 **I2C3SEC**: secure access mode for I2C3
0: nonsecure
1: secure
- Bit 25 **LPUART1SEC**: secure access mode for LPUART
0: nonsecure
1: secure
- Bit 24 **SPI5SEC**: secure access mode for SPI5
0: nonsecure
1: secure
- Bits 23:20 Reserved, must be kept at reset value.
- Bit 19 **USBSEC**: secure access mode for USB
0: nonsecure
1: secure
- Bit 18 **SAI2SEC**: secure access mode for SAI2
0: nonsecure
1: secure
- Bit 17 **SAI1SEC**: secure access mode for SAI1
0: nonsecure
1: secure
- Bit 16 **SPI6SEC**: secure access mode for SPI6
0: nonsecure
1: secure
- Bit 15 **SPI4SEC**: secure access mode for SPI4
0: nonsecure
1: secure
- Bit 14 **TIM17SEC**: secure access mode for TIM17
0: nonsecure
1: secure
- Bit 13 **TIM16SEC**: secure access mode for TIM16
0: nonsecure
1: secure
- Bit 12 **TIM15SEC**: secure access mode for TIM15
0: nonsecure
1: secure
- Bit 11 **USART1SEC**: secure access mode for USART1
0: nonsecure
1: secure
- Bit 10 **TIM8SEC**: secure access mode for TIM8
0: nonsecure
1: secure

Bit 9 **SPI1SEC**: secure access mode for SPI1

0: nonsecure

1: secure

Bit 8 **TIM1SEC**: secure access mode for TIM1

0: nonsecure

1: secure

Bits 7:5 Reserved, must be kept at reset value.

Bit 4 **COMPSEC**: secure access mode for COMP

0: nonsecure

1: secure

Bit 3 Reserved, must be kept at reset value.

Bit 2 **UCPDSEC**: secure access mode for UCPD

0: nonsecure

1: secure

Bit 1 **FDCAN2SEC**: secure access mode for FDCAN2

0: nonsecure

1: secure

Bit 0 **FDCAN1SEC**: secure access mode for FDCAN1

0: nonsecure

1: secure

5.6.4 GTZC1 TZSC secure configuration register 3 (GTZC1_TZSC_SECCFGR3)

Address offset: 0x018

Reset value: 0x0000 0000

Write-secure access only.

This register can be written only by a secure privileged transaction when the corresponding GTZC1_TZSC_PRIVCFGR is set to 1. If a given PRIV is not set, the equivalent SEC bit can be written by a secure unprivileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	RAMC FGSEC	Res.	OCTOS PI1SE C	FMCSE C	SDMM C2SEC	SDMM C1SEC	PKASE C	SAESS EC	RNGS EC	HASHS EC	AESSE C
					rw		rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DCMIS EC	ADC12 SEC	DCAC HESEC	ICACH ESEC	ETHSE C	FMACS EC	CORDI CSEC	CRCSE C	Res.	ADC3S EC	Res.	CCBSE C	Res.	I3C2SE C	VREFB UFSEC	LPTIM6 SEC
rw	rw	rw	rw	rw	rw	rw	rw		rw		rw		rw	rw	rw

Bits 31:27 Reserved, must be kept at reset value.

- Bit 26 **RAMCFGSEC**: secure access mode for RAMSCFG
0: nonsecure
1: secure
- Bit 25 Reserved, must be kept at reset value.
- Bit 24 **OCTOSPI1SEC**: secure access mode for OCTOSPI1
0: nonsecure
1: secure
- Bit 23 **FMCSEC**: secure access mode for FMC
0: nonsecure
1: secure
- Bit 22 **SDMMC2SEC**: secure access mode for SDMMC2
0: nonsecure
1: secure
- Bit 21 **SDMMC1SEC**: secure access mode for SDMMC1
0: nonsecure
1: secure
- Bit 20 **PKASEC**: secure access mode for PKA
0: nonsecure
1: secure
- Bit 19 **SAESSEC**: secure access mode for SAES
0: nonsecure
1: secure
- Bit 18 **RNGSEC**: secure access mode for RNG
0: nonsecure
1: secure
- Bit 17 **HASHSEC**: secure access mode for HASH
0: nonsecure
1: secure
- Bit 16 **AESSEC**: secure access mode for AES
0: nonsecure
1: secure
- Bit 15 **DCMISEC**: secure access mode for DCMI
0: nonsecure
1: secure
- Bit 14 **ADC12SEC**: secure access mode for ADC1 and ADC2
0: nonsecure
1: secure
- Bit 13 **DCACHESEC**: secure access mode for DCACHE
0: nonsecure
1: secure
- Bit 12 **ICACHESEC**: secure access mode for ICACHE
0: nonsecure
1: secure

- Bit 11 **ETHSEC**: secure access mode for register of ETH
0: nonsecure
1: secure
- Bit 10 **FMACSEC**: secure access mode for FMAC
0: nonsecure
1: secure
- Bit 9 **CORDICSEC**: secure access mode for CORDIC
0: nonsecure
1: secure
- Bit 8 **CRCSEC**: secure access mode for CRC
0: nonsecure
1: secure
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **ADC3SEC**: secure access mode for ADC3
0: nonsecure
1: secure
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **CCBSEC**: secure access mode for CCB
0: nonsecure
1: secure
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **I3C2SEC**: secure access mode for I3C2
0: nonsecure
1: secure
- Bit 1 **VREFBUFSEC**: secure access mode for VREFBUF
0: nonsecure
1: secure
- Bit 0 **LPTIM6SEC**: secure access mode for LPTIM6
0: nonsecure
1: secure

5.6.5 GTZC1 TZSC privilege configuration register 1 (GTZC1_TZSC_PRIVCFGR1)

Address offset: 0x020

Reset value: 0x0000 0000

Write-privileged access only.

This register can be read or written only by a secure privileged transaction when the corresponding GTZC1_TZSC_SECCFGR signal is set to 1. If a given SEC bit is not set, the equivalent PRIV bit can be read/written by a nonsecure privileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM2 PRIV	DTSPR IV	UART1 2PRIV	UART9 PRIV	UART8 PRIV	UART7 PRIV	DAC1P RIV	HDMIC ECPRI V	USART 11PRIV	USART 10PRIV	USART 6PRIV	CRSPR IV	I3C1PR IV	I2C2PR IV	I2C1PR IV	UART5 PRIV
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UART4 PRIV	USART 3PRIV	USART 2PRIV	SPI3P RIV	SPI2P RIV	IWDGP RIV	WWDG PRIV	TIM14P RIV	TIM13P RIV	TIM12P RIV	TIM7P RIV	TIM6P RIV	TIM5P RIV	TIM4P RIV	TIM3P RIV	TIM2P RIV
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **LPTIM2PRIV**: privileged access mode for LPTIM2

0: unprivileged
1: privileged

Bit 30 **DTSPRIV**: privileged access mode for DTS

0: unprivileged
1: privileged

Bit 29 **UART12PRIV**: privileged access mode for UART12

0: unprivileged
1: privileged

Bit 28 **UART9PRIV**: privileged access mode for UART9

0: unprivileged
1: privileged

Bit 27 **UART8PRIV**: privileged access mode for UART8

0: unprivileged
1: privileged

Bit 26 **UART7PRIV**: privileged access mode for UART7

0: unprivileged
1: privileged

Bit 25 **DAC1PRIV**: privileged access mode for DAC1

0: unprivileged
1: privileged

- Bit 24 **HDMICECPRIV**: privileged access mode for HDMICEC
0: unprivileged
1: privileged
- Bit 23 **USART11PRIV**: privileged access mode for USART11
0: unprivileged
1: privileged
- Bit 22 **USART10PRIV**: privileged access mode for USART10
0: unprivileged
1: privileged
- Bit 21 **USART6PRIV**: privileged access mode for USART6
0: unprivileged
1: privileged
- Bit 20 **CRSPRIV**: privileged access mode for CRS
0: unprivileged
1: privileged
- Bit 19 **I3C1PRIV**: privileged access mode for I3C1
0: unprivileged
1: privileged
- Bit 18 **I2C2PRIV**: privileged access mode for I2C2
0: unprivileged
1: privileged
- Bit 17 **I2C1PRIV**: privileged access mode for I2C1
0: unprivileged
1: privileged
- Bit 16 **UART5PRIV**: privileged access mode for UART5
0: unprivileged
1: privileged
- Bit 15 **UART4PRIV**: privileged access mode for UART4
0: unprivileged
1: privileged
- Bit 14 **USART3PRIV**: privileged access mode for USART3
0: unprivileged
1: privileged
- Bit 13 **USART2PRIV**: privileged access mode for USART2
0: unprivileged
1: privileged
- Bit 12 **SPI3PRIV**: privileged access mode for SPI3
0: unprivileged
1: privileged
- Bit 11 **SPI2PRIV**: privileged access mode for SPI2
0: unprivileged
1: privileged
- Bit 10 **IWDGPRIV**: privileged access mode for IWDG
0: unprivileged
1: privileged

- Bit 9 **WWDGPRIV**: privileged access mode for WWDG
0: unprivileged
1: privileged
- Bit 8 **TIM14PRIV**: privileged access mode for TIM14
0: unprivileged
1: privileged
- Bit 7 **TIM13PRIV**: privileged access mode for TIM13
0: unprivileged
1: privileged
- Bit 6 **TIM12PRIV**: privileged access mode for TIM12
0: unprivileged
1: privileged
- Bit 5 **TIM7PRIV**: privileged access mode for TIM7
0: unprivileged
1: privileged
- Bit 4 **TIM6PRIV**: privileged access mode for TIM6
0: unprivileged
1: privileged
- Bit 3 **TIM5PRIV**: privileged access mode for TIM5
0: unprivileged
1: privileged
- Bit 2 **TIM4PRIV**: privileged access mode for TIM4
0: unprivileged
1: privileged
- Bit 1 **TIM3PRIV**: privileged access mode for TIM3
0: unprivileged
1: privileged
- Bit 0 **TIM2PRIV**: privileged access mode for TIM2
0: unprivileged
1: privileged

5.6.6 GTZC1 TZSC privilege configuration register 2 (GTZC1_TZSC_PRIVCFGR2)

Address offset: 0x024

Reset value: 0x0000 0000

Write-privileged access only.

This register can be read or written only by a secure privileged transaction when the corresponding GTZC1_TZSC_SECCFGR signal is set to 1. If a given SEC bit is not set, the equivalent PRIV bit can be read/written by a nonsecure privileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM5 PRIV	LPTIM4 PRIV	LPTIM3 PRIV	LPTIM1 PRIV	I2C4PR IV	I2C3PR IV	LPUART1PRIV	SPI5P RIV	Res.	Res.	Res.	Res.	USBPR IV	SAI2P RIV	SAI1P RIV	SPI6P RIV
rw	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI4P RIV	TIM17P RIV	TIM16P RIV	TIM15P RIV	USART 1PRIV	TIM8P RIV	SPI1P RIV	TIM1P RIV	Res.	Res.	Res.	COMP PRIV	Res.	UCPDPRIV	FDCAN 2PRIV	FDCAN 1PRIV
rw	rw	rw	rw	rw	rw	rw	rw				rw		rw	rw	rw

Bit 31 **LPTIM5PRIV**: privileged access mode for LPTIM5

0: unprivileged

1: privileged

Bit 30 **LPTIM4PRIV**: privileged access mode for LPTIM4

0: unprivileged

1: privileged

Bit 29 **LPTIM3PRIV**: privileged access mode for LPTIM3

0: unprivileged

1: privileged

Bit 28 **LPTIM1PRIV**: privileged access mode for LPTIM1

0: unprivileged

1: privileged

Bit 27 **I2C4PRIV**: privileged access mode for I2C4

0: unprivileged

1: privileged

Bit 26 **I2C3PRIV**: privileged access mode for I2C3

0: unprivileged

1: privileged

Bit 25 **LPUART1PRIV**: privileged access mode for LPUART

0: unprivileged

1: privileged

Bit 24 **SPI5PRIV**: privileged access mode for SPI5

0: unprivileged

1: privileged

Bits 23:20 Reserved, must be kept at reset value.

Bit 19 **USBPRIV**: privileged access mode for USB

0: unprivileged

1: privileged

Bit 18 **SAI2PRIV**: privileged access mode for SAI2

0: unprivileged

1: privileged

Bit 17 **SAI1PRIV**: privileged access mode for SAI1

0: unprivileged

1: privileged

- Bit 16 **SPI6PRIV**: privileged access mode for SPI6
0: unprivileged
1: privileged
- Bit 15 **SPI4PRIV**: privileged access mode for SPI4
0: unprivileged
1: privileged
- Bit 14 **TIM17PRIV**: privileged access mode for TIM17
0: unprivileged
1: privileged
- Bit 13 **TIM16PRIV**: privileged access mode for TIM16
0: unprivileged
1: privileged
- Bit 12 **TIM15PRIV**: privileged access mode for TIM15
0: unprivileged
1: privileged
- Bit 11 **USART1PRIV**: privileged access mode for USART1
0: unprivileged
1: privileged
- Bit 10 **TIM8PRIV**: privileged access mode for TIM8
0: unprivileged
1: privileged
- Bit 9 **SPI1PRIV**: privileged access mode for SPI1
0: unprivileged
1: privileged
- Bit 8 **TIM1PRIV**: privileged access mode for TIM1
0: unprivileged
1: privileged
- Bits 7:5 Reserved, must be kept at reset value.
- Bit 4 **COMPPRIV**: privileged access mode for COMP
0: unprivileged
1: privileged
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **UCPDPRIV**: privileged access mode for UCPD
0: unprivileged
1: privileged
- Bit 1 **FDCAN2PRIV**: privileged access mode for FDCAN2
0: unprivileged
1: privileged
- Bit 0 **FDCAN1PRIV**: privileged access mode for FDCAN1
0: unprivileged
1: privileged

5.6.7 GTZC1 TZSC privilege configuration register 3 (GTZC1_TZSC_PRIVCFGR3)

Address offset: 0x028

Reset value: 0x0000 0000

Write-privileged access only.

This register can be read or written only by a secure privileged transaction when the corresponding GTZC1_TZSC_SECCFGR signal is set to 1. If a given SEC bit is not set, the equivalent PRIV bit can be read/written by a nonsecure privileged transaction.

Read accesses are authorized for any type of transactions, secure or not, privileged or not.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	RAMC FGPRIV	Res.	OCTOS PI1PRIV	FMCP RIV	SDMM C2PRIV	SDMM C1PRIV	PKAPR IV	SAESP RIV	RNGP RIV	HASHP RIV	AESPR IV
					rw		rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DCMIP RIV	ADC12 PRIV	DCAC HEPRIV	ICACH EPRIV	ETHPR IV	FMACP RIV	CORDI CPRIV	CRCP RIV	Res.	ADC3P RIV	Res.	CCBPR IV	Res.	I3C2PR IV	VREFB UFPRIV	LPTIM6 PRIV
rw	rw	rw	rw	rw	rw	rw	rw		rw		rw		rw	rw	rw

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **RAMCFGPRIV**: privileged access mode for RAMSCFG

0: unprivileged

1: privileged

Bit 25 Reserved, must be kept at reset value.

Bit 24 **OCTOSPI1PRIV**: privileged access mode for OCTOSPI1

0: unprivileged

1: privileged

Bit 23 **FMCPRIV**: privileged access mode for FMC

0: unprivileged

1: privileged

Bit 22 **SDMMC2PRIV**: privileged access mode for SDMMC2

0: unprivileged

1: privileged

Bit 21 **SDMMC1PRIV**: privileged access mode for SDMMC1

0: unprivileged

1: privileged

Bit 20 **PKAPRIV**: privileged access mode for PKA

0: unprivileged

1: privileged

Bit 19 **SAESPRIV**: privileged access mode for SAES

0: unprivileged

1: privileged

- Bit 18 **RNGPRIV**: privileged access mode for RNG
0: unprivileged
1: privileged
- Bit 17 **HASHPRIV**: privileged access mode for HASH
0: unprivileged
1: privileged
- Bit 16 **AESPRIV**: privileged access mode for AES
0: unprivileged
1: privileged
- Bit 15 **DCMIPRIV**: privileged access mode for DCMI
0: unprivileged
1: privileged
- Bit 14 **ADC12PRIV**: privileged access mode for ADC1 and ADC2
0: unprivileged
1: privileged
- Bit 13 **DCACHEPRIV**: privileged access mode for DCACHE
0: unprivileged
1: privileged
- Bit 12 **ICACHEPRIV**: privileged access mode for ICACHE
0: unprivileged
1: privileged
- Bit 11 **ETHPRIV**: privileged access mode for register of ETH
0: unprivileged
1: privileged
- Bit 10 **FMACPRIV**: privileged access mode for FMAC
0: unprivileged
1: privileged
- Bit 9 **CORDICPRIV**: privileged access mode for CORDIC
0: unprivileged
1: privileged
- Bit 8 **CRCPRIV**: privileged access mode for CRC
0: unprivileged
1: privileged
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **ADC3PRIV**: privileged access mode for ADC3
0: unprivileged
1: privileged
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **CCBPRIV**: privileged access mode for CCB
0: unprivileged
1: privileged
- Bit 3 Reserved, must be kept at reset value.

Bit 2 **I3C2PRIV**: privileged access mode for I3C2

0: unprivileged

1: privileged

Bit 1 **VREFBUFPRIV**: privileged access mode for VREFBUF

0: unprivileged

1: privileged

Bit 0 **LPTIM6PRIV**: privileged access mode for LPTIM6

0: unprivileged

1: privileged

5.6.8 GTZC1 TZSC memory x subregion A watermark configuration register (GTZC1_TZSC_MPCWMxACFGR)

Address offset: $0x40 + 0x10 \cdot (x - 1)$ ($x = 1$ to 4)

Reset value: 0x0000 0000

Secure privilege access only.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	PRIV	SEC	Res.	Res.	Res.	Res.	Res.	Res.	SRLOCK	SREN
						rw	rw							rs	rw

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 **PRIV**: Privileged subregion A of base region x

This bit is taken into account only if SREN is set.

0: Privileged and unprivileged accesses are granted in subregion A.

1: Only privileged accesses are granted in subregion A of region x.

Bit 8 **SEC**: Secure subregion A of base region x

This bit is taken into account only if SREN is set.

0: Only nonsecure data accesses are granted to subregion A of region x.

1: Only secure data accesses are granted to subregion A of region x.

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **SRLOCK**: Subregion A lock

This bit, once set, can be cleared only by a system reset.

0: GTZC1_TZSC_MPCWMxCFGR, GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR can be written.

1: Writes to GTZC1_TZSC_MPCWMxCFGR, GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR are ignored.

Bit 0 **SREN**: Subregion A enable

0: subregion A is disabled. Access control of base region x applies to any access between this subregion start- and end-addresses.

1: subregion A of region x is enabled. Access control defined in GTZC1_TZSC_MPCWMx_CFGR applies to any access between this subregion start- and end-addresses, defined in GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR.

Note: Watermark controlled external memories start fully nonsecure/unprivileged at reset when TZEN = 0xC3. When TZEN = 0xB4, they start fully secure/fully privileged (inverted reset-value).

5.6.9 GTZC1 TZSC memory x subregion A watermark register (GTZC1_TZSC_MPCWMxAR)

Address offset: $0x44 + 0x10 \cdot (x - 1)$ ($x = 1$ to 4)

Reset value: 0x0000 0000

The given reset value is valid when TZEN = 0xB4. The reset value is 0x0800 0000 when TZEN = 0xC3.

Secure privilege access only.

When SUBA_START + SUBA_LENGTH is higher than the maximum size allowed for the memory, a saturation of SUBA_LENGTH is applied automatically.

When an overlap of subregion A and B exists, secure/privileged attributes of both subregions apply on the common section (see [Section 5.4.3](#))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SUBA_LENGTH[11:0]											
				r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SUBA_START[10:0]										
					r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **SUBA_LENGTH[11:0]**: Length of subregion A in region x

This field defines the length of the subregion A, to be multiplied by the granularity defined in [Table 31](#).

When SUBA_START + SUBA_LENGTH is higher than the maximum size allowed for the memory, a saturation of SUBA_LENGTH is applied automatically. If SUBA_LENGTH = 0, subregion A is disabled (SREN bit in GTZC1_TZSC_MPCWMxACFGR is cleared).

Bits 15:11 Reserved, must be kept at reset value.

Bits 10:0 **SUBA_START[10:0]**: Start of subregion A in region x

This field defines the address offset of the subregion A, to be multiplied by the granularity defined in [Table 31](#), versus the start of the region x.

Watermark controlled external memories start fully nonsecure at reset when TZEN = 0xC3. When TZEN = 0xB4, they start fully secure (inverted reset value).

5.6.10 GTZC1 TZSC memory x subregion B watermark configuration register (GTZC1_TZSC_MPCWMxBCFGR)

Address offset: $0x48 + 0x10 \cdot (x - 1)$ ($x = 1$ to 4)

Reset value: $0x0000\ 0000$

Secure privilege access only.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	PRIV	SEC	Res.	Res.	Res.	Res.	Res.	Res.	SRLOCK	SREN
						rw	rw							rs	rw

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 **PRIV**: Privileged subregion B of base region x

This bit is taken into account only if SREN is set.

0: Privileged and unprivileged accesses are granted in subregion A.

1: Only privileged accesses are granted in subregion A of region x.

Bit 8 **SEC**: Secure subregion B of base region x

This bit is taken into account only if SREN is set.

0: Only nonsecure data accesses are granted to subregion A of region x.

1: Only secure data accesses are granted to subregion A of region x.

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **SRLOCK**: Subregion B lock

This bit, once set, can be cleared only by a system reset.

0: GTZC1_TZSC_MPCWMxCFGR, GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR can be written.

1: Writes to GTZC1_TZSC_MPCWMxCFGR, GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR are ignored.

Bit 0 **SREN**: Subregion B enable

0: subregion B is disabled. Access control of base region x applies to any access between this subregion start- and end-addresses.

1: subregion B of region x is enabled. Access control defined in GTZC1_TZSC_MPCWMxCFGR applies to any access between this subregion start- and end-addresses, defined in GTZC1_TZSC_MPCWMxAR and GTZC1_TZSC_MPCWMxBR.

Note: Watermark controlled external memories start fully nonsecure/unprivileged at reset when TZEN = 0xC3. When TZEN = 0xB4, they start fully secure/fully privileged (inverted reset-value).

5.6.11 GTZC1 TZSC memory x subregion B watermark register (GTZC1_TZSC_MPCWMxBR)

Address offset: $0x4C + 0x10 \cdot (x - 1)$ ($x = 1$ to 4)

Reset value: $0x0000\ 0000$

The given reset value is valid when TZEN = 0xB4. The reset value is $0x0800\ 0000$ when TZEN = 0xC3.

Secure privilege access only.

When SUBB_START + SUBB_LENGTH is higher than the maximum size allowed for the memory, a saturation of SUBB_LENGTH is applied automatically.

When an overlap of subregion A and B exists, secure/privileged attributes of both subregions apply on the common section (see [Section 5.4.3](#))

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SUBB_LENGTH[11:0]											
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	SUBB_START[10:0]										
					rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bits 27:16 **SUBB_LENGTH[11:0]**: Length of subregion B in region x

This field defines the length of the subregion B, to be multiplied by the granularity defined in [Table 31](#).

When SUBB_START + SUBB_LENGTH is higher than the maximum size allowed for the memory, a saturation of SUBB_LENGTH is applied automatically.

If SUBB_LENGTH = 0, the subregion B is disabled (SREN bit in GTZC1_TZSC_MPCMWxBCFGR is cleared).

Bits 15:11 Reserved, must be kept at reset value.

Bits 10:0 **SUBB_START[10:0]**: Start of subregion B in region x

This field defines the address offset of the subregion B, to be multiplied by the granularity defined in [Table 31](#), versus the start of the region x.

Watermark controlled external memories start fully nonsecure at reset when TZEN = 0xC3. When TZEN = 0xB4, they start fully secure (inverted reset value).

5.6.12 GTZC1 TZSC register map

Table 39. GTZC1 TZSC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x000	GTZC1_TZSC_CR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	LCK			
	Reset value																																	0			
0x004-0x00C	Reserved	Reserved																																			
0x010	GTZC1_TZSC_SECCFGR1	LPTIM2SEC	DTSEC	UART12SEC	UART9SEC	UART8SEC	UART7SEC	DAC1SEC	HDMICECSEC	USART11SEC	USART10SEC	USART6SEC	CRSSEC	I3C1SEC	I2C2SEC	I2C1SEC	UART4SEC	UART3SEC	USART2SEC	SPI3SEC	SPI2SEC	IWDGSEC	VWDGSEC	TIM14SEC	TIM13SEC	TIM12SEC	TIM7SEC	TIM6SEC	TIM5SEC	TIM4SEC	TIM3SEC	TIM2SEC					
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x014	GTZC1_TZSC_SECCFGR2	LPTIM5SEC	LPTIM4SEC	LPTIM3SEC	LPTIM1SEC	I2C4SEC	I2C3SEC	LPUART1SEC	SPI5SEC	Res	Res	Res	Res	USBSEC	SAI2SEC	SAI1SEC	SPI6SEC	SPI4SEC	TIM17SEC	TIM16SEC	TIM15SEC	USART1SEC	TIM8SEC	SPI1SEC	TIM1SEC	Res	Res	Res	COMPSEC	Res	UCPDSEC	FDCAN2SEC	FDCAN1SEC				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x018	GTZC1_TZSC_SECCFGR3	Res	Res	Res	Res	Res	RAMCFGSEC	Res	OCTOSPI1SEC	FMCSEC	SDMMC2SEC	SDMMC1SEC	PKASEC	SAESSEC	RNGSEC	HASHSEC	AESSEC	DCMISEC	ADC12SEC	DCACHESEC	ICACHESEC	ETHSEC	FMACSEC	CORDICSEC	CRCSEC	Res	ADC3SEC	Res	CCBSEC	Res	I3C2SEC	VREFBUFSEC	LPTIM6SEC				
	Reset value						0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x01C	Reserved	Reserved																																			
0x020	GTZC1_TZSC_PRIVCFGR1	LPTIM2PRIV	DTSPRIV	UART12PRIV	UART9PRIV	UART8PRIV	UART7PRIV	DAC1PRIV	HDMICECPRIV	USART11PRIV	USART10PRIV	USART6PRIV	CRSPRIV	I3C1PRIV	I2C2PRIV	I2C1PRIV	UART5PRIV	UART4PRIV	USART3PRIV	USART2PRIV	SPI3PRIV	SPI2PRIV	IWDGPRIV	VWDGPRIV	TIM14PRIV	TIM13PRIV	TIM12PRIV	TIM7PRIV	TIM6PRIV	TIM5PRIV	TIM4PRIV	TIM3PRIV	TIM2PRIV				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x024	GTZC1_TZSC_PRIVCFGR2	LPTIM5PRIV	LPTIM4PRIV	LPTIM3PRIV	LPTIM1PRIV	I2C4PRIV	I2C3PRIV	LPUART1PRIV	SPI5PRIV	Res	Res	Res	Res	USBPRIV	SAI2PRIV	SAI1PRIV	SPI6PRIV	SPI4PRIV	TIM17PRIV	TIM16PRIV	TIM15PRIV	USART1PRIV	TIM8PRIV	SPI1PRIV	TIM1PRIV	Res	Res	Res	COMPRIV	Res	UCPDPRIV	FDCAN2PRIV	FDCAN1PRIV				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x028	GTZC1_TZSC_PRIVFGR3	Res	Res	Res	Res	Res	RAMCFGPRIV	Res	OCTOSPI1PRIV	FMCPRIV	SDMMC2PRIV	SDMMC1PRIV	PKAPRIV	SAESPRIV	RNGPRIV	HASHPRIV	AESPRIV	DCMIPRIV	ADC12PRIV	DCACHEPRIV	ICACHEPRIV	ETHPRIV	FMACPRIV	CORDICPRIV	CRCPRIV	Res	ADC3PRIV	Res	CCBPRIV	Res	I3C2PRIV	VREFBUFPRIV	LPTIM6PRIV				
	Reset value						0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x02C-0x03C	Reserved	Reserved																																			
0x040	GTZC1_TZSC_MPCWM1ACFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN				
	Reset value																							0	0							0	0				
0x44	GTZC1_TZSC_MPCWM1AR	Res	Res	Res	Res	SUBA_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBA_START[10:0]													
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0			
0x048	GTZC1_TZSC_MPCWM1BCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN				
	Reset value																							0	0							0	0				

Table 39. GTZC1 TZSC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x04C	GTZC1_TZSC_MPCWM1BR	Res	Res	Res	Res	SUBB_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBB_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x050	GTZC1_TZSC_MPCWM2ACFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x54	GTZC1_TZSC_MPCWM2AR	Res	Res	Res	Res	SUBA_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBA_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x058	GTZC1_TZSC_MPCWM2BCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x05C	GTZC1_TZSC_MPCWM2BR	Res	Res	Res	Res	SUBB_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBB_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x060	GTZC1_TZSC_MPCWM3ACFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x64	GTZC1_TZSC_MPCWM3AR	Res	Res	Res	Res	SUBA_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBA_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x068	GTZC1_TZSC_MPCWM3BCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x06C	GTZC1_TZSC_MPCWM3BR	Res	Res	Res	Res	SUBB_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBB_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x070	GTZC1_TZSC_MPCWM4ACFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x74	GTZC1_TZSC_MPCWM4AR	Res	Res	Res	Res	SUBA_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBA_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0
0x078	GTZC1_TZSC_MPCWM4BCFGR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PRIV	SEC	Res	Res	Res	Res	Res	Res	SRLOCK	SREN	
	Reset value																							0	0							0	0	
0x07C	GTZC1_TZSC_MPCWM4BR	Res	Res	Res	Res	SUBB_LENGTH[11:0]													Res	Res	Res	Res	Res	SUBB_START[10:0]										
	Reset value					0	0	0	0	0	0	0	0	0	0	0	0							0	0	0	0	0	0	0	0	0	0	0

Refer to [Table 30: GTZC1 subblock address offset](#).

5.7 GTZC1 TZIC registers

All registers are accessed only by words (32-bit).

5.7.1 GTZC1 TZIC interrupt enable register 1 (GTZC1_TZIC_IER1)

Address offset: 0x000

Reset value: 0x0000 0000

Secure privileged access only.

This register is used to enable interrupt of illegal access.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM2 IE	DTSIE	UART1 2IE	UART9 IE	UART8 IE	UART7 IE	DAC1 IE	HDMIC ECIE	USART 11IE	USART 10IE	USART 6IE	CRSIE	I3C1IE	I2C2IE	I2C1IE	UART5 IE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UART4 IE	USART 3IE	USART 2IE	SPI3IE	SPI2IE	IWDG IE	WWDG IE	TIM14 IE	TIM13 IE	TIM12 IE	TIM7IE	TIM6IE	TIM5IE	TIM4IE	TIM3IE	TIM2IE
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **LPTIM2IE**: illegal access interrupt enable for LPTIM2

0: interrupt disabled
1: interrupt enabled

Bit 30 **DTSIE**: illegal access interrupt enable for DTS

0: interrupt disabled
1: interrupt enabled

Bit 29 **UART12IE**: illegal access interrupt enable for UART12

0: interrupt disabled
1: interrupt enabled

Bit 28 **UART9IE**: illegal access interrupt enable for UART9

0: interrupt disabled
1: interrupt enabled

Bit 27 **UART8IE**: illegal access interrupt enable for UART8

0: interrupt disabled
1: interrupt enabled

Bit 26 **UART7IE**: illegal access interrupt enable for UART7

0: interrupt disabled
1: interrupt enabled

Bit 25 **DAC1IE**: illegal access interrupt enable for DAC1

0: interrupt disabled
1: interrupt enabled

Bit 24 **HDMICECIE**: illegal access interrupt enable for HDMICEC

0: interrupt disabled
1: interrupt enabled

- Bit 23 **USART11IE**: illegal access interrupt enable for USART11
0: interrupt disabled
1: interrupt enabled
- Bit 22 **USART10IE**: illegal access interrupt enable for USART10
0: interrupt disabled
1: interrupt enabled
- Bit 21 **USART6IE**: illegal access interrupt enable for USART6
0: interrupt disabled
1: interrupt enabled
- Bit 20 **CRSIE**: illegal access interrupt enable for CRS
0: interrupt disabled
1: interrupt enabled
- Bit 19 **I3C1IE**: illegal access interrupt enable for I3C1
0: interrupt disabled
1: interrupt enabled
- Bit 18 **I2C2IE**: illegal access interrupt enable for I2C2
0: interrupt disabled
1: interrupt enabled
- Bit 17 **I2C1IE**: illegal access interrupt enable for I2C1
0: interrupt disabled
1: interrupt enabled
- Bit 16 **UART5IE**: illegal access interrupt enable for UART5
0: interrupt disabled
1: interrupt enabled
- Bit 15 **UART4IE**: illegal access interrupt enable for UART4
0: interrupt disabled
1: interrupt enabled
- Bit 14 **USART3IE**: illegal access interrupt enable for USART3
0: interrupt disabled
1: interrupt enabled
- Bit 13 **USART2IE**: illegal access interrupt enable for USART2
0: interrupt disabled
1: interrupt enabled
- Bit 12 **SPI3IE**: illegal access interrupt enable for SPI3
0: interrupt disabled
1: interrupt enabled
- Bit 11 **SPI2IE**: illegal access interrupt enable for SPI2
0: interrupt disabled
1: interrupt enabled
- Bit 10 **IWDGIE**: illegal access interrupt enable for IWDG
0: interrupt disabled
1: interrupt enabled
- Bit 9 **WWDGIE**: illegal access interrupt enable for WWDG
0: interrupt disabled
1: interrupt enabled

Bit 8 **TIM14IE**: illegal access interrupt enable for TIM14

0: interrupt disabled

1: interrupt enabled

Bit 7 **TIM13IE**: illegal access interrupt enable for TIM13

0: interrupt disabled

1: interrupt enabled

Bit 6 **TIM12IE**: illegal access interrupt enable for TIM12

0: interrupt disabled

1: interrupt enabled

Bit 5 **TIM7IE**: illegal access interrupt enable for TIM7

0: interrupt disabled

1: interrupt enabled

Bit 4 **TIM6IE**: illegal access interrupt enable for TIM6

0: interrupt disabled

1: interrupt enabled

Bit 3 **TIM5IE**: illegal access interrupt enable for TIM5

0: interrupt disabled

1: interrupt enabled

Bit 2 **TIM4IE**: illegal access interrupt enable for TIM4

0: interrupt disabled

1: interrupt enabled

Bit 1 **TIM3IE**: illegal access interrupt enable for TIM3

0: interrupt disabled

1: interrupt enabled

Bit 0 **TIM2IE**: illegal access interrupt enable for TIM2

0: interrupt disabled

1: interrupt enabled

5.7.2 GTZC1 TZIC interrupt enable register 2 (GTZC1_TZIC_IER2)

Address offset: 0x004

Reset value: 0x0000 0000

Secure privileged access only.

This register is used to enable interrupt of illegal access.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM5IE	LPTIM4IE	LPTIM3IE	LPTIM1IE	I2C4IE	I2C3IE	LPUART1IE	SPI5IE	Res.	Res.	Res.	Res.	USBIE	SAI2IE	SAI1IE	SPI6IE
rw	rw	rw	rw	rw	rw	rw	rw					rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI4IE	TIM17IE	TIM16IE	TIM15IE	USART1IE	TIM8IE	SPI1IE	TIM1IE	Res.	Res.	Res.	COMP1IE	Res.	UCPDIE	FDCAN2IE	FDCAN1IE
rw	rw	rw	rw	rw	rw	rw	rw				rw		rw	rw	rw

- Bit 31 **LPTIM5IE**: illegal access interrupt enable for LPTIM5
0: interrupt disabled
1: interrupt enabled
- Bit 30 **LPTIM4IE**: illegal access interrupt enable for LPTIM4
0: interrupt disabled
1: interrupt enabled
- Bit 29 **LPTIM3IE**: illegal access interrupt enable for LPTIM3
0: interrupt disabled
1: interrupt enabled
- Bit 28 **LPTIM1IE**: illegal access interrupt enable for LPTIM1
0: interrupt disabled
1: interrupt enabled
- Bit 27 **I2C4IE**: illegal access interrupt enable for I2C4
0: interrupt disabled
1: interrupt enabled
- Bit 26 **I2C3IE**: illegal access interrupt enable for I2C3
0: interrupt disabled
1: interrupt enabled
- Bit 25 **LPUART1IE**: illegal access interrupt enable for LPUART
0: interrupt disabled
1: interrupt enabled
- Bit 24 **SPI5IE**: illegal access interrupt enable for SPI5
0: interrupt disabled
1: interrupt enabled
- Bits 23:20 Reserved, must be kept at reset value.
- Bit 19 **USBIE**: illegal access interrupt enable for USB
0: interrupt disabled
1: interrupt enabled
- Bit 18 **SAI2IE**: illegal access interrupt enable for SAI2
0: interrupt disabled
1: interrupt enabled
- Bit 17 **SAI1IE**: illegal access interrupt enable for SAI1
0: interrupt disabled
1: interrupt enabled
- Bit 16 **SPI6IE**: illegal access interrupt enable for SPI6
0: interrupt disabled
1: interrupt enabled
- Bit 15 **SPI4IE**: illegal access interrupt enable for SPI4
0: interrupt disabled
1: interrupt enabled
- Bit 14 **TIM17IE**: illegal access interrupt enable for TIM17
0: interrupt disabled
1: interrupt enabled

- Bit 13 **TIM16IE**: illegal access interrupt enable for TIM16
0: interrupt disabled
1: interrupt enabled
- Bit 12 **TIM15IE**: illegal access interrupt enable for TIM15
0: interrupt disabled
1: interrupt enabled
- Bit 11 **USART1IE**: illegal access interrupt enable for USART1
0: interrupt disabled
1: interrupt enabled
- Bit 10 **TIM8IE**: illegal access interrupt enable for TIM8
0: interrupt disabled
1: interrupt enabled
- Bit 9 **SPI1IE**: illegal access interrupt enable for SPI1
0: interrupt disabled
1: interrupt enabled
- Bit 8 **TIM1IE**: illegal access interrupt enable for TIM1
0: interrupt disabled
1: interrupt enabled
- Bits 7:5 Reserved, must be kept at reset value.
- Bit 4 **COMPIE**: illegal access interrupt enable for COMP
0: interrupt disabled
1: interrupt enabled
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **UCPDIE**: illegal access interrupt enable for UCPD
0: interrupt disabled
1: interrupt enabled
- Bit 1 **FDCAN2IE**: illegal access interrupt enable for FDCAN2
0: interrupt disabled
1: interrupt enabled
- Bit 0 **FDCAN1IE**: illegal access interrupt enable for FDCAN1
0: interrupt disabled
1: interrupt enabled

5.7.3 GTZC1 TZIC interrupt enable register 3 (GTZC1_TZIC_IER3)

Address offset: 0x008

Reset value: 0x0000 0000

Secure privileged access only.

This register is used to enable interrupt of illegal access.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	RAMCFGIE	Res.	OCTOSPI1IE	FMCIE	SDMMC2IE	SDMMC1IE	PKAIE	SAESIE	RNGIE	HASHIE	AESIE
					rw		rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DCMIIE	ADC12IE	DCACHEIE	ICACHEIE	ETHIE	FMACEIE	CORDCIE	CRCIE	Res.	ADC3IE	Res.	CCBIE	Res.	I3C2IE	VREFBUFIE	LPTIM6IE
rw	rw	rw	rw	rw	rw	rw	rw		rw		rw		rw	rw	rw

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **RAMCFGIE**: illegal access interrupt enable for RAMSCFG

0: interrupt disabled

1: interrupt enabled

Bit 25 Reserved, must be kept at reset value.

Bit 24 **OCTOSPI1IE**: illegal access interrupt enable for OCTOSPI1

0: interrupt disabled

1: interrupt enabled

Bit 23 **FMCIE**: illegal access interrupt enable for FMC

0: interrupt disabled

1: interrupt enabled

Bit 22 **SDMMC2IE**: illegal access interrupt enable for SDMMC2

0: interrupt disabled

1: interrupt enabled

Bit 21 **SDMMC1IE**: illegal access interrupt enable for SDMMC1

0: interrupt disabled

1: interrupt enabled

Bit 20 **PKAIE**: illegal access interrupt enable for PKA

0: interrupt disabled

1: interrupt enabled

Bit 19 **SAESIE**: illegal access interrupt enable for SAES

0: interrupt disabled

1: interrupt enabled

Bit 18 **RNGIE**: illegal access interrupt enable for RNG

0: interrupt disabled

1: interrupt enabled

Bit 17 **HASHIE**: illegal access interrupt enable for HASH

0: interrupt disabled

1: interrupt enabled

Bit 16 **AESIE**: illegal access interrupt enable for AES

0: interrupt disabled

1: interrupt enabled

Bit 15 **DCMIIE**: illegal access interrupt enable for DCMI

0: interrupt disabled

1: interrupt enabled

- Bit 14 **ADC12IE**: illegal access interrupt enable for ADC1 and ADC2
0: interrupt disabled
1: interrupt enabled
- Bit 13 **DCACHEIE**: illegal access interrupt enable for DCACHE
0: interrupt disabled
1: interrupt enabled
- Bit 12 **ICACHEIE**: illegal access interrupt enable for ICACHE
0: interrupt disabled
1: interrupt enabled
- Bit 11 **ETHIE**: illegal access interrupt enable for register of ETH
0: interrupt disabled
1: interrupt enabled
- Bit 10 **FMACIE**: illegal access interrupt enable for FMAC
0: interrupt disabled
1: interrupt enabled
- Bit 9 **CORDICIE**: illegal access interrupt enable for CORDIC
0: interrupt disabled
1: interrupt enabled
- Bit 8 **CRCIE**: illegal access interrupt enable for CRC
0: interrupt disabled
1: interrupt enabled
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **ADC3IE**: illegal access interrupt enable for ADC3
0: interrupt disabled
1: interrupt enabled
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **CCBIE**: illegal access interrupt enable for CCB
0: interrupt disabled
1: interrupt enabled
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **I3C2IE**: illegal access interrupt enable for I3C2
0: interrupt disabled
1: interrupt enabled
- Bit 1 **VREFBUFIE**: illegal access interrupt enable for VREFBUF
0: interrupt disabled
1: interrupt enabled
- Bit 0 **LPTIM6IE**: illegal access interrupt enable for LPTIM6
0: interrupt disabled
1: interrupt enabled

5.7.4 GTZC1 TZIC interrupt enable register 4 (GTZC1_TZIC_IER4)

Address offset: 0x00C

Reset value: 0x0000 0000

Secure privileged access only.

This register is used to enable interrupt of illegal access.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	MPCB B3_RE GIE	SRAM3 IE	MPCB B2_RE GIE	SRAM2 IE	MPCB B1_RE GIE	SRAM1 IE	Res.	Res.	Res.	BKPSR AMIE	FMC_ MEMIE	OCTOS PI1_M EMIE	TZIC1I E	TZSC1I E
		rw	rw	rw	rw	rw	rw				rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	PLAY1I E	EXTIIE	RCCIE	PWRIE	TAMPI E	RTCIE	SBSIE	Res.	OTFDE C1IE	FLASHI E	FLASH _REGI E	GPDM A2IE	GPDM A1IE
			rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **MPCBB3_REGIE**: illegal access interrupt enable for MPCBB3 registers

0: interrupt disabled
1: interrupt enabled

Bit 28 **SRAM3IE**: illegal access interrupt enable for SRAM3

0: interrupt disabled
1: interrupt enabled

Bit 27 **MPCBB2_REGIE**: illegal access interrupt enable for MPCBB2 registers

0: interrupt disabled
1: interrupt enabled

Bit 26 **SRAM2IE**: illegal access interrupt enable for SRAM2

0: interrupt disabled
1: interrupt enabled

Bit 25 **MPCBB1_REGIE**: illegal access interrupt enable for MPCBB1 registers

0: interrupt disabled
1: interrupt enabled

Bit 24 **SRAM1IE**: illegal access interrupt enable for SRAM1

0: interrupt disabled
1: interrupt enabled

Bits 23:21 Reserved, must be kept at reset value.

Bit 20 **BKPSRAMIE**: illegal access interrupt enable for MPCWM4 (BKPSRAM) memory bank

0: interrupt disabled
1: interrupt enabled

- Bit 19 **FMC_MEMIE**: illegal access interrupt enable for MPCWM2 (FMC_NOR bank), MPCWM3 (FMC_NAND bank and FMC_SDRAM bank 1), and MPCWM4 (FMC_SDRAM bank 2)
0: interrupt disabled
1: interrupt enabled
- Bit 18 **OCTOSPI1_MEMIE**: illegal access interrupt enable for MPCWM1 (OCTOSPI1) memory bank
0: interrupt disabled
1: interrupt enabled
- Bit 17 **TZIC1IE**: illegal access interrupt enable for GTZC1 TZIC registers
0: interrupt disabled
1: interrupt enabled
- Bit 16 **TZSC1IE**: illegal access interrupt enable for GTZC1 TZSC registers
0: interrupt disabled
1: interrupt enabled
- Bits 15:13 Reserved, must be kept at reset value.
- Bit 12 **PLAY1IE**: illegal access interrupt enable for PLAY1
0: interrupt disabled
1: interrupt enabled
- Bit 11 **EXTIIE**: illegal access interrupt enable for EXTI
0: interrupt disabled
1: interrupt enabled
- Bit 10 **RCCIE**: illegal access interrupt enable for RCC
0: interrupt disabled
1: interrupt enabled
- Bit 9 **PWRIE**: illegal access interrupt enable for PWR
0: interrupt disabled
1: interrupt enabled
- Bit 8 **TAMPIE**: illegal access interrupt enable for TAMP
0: interrupt disabled
1: interrupt enabled
- Bit 7 **RTCIE**: illegal access interrupt enable for RTC
0: interrupt disabled
1: interrupt enabled
- Bit 6 **SBSIE**: illegal access interrupt enable for SBS
0: interrupt disabled
1: interrupt enabled
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **OTFDEC1IE**: illegal access interrupt enable for OTFDEC1
0: interrupt disabled
1: interrupt enabled
- Bit 3 **FLASHIE**: illegal access interrupt enable for FLASH memory
0: interrupt disabled
1: interrupt enabled

Bit 2 **FLASH_REGIE**: illegal access interrupt enable for FLASH registers

0: interrupt disabled

1: interrupt enabled

Bit 1 **GPDMA2IE**: illegal access interrupt enable for GPDMA2

0: interrupt disabled

1: interrupt enabled

Bit 0 **GPDMA1IE**: illegal access interrupt enable for GPDMA1

0: interrupt disabled

1: interrupt enabled

5.7.5 GTZC1 TZIC status register 1 (GTZC1_TZIC_SR1)

Address offset: 0x010

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM2 F	DTSF	UART1 2F	UART9 F	UART8 F	UART7 F	DAC1F	HDMIC ECF	USART 11F	USART 10F	USART 6F	CRSF	I3C1F	I2C2F	I2C1F	UART5 F
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
UART4 F	USART 3F	USART 2F	SPI3F	SPI2F	IWDGF	WWDG F	TIM14F	TIM13F	TIM12F	TIM7F	TIM6F	TIM5F	TIM4F	TIM3F	TIM2F
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **LPTIM2F**: illegal access flag for LPTIM2

0: no illegal access event

1: illegal access event

Bit 30 **DTSF**: illegal access flag for DTS

0: no illegal access event

1: illegal access event

Bit 29 **UART12F**: illegal access flag for UART12

0: no illegal access event

1: illegal access event

Bit 28 **UART9F**: illegal access flag for UART9

0: no illegal access event

1: illegal access event

Bit 27 **UART8F**: illegal access flag for UART8

0: no illegal access event

1: illegal access event

Bit 26 **UART7F**: illegal access flag for UART7

0: no illegal access event

1: illegal access event

- Bit 25 **DAC1F**: illegal access flag for DAC1
0: no illegal access event
1: illegal access event
- Bit 24 **HDMICECF**: illegal access flag for HDMICEC
0: no illegal access event
1: illegal access event
- Bit 23 **USART11F**: illegal access flag for USART11
0: no illegal access event
1: illegal access event
- Bit 22 **USART10F**: illegal access flag for USART10
0: no illegal access event
1: illegal access event
- Bit 21 **USART6F**: illegal access flag for USART6
0: no illegal access event
1: illegal access event
- Bit 20 **CRSF**: illegal access flag for CRS
0: no illegal access event
1: illegal access event
- Bit 19 **I3C1F**: illegal access flag for I3C1
0: no illegal access event
1: illegal access event
- Bit 18 **I2C2F**: illegal access flag for I2C2
0: no illegal access event
1: illegal access event
- Bit 17 **I2C1F**: illegal access flag for I2C1
0: no illegal access event
1: illegal access event
- Bit 16 **UART5F**: illegal access flag for UART5
0: no illegal access event
1: illegal access event
- Bit 15 **UART4F**: illegal access flag for UART4
0: no illegal access event
1: illegal access event
- Bit 14 **USART3F**: illegal access flag for USART3
0: no illegal access event
1: illegal access event
- Bit 13 **USART2F**: illegal access flag for USART2
0: no illegal access event
1: illegal access event
- Bit 12 **SPI3F**: illegal access flag for SPI3
0: no illegal access event
1: illegal access event
- Bit 11 **SPI2F**: illegal access flag for SPI2
0: no illegal access event
1: illegal access event

- Bit 10 **IWDGF**: illegal access flag for IWDG
0: no illegal access event
1: illegal access event
- Bit 9 **WWDGF**: illegal access flag for WWDG
0: no illegal access event
1: illegal access event
- Bit 8 **TIM14F**: illegal access flag for TIM14
0: no illegal access event
1: illegal access event
- Bit 7 **TIM13F**: illegal access flag for TIM13
0: no illegal access event
1: illegal access event
- Bit 6 **TIM12F**: illegal access flag for TIM12
0: no illegal access event
1: illegal access event
- Bit 5 **TIM7F**: illegal access flag for TIM7
0: no illegal access event
1: illegal access event
- Bit 4 **TIM6F**: illegal access flag for TIM6
0: no illegal access event
1: illegal access event
- Bit 3 **TIM5F**: illegal access flag for TIM5
0: no illegal access event
1: illegal access event
- Bit 2 **TIM4F**: illegal access flag for TIM4
0: no illegal access event
1: illegal access event
- Bit 1 **TIM3F**: illegal access flag for TIM3
0: no illegal access event
1: illegal access event
- Bit 0 **TIM2F**: illegal access flag for TIM2
0: no illegal access event
1: illegal access event

5.7.6 GTZC1 TZIC status register 2 (GTZC1_TZIC_SR2)

Address offset: 0x014

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPTIM5F	LPTIM4F	LPTIM3F	LPTIM1F	I2C4F	I2C3F	LPUART1F	SPI5F	Res.	Res.	Res.	Res.	USBF	SAI2F	SAI1F	SPI6F
r	r	r	r	r	r	r	r					r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI4F	TIM17F	TIM16F	TIM15F	USART1F	TIM8F	SPI1F	TIM1F	Res.	Res.	Res.	COMP F	Res.	UCPDF	FDCAN2F	FDCAN1F
r	r	r	r	r	r	r	r				r		r	r	r

Bit 31 **LPTIM5F**: illegal access flag for LPTIM5

0: no illegal access event

1: illegal access event

Bit 30 **LPTIM4F**: illegal access flag for LPTIM4

0: no illegal access event

1: illegal access event

Bit 29 **LPTIM3F**: illegal access flag for LPTIM3

0: no illegal access event

1: illegal access event

Bit 28 **LPTIM1F**: illegal access flag for LPTIM1

0: no illegal access event

1: illegal access event

Bit 27 **I2C4F**: illegal access flag for I2C4

0: no illegal access event

1: illegal access event

Bit 26 **I2C3F**: illegal access flag for I2C3

0: no illegal access event

1: illegal access event

Bit 25 **LPUART1F**: illegal access flag for LPUART

0: no illegal access event

1: illegal access event

Bit 24 **SPI5F**: illegal access flag for SPI5

0: no illegal access event

1: illegal access event

Bits 23:20 Reserved, must be kept at reset value.

Bit 19 **USBF**: illegal access flag for USB

0: no illegal access event

1: illegal access event

Bit 18 **SAI2F**: illegal access flag for SAI2

0: no illegal access event

1: illegal access event

Bit 17 **SAI1F**: illegal access flag for SAI1

0: no illegal access event

1: illegal access event

Bit 16 **SPI6F**: illegal access flag for SPI6

0: no illegal access event

1: illegal access event

- Bit 15 **SPI4F**: illegal access flag for SPI4
0: no illegal access event
1: illegal access event
- Bit 14 **TIM17F**: illegal access flag for TIM17
0: no illegal access event
1: illegal access event
- Bit 13 **TIM16F**: illegal access flag for TIM16
0: no illegal access event
1: illegal access event
- Bit 12 **TIM15F**: illegal access flag for TIM15
0: no illegal access event
1: illegal access event
- Bit 11 **USART1F**: illegal access flag for USART1
0: no illegal access event
1: illegal access event
- Bit 10 **TIM8F**: illegal access flag for TIM8
0: no illegal access event
1: illegal access event
- Bit 9 **SPI1F**: illegal access flag for SPI1
0: no illegal access event
1: illegal access event
- Bit 8 **TIM1F**: illegal access flag for TIM1
0: no illegal access event
1: illegal access event
- Bits 7:5 Reserved, must be kept at reset value.
- Bit 4 **COMPF**: illegal access flag for COMP
0: no illegal access event
1: illegal access event
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **UCPDF**: illegal access flag for UCPD
0: no illegal access event
1: illegal access event
- Bit 1 **FDCAN2F**: illegal access flag for FDCAN2
0: no illegal access event
1: illegal access event
- Bit 0 **FDCAN1F**: illegal access flag for FDCAN1
0: no illegal access event
1: illegal access event

5.7.7 GTZC1 TZIC status register 3 (GTZC1_TZIC_SR3)

Address offset: 0x018

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	RAMC FGF	Res.	OCTOS PI1F	FMCF	SDMM C2F	SDMM C1F	PKAF	SAESF	RNGF	HASHF	AESF
					r		r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DCMIF	ADC12 F	DCAC HEF	ICACH EF	ETHF	FMACF	CORDI CF	CRCF	Res.	ADC3F	Res.	CCBF	Res.	I3C2F	VREFB UFF	LPTIM6 F
r	r	r	r	r	r	r	r		r		r		r	r	r

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **RAMCFGF**: illegal access flag for RAMSCFG

- 0: no illegal access event
- 1: illegal access event

Bit 25 Reserved, must be kept at reset value.

Bit 24 **OCTOSPI1F**: illegal access flag for OCTOSPI1

- 0: no illegal access event
- 1: illegal access event

Bit 23 **FMCF**: illegal access flag for FMC

- 0: no illegal access event
- 1: illegal access event

Bit 22 **SDMMC2F**: illegal access flag for SDMMC2

- 0: no illegal access event
- 1: illegal access event

Bit 21 **SDMMC1F**: illegal access flag for SDMMC1

- 0: no illegal access event
- 1: illegal access event

Bit 20 **PKAF**: illegal access flag for PKA

- 0: no illegal access event
- 1: illegal access event

Bit 19 **SAESF**: illegal access flag for SAES

- 0: no illegal access event
- 1: illegal access event

Bit 18 **RNGF**: illegal access flag for RNG

- 0: no illegal access event
- 1: illegal access event

Bit 17 **HASHF**: illegal access flag for HASH

- 0: no illegal access event
- 1: illegal access event

- Bit 16 **AESF**: illegal access flag for AES
0: no illegal access event
1: illegal access event
- Bit 15 **DCMIF**: illegal access flag for DCMIF
0: no illegal access event
1: illegal access event
- Bit 14 **ADC12F**: illegal access flag for ADC1 and ADC2
0: no illegal access event
1: illegal access event
- Bit 13 **DCACHEF**: illegal access flag for DCACHE
0: no illegal access event
1: illegal access event
- Bit 12 **ICACHEF**: illegal access flag for ICACHE
0: no illegal access event
1: illegal access event
- Bit 11 **ETHF**: illegal access flag for register of ETH
0: no illegal access event
1: illegal access event
- Bit 10 **FMACF**: illegal access flag for FMAF
0: no illegal access event
1: illegal access event
- Bit 9 **CORDICF**: illegal access flag for CORDIC
0: no illegal access event
1: illegal access event
- Bit 8 **CRCF**: illegal access flag for CRC
0: no illegal access event
1: illegal access event
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **ADC3F**: illegal access flag for ADC3
0: no illegal access event
1: illegal access event
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **CCBF**: illegal access flag for CCB
0: no illegal access event
1: illegal access event
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **I3C2F**: illegal access flag for I3C2
0: no illegal access event
1: illegal access event
- Bit 1 **VREFBUFF**: illegal access flag for VREFBUF
0: no illegal access event
1: illegal access event

Bit 0 **LPTIM6F**: illegal access flag for LPTIM6

0: no illegal access event

1: illegal access event

5.7.8 GTZC1 TZIC status register 4 (GTZC1_TZIC_SR4)

Address offset: 0x01C

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	MPCB B3_RE GF	SRAM3 F	MPCB B2_RE GF	SRAM2 F	MPCB B1_RE GF	SRAM1 F	Res.	Res.	Res.	BKPSR AMF	FMC MEMF	OCTOS PI1_M EMF	TZIC1F	TZSC1 F
		r	r	r	r	r	r				r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	PLAY1 F	EXTIF	RCCF	PWRF	TAMPF	RTCF	SBSF	Res.	OTFDE C1F	FLASH F	FLASH _REGF	GPDM A2F	GPDM A1F
			r	r	r	r	r	r	r		r	r	r	r	r

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **MPCBB3_REGF**: illegal access flag for MPCBB3 registers

0: no illegal access event

1: illegal access event

Bit 28 **SRAM3F**: illegal access flag for SRAM3

0: no illegal access event

1: illegal access event

Bit 27 **MPCBB2_REGF**: illegal access flag for MPCBB2 registers

0: no illegal access event

1: illegal access event

Bit 26 **SRAM2F**: illegal access flag for SRAM2

0: no illegal access event

1: illegal access event

Bit 25 **MPCBB1_REGF**: illegal access flag for MPCBB1 registers

0: no illegal access event

1: illegal access event

Bit 24 **SRAM1F**: illegal access flag for SRAM1

0: no illegal access event

1: illegal access event

Bits 23:21 Reserved, must be kept at reset value.

Bit 20 **BKPSRAMF**: illegal access flag for MPCWM4 (BKPSRAM) memory bank

0: no illegal access event

1: illegal access event

- Bit 19 **FMC_MEMF**: illegal access flag for MPCWM2 (FMC_NOR bank), MPCWM3 (FMC_NAND bank and FMC_SDRAM bank 1), and MPCWM4 (FMC_SDRAM bank 2)
0: no illegal access event
1: illegal access event
- Bit 18 **OCTOSPI1_MEMF**: illegal access flag for MPCWM1 (OCTOSPI1) memory bank
0: no illegal access event
1: illegal access event
- Bit 17 **TZIC1F**: illegal access flag for GTZC1 TZIC registers
0: no illegal access event
1: illegal access event
- Bit 16 **TZSC1F**: illegal access flag for GTZC1 TZSC registers
0: no illegal access event
1: illegal access event
- Bits 15:13 Reserved, must be kept at reset value.
- Bit 12 **PLAY1F**: illegal access flag for PLAY1
0: no illegal access event
1: illegal access event
- Bit 11 **EXTIF**: illegal access flag for EXTI
0: no illegal access event
1: illegal access event
- Bit 10 **RCCF**: illegal access flag for RCC
0: no illegal access event
1: illegal access event
- Bit 9 **PWRF**: illegal access flag for PWR
0: no illegal access event
1: illegal access event
- Bit 8 **TAMPF**: illegal access flag for TAMP
0: no illegal access event
1: illegal access event
- Bit 7 **RTCF**: illegal access flag for RTC
0: no illegal access event
1: illegal access event
- Bit 6 **SBSF**: illegal access flag for SBS
0: no illegal access event
1: illegal access event
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **OTFDEC1F**: illegal access flag for OTFDEC1
0: no illegal access event
1: illegal access event
- Bit 3 **FLASHF**: illegal access flag for FLASH memory
0: no illegal access event
1: illegal access event

Bit 2 **FLASH_REGF**: illegal access flag for FLASH registers

0: no illegal access event

1: illegal access event

Bit 1 **GPDMA2F**: illegal access flag for GPDMA2

0: no illegal access event

1: illegal access event

Bit 0 **GPDMA1F**: illegal access flag for GPDMA1

0: no illegal access event

1: illegal access event

5.7.9 GTZC1 TZIC flag clear register 1 (GTZC1_TZIC_FCR1)

Address offset: 0x020

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLPTIM2F	CDTSF	CUART12F	CUART9F	CUART8F	CUART7F	CDAC1F	CHDMI CECF	CUSART11F	CUSART10F	CUSART6F	CCRSF	CI3C1F	CI2C2F	CI2C1F	CUART5F
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CUART4F	CUSART3F	CUSART2F	CSPI3F	CSPI2F	CIWDGF	CWWDGF	CTIM14F	CTIM13F	CTIM12F	CTIM7F	CTIM6F	CTIM5F	CTIM4F	CTIM3F	CTIM2F
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bit 31 **CLPTIM2F**: clear the illegal access flag for LPTIM2

0: no action

1: status flag cleared

Bit 30 **CDTSF**: clear the illegal access flag for DTS

0: no action

1: status flag cleared

Bit 29 **CUART12F**: clear the illegal access flag for UART12

0: no action

1: status flag cleared

Bit 28 **CUART9F**: clear the illegal access flag for UART9

0: no action

1: status flag cleared

Bit 27 **CUART8F**: clear the illegal access flag for UART8

0: no action

1: status flag cleared

Bit 26 **CUART7F**: clear the illegal access flag for UART7

0: no action

1: status flag cleared

- Bit 25 **CDAC1F**: clear the illegal access flag for DAC1
0: no action
1: status flag cleared
- Bit 24 **CHDMICECF**: clear the illegal access flag for HDMICEC
0: no action
1: status flag cleared
- Bit 23 **CUSART11F**: clear the illegal access flag for USART11
0: no action
1: status flag cleared
- Bit 22 **CUSART10F**: clear the illegal access flag for USART10
0: no action
1: status flag cleared
- Bit 21 **CUSART6F**: clear the illegal access flag for USART6
0: no action
1: status flag cleared
- Bit 20 **CCRSF**: clear the illegal access flag for CRS
0: no action
1: status flag cleared
- Bit 19 **CI3C1F**: clear the illegal access flag for I3C1
0: no action
1: status flag cleared
- Bit 18 **CI2C2F**: clear the illegal access flag for I2C2
0: no action
1: status flag cleared
- Bit 17 **CI2C1F**: clear the illegal access flag for I2C1
0: no action
1: status flag cleared
- Bit 16 **CUART5F**: clear the illegal access flag for UART5
0: no action
1: status flag cleared
- Bit 15 **CUART4F**: clear the illegal access flag for UART4
0: no action
1: status flag cleared
- Bit 14 **CUSART3F**: clear the illegal access flag for USART3
0: no action
1: status flag cleared
- Bit 13 **CUSART2F**: clear the illegal access flag for USART2
0: no action
1: status flag cleared
- Bit 12 **CSPI3F**: clear the illegal access flag for SPI3
0: no action
1: status flag cleared
- Bit 11 **CSPI2F**: clear the illegal access flag for SPI2
0: no action
1: status flag cleared

- Bit 10 **CIWDGF**: clear the illegal access flag for IWDG
0: no action
1: status flag cleared
- Bit 9 **CWWDGF**: clear the illegal access flag for WWDG
0: no action
1: status flag cleared
- Bit 8 **CTIM14F**: clear the illegal access flag for TIM14
0: no action
1: status flag cleared
- Bit 7 **CTIM13F**: clear the illegal access flag for TIM13
0: no action
1: status flag cleared
- Bit 6 **CTIM12F**: clear the illegal access flag for TIM12
0: no action
1: status flag cleared
- Bit 5 **CTIM7F**: clear the illegal access flag for TIM7
0: no action
1: status flag cleared
- Bit 4 **CTIM6F**: clear the illegal access flag for TIM6
0: no action
1: status flag cleared
- Bit 3 **CTIM5F**: clear the illegal access flag for TIM5
0: no action
1: status flag cleared
- Bit 2 **CTIM4F**: clear the illegal access flag for TIM4
0: no action
1: status flag cleared
- Bit 1 **CTIM3F**: clear the illegal access flag for TIM3
0: no action
1: status flag cleared
- Bit 0 **CTIM2F**: clear the illegal access flag for TIM2
0: no action
1: status flag cleared

5.7.10 GTZC1 TZIC flag clear register 2 (GTZC1_TZIC_FCR2)

Address offset: 0x024

Reset value: 0x0000 0000

Secure privileged access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CLPTIM5F	CLPTIM4F	CLPTIM3F	CLPTIM1F	CI2C4F	CI2C3F	CLPUART1F	CSPI5F	Res.	Res.	Res.	Res.	CUSBF	CSAI2F	CSAI1F	CSPI6F
w	w	w	w	w	w	w	w					w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CSPI4F	CTIM17F	CTIM16F	CTIM15F	CUSART1F	CTIM8F	CSPI1F	CTIM1F	Res.	Res.	Res.	CCOMPF	Res.	CUCPDF	CFDCA N2F	CFDCA N1F
w	w	w	w	w	w	w	w				w		w	w	w

Bit 31 **CLPTIM5F**: clear the illegal access flag for LPTIM5

0: no action

1: status flag cleared

Bit 30 **CLPTIM4F**: clear the illegal access flag for LPTIM4

0: no action

1: status flag cleared

Bit 29 **CLPTIM3F**: clear the illegal access flag for LPTIM3

0: no action

1: status flag cleared

Bit 28 **CLPTIM1F**: clear the illegal access flag for LPTIM1

0: no action

1: status flag cleared

Bit 27 **CI2C4F**: clear the illegal access flag for I2C4

0: no action

1: status flag cleared

Bit 26 **CI2C3F**: clear the illegal access flag for I2C3

0: no action

1: status flag cleared

Bit 25 **CLPUART1F**: clear the illegal access flag for LPUART

0: no action

1: status flag cleared

Bit 24 **CSPI5F**: clear the illegal access flag for SPI5

0: no action

1: status flag cleared

Bits 23:20 Reserved, must be kept at reset value.

Bit 19 **CUSBF**: clear the illegal access flag for USB

0: no action

1: status flag cleared

Bit 18 **CSAI2F**: clear the illegal access flag for SAI2

0: no action

1: status flag cleared

Bit 17 **CSAI1F**: clear the illegal access flag for SAI1

0: no action

1: status flag cleared

Bit 16 **CSPI6F**: clear the illegal access flag for SPI6

0: no action

1: status flag cleared

- Bit 15 **CSPI4F**: clear the illegal access flag for SPI4
0: no action
1: status flag cleared
- Bit 14 **CTIM17F**: clear the illegal access flag for TIM17
0: no action
1: status flag cleared
- Bit 13 **CTIM16F**: clear the illegal access flag for TIM16
0: no action
1: status flag cleared
- Bit 12 **CTIM15F**: clear the illegal access flag for TIM15
0: no action
1: status flag cleared
- Bit 11 **CUSART1F**: clear the illegal access flag for USART1
0: no action
1: status flag cleared
- Bit 10 **CTIM8F**: clear the illegal access flag for TIM8
0: no action
1: status flag cleared
- Bit 9 **CSPI1F**: clear the illegal access flag for SPI1
0: no action
1: status flag cleared
- Bit 8 **CTIM1F**: clear the illegal access flag for TIM1
0: no action
1: status flag cleared
- Bits 7:5 Reserved, must be kept at reset value.
- Bit 4 **CCOMPF**: clear the illegal access flag for COMP
0: no action
1: status flag cleared
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **CUCPDF**: clear the illegal access flag for UCPD
0: no action
1: status flag cleared
- Bit 1 **CFDCAN2F**: clear the illegal access flag for FDCAN2
0: no action
1: status flag cleared
- Bit 0 **CFDCAN1F**: clear the illegal access flag for FDCAN1
0: no action
1: status flag cleared

5.7.11 GTZC1 TZIC flag clear register 3 (GTZC1_TZIC_FCR3)

Address offset: 0x028

Reset value: 0x0000 0000

Secure privilege access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	CRAM CFGF	Res.	COCT OSPI1 F	CFMCF	CSDM MC2F	CSDM MC1F	CPKAF	CSAES F	CRNG F	CHASH F	CAESF
					w		w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CDCMI F	CADC1 2F	CDCA CHEF	CICAC HEF	CETHF	CFMA CF	CCOR DICF	CCRCF	Res.	CADC3 F	Res.	CCCBF	Res.	CI3C2F	CVREF BUFF	CLPT1 M6F
w	w	w	w	w	w	w	w		w		w		w	w	w

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **CRAMCFGF**: clear illegal access flag for RAMSCFG

0: no action

1: status flag cleared

Bit 25 Reserved, must be kept at reset value.

Bit 24 **COCTOSPI1F**: clear illegal access flag for OCTOSPI1

0: no action

1: status flag cleared

Bit 23 **CFMCF**: clear illegal access flag for FMC

0: no action

1: status flag cleared

Bit 22 **CSDMMC2F**: clear illegal access flag for SDMMC2

0: no action

1: status flag cleared

Bit 21 **CSDMMC1F**: clear illegal access flag for SDMMC1

0: no action

1: status flag cleared

Bit 20 **CPKAF**: clear illegal access flag for PKA

0: no action

1: status flag cleared

Bit 19 **CSAESF**: clear illegal access flag for SAES

0: no action

1: status flag cleared

Bit 18 **CRNGF**: clear illegal access flag for RNG

0: no action

1: status flag cleared

Bit 17 **CHASHF**: clear illegal access flag for HASH

0: no action

1: status flag cleared

Bit 16 **CAESF**: clear illegal access flag for AES

0: no action

1: status flag cleared

- Bit 15 **CDCMIF**: clear illegal access flag for DCMI
0: no action
1: status flag cleared
- Bit 14 **CADC12F**: clear illegal access flag for ADC1 and ADC2
0: no action
1: status flag cleared
- Bit 13 **CDCACHEF**: clear illegal access flag for DCACHE
0: no action
1: status flag cleared
- Bit 12 **CICACHEF**: clear illegal access flag for ICACHE
0: no action
1: status flag cleared
- Bit 11 **CETHF**: clear illegal access flag for register of ETH
0: no action
1: status flag cleared
- Bit 10 **CFMACF**: clear illegal access flag for FMAC
0: no action
1: status flag cleared
- Bit 9 **CCORDICF**: clear illegal access flag for CORDIC
0: no action
1: status flag cleared
- Bit 8 **CCRCF**: clear illegal access flag for CRC
0: no action
1: status flag cleared
- Bit 7 Reserved, must be kept at reset value.
- Bit 6 **CADC3F**: clear illegal access flag for ADC3
0: no action
1: status flag cleared
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **CCCBF**: clear illegal access flag for CCB
0: no action
1: status flag cleared
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **CI3C2F**: clear illegal access flag for I3C2
0: no action
1: status flag cleared
- Bit 1 **CVREFBUFF**: clear illegal access flag for VREFBUF
0: no action
1: status flag cleared
- Bit 0 **CLPTIM6F**: clear illegal access flag for LPTIM6
0: no action
1: status flag cleared

5.7.12 GTZC1 TZIC flag clear register 4 (GTZC1_TZIC_FCR4)

Address offset: 0x02C

Reset value: 0x0000 0000

Secure privilege access only.

Refer to the device datasheet for the peripheral availability. If not present, consider the associated bit as reserved, and keep it at reset value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	CMPC BB3_R EGF	CSRA M3F	CMPC BB2_R EGF	CSRA M2F	CMPC BB1_R EGF	CSRA M1F	Res.	Res.	Res.	CBKPS RAMF	CFMC_ MEMF	COCT OSPI1_ MEMF	CTZIC1 F	CTZSC 1F
		w	w	w	w	w	w				w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CPLAY 1F	CEXTI F	CRCCF	CPWR F	CTAMP F	CRTCF	CSBSF	Res.	COTFD EC1F	CFLAS HF	CFLAS H_REG F	CGPD MA2F	CGPD MA1F
			w	w	w	w	w	w	w		w	w	w	w	w

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **CMPCBB3_REGF**: clear the illegal access flag for MPCBB3 registers

0: no action

1: status flag cleared

Bit 28 **CSRAM3F**: clear the illegal access flag for SRAM3

0: no action

1: status flag cleared

Bit 27 **CMPCBB2_REGF**: clear the illegal access flag for MPCBB2 registers

0: no action

1: status flag cleared

Bit 26 **CSRAM2F**: clear the illegal access flag for SRAM2

0: no action

1: status flag cleared

Bit 25 **CMPCBB1_REGF**: clear the illegal access flag for MPCBB1 registers

0: no action

1: status flag cleared

Bit 24 **CSRAM1F**: clear the illegal access flag for SRAM1

0: no action

1: status flag cleared

Bits 23:21 Reserved, must be kept at reset value.

Bit 20 **CBKPSRAMF**: clear the illegal access flag for MPCWM4 (BKPSRAM) memory bank

0: no action

1: status flag cleared

Bit 19 **CFMC_MEMF**: clear the illegal access flag for MPCWM2 (FMC_NOR bank), MPCWM3 (FMC_NAND bank and FMC_SDRAM bank 1), and MPCWM4 (FMC_SDRAM bank 2)

0: no action

1: status flag cleared

- Bit 18 **COCTOSPI1_MEMF**: clear the illegal access flag for MPCWM1 (OCTOSPI1) memory bank
0: no action
1: status flag cleared
- Bit 17 **CTZIC1F**: clear the illegal access flag for GTZC1 TZIC registers
0: no action
1: status flag cleared
- Bit 16 **CTZSC1F**: clear the illegal access flag for GTZC1 TZSC registers
0: no action
1: status flag cleared
- Bits 15:13 Reserved, must be kept at reset value.
- Bit 12 **CPLAY1F**: clear the illegal access flag for PLAY1
0: no action
1: status flag cleared
- Bit 11 **CEXTIF**: clear the illegal access flag for EXTI
0: no action
1: status flag cleared
- Bit 10 **CRCCF**: clear the illegal access flag for RCC
0: no action
1: status flag cleared
- Bit 9 **CPWRF**: clear the illegal access flag for PWR
0: no action
1: status flag cleared
- Bit 8 **CTAMPF**: clear the illegal access flag for TAMP
0: no action
1: status flag cleared
- Bit 7 **CRTCF**: clear the illegal access flag for RTC
0: no action
1: status flag cleared
- Bit 6 **CSBSF**: clear the illegal access flag for SBS
0: no action
1: status flag cleared
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **COTFDEC1F**: clear the illegal access flag for OTFDEC1
0: no action
1: status flag cleared
- Bit 3 **CFLASHF**: clear the illegal access flag for FLASH memory
0: no action
1: status flag cleared
- Bit 2 **CFLASH_REGF**: clear the illegal access flag for FLASH registers
0: no action
1: status flag cleared
- Bit 1 **CGPDMA2F**: clear the illegal access flag for GPDMA2
0: no action
1: status flag cleared

Bit 0 **CGPDMA1F**: clear the illegal access flag for GPDMA1

0: no action

1: status flag cleared

5.7.13 GTZC1 TZIC register map

Table 40. GTZC1 TZIC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	GTZC1_TZIC_IER1	LPTIM2IE	DTSIE	UART12IE	UART9IE	UART8IE	UART7IE	DAC1IE	HDMICECIE	USART11IE	USART10IE	USART6IE	CRSIE	I3C1IE	I2C2IE	I2C1IE	UART5IE	UART4IE	USART3IE	USART2IE	SPI3IE	SPI2IE	IWDGIE	WWDGIE	TIM14IE	TIM13IE	TIM12IE	TIM7IE	TIM6IE	TIM5IE	TIM4IE	TIM3IE	TIM2IE
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004	GTZC1_TZIC_IER2	LPTIM5IE	LPTIM4IE	LPTIM3IE	LPTIM1IE	I2C4IE	I2C3IE	LPUART1IE	SPI5IE	Res.	Res.	Res.	Res.	USBIE	SAI2IE	SAI1IE	SPI6IE	SPI4IE	TIM17IE	TIM16IE	TIM15IE	USART11IE	TIM8IE	SPI1IE	TIM1IE	Res.	Res.	Res.	COMPIE	Res.	UCPDIE	FDCAN2IE	FDCAN1IE
	Reset value	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008	GTZC1_TZIC_IER3	Res.	Res.	Res.	Res.	Res.	RAMCFGIE	Res.	OCTOSP11IE	FMCIIE	SDMMC2IE	SDMMC1IE	PKAIE	SAESIE	RNGIE	HASHIE	AESIE	DCMIIE	ADC12IE	DCACHEIE	ICACHEIE	ETHIE	FMACIE	CORDICIE	CRCIE	Res.	ADC3IE	Res.	CCBIE	Res.	I3C2IE	VREFBUFIE	LPTIM6IE
	Reset value						0		0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	GTZC1_TZIC_IER4	Res.	Res.	MPCBB3_REGIE	SRAM3IE	MPCBB2_REGIE	SRAM2IE	MPCBB1_REGIE	SRAM1IE	Res.	Res.	Res.	BKPSRAMIE	FMC_MEMIE	OCTOSP11_MEMIE	TZIC1IE	TZSC1IE	Res.	Res.	Res.	PLAY1IE	EXT1IE	RCCIE	PWRIE	TAMPIE	RTCIE	SBSIE		OTFDEC1IE	FLASHIE	FLASH_REGIE	GPDMA2IE	GPDMA1IE
	Reset value			0	0	0	0	0	0				0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0
0x010	GTZC1_TZIC_SR1	LPTIM2F	DTSF	UART12F	UART9F	UART8F	UART7F	DAC1F	HDMICECF	USART11F	USART10F	USART6F	CRSF	I3C1F	I2C2F	I2C1F	UART5F	UART4F	USART3F	USART2F	SPI3F	SPI2F	IWDGF	WWDGF	TIM14F	TIM13F	TIM12F	TIM7F	TIM6F	TIM5F	TIM4F	TIM3F	TIM2F
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	GTZC1_TZIC_SR2	LPTIM5F	LPTIM4F	LPTIM3F	LPTIM1F	I2C4F	I2C3F	LPUART1F	SPI5F	Res.	Res.	Res.	Res.	USBF	SAI2F	SAI1F	SPI6F	SPI4F	TIM17F	TIM16F	TIM15F	USART11F	TIM8F	SPI1F	TIM1F	Res.	Res.	Res.	COMPF	Res.	UCPDF	FDCAN2F	FDCAN1F
	Reset value	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018	GTZC1_TZIC_SR3	Res.	Res.	Res.	Res.	Res.	RAMCFGF	Res.	OCTOSP11F	FMCF	SDMMC2F	SDMMC1F	PKAF	SAESF	RNGF	HASHF	AESF	DCMIF	ADC12F	DCACHEF	ICACHEF	ETHF	FMACF	CORDICF	CRCF	Res.	ADC3F	Res.	CCBF	Res.	I3C2F	VREFBUFF	LPTIM6F
	Reset value						0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x01C	GTZC1_TZIC_SR4	Res.	Res.	MPCBB3_REGF	SRAM3F	MPCBB2_REGF	SRAM2F	MPCBB1_REGF	SRAM1F	Res.	Res.	Res.	BKPSRAMF	FMC_MEMF	OCTOSP11_MEMF	TZIC1F	TZSC1F	Res.	Res.	Res.	PLAY1F	EXT1F	RCCF	PWRF	TAMPF	RTCF	SBSF	Res.	OTFDEC1F	FLASHF	FLASH_REGF	GPDMA2F	GPDMA1F
	Reset value			0	0	0	0	0	0				0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0

Table 40. GTZC1 TZIC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x020	GTZC1_TZIC_FCR1	CLPTIM2F	CDTSF	CUART12F	CUART9F	CUART8F	CUART7F	CDAC1F	CHDMICECF	CUSART11F	CUSART10F	CUSART6F	CCRSF	CI3C1F	CI2C2F	CI2C1F	CUART5F	CUART4F	CUSART3F	CUSART2F	CSP13F	CSP12F	CIWDGF	CWWDGF	CTIM14F	CTIM13F	CTIM12F	CTIM7F	CTIM6F	CTIM5F	CTIM4F	CTIM3F	CTIM2F
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x024	GTZC1_TZIC_FCR2	CLPTIM5F	CLPTIM4F	CLPTIM3F	CLPTIM1F	CI2C4F	CI2C3F	CLPUART1F	CSP15F	Res.	Res.	Res.	Res.	CUSBF	CSA12F	CSA11F	CSP16F	CSP14F	CTIM17F	CTIM16F	CTIM15F	CUSART11F	CTIM8F	CSP11F	CTIM1F	Res.	Res.	Res.	CCOMPF	Res.	CUCPDF	CFDCAN2F	CFDCAN1F
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x028	GTZC1_TZIC_FCR3	Res.	Res.	Res.	Res.	Res.	CRAMCFGF	Res.	COCTOSPI1F	CFMCF	CSDMMC2F	CSDMMC1F	CPKAF	CSAESF	CRNGF	CHASHF	CAESF	CDCMIF	CADC12F	CDCACHEF	CICACHEF	CETHF	CFMACF	CCORDICF	CCRCF	Res.	CADC3F	Res.	CCCBF	Res.	CI3C2F	CVREFBUFF	CLPTIM6F
	Reset value						0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x02C	GTZC1_TZIC_FCR4	Res.	Res.	CMPCBB3_REGF	CSRAM3F	CMPCBB2_REGF	CSRAM2F	CMPCBB1_REGF	CSRAM1F	Res.	Res.	Res.	CBKPSRAMF	CFMC_MEMF	COCTOSPI1_MEMF	CTZIC1F	CTZSC1F	Res.	Res.	Res.	CPLAY1F	CEXTIF	CRCCF	CPWRF	CTAMPF	CRTCF	CSBSF	Res.	COTFDEC1F	CFLASHF	CFLASH_REGF	CGPDMA2F	CGPDMA1F
	Reset value			0	0	0	0	0	0				0	0	0	0	0				0	0	0	0	0	0	0		0	0	0	0	0

Refer to [Table 30: GTZC1 subblock address offset](#).

5.8 GTZC1 MPCBBz registers (z = 1 to 3)

All registers are accessed only by words (32-bit).

5.8.1 GTZC1 SRAMz MPCBB control register (GTZC1_MPCBBz_CR) (z = 1 to 3)

Address offset: 0x000

Reset value: 0x0000 0000

Secure privileged access only.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRWILADIS	INVSECSTATE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw														
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GLOCK
															rs

Bit 31 **SRWILADIS**: secure read/write illegal access disable

This bit disables the detection of an illegal access when a secure read/write transaction access a nonsecure blocks of the block-based SRAM (secure fetch on nonsecure block is always considered illegal).

0: enabled, secure read/write access not allowed on nonsecure SRAM block

1: disabled, secure read/write access allowed on nonsecure SRAM block

Bit 30 **INVSECSTATE**: SRAMx clocks security state

This bit is used to define the internal SRAMs clocks control in RCC as secure or not.

0: SRAMs clocks are secure if a secure area exists in the MPCBB. It is non secure if there is no secure area.

1: SRAMs clocks are nonsecure even if a secure area exists in the MPCBB, and secure even if no secure block is set in the MPCBB.

Bits 29:1 Reserved, must be kept at reset value.

Bit 0 **GLOCK**: lock the control register of the MPCBB until next reset

This bit is cleared by default, and once set, it cannot be reset until system reset.

0: control register not locked

1: control register locked

5.8.2 GTZC1 SRAMz MPCBB configuration lock register 1 (GTZC1_MPCBBz_CFGLOCK1) (z = 1 to 3)

Address offset: 0x010

Reset value: 0x0000 0000

Secure privileged access only.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SPLCK 31	SPLCK 30	SPLCK 29	SPLCK 28	SPLCK 27	SPLCK 26	SPLCK 25	SPLCK 24	SPLCK 23	SPLCK 22	SPLCK 21	SPLCK 20	SPLCK 19	SPLCK 18	SPLCK 17	SPLCK 16
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPLCK 15	SPLCK 14	SPLCK 13	SPLCK 12	SPLCK 11	SPLCK 10	SPLCK 9	SPLCK 8	SPLCK 7	SPLCK 6	SPLCK 5	SPLCK 4	SPLCK 3	SPLCK 2	SPLCK 1	SPLCK 0
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **SPLCKy**: Security/privilege configuration lock for super-block (y = 31 to 0)

This bit is set by software and can be cleared only by system reset.

0: GTZC1_MPCBBz_SECCFGRy and GTZC1_MPCBBz_PRIVCFGRy can be written.

1: Writes to GTZC1_MPCBBz_SECCFGRy and GTZC1_MPCBBz_PRIVCFGRy are ignored

5.8.3 GTZC1 SRAMz MPCBB security configuration for super-block x register (GTZC1_MPCBBz_SECCFGRx) (z = 1 to 3)

Address offset: $0x100 + 0x4 * x$, (x = 0 to 31)

Reset value: 0xFFFF FFFF

The given reset value is valid when TZEN = 0xB4. The reset value is 0x0000 0000 when TZEN = 0xC3.

Write access to this register is secure only. Any read is allowed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SEC31	SEC30	SEC29	SEC28	SEC27	SEC26	SEC25	SEC24	SEC23	SEC22	SEC21	SEC20	SEC19	SEC18	SEC17	SEC16
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **SECy**: Security configuration for block y (y = 31 to 0)

0: Nonsecure access only to block y, belonging to super-block x. Secure access is also allowed if the SRWILADIS bit is set in GTZC1_MPCBBz_CR.

1: Secure access only to block y, belonging to super-block x.

Unprivileged write to this bit is ignored if PRIVy bit is set in GTZC1_MPCBBz_PRIVCFGRx.

Writes are ignored if SPLCKx bit is set in GTZC1_MPCBBz_CFGLOCK.

5.8.4 GTZC1 SRAMz MPCBB privileged configuration for super-block x register (GTZC1_MPCBBz_PRIVCFGRx) (z = 1 to 3)

Address offset: $0x200 + 0x4 * x$, (x = 0 to 31)

Reset value: 0xFFFF FFFF

The given reset value is valid when TZEN = 0xB4. The reset value is 0x0000 0000 when TZEN = 0xC3. Write access to this register is privileged only. Any read is allowed.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRIV31	PRIV30	PRIV29	PRIV28	PRIV27	PRIV26	PRIV25	PRIV24	PRIV23	PRIV22	PRIV21	PRIV20	PRIV19	PRIV18	PRIV17	PRIV16
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRIV15	PRIV14	PRIV13	PRIV12	PRIV11	PRIV10	PRIV9	PRIV8	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **PRIVy**: Privileged configuration for block y, belonging to super-block x (y = 31 to 0).

0: Privileged and unprivileged access to block y, belonging to super-block x

1: Only privileged access to block y, belonging to super-block x

Nonsecure write to this bit is ignored if SECy bit is set in GTZC1_MPCBBz_SECCFGRx.

Writes are ignored if SPLCKx bit is set in GTZC1_MPCBBz_CFGLOCK.

5.8.5 GTZC1 MPCBBz register map (z = 1 to 3)

Table 41. GTZC1 MPCBBz register map and reset values (z = 1 to 3)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0				
0x000	GTZC1_MPCBBz_CR	SRWLADIS	INVSECSTATE	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0				
	Reset value	0	0																														0				
0x004-0x00C	Reserved	Reserved																																			
0x010	GTZC1_MPCBBz_CFGLOCK1	SPLCK31	SPLCK30	SPLCK29	SPLCK28	SPLCK27	SPLCK26	SPLCK25	SPLCK24	SPLCK23	SPLCK22	SPLCK21	SPLCK20	SPLCK19	SPLCK18	SPLCK17	SPLCK16	SPLCK15	SPLCK14	SPLCK13	SPLCK12	SPLCK11	SPLCK10	SPLCK9	SPLCK8	SPLCK7	SPLCK6	SPLCK5	SPLCK4	SPLCK3	SPLCK2	SPLCK1	SPLCK0				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x014-0x0FC	Reserved	Reserved																																			
0x100 + 0x04 *x (x = 0 to 31)	GTZC1_MPCBBz_SECCFGRx	SEC31	SEC30	SEC29	SEC28	SEC27	SEC26	SEC25	SEC24	SEC23	SEC22	SEC21	SEC20	SEC19	SEC18	SEC17	SEC16	SEC15	SEC14	SEC13	SEC12	SEC11	SEC10	SEC9	SEC8	SEC7	SEC6	SEC5	SEC4	SEC3	SEC2	SEC1	SEC0				
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1					
0x180-0x1FC	Reserved	Reserved																																			
0x200 + 0x04 *x (x = 0 to 31)	GTZC1_MPCBBz_PRIVCFGRx	PRIV31	PRIV30	PRIV29	PRIV28	PRIV27	PRIV26	PRIV25	PRIV24	PRIV23	PRIV22	PRIV21	PRIV20	PRIV19	PRIV18	PRIV17	PRIV16	PRIV15	PRIV14	PRIV13	PRIV12	PRIV11	PRIV10	PRIV9	PRIV8	PRIV7	PRIV6	PRIV5	PRIV4	PRIV3	PRIV2	PRIV1	PRIV0				
	Reset value	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1					

Refer to [Table 30: GTZC1 subblock address offset](#).

6 RAMs configuration controller (RAMCFG)

6.1 RAMCFG introduction

The RAMCFG configures the features of the internal SRAMs (SRAM1, SRAM2, SRAM3, and BKPSRAM).

6.2 RAMCFG main features

The internal SRAMs support the features listed hereafter, configured in RAMCFG:

- Error code correction (ECC):
 - Single error detection and correction with interrupt generation
 - Double error detection with interrupt or NMI generation
 - Status with failing address
- Write protection (1-Kbyte granularity)
- SRAM software erase

6.3 RAMCFG functional description

6.3.1 Internal SRAMs features

Four SRAMs are embedded in the devices, each with specific features:

- SRAM1, SRAM2, SRAM3 are the main SRAMs (see [Table 42](#)), made of several blocks that can be powered down in Stop mode to reduce consumption.
- The backup SRAM (BKPSRAM) can be retained in all low-power modes and when V_{DD} is off in VBAT mode. Refer to [Section 10: Power control \(PWR\)](#) for more details.
- SRAM2 is erased when a system reset occurs if the SRAM2_RST option bit is selected in the flash memory user option bytes. SRAM1 and SRAM3 are erased when a system reset occurs if the SRAM13_RST option bit is selected in the flash memory user option bytes. Refer to [Section 7: Embedded flash memory \(FLASH\)](#) for more details.
- SRAM2 and (optionally) backup SRAM are protected by the tamper detection circuit, and are erased by hardware in case of tamper detection. Refer to [Section 52: Tamper and backup registers \(TAMP\)](#) for more details.
- The RAMCFG embeds the registers related to the internal SRAMs ECC, write protection and software erase.

Table 42. SRAMs density (Kbytes)

Products	SRAM1	SRAM2	SRAM3
STM32H562/63/73xx	256	64	320
STM32H523/33xx	128	80	64
STM32H543/53xx	128	80	96

[Table 43](#) summarizes the features supported by each internal SRAM.

Table 43. Internal SRAMs features

SRAM feature	SRAM1	SRAM2	SRAM3	BKPSRAM
Optional retention in Standby mode	-	-	-	X
Optional retention in VBAT mode	-	-	-	X
Optional retention in Stop mode	X	X	X	X
Erased with product state regression	X	X	X	X
Erased with tamper detection	-	X	-	X ⁽¹⁾
Erased with Backup domain reset	-	-	-	X
Optionally erased with system reset	X	X	X	-
Software erase	X	X	X	X
ECC	-	X	X ⁽²⁾	X
Write protection	-	X	-	-

1. Optional: BKPSRAM can be configured to be erased or not on tamper detection.

2. Not available on STM32H523/33xx devices.

6.3.2 Error code correction (SRAM2, SRAM3, BKPSRAM)

The ECC is supported by SRAM2, SRAM3^(a) and BKPSRAM when enabled with the SRAM2_ECC, SRAM3_ECC^(b) and BKPSRAM_ECC user option bits. Refer to [Section 7: Embedded flash memory \(FLASH\)](#) for more details.

Seven ECC bits are added per 32 bits of SRAM, allowing two bits error detection and one bit error correction on memory read access.

As the ECC is calculated and checked for a 32-bit word, the byte and half-word write accesses are managed by the SRAM interface by first reading the whole word, then write the word again with the new byte/half-word value. ECC double errors are also detected during these byte or half-word AHB write accesses (read/modify/write done by interface). The byte or half-word write access latency is two AHB clock cycles.

Caution: In case of a byte or half-word write on SRAM with ECC, the read/modify/write operation is done in a buffer. The buffer content is written into the SRAM two AHB clock cycles after the SRAM AHB is released (when SRAM is no more accessed).

Single and double ECC errors

When a single error is detected, it is automatically corrected. SEDC and CSEDC bits are set, respectively, in the [RAMCFG memory interrupt status register \(RAMCFG_MxISR\)](#) and [RAMCFG memory x interrupt clear register x \(RAMCFG_MxICR\)](#). The associated ECC single error address is updated only if the single error interrupt is enabled (SEIE bit of [RAMCFG memory x interrupt enable register \(RAMCFG_MxIER\)](#) is set). An interrupt is

a. Not available on STM32H523/33xx devices.

b. Refer to the device datasheet for the actual availability. If not present, consider this bit as reserved, and keep it at reset value.

generated if enabled by the SEIE bit. The failing address is stored in the *RAMCFG memory x ECC single error address register (RAMCFG_MxSEAR)* if the ALE bit is set in the *RAMCFG memory x control register (RAMCFG_MxCR)*.

Caution: Single errors cannot be detected when the SEDC bit is set.

When a double error is detected, the DED and CDED bits are set in the *RAMCFG memory interrupt status register (RAMCFG_MxISR)* and *RAMCFG memory x interrupt clear register x (RAMCFG_MxICR)*, respectively. The associated ECC double error address is updated only if double error interrupt or NMI is enabled (DEIE bit or ECCNMI bit of *RAMCFG memory x interrupt enable register (RAMCFG_MxIER)* is set). An interrupt or NMI is generated if enabled by the DEIE or ECCNMI bit in the *RAMCFG memory x interrupt enable register (RAMCFG_MxIER)*. The failing address is stored in the *RAMCFG memory x ECC double error address register (RAMCFG_MxDEAR)* if the ALE bit is set in the *RAMCFG memory x control register (RAMCFG_MxCR)*.

Caution: Double errors cannot be detected when the DED bit is set.

SRAM3 ECC specific management

On STM32H562/563/573xx devices, when the ECC is enabled for SRAM3, only the first 256 Kbytes of SRAM3 are with ECC. The next 64 Kbytes are without ECC, and the last block is used to store the ECC, hence it cannot be used by the application.

On STM32H543/553xx devices, when the ECC is enabled for SRAM3, only the first 64 Kbytes of SRAM3 are with ECC. The next 16 Kbytes are without ECC, and the last block is used to store the ECC, hence it cannot be used by the application.

Figure 21 shows the SRAM areas, when SRAM2 and SRAM3 ECC are enabled. Figure 22 shows the SRAM areas, when SRAM2 ECC and SRAM3 are enabled. Figure 23 shows the SRAM areas, when SRAM2 and SRAM3 ECC are enabled.

Figure 21. Memory map: SRAM1, SRAM2/3 with ECC (STM32H562/63/73xx devices)

Address offset			
SRAM3	0x9 FFFF	64 Kbytes	Reserved (ECC storage)
	0x9 0000		
	0x8 FFFF	64 Kbytes	SRAM with ECC
		64 Kbytes	
		64 Kbytes	
SRAM2	0x5 0000	48 Kbytes	SRAM without ECC
	0x4 FFFF	16 Kbytes	
	0x4 0000	64 Kbytes	
SRAM1	0x3 FFFF	64 Kbytes	SRAM without ECC
		64 Kbytes	
		64 Kbytes	
	0x0	64 Kbytes	

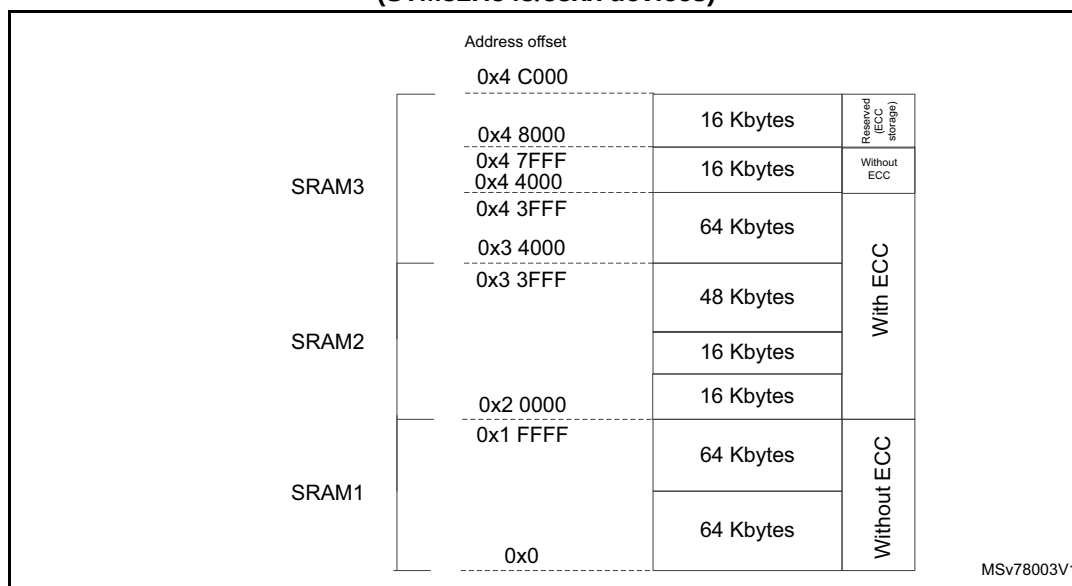
MSv68813V1

Figure 22. Memory map: SRAM1/3, SRAM2 with ECC (STM32H523/33xx devices)

Address offset			
SRAM3	0x4 3FFF	64 Kbytes	Without ECC
	0x3 4000		
SRAM2	0x3 3FFF	48 Kbytes	With ECC
		16 Kbytes	
	0x2 0000	16 Kbytes	
SRAM1	0x1 FFFF	64 Kbytes	Without ECC
	0x0	64 Kbytes	

MSv56369V1

**Figure 23. Memory map: SRAM1, SRAM2/3 with ECC
(STM32H543/53xx devices)**



When ECC is enabled by user option bits, the ECCE bit is automatically set after system reset in the related [RAMCFG memory x control register \(RAMCFG_MxCR\)](#).

The ECC can be deactivated by executing the following software sequence:

1. Write 0xAE in the [RAMCFG memory x ECC key register \(RAMCFG_MxECCKEYR\)](#).
2. Write 0x75 in the [RAMCFG memory x ECC key register \(RAMCFG_MxECCKEYR\)](#).
3. Write 0 in the ECCE bit of the [RAMCFG memory x control register \(RAMCFG_MxCR\)](#).

When ECC is deactivated (ECCE = 0), the SRAM3 ECC storage area can be read and written for ECC user test purpose. When the ECC is activated (ECCE = 1), this area is reserved for ECC storage purpose and cannot be neither read, nor written.

6.3.3 Write protection (SRAM2)

The SRAM2 is made of 1-Kbyte pages, each of them can be write-protected by setting its corresponding PxWP bit in the [RAMCFG memory 2 write protection register 1 \(RAMCFG_M2WPR1\)](#), [RAMCFG memory 2 write protection register 2 \(RAMCFG_M2WPR2\)](#), and [RAMCFG memory 2 write protection register 3 \(RAMCFG_M2WPR3\)](#)^(a).

Table 44. SRAM2 features

Products	Page size	Number of pages	PxWP bits
STM32H562/63/73xx	1 Kbyte	64	0 to 63

a. This register is available only on STM32H523/533/543/553xx devices. If not present, consider this register as reserved, and keep its bits at reset value.

Table 44. SRAM2 features (continued)

Products	Page size	Number of pages	PxWP bits
STM32H523/33xx	1 Kbyte	80	0 to 79
STM32H543/53xx			

6.3.4 Software erase

SRAM erase can be requested by executing this software sequence:

1. Write 0xCA in the *RAMCFG memory x erase key register (RAMCFG_MxERKEYR)*.
2. Write 0x53 in the *RAMCFG memory x erase key register (RAMCFG_MxERKEYR)*.
3. Write 1 in the SRAMER bit of the *RAMCFG memory x control register (RAMCFG_MxCR)*.

SRAMBUSY flag is set in the related SRAM interrupt status register as long as the erase is ongoing.

The total duration of each SRAM erase is N AHB clock cycles, where N is the size of the SRAM in 32-bit words.

If the SRAM is read or written while an erase is ongoing, wait states are inserted on the AHB bus until the end of the operation.

6.4 RAMCFG low-power modes

Table 45. Effect of low-power modes on RAMCFG

Mode	Description
Sleep	No effect. RAMCFG interrupts cause the device to exit the Sleep mode
Stop	The content of RAMCFG registers is kept
Standby	The RAMCFG peripheral is powered down and must be reinitialized after exiting Standby

6.5 RAMCFG interrupts

Table 46. RAMCFG interrupt requests

Interrupt acronym	Interrupt event	Event flag	Enable control bit(s)	Interrupt clear method	Exit Sleep mode	Exit Stop mode	Exit Standby modes
RAMCFG	ECC single error detection and correction	SEDC	SEIE	Write 1 in CSEDC	Yes	No	No
	ECC double error detection	DED	DEIE = 1 and ECCNMI = 0	Write 1 in CDED	Yes	No	No
NMI	ECC double error detection	DED	ECCNMI	Write 1 in CDED	Yes	No	No

6.6 RAMCFG registers

In the registers described below, x refers to:

- SRAM1/2/3 when x = 1/2/3, respectively
- BKPSRAM when x = 5

6.6.1 RAMCFG memory x control register (RAMCFG_MxCR)

Address offset: $0x000 + 0x040 * (x - 1)$, (x = 1, 2, 3, 5)

Reset value: 0x0000 000X

ECCE reset value depends on ECC enable user option bit.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	SRAMER	Res.	Res.	Res.	ALE	Res.	Res.	Res.	ECCE
							rs				rw				rw

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **SRAMER**: SRAM erase

This bit can be set by software only after writing the unlock sequence in the ERASEKEY field of the RAMCFG_MxERKEYR register. Setting this bit starts the SRAM erase. This bit is automatically cleared by hardware at the end of the erase operation.

0: No erase operation on going
1: Erase operation on going

Bits 7:5 Reserved, must be kept at reset value.

Bit 4 **ALE**: Address latch enable

0: Failing address not stored in the SRAMx ECC single/double error address registers
1: Failing address stored in the SRAMx ECC single/double error address registers

Note: For SRAM1 and SRAM3 (when ECC is not available) this bit is reserved and must be kept at reset value in control registers.

Bits 3:1 Reserved, must be kept at reset value.

Bit 0 **ECCE**: ECC enable.

This bit reset value is defined by the user option bit configuration. When set, it can be cleared by software only after writing the unlock sequence in the RAMCFG_MxECCKEYR register.

0: ECC disabled
1: ECC enabled

Note: For SRAM1 and SRAM3 (when ECC is not available) this bit is reserved and must be kept at reset value in control registers.

6.6.2 RAMCFG memory x interrupt enable register (RAMCFG_MxIER)

Address offset: $0x004 + 0x040 * (x - 1)$, (x = 2, 3, 5)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCNMI	Res.	DEIE	SEIE
												rs		rw	rw

Bits 31:4 Reserved, must be kept at reset value.

Bit 3 **ECCNMI**: Double error NMI

This bit is set by software and cleared only by a global RAMCFG reset.

0: NMI not generated in case of ECC double error

1: NMI generated in case of ECC double error

Note: if ECCNMI is set, the RAMCFG maskable interrupt is not generated whatever DEIE bit value.

Bit 2 Reserved, must be kept at reset value.

Bit 1 **DEIE**: ECC double error interrupt enable

0: Double error interrupt disabled

1: Double error interrupt enabled

Bit 0 **SEIE**: ECC single error interrupt enable

0: Single error interrupt disabled

1: Single error interrupt enabled

6.6.3 RAMCFG memory interrupt status register (RAMCFG_MxISR)

Address offset: 0x008 + 0x040 * (x - 1), (x = 1, 2, 3, 5)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	SRAM BUSY	Res.	Res.	Res.	Res.	Res.	Res.	DED	SEDC
							r							r	r

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **SRAMBUSY**: SRAM busy with erase operation

0: No erase operation on going

1: Erase operation on going

Note: Depending on the SRAM, the erase operation can be performed due to software request, system reset if the option bit is enabled, tamper detection or readout protection regression. Refer to [Table 43](#).

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **DED**: ECC double error detected

0: No double error

1: Double error detected

Note: For SRAM1 and SRAM3 (when ECC is not available) this bit is reserved and must be kept at reset value in control registers.

Bit 0 **SEDC**: ECC single error detected and corrected

0: No single error

1: Single error detected and corrected

Note: For SRAM1 and SRAM3 (when ECC is not available) this bit is reserved and must be kept at reset value in control registers.

6.6.4 RAMCFG memory x ECC single error address register (RAMCFG_MxSEAR)

Address offset: $0x00C + 0x040 * (x - 1)$, ($x = 2, 3, 5$)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ESEA[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ESEA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **ESEA[31:0]**: ECC single error address

When the ALE bit is set in the RAMCFG_MxCR register, this field is updated with the address corresponding to the ECC single error.

6.6.5 RAMCFG memory x ECC double error address register (RAMCFG_MxDEAR)

Address offset: $0x010 + 0x040 * (x - 1)$, ($x = 2, 3, 5$)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
EDEA[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDEA[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **EDEA[31:0]**: ECC double error address

When the ALE bit is set in the RAMCFG_MxCR register, this field is updated with the address corresponding to the ECC double error.

6.6.6 RAMCFG memory x interrupt clear register x (RAMCFG_MxICR)

Address offset: $0x014 + 0x040 * (x - 1)$, ($x = 2, 3, 5$)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CDED	CSEDC
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **CDED**: Clear ECC double error detected

Writing 1 to this flag clears the DED bit in the RAMCFG_MxISR register. Reading this flag returns the DED value.

Bit 0 **CSEDC**: Clear ECC single error detected and corrected

Writing 1 to this flag clears the SEDC bit in the RAMCFG_MxISR register. Reading this flag returns the SEDC value.

6.6.7 RAMCFG memory 2 write protection register 1 (RAMCFG_M2WPR1)

Address offset: 0x058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
P31WP	P30WP	P29WP	P28WP	P27WP	P26WP	P25WP	P24WP	P23WP	P22WP	P21WP	P20WP	P19WP	P18WP	P17WP	P16WP
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P15WP	P14WP	P13WP	P12WP	P11WP	P10WP	P9WP	P8WP	P7WP	P6WP	P5WP	P4WP	P3WP	P2WP	P1WP	P0WP
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **PyWP**: SRAM2 1-Kbyte page y write protection (y = 31 to 0)

These bits are set by software and cleared only by a global RAMCFG reset.

0: Write protection of SRAM2 1-Kbyte page y is disabled.

1: Write protection of SRAM2 1-Kbyte page y is enabled.

6.6.8 RAMCFG memory 2 write protection register 2 (RAMCFG_M2WPR2)

Address offset: 0x05C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
P63WP	P62WP	P61WP	P60WP	P59WP	P58WP	P57WP	P56WP	P55WP	P54WP	P53WP	P52WP	P51WP	P50WP	P49WP	P48WP
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P47WP	P46WP	P45WP	P44WP	P43WP	P42WP	P41WP	P40WP	P39WP	P38WP	P37WP	P36WP	P35WP	P34WP	P33WP	P32WP
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **PyWP**: SRAM2 1-Kbyte page y write protection (y = 63 to 32)

These bits are set by software and cleared only by a global RAMCFG reset.

0: Write protection of SRAM2 1-Kbyte page y is disabled.

1: Write protection of SRAM2 1-Kbyte page y is enabled.

6.6.9 RAMCFG memory 2 write protection register 3 (RAMCFG_M2WPR3)

Address offset: 0x060

Reset value: 0x0000 0000

This register is available only for STM32H523/33/43/53xx devices.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
P79WP	P78WP	P77WP	P76WP	P75WP	P74WP	P73WP	P72WP	P71WP	P70WP	P69WP	P68WP	P67WP	P66WP	P65WP	P64WP
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **PyWP**: SRAM2 1-Kbyte page y write protection (y = 79 to 64)

These bits are set by software and cleared only by a global RAMCFG reset.

0: Write protection of SRAM2 1-Kbyte page y is disabled.

1: Write protection of SRAM2 1-Kbyte page y is enabled.

6.6.10 RAMCFG memory x ECC key register (RAMCFG_MxECCKEYR)

Address offset: 0x024 + 0x040 * (x - 1), (x = 2, 3, 5)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCKEY[7:0]							
								w	w	w	w	w	w	w	w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **ECCKEY[7:0]**: ECC write protection key

The following steps are required to unlock the write protection of the ECCE bit in the RAMCFG_MxCR register.

1) Write 0xAE into ECCKEY[7:0].

2) Write 0x75 into ECCKEY[7:0].

Note: Writing a wrong key reactivates the write protection.

6.6.11 RAMCFG memory x erase key register (RAMCFG_MxERKEYR)

Address offset: $0x028 + 0x040 * (x - 1)$, ($x = 1$ to 5)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERASEKEY[7:0]							
								w	w	w	w	w	w	w	w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **ERASEKEY[7:0]**: Erase write protection key

The following steps are required to unlock the write protection of the SRAMER bit in the RAMCFG_MxCR register.

- 1) Write 0xCA into ERASEKEY[7:0].
- 2) Write 0x53 into ERASEKEY[7:0].

Note: Writing a wrong key reactivates the write protection.

6.6.12 RAMCFG register map

Table 47. RAMCFG register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0					
0x00	RAMCFG_M1CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	Res.	Res.	Res.	ALE	Res.	Res.	Res.	ECCE					
	Reset value																								0				0				x					
0x04	Reserved	Reserved																																				
0x08	RAMCFG_M1ISR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.					
	Reset value																								0													
0x0C to 0x24	Reserved	Reserved																																				
0x28	RAMCFG_M1ERKEYR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERASEKEY[7:0]												
	Reset value																									0	0	0	0	0	0	0	0					
0x2C to 0x3C	Reserved	Reserved																																				
0x40	RAMCFG_M2CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	Res.	Res.	Res.	Res.	ALE	Res.	Res.	ECCE					
	Reset value																								0				0				x					
0x44	RAMCFG_M2IER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECCNMI	Res.	DEIE	SEIE					
	Reset value																												0		0	0	0					
0x048	RAMCFG_M2ISR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	0	Res.	Res.	Res.	Res.	Res.	Res.	DED	SEDC					
	Reset value																								0						0	0	0					
0x04C	RAMCFG_M2SEAR	ESEA[31:0]																																				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x050	RAMCFG_M2DEAR	EDEA[31:0]																																				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x054	RAMCFG_M2ICR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CDED	CSEDC					
	Reset value																														0	0	0					
0x058	RAMCFG_M2WPR1	P31WP	P30WP	P29WP	P28WP	P27WP	P26WP	P25WP	P24WP	P23WP	P22WP	P21WP	P20WP	P19WP	P18WP	P17WP	P16WP	P15WP	P14WP	P13WP	P12WP	P11WP	P10WP	P9WP	P8WP	P7WP	P6WP	P5WP	P4WP	P3WP	P2WP	P1WP	P0WP					
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x05C	RAMCFG_M2WPR2	P63WP	P62WP	P61WP	P60WP	P59WP	P58WP	P57WP	P56WP	P55WP	P54WP	P53WP	P52WP	P51WP	P50WP	P49WP	P48WP	P47WP	P46WP	P45WP	P44WP	P43WP	P42WP	P41WP	P40WP	P39WP	P38WP	P37WP	P36WP	P35WP	P34WP	P33WP	P32WP					
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x060	RAMCFG_M2WPR3 ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	P79WP	P78WP	P77WP	P76WP	P75WP	P74WP	P73WP	P72WP	P71WP	P70WP	P69WP	P68WP	P67WP	P66WP	P65WP	P64WP					
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					
0x064	RAMCFG_M2ECKEYR	ECKEY[7:0]																																				
	Reset value																																					

Table 47. RAMCFG register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0			
0x068	RAMCFG_M2ERKEYR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ERASEKEY[7:0]										
	Reset value																									0	0	0	0	0	0	0	0			
0x06C to 0x07C	Reserved	Reserved																																		
0x080	RAMCFG_M3CR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	SRAMER	Res	Res	Res	Res	Res	Res	ECCE			
	Reset value																									0				0			0			
0x084	RAMCFG_M3IER	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	DEIE	SEIE			
	Reset value																												0	ECCNMI		0	0			
0x088	RAMCFG_M3ISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	SRAMBUSY	Res	Res	Res	Res	Res	DED	SEDC			
	Reset value																									0						0	0			
0x08C	RAMCFG_M3SEAR	ESEA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
0x090	RAMCFG_M3DEAR	EDEA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x094	RAMCFG_M3ICR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	CDED	CSEDC			
	Reset value																															0	0			
0x098 to 0x0A0	Reserved	Reserved																																		
0x0A4	RAMCFG_M3ECCKEYR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ECCKEY[7:0]										
	Reset value																										0	0	0	0	0	0	0			
0x0A8	RAMCFG_M3ERKEYR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ERASEKEY[7:0]										
	Reset value																										0	0	0	0	0	0	0			
0x0AC to 0x0FC	Reserved	Reserved																																		
0x100	RAMCFG_M5CR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	SRAMER	Res	Res	Res	Res	Res	Res	ECCE			
	Reset value																									0				0			x			
0x104	RAMCFG_M5IER	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	ECCNMI	Res	DEIE	SEIE		
	Reset value																													0		0	0			
0x108	RAMCFG_M5ISR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	0	SRAMBUSY	Res	Res	Res	Res	Res	DED	SEDC			
	Reset value																									0						0	0			
0x10C	RAMCFG_M5SEAR	ESEA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x110	RAMCFG_M5DEAR	EDEA[31:0]																																		
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			

Table 47. RAMCFG register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0						
0x114	RAMCFG_M5ICR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CDED	CSEDC						
	Reset value																														0	0							
0x118 to 0x120	Reserved	Reserved																																					
0x124	RAMCFG_M5ECKKEYR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ECKKEY[7:0]													
	Reset value																									0	0	0	0	0	0	0	0						
0x128	RAMCFG_M5ERKEYR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERASEKEY[7:0]													
	Reset value																									0	0	0	0	0	0	0	0						

1. Available only on STM32H523/33/43/53xx devices.

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

7 Embedded flash memory (FLASH)

7.1 FLASH introduction

The embedded flash memory (FLASH) manages the accesses of any master to the up to 2 Mbytes of embedded nonvolatile memory. It implements read, program, and erase operations, error corrections, as well as various integrity and confidentiality protection mechanisms.

FLASH manages the automatic loading of nonvolatile user option bytes at power-on reset, and implements the dynamic update of these options. It also features a high-cycle data area, a one-time-programmable (OTP) area, a secure key storage area (OBKeys), and a read-only area configured by STMicroelectronics during manufacturing.

7.2 FLASH main features

- Up to 2 Mbytes of nonvolatile memory, divided into two 1-Mbyte banks
- Flash memory read operations supporting multiple lengths: 128 bits, 64 bits, 32 bits, 16 bits, or one byte
- Flash memory programming by 128 (user area, OBKeys) and by 16 bits (OTP and flash high-cycle data area)
- 8-Kbyte sector erase, bank erase and dual-bank mass erase
- Dual-bank organization supporting:
 - Simultaneous operations: read-while-write (program and erase) is supported, including flash high-cycle data area. The two banks share the same interface, hence write and erase cannot be performed in parallel (write-while-write is not supported).
 - Bank swapping: the address mapping of the user memory of each bank can be swapped, along with the corresponding registers. Security flags remain valid for the physical bank, so the data are not revealed by swapping to a bank with lower security configuration.
- Error code correction (ECC): one error detection/correction, or two errors detection per 128-bit flash word using nine ECC bits, on 16-bit words with six bits within configurable flash high-cycle data area.
- User configurable nonvolatile option bytes
- Flash memory enhanced protections, activated by option bytes
 - different product states for protecting memory content from debug access
 - sector group write-protection (WRPSG), protecting up to 32 groups of four sectors (32 Kbytes) per bank
 - two secure-only areas (one per user flash bank): when enabled, these areas are accessible only if the microcontroller operates in Secure access mode
 - HDP protection, providing temporal isolation for startup code
- 2-Kbyte one-time programmable (OTP) area
- Read-only area configured by STMicroelectronics
- Prefetch reads the next sequential instruction from flash memory

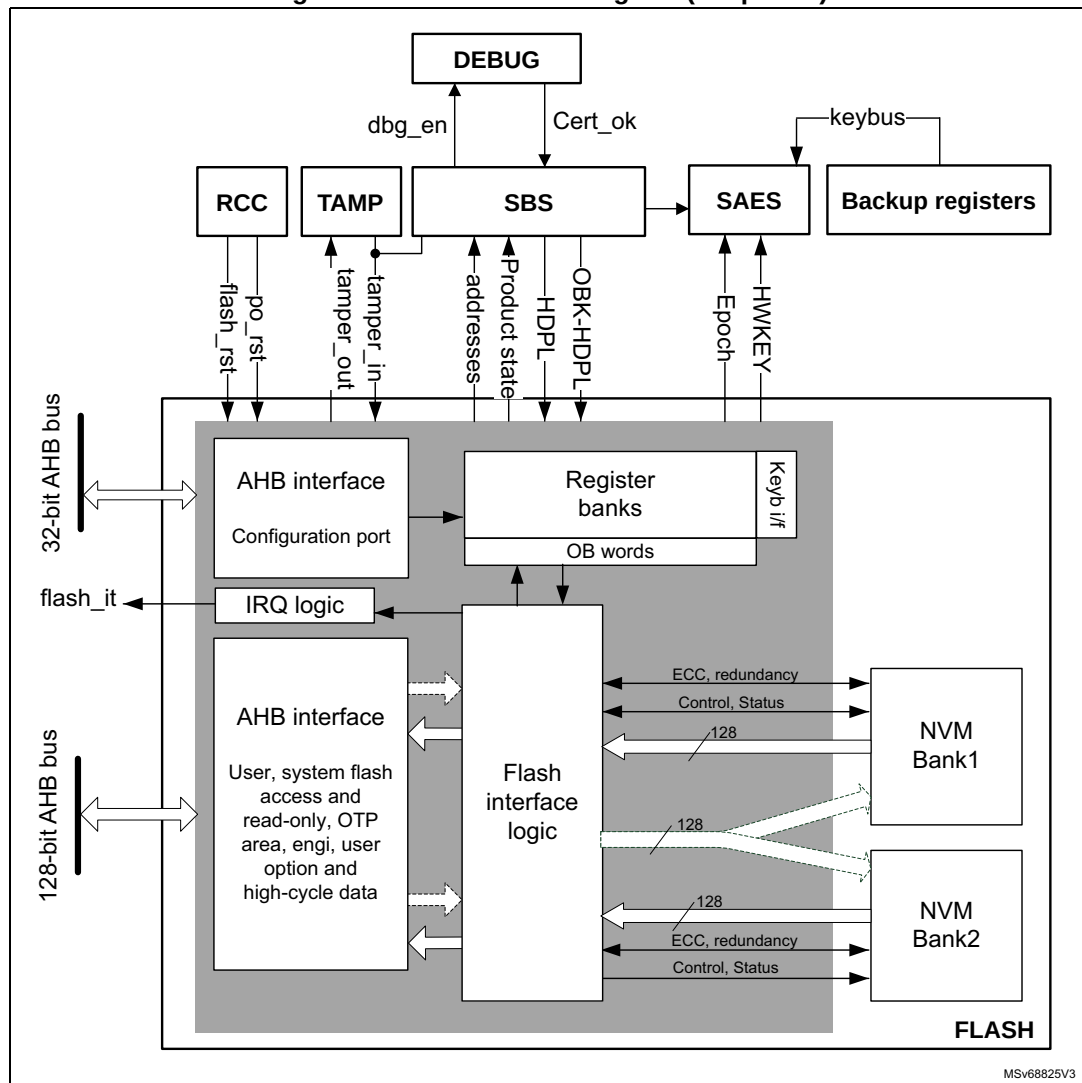
- Up to 48 Kbytes per bank supporting high-cycling capability (100 kcycles), to be used for data (EEPROM emulation)

7.3 FLASH functional description

7.3.1 FLASH block diagram

Figure 24 shows the embedded flash memory block diagram.

Figure 24. FLASH block diagram (simplified)



7.3.2 FLASH signals

The flash memory has two AHB connections, namely the flash AHB register interface and the main AHB interface.

Flash AHB register interface

- Data size is 32 bits
- Except for some registers (FLASH_NS/SECKEYR, FLASH_NS/SECOBKKEYR and FLASH_OPTKEYR, used to insert unlock sequences for control, and option registers that can be written by 32 bits), it is possible to read and write all registers by 8, 16 and 32 bits.
- When unlock sequence for control and option registers is wrong, a bus error is raised, otherwise no read or write errors are generated on the bus.

Main AHB interface

The AHB data bus size is 128 bits. This interface is used to handle three different targets:

- Code placed in user and system memory, protected by 9 bits of ECC.
- Secure keys placed in OBKey sectors, protected by 9 bits of ECC.
- OTP, read-only and flash memory high-cycle data area. protected by 6 bits of ECC.

The main AHB interface is implemented as follows:

- User and system memory, OBKeys storage:
 - Supports multiple length: 128-, 64-, 32-, 16- and 8-bit data width.
 - There is a read buffer of 128 bits for each bank where the last data read is stored. If data are available in the read buffer, no read access is given to the flash memory. Buffer is flushed when write access, OTP access, OBK swap, OBK alternate sector erase, high-cycle data area access, user option change request or erase operation occur.
 - There is a prefetch of the same size as the read buffer.
 - A 9-bit ECC is associated to each 128-bit data flash memory word.
- OTP, read-only and flash high-cycle data
 - Two dedicated data buffer of 137 bits are used to manage 16-bit data with 6-bit ECC
 - Reading two times the same address triggers two flash read accesses.
 - During a read access, two wait states are added in addition to the memory wait states. These wait states are necessary to parse the data buffer.
 - Each write access triggers a write in the flash memory.
 - 6-bit ECC is associated to each 16-bit data.

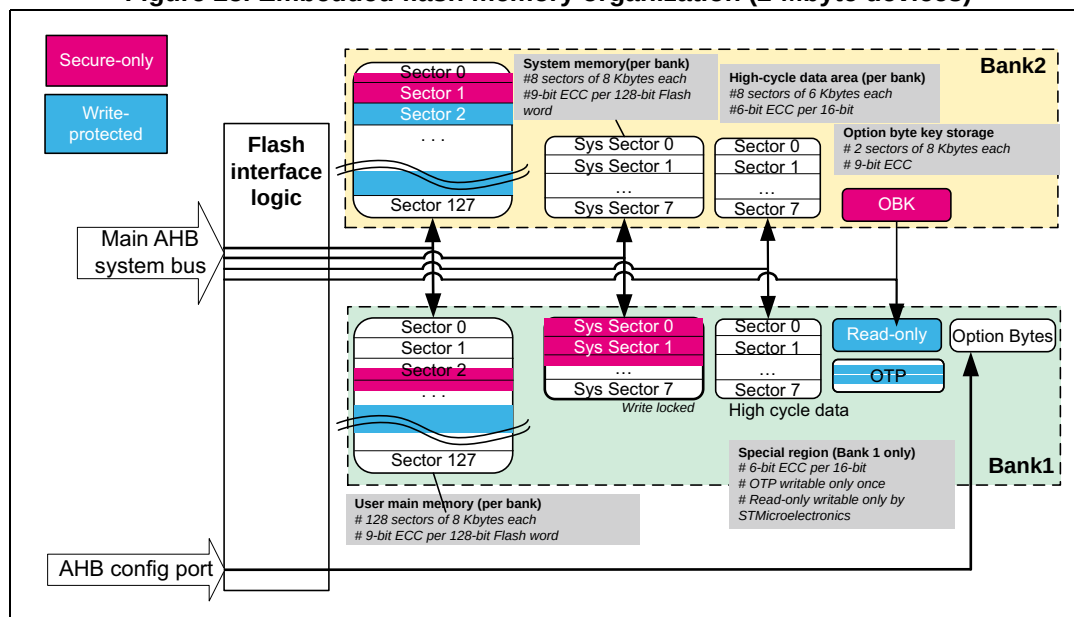
By default, all the AHB memory range is cacheable. For regions where caching is not practical (OTP, RO, data area), MPU must be used to disable local cacheability.

7.3.3 Flash memory architecture and usage

Flash memory architecture

Figure 25 shows the organization supported by the embedded flash memory. By default, all the AHB memory range is cacheable. For regions where caching is impractical (OTP, read-only, data) use MPU to disable local cacheability.

Figure 25. Embedded flash memory organization (2-Mbyte devices)



The embedded flash nonvolatile memory is composed of:

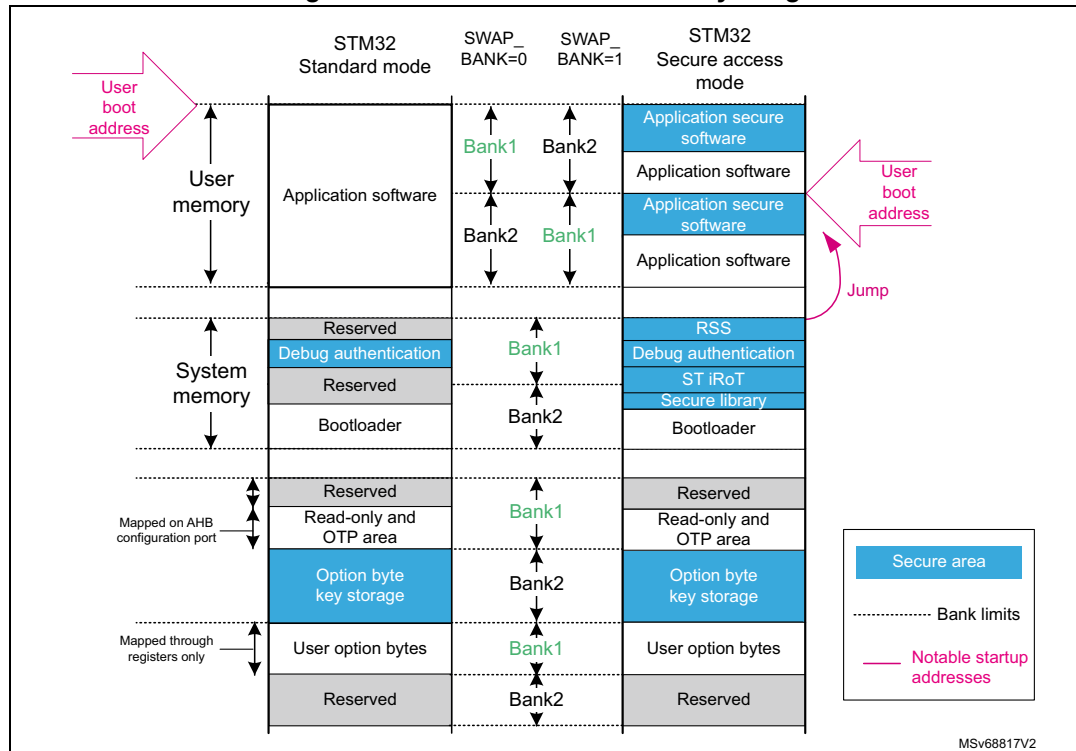
- A main memory block, organized in two banks. Each bank is divided in sectors of 8 Kbytes each, and features flash-word rows of 128 bits + 9 bits of ECC per word (see [Table 51](#)).
- A system memory block of 128 Kbytes, divided into two 64-Kbyte banks. Each bank is divided in eight 8-Kbyte sectors. The system flash memory is ECC protected (9-bit ECC per 128-bit word).
- A set of nonvolatile option bytes loaded at reset by the embedded flash memory and accessible by the application software only through the AHB configuration register interface.
- A 2-Kbyte one-time-programmable (OTP) area that can be written only once by the application software.
- A 2-Kbyte read-only area. It contains a unique device ID and product information.
- Two memory sectors (2 x 8 Kbytes) of secure key storage, OBKeys.
- Up to 16 sectors of user flash memory with high cycling capability (100 K cycles) for data, 8 sectors per bank.

The overall flash memory architecture and its corresponding access interface is summarized in [Table 51](#).

Partition usage

Figure 26 shows how the embedded flash memory is used by STMicroelectronics and by the application software.

Figure 26. Embedded flash memory usage



User and system memories are used differently, according to product state and other option bytes settings:

- The user memory contains the application code and data, while the system memory is used with root secure services (RSS), the debug authentication code, and the STM32 bootloader. When a reset occurs, the core jumps to the boot address configured through the BOOT pin, the (SEC/NS)BOOTADD option bytes, and the product state.
- The unique boot entry (BOOT_UBE) is set to either STiRoT located in the system flash memory, or iRoT in the user flash memory. This feature is available only on cryptography enabled devices.
- Secure service library in system flash memory is available in secure mode.
- If the debugger is attached to the product, the entry point is the debug authentication policy, used to unlock the device via the SBS when attached to debugger. A digital signature must be provided to perform a regression to product state, where debug is allowed.
- If (SEC/NS)BOOTADD is not yet configured, dedicated libraries can be used for secure boot. They are located in the system flash memory:
 - ST libraries in system flash memory assist the application software boot with special features, such as secure boot and secure firmware install (SFI-RSS)
 - STiRoT (immutable root of trust) secure software in user flash memory is used for secure firmware update and provisioning (SFU)

Note: For more information on option byte setup for boot, refer to [Section 7.4.6](#).

7.3.4 FLASH read operations

Read operation overview

Read access to main and system flash memory operates as follows:

- There is a 128-bit read data buffer associated to each bank, which stores the last data read. If several consecutive read accesses request data belonging to the same flash data word (128 bits), data are read directly from the current data read buffer, without triggering additional flash read operations. This mechanism occurs each time a read access is granted. When a read access is rejected for security reasons, the corresponding read error response is issued by the embedded flash memory and no read operation to flash memory is triggered.
- The read data buffer is disabled when write access or OTP access, user option change request, OBK swap, OBK erase or other erase operation occur.

Read access to OTP, RO and flash high-cycle data operates as follows:

1. Flash data word of 137 bits is read and stored in a temporary buffer.
2. The interface parses the 137-bit data word, and selects the 16- or 32-bit data requested.
3. While parsing the 137-bit data word, (two) wait states are added, and the AHB bus is stalled.
4. If the application reads an OTP data or flash high-cycle data not previously written, a **double ECC error** is reported and **only a word full of set bits is returned** (see [Section 7.3.9](#) for details). The read data (in 16 bits) is stored in FLASH_ECCDR register, so that the user can identify if the double ECC error is due to a virgin data or a real ECC error.
5. Reading two times the same address triggers two reads in the flash memory.
6. For 8-bit accesses, an AHB bus error is generated.

Note: The embedded flash memory can perform single-error correction and double-error detection while read operations are being executed (see [Section 7.3.8](#)).

Instruction prefetch

The Cortex-M33 fetches instructions and literal pools (constants/data) over the C-Bus and through the instruction cache, if it is enabled. The prefetch block aims at increasing the efficiency of C-Bus accesses when the instruction cache is enabled, by reducing the cache refill latency.

Prefetch is efficient in case of sequential code; prefetch in the flash memory allows the next sequential instruction line to be read from the memory, while the current instruction line is being filled in instruction cache and executed by the CPU.

Prefetch is enabled by setting the PRFTEN bit in the FLASH access control register (FLASH_ACR). PRFTEN must be set only if at least one wait state is needed to access the flash memory.

Adjusting read timing constraints

The embedded clock must be enabled and running before reading data from a nonvolatile memory.

To correctly read data from the memory, the number of wait states (LATENCY) must be correctly programmed in the access control register (FLASH_ACR), according to the main AHB interface clock frequency, and the internal voltage range of the device (V_{core}).

[Table 48](#) shows the correspondence between the number of wait states (LATENCY), the programming delay parameter (WRHIGHFREQ), the embedded flash memory clock frequency, and the supply voltage range.

Table 48. Recommended number of wait states and programming delay

Number of wait states (LATENCY)	Programming delay (WRHIGHFREQ)	Interface clock frequency vs. V_{CORE} range ⁽¹⁾			
		VOS3 range 0.95 to 1.05 V	VOS2 range 1.05 to 1.15 V	VOS1 range 1.15 to 1.26 V	VOS0 range 1.30 to 1.40 V
0 WS (1 FLASH clock cycle)	00	0 to 20 MHz	0 to 30 MHz	0 to 34 MHz	0 to 42 MHz
1 WS (2 FLASH clock cycles)		20 to 40 MHz	30 to 60 MHz	34 to 68 MHz	42 to 84 MHz
2 WS (3 FLASH clock cycles)	01	40 to 60 MHz	60 to 90 MHz	68 to 102 MHz	84 to 126 MHz
3 WS (4 FLASH clock cycles)		60 to 80 MHz	90 to 120 MHz	102 to 136 MHz	126 to 168 MHz
4 WS (5 FLASH clock cycles)	10	80 to 100 MHz	120 to 150 MHz	136 to 170 MHz	168 to 210 MHz
5 WS (6 FLASH clock cycles)		N/A	N/A	170 to 200 MHz	210 to 250 MHz

1. Voltage range from 1.26 to 1.30 V is not supported.

Adjusting system frequency

After power-on, the embedded flash memory is clocked by the 64 MHz high-speed internal oscillator (HSI), with a voltage range set at a scaled value of VOS3: a conservative 3 wait-state latency is specified in FLASH_ACR register (see [Table 48](#)).

When changing the bus frequency, the application software must follow the sequence described below, to tune the number of wait states required to access the memory.

To increase the CPU frequency:

1. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in the FLASH_ACR register, as described in [Table 48](#).
2. Check that the new number of wait states is taken into account by reading back the FLASH_ACR register.
3. Modify the embedded flash memory clock source and/or the clock prescaler in the RCC_CFGR register of the reset and clock controller (RCC).
4. Check that the new embedded flash memory clock source and/or the new AHB clock prescaler value are taken in account by reading back the embedded flash memory clock source status and/or the prescaler value in the RCC_CFGR register of the reset and clock controller (RCC).

To decrease the CPU frequency:

1. Modify the embedded flash memory clock source and/or the clock prescaler in the RCC_CFGR register of reset and clock controller (RCC).
2. Check that the embedded flash memory new clock source and/or the new clock prescaler value are taken into account by reading back the embedded flash memory clock source status and/or the AHB interface prescaler value in the RCC_CFGR register of reset and clock controller (RCC).
3. If necessary, program the LATENCY and WRHIGHFREQ bits to the right value in FLASH_ACR register, as described in [Table 48](#).
4. Check that the new number of wait states has been taken into account by reading back the FLASH_ACR register.

Error code correction (ECC)

The memory embeds an error correction mechanism. Single-error correction and double-error detection are performed for each read operation. For more details, refer to [Section 7.3.8](#).

Read errors

When the ECC mechanism is unable to correct the read operation, the memory reports read errors, as described in [Section 7.9.10](#).

Read interrupts

See [Section 7.10](#) for details.

7.3.5 FLASH program operations

Program operation overview

Program operation consists in issuing write commands. The memory supports the execution of only one write-command at a time. Write-while-write is not supported. Nothing prevents overwriting a non-virgin flash word, but this is not recommended. The result may lead to invalid data and inconsistent ECC code.

User flash, OBK storage and system flash memories sectors

For the user and system flash memories, 9-bit ECC is associated to each 128-bit data flash word. In this case, the embedded flash memory must always perform write operations to nonvolatile memory with a 128-bit word granularity. Once the write buffer is full (128 bits), the Busy flag is set, and a programming operation is triggered.

There is a write buffer common to Bank1 and 2, which supports multiple write-access types (128, 64, 32, 16 or 8 bits). The application can decide to write from 8 bits to 128 words. In this case, a force-write mechanism to the 128 bits + ECC is used (see FW bit of FLASH_NS/SECCR register).

When the write request is issued to the memory, any new write request stalls the main AHB bus. Moreover, while a write operation is ongoing, any new read request to the same bank stalls the main AHB bus.

OTP, RO, flash high-cycle data

When the target memory is OTP, RO and flash high-cycle data sectors, 6-bits ECC code is associated to each 16-bit data flash word. The embedded flash memory supports 16- or 32-bit write operations (8-bit write operations are not supported). For 8-bit accesses, write accesses are ignored. There is no write data buffer. Each write access triggers a write in the flash memory.

Note: The OTP area is typically write-protected on the final product, as described in [Section 7.3.9](#).

The write protection check is performed at the reception of the write request (during address phase). Write protection is not checked anymore at the output of the write buffer. If a write protection violation is detected, the write operation is canceled, and write protection error (WRPERR) is raised in FLASH_NS/SECSR register.

Note: For write protections of main flash, ICP, and OTP see [Section 7.6](#).

Monitoring ongoing write operations

The application software can use a status flag located in FLASH_NS/SECSR to monitor ongoing write operations. Since only one operation is possible at a time, this flag indicates if any operation (write, erase, option change) is ongoing, whatever the bank.

- **BSY**: indicates that an effective write, erase, option byte change, OBK swap, OBK alt sector erase is ongoing in the nonvolatile memory. This flag is not dedicated to a specific bank. It is set when an operation is starting on the memory, whatever the bank. An operation is triggered by:
 - An erase (FLASH_NS/SECCR.STRT)
 - A write (FLASH_NS/SECCR.PG + AHB write)
 - An option modification (FLASH_OPTCR.OPTSTRT)
 - OBKeys sector swap and OBKeys sector erase (FLASH_NS/SECOBKCFGR.SWAP_SECT_REQ and FLASH_NS/SECOBKCFGR.ALT_SECT_ERASE)

They are cleared when the current operation ends, or in case of error.

- **WBNE**: this bit indicates that the embedded flash memory is waiting for new data to complete the 128-bit write buffer. In this state the write buffer is not empty. It is reset as soon as the application software fills the write buffer, or forces the writes by using FW bit in FLASH_NS/SECCR, or an error is detected. When WBNE is high, it is not possible to launch an erase, an option modification, an OBK swap or OBK alternate sector erase operation on flash memory.
- **DBNE**: this bit indicates that the data buffer for parsing 16-bits data is not empty:
 - 16-bit data write access is received, and the data buffer is being filled. It is set at the receipt of a valid write access, and reset as soon as the write request preparation has been processed.

Note: If the memory is busy at the receipt of the AHB write request, the CPU execution is stalled.

Enabling write operations

Before programming the user flash memory in Bank1 or in Bank2, the application software must ensure that the PG bit is set to 1 in FLASH_NS/SECCR. If not, an unlock sequence must be used (see [Section 7.6.7](#)), and the PG bit must be set.

When an option byte or an option byte key must be modified, or a mass erase must be started, the application software must ensure that FLASH_OPTCR is unlocked. If this is not the case, an unlock sequence must be used (see [Section 7.6.7](#)).

A separate mechanism with similar use exists for FLASH_NS/SECOBKCFGR. The control register must be unlocked prior to the start of any OBKeys storage modification. Writing the correct sequence to the FLASH_NS/OBKKEYR unlocks FLASH_NS/OBKCFGR. FLASH_SECOBKKEYR is linked to FLASH_SECOBKCFGR, as described in [Section 7.6.7](#).

Note: *The application software must not unlock an already unlocked register, otherwise this register remains locked until the next system reset.*

If needed, the application software can update the programming delay, as described in [Adjusting programming timing constraints](#).

Writing to the FLASH control register FLASH_NS/SECCR and FLASH_OPTCR

The FLASH_NS/SECCR, FLASH_OPTCR and FLASH_NS/SECOBKCFGR registers are not accessible in write mode when the BSY bit is set. Any attempt to write these registers while the BSY bits is set causes the AHB bus to stall until SEC/NSBSY bit is cleared.

Single-write sequence

The recommended single-write sequence is the following:

1. Make sure protection mechanism does not prevent programming
2. Check that no memory operations are ongoing by checking the BSY bit in the FLASH_NS/SECSR register and CDBNE bits in the FLASH_NS/SECSR register. Check that the write buffer is empty by checking the WBNE bit in the FLASH_NS/SECSR register
3. Check and clear all the error flags due to previous programming/erase operation
4. Unlock the FLASH_NS/SECCR register, as described in [Section 7.6.7](#) (only if this register is not already unlocked)
5. Enable write operations by setting PG bit in the FLASH_NS/SECCR register
6. Write one flash-word at aligned address

Note: *NS/SECWBNE flag indicates if the 128-bit write buffer is waiting for new data.*

Note: *No erase request, options change request, OBK operation is allowed between the first write and the completion of the write operation.*

7. Wait for the BSY bit to be cleared in the corresponding FLASH_NS/SECSR register
8. Clear PG bit in FLASH_NS/SECCR register if there are not any more programming requests

If step 6 is executed incrementally (for example byte per byte), the write buffer can become partially filled. In this case the application software can decide to force-write what is stored in the write buffer by using FW bit in FLASH_NS/SECCR register. In this particular case, the unwritten bits are automatically set to 1. If no bit in the write buffer is cleared to 0, the FW bit has no effect.

Note: *The usage of a force-write operation prevents the application from updating, in a later stage, the missing bits with a value different from 1. This can lead to unexpected or inconsistent data, or ECC.*

Adjusting programming timing constraints

Program operation timing constraints depend of the memory clock frequency, which directly impacts the performance. If timing constraints are too tight, the nonvolatile memory does not operate correctly, if they are too lax, the programming speed is not optimal.

The user must therefore trim the optimal programming delay through the WRHIGHFREQ parameter in the FLASH_ACR register. Refer to [Table 48](#) for the recommended programming delay, depending upon the memory clock frequency.

FLASH_ACR configuration register is common to both banks.

The application software must check that no program/erase operation is ongoing before modifying WRHIGHFREQ.

Caution: Modifying WRHIGHFREQ while programming/erasing the memory can corrupt its content.

Programming errors

When a program operation fails, an error is reported, as described in [Section 7.9](#).

Programming interrupts

See [Section 7.10: FLASH interrupts](#) for details.

7.3.6 FLASH erase operations

Erase operation overview

The memory can perform erase operations on 8-Kbyte user sectors, on one user flash memory bank, or on two user flash memory banks (for example mass erase). For more details in user flash memory, ICP, user options and OTP erase protection, see [Section 7.6](#).

Erase commands are issued through the AHB configuration interface. The memory supports one operation at a time, if it receives simultaneously a write and an erase request an error flag is raised, and both operations are canceled. See [Section 7.9](#) for details.

After successful completion of erase, all the bytes in the erased flash sectors are set to the flash default 0xFF value.

Erase and WRP

If the application software attempts to erase a write-protected user sector, the sector erase operation is aborted, and the WRPERR flag is raised in the FLASH_NS/SECSR register, as described in [Section 7.9.2](#).

Flash busy

Busy signals is described in [Monitoring ongoing write operations](#).

Writing to the FLASH control register FLASH_NS/SECCR and FLASH_OPTCR

Refer to [Writing to the FLASH control register FLASH_NS/SECCR and FLASH_OPTCR](#).

Enabling erase operations

Before erasing a sector, the application software must make sure that FLASH_NS/SECCR is unlocked. If this is not the case, an unlock sequence must be used (see [Section 7.6.7](#)).

Note: The application software must not unlock a register that is already unlocked, otherwise this register remains locked until next system reset. This can be used to deliberately lock-out a register from further accesses.

Similar constraints apply to bank erase requests.

Standard flash sector erase sequence

To erase an 8- or 6-Kbyte data user sector without security protections, proceed as follows:

1. Make sure protection mechanism does not prevent sector erase (WRP, secure flag, HDP).
2. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_NSSR register, and that the write buffer is empty by checking the WBNE bit in the FLASH_NSSR register.
3. Check and clear all the nonsecure error flags due to previous programming/erase operation. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_NSCR register, as described in [Section 7.6.7](#) (only if this register is not already unlocked).
5. Set the BKSEL bit, the SER bit and SNB bitfield in the FLASH_NSCR register. BKSEL indicates in which physical bank sector must be erased, then SER indicates a sector erase operation, while SNB contains the target sector number. In case of data sector, use the number of the corresponding regular sector.
6. Set the STRT bit in the FLASH_NSCR register.
7. Wait for the BSY bit to be cleared in the FLASH_NSSR register.
8. STRT bit is automatically cleared at the end of the sector erase, or in case of error.
9. Clear SER in FLASH_NSCR register if there are not anymore sector erase request to be issued.

Note: If another erase flag is requested simultaneously to the sector erase, a PGSERR error is generated.

Secure flash memory sector erase sequence

To erase an 8- or 6-Kbyte data secure configured user sector, proceed as follows:

1. Make sure write protection or HDP mechanism does not prevent sector erase.
2. Ensure that no memory operations are ongoing (by checking the BSY and DBNE bits in the FLASH_SECSR register), and that the write buffer is empty (by checking the WBNE bit in the FLASH_SECSR register).
3. Check and clear all the secure error flags due to previous programming/erase operations. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_SECCR register, as described in [Section 7.6.7](#) (only if this register is not already unlocked).
5. Set the BKSEL bit, the SER bit and SNB bitfield in the FLASH_SECCR register. BKSEL indicates in which physical bank sector must be erased, then SER indicates a sector erase operation, while SNB contains the target secure sector number. In case of data sector, use the number of the corresponding regular sector.

6. Set the STRT bit in the FLASH_SECCR register.
7. Wait for the BSY bit to be cleared in the FLASH_SECSR register.
8. STRT bit is automatically cleared at the end of the sector erase or in case of error.
9. Clear SER in FLASH_SECCR register if there are not any secure sector erase request to be issued.

Note: If another erase flag is requested simultaneously to the sector erase, a PGSERR error is generated.

Standard flash memory bank erase sequence

To erase bank where no sector is configured as secure:

1. Make sure protection mechanism does not prevent sector erase.
2. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_NSSR register and that the write buffer is empty by checking the WBNE bit in the same register.
3. Check and clear all the nonsecure error flags due to previous programming/erase operation. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_NSCR register, as described in [Section 7.6.7](#) (only if this register is not already unlocked).
5. Set the BKSEL bit and the BER bit in the FLASH_NSCR register to the targeted physical bank (swap setting is ignored).
6. Set the STRT bit in the FLASH_NSCR register to start the bank erase operation. Then wait until the BSY bit is cleared in the FLASH_NSSR register.
7. STRT bit is automatically cleared at the end of erase sequence or in case of error.
8. Clear BER in FLASH_NSCR register if there is not other bank erase request to be issued.

Secure flash memory bank erase sequence

To erase bank where all sectors are configured with secure flag:

1. Make sure protection mechanism does not prevent sector erase (for mass erase the HDP is also considered, it cannot be fully executed from HDPL= 2, 3 if HDP is defined).
2. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_SECSR register and that the write buffer is empty by checking the WBNE bit in the same register.
3. Check and clear all the secure error flags due to previous programming/erase operation. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_SECCR register, as described in [Section 7.6.7](#) (only if this register is not already unlocked).
5. Set the BKSEL bit and the BER bit in the FLASH_SECCR register to the targeted physical bank (swap setting is ignored).
6. Set the STRT bit in the FLASH_SECCR register to start the bank erase operation. Then wait until the BSY bit is cleared in the FLASH_SECSR register.
7. STRT bit is automatically cleared at the end of erase sequence or in case of error.
8. Clear BER in FLASH_SECCR register if there is not other bank erase request to be issued.

Flash mass erase sequence

To erase all sectors of both banks, using nonsecure access, all sectors must be configured as nonsecure. The application software can set the MER bit to 1 in FLASH_NSCR register, as described below:

1. Make sure protection mechanisms do not prevent mass erase ([Section 7.6](#)).
2. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_NSSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_NSSR register.
3. Check and clear all the nonsecure error flags due to previous programming/erase operation. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_NSCR register as described in [Section 7.6.7](#) (only if the registers are not already unlocked).
5. Set the MER bit to 1 in FLASH_NSCR register.
6. Set the STRT bit in the FLASH_NSCR register. Then wait until BSY bit is cleared in the FLASH_NSSR register.
7. STRT bit is cleared automatically at the end of the erase sequence, or in case of error.
8. Clear MER in FLASH_NSCR register.

Secure flash memory mass erase sequence

To erase all sectors of both banks simultaneously, using secure access, all sectors must be configured as secure. The application software can set the MER bit to 1 in FLASH_SECCR register, as described below:

1. Make sure protection mechanisms do not prevent mass erase ([Section 7.6](#)).
2. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_SECSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_SECSR register.
3. Check and clear all the secure error flags due to previous programming/erase operation. Refer to [Section 7.9](#) for details.
4. Unlock the FLASH_SECCR register as described in [Section 7.6.7](#) (only if the registers are not already unlocked).
5. Set the MER bit to 1 in FLASH_SECCR register.
6. Set the STRT bit in the FLASH_SECCR register. Then wait until BSY bit is cleared in the FLASH_SECSR register.
7. STRT bit is cleared automatically at the end of the erase sequence, or in case of error.
8. Clear MER in FLASH_SECCR register.

Note: Mass and bank erase also erase high-cycle data sectors aliased from the erased bank.

7.3.7 FLASH parallel operations

As the memory is divided into two independent banks, the embedded flash memory interface supports a read in one bank while a write (RWW: read while write) or an erase is executed in the other bank. It does not support write-while-write, nor read-while-read. Same is valid for the high-cycle data area, system flash libraries, or the OBK (located on Bank2).

In all cases, the sequences described in [Section 7.3.4](#), [Section 7.3.5](#) and [Section 7.3.6](#) apply.

7.3.8 FLASH error protections

Error correction codes (ECC)

The embedded flash memory supports an error correction code (ECC) mechanism, based on the SECDED algorithm, to correct single errors and detect double errors.

This mechanism uses nine ECC bits per 128-bit flash word, and applies to user and system memory. For read-only, OTP, flash high-cycle data, a stronger six ECC bits per 16-bit word is used. A double ECC error is generated for an OTP or flash high-cycle data virgin word (for example a word with 22 bits at 1). When this OTP or flash high-cycle data word is no more virgin, the ECC error disappears.

More specifically, during each read operation from a 128-bit flash word, the embedded flash memory retrieves the 9-bit ECC information, computes the ECC of the flash word, and compares the result with the reference value. If they do not match, the corresponding ECC error is raised, as described in [Section 7.9.10](#).

During each program operation, a 9-bit ECC code is associated to each 128-bit data flash word, and the resulting 137-bit flash word information is written in nonvolatile memory.

A similar mechanism applies to read-only and OTP areas, but with 6-bit ECC for 16-bit data.

7.3.9 OTP and RO memory access

OTP and RO memory are accessed through main AHB interface. The OTP is accessible at addresses 0x08FF_F000 to 0x08FF_F7FF, and the read-only section is accessible from 0x08FF_F800 to 0x08FF_FFFF.

FLASH one-time programmable area

The embedded flash memory offers a 2048-byte memory area dedicated to application non-confidential, one-time programmable data (OTP). This area is composed by 1024 words of 16 bits (plus 6 bits of ECC). It cannot be erased, and can be written only once. The OTP area can be accessed through the main AHB interface from address 0x08FF_F000 to 0x08FF_F7FE.

OTP data can be programmed by the application software by 16-bit chunks. Overwriting an already programmed 16-bit half-word can lead to data and ECC errors, and is therefore not supported.

Note: The OTP area is virgin when the device is delivered by STMicroelectronics.

When reading OTP data with a single error corrected or a double error detected, the embedded flash memory reports read errors, as described in [Section 7.9.10](#).

When reading OTP data not written by the application software (such as virgin OTP), the ECC correction reports a double-error detection (ECCD), and the data are to be found in the FLASH_ECCDR register. ECCD implies an NMI raised.

OTP write protection

OTP data are organized as 32 blocks of 32 OTP words, as shown in [Table 49](#). An entire OTP block can be protected (locked) from write accesses by setting the LOCKBLi bit (i = 0 to 31) corresponding to each OTP block in the FLASH_OTPBLR register. A block can be write-protected, if it has been programmed (even partially) or not.

The OTP block locking operation is irreversible, and independent from the product life state.

Note: The OTP area can be accessed only in read mode.

Table 49. Flash memory OTP organization

OTP block	AHB address	AHB word		Lock bit
		[31:16]	[15:0]	
Block 0	0x08FF F000	OTP001	OTP000	LOCKBL0
	0x08FF F004	OTP003	OTP002	
	...			
	0x08FF F03C	OTP031	OTP030	
Block 1	0x08FF F040	OTP033	OTP032	LOCKBL1
	0x08FF F044	OTP035	OTP034	
	...			
	0x08FF F07C	OTP063	OTP062	
Block 2	0x08FF F080	OTP065	OTP064	LOCKBL2
	0x08FF F084	OTP067	OTP066	
	...			
	0x08FF F0BC	OTP95	OTP94	
...				
Block 31	0x08FF F7C0	OTP993	OTP992	LOCKBL31
	0x08FF F7C4	OTP995	OTP994	
	...			
	0x08FF F7FC	OTP1023	OTP1022	

OTP write sequence

Follow the sequence below to write an OTP word:

1. Check that no memory operations are ongoing by checking the BSY bit in the FLASH_NSSR register and that the data buffer is empty by checking the DBNE bit in the FLASH_NSSR register.
2. Check and clear all the error flags due to previous programming/erase operation.
3. Set PG bit in the FLASH_NSCR register.
4. Check the protection status of the target OTP word (see [Table 49](#)). The corresponding LOCKBLi bit must not be set to 1.
5. Write two OTP words (32 bits) corresponding to the 4-byte aligned address shown in [Table 49](#). Alternatively, the application software can program separately the 16-bit MSB or 16-bit LSB. In this case the first 16-bit write operation starts immediately, without waiting for the second one.
6. Wait for the BSY bit to be cleared in the FLASH_NSSR register.
7. Clear PG bit in FLASH_NSCR register if there is not any programming request anymore in the bank.
8. Optionally, lock the OTP block using LOCKBLi to prevent further data changes.

Note: Do not write twice an OTP 16-bit word, otherwise an ECC error is generated.
 Writing OTP data at byte level is not supported, and generates a bus error.
 To avoid data corruption, it is important to complete the OTP write process (for example by reading back the OTP value), before starting an option change.

Flash read-only area

The embedded flash memory offers a 2-Kbyte area to store read-only data. This area can be accessed through the AHB main port, and is protected by a robust ECC scheme, as detailed in [Section 7.3.8](#).

The read-only information (programmed by STMicroelectronics) that can be used by the application software is detailed in [Table 50](#).

Table 50. Read-only public data organization

Read-only data name	Address	Comment
Unique device ID	0x08FF F800	U_ID[31:0]
	0x08FF F804	U_ID[63:32]
	0x08FF F808	U_ID[96:64]
Flash memory size/ package	0x08FF F80C	Flash memory size[15:0] / Package code[15:0]
Reserved	0x08FF F810 to 0x08FF FFFF	Reserved information

7.3.10 Flash high-cycle data

The embedded flash memory offers up to 96 Kbytes (maximum) memory area with high-cycling capability (100 kcycles) to store data and emulate EEPROM. It can be accessed through the AHB system port from address 0x0900_0000 to 0x0901_7FFF (see [Figure 27](#)). It is mapped in the 8 (or 4) last sectors of Bank 1 and 2. This area is protected by a robust 6-bit ECC, enabling a 16-bit read and write granularity, at the expense of having sector size shrunk to 6 Kbytes.

A threshold per bank (EDATA(1/2)_STRT) is programmable to determine the beginning of the data flash area. By default, the whole memory is used for code.

For example, if 48 Kbytes of data are needed in Bank 1, set EDATA1_EN to 1, and EDATA1_STRT to 7. If no data are needed in Bank2, set EDATA2_EN to 0. In this case the data are accessible from address 0x0900_0000 to 0x0900_BFFF for Bank 1.

For greater efficiency, it is recommended to use sector on the other bank for flash high-cycle data, so that the application benefits from RWW capability of the dual-bank arrangement.

When SWAP_BANK feature is enabled, the banks are swapped: the flash high-cycle data in Bank 2 are accessible from 0x0900_000 to 0x0900_BFFF, and the data in Bank 1 are accessible from 0x0900_C000 to 0x0901_7FFF.

A bus error is generated on:

- Attempt to access an address between 0x0900_0000 to 0x0901_7FFF and this address is not valid (EDATA(1/2)_EN not enabled or EDATA(1/2)_STRT not correct).
- Attempt to fetch instructions from flash high-cycle data area.

Erasing the data area sector is possible by normal erase request for the corresponding user flash sector (120-127 for the STM32H562/563/573xx devices, 24-31 for the STM32H523/33xx devices).

Protections and security of high-cycle area are detailed in [Section 7.6.9](#).

Figure 27. Flash high-cycle data memory map on 2-Mbyte devices

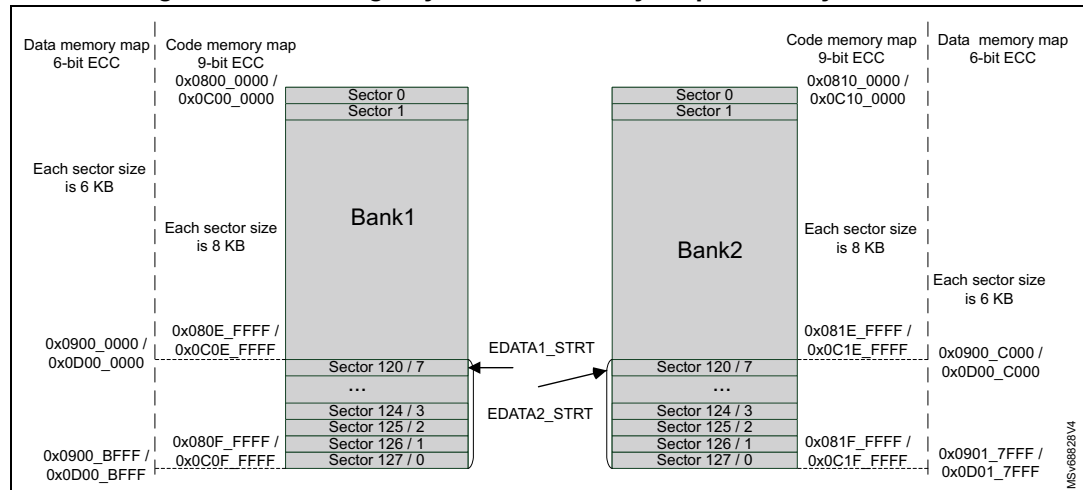


Figure 28. Flash high-cycle data memory map on 1-Mbyte devices

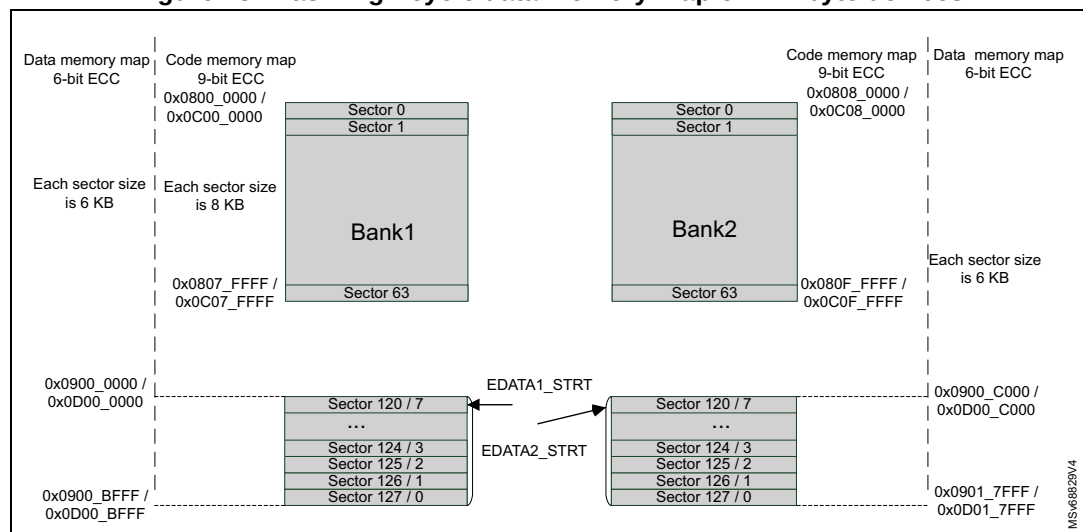


Figure 29. Flash high-cycle data memory map on 512-Kbyte devices

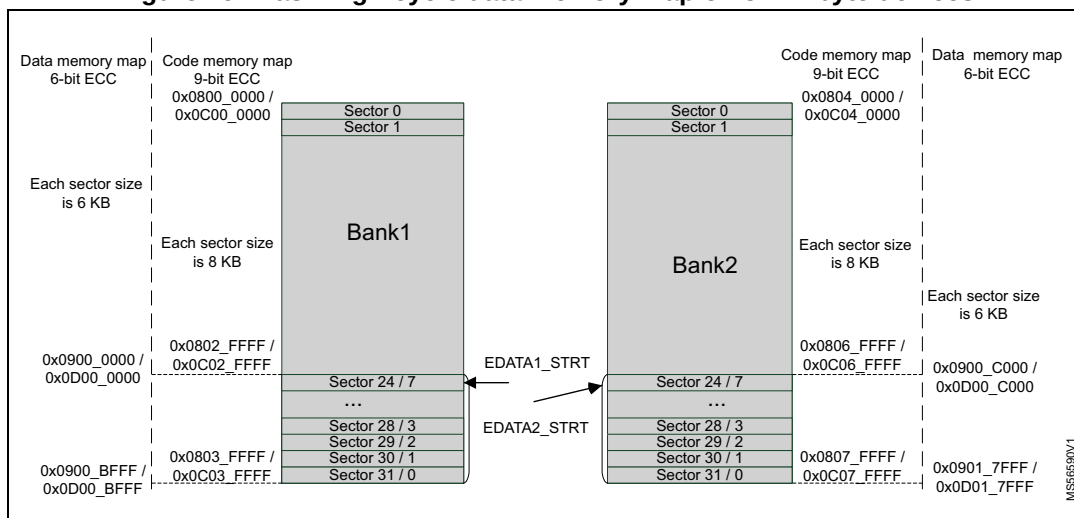
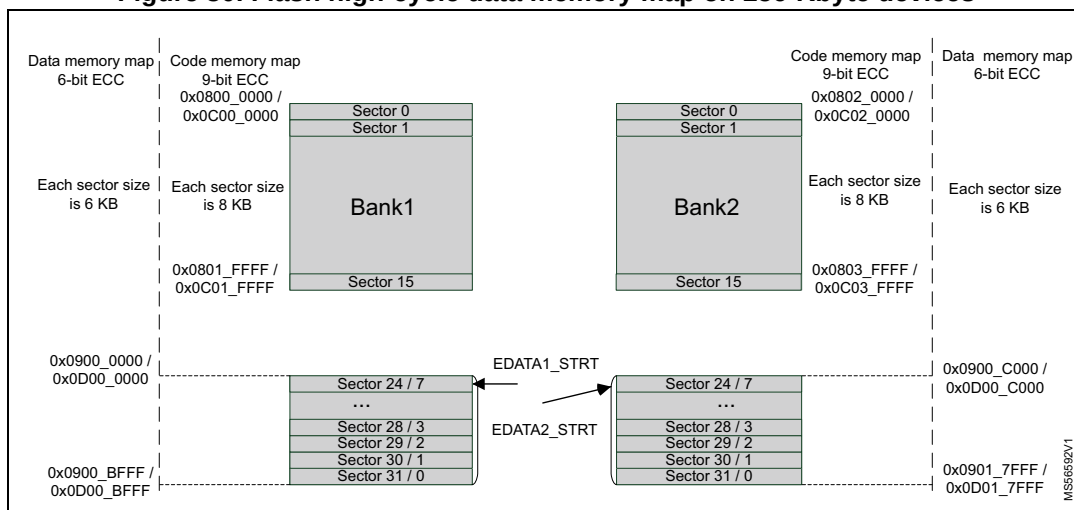


Figure 30. Flash high-cycle data memory map on 256-Kbyte devices



Note: When flash high-cycle data area on Bank1 is enabled, the code memory map is not continuous from Bank1 to Bank2 on 2M- and 512K-byte devices.

7.3.11 Flash bank swapping

Bank1 and Bank2 can be swapped for the user flash. This feature can be used after a firmware upgrade to restart the device on the new firmware. Bank swapping is an user option byte flag controlled by the SWAP_BANK bit of the FLASH_OPTCR register.

Bank specific settings for data area and security attributes follow the original bank and its contents. Control bit BKSEL always refers to physical bank, not the SWAP_BANK setting.

[Table 51](#) shows the accessible memory, depending upon the SWAP_BANK bit value.

Table 51. Memory map and swapping options (STM32H562/563/573xx devices)

Area	Corresponding bank		Start address	End address	Size (bytes)	Region name
	SWAP_BANK = 0	SWAP_BANK = 1				
User main memory	Bank1	Bank2	0x0800 0000	0x0800 1FFF	8 K	Sector 0
			0x0800 2000	0x0800 3FFF	8 K	Sector 1
			...	...	...	...
			0x080F E000	0x080F FFFF	8 K	Sector 127
	Bank2	Bank1	0x0810 0000	0x0810 1FFF	8 K	Sector 0
			0x0810 2000	0x0810 3FFF	8 K	Sector 1
			...	...	...	...
			0x081F E000	0x081F FFFF	8 K	Sector 127
System memory	Bank1		0x0BF8 0000	0x0BF8 1FFF	8 K	System 1 Sector 0
			0x0BF8 2000	0x0BF8 3FFF	8 K	System 1 Sector 1
			...	...	...	...
			0x0BF8 E000	0x0BF8 FFFF	8 K	System 1 Sector 7
	Bank2		0x0BF9 0000	0x0BF9 1FFF	8 K	System 2 Sector 0
			0x0BF9 0000	0x0BF9 3FFF	8 K	System 2 Sector 1
			...	...	...	...
			0x0BF9 E000	0x0BF9 FFFF	8 K	System 2 Sector 7

Table 52. Memory map and swapping options (STM32H543/553xx devices)

Area	Corresponding bank		Start address	End address	Size (bytes)	Region name
	SWAP_BANK = 0	SWAP_BANK = 1				
User main memory	Bank1	Bank2	0x0800 0000	0x0800 1FFF	8K	Sector 0
			0x0800 2000	0x0800 3FFF	8K	Sector 1
			...	...	...	...
			0x0807 E000	0x0807 FFFF	8K	Sector 63
	Bank2	Bank1	0x0808 0000	0x0808 1FFF	8K	Sector 0
			0x0808 2000	0x0808 3FFF	8K	Sector 1
			...	...	...	...
			0x080F E000	0x080F FFFF	8K	Sector 63

Table 52. Memory map and swapping options (STM32H543/553xx devices) (continued)

Area	Corresponding bank		Start address	End address	Size (bytes)	Region name
	SWAP_BANK = 0	SWAP_BANK = 1				
System memory	Bank1		0x0BF8 0000	0x0BF8 1FFF	8K	System 1 Sector 0
			0x0BF8 2000	0x0BF8 3FFF	8K	System 1 Sector 1
			...	...	...	...
			0x0BF9 6000	0x0BF9 7FFF	8K	System 1 Sector 11
	Bank2		0x0BF9 8000	0x0BF9 9FFF	8K	System 2 Sector 0
			0x0BF9 A000	0x0BF9 BFFF	8K	System 2 Sector 1
			...	...	...	...
			0x0BFA E000	0x0BFA FFFF	8K	System 2 Sector 11

Table 53. Memory map and swapping options (STM32H523/533xx devices)

Area	Corresponding bank		Start address	End address	Size (bytes)	Region name
	SWAP_BANK = 0	SWAP_BANK = 1				
User main memory	Bank1	Bank2	0x0800 0000	0x0800 1FFF	8 K	Sector 0
			0x0800 2000	0x0800 3FFF	8 K	Sector 1
			⋮	⋮	⋮	⋮
			0x0803 E000	0x0803 FFFF	8 K	Sector 31
	Bank2	Bank1	0x0804 0000	0x0804 1FFF	8 K	Sector 0
			0x0804 2000	0x0804 3FFF	8 K	Sector 1
			⋮	⋮	⋮	⋮
			0x0807 E000	0x0807 FFFF	8 K	Sector 31
System memory	Bank1		0x0BF8 0000	0x0BF8 1FFF	8 K	System 1 Sector 0
			0x0BF8 2000	0x0BF8 3FFF	8 K	System 1 Sector 1
			...	...	...	...
			0x0BF8 E000	0x0BF8 FFFF	8 K	System 1 Sector 7
	Bank2		0x0BF9 0000	0x0BF9 1FFF	8 K	System 2 Sector 0
			0x0BF9 0000	0x0BF9 3FFF	8 K	System 2 Sector 1
			...	...	...	...
			0x0BF9 E000	0x0BF9 FFFF	8 K	System 2 Sector 7

The SWAP_BANK bit in FLASH_OPTCR register is loaded from the SWAP_BANK option bit only after system reset or POR.

To change the SWAP_BANK bit (for example to apply a new firmware update), follow the sequence below:

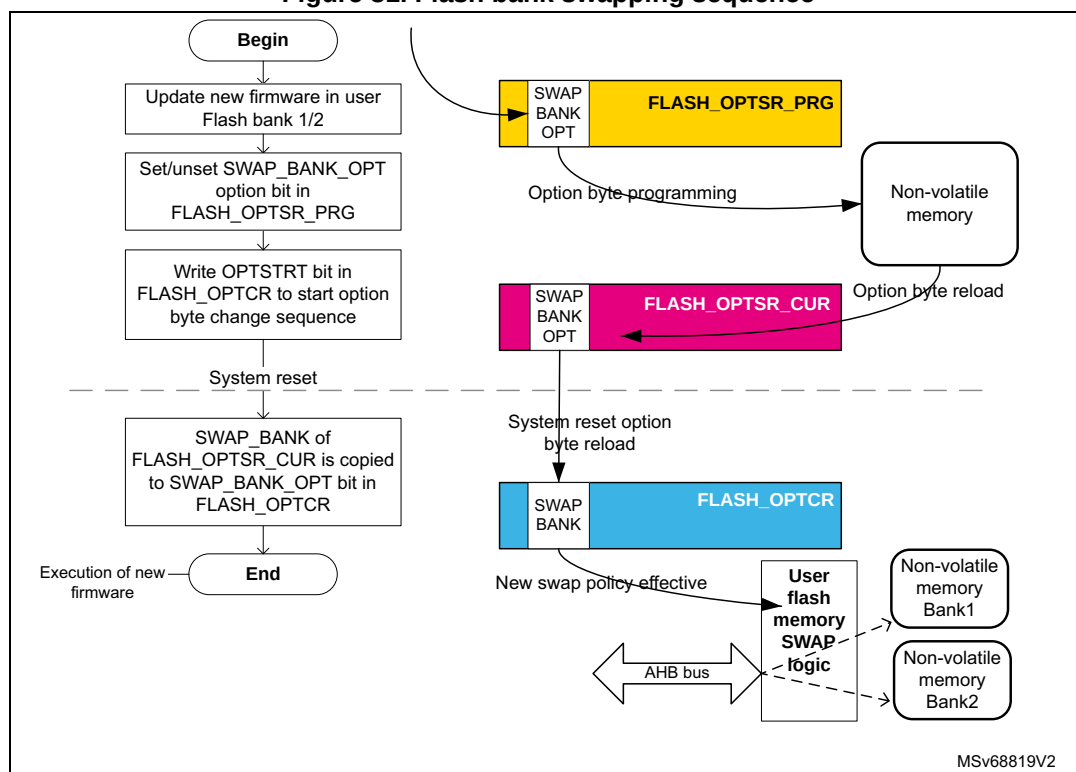
1. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_NS/SECSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_NS/SECSR register.
2. Clear all error flags due to a previous operation.
3. Unlock OPTLOCK bit, if not already unlocked.
4. Set the new desired SWAP_BANK value in the FLASH_OPTSR_PRG register.
5. Start the option byte change sequence by setting the OPTSTRT bit in the FLASH_OPTCR register.
6. Once the option byte change has completed, FLASH_OPTSR_CUR contains the expected SWAP_BANK value, but SWAP_BANK bit in FLASH_OPTCR has not yet been modified and the bank swapping is not yet effective.
7. Force a system reset or a POR. When the reset rises up, the bank swapping is effective (SWAP_BANK value updated in FLASH_OPTCR) and the new firmware shall be executed.

Note: The SWAP_BANK bit in FLASH_OPTCR is read-only, and cannot be modified by the application software.

The SWAP_BANK option bit in FLASH_OPTSR_PRG can be modified whatever the product state. Instead of being locked by PRODUCT_STATE, it is locked by (NS/SEC)BOOT_LOCK User OB.

Figure 31 gives an overview of the bank swapping sequence.

Figure 31. Flash bank swapping sequence



7.3.12 FLASH reset and clocks

Reset management

The embedded flash memory can be reset by a core domain reset, driven by the reset and clock control (RCC). The main effects of this reset are the following:

- All registers, except for option byte registers, are cleared, including read and write latencies. If the bank swapping option is changed, it is applied.
- Most control registers are automatically protected against write operations. To unprotect them, new unlock sequences must be used as described in [Section 7.6.7](#).

The memory can be reset by a power-on core domain reset, driven by the reset and clock control (RCC). When the reset falls, all option byte registers are reset. When the reset rises up, the option bytes are loaded, potentially applying new features. During this loading sequence, the device remains under reset, and the memory is not accessible.

The reset signal can have a critical impact on the memory: the content is not guaranteed if a device reset occurs during a write or erase operation.

Reset occurring during flash operation

If a reset occurs during a flash operation (programming, erase or option change), the content of the memory is not guaranteed. It is mandatory for integrity to restart the operation. The status register FLASH_OPSR gives information about operations interrupted by a reset.

FLASH_OPSR.CODE_OP gives opcode of operation. [Table 54](#) indicates how to use FLASH_OPSR, and which operation is required.

Table 54. Recommended reactions to FLASH_OPSR contents

CODE_OP	Operation interrupted	OTP_OP	SYSF_OP	BK_OP	DATA_OP	Flash area	ADDR_OP min	ADDR_OP max	Recommended action
0x000	No operation ongoing while reset	0	0	0	0	_(1)	-	-	No extra action
0x001	Write operation	0	0	0/1	0	User flash	0x0000	0xFFFF	Erase sector and rewrite
		0	1	0/1	0	System flash	0x0000	0x0FFF	
		1	0	0	0	OTP	0x0600	0x07FF	
		1	0	1	0	OBKeys ⁽²⁾	0x0000	0x03FF	
		0	0	0/1	1	Data area ⁽³⁾	0xF000	0xFFFF	
0x010	OBK alternate sector erase	0	0	1	0	OBKeys	0	0	Relaunch the alternate sector erase
0x011	Sector erase ⁽⁴⁾	0	0	0/1	0	User flash	0x0000	0xFFFF	Relaunch sector erase
		0	1	0/1	0	System flash	0x0000	0x0FFF	
0x100	Bank erase	0	0	0/1	0	User flash	-	-	Relaunch bank erase
0x101	Mass erase	0	0	0	0		-	-	Relaunch mass erase

Table 54. Recommended reactions to FLASH_OPSR contents (continued)

CODE_OP	Operation interrupted	OTP_OP	SYSF_OP	BK_OP	DATA_OP	Flash area	ADDR_OP min	ADDR_OP max	Recommended action
0x110	Option change	0	0	0	0	User configuration	-	-	New attempt on option change
0x111	OBK swap sector	0	0	1	0	OBKeys	-	-	Erase alternate sector, reprogram new OBKeys and relaunch swap

1. Dash represents "does not matter".
2. Depends on current OBK sector.
3. Depends on EDATA setting in the OB.
4. Addresses indicated are aligned to sector start, data area sectors are erased by erase request to corresponding user flash memory sector.

Clock management

The memory uses the microcontroller system clock (sys_ck), here the AHB interface clock.

7.4 FLASH option bytes

7.4.1 About option bytes

The memory includes a set of nonvolatile option bytes. They are loaded at power-on reset and can be read and modified only through configuration registers. This section details:

- when option bytes are loaded
- how application software can modify them
- their detailed list, together with their initial values (before the first option byte change, user default configuration).

7.4.2 Option bytes loading

There are multiple ways of loading the option bytes:

- **Power-on wake-up**
When the device is first powered, the embedded flash memory automatically loads all the option bytes. During the option byte loading sequence, the device remains under reset and the embedded flash memory cannot be accessed.
- **Wake-up from system Standby**
When the core power domain, which contains the embedded flash memory, is switched from Standby mode to Run mode, the embedded flash memory behaves as during a

power-on sequence. **During loading time the device is not under reset, unlike power-on sequence.**

- **Dedicated option byte reloading by the application**

When the user application successfully modifies the option byte content through the memory registers, the nonvolatile option bytes are programmed and the memory automatically reloads all option bytes to update the option registers.

Note: The option byte read sequence is protected by error correction code. In case of error, the option bytes are loaded with default values (see [Section 7.4.3](#)), different from the initial values (user default configuration), and more restrictive.

7.4.3 Option bytes modification

Changing user option bytes

A user option byte change operation can be used to modify the configuration and the protection settings saved in the nonvolatile option byte area.

There are several rules enforced when attempting to change a user option byte, they are summarized in [Section 7.4.7](#). Failing to stick to those rules usually results in errors, described in [Section 7.9.12](#).

The embedded flash memory features two sets of option byte registers:

- The first register set contains the current values of the option bytes. Their names have the `_CUR` extension. All these registers are read-only. Their values are automatically loaded from the nonvolatile memory after power-on reset, wake-up from system standby or after an option byte change operation.
- The second register set allows the modification of the option bytes. Their names contain the `_PRG` extension. All “`_PRG`” registers can be accessed in read/write mode.

When the `OPTLOCK` bit in `FLASH_OPTCR` register is set, it is not possible to modify the `FLASH_XXX_PRG` registers.

When `OPTSTRT` bit is set to 1, the memory checks the programming sequence (`PGSERR`) and the conditions described in [Section 7.4.7](#) (`OPTCHANGEERR`). If no error has been detected (`PGSERR/OPTCHANGEERR`), the flash interface launches the option byte modification, and updates the option byte registers with `_CUR` extension.

If one of the condition described in [Section 7.4.7](#), [Section 7.9.12](#) or [Section 7.9.5](#), alternatively [Section 7.9.4](#) is not respected, the memory aborts the option byte change operation. In this case, the `FLASH_XXX_PRG` registers are not overwritten by the current option value. The user application can check what was wrong in their configuration.

Unlocking the option byte modification

After reset, the `OPTLOCK` bit is set to 1 and the `FLASH_OPTCR` is locked. As a result, the application software must unlock the option configuration register before attempting to change the option bytes. The `FLASH_OPTCR` unlock sequence is described in [Section 7.6.7](#).

Option bytes modification sequence

To modify user option bytes, follow the sequence below:

1. Check that no memory operations are ongoing by checking the BSY bit in the FLASH_NS/SECSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_NS/SECSR register.
2. Check the data buffer is empty (DBNE = 0) in FLASH_NS/SECSR register.
3. Clear all error flags due to a previous operation.
4. Unlock FLASH_OPTCR register as described in [Section 7.6.7](#), unless the register is already unlocked.
5. Write the desired new option byte values in the corresponding option registers (FLASH_XXX_PRG).
6. Set the option byte start change OPTSTRT bit to 1 in the FLASH_OPTCR register.
7. Wait until BSY bit is cleared in FLASH_NS/SECSR register.
8. OPTSTRT bit is cleared automatically at the end of the sequence (or in case of error).
9. Reset the device. This step is always recommended, becomes mandatory when one of the following option bytes is impacted:
 - a) SECBOOTADDR
 - b) NSBOOTADDR
 - c) TZEN
 - d) BOOT_UBE

Note: If a reset or a power-down occurs while the option byte modification is ongoing, the original option byte value is kept. A new option byte modification sequence is required to program it.

Option bytes overview

[Table 55](#) lists all the user option bytes managed through the memory registers, as well as the initial values before the first option byte change (user default configuration).

Table 55. Option bytes organization

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
FLASH_OPTSR	SWAP BANK	Res.	BOOT_UBE								IWDG_STDBY	IWDG_STOP	Res.	Res.	IO_VDDIO2_HSLV	IO_VDD_HSLV	PRODUCT_STATE								NRST_STDBY	NRST_STOP	Res.	WWDG_SW	IWDG_SW	BORH_EN	BOR_LEV		
	0										1	0													1	1		0	1	0	0	1	1
FLASH_OPTSR2	TZEN								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBPD_DIS	Res.	SRAM2_ECC	SRAM3_ECC	BKPRAM_ECC	SRAM2_RST	SRAM13_RST	Res.	Res.	
									1	1	0	0	0	0	1	1															1		1

Table 55. Option bytes organization (continued)

[illegible]

Table 55. Option bytes organization (continued)

Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FLASH_SECEPOCHR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC_EPOCH																							
									0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
FLASH_HDP1R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_END								HDP1_STRT															
																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
FLASH_HDP2R	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_END								HDP2_STRT															
																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1

7.4.4 Description of user and system option bytes

The general-purpose option bytes that can be used by the application are listed below:

- Watchdog management
 - IWDG_STOP: independent watchdog IWDG (also known as WDGLS_CD) counter active in Stop mode if 1 (stop counting or freeze if 0)
 - IWDG_STDBY: independent watchdog IWDG (also known as WDGLS_CD) counter active in Standby mode if 1 (stop counting or freeze if 0)
 - IWDG_SW: hardware (0) or software (1) IWDG (also known as WDGLS_CD) watchdog control selection

Note: If the hardware watchdog “control selection” feature is enabled (set to 0), the watchdog is automatically enabled at power-on, thus generating a reset unless the watchdog key register is written to or the down-counter is reloaded before the end-of-count is reached.

Depending on the configuration of IWDG_STOP and IWDG_STDBY options, the IWDG can continue counting (1) or not (0) when the device is, respectively, in Stop or Standby mode. When the IWDG is kept running during Stop or Standby mode, it can wake up the device from these modes.

- Reset management
 - BOR_LEV: Brownout level option, indicating the supply level threshold that activates/releases the reset
 - BORH_EN: enabling a high BOR level
 - NRST_STDBY: generates a reset when entering Standby mode if cleared to 0
 - NRST_STOP: generates a reset when entering Stop mode if cleared to 0.

Note: Whenever a Standby (Stop) mode entry sequence is successfully executed, the device is reset instead of entering Standby (Stop) mode if NRST_STDBY (NRST_STOP) is cleared to 0.

- Device options
 - IO_VDDIO2_HSLV: enables the configuration of pads below 2.7 V for VDDIO2 power rail if set to 1
 - IO_VDD_HSLV: enables the configuration of pads below 2.7 V for VDD power rail if set to 1
 - USBPD_DIS: USB Type-C® dead battery pull-down behavior on UCPDx_CC1

At delivery, the values programmed in the user option bytes are the following:

- Watchdog management
 - IWDG (also known as WDGLS_CD) active in Standby and Stop modes: 0x1
 - IWDG (also known as WDGLS_CD) not automatically enabled at power-on: 0x1
- Reset management
 - BOR: brownout level option (reset level = 2.1 V): 0x0. A reset is not generated when the device enters Standby, Stop low-power mode (value = 0x1)
- Device working in the full voltage range with I/O speed optimization at low-voltage disabled (IO_VDDIO2_HSLV = 0 and IO_VDD_HSLV = 0)

Refer to [Section 7.11](#) for details.

7.4.5 Description of data protection option bytes

The option bytes that can be used to enhance data protection are listed below:

- PRODUCT_STATE[7:0]: A product life cycle state (see [Section 7.6.11](#) for details).
- WRPSGn1/2: write protection option of the corresponding group of four consecutive sectors in Bank1 (respectively Bank2). It is active low. Refer to [Section 7.6.8](#) for details.
 - Bit N: Group embedding sectors $4 \times N$ to $4 \times N + 3$
- SECWMx: TrustZone® secure only watermark area definition (refer to [Section 7.6.1](#) for details).
 - SECWM1_STRT (respectively SECWM1_END) contains the first (respectively last) sector of the secure access only zone in Bank1
 - SECWM2_STRT (respectively SECWM2_END) contains the first (respectively last) sector of the secure access only zone in Bank2.
- TZEN: this nonvolatile option can be used by the application to activate the secure access mode, as described in [Section 7.6](#). For security reasons the TZEN is stored with redundancy on 8 bits. While the TZEN OB value is set immediately after programming, to actually use TZ features, reset is required to set the TZ_STATE in SBS.

When TZEN is activated, secure watermark settings and secure boot address can be reset to default to prevent a deadlock in case the previous values were not correct.

- HDPx: Secure hide protection - HDPL exclusive area control in the user flash memory.

When factory programmed values of the data protection option bytes are the following:

- Flash bank erase operations do not impact watermarked secure data areas
- Secure watermark areas protections disabled (start addresses higher than end addresses)
- Write protection disabled (all option byte bits set to 1)
- TrustZone Secure access mode disabled (TZEN option byte value = 0xC3)

Refer to [Section 7.11](#) for details.

7.4.6 Description of boot address option bytes

Below the list of option bytes that can be used to configure the appropriate boot address for an application:

- PRODUCT_STATE

- **BOOT_UBE**: Selects either STiRoT or User flash as default boot address. Also used by RSS(SFI) to choose either Bootloader or STiRoT as next stage. Available only on products embedding cryptographic acceleration (STM32H533/573xx).
- **NSBOOTADD**: Selects default boot address when TZ is disabled.
- **SECBOOTADD**: Selects default boot address when TZ is enabled.
- **(NS/SEC)BOOT_LOCK**: Protects the boot configuration from further modification attempts.
- **SWAP_BANK**: bank swapping option, set to 1 to swap user flash banks after boot (see [Section 7.3.11](#)). If **BOOT_LOCK** corresponding to TZEN state (SEC/NS) is active, the value in **SWAP_BANK** is fixed, read-only.

When STMicroelectronics delivers the device, the **PRODUCT_STATE** is Open, **BOOT_LOCK** is not set (0xC3) and the **BOOT_UBE** is set to user flash (0xB4). Addresses are **SECBOOTADD** = 0x0C00 0000, **NSBOOTADD** = 0x0800 0000.

Refer to [Section 7.11](#) for details.

7.4.7 Specific rules for modifying option bytes

On top of **OPTLOCK** bit and register access rules, there are other protections for selected security-sensitive option byte fields.

Different option bytes can be modified simultaneously, but if they rely on each other for protection, both states are checked. For example, when trying to modify **PRODUCT_STATE** and **TZEN** simultaneously, resulting state must be coherent with the rules. With few exceptions, listed in this section, failing to uphold the rules results in raising **OPTCHANGEERR** flag ([Section 7.9.12](#) for additional details).

Table 56. Specific modifying rules

Option byte	HDPL	TZ secure	Value	Product state
PRODUCT_STATE	1 ⁽¹⁾	Some values only when TZ is enabled	Set of possible transitions	Set of possible transitions (Table 48)
HDP	1 ⁽²⁾	⁽³⁾	-	Refer to Table 57
TZEN	-	-	-	Regression, Open or Provisioning
EPOCH	-	-	PRG > CUR	Regression
EPOCH_NS	-	-	PRG > CUR	NS-Regression, Regression
SECWM	-	Secure mode only ⁽²⁾	-	Refer to Table 57
BOOT_UBE	-	-	-	Open or Provisioning, SECBOOT_LOCK disabled
SECBOOTADD	-	Secure mode only ⁽²⁾	-	
NSBOOTADD	-	-	-	Open or Provisioning, NSBOOT_LOCK disabled
LOCKBL	-	-	One way switch ⁽²⁾	-
SWAP_BANK	-	Fixed if BOOT_LOCK corresponding to TZEN is set	-	-
SECBOOT_LOCK	-	Secure mode only ⁽²⁾	-	Open, Regression or Provisioning to unlock
NSBOOT_LOCK	-	-	-	

1. Most transitions are possible regardless of HDPL, only a few require specific HDPL, see text below.
2. No OPTCHANGEERR raised in violation of this.
3. Dash means there is no limitation from this side.

Even in the closed PRODUCT_STATE progression, some OBs can still be modified, if all the other constraints are satisfied. An overview is presented in [Table 57](#).

Table 57. OB modifiable in closed product

PRODUCT_STATE	OBs that can be modified
TZ-Closed	SWAP_BANK, LOCKBL, PRODUCT_STATE, SECWM, HDP
Closed	SWAP_BANK, LOCKBL, PRODUCT_STATE
Locked	SWAP_BANK, LOCKBL

Specific rules must be respected to update the following OB:

- **PRODUCT_STATE**
PRODUCT_STATE transitions follow a state machine with two types of transitions. Locking down the product is allowed without restriction. Opening the product is possible only using a debug interface and following a digital signature verification. The regression to a less secure state results in the erase of the protected content. More details are given in [Section 7.6.11](#).
Summary: Selected changes are possible only in HDPL1. Some states are accessible only when TZEN is enabled.
- **HDP**
Can be modified only in HDPL1. Do not set it to overlap with flash high-cycle data area.
- **TrustZone access mode (TZEN)**
Can be changed only in Open, Regression and Provisioning.
- **SEC_EPOCH and NS_EPOCH**
The value programmed must be greater than current value. The increment is done in specific product states: Regression and NS-Regression.
- **Secure watermark area (SECWM1/2_STRT and SECWM1/2_END)**
Can be changed only in secure mode (TZ_state = 0xB4). Automatically reset to default value when TZEN is enabled. Do not set it to overlap with flash high-cycle data area.
- **BOOT_UBE**
Available only on cryptography enabled devices. Can be changed only in product states open for debug like Open and also in Provisioning. SECBOOT_LOCK must be disabled to change.

- **SECBOOTADD**
Can be changed only in Open and Provisioning. Locked by SECBOOT_LOCK. Automatically reset to default value when TZEN is enabled.
- **NSBOOTADD**
Can be changed only in Open and Provisioning. Locked by NSBOOT_LOCK.
- **LOCKBL**
Can be changed freely only in one direction. A permanent irreversible switch.
- **SWAP_BANK**
Not modifiable when both TZEN and SECBOOT_LOCK are 0xB4 (set) or when TZEN = 0xC3 (disabled) and NSBOOT_LOCK is active (0xB4).
- **SECBOOT_LOCK**
Can be changed only freely in locking direction. Unlock is possible in Open, Provisioning and Regression.
- **NSBOOT_LOCK**
Can be changed only freely in locking direction. Unlock is possible in Open, Provisioning and Regression.

Note: For all user option bytes above: default values are loaded and Tamper is signaled when a double ECC error occurs during OBL.

7.5 Option bytes key (OBK) management

This section describes the key storage tied with SAES and OBK-HDPL value set by the SBS (refer to [Section 14.3.7: SBS hardware secure storage control](#)). The OBKs are stored in 128 bits and protected by 9-bit ECC (SEC/DED). They are written alternatively in two sectors.

7.5.1 OBK loading

OBKs are memory mapped, and accessible through the main AHB bus (C-Bus), starting from address 0x0FFD 0000. A 9-bit ECC is associated to each 128-bit data flash word.

Note: Read access is allowed in the current and alternate sector, except for the last address of the current and alternate sector. In case of read access, no error is reported but the read data is always 0x0 (see [Section 7.6.12](#)).

7.5.2 OBK access per HDPL level

[Table 58](#) describes the option byte key areas. The key storage is not dedicated to a particular key, and the usage is defined by the application.

An option byte key can be accessed only if the OBK-HDPL (set in SBS) matches the HDPL associated to the storage offset, indicated below (refer to [Section 14.3.7: SBS hardware secure storage control](#)). If this is not the case, an OBKERR error is raised.

Table 58. Option bytes key area

OBK-HDPL	Address offset start	Address offset end	Comment
-	0x0000	0x00FF	Reserved
1	0x0100	0x08FF	OEMiRoT keys

Table 58. Option bytes key area (continued)

OBK-HDPL	Address offset start	Address offset end	Comment
2	0x0900	0x0BFF	uRoT, OS or secure application
3	0x0C00	0x17FF	HDPL 3 secure keys
3	0x1800	0x1FEF	HDPL 3 nonsecure keys

7.5.3 OBK programming sequence

The sequence to write OBK is the same as a write to main flash except that the “alternate sector” (ALT_SECT) bit must be set to write in the alternate OBK sector. The ALT_SECT bit is checked each time a write access is received. If it does not match the one stored in the write buffer, an error is raised.

Note: Before writing in the alternate OBK sector the user must ensure that the whole key space (128, 256, or 512 bits) is free, by reading the alternate OBK sector.
If the alternate sector is not free, erase the sector using the command ALT_SECT_ERASE.

Secure programming method

Using the secure register if TrustZone is active (TZ_STATE = 0xB4):

1. Program the ALT_SECT bit.
2. Make sure protection mechanism does not prevent programming (TZ, PRIV and OBK-HDPL).
3. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_SECSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_SECCR register.
4. Check and clear all the secure error flags due to previous operation.
5. Unlock the FLASH_SECCR register only if this register is not already unlocked, as described in [Section 7.6.7](#).
6. Enable write operations by setting the SECPG bit in the FLASH_SECCR register.
7. Write one flash-word corresponding to 16-byte data starting at 16-byte aligned address.

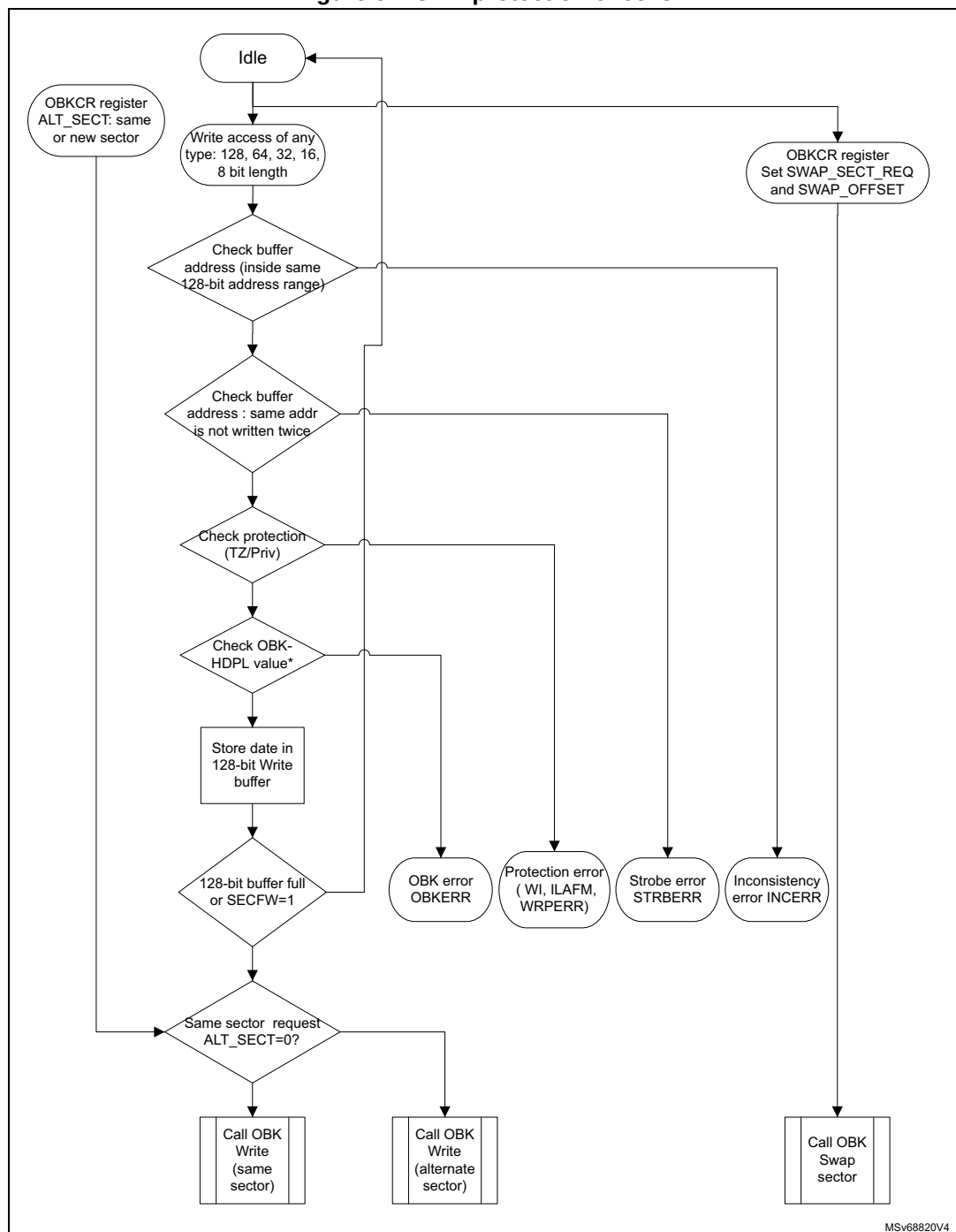
Note: WBNE flag indicates if the 128-bit write buffer is waiting for new data.

Note: No erase request, options change request, OBK swap sector or OBK alternate sector erase is allowed between first write and the completion of flash write operation.

8. Wait for the BSY bit to be cleared in the corresponding FLASH_SECSR register.
9. Clear PG bit in FLASH_SECCR register if there is not any programming request anymore in the bank.

If step 7 is executed incrementally (byte per byte), the write buffer can become partially filled. In this case the application software can decide to force-write what is stored in the write buffer by using FW bit in FLASH_SECCR register. In this particular case, the unwritten bits are automatically set to 1. If no bit in the write buffer is cleared to 0, the FW bit has no effect.

Figure 32. OBK protection checks

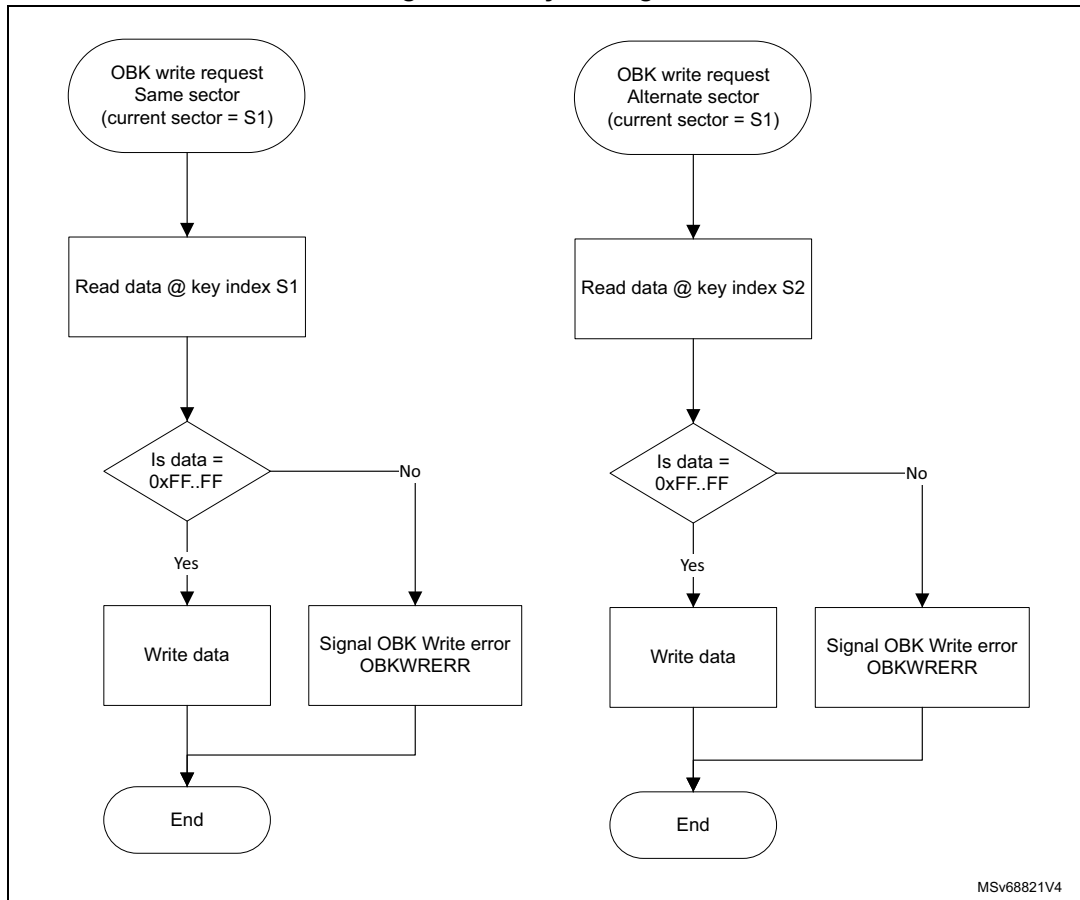


7.5.4 OBK programming finite state machine

When the OBK write buffer is filled, the OBK programming is launched automatically in the current OBK sector or in the alternate OBK sector, depending upon the ALT_SECT bit.

If the address is not virgin, the OBKWRERR is raised.

Figure 33. Key writing flow



7.5.5 OBK swap sector

The user can request a swap of the option byte keys by setting the bit `SWAP_SECT_REQ` in the `NS/SECOBKCFGR` register. The number of keys swapped is defined by the `SWAP_OFFSET` in `NS/SECOBKCFGR` register.

Swap is limited by `OBK-HDPL` value, the following `SWAP_OFFSET` values are permitted:

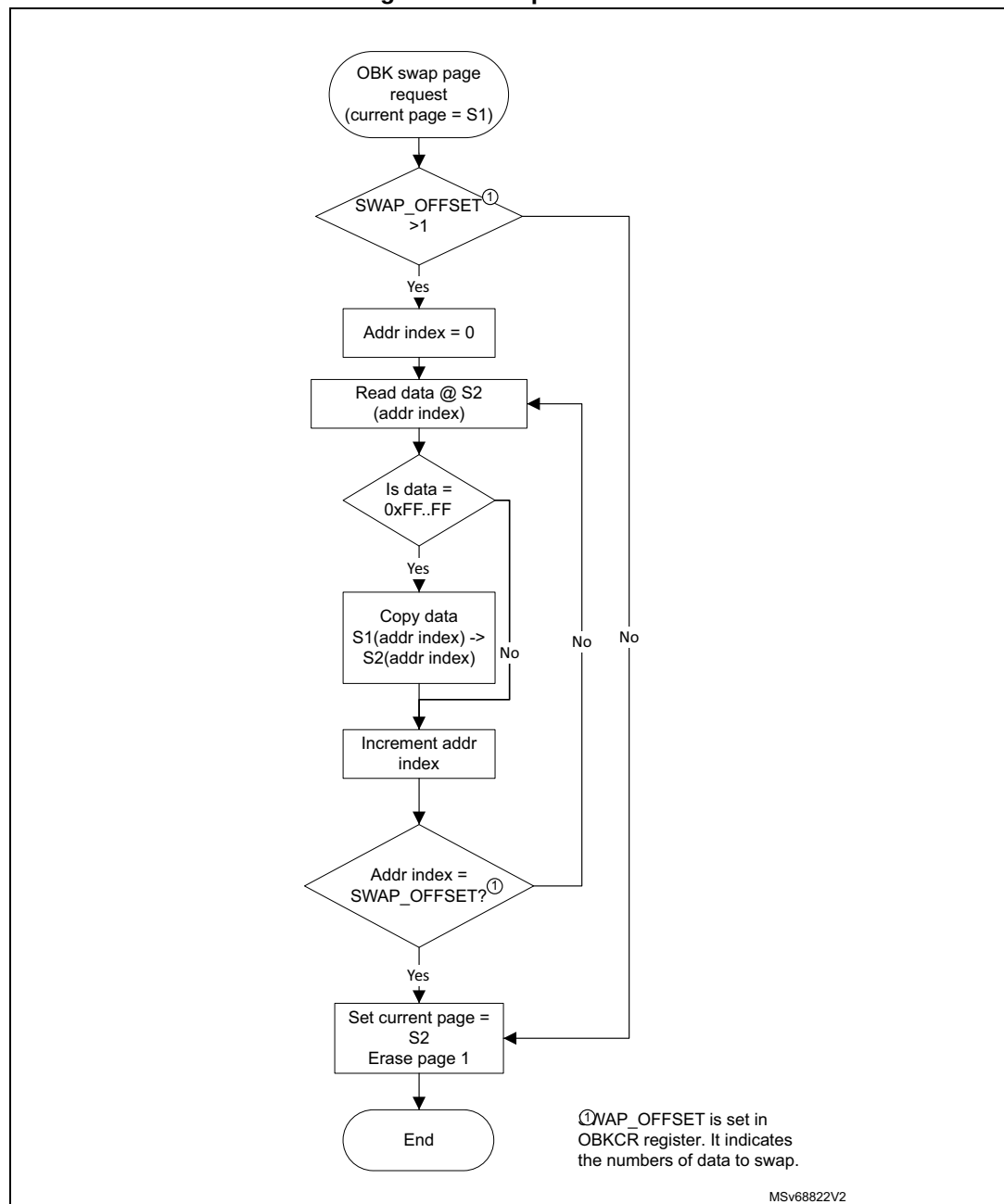
- `OBK_HDPL1_OFFSET` = 144
- `OBK_HDPL2_OFFSET` = 192
- `OBK_HDPL3SEC_OFFSET` = 384
- `OBK_HDPL3NS_OFFSET` = 511

The programming sequence is the following:

1. Check that no memory operations are ongoing by checking the `BSY` and `DBNE` bit in the `FLASH_SEC/NSSR` register and that the write buffer is empty by checking the `WBNE` bit in the `FLASH_SEC/NSSR` register.
2. Check and clear all the secure/nonsecure error flags due to previous operation.
3. Unlock `NS/SECOBKCFGR` register if not unlocked already thanks to `FLASH_NS/SECOBKKEYR`.

4. Define the numbers of keys to be swapped thanks to SWAP_OFFSET field in NS/SECOBKCFGR. SWAP_OFFSET must be equal to or greater than OBK_HDPL<N-1>_OFFSET.
5. Set bit SWAP_SECT_REQ in NS/SECOBKCFGR.
6. Wait for BSY bit to be cleared. In case of error OBKERR flag is set.
7. SWAP_SECT_REQ is cleared automatically at the end of the operation or in case of error.

Figure 34. Swap workflow



While the OBK SWAP operation is ongoing, the BSY flag is set and no other operation (write, erase, user opt change) can be launched in parallel.

7.5.6 OBK alternate sector erase

The user can request the erase of the alternate OBK sector by setting the bit ALT_SECT_ERASE in the FLASH_NS/SECOBKCFGR register.

The programming sequence must be:

1. Check that no memory operations are ongoing by checking the BSY and DBNE bits in the FLASH_SEC/NSSR register and that the write buffer is empty by checking the WBNE bit in the FLASH_SEC/NSSR register.
2. Check and clear all the secure/nonsecure error flags due to previous operation.
3. Set bit ALT_SECT_ERASE in NS/SECOBKCFGR.
4. Wait for BSY bit to be cleared.
5. ALT_SECT_ERASE is cleared automatically at the end of the operation or in case of error.

7.6 FLASH security and protections

To protect sensitive information against unwanted operations (such as reading confidential areas, illegal programming of immutable sectors, or malicious flash memory erasing), the FLASH implements the following protection mechanisms:

- TrustZone backed watermark and block security protection
- Temporal isolation protection (HDP)
- Configuration protection
- User flash write protection
- Device nonvolatile security life cycle and application boot state management
- OTP locking

The microcontroller can have TrustZone active (TZ_STATE = 0xB4) or not (TZ_STATE = 0xC3).

While TrustZone is active the flash memory can be accessed in four basic modes:

- nonsecure and unprivileged
- nonsecure and privileged
- secure and privileged
- secure and unprivileged

While TrustZone is disabled, only two modes are possible:

- nonsecure and unprivileged
- nonsecure and privileged

The flash interface evaluates the access restrictions in the following order:

1. TrustZone security
2. Write protection (for write access)
3. HDP
4. Privilege

7.6.1 TrustZone security protection

The global TrustZone system security is activated by setting the TZEN option in the FLASH_OPTSR2_PRG register. The actual state of TZ is however determined by the SBS and the setting of TZ_STATE (refer to [Section 14.3.5: SBS boot control](#)).

When TrustZone is active (TZ_STATE = 0xB4), additional security features are available:

- Secure watermark-based user options bytes defining secure areas
- Secure or nonsecure block-based areas can be configured on-the-fly after reset. This is a volatile secure area
- Additional product states associated with **closed_secure** provide protection of secure domain against external access (debug or bootloader)
- Erase or program operation can be performed in secure or nonsecure mode with associated configuration bit. When the TrustZone is disabled (TZ_STATE = 0xC3), the above features are deactivated and all secure registers are RAZ/WI.

Activating TrustZone security

When the TrustZone is activated (TZ_STATE = 0xB4), the SECBOOTADD and the secure watermark areas are set to a secure default value. Default settings are also used in case of corrupted configuration. In this default state, user flash memory is secure (see [Table 59](#)).

Table 59. Default secure watermark

Secure watermark in case of configuration error	Security attribute
SECWMx_STRT = 0 SECWMx_END = 0x7F ⁽¹⁾	All flash memory is secure

1. Default and maximum values depend upon the total number of sectors in the user flash memory.

Illegal access generation

A nonsecure access to a secure flash memory area is RAZ/WI, it generates an illegal access event. An illegal access interrupt is generated if the FLASHIE illegal access interrupt is enabled in the TZIC_IER2 register.

A nonsecure access to a secure flash register generates an illegal access event. An illegal access interrupt is generated if the FLASH_REGIE illegal access interrupt is enabled in the TZIC_IER2 register.

Table 60. Flash memory TZ protection summary

TZ protection (TZ_STATE = 0xB4)		Main flash	
		Nonsecure sector	Secure sector
Secure	Fetch	BUS ERROR	OK
	Read	RAZ, ILAFM	
	Write	WI, WRPERR, ILAFM	If not WRP: OK else: WI, WRPERR
	Erase		

Table 60. Flash memory TZ protection summary (continued)

TZ protection (TZ_STATE = 0xB4)		Main flash	
		Nonsecure sector	Secure sector
Nonsecure	Fetch	OK	BUS ERROR
	Read		RAZ, ILAFM
	Write	If not WRP: OK else: WI, WRPERR	WI, WRPERR, ILAFM
	Erase		

Table 61. TZ protection and bank or mass erase summary

TZ protection (TZ_STATE = 0xB4)	Main flash		
	Nonsecure flash	Secure flash	Mix of secure and nonsecure
MES	WI, WRPERR, ILAFM	OK	WI, WRPERR, ILAFM
MENS	OK	WI, WRPERR, ILAFM	WI, WRPERR, ILAFM

Deactivate TrustZone security

Reversing of the TZ setting in OB (TZEN) is possible when the product state is ST-RoT-Ready or Provisioning. Deactivating TZEN from other states requires full regression (Regression state).

When the TrustZone is deactivated (TZ_STATE = 0xC3) after option bytes loading, the following security features are deactivated:

- Watermark-based secure area (refer to [Watermark-based secure flash area protection](#))
- block-based secure area (refer to [Section 7.6.3](#))
- Secure interrupts (refer to [Section 7.9.11](#))
- All secure registers are RAZ/WI.

Watermark-based secure flash area protection

When TrustZone security is active (TZ_STATE = 0xB4), a part of the flash memory can be protected against nonsecure read and write accesses. Up to two different nonvolatile secure areas can be defined by option bytes and can be read or written by a secure access only: one area per bank can be selected with a sector granularity.

The secure areas are defined by a start sector offset and end sector offset using the SECWMx_STRT and SECWMx_END option bytes. These offsets are defined in the secure watermark registers address registers [FLASH security watermark for Bank1 \(FLASH_SECWM1R_CUR\)](#), [FLASH security watermark for Bank2 \(FLASH_SECWM2R_CUR\)](#) and modified by corresponding FLASH_XXX_PRG registers.

The SECWMx_STRT and SECWMx_END option bytes can be modified only by secure firmware.

Table 62. Secure watermark-based area

Secure watermark option bytes values (x = 1, 2)	Secure watermark protection area
SECWMx_STRT > SECWMx_END	Nonsecure area
SECWMx_STRT = SECWMx_END	One sector defined by SECWMx_STRT is secure watermark-based protected
SECWMx_STRT < SECWMx_END	The area between SECWMx_STRT and SECWMx_END is secure watermark-based protected

Caution: Switching a memory area from secure to nonsecure does not erase its content. The user secure software must perform the needed operation to erase the secure area before switching an area to nonsecure attribute whenever is needed. It is also recommended to flush the instruction cache.

7.6.2 Hide protection (HDP)

The HDP area is independent of the flash watermark-based secure area. One nonvolatile secure hide protection (HDP) area per bank can be defined with a sector granularity. Access to the hide protection area can be denied by progressing the HDPL level in the SBS.

When the HDPL = 1, no user flash is protected by the HDP. With HDPL ≥ 2 the user OB defined HDP area in each bank is closed. No read, write, fetch or erase is allowed in the HDP area.

The HDP area can be extended, the extension setting is only a register value, not stored in OB. With HDPL = 3 the HDP area remains activated and the extension is added. If extension is added while the base user OB defined area is not active, the extension covers one more sector, because HDPx_END sector is also protected.

The HDPL level can be only cleared by a system reset, there is no means to deactivate the HDP area.

The protected HDP area is defined by setting its size using start and end sectors in a similar way as the secure watermark.

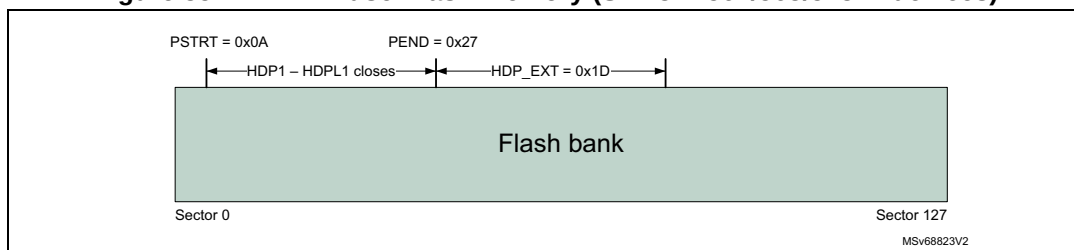
The size of the inaccessible area can be extended using register FLASH_HDPEXTR values HDPx_EXT, representing the number of sectors added to the HDP area (past the HDPx_END sector). The volatile HDPx_EXT value can be increased, but cannot decrement.

If HDPx_STRT > HDPx_END the extension covers (HDPx_EXT + 1) size area between sectors marked by HDPx_END and HDPx_END + HDPx_EXT.

By default HDPx_END is set to 0 and HDPx_STRT is set to 1. This means that the HDP area in user flash has a 0 size, and extends from the flash start address (first sector).

For example, to protect area by HDP from the address 0x0801 4000 (included) to the address 0x0804 FFFF (included):

- For physical Bank1, the option byte registers must be programmed with:
 - HDP1_STRT = 0x0A
 - HDP1_END = 0x27

Figure 35. HDP^(a) in user flash memory (STM32H562/563/573xx devices)

- Once the HDPL is incremented to 2, the marked area becomes inaccessible. The HDPL = 2 code then sets the extension size to 29. Once the HDPL is incremented to 3, even the extension area becomes inaccessible.
- Alternatively, if:
 - HDP1_END = 0, HDP1_START = 1, HDPx_EXT = 1:
Sector 0 and Sector 1 are HDP protected.
 - HDP1_END = 2, HDP1_START = 7, HDPx_EXT = 3 (a rather unusual example):
Sectors 2 to 5 are HDP protected.
- If the two banks are swapped, the protection defined to physical Bank1 remains on the physical Bank1, unaffected by swapping. Separate protection applies to physical Bank2 and the option bytes registers must also be programmed with:
 - HDP2_STRT = 0x0A
 - HDP2_END = 0x27

Note: For more details on the bank swapping mechanism, refer to [Section 7.6.6](#). The values have an upper limit, 127 for STM32H562/63/73xx devices, 31 for STM32H523/33xx devices.

Table 63. Secure hide protection

HDPx watermark option byte values (x = 1, 2)		Hide protection area
HDPL ≤ 1	-	No inaccessible HDP area in user flash memory
HDPL = 2	HDPx_END < HDPx_STRT	No inaccessible HDP area in user flash memory
	HDPx_END = HDPx_STRT	Exactly one sector is protected by HDP
	HDPx_END > HDPx_STRT	The area between HDPx_STRT and HDPx_END is HDP protected.
HDPL = 3	HDPx_END < HDPx_STRT and HDPx_EXT = 0	No inaccessible HDP area in user flash memory
	HDPx_END ≥ HDPx_STRT or HDPx_EXT > 0	The area between min (HDPx_STRT, HDPx_END) and (HDPx_END + HDPx_EXT) is HDP protected

a. The HDP works analogically, the number of sectors is the only differentiator.

Table 64. HDP protections summary

HDP protection	User flash			
	Access to OB HDP area		Access to EXT HDP area	
	HDPL 2 or 3	HDPL1	HDPL3	HDPL1 or HDPL 2
Fetch	BUS ERROR	OK	BUS ERROR	OK
Read	RAZ		RAZ	
Write	WI, WRPERR	If not WRP: OK Else: WI, WRPERR	WI, WRPERR	If not WRP: OK Else: WI, WRPERR
Erase (mass erase)				

7.6.3 Block-based secure flash memory area protection

Any sector can be programmed on-the-fly as secure or nonsecure using the block-based configuration registers. FLASH_SECBB1x/FLASH_SECBB2x registers are used to configure the security attribute for, respectively, sectors in Bank1/Bank2.

When the sector security attribute bit SECBBx[i] is set, the security attribute is the same as the secure watermark-based area. The secure sector is accessible only by a secure access.

If the SECBBx[i] bit is set or reset for a sector already included in a secure watermark-based area, the sector keeps the watermark-based protection security attributes.

To modify a block-based sector security attribute, it is recommended to:

- Check that no flash operation is ongoing on the related sector.
- Add an ISB instruction after modifying the sector security attribute SECBBx[i].

Caution: Switching a sector from secure to nonsecure does not erase the content of the associated sector. User secure software must perform the needed operations before switching to non-secure attribute:

- Erase sector content.
- Invalidate the instruction cache.

Note: For SECBBx[i] access control, refer to [Table 65](#).

Execute protection for S-Bus (AHB register bus) is done in AHB decoder, not in flash memory interface itself.

Table 65. Secure configuration block-based registers access conditions

Access			Corresponding sector privilege status	Sector setting in SECBBxy
Fetch	Secure/ nonsecure	Privilege/ unprivilege	-	BUS ERROR
Read		Privilege/ unprivilege	-	OK
Write	Secure	Privilege	-	OK
		Unprivilege	0	OK
			1	WI
	Nonsecure	Privilege/ unprivilege	-	WI, ILAP

7.6.4 Block-based privileged flash memory area protection

Any sector can be programmed on-the-fly as privileged or unprivileged using the block-based configuration registers. FLASH_PRIVBB1x and FLASH_PRIVBB2x registers are used to configure the privilege attribute for, respectively, sectors in Bank1 and Bank2.

When the sector privilege attribute PRIVBByx[i] bit is set, the sector is accessible only by a privileged access. An unprivileged sector is accessible by a privileged or unprivileged access.

To modify a block-based privilege attribution, it is recommended to:

- Check that no flash operation is ongoing on the related sector.
- Add an ISB instruction after modifying the sector security attribute PRIVBByx[i].

Caution: Switching a sector from privileged to unprivileged does not erase the content of the associated sector.

Table 66. Privilege protection summary

Access (TZ not relevant)		Main flash	
		Unprivileged sector	Privileged sector
Privilege	Fetch	OK	
	Read		
	Write		
	Erase		
Unprivilege	Fetch	OK	RAZ
	Read		WI, WRPERR
	Write		
	Erase		

Table 67. Privilege and mass or bank erase

Access (TZ state not relevant)	Main flash		
	Unprivileged FLASH	Privileged FLASH	Mix unprivileged and privileged FLASH
MEP	OK		
MENP	OK	WI, WRPERR	

Note: For PRIVBByx[i] access control, refer to [Table 68](#) and [Table 69](#).

Table 68. Privilege configuration register access conditions (TZ enabled)

Access			Corresponding sector secure status ⁽¹⁾	Sector setting access in PRIVBBxy
Fetch	Privileged/ unprivileged	Secure/ nonsecure	-	BUS ERROR
Read			-	OK (all sectors)

Table 68. Privilege configuration register access conditions (TZ enabled) (continued)

Access			Corresponding sector secure status ⁽¹⁾	Sector setting access in PRIVBBxy
Write	Privileged	Secure	-	OK (all sectors)
		Nonsecure	NS	OK (only the sector corresponding to the bit)
			S	WI (only the sector corresponding to the bit)
	Unprivileged	Secure/nonsecure	-	WI

1. Either watermark or block-based.

Table 69. Privilege configuration register access conditions (TZ disabled)

Access		Sector setting access in PRIVBBRxy
Fetch	Privileged/unprivileged	BUS ERROR
Read		OK
Write	Privileged	OK
	Unprivileged	WI

7.6.5 Flash memory register privileged and unprivileged modes

The flash registers can be read and written by privileged and unprivileged accesses depending on the SPRIV and NSPRIV bits in [FLASH privilege configuration register \(FLASH_PRIVCFGR\)](#):

- When the SPRIV (respectively NSPRIV) bit is reset, all secure (respectively non-secure) flash registers can be read and written by both privileged or unprivileged access.
- When the SPRIV (respectively NSPRIV) bit is set, all secure (respectively nonsecure) flash registers can be read and written by privileged access only. Unprivileged access to a privileged registers is RAZ/WI.

The registers related to key storage (FLASH_NSOKBKFGR, FLASH_SECOBKCFGR, FLASH_NSOKBKKEYR, FLASH_SECOBKKEYR) are exceptions to this rule and can be only accessed in privilege, regardless of other settings.

The next table summarizes flash registers access control.

Table 70. Flash register accesses⁽¹⁾

Access			Nonsecure register		Secure register	
			NSPRIV = 1	NSPRIV = 0	SPRIV = 1	SPRIV = 0
X	Secure/nonsecure	Privileged/unprivileged	Bus error			
R/W	Secure	Privileged	OK			
R/W		Unprivileged	RAZ, WI	OK	RAZ, WI	OK

Table 70. Flash register accesses⁽¹⁾ (continued)

Access			Nonsecure register		Secure register	
			NSPRIV = 1	NSPRIV = 0	SPRIV = 1	SPRIV = 0
R/W	Nonsecure	Privileged	OK		RAZ, WI, ILAP	
R/W		Unprivileged	RAZ, WI	OK		

1. Some registers are privilege only, not configurable.

7.6.6 Flash memory banks attributes in case of bank swap

The SWAP_BANK option bit modifies the address of each bank in the memory map. When SWAP_BANK is reset, flash Bank1 is mapped at the lower address range. When SWAP_BANK is set, flash Bank1 is mapped at the higher address range. flash bank attributes follow their bank contents so there is no need to modify their setting registers when swapping banks:

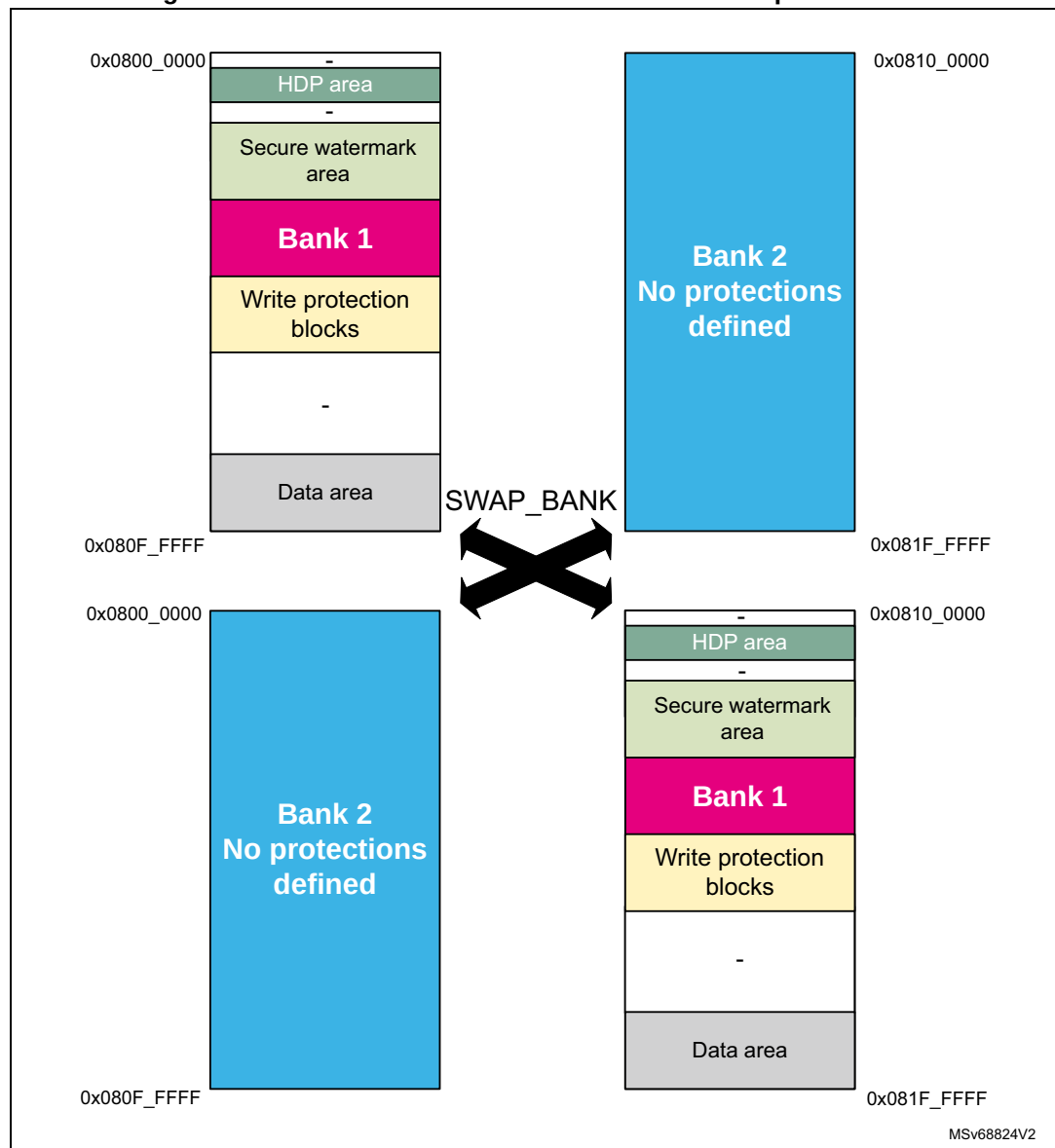
- Flash secure watermark x FLASH_SECWMx
- Flash write protection sector group FLASH_WRPSG (refer to [Section 7.6.8](#))
- Hide protection in FLASH_HDPx registers
- Flash secure block based bank x register y FLASH_SECBBxy
- Flash privilege block based bank x register y FLASH_PRIVBBxy
- The data area configuration FLASH_EDATA

The SWAP_BANK is rendered immutable by setting BOOT_LOCK in the user OB. If TZEN is enabled, then SECBOOT_LOCK locks the option. If TZEN is disabled, then NSBOOT_LOCK locks the SWAP_BANK option.

Note: The BK_ECC bit in the [FLASH ECC detection register \(FLASH_ECCDETR\)](#) and [FLASH ECC correction register \(FLASH_ECCCORR\)](#), BKSEL bit in [FLASH nonsecure control register \(FLASH_NSCR\)](#) and BKSEL bit in [FLASH secure control register \(FLASH_SECCR\)](#) always refers to Bank1 (respectively Bank2) when it is low (respectively high), regardless of the SWAP_BANK value.

Figure 36 shows how security attributes and protections behave in case of bank swap.

Figure 36. Protection attributes in case of bank swap illustration



7.6.7 Flash memory configuration protection

The memory uses hardware mechanisms to protect the following assets against unwanted or spurious modifications (such as software bugs):

- Option bytes change
- Write operations
- Erase commands
- Interrupt masking

The memory configuration registers protection is summarized in [Table 71](#). Registers not present in this table are not protected by a key.

Table 71. Flash interface register protection summary

Register name	Unlocking register	Protected asset
FLASH_NSCR	FLASH_NSKEYR	Write/erase control
FLASH_SECCR	FLASH_SECKEYR	Secure write/erase control
FLASH_OPTCR + all _PRG registers	FLASH_OPTKEYR	Flash bank option byte words change
FLASH_NSObKCFGR	FLASH_NSObKKEYR	Flash OBKey storage manipulation
FLASH_SECOBKCFGR	FLASH_SECOBKKEYR	

7.6.8 Write protection

The purpose of the write protection is to prevent unwanted modifications to code and/or data.

Any group of four consecutive 8-Kbyte sectors can be independently write-protected or unprotected by clearing/setting the corresponding WRPSGn1/2 bit in the FLASH_WRPSG1/2 register (see [FLASH write sector group protection for Bank1 \(FLASH_WRP1R_CUR\)](#) and [FLASH write sector group protection for Bank2 \(FLASH_WRP2R_CUR\)](#)).

A write-protected group of sectors cannot be erased nor programmed. As a result, a bank erase cannot be performed if one group of sectors is write-protected, unless a PRODUCT_STATE transition to OPEN is triggered (erasing the whole user flash memory).

Note: Write protection errors are documented in [Section 7.9](#).

7.6.9 Flash high-cycle data protections

The memory can be configured to have 96-Kbyte memory area with high-cycling capability (100 kcycles) to store data and emulate EEPROM, see [Section 7.3.10](#).

Option byte controlled security features for user flash memory prevent sectors from being used as high-cycle data area. No OPTCHANGEERR is raised in case of setting conflict, it only makes the sectors unusable. Details are listed in [Table 72](#).

Volatile settings using FLASH_SECBBxx and FLASH_PRIVBBxx can be applied. In this case the security setting of user flash sector 127 applies to flash high-cycle data sector 0 and so on, as indicated in [Figure 27](#), [Figure 28](#), and [Figure 31](#).

Volatile settings of HDP extension that extends into sectors of data area cause the data area to be inaccessible.

Table 72. High-cycle area protection summary: access to data area address range

TZ protection (when TZ_STATE = 0xB4) ⁽¹⁾		Data sector enabled						Data sector not enabled	
		HDP protected	OB secure area	SecBB protected		NS sector			
				16/32 bits access	Other	16/32 bits access	Other		
Secure	Fetch	BUS ERROR	BUS ERROR						BUS ERROR
	Read		OK	BUS ERROR	RAZ, ILAC				
	Write		If not WRP: OK else WI, WRPERR		WI, WRPERR, ILAFM				
	Erase	Same protection as in corresponding classic user flash sector							
Nonsecure	Fetch	BUS ERROR	BUS ERROR						
	Read		RAZ, ILAC		OK	BUS ERROR			
	Write		WI, WRPERR, ILAFM		If not WRP: OK else WI, WRPERR				
	Erase	Same protection as in corresponding classic user flash sector							

1. If TZ_STATE = 0xC3, there is no secure state access, no SecBB, and no Secure Area. For HDP and NS access the rules remain the same.

Table 73. HDP protected definition

User flash	OB HDP area enabled	HDP extension enabled
HDPL1	HDP protected = 0	HDP protected = 0
HDPL2	HDP protected = 1	HDP protected = 0
HDPL3	HDP protected = 1	HDP protected = 1

Table 74. Privileged sectors and data area - Access to data area address range

Access		Data area address range access	
		Unprivileged sector	Privileged sector
Privileged	Fetch	BUS ERROR	
	Read	OK	OK
	Write	OK	OK
Unprivileged	Fetch	BUS ERROR	
	Read	OK	RAZ, ILAC
	Write	OK	WI, WRPERR

7.6.10 Life cycle management

Nonvolatile or debug states are determined by the product state set by user option bytes. Debug possibility and possibility to change selected security settings are related to it. While the state is stored in the OB, it is the SBS that controls the debug policy.

OBK storage access conditions are unaffected by the PRODUCT_STATE, but keys can be evicted in regression transitions.

HDP works regardless of the debug state. Even in debug the HDP area is hidden when HDPL increments.

The following states are defined:

Table 75. Product states, debug states and debug policy

PRODUCT_STATE	Code in OB	Description
Open	0xED	User flash open (TZ secure and nonsecure open). External access (debug) N and NS enabled (~RDP0).
Provisioning	0x17	Provisioning - Immutable root of trust is being installed. Open if TZEN = 0xC3. The debug is opened in HDPL3-NS.
iRoT-Provisioned	0x2E	The immutable root of trust is installed (~RDP 0.5).
TZ-Closed	0xC6	State for debugging the NS application. Debug is restrained to nonsecure areas.
Closed	0x72	State for running a secure application. Debug disabled, regression is possible.
Locked	0x5C	Transition to other state, policy change or debug not permitted ⁽¹⁾ .
Regression	0x9A	The temporary state initiated by the debug authentication system in transition to Open.
NS-Regression	0xA3	The temporary state initiated by the debug authentication system in transition to TZ-Closed.

1. The same situation is Closed state with incorrect or not defined debug certificates.

1. No debug protection

Read, program and erase operations into the flash main memory area are possible. Least restriction on the option bytes.

2. Nonsecure debug only

All read and write operations (if no write protection is set) from/to the nonsecure flash memory are possible. The debug access to secure area is prohibited. Debug access to nonsecure area remains possible.

The following rules are enforced:

- User mode: code executing in user mode (boot flash) can access flash main memory and option bytes with all operations (read, erase, program).
- Nonsecure debug mode: The secure flash memory areas are inaccessible; the nonsecure flash memory areas remain accessible for debug purpose.
- Boot RAM mode: boot from SRAM is not possible.
- Transition to Open state is possible with the consequence of flash mass erase and key slots revocation. Depends on debug unlock policy (see below).

3. Full product protection

All debug features are disabled and following rules enforced:

- User mode: code executing in user mode (boot flash) can access flash main memory and selected option bytes with all operations (read, erase, program).
- Boot RAM mode: boot from SRAM is not possible.
- Transition to TZ-Closed or Open state is possible with the consequence of flash mass erase and key slots revocation. Depends on debug unlock policy (see below).

7.6.11 Product state transitions

Progressing towards a more closed state is the normal product life cycle, that does not require security measures. Transition towards Open state is a regression, controlled by the debug authentication control. If debug unlock policy is set to "locked", no regression is accepted. Locked state can be reinforced by invalidating the debug authentication certificates.

All transitions not listed in this chapter are invalid.

There is no restriction on reading the current product state.

Transitions in progress direction

Transition from Open to any of the closed states is matter of correct product configuration and provisioning. The transitions must be done in correct order.

The normal progression is:

- Open to Provisioning or iRoT-Provisioned
- Provisioning to iRoT-Provisioned
- Provisioning or iRoT-Provisioned to TZ-Closed, Locked or Closed
- TZ-Closed to Locked or Closed

Transition is managed either by a software running on the device, or directly, using a debug interface. Transitions from Closed are only possible by software running on the device. By software it is assumed that transitions are triggered by bootloader, or root-of-trust services, but generally any software running on the device can do a progress transition.

Transition to Open state

This transition is a full regression. The starting state is any except Open and Locked. The debug tools are used to authenticate debug regression access rights with the debug authentication library, running on the device in HDPL1 (refer to [Section 14.3.6: SBS debug control](#)). After verifying the credentials, it puts the device into intermediate state Regression. From this state the device regresses securely in HDPL0 to Open.

The transition has the following consequence:

- Epoch secure counter is incremented
- Key slots revocation
- Product state updated
- User options updated (security settings reset, TZEN reset)
- Flash memory mass erase
- Backup RAM erase is requested

Transition from closed to closed-secure debug state

This transition opens the device to authorize the debug of the nonsecure application software without compromising the security of the ROT functions.

The starting state is Closed. The debug tools are used to authenticate debug regression access rights with the debug authentication library, running on the device in the HDPL1 (refer to [Section 14.3.6: SBS debug control](#)). After verifying the credentials, it puts the device into intermediate state NS-Regression. From this state, the device regresses securely in HDPL0 to Closed.

The transition has the following consequence:

- Epoch nonsecure counter is incremented
- Nonsecure slots revocation
- Product state updated
- Flash memory sectors not covered by the secure watermark are erased
- Backup RAM erase is requested

Table 76. PRODUCT_STATE transitions

From	To ⁽¹⁾							
	Open	Provisioning	iRoT-Provisioned	TZ-Closed ⁽²⁾	Closed	Locked	Regression	NS-Regression ⁽²⁾
Open	-	OK	OK	X	X	X	X	X
Provisioning	X	-	OK	OK	OK	OK	OK (HDPL1)	X
iRoT-Provisioned	X	X	-	OK	OK	OK	OK (HDPL1)	X
TZ-Closed	X	X	X	-	OK	OK	OK (HDPL1)	X
Closed	X	X	X	X	-	X	OK (HDPL1)	OK (HDPL1)
Locked	X	X	X	X	X	-	X	X

Table 76. PRODUCT_STATE transitions (continued)

From	To ⁽¹⁾							
	Open	Provisioning	iRoT-Provisioned	TZ-Closed ⁽²⁾	Closed	Locked	Regression	NS-Regression ⁽²⁾
Regression	OK (HDPL0)	X	X	X	X	X	-	X
NS-Regression	X	X	X	OK (HDPL0)	X	X	X	-

1. X = transitions not permitted (OPTCHANGEERR raised), - = no change, OK = valid transitions.
2. To transition into this state, both the current and programmed values of TZEN must be enabled (TZEN = 0xB4), otherwise the change is ignored with OPTCHANGEERR.

In [Table 76](#), X indicates transitions that are not permitted, a dash indicates no change, OK indicates valid transitions. Some transitions are possible only in correct HDPL provided in parenthesis. Attempts to do an illegal transition result in OPTCHANGEERR.

Incorrect PRODUCT_STATE (no OBL) is interpreted as Locked.

7.6.12 OBK protection

At the receipt of an OBK access (read/write/execute), secure and privilege attributes are first checked, followed by OBK-HDPL. The OBK-HDPL value must exactly match the HDPL assigned to the key location, otherwise OBKERROR flag is raised. Flash interface response is described in [Table 77](#).

Table 77. TZ OBK protection summary

TZ protection (when TZ_STATE = 0xB4)	OBK access		
	No tamper		Tamper
	HDPL0/1/2/3	OBK selector	
XS	BUS ERROR	BUS ERROR	RAZ, WI
RS	OK if privilege, otherwise RAZ	RAZ	
WS	OK if privilege, otherwise WI, WRPERR	WI, if not privilege also WRPERR	
XNS	BUS ERROR	BUS ERROR	
RNS	RAZ, ILAFM	RAZ, ILAFM	
WNS	WI, WRPERR, ILAFM	WI, WRPERR, ILAFM	
Erase	NA	NA	NA

Table 78. OBK protection summary with TZ disabled

TZ protection (when TZ_STATE = 0xC3)	OBK access		
	No tamper		Tamper
	HDPL1/2/3	OBK selector	
XNS	BUS ERROR	BUS ERROR	RAZ, WI
RNS	OK if privileged, otherwise RAZ	RAZ	
WNS	OK if privilege, otherwise WI, WRPERR	WI if privileged, otherwise WI, WRPERR	
Erase	NA	NA	NA

Note: Cacheable access is managed at system level and not in the flash interface. The MPU must be configured to disable cache where it is not desirable.

In case of tamper, all keys are read as 0x00s.

There are also rules for accessing the OBKFGCR and OBKKEYR registers.

Table 79. Access conditions to secure control register

-	S/NS	P/NP	TZ state	SECOBKCFGR and SECOBKKEYR access
X	-	-	-	Bus error
R/W	S	P	Active	OK
R/W	S	NP	Active	RAZ, WI
R/W	NS	P	Active	RAZ, WI, ILAP
R/W	NS	NP	Active	RAZ, WI, ILAP
R/W	-	-	Deactivated	RAZ, WI

Table 80. Access conditions to nonsecure control register

	S/NS	P/NP	TZ state	NSOBKCFGR and NSOBKKEYR access
X	-	-	-	Bus error
R/W	-	-	Active	RAZ, WI
R/W	-	P	Deactivated	OK
R/W	-	NP	Deactivated	RAZ, WI

7.6.13 One-time-programmable and read-only memory protections

Sections of OTP/RO in the flash memory are described in details in [Section 7.3.9](#). There is no protection provided by the flash memory interface other than the dedicated write protection.

No write is possible to the RO area, once it was established in manufacturing. The [OTP write protection](#) describes a method of locking out the OTP area.

The access conditions are summarized in [Table 81](#).

Table 81. OTP/RO access constraints

-	8 Kbytes sector dedicated to RO/OTP			
	0x08FF_E000 - E7FF	0x08FF_E800 - EFFF Reserved	0x08FF_F000 - F7FF OTP	0x08FF_F800 - FFFF RO
XS	BUS ERROR	BUS ERROR		
RS		RAZ, ILAC		
WS		WI, WRPERR, ILAFM		
XNS	BUS ERROR	BUS ERROR		
RNS		If correct size ⁽¹⁾ RAZ, else BUS ERROR	If correct size OK, else BUS ERROR	
WNS		If correct size WI, else BUS ERROR	BUS ERROR if incorrect size. WRPERR if block is protected by LOCKBL, else OK.	If correct size WI, else BUS ERROR
Erase	WI			

1. Word size is 16 bits. 32-bit access is possible. Other access attempt leads to bus error.

7.7 System memory

7.7.1 Introduction

System memory stores RSS (root secure services) firmware programmed by ST during production. The RSS provides runtime services to user firmware.

When TrustZone is enabled (user sets TZEN bitfield to 0xB4) then RSS provides secure services to secure user firmware only; when TrustZone is disabled (user sets TZEN bitfield to 0xC3), RSS provides services to user firmware.

7.7.2 RSS user functions

The RSS provides runtime services thanks to RSS library, whose functions are exposed to user within the CMSIS device header file provided by the STM32CubeH5 firmware package (see UM3065 “*Getting started with STM32CubeH5 for STM32H5 series*” for more details).

RSS provides services through two different libraries, RSSLIB and NSSLIB.

Table 82. RSSLIB/NSSLIB accesses

TZEN configuration	User FW Nonsecure	User FW Secure	Bootloader/JTAG/SWD ⁽¹⁾
Enabled (0xB4)	N/A	RSSLIB	RSSLIB
Disabled (0xC3)	NSSLIB	N/A	

1. When the devices boot in bootloader, that is, in product state OPEN (with BOOT pin set to 1 and UBE set to 0xB4), or PROVISIONING.

For portability, security, and safety reasons, avoid calling library functions directly in your code. Instead, always use the official ST header files and libraries.

Table 83. RSSLIB/NSSLIB entry point access

C defined macro	Location in flash memory
RSSLIB_PFUNC	0xBF9FB68
NSSLIB_PFUNC	0xBF9FB6C

RSSLIB

The secure user firmware calls RSSLIB functions using RSSLIB_PFUNC C defined macro, which points to a location within nonsecure system memory. Before calling RSSLIB functions, the secure user firmware must define a nonsecure region above this location within SAU of the Cortex[®]-M33, starting from RSSLIB_SYS_FLASH_NS_PFUNC_START (0xBF9FB78), up to RSSLIB_SYS_FLASH_NS_PFUNC_END (0xBF9FB84). These last addresses are provided within the CMSIS device header file.

The user can set this nonsecure region either by using the CMSIS system partition header file, or by implementing its own code for SAU setup. The CMSIS system partition header file is part of the STM32CubeH5 firmware package.

Table 84. RSS lib interface functions

Library	Location in flash memory	Function	Function address offset	Attributes
RSSLIB_PFUNC	0xBF9FB68	JumpHDPLv2	0x10	Secure callable function
		JumpHDPLv3	0x14	
		JumpHDPLv3NS	0x18	
		DataProvisioning	0x00	Nonsecure callable function

DataProvisioning

Secure attribute: Nonsecure callable function, no substitute is available if TrustZone is disabled. In such case, OBK has to be managed by using register access.

Prototype:

```
uint32_t DataProvisioning (RSSLIB_DataProvisioningConf_t *pConfig)
```

User code function call example:

```
RSSLIB_PFUNC→NSC.DataProvisioning(&pConfig)
```

Arguments:

pConfig: input parameter. **RSSLIB_DataProvisioningConf_t**

C structure definition is described below:

```
typedef struct
{
    uint32_t *pSource;
    uint32_t *pDestination;
    uint32_t Size;
    uint32_t DoEncryption;
    uint32_t Crc;
```

```
} RSSLIB_DataProvisioningConf_t;
```

Structure elements

pSource	Provides the address of data to be provisioned. Must be within SRAM3 address range (nonsecure aliases).
pDestination	Provides the address where to store data to be provisioned. Must be within OBKeys address range. To be aligned on 16 bytes.
Size	Provides the size of data to be provisioned (the number of bytes, must be a multiple of 16).
DoEncryption	Notifies RSSLIB_DataProvisioning function if it must encrypt or not data within OBKeys. DoEncrypt can be either 0xF5F5A0AAU or 0xCACA0AA0U (the only one allowed on STM32H563xx devices) – 0xF5F5A0AAU: notifies RSSLIB_DataProvisioning to encrypt data with relevant DHUK before programming it within OBKeys. DHUK is selected according to pDestination value. – 0xCACA0AA0U: notifies RSSLIB_DataProvisioning to program in clear data within OBKeys.
Crc	CRC over full source data buffer, pConfig->pDestination value, pConfig->Size value and finally pConfig->DoEncryption value. CRC computation uses CRC-32 (Ethernet): – CRC polynomial: 0x04C11DB7U – Initial value: 0xFFFFFFFFU

Returned values

0xEAEAEAEU	Success
0xF5F5E0EU	Error: – pConfig or pSource are not in SRAM3 – pDestination is not aligned on 16 bytes – pDestination + Size does not fit within an OBKey section
0xF5F50E0EU	Error: Size is not a multiple of 16
0xF5F58080U	Error: computed CRC is not the expected one provided within pConfig->CRC
0xF5F58008U	Error: cannot program data within OBKeys destination section
0xF5F5E00EU	Error: wrong DoEncryption parameter value
0xF5F50880U	Error: encryption requested, butw platform does not support it
0xF5F50EE0U	Error: encryption error
0xF5F50808U.	Error: OBKeys programming error

RSSLIB_DataProvisioning receives in input a data buffer and programs it within OBKeys. A CRC prevents any data and parameters tampering issue.

The destination address (**pDestination**) added to the size (**Size**) of the data to be provisioned must not cross the OBKeys section boundaries, listed below.

Table 85. OBKeys section boundaries

OBKeys Level	Address
OBKeys Level1	Start address: 0x0FFD0100UL
	End address: 0x0FFD08FFUL
OBKeys Level2	Start address: 0x0FFD0900UL
	End address: 0x0FFD0BFFUL
OBKeys Level3 secure	Start address: 0x0FFD0C00UL
	End address: 0x0FFD17FFUL
OBKeys Level3 nonsecure	Start address: 0x0FFD1800UL
	End address: 0x0FFD1FEFUL

When requested through **pConfig->DoEncryption** parameter, **RSSLIB_DataProvisioning** encrypts data before programming them within OBKeys.

RSSLIB_DataProvisioning uses AES CBC 128 bits with:

- IV: defined as (using C definition format)
`uint32_t IV = {0x8001D1CEU, 0xD1CED1CEU, 0xD1CE8001U, 0xCED1CED1U};`
- Key: DHUK corresponding to the targeted OBKeys section defined by **pConfig->pDestination** parameter.

Note: Data encryption within OBkeys is supported only by STM32H533/573xx devices.

JumpHDPLvl2

Secure attribute: Secure callable function.

Prototype:

```
uint32_t JumpHDPLvl2(uint32_t VectorTableAddr, uint32_t MPUIndex)
```

User code function call example:

```
RSSLIB_PFUNC->S.JumpHDPLvl2((uint32_t)NextVectorTableAddr, 1U );
```

Arguments:

- VectorTableAddr:
 - Input parameter, address of the next vector table to apply.
 - The vector table format is the one used by the Cortex-M33 core.
- MPUIndex:
 - Input parameter, MPU region index. Caller function must define (but keep disabled) the corresponding MPU region before calling **JumpHDPLvl2**. The function enables the MPU region before jumping to the reset handler of the vector table. The vector table reset handler function belongs to the MPU region.

User calls `JumpHDPLv2` to close user Flash HDPL1 area by incrementing HDPL to 2, then jump to the reset handler embedded within the vector table, whose address is passed as input parameter.

After closing HDPL1, `JumpHDPLv2` enables the MPU region provided as input parameter. Once the MPU is enabled, the function sets the SP to the address provided by the passed vector table, and jumps to the reset handler function supported by it. `JumpHDPLv2` does not set the new vector table.

On successful execution, the function does not return and does not push LR onto the stack.

In case of failure (bad input parameter value), `RSSLIB_Sec_JumpHDPLv2` returns `0xF5F5F5F5`.

JumpHDPLv3

Secure attribute: Secure callable function.

Prototype:

```
uint32_t JumpHDPLv3(uint32_t VectorTableAddr, uint32_t MPUIndex)
```

User code function call example:

```
RSSLIB_PFUNC->S.JumpHDPLv3((uint32_t)NextVectorTableAddr, 1U );
```

Arguments:

- `VectorTableAddr`:
 - Input parameter, address of the next vector table to apply.
 - The vector table format is the one used by the Cortex-M33 core.
- `MPUIndex`:
 - Input parameter, MPU region index. Caller function must define but keep disabled the corresponding MPU region before calling `JumpHDPLv3`. The function enables the MPU region before jumping to the reset handler of the vector table, whose function belongs to the MPU region.

User calls `JumpHDPLv3` to close user Flash HDPL1 and HDPL2 areas by incrementing HDPL up to 3, then jump to the reset handler embedded within the vector table, whose address is passed as input parameter.

After closing HDPL1/2, `JumpHDPLv3` enables the MPU region provided as input parameter. Once the MPU is enabled, the function sets the SP to the address provided by the passed vector table, and jumps to the reset handler function supported by the vector table. `JumpHDPLv3` does not set the new vector table.

On successful execution, the function does not return and does not push LR onto the stack.

In case of failure (bad input parameter value), `JumpHDPLv3` returns `0xF5F5F5F5`.

JumpHDPLv3NS

Secure attribute: Secure callable function.

Prototype:

```
uint32_t JumpHDPLv3NS(uint32_t VectorTableAddr)
```

User code function call example:

```
RSSLIB_PFUNC->S.JumpHDPLv3NS((uint32_t)NextVectorTableAddr, 1U );
```

Arguments:

- VectorTableAddr:
 - Input parameter, address of the next vector table to apply.
 - The vector table format is the one used by the Cortex-M33 core.

User calls JumpHDPLv3NS to close user flash HDPL1 and HDPL2 areas by incrementing HDPL up to 3, to move from secure to nonsecure domain, then jump to the nonsecure reset handler embedded within the vector table, whose address is passed as input parameter.

After closing HDPL1/2, JumpHDPLv3 jumps to the nonsecure reset handler function supported by the vector table. JumpHDPLv3NS does not set the new vector table.

On successful execution, the function does not return and does not push LR onto the stack.

In case of failure (bad input parameter value), JumpHDPLv3NS returns 0xF5F5F5F5.

NSSLIB

The user firmware calls only the NSSLIB function when TrustZone is disabled (TZEN bitfield is set to 0xC3).

The user firmware calls NSSLIB functions using NSSLIB_PFUNC C defined macro, which points to a location within system memory.

Table 86. NSS lib interface functions

Library	Location in flash memory	Function	Function address offset	Attributes
NSSLIB_PFUNC	0xBF9FB6C	JumpHDPLv2	0x00	Nonsecure function
		JumpHDPLv3	0x04	

JumpHDPLv2

Prototype:

```
uint32_t JumpHDPLv2(uint32_t VectorTableAddr, uint32_t MPUIndex)
```

User code function call example:

```
NSSLIB_PFUNC->JumpHDPLv2((uint32_t)NextVectorTableAddr, 1U );
```

Arguments:

- VectorTableAddr:
 - Input parameter, address of the next vector table to apply.
 - The vector table format is the one used by the Cortex-M33 core.
- MPUIndex:
 - Input parameter, MPU region index. Caller function shall define but keep disable the corresponding MPU region before calling JumpHDPLv2. The function enables the MPU region before jumping to the reset handler of the vector table. The vector table reset handler function belongs to the MPU region.

User calls JumpHDPLv2 to close user Flash HDPL1 area by incrementing HDPL to 2, and to jump to the reset handler embedded within the vector table which address is passed as input parameter

After closing HDPL1, JumpHDPLv2 enables the MPU region provided as input parameter. Once the MPU is enabled, the function sets the SP to the address provided by the passed vector table and jumps to the reset handler function supported by the vector table too. JumpHDPLv2 does not set the new vector table.

On successful execution, the function does not return and does not push LR onto the stack.

In case of failure (bad input parameter value), `RSSLIB_Sec_JumpHDPLv2` returns `0xF5F5F5F5`.

JumpHDPLv3

Prototype:

```
uint32_t JumpHDPLv3(uint32_t VectorTableAddr, uint32_t MPUIndex)
```

User code function call example:

```
NSSLIB_PFUNC->JumpHDPLv3((uint32_t)NextVectorTableAddr, 1U );
```

Arguments:

- VectorTableAddr:
 - Input parameter, address of the next vector table to apply.
 - The vector table format is the one used by the Cortex-M33 core.
- MPUIndex:
 - Input parameter, MPU region index. Caller function shall define but keep disable the corresponding MPU region before calling JumpHDPLv3. The function enables the MPU region before jumping to the reset handler of the vector table. The vector table reset handler function belongs to the MPU region.

User calls JumpHDPLv3 to close user Flash HDPL1 and HDPL2 areas by incrementing HDPL up to 3, and then jump to the reset handler embedded within the vector table, whose address is passed as input parameter

After closing HDPL1/2, JumpHDPLv3 enables the MPU region provided as input parameter. Once the MPU is enabled, the function sets the SP to the address provided by the passed vector table, and jumps to the reset handler function supported by it. JumpHDPLv3 does not set the new vector table.

On successful execution, the function does not return and does not push LR onto the stack.
In case of failure (bad input parameter value), JumpHDPLv13 returns 0xF5F5F5F5.

7.8 FLASH low-power modes

[Table 87](#) summarizes the memory behavior in STM32 low-power modes. Embedded flash memory belongs to the core domain.

Table 87. Effect of low-power modes on the embedded flash memory

Power mode	Core domain voltage range	Allowed if FLASH busy	FLASH power mode
Run	VOS0/1/2/3	Yes	Run
Stop1 (clock stopped)	SVOS3/4/5	No	Clock gated or stopped in case of SVOS5
Standby	Off	No	Off

When the system state changes or within a given system state, the memory can operate with a different voltage supply range (VOS), according to the application. The procedure to switch the memory into power modes (run, clock gated, stopped, off) is described hereafter.

Note: For more information on the microcontroller power states, refer to [Section 10: Power control \(PWR\)](#).

Managing the FLASH domain switching to Stop or Standby

As explained in [Table 87](#), if the memory informs the reset and clock controller (RCC) that it is busy (BSY, DBNE, WBNE is set), the microcontroller cannot switch the core domain to Stop or Standby mode.

There are two ways to release the memory:

- Reset the WBNE busy flag in FLASH_NS/SECSR register by any of the following actions:
 - Complete the write buffer with missing data.
 - Force the write operation without filling the missing data by activating the FW bit in FLASH_NS/SECCR register. This forces all missing data “high”.
- Poll BSY busy bits in FLASH_NS/SECSR register until they are cleared. This indicates that all recorded write, erase and option change operations are complete.

The microcontroller can then switch the domain to Stop or Standby mode.

7.9 FLASH error management

7.9.1 Introduction

The memory automatically reports when an error occurs during a read, program or erase operation. A wide range of errors are reported:

- *Nonsecure write protection error (WRPERR)*
- *Secure write protection error (WRPERR)*
- *Non secure programming sequence error (PGSERR)*
- *Secure programming sequence error (PGSERR)*
- *Secure strobe error (STRBERR)*
- *Error correction code error (ECCC, ECCD)*
- *Illegal access (ILAFM/ILAP)*
- *Option byte change error (OPTCHANGEERR)*
- OBK nonsecure general error (OBKERR)
- OBK secure general error (OBKERR)
- OBK nonsecure write error (OBKWERR)
- OBK secure write error (OBKWERR)

The application software can individually enable the interrupt for each error, as detailed in [Section 7.10](#).

The flash memory interface uses different interrupt lines to trigger the event. There are two direct lines to NVIC, one for secure interrupts, other for non secure. Third line leads to TZIC, for the ILAFM interrupt.

Note: For all errors, the application software must clear the error flag before attempting a new modify operation.

Since there is just one write buffer and only one operation is allowed at a time, the write control bits and status bits are shared by both banks:

- Write control bits (PG, FW, EOPIE, WRPERRIE, PGSERRIE, STRBERRIE, INCERRIE, CLR_EOP, CLR_WRPERR, CLR_STRBERR, CLR_INCERR, CLR_PGSERR) controls Bank1 and 2 at the same time.
- Write status flag reports (WBNE, DBNE, EOP, WRPERR, PGSERR, STRBERR, INCERR, BSY) error for Bank1 and 2 at the same time.

7.9.2 Nonsecure write protection error (WRPERR)

When an illegal nonsecure erase/program operation is attempted to the nonvolatile memory, the flash interface sets the write protection error flag WRPERR in FLASH_NSSR register.

An erase operation is rejected and flagged as illegal if it targets one of the following memory areas:

- A sector write-protected with WRPSGn
- An HDP area while the HDPL has made it inaccessible
- Secure (TZ) state not matching the erased memory attribute
- Attempt to erase privilege sector within unprivileged mode

A program operation is ignored and flagged as illegal if it targets one of the following memory areas:

- System flash memory
- A user sector write-protected with WRPSGn
- An OTP block, locked with LOCKBL
- A read-only section
- A reserved area
- Secure area
- Privileged area from within unprivileged mode
- OBK access conditions check error

When WRPERR flag is raised, the operation is rejected and nothing is changed in the corresponding bank. If this error is detected, the write buffer is invalidated.

Note: WRPERR flag must be cleared before any erase/program operation.

WRPERR flag is cleared by setting CLR_WRPERR bit to 1 in FLASH_NSCCR register.

If WRPERRIE bit in FLASH_NSCR register is set to 1, an interrupt is generated when WRPERR flag is raised (see [Section 7.10](#) for details).

7.9.3 Secure write protection error (WRPERR)

When an illegal secure erase/program operation is attempted to the nonvolatile memory, the flash interface sets the write protection error flag WRPERR in FLASH_SECSR register.

An erase operation is rejected and flagged as illegal if it targets one of the following memory areas:

- A sector write-protected with WRPSGn
- An HDP area while the HDPL has made it inaccessible
- Secure (TZ) state not matching the erased memory attribute
- Attempt to erase privilege sector within unprivileged mode

A program operation is ignored and flagged as illegal if it targets one of the following memory areas:

- System flash memory
- A user sector write-protected with WRPSGn
- An OTP block, locked with LOCKBL
- A read-only section
- A reserved area
- Nonsecure area
- Privileged area from within unprivileged mode
- OBK access condition check error

When WRPERR flag is raised, the operation is rejected and nothing is changed in the corresponding bank.

If this error is detected the write buffer is invalidated.

Note: WRPERR flag must be cleared before any erase/program operation.

WRPERR flag is cleared by setting CLR_WRPERR bit to 1 in FLASH_SECCCR register.

If WRPERRIE bit in FLASH_SECCR register is set to 1, an interrupt is generated when WRPERR flag is raised (see [Section 7.10](#) for details).

7.9.4 Non secure programming sequence error (PGSERR)

When the nonsecure programming sequence is incorrect, the flash interface sets the programming sequence error flag PGSERR in FLASH_NSSR register.

More specifically, PGSERR flag is set when one of below conditions is met:

- An error like INCERR, WRPERR, PGSERR, STRBERR, OPTCHANGEERR, OBKERR or OBKWERR have not been cleared before requesting a new write or erase operation, OBK operation or options change.
- For erase, PGSERR is set if:
 - missing erase operation (FLASH_NSCR): STRT = 1 with (MER = 0, SER = 0 and BER = 0)
 - sector and bank erase requested at the same time (NSSTRT and more than one of MER, SER and BER)
 - PG set during erase operation (it ensures that no erase req and write req happen at the same time): (STRT = 1) with (PG = 1)
 - Erase operation is started while write buffer is waiting for next data: (STRT = 1) with WBNE = 1.
 - STRT = 1 set by secure access.
 - Erase operation is started while DBNE = 1.
- For programming, PGSERR is set if:
 - missing write flag: A write operation is requested but the program enable bit PG has not been set in FLASH_NSCR register prior to the request.
 - write operation to flash high-cycle data is requested but PG = 0.
 - AHB write request is received and (SER = 1, BER = 1 or MER = 1)
 - 16 bit data access requested while WBNE = 1
- For options change, PGSERR is set if:
 - option change is started while write buffer is waiting for next data: OPTSTRT = 1 with WBNE = 1
 - OPTSTRT set with nonsecure DBNE = 1.
- For options byte key storage access, PGSERR is set if:
 - error flags from previous operation not cleared
 - ALT_SECT_ERASE and SWAP_SECT_REQ in FLASH_NSObKCFGR are set at the same time
 - SWAP_OFFSET value is wrong: when SWAP_SECT_REQ is set to 1, the SWAP_OFFSET should be equal or greater than $OBK_HDPL < N - 1 > _SWAP_OFFSET$
 - ALT_SECT_ERASE or OBK_SWAP set while DBNE = 1
 - OBK-HDPL is incorrect and nonsecure OBK_SWAP is requested
 - If OBKSWAP or OBK alternate erase is started while write buffer is waiting for new data (WBNE = 1).

When PGSERR flag is raised as consequence of failed write attempt (flash programming), the current program operation is aborted and nothing is changed in the corresponding bank. The write data buffer is also invalidated.

Note: When PGSEERR flag is raised, there is a risk that the last write operation performed by the application has been lost because of the above protection mechanism. It is recommended to generate interrupts on PGSEERR and verify in the interrupt handler if the last write operation has been successful, by reading back the value in the flash memory.

The PGSEERR flag also blocks any new program operation. This means that PGSEERR must be cleared before starting a new program operation.

PGSEERR flag is cleared by setting CLR_PGSEERR bit to 1 in FLASH_NSCCR register.

If PGSEERRIE bit in FLASH_NSCR register is set to 1, an interrupt is generated when PGSEERR flag is raised. See [Section 7.10](#) for details.

7.9.5 Secure programming sequence error (PGSEERR)

When the secure programming sequence is incorrect, the flash interface sets the programming sequence error flag PGSEERR in FLASH_SECSR register.

More specifically, PGSEERR flag is set when one of below conditions is met:

- An error like INCERR, WRPERR, PGSEERR, STRBERR, OBKERR, or OBKWERR has not been cleared before requesting a new write or erase operation, OBK operation or options change.
- For erase, PGSEERR is set if:
 - missing erase operation (FLASH_SECCR): STRT = 1 with (SER = 0 and BER = 0, MER = 0)
 - sector and bank erase requested at the same time (SECSTRT and more than one of MER, SER, and BER)
 - PG set during erase operation (it ensures that no erase request and write request happen at the same time): (STRT = 1) with (PG = 1)
 - erase operation is started while write buffer is waiting for next data: (STRT = 1) with WBNE = 1.
 - operation started while secure DBNE = 1.
- For programming, PGSEERR is set if:
 - missing write flag: a write operation is requested but the program enable bit PG has not been set in FLASH_SECCR register prior to the request
 - write operation to flash high-cycle data is requested but PG = 0
 - AHB write request is received and (SER = 1, SBER = 1, or MER = 1)
 - 16 bit data access requested while WBNE = 1
- For options change, PGSEERR is set if:
 - Option change is started while write buffer is waiting for next data: OPTSTRT = 1 with WBNE = 1
 - OPTSTRT set with secure DBNE = 1
- For options byte key storage access, SECPGSEERR is set if:
 - error flags from previous operation not cleared
 - ALT_SECT_ERASE and SWAP_SECT_REQ are set at the same time
 - SWAP_OFFSET value is wrong: when SWAP_SECT_REQ = 1, SWAP_OFFSET must be equal or greater than OBK_HDPL<N-1>_SWAP_OFFSET.
 - SWAP_SECT_REQ or OBK_SWAP set while DBNE = 1
 - OBK-HDPL is incorrect and secure OBK_SWAP is requested

- If OBKSWAP or OBK alternate erase is started while write buffer is waiting for new data (WBNE = 1)

When PGSERR flag is raised as consequence of failed write attempt (flash programming), the current program operation is aborted and nothing is changed in the corresponding bank. The write data buffer is also invalidated.

Note: When PGSERR flag is raised, there is a risk that the last write operation performed by the application has been lost because of the above protection mechanism. It is recommended to generate interrupts on PGSERR and verify in the interrupt handler if the last write operation has been successful by reading back the value in the flash memory.

The PGSERR flag also blocks any new program operation. This means that PGSERR must be cleared before starting a new program operation.

PGSERR flag is cleared by setting CLR_SECPGSERR bit to 1 in FLASH_SECCCR register.

If PGSERRIE bit in FLASH_SECCR register is set to 1, an interrupt is generated when PGSERR flag is raised. See [Section 7.10](#) for details.

7.9.6 Nonsecure strobe error (STRBERR)

When the nonsecure application software writes several times to the same byte in the write buffer, the flash interface sets the strobe error flag STRBERR (FLASH_NSSR) whatever the target bank of the write access.

When STRBERR flag is raised, the current program operation is aborted and the write buffer is invalidated.

STRBERR flag is cleared by setting CLR_STRBERR bit to 1 in FLASH_NSCCR register.

If STRBERRIE bit in FLASH_NSCR register is set to 1, an interrupt is generated when STRBERR flag is raised. See [Section 7.10](#) for details.

7.9.7 Secure strobe error (STRBERR)

When the secure application software writes several times to the same byte in the write buffer, the memory sets the strobe error flag STRBERR (FLASH_SECSR) whatever the target bank of the write access.

When STRBERR flag is raised, the current program operation is aborted and the write buffer is invalidated.

STRBERR flag is cleared by setting CLR_STRBERR bit to 1 in FLASH_SECCCR register.

If STRBERRIE bit in FLASH_SECCR register is set to 1, an interrupt is generated when STRBERR flag is raised. See [Section 7.10](#) for details.

7.9.8 Nonsecure inconsistency error (INCERR)

When a programming inconsistency in nonsecure access is detected, the flash interface sets the inconsistency error flag INCERR in register FLASH_NSSR.

More specifically, INCERR flag is set when one of the following conditions is met:

- A write operation is attempted before completion of the previous write operation, for example:
 - The application software starts a write operation to fill the 128-bit write buffer, but sends a new write burst request to a different flash memory address before the buffer is full.
 - One master starts a write operation, but before the buffer is full, another master starts a new write operation to the same address or to a different address.
 - ALT_SECT in FLASH_NSOKCFGR changed while filling the write buffer.

Note: INCERR flag must be cleared before starting a new write operation, otherwise a sequence error (PGSERR) is raised.

It is recommended to follow the following sequence to avoid losing data when an inconsistency error occurs:

1. Execute a handler routine when INCERR flag is raised.
2. Stop all write requests to the memory.
3. Clear INCERR bit.
4. Restart the write operations from where they have been interrupted.

INCERR flag is cleared by setting CLR_INCERR bit to 1 in FLASH_NSCCR register.

If INCERRIE bit in FLASH_NSCR register is set to 1, an interrupt is generated when INCERR flag is raised (see [Section 7.10](#) for details).

7.9.9 Secure inconsistency error (INCERR)

When a programming inconsistency is detected, the flash interface sets the inconsistency error flag INCERR in register FLASH_SECSR.

More specifically, INCERR flag is set when one of the following conditions is met:

- A write operation is attempted before completion of the previous write operation, for example:
 - The application software starts a write operation to fill the 128-bit write buffer, but sends a new write burst request to a different flash memory address before the buffer is full.
 - One master starts a write operation, but before the buffer is full, another master starts a new write operation to the same address or to a different address.

Note: INCERR flag must be cleared before starting a new write operation, otherwise a sequence error (PGSERR) is raised.

It is recommended to follow the sequence below to avoid losing data when an inconsistency error occurs:

1. Execute a handler routine when INCERR flag is raised.
2. Stop all write requests to the flash memory.
3. Clear INCERR bit.
4. Restart the write operations from where they have been interrupted.

INCERR flag is cleared by setting CLR_INCERR bit to 1 in FLASH_SECCR register.

If INCERRIE bit in FLASH_SECCR register is set to 1, an interrupt is generated when INCERR flag is raised (see [Section 7.10](#) for details).

7.9.10 Error correction code error (ECCC, ECCD)

When a single-error correction is detected during a read, the flash interface sets the single-error correction flag ECCC in FLASH_ECCCORR register.

When two ECC errors are detected during a read, the flash interface sets the double error detection flag ECCD in FLASH_ECCDETR register.

When the ECCC flag is raised, the corrected read data are returned. The application can ignore the error, and request new read operations. When the ECCD the flag is raised, an NMI is generated, it can be masked in SBS registers (*SBS flitf ECC NMI mask register (SBS_ECCNMIR)*) for data access (OTP, data area, RO data). Software must invalidate the instruction cache (CACHEINV = 1) in the NMI interrupt service routine when the ECCD flag is set.

When ECCC or ECCD flag is raised, the address of the flash word that generated the error is saved in the FLASH_ECCCORR (FLASH_ECCDETR) register. If the address corresponds to a read-only area or to an OTP area or flash high-cycle data, the OTP_ECC bit is also set to 1 in the FLASH_ECCCORRR (FLASH_ECCDETR) register. This register is automatically cleared when the associated flag that generated the error is reset.

A BK_ECC flag indicates in which flash bank the error occurred.

A SYSF_ECC flag indicates an error detected in the system flash area.

Table 88. Locating ECC failure

OTP_ECC	SYSF_ECC	BK_ECC	EDATA_ECC	OBK_ECC	Flash area	ADDR_ECC min	ADDR_ECC max
0	0	0/1	0	0	User flash	0x0000	0xFFFF
0	1	0/1	0	0	System flash	0x0000	0x0FFF
1	0	0	0	0	OTP	0x0600	0x07FF
0	0	1	0	1	OBKeys	0x0000	0x03FF
0	0	0/1	1	0	Data area sector 7	0xF000	0xF1FF
0	0	0/1	1	0	Data area sector 6	0xF200	0xF3FF
0	0	0/1	1	0	Data area sector 5	0xF400	0xF5FF
0	0	0/1	1	0	Data area sector 4	0xF600	0xF7FF
0	0	0/1	1	0	Data area sector 3	0xF800	0xF9FF
0	0	0/1	1	0	Data area sector 2	0xFA00	0xFBFF
0	0	0/1	1	0	Data area sector 1	0xFC00	0xFDFF
0	0	0/1	1	0	Data area sector 0	0xFE00	0xFFFF

Note: In case of successive single correction or double detection errors, only the address corresponding to the first error is stored in FLASH_ECCCORR (FLASH_ECCDETR) register.

It is mandatory to clear ECCC or ECCD flags before starting a new read operation.

As the ECC interface is shared by the two banks, if the same error is registered simultaneously, a physical Bank1 error is registered. Both errors are registered if one bank reports ECCD and the other bank ECCC.

ECCC (respectively ECCD) flag is cleared by setting to 1 ECCC bit (respectively ECCD bit) in FLASH_ECCCORR (FLASH_ECCDETR) register.

If ECCC bit in FLASH_ECCCORR register is set to 1, an interrupt is generated when ECCC flag is raised. Only NMI is generated for the ECCD. See [Section 7.10](#) for details.

7.9.11 Illegal access (ILAFM/ILAP)

Illegal access is a signal to the TZIC, triggering a secure interrupt in there. It is complementary to the other interrupts and only generated when the TZ is enabled (TZ_STATE = 0xB4).

It can be masked only on GTZC level.

ILAFM is generated when rules on secure flash memory access are violated:

- Attempt to access secure memory location in nonsecure mode.
- Attempt to access nonsecure memory location in secure mode.

ILAP is generated on:

- Attempt to access secure register in nonsecure mode.

If ILAFM is detected during write, the write buffer is invalidated.

Note: ILAFM and ILAP has no flag within the flash memory controller to clear to resume operation. It is dealt within the TZIC.

7.9.12 Option byte change error (OPTCHANGEERR)

When the flash interface finds an error during an option change operation, it aborts the operation and sets the option byte change error flag OPTCHANGEERR in FLASH_NSSR register.

The error is raised after OPTSTRT bit set if:

- TZEN is not set and transition to TZ-Closed or NS-Regression state is requested. Also any other forbidden PRODUCT_STATE transition ([Table 76](#)).
- SWAP_BANK is locked out by combination of TZEN and BOOT_LOCK.
- OB modification is attempted in wrong HDPL (selected PRODUCT_STATE transitions)
- Attempt to modify OB with invalid value
 - New EPOCH value not greater than current
 - Value not recognized by the specification (enumerated magic numbers only)
- Attempt to modify OB in product state where it is not allowed (usually a state open for debug is required for change).

Note: Exceptions and details provided in the OB description (see [Section 7.4.7](#) and [Table 56](#)).

OPTCHANGEERR flag is cleared by setting CLR_OPTCHANGEERR bit to 1 in FLASH_NSSCR register.

If OPTCHANGEERRIE bit in FLASH_NSCR register is set to 1, an interrupt is generated when OPTCHANGEERR flag is raised (see [Section 7.10](#) for details).

It is mandatory to clean the OPTCHANGEERR flag before starting a new operation (option change, erase or write).

7.9.13 Miscellaneous HardFault errors

The following events generate a bus error on the corresponding bus interface:

- On main AHB system bus for access targeting code and data with 9 bits ECC.
 - Fetching from secure user flash memory in nonsecure mode.
 - Fetching from nonsecure user flash memory in secure mode.
 - Access to invalid address (including data addresses forbidden for code use).
 - Fetching from HDP area with incorrect HDPL value.
 - Fetching from privilege area in unprivilege mode.
- On AHB configuration or system bus for accesses targeting OTP/RO (all addresses using 6bits ECC):
 - wrong key input to FLASH_NS/SECKEYR or FLASH_OPTKEYR.
 - 8-bit accesses to system AHB interface.
 - Wrong unlock sequence on a register.

7.9.14 OBK error cases (OBKERR, OBKWERR)

Four error types can be detected while filling the OBK write buffer: STRBERR, INCERR, WRPERR or OBKERR. The OBKERR is specific to the OBK access and cannot be raised by access to any other flash memory asset.

Note: For STRBERR, INCERR and WRPERR, nonsecure or secure flags are used depending on the AHB access type (sec or nonsecure). If a nonsecure access is received, the nonsecure error flags are used. If a secure access is received, the secure error flags are used.

For OBKERR and OBKWERR, nonsecure or secure flags are used depending on the value of the secure state. If the SOC is in secure state (TZ_STATE active), the secure error flags (OBKERR and OBKWERR) are used. Otherwise the nonsecure error flags (OBKERR and OBKWERR) are used.

OBKERR is raised when:

- the OBK-HDPL (input signal from SBS) does not match the HDPL associated to the key during a read or write access, and the TZ and PRIV attributes are correct (See [Section 7.5.2](#) for more details).

If OBKERR is raised during write access, the write buffer is flushed. If the OBKERR is raised on read access, the write buffer is intact. In both cases the flag must be cleared to enable write/erase access to flash memory.

OBKWERR is detected after filling the write buffer and raised if:

- the address is not virgin during a write access.
- OBK selector in the alternate sector is not virgin during a SWAP operation.

If OBKWERR is raised, the write buffer is flushed. The flag must be cleared to enable write/erase access to flash memory.

7.10 FLASH interrupts

The flash interface can generate a maskable interrupt to signal the following events on a given bank:

- Read and write errors (see [Section 7.9](#))
 - Single ECC error correction during read operation
 - Write inconsistency error
 - Bad programming sequence
 - Strobe error during write operations
 - option change operation error
- Security errors (see [Section 7.9](#))
 - Write protection error
 - Illegal access error - signal to TZIC
- Miscellaneous events (described below)
 - End of programming

A NMI is raised on double ECC error detection during read operation.

Two interrupt lines gather all the error sources, one for errors in nonsecure access and second for errors that happen in secure access.

The user can individually enable or disable flash interface interrupt sources by changing the mask bits in the FLASH_NS/SECCR, FLASH_ECCCORR and FLASH_ECCDETR register. Setting the appropriate mask bit to 1 enables the interrupt.

Note: Prior to writing, FLASH_NS/SECCR register must be unlocked as explained in [Section 7.6.7](#).

[Table 89](#) gives a summary of the available flash interface interrupt features. As mentioned in the table below, some flags need to be cleared before a new operation is triggered.

Table 89. Flash interrupt request

Interrupt event Event flag	Error flag label in register	Enable control bit	Clear flag to resume operation
End of operation event	EOP	EOPIE	N/A
Write protection error	WRPERR	WRPERRIE	Yes
Programming sequence error	PGSERR	PGSERRIE	Yes
Strobe error	STRBERR	STRBERRIE	Yes
OBK write error	OBKWERR	OBKWERRIE	Yes
OBK general error	OBKERR	OBKERRIE	Yes
Inconsistency error	INCERR	INCERRIE	Yes
Illegal access error	ILAFM	TZEN	No
Option byte error	OPTCHANGEERR	-	Yes

Table 89. Flash interrupt request (continued)

Interrupt event Event flag	Error flag label in register	Enable control bit	Clear flag to resume operation
ECC single error correction event	ECCC	ECCCIE	No
ECC double error detection event ⁽¹⁾	ECCD	Disabled	No

1. NMI.

The status of the individual maskable interrupt sources described in [Table 89](#) (except for option byte error and ECC) can be read from the FLASH_NS/SECSR register. They can be cleared by setting to 1 the adequate bit in FLASH_NS/SECCCR register.

Note: No unlocking mechanism is required to clear an interrupt.

End of operation event

Setting the end of operation interrupt enable bit (EOPIE) in the FLASH_NS/SECCR register enables the generation of an interrupt at the end of an erase operation, a program operation, or an option byte change.

When managing the OBKey storage area, the EOP is associated also with the OBK swap operation and the alternate area erase operation. EOP bit in the FLASH_NS/SECSR register is also set when one of these events occurs.

Setting CLR_EOP bit to 1 in FLASH_NS/SECCCR register clears EOP flag.

7.11 FLASH registers

Each register has an offset address and a reset value. In case of registers representing option bytes, the reset value is determined by the OBL process. In case of success the reset value is loaded from OB, in case of failure, a highly restrictive default value is set.

7.11.1 FLASH access control register (FLASH_ACR)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

For more details, refer to [Section 7.3.4](#) and [Section 7.3.5](#).

Address offset: 0x000

Reset value: 0x0000 0013

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRF TEN	Res.	Res.	WRHIGHFREQ [1:0]		LATENCY[3:0]			
							rw			rw	rw	rw	rw	rw	rw

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **PRFTEN**: Prefetch enable. When bit value is modified, user must read back ACR register to be sure PRFTEN has been taken into account.

Bits used to control the prefetch.

0: prefetch disabled.

1: prefetch enabled when latency is at least one wait state.

Bits 7:6 Reserved, must be kept at reset value.

Bits 5:4 **WRHIGHFREQ[1:0]**: Flash signal delay

These bits are used to control the delay between nonvolatile memory signals during programming operations. Application software has to program them to the correct value depending on the memory interface frequency. Please refer to [Table 48](#) for details.

Note: No check is performed to verify that the configuration is correct.

Two WRHIGHFREQ values can be selected for some frequencies.

Bits 3:0 **LATENCY[3:0]**: Read latency

These bits are used to control the number of wait states used during read operations on both nonvolatile memory banks. The application software has to program them to the correct value depending on the memory interface frequency and voltage conditions.

0000: no wait states used to read a word from nonvolatile memory

0001: one wait state used to read a word from nonvolatile memory

0010: two wait states used to read a word from nonvolatile memory

...

0111: seven wait states used to read a word from nonvolatile memory

1111: 15 wait states used to read a word from nonvolatile memory

Note: No check is performed by hardware to verify that the configuration is correct.

7.11.2 FLASH nonsecure key register (FLASH_NSKEYR)

This register is nonsecure. It can be written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

FLASH_NSKEYR is a write-only register. The following values must be programmed consecutively to unlock FLASH_NSCR register and allow programming/erasing it.

- First key = 0x4567 0123
- Second key = 0xCDEF 89AB

A wrong sequence locks the FLASH_NSCR register until next system reset.

Address offset: 0x004

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NSKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **NSKEY[31:0]**: Nonvolatile memory nonsecure configuration access unlock key

7.11.3 FLASH secure key register (FLASH_SECKEYR)

This register is secure. It can be written only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

FLASH_SECKEYR is a write-only register. The following values must be programmed consecutively to unlock FLASH_SECCR register and allow programming/erasing it. A wrong sequence locks the FLASH_SECCR register until next system reset.

First key = 0x4567 0123

Second key = 0xCDEF 89AB

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **SECKEY[31:0]**: Nonvolatile memory secure configuration access unlock key

7.11.4 FLASH option key register (FLASH_OPTKEYR)

This write-only register is nonsecure. It can be written by both secure and nonsecure accesses, and can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

The following values must be programmed consecutively to unlock FLASH_OPTCR register

- First key = 0x0819 2A3B
- Second key = 0x4C5D 6E7F

A wrong sequence locks FLASH_OPTCR register until next system reset.

Address offset: 0x00C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OPTKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OPTKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **OPTKEY[31:0]**: FLASH option bytes control access unlock key

7.11.5 FLASH nonsecure OBK key register (FLASH_NSObKKEYR)

This is a write-only register. The following values must be programmed consecutively to unlock it register (a wrong sequence locks the register until next system reset).

- First key = 0x192A 083B
- Second key = 0x6E7F 4C5D

This register is nonsecure. This register can be written only by privileged access.

Address offset: 0x0010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSObKKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NSObKKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **NSObKKEY[31:0]**: FLASH nonsecure option bytes keys control access unlock key

7.11.6 FLASH secure OBK key register (FLASH_SECOBKKEYR)

This is a write-only register. The following values must be programmed consecutively to unlock it (a wrong sequence locks the register until the next system reset)

- First key = 0x192A 083B
- Second key = 0x6E7F 4C5D

This register is secure, it can be written only by privileged access.

Address offset: 0x0014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECOBKKEY[31:16]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECOBKKEY[15:0]															
w	w	w	w	w	w	w	w	w	w	w	w	w	w	w	w

Bits 31:0 **SECOBKKEY[31:0]**: FLASH secure option bytes keys control access unlock key

7.11.7 FLASH operation status register (FLASH_OPSR)

This register is nonsecure. This register can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x0018

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CODE_OP[2:0]			Res.	Res.	Res.	Res.	OTP_OP	SYSF_OP	BK_OP	DATA_OP	Res.	ADDR_OP[19:16]			
r	r	r					r	r	r	r		r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR_OP[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:29 **CODE_OP[2:0]**: Flash memory operation code

000: No flash operation on going during previous reset

001: Single write operation interrupted

010: OBK alternate sector erase

011: Sector erase operation interrupted

100: Bank erase operation interrupted

101: Mass erase operation interrupted

110: Option change operation interrupted

111: OBK swap sector request

Bits 28:25 Reserved, must be kept at reset value.

Bit 24 **OTP_OP**: OTP operation interrupted

Indicates that reset interrupted an ongoing operation in OTP area (or OBKeys area).

Bit 23 **SYSF_OP**: Operation in system flash memory interrupted

Indicates that reset interrupted an ongoing operation in system flash.

Bit 22 **BK_OP**: Interrupted operation bank

It indicates which bank was concerned by operation.

Bit 21 **DATA_OP**: Flash high-cycle data area operation interrupted
It indicates if flash high-cycle data area is concerned by operation.

Bit 20 Reserved, must be kept at reset value.

Bits 19:0 **ADDR_OP[19:0]**: Interrupted operation address

7.11.8 FLASH option control register (FLASH_OPTCR)

This register is nonsecure. It can be read and written by both secure and nonsecure access, and protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Access: No wait states when no memory operations are ongoing. The FLASH_OPTCR register is not accessible in write mode when the BSY bit is set. Any attempt to write to it while the BSY bit set causes the AHB bus to stall until the BSY bit is cleared.

Address offset: 0x01C

Reset value: 0xX000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r															
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPT STRT	OPT LOCK
														rw	rs

Bit 31 **SWAP_BANK**: Bank swapping option configuration bit

SWAP_BANK controls whether Bank1 and Bank2 are swapped or not. This bit is loaded with the SWAP_BANK bit of FLASH_OPTSR_CUR register only after reset or POR.

0: Bank1 and Bank2 not swapped

1: Bank1 and Bank2 swapped

Bits 30:2 Reserved, must be kept at reset value.

Bit 1 **OPTSTRT**: Option byte start change option configuration bit

OPTSTRT triggers an option byte change operation. The user can set OPTSTRT only when the OPTLOCK bit is cleared to 0. It is set only by Software and cleared when the option byte change is completed or an error occurs (PGSERR or OPTCHANGEERR). It is reset at the same time as BSY bit.

The user application cannot modify any FLASH_XXX_PRG flash interface register until the option change operation has been completed.

Before setting this bit, the user has to write the required values in the FLASH_XXX_PRG registers. The FLASH_XXX_PRG registers are locked until the option byte change operation has been executed in nonvolatile memory.

Bit 0 **OPTLOCK**: FLASH_OPTCR lock option configuration bit

The OPTLOCK bit locks the FLASH_OPTCR register as well as all _PRG registers. The correct write sequence to FLASH_OPTKEYR register unlocks this bit. If a wrong sequence is executed, or the unlock sequence to FLASH_OPTKEYR is performed twice, this bit remains locked until next system reset.

It is possible to set OPTLOCK by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When OPTLOCK changes from 0 to 1, the others bits of FLASH_OPTCR register do not change.

0: FLASH_OPTCR register unlocked

1: FLASH_OPTCR register locked.

7.11.9 FLASH nonsecure status register (FLASH_NSSR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x020

Reset value: 0x0000 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPT CHANGE ERR	OBKW ERR	OBK ERR	INC ERR	STRB ERR	PGS ERR	WRP ERR	EOP
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBNE	Res.	WBNE	BSY
												r		r	r

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **OPTCHANGEERR**: Option byte change error flag

OPTCHANGEERR flag indicates that an error occurred during an option byte change operation. When OPTCHANGEERR is set to 1, the option byte change operation did not successfully complete. An interrupt is generated when this flag is raised if the OPTCHANGEERRIE bit of FLASH_NSCR register is set to 1.

Writing 1 to CLR_OPTCHANGEERR of register FLASH_NSCCR clears OPTCHANGEERR.

0: no option byte change errors occurred

1: one or more errors occurred during an option byte change operation.

Note: The OPTSTRT bit in FLASH_OPTCR cannot be set while OPTCHANGEERR is set.

Bit 22 **OBKWERR**: OBK write error flag

OBKWERR flag is raised when the address is not virgin on a write access to the OBK storage. Alternatively also when the OBK selector in the alternate sector is not virgin during a swap operation.

0: no OBK write error occurred

1: an OBK write error occurred

- Bit 21 **OBKERR**: OBK general error flag
OBKERR flag is raised when the OBK-HDPL signal from the SBS does not match the HDPL value associated with the key slot during access to the key location. Alternatively also when the ALT_SECT is unexpectedly changed while the write buffer is being filled.
0: no OBK general error occurred
1: an OBK general error occurred
- Bit 20 **INCERR**: inconsistency error flag
NSINCERR flag is raised when a inconsistency error occurs. An interrupt is generated if INCERRIE is set to 1. Writing 1 to CLR_INCERR bit in the FLASH_NSCCR register clears NSINCERR.
0: no inconsistency error occurs
1: a inconsistency error occurs
- Bit 19 **STRBERR**: strobe error flag
STRBERR flag is raised when a strobe error occurs (when the master attempts to write several times the same byte in the write buffer). An interrupt is generated if the STRBERRIE bit is set to 1. Writing 1 to CLR_STRBERR bit in FLASH_NSCCR register clears STRBERR.
0: no strobe error occurred
1: a strobe error occurred
- Bit 18 **PGSERR**: programming sequence error flag
PGSERR flag is raised when a sequence error occurs. An interrupt is generated if the PGSERRIE bit is set to 1. Writing 1 to CLR_PGSERR bit in FLASH_NSCCR register clears PGSERR.
0: no sequence error occurred
1: a sequence error occurred
- Bit 17 **WRPERR**: write protection error flag
WRPERR flag is raised when a protection error occurs during a program operation. An interrupt is also generated if the WRPERRIE is set to 1. Writing 1 to CLR_WRPERR bit in FLASH_NSCCR register clears WRPERR.
0: no write protection error occurred
1: a write protection error occurred
- Bit 16 **EOP**: end of operation flag
EOP flag is set when a operation (program/erase) completes. An interrupt is generated if the EOPIE is set to 1. It is not necessary to reset EOP before starting a new operation. EOP bit is cleared by writing 1 to CLR_EOP bit in FLASH_NSCCR register.
0: no operation completed
1: a operation completed
- Bits 15:4 Reserved, must be kept at reset value.
- Bit 3 **DBNE**: data buffer not empty flag
DBNE flag is set when the flash interface is processing 6-bits ECC data in dedicated buffer. This bit cannot be set to 0 by software. The hardware resets it once the buffer is free.
0: data buffer not used
1: data buffer used, wait
- Bit 2 Reserved, must be kept at reset value.

Bit 1 **WBNE**: write buffer not empty flag

WBNE flag is set when the flash interface is waiting for new data to complete the write buffer. In this state, the write buffer is not empty. WBNE is reset by hardware each time the write buffer is complete or the write buffer is emptied following one of the events below:

- the application software forces the write operation using FW bit in FLASH_NSCR
- the memory detects an error that involves data loss

This bit cannot be reset by software writing 0 directly. To reset it, clear the write buffer by performing any of the above listed actions, or send the missing data.

0: write buffer empty or full

1: write buffer waiting data to complete

Bit 0 **BSY**: busy flag

BSY flag indicates that a flash memory is busy by an operation (write, erase, option byte change, OBK operation). It is set at the beginning of a flash memory operation and cleared when the operation finishes, or an error occurs.

0: no programming, erase or option byte change operation being executed

1: programming, erase or option byte change operation being executed

7.11.10 FLASH secure status register (FLASH_SECSR)

This register is secure. It can be read only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x024

Reset value: 0x0000 000X

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OBKW ERR	OBK ERR	INC ERR	STRB ERR	PGS ERR	WRP ERR	EOP
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBNE	Res.	WBNE	BSY
												r		r	r

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **OBKWERR**: OBK write error flag

OBKWERR flag is raised when the address is not virgin on a write access to the OBK storage. Alternatively also when the OBK selector in the alternate sector is not virgin during a swap operation.

0: no OBK write error occurred

1: an OBK write error occurred

Bit 21 **OBKERR**: OBK general error flag

OBKERR flag is raised when the OBK-HDPL signal from the SBS does not match the HDPL value associated with the key slot during access to the key location. Alternatively also when the ALT_SECT is unexpectedly changed while the write buffer is being filled.

0: no OBK general error occurred

1: an OBK general error occurred

- Bit 20 **INCERR**: inconsistency error flag
 INCERR flag is raised when a inconsistency error occurs. An interrupt is generated if INCERRIE is set to 1. Writing 1 to CLR_INCERR bit in the FLASH_SECCCR register clears INCERR.
 0: no inconsistency error occurred
 1: a inconsistency error occurred
- Bit 19 **STRBERR**: strobe error flag
 STRBERR flag is raised when a strobe error occurs (when the master attempts to write several times the same byte in the write buffer). An interrupt is generated if the STRBERRIE bit is set to 1. Writing 1 to CLR_STRBERR bit in FLASH_SECCCR register clears STRBERR.
 0: no strobe error occurred
 1: a strobe error occurred
- Bit 18 **PGSERR**: programming sequence error flag
 PGSERR flag is raised when a sequence error occurs. An interrupt is generated if the PGSERRIE bit is set to 1. Writing 1 to CLR_PGSERR bit in FLASH_SECCCR register clears PGSERR.
 0: no sequence error occurred
 1: a sequence error occurred
- Bit 17 **WRPERR**: write protection error flag
 WRPERR flag is raised when a protection error occurs during a program operation. An interrupt is also generated if the WRPERRIE is set to 1. Writing 1 to CLR_WRPERR bit in FLASH_SECCCR register clears WRPERR.
 0: no write protection error occurred
 1: a write protection error occurred
- Bit 16 **EOP**: end of operation flag
 EOP flag is set when a operation (program/erase) completes. An interrupt is generated if the EOPIE is set to. It is not necessary to reset EOP before starting a new operation. EOP bit is cleared by writing 1 to CLR_EOP bit in FLASH_SECCCR register.
 0: no operation completed
 1: a operation completed
- Bits 15:4 Reserved, must be kept at reset value.
- Bit 3 **DBNE**: data buffer not empty flag
 DBNE flag is set when the memory interface is processing 6-bits ECC data in dedicated buffer. This bit cannot be set to 0 by software. The hardware resets it once the buffer is free.
 0: data buffer not used
 1: data buffer used, wait
- Bit 2 Reserved, must be kept at reset value.
- Bit 1 **WBNE**: write buffer not empty flag
 WBNE flag is set when the flash interface is waiting for new data to complete the write buffer. In this state, the write buffer is not empty. WBNE is reset by hardware each time the write buffer is complete or the write buffer is emptied following one of the events below:
- the application software forces the write operation using FW bit in FLASH_SECCCR
 - the flash interface detects an error that involves data loss
- This bit cannot be reset by writing 0 directly by software. To reset it, clear the write buffer by performing any of the above listed actions, or send the missing data.
 0: write buffer empty or full
 1: write buffer waiting data to complete

Bit 0 **BSY**: busy flag

BSY flag indicates that a FLASH memory is busy (write, erase, option byte change, OBK operations). It is set at the beginning of a flash memory operation and cleared when the operation finishes or an error occurs.

0: no programming, erase or option byte change operation being executed

1: programming, erase or option byte change operation being executed

7.11.11 FLASH nonsecure control register (FLASH_NSCR)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses, and can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Access: No wait states when no memory operations are ongoing. The FLASH_NSCR register is not accessible in write mode when the BSY bit is set. Any attempt to write to it with the BSY bit set causes the AHB bus to stall until the BSY bit is cleared.

Address offset: 0x028

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BKSEL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPT CHANGE ERRIE	OBKW ERRIE	OBK ERRIE	INC ERRIE	STRB ERRIE	PGS ERRIE	WRP ERRIE	EOPIE
rw								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MER	Res.	Res.	SNB[6:0]							STRT	FW	BER	SER	PG	LOCK
rw			rw	rw	rw	rw	rw	rw	rw	rs	rw	rw	rw	rw	rs

Bit 31 **BKSEL**: Bank selector bit

Can be programmed only when LOCK is cleared to 0. The bit selects physical bank, SWAP_BANK setting is ignored.

0: Bank1 is selected for Bank erase / sector erase / interrupt enable

1: Bank2 is selected for BER / SER

Bits 30:24 Reserved, must be kept at reset value.

Bit 23 **OPTCHANGEERRIE**: Option byte change error interrupt enable bit

This bit controls if an interrupt must be generated when an error occurs during an option byte change. It can be programmed only when LOCK bit is cleared to 0.

0: no interrupt is generated when an error occurs during an option byte change

1: an interrupt is generated when an error occurs during an option byte change.

Bit 22 **OBKWERRIE**: OBK write error interrupt enable bit

OBKWERRIE enables generation of interrupt in case of OBK specific write error. This bit can be programmed only when LOCK bit is cleared to 0.

0: no interrupt is generated on OBK write error

1: an interrupt is generated on OBK write error

Bit 21 **OBKERRIE**: OBK general error interrupt enable bit

OBKERRIE enables generating an interrupt in case of OBK specific access error. This bit can be programmed only when LOCK bit is cleared to 0.

0: no interrupt is generated on OBK general access error

1: an interrupt is generated on OBK general access error

Bit 20 **INCERRIE**: inconsistency error interrupt enable bit

When INCERRIE bit is set to 1, an interrupt is generated when an inconsistency error occurs during a write operation. INCERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a inconsistency error occurs

1: interrupt generated when a inconsistency error occurs.

Bit 19 **STRBERRIE**: strobe error interrupt enable bit

When STRBERRIE bit is set to 1, an interrupt is generated when a strobe error occurs (the master programs several times the same byte in the write buffer) during a write operation. STRBERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a strobe error occurs

1: interrupt generated when strobe error occurs.

Bit 18 **PGSERRIE**: programming sequence error interrupt enable bit

When this bit is set to 1, an interrupt is generated when a sequence error occurs during a program operation. PGSERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a sequence error occurs

1: interrupt generated when sequence error occurs

Bit 17 **WRPERRIE**: write protection error interrupt enable bit

When this bit is set to 1, an interrupt is generated when a protection error occurs during a program operation. WRPERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a protection error occurs

1: interrupt generated when a protection error occurs

Bit 16 **EOPIE**: end of operation interrupt control bit

Setting EOPIE bit to 1 enables the generation of an interrupt at the end of a program or erase operation. EOPIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated at the end of operation.

1: interrupt enabled when at the end of operation

Bit 15 **MER**: mass erase request

Setting MER bit to 1 requests a mass erase operation (user flash memory only). MER can be programmed only when LOCK is cleared to 0.

If BER or SER are both set, a PGSERR is raised.

0: mass erase not requested

1: mass erase requested

Note: An error is triggered when a mass erase is required and some sectors are protected.

Bits 14:13 Reserved, must be kept at reset value.

Bits 12:6 **SNB[6:0]**: sector erase selection number

These bits are used to select the target sector for an erase operation, otherwise they are not used. They can be programmed only when LOCK is cleared to 0.

0x00: Sector 0 selected

0x01: Sector 1 selected

..

0x7F: Sector 127 selected

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bit 5 **STRT**: erase start control bit

STRT bit is used to start a sector erase or a bank erase operation. STRT can be programmed only when LOCK is cleared to 0.

STRT is reset at the end of the operation or when an error occurs. It cannot be reset by software.

Bit 4 FW: write forcing control bit

FW forces a write operation even if the write buffer is not full. In this case all bits not written are set to 1 by hardware. FW can be programmed only when LOCK is cleared to 0.

The memory resets FW when the corresponding operation has been acknowledged.

Note: Using a force-write operation prevents the application from updating later the missing bits with something else than 1, because it is likely that it leads to permanent ECC error.

Write forcing is effective only if the write buffer is not empty and was filled by nonsecure access (in particular, FW does not start several write operations when the force-write operations are performed consecutively).

Since there is just one write buffer, FW can force a write in Bank1 or Bank2.

Bit 3 BER: erase request

Setting BER bit to 1 requests a bank erase operation (user flash memory only). BER can be programmed only when LOCK is cleared to 0.

If MER and SER are also set, a PGSEERR is raised.

0: bank erase not requested

1: bank erase requested

Note: Write protection error is triggered when a bank erase is required and some sectors are protected.

Bit 2 SER: sector erase request

Setting SER bit to 1 requests a sector erase. SER can be programmed only when LOCK is cleared to 0.

If MER and SER are also set, a PGSEERR is raised.

0: sector erase not requested

1: sector erase requested

Bit 1 PG: programming control bit

PG can be programmed only when LOCK is cleared to 0.

PG allows programming in Bank1 and Bank2.

0: programming disabled

1: programming enabled

Bit 0 LOCK: configuration lock bit

This bit locks the FLASH_NSCR register. The correct write sequence to FLASH_NSKEYR register unlocks this bit. If a wrong sequence is executed, or if the unlock sequence to FLASH_NSKEYR is performed twice, this bit remains locked until the next system reset.

LOCK can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK changes from 0 to 1, the other bits of FLASH_NSCR register do not change.

0: FLASH_NSCR register unlocked

1: FLASH_NSCR register locked

7.11.12 FLASH secure control register (FLASH_SECCR)

This register is secure, can be read and written only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

Access: No wait states when no memory operations are ongoing. The FLASH_SECCR register is not accessible in write mode when the BSY bit is set. Any attempt to write to it with the BSY bit set causes the AHB bus to stall until the BSY bit is cleared.

Address offset: 0x02C

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BKSEL	Res.	INV	Res.	Res.	Res.	Res.	Res.	Res.	OBKW ERRIE	OBK ERRIE	INC ERRIE	STRB ERRIE	PGS ERRIE	WRP ERRIE	EOPIE
rw		rw							rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MER	Res.	Res.	SNB[6:0]							STRT	FW	BER	SER	PG	LOCK
rw			rw	rw	rw	rw	rw	rw	rw	rs	rw	rw	rw	rw	rs

Bit 31 BKSEL: Bank selector bit

BKSEL can only be programmed when LOCK is cleared to 0. The bit selects physical bank, SWAP_BANK setting is ignored.

0: Bank1 is selected for Bank erase / sector erase / interrupt enable

1: Bank2 is selected for BER / SER

Bit 30 Reserved, must be kept at reset value.

Bit 29 INV: Flash memory security state invert.

This bit inverts the flash memory security state.

Bits 28:23 Reserved, must be kept at reset value.

Bit 22 OBKWERRIE: OBK write error interrupt enable bit

OBKWERRIE enables generation of interrupt in case of OBK specific write error.

OBKWERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt is generated on OBK write error

1: an interrupt is generated on OBK write error

Bit 21 OBKERRIE: OBK general error interrupt enable bit

OBKERRIE enables generating an interrupt in case of OBK specific access error.

OBKERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt is generated on OBK general access error

1: an interrupt is generated on OBK general access error

Bit 20 INCERRIE: inconsistency error interrupt enable bit

When INCERRIE bit is set to 1, an interrupt is generated when an inconsistency error occurs during a write operation. INCERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a inconsistency error occurs

1: interrupt generated when a inconsistency error occurs.

Bit 19 STRBERRIE: strobe error interrupt enable bit

When STRBERRIE bit is set to 1, an interrupt is generated when a strobe error occurs (the master programs several times the same byte in the write buffer) during a write operation. STRBERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a strobe error occurs

1: interrupt generated when strobe error occurs.

Bit 18 PGSERRIE: programming sequence error interrupt enable bit

When PGSERRIE is set to 1, an interrupt is generated when a sequence error occurs during a program operation. PGSERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a sequence error occurs

1: interrupt generated when sequence error occurs

Bit 17 **WRPERRIE**: write protection error interrupt enable bit

When WRPERRIE bit is set to 1, an interrupt is generated when a protection error occurs during a program operation. WRPERRIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated when a protection error occurs
1: interrupt generated when a protection error occurs

Bit 16 **EOPIE**: end of operation interrupt control bit

Setting EOPIE bit to 1 enables the generation of an interrupt at the end of a program/erase operation. EOPIE can be programmed only when LOCK is cleared to 0.

0: no interrupt generated at the end of operation.
1: interrupt enabled when at the end of operation

Bit 15 **MER**: mass erase request

Setting MER bit to 1 requests a mass erase operation (user flash memory only). MER can be programmed only when LOCK is cleared to 0.

If BER or SER are also set, a PGSERR is raised.

0: mass erase not requested
1: mass erase requested

Note: An error is triggered when a mass erase is required and some sectors are protected.

Bits 14:13 Reserved, must be kept at reset value.

Bits 12:6 **SNB[6:0]**: sector erase selection number

These bits are used to select the target sector for an erase operation, otherwise they are unused. SNB can be programmed only when LOCK is cleared to 0.

0x00: Sector 0 selected

0x01: Sector 1 selected

..

0x7F: Sector 127 selected

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bit 5 **STRT**: erase start control bit

STRT bit is used to start a sector erase or a bank erase operation. STRT can be programmed only when LOCK is cleared to 0.

STRT is reset at the end of the operation or when an error occurs. It cannot be reset by software.

Bit 4 **FW**: write forcing control bit

FW forces a write operation even if the write buffer is not full. In this case all bits not written are set to 1 by hardware. FW can be programmed only when LOCK is cleared to 0.

The memory resets FW when the corresponding operation has been acknowledged.

Note: Using a force-write operation prevents the application from updating later the missing bits with something else than 1, because it is likely that it leads to permanent ECC error.

Write forcing is effective only if the write buffer is not empty and was filled by secure access (in particular, FW does not start several write operations when the force-write operations are performed consecutively).

Since there is just one write buffer, FW can force a write in Bank1 or Bank2.

Bit 3 BER: erase request

Setting BER bit to 1 requests a bank erase operation (user flash memory only). BER can be programmed only when LOCK is cleared to 0.

If MER and SER are also set, a PGSERR is raised.

0: bank erase not requested

1: bank erase requested

Note: A write protection error is triggered when a bank erase is required and some sectors are protected.

Bit 2 SER: sector erase request

Setting SER bit to 1 requests a sector erase. SER can be programmed only when LOCK is cleared to 0.

If BER and MER are also set, a PGSERR is raised.

0: sector erase not requested

1: sector erase requested

Bit 1 PG: programming control bit

PG can be programmed only when LOCK is cleared to 0.

PG allows programming in Bank1 and Bank2.

0: programming disabled

1: programming enabled

Bit 0 LOCK: configuration lock bit

This bit locks the FLASH_SECCR register. The correct write sequence to FLASH_SECKEYR register unlocks this bit. If a wrong sequence is executed, or if the unlock sequence to FLASH_NSKEYR is performed twice, this bit remains locked until the next system reset.

LOCK can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK changes from 0 to 1, the other bits of FLASH_SECCR register do not change.

0: FLASH_SECCR register unlocked

1: FLASH_SECCR register locked

7.11.13 FLASH nonsecure clear control register (FLASH_NSSCR)

This register is nonsecure. It can be written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x030

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLR_OPTCHANGEERR	CLR_OBKWERR	CLR_OBKERR	CLR_INCERR	CLR_STRBERR	CLR_PGSEERR	CLR_WRPERR	CLR_EOP
								w	w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **CLR_OPTCHANGEERR:** Clear the flag corresponding flag in FLASH_NSSR by writing this bit.

- Bit 22 **CLR_OBKWERR**: OBKWERR flag clear bit.
Setting this bit to 1 resets to 0 OBKWERR flag in FLASH_NSSR register.
- Bit 21 **CLR_OBKERR**: OBKERR flag clear bit.
Setting this bit to 1 resets to 0 OBKERR flag in FLASH_NSSR register.
- Bit 20 **CLR_INCERR**: INCERR flag clear bit
Setting this bit to 1 resets to 0 INCERR flag in FLASH_NSSR register.
- Bit 19 **CLR_STRBERR**: STRBERR flag clear bit
Setting this bit to 1 resets to 0 STRBERR flag in FLASH_NSSR register.
- Bit 18 **CLR_PGSERR**: PGSERR flag clear bit
Setting this bit to 1 resets to 0 PGSERR flag in FLASH_NSSR register.
- Bit 17 **CLR_WRPERR**: WRPERR flag clear bit
Setting this bit to 1 resets to 0 WRPERR flag in FLASH_NSSR register.
- Bit 16 **CLR_EOP**: EOP flag clear bit
Setting this bit to 1 resets to 0 EOP flag in FLASH_NSSR register.
- Bits 15:0 Reserved, must be kept at reset value.

7.11.14 FLASH secure clear control register (FLASH_SECCCR)

This register is secure. It can be written only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x034

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLR_OBKWERR	CLR_OBKERR	CLR_INCERR	CLR_STRBERR	CLR_PGSERR	CLR_WRPERR	CLR_EOP
									w	w	w	w	w	w	w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:23 Reserved, must be kept at reset value.

- Bit 22 **CLR_OBKWERR**: OBKWERR flag clear bit
Setting this bit to 1 resets to 0 OBKWERR flag in FLASH_SECSR register.
- Bit 21 **CLR_OBKERR**: OBKERR flag clear bit
Setting this bit to 1 resets to 0 OBKERR flag in FLASH_SECSR register.
- Bit 20 **CLR_INCERR**: INCERR flag clear bit
Setting this bit to 1 resets to 0 INCERR flag in FLASH_SECSR register.
- Bit 19 **CLR_STRBERR**: STRBERR flag clear bit
Setting this bit to 1 resets to 0 STRBERR flag in FLASH_SECSR register.
- Bit 18 **CLR_PGSERR**: PGSERR flag clear bit
Setting this bit to 1 resets to 0 PGSERR flag in FLASH_SECSR register.

Bit 17 **CLR_WRPERR**: WRPERR flag clear bit

Setting this bit to 1 resets to 0 WRPERR flag in FLASH_SECSR register.

Bit 16 **CLR_EOP**: EOP flag clear bit

Setting this bit to 1 resets to 0 EOP flag in FLASH_SECSR register.

Bits 15:0 Reserved, must be kept at reset value.

7.11.15 FLASH privilege configuration register (FLASH_PRIVCFGR)

Address offset: 0x03C

Reset value: 0x0000 0000

This register can be read by both privileged and unprivileged access. NSPRIV is a nonsecure bit. SPRIV is a secure bit. It can be written only by privileged mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV	SPRIV
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **NSPRIV**: privilege attribute for non secure registers

0: access to non secure registers is always granted

1: access to non secure registers is denied in case of unprivileged access.

Bit 0 **SPRIV**: privilege attribute for secure registers

0: access to secure registers is always granted

1: access to secure registers is denied in case of unprivileged access.

7.11.16 FLASH nonsecure OBK configuration register (FLASH_NSObKCFGR)

Register is only available when TZ_STATE = 0xC3. This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can only be accessed by privilege code. This register is not accessible in write mode when the BSY bit is set. Any attempt to write to it with the BSY bit set causes the AHB bus to stall until the BSY bit is cleared.

Address offset: 0x040

Reset value: 0x01FF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWAP_OFFSET[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALT_SECT_ERASE	ALT_SECT	SWAP_SECT_REQ	LOCK
												rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:16 **SWAP_OFFSET[8:0]**: Key index (offset /16 bits) pointing for next swap.

0x00: means that no OBK is copied from current to alternate OBK sector during SWAP operation.

0x01 means that only the first OBK data (128 bits) are copied from current to alternate OBK sector

0x02 means that the two first OBK data are copied

...

0x1FF: means that all OBK data (511) are copied

Bits 15:4 Reserved, must be kept at reset value.

Bit 3 **ALT_SECT_ERASE**: alternate sector erase bit

0: do not touch OBK sector

1: erase the alternate OBK sector

When ALT_SECT bit is set, use this bit to generate an erase command for the OBK alternate sector. It is set only by Software and cleared when the OBK swap operation is completed or an error occurs (PGSERR). It is reset at the same time as BUSY bit.

Bit 2 **ALT_SECT**: alternate sector bit

0: current OBK sector is mapped to OBK address range for access

1: alternate OBK sector is mapped to OBK address range for access

This bit must not change while filling the write buffer, otherwise an error (OBKERR) is generated

Bit 1 **SWAP_SECT_REQ**: OBK swap sector request bit

0: no swap requested

1: launch the sector swap

When set, all the OBKs not updated in the alternate sector are copied from current sector to alternate one.

The SWAP_OFFSET must have a minimum value to launched the swap. The minimum value is 16 for OBK-HDPL = 1, 144 for OBK-HDPL = 2, and 192 for OBK-HDPL = 3.

Bit 0 **LOCK**: OBKCFGR lock option configuration bit

This bit locks the register, the correct write sequence unlocks it. If a wrong sequence is executed, or if the unlock sequence is performed twice, this bit remains locked until the next system reset. LOCK can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK changes from 0 to 1, the other bits of FLASH_NSCR register do not change.

0: FLASH_NSObKCFGR register unlocked

1: FLASH_NSObKCFGR register locked

7.11.17 FLASH secure OBK configuration register (FLASH_SECOBKCFGR)

This register can be accessed only in secure privileged mode. The FLASH_SECOBKCFGR register is not accessible in write mode when the BSY bit is set. Any attempt to write to it with the BSY bit set causes the AHB bus to stall until the BSY bit is cleared.

Address offset: 0x044

Reset value: 0x01FF 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWAP_OFFSET[8:0]								
							rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALT_SECT_ERASE	ALT_SECT	SWAP_SECT_REQ	LOCK
												rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bits 24:16 **SWAP_OFFSET[8:0]**: key index (offset /16 bits) pointing for next swap.

0x0: no OBK is copied from current to alternate OBK sector during SWAP operation.

0x1: only the first OBK data (128 bits) are copied from current to alternate OBK sector

0x2: the two first OBK data are copied

...

0x1FF: the 511 first OBK data are copied

Bits 15:4 Reserved, must be kept at reset value.

Bit 3 **ALT_SECT_ERASE**: alternate sector erase bit

0: do not touch OBK sector

1: erase the alternate OBK sector

When ALT_SECT bit is set, use this bit to generate an erase command for the OBK alternate sector. It is set only by Software and cleared when the OBK swap operation is completed or an error occurs (PGSERR). It is reset at the same time as the BUSY bit.

Bit 2 **ALT_SECT**: alternate sector bit

0: current OBK sector is mapped to OBK address range for access

1: alternate OBK sector is mapped to OBK address range for access

This bit must not change while filling the write buffer, otherwise an error is generated

Bit 1 **SWAP_SECT_REQ**: OBK swap sector request bit

0: no swap requested

1: launch the sector swap

When set, all the OBKs not updated in the alternate sector are copied from current sector to alternate one.

SWAP_OFFSET must have a minimum value to launch the swap. The minimum value is 16 for OBK-HDPL = 1, 144 for OBK-HDPL = 2, and 192 for OBK-HDPL = 3.

Bit 0 **LOCK**: OBKCFGR lock option configuration bit

This bit locks the FLASH_OBKCFGR register. The correct write sequence to FLASH_SECOBKKEYR register unlocks this bit. If a wrong sequence is executed, or if the unlock sequence to FLASH_SECOBKKEYR is performed twice, this bit remains locked until the next system reset. LOCK can be set by programming it to 1. When set to 1, a new unlock sequence is mandatory to unlock it. When LOCK changes from 0 to 1, the other bits of FLASH_NSCR register do not change.

0: FLASH_OBKCFGR register unlocked

1: FLASH_OBKCFGR register locked

7.11.18 FLASH HDP extension register (FLASH_HDPEXTR)

All bits in this register are nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

The register can be modified only in $HDPL \leq 2$.

The values in this register cannot be decremented. Attempts to write a value lower than the current one are ignored.

Address offset: 0x048

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_EXT[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_EXT[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HDP2_EXT[6:0]**: HDP area extension in 8 Kbytes sectors in Bank2. Extension is added after the HDP2_END sector (included).

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **HDP1_EXT[6:0]**: HDP area extension in 8 Kbytes sectors in Bank1. Extension is added after the HDP1_END sector (included).

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.19 FLASH option status register (FLASH_OPTSR_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Default value: 0x0030 5CD8 (value in case of double ECC issue during OBL).

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x050

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK	Res.	BOOT_UB[7:0]							IWDG_STDBY	IWDG_STOP	Res.	Res.	IO_VDDIO2_HSLV	IO_VDD_HSLV	
r		r	r	r	r	r	r	r	r	r			r	r	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRODUCT_STATE[7:0]								NRST_STDBY	NRST_STOP	Res.	WWDG_SW	IWDG_SW	BORH_EN	BOR_LEV[1:0]	
r	r	r	r	r	r	r	r	r	r		r	r	r	r	r

- Bit 31 **SWAP_BANK**: Bank swapping option status bit
 SWAP_BANK reflects whether Bank1 and Bank2 are swapped or not.
 SWAP_BANK is loaded to SWAP_BANK of FLASH_OPTCR after a reset.
 0: Bank1 and Bank2 not swapped
 1: Bank1 and Bank2 swapped
- Bit 30 Reserved, must be kept at reset value.
- Bits 29:22 **BOOT_UBE[7:0]**: Available only on cryptography enabled devices.
 Unique boot entry control, selects either ST or OEMiRoT for secure boot.
 0xC3: STiRoT (system flash) selected
 0xB4: OEMiRoT (user flash) selected. In Open PRODUCT_STATE this value selects bootloader. Default value.
- Bit 21 **IWDG_STDBY**: IWDG Standby mode freeze option status bit
 When set the independent watchdog IWDG is frozen in system Standby mode.
 0: Independent watchdog frozen in Standby mode
 1: Independent watchdog keep running in Standby mode.
- Bit 20 **IWDG_STOP**: IWDG Stop mode freeze option status bit
 When set the independent watchdog IWDG is in system Stop mode.
 0: Independent watchdog frozen in system Stop mode
 1: Independent watchdog keep running in system Stop mode.
- Bits 19:18 Reserved, must be kept at reset value.
- Bit 17 **IO_VDDIO2_HSLV**: High-speed IO at low V_{DDIO2} voltage configuration bit.
 This bit can be set only with V_{DDIO2} below 2.7 V.
 0: High-speed IO at low V_{DDIO2} voltage feature disabled (V_{DDIO2} can exceed 2.7 V)
 1: High-speed IO at low V_{DDIO2} voltage feature enabled (V_{DDIO2} remains below 2.7 V)
- Bit 16 **IO_VDD_HSLV**: High-speed IO at low V_{DD} voltage configuration bit.
 This bit can be set only with V_{DD} below 2.7 V.
 0: High-speed IO at low V_{DD} voltage feature disabled (V_{DD} can exceed 2.7 V)
 1: High-speed IO at low V_{DD} voltage feature enabled (V_{DD} remains below 2.7 V)
- Bits 15:8 **PRODUCT_STATE[7:0]**: Life state code (based on Hamming 8,4). See [Section 7.6.11](#).
- Bit 7 **NRST_STDBY**: Core domain Standby entry reset option status bit
 0: a reset is generated when entering Standby mode on core domain
 1: no reset generated when entering Standby mode on core domain.
- Bit 6 **NRST_STOP**: Core domain Stop entry reset option status bit
 0: a reset is generated when entering Stop mode on core domain
 1: no reset generated when entering Stop mode on core domain.
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **WWDG_SW**: WWDG control mode option status bit
 0: WWDG watchdog is controlled by hardware
 1: WWDG watchdog is controlled by software
- Bit 3 **IWDG_SW**: IWDG control mode option status bit
 0: IWDG watchdog is controlled by hardware
 1: IWDG watchdog is controlled by software
- Bit 2 **BORH_EN**: Brownout high enable
 0: disable
 1: enable

Bits 1:0 **BOR_LEV[1:0]**: Brownout level option status bit

These bits reflect the power level that generates a system reset.

00 or 11: BOR Level 1, the threshold level is low (~ 2.1 V)

01: BOR Level 2, the threshold level is medium (~ 2.4 V)

10: BOR Level 3, the threshold level is high (~ 2.7 V)

7.11.20 FLASH option status register (FLASH_OPTSR_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits. Values after reset reflect the current values of the corresponding option bits.

Address offset: 0x054

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SWAP_BANK	Res.	BOOT_UB[7:0]								IWDG_STDBY	IWDG_STOP	Res.	Res.	IO_VD DIO2_HSLV	IO_VD D_HSLV
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRODUCT_STATE[7:0]								NRST_STDBY	NRST_STOP	Res.	WWDG_SW	IWDG_SW	BORH_EN	BOR_LEV[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw

Bit 31 **SWAP_BANK**: Bank swapping option configuration bit

SWAP_BANK option bit is used to configure whether the Bank1 and Bank2 are swapped or not. This bit is loaded with the SWAP_BANK bit of FLASH_OPTSR_CUR register after a reset.

0: Bank1 and Bank2 not swapped

1: Bank1 and Bank2 swapped

Bit 30 Reserved, must be kept at reset value.

Bits 29:22 **BOOT_UB[7:0]**: Available only on cryptography enabled devices.

Unique boot entry control, selects either ST or OEMiRoT for secure boot.

0xC3: STiRoT (system flash) selected

0xB4: OEMiRoT (user flash) selected.

In Open PRODUCT_STATE this value selects bootloader. Default value.

Bit 21 **IWDG_STDBY**: IWDG Standby mode freeze option status bit

When set the independent watchdog IWDG is frozen in system Standby mode.

0: Independent watchdog frozen in Standby mode

1: Independent watchdog keep running in Standby mode.

Bit 20 **IWDG_STOP**: IWDG Stop mode freeze option status bit

When set the independent watchdog IWDG is in system Stop mode.

0: Independent watchdog frozen in system Stop mode

1: Independent watchdog keep running in system Stop mode.

Bits 19:18 Reserved, must be kept at reset value.

- Bit 17 **IO_VDDIO2_HSLV**: High-speed IO at low V_{DDIO2} voltage configuration bit.
 This bit can be set only with V_{DDIO2} below 2.7 V.
 0: High-speed IO at low V_{DDIO2} voltage feature disabled (V_{DDIO2} can exceed 2.7 V)
 1: High-speed IO at low V_{DDIO2} voltage feature enabled (V_{DDIO2} remains below 2.7 V)
- Bit 16 **IO_VDD_HSLV**: High-speed IO at low VDD voltage configuration bit.
 This bit can be set only with V_{DD} below 2.7 V.
 0: High-speed IO at low V_{DD} voltage feature disabled (V_{DD} can exceed 2.7 V)
 1: High-speed IO at low V_{DD} voltage feature enabled (V_{DD} remains below 2.7 V)
- Bits 15:8 **PRODUCT_STATE[7:0]**: Life state code (based on Hamming 8,4). See [Section 7.6.11](#).
- Bit 7 **NRST_STDBY**: Core domain Standby entry reset option configuration bit
 0: a reset is generated when entering Standby mode on core domain
 1: no reset generated when entering Standby mode on core domain.
- Bit 6 **NRST_STOP**: Core domain Stop entry reset option configuration bit
 0: a reset is generated when entering Stop mode on core domain
 1: no reset generated when entering Stop mode on core domain.
- Bit 5 Reserved, must be kept at reset value.
- Bit 4 **WWDG_SW**: WWDG control mode option configuration bit
 0: WWDG watchdog is controlled by hardware
 1: WWDG watchdog is controlled by software
- Bit 3 **IWDG_SW**: IWDG control mode option configuration bit
 0: IWDG watchdog is controlled by hardware
 1: IWDG watchdog is controlled by software
- Bit 2 **BORH_EN**: Brownout high enable configuration bit
 0: disabled
 1: enabled
- Bits 1:0 **BOR_LEV[1:0]**: Brownout level option configuration bit
 These bits reflects the power level that generates a system reset.
 00 or 11: BOR Level 1, the threshold level is low (~ 2.1 V)
 01: BOR Level 2, the threshold level is medium (~ 2.4 V)
 10: BOR Level 3, the threshold level is high (~ 2.7 V)

7.11.21 FLASH nonsecure EPOCH register (FLASH_NSEPOCHR_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x060

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NS_EPOCH[23:16]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NS_EPOCH[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:0 **NS_EPOCH[23:0]**: Nonvolatile nonsecure EPOCH counter

7.11.22 FLASH secure EPOCH register (FLASH_SECEPOCHR_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x068

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC_EPOCH[23:16]							
								r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SEC_EPOCH[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:24 Reserved, must be kept at reset value.

Bits 23:0 **SEC_EPOCH[23:0]**: Nonvolatile secure EPOCH counter

7.11.23 FLASH option status register 2 (FLASH_OPTSR2_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Default value: 0xB400 0170

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x070

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TZEN[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
r	r	r	r	r	r	r	r								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBPD _DIS	Res.	SRAM2 _ECC	SRAM3 _ECC	BKPRAM _ECC	SRAM2 _RST	SRAM13 _RST	Res.	Res.
							r		r	r	r	r	r		

Bits 31:24 **TZEN[7:0]**: TrustZone enable configuration bits

This bit enables the device is in TrustZone mode during an option byte change.

0xC3: TrustZone disabled

0xB4: TrustZone enabled.

Bits 23:15 Reserved, must be kept at reset value.

Bits 14:9 Reserved, must be kept at reset value.

Bit 8 **USBPD_DIS**: USB power delivery configuration option bit for UCPDx_CC1 pin only

This bit enables USB Type-C dead battery pull-down behavior on UCPDx_CC1 pin.

0: Enabled. For USB Type-C® and Power Delivery applications: enables correct dead-battery behavior of UCPDx_CC1 pin.

1: Disabled: Stops the dead battery pull-down behavior on the UCPDx_CC1 pin after power-up.

Bit 7 Reserved, must be kept at reset value.

Bit 6 **SRAM2_ECC**: SRAM2 ECC detection and correction disable

0: SRAM2 ECC check enabled

1: SRAM2 ECC check disabled

Bit 5 **SRAM3_ECC**: SRAM3 ECC detection and correction disable

0: SRAM3 ECC check enabled

1: SRAM3 ECC check disabled

Bit 4 **BKPRAM_ECC**: Backup RAM ECC detection and correction disable

0: BKPRAM ECC check enabled

1: BKPRAM ECC check disabled

Bit 3 **SRAM2_RST**: SRAM2 erase when system reset

0: SRAM2 erased when a system reset occurs

1: SRAM2 not erased when a system reset occurs.

Bit 2 **SRAM13_RST**: SRAM1 and SRAM3 erase upon system reset

0: SRAM1 and SRAM3 erased when a system reset occurs

1: SRAM1 and SRAM3 not erased when a system reset occurs

Bits 1:0 Reserved, must be kept at reset value.

7.11.24 FLASH option status register 2 (FLASH_OPTSR2_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits. Values after reset reflects the current values of the corresponding option bits.

Address offset: 0x074

Reset value: 0XXXXX XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TZEN[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBPD_DIS	Res.	SRAM2_ECC	SRAM3_ECC	BKPRAM_ECC	SRAM2_RST	SRAM1_3_RST	Res.	Res.
							rw		rw	rw	rw	rw	rw		

Bits 31:24 **TZEN[7:0]**: TrustZone enable configuration bits

This bit enables the device is in TrustZone mode during an option byte change.

0xC3: TrustZone disabled

0xB4: TrustZone enabled

Bits 23:15 Reserved, must be kept at reset value.

Bits 14:9 Reserved, must be kept at reset value.

Bit 8 **USBPD_DIS**: USB power delivery configuration option bit for UCPDx_CC1 pin only

This bit enables USB Type-C dead battery pull-down behavior on UCPDx_CC1 pin.

0: Enabled. For USB Type-C® and Power Delivery applications: enables correct dead-battery behavior of UCPDx_CC1 pin.

1: Disabled: Stops the dead battery pull-down behavior on the UCPDx_CC1 pin after power-up.

Bit 7 Reserved, must be kept at reset value.

Bit 6 **SRAM2_ECC**: SRAM2 ECC detection and correction disable

0: SRAM2 ECC check enabled

1: SRAM2 ECC check disabled

Bit 5 **SRAM3_ECC**: SRAM3 ECC detection and correction disable

0: SRAM3 ECC check enabled

1: SRAM3 ECC check disabled

Bit 4 **BKPRAM_ECC**: Backup RAM ECC detection and correction disable

0: BKPRAM ECC check enabled

1: BKPRAM ECC check disabled

Bit 3 **SRAM2_RST**: SRAM2 erase when system reset

0: SRAM2 erased when a system reset occurs

1: SRAM2 not erased when a system reset occurs.

Note: SRAM erase is triggered by option byte change operation, when enabling this feature.

Bit 2 **SRAM1_3_RST**: SRAM1 and SRAM3 erase upon system reset

0: SRAM1 and SRAM3 erased when a system reset occurs

1: SRAM1 and SRAM3 not erased when a system reset occurs

Note: SRAM erase is triggered by option byte change operation, when enabling this feature.

Bits 1:0 Reserved, must be kept at reset value.

7.11.25 FLASH nonsecure boot register (FLASH_NSBOOTR_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register reflects the current values of the corresponding option bits.

Address offset: 0x080

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSBOOTADD[23:8]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NSBOOTADD[7:0]								NSBOOT_LOCK[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:8 **NSBOOTADD[23:0]**: Non secure unique boot entry address
These bits reflect the Non secure UBE address

Bits 7:0 **NSBOOT_LOCK[7:0]**: Field locking the values of SWAP_BANK, and NSBOOTADD settings.
0xC3: SWAP_BANK and NSBOOTADD can still be modified following their individual rules.
0xB4: NSBOOTADD is frozen. SWAP_BANK can be modified only with TZEN set to 0xB4 (enabled).

7.11.26 FLASH nonsecure boot register (FLASH_NSBOOTR_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses, and protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x084

Reset value: 0xFFFF XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
NSBOOTADD[23:8]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
NSBOOTADD[7:0]								NSBOOT_LOCK[7:0]							
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:8 **NSBOOTADD[23:0]**: Non secure unique boot entry address
These bits allow configuring the Non secure BOOT address

Bits 7:0 **NSBOOT_LOCK[7:0]**: Field locking the values of SWAP_ BANK, and NSBOOTADD settings.
 0xC3: SWAP_ BANK and NSBOOTADD can still be modified following their individual rules.
 0xB4: NSBOOTADD is frozen. SWAP_ BANK can be modified only with TZEN set to 0xB4 (enabled).

7.11.27 FLASH secure boot register (FLASH_SECBOOTR_CUR)

This register is secure. It can be read only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

This register reflects the current values of the corresponding option bits.

Address offset: 0x088

Reset value: 0XXXXX XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECBOOTADD[23:8]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECBOOTADD[7:0]								SECBOOT_LOCK[7:0]							
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:8 **SECBOOTADD[23:0]**: Unique boot entry secure address

These bits reflect the Secure UBE address

Bits 7:0 **SECBOOT_LOCK[7:0]**: Field locking the values of UBE, SWAP_ BANK, and SECBOOTADD settings.

0xC3: BOOT_UBE, SWAP_ BANK and SECBOOTADD can still be modified following their individual rules.

0xB4: BOOT_UBE and SECBOOTADD are frozen. SWAP_ BANK can be modified only with TZEN set to 0xC3 (disabled).

7.11.28 FLASH secure boot register (FLASH_BOOTR_PRG)

This register is secure, can be read and written only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x08C

Reset value: 0XXXXX XXXX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECBOOTADD[23:8]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECBOOTADD[7:0]								SECBOOT_LOCK[7:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:8 **SECBOOTADD[23:0]**: Secure unique boot entry address.

These bits allow configuring the secure UBE address.

Bits 7:0 **SECBOOT_LOCK[7:0]**: Field locking the values of UBE, SWAP_BANK, and SECBOOTADD setting.

0xC3: BOOT_UBE, SWAP_BANK and SECBOOTADD can still be modified following their individual rules.

0xB4: BOOT_UBE and SECBOOTADD are frozen. SWAP_BANK can be modified only with TZEN set to 0xC3 (disabled).

7.11.29 FLASH nonsecure OTP block lock (FLASH_OTPBLR_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register reflects the current values of the corresponding option bits.

Address offset: 0x090

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCKBL[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LOCKBL[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **LOCKBL[31:0]**: OTP block lock

Block n corresponds to OTP 16-bit word $32 \times n$ to $32 \times n + 31$.

LOCKBL[n] = 1 indicates that all OTP 16-bit words in OTP Block n are locked and attempt to program them results in WRPERR.

LOCKBL[n] = 0 indicates that all OTP 16-bit words in OTP Block n are not locked.

When one block is locked, it is not possible to remove the write protection.

Also if not locked, it is not possible to erase OTP words.

7.11.30 FLASH nonsecure OTP block lock (FLASH_OTPBLR_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x094

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LOCKBL[31:16]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LOCKBL[15:0]															
rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs	rs

Bits 31:0 **LOCKBL[31:0]**: OTP block lockBlock n corresponds to OTP 16-bit word $32 \times n$ to $32 \times n + 31$.LOCKBL[n] = 1 indicates that all OTP 16-bit words in OTP Block n are locked and attempt to program them results in WRPERR.LOCKBL[n] = 0 indicates that all OTP 16-bit words in OTP Block n are not locked.

When one block is locked, it is not possible to remove the write protection.

LOCKBL bits can be set if the corresponding bit in FLASH_OTPBLR_CUR is cleared.

7.11.31 FLASH secure block based register for Bank1 (FLASH_SECBB1Rx)

These registers (only one for STM32H523/33xx devices, as $x = 1$) are secure. They can be read and written only by secure access. A nonsecure read/write access is RAZ/WI.

Address offset: $0x0A0 + 0x004 \times (x-1)$, ($x = 1$ to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECBB1[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECBB1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **SECBB1[31:0]**: Secure/nonsecure 8 Kbytes flash Bank1 sector attributes ($y = 0$ to 31)0: sector $(32 \times (x-1) + y)$ in Bank1 is non secure.1: sector $(32 \times (x-1) + y)$ in Bank1 is secure.

7.11.32 FLASH privilege block based register for Bank1 (FLASH_PRIVBB1Rx)

These registers (only one for STM32H523/33xx devices, as $x = 1$) are privileged. They can be read and written only by a privileged access. This register can be protected against nonsecure write access.

Address offset: $0x0C0 + 0x004 \times (x-1)$, ($x = 1$ to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRIVBB1[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRIVBB1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **PRIVBB1[31:0]**: Privileged/unprivileged 8-Kbyte flash Bank1 sector attribute (y = 0 to 31)

0: sectors $(32 * (x-1) + y)$ in Bank1 is unprivileged.

1: sector $(32 * (x-1) + y)$ in Bank1 is privileged.

7.11.33 FLASH security watermark for Bank1 (FLASH_SECWM1R_CUR)

This read-only register is nonsecure, it can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register. It reflects the current values of the corresponding option bits.

Address offset: 0x0E0

Reset value: 0x00XX 00XX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_END[6:0]						
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_STRT[6:0]						
									r	r	r	r	r	r	r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **SECWM1_END[6:0]**: Bank1 security WM area 1 end sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **SECWM1_STRT[6:0]**: Bank1 security WM area 1 start sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.34 FLASH security watermark for Bank1 (FLASH_SECWM1R_PRG)

This register is secure, can be read and written only by secure access. A nonsecure read/write access is RAZ/WI. This register can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register. This register is used to program values in corresponding option bits.

Address offset: 0x0E4

Reset value: 0x00XX 00XX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_END[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_STRT[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **SECWM1_END[6:0]**: Bank1 security WM area 1 end sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **SECWM1_STRT[6:0]**: Bank1 security WM area 1 start sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.35 FLASH write sector group protection for Bank1 (FLASH_WRP1R_CUR)

This register is nonsecure. It can be read by both secure and nonsecure accesses, and can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x0E8

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WRPSG1[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WRPSG1[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **WRPSG1[31:0]**: Bank1 sector group protection option status byte

Each FLASH_WRP1R_CUR bit reflects the write protection status of the corresponding group of four consecutive sectors in Bank1 (0: the group is write-protected; 1: the group is not write-protected)

Bit 0: Group embedding sectors 0 to 3

Bit 1: Group embedding sectors 4 to 7

Bit N: Group embedding sectors 4 x N to 4 x N + 3

Bit 31: Group embedding sectors 124 to 127

Note: In STM32H523/33xx devices only bits [7:0] are used. Consider bits [31:8] as reserved, and keep them at reset value.

7.11.36 FLASH write sector group protection for Bank1 (FLASH_WRP1R_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x0EC

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WRPSG1[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WRPSG1[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **WRPSG1[31:0]**: Bank1 sector group protection option status byte

Setting WRPSG1 bits to 0 write protects the corresponding group of four consecutive sectors in Bank1 (0: the group is write-protected; 1: the group is not write-protected)

Bit 0: Group embedding sectors 0 to 3

Bit 1: Group embedding sectors 4 to 7

Bit N: Group embedding sectors 4 x N to 4 x N + 3

Bit 31: Group embedding sectors 124 to 127

Note: In STM32H523/33xx devices only bits [7:0] are used. Consider bits [31:8] as reserved, and keep them at reset value.

7.11.37 FLASH data sector configuration Bank1 (FLASH_EDATA1R_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. The register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding options bits.

Address offset: 0x0F0

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDATA1_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_STRT[2:0]		
r													r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **EDATA1_EN**: Bank1 flash high-cycle data enable

- 0: No flash high-cycle data area
- 1: Flash high-cycle data is used

Bits 14:3 Reserved, must be kept at reset value.

Bits 2:0 **EDATA1_STRT[2:0]**:

EDATA1_STRT contains the start sectors of the flash high-cycle data area in Bank1. There is no hardware effect to those bits. They shall be managed by ST tools in Flasher.

000: The last sector of Bank1 is reserved for flash high-cycle data

001: The two last sectors of Bank1 are reserved for flash high-cycle data

010: The three last sectors of Bank1 are reserved for flash high-cycle data

...

111: The eight last sectors of Bank1 are reserved for flash high-cycle data

7.11.38 FLASH data sector configuration Bank1 (FLASH_EDATA1R_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding options bits.

Address offset: 0xF4

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDATA1_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_STRT[2:0]		
rw													rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **EDATA1_EN**: Bank1 flash high-cycle data enable

- 0: No flash high-cycle data area
- 1: Flash high-cycle data is used

Bits 14:3 Reserved, must be kept at reset value.

Bits 2:0 **EDATA1_STRT[2:0]**:

EDATA1_STRT contains the start sectors of the flash high-cycle data area in Bank1. There is no hardware effect to those bits. They must be managed by ST tools in Flasher.

000: The last sector of Bank1 is reserved for flash high-cycle data

001: The two last sectors of Bank1 are reserved for flash high-cycle data

010: The three last sectors of Bank1 are reserved for flash high-cycle data

...

111: The eight last sectors of Bank1 are reserved for flash high-cycle data

7.11.39 FLASH HDP Bank1 configuration (FLASH_HDP1R_CUR)

This register is nonsecure. It can be read by both secure and nonsecure accesses, and protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register. This register is initially loaded with the values of the corresponding option bits.

Address offset: 0x0F8

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_END[6:0]						
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_STRT[6:0]						
									r	r	r	r	r	r	r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HDP1_END[6:0]**: HDPL barrier end set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **HDP1_STRT[6:0]**: HDPL barrier start set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.40 FLASH HDP Bank1 configuration (FLASH_HDP1R_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure access, and can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

The register is accessible only in HDPL1. In HDPL2 or HDPL3 it is WI, RAZ.

This register is used to program values in corresponding option bits.

Address offset: 0x0FC

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_END[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_STRT[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HDP1_END[6:0]**: HDPL barrier end set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **HDP1_STRT[6:0]**: HDPL barrier start set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.41 FLASH ECC correction register (FLASH_ECCCORR)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x100

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	ECCC	Res.	Res.	Res.	Res.	ECCCI E	OTP_E CC	SYSF _ECC	BK _ECC	EDATA _ECC	OBK _ECC	Res.	Res.	Res.	Res.
	rw					rw	r	r	r	r	r				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR_ECC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 Reserved, must be kept at reset value.

Bit 30 **ECCC**: ECC correction set by hardware when single ECC error has been detected and corrected.

Cleared by writing 1.

Bits 29:26 Reserved, must be kept at reset value.

Bit 25 **ECCCI**: ECC single correction error interrupt enable bit

When ECCCI bit is set to 1, an interrupt is generated when an ECC single correction error occurs during a read operation.

0: no interrupt generated when an ECC single correction error occurs

1: nonsecure interrupt generated when an ECC single correction error occurs

Bit 24 **OTP_ECC**: OTP ECC error bit

This bit is set to 1 when one single ECC correction occurred during the last successful read operation from the read-only/ OTP area. The address of the ECC error is available in ADDR_ECC bitfield.

Bit 23 **SYSF_ECC**: ECC fail for corrected ECC error in system flash memory

It indicates if system flash memory is concerned by ECC error.

Bit 22 **BK_ECC**: ECC fail bank for corrected ECC error

It indicates which bank is concerned by ECC error

Bit 21 **EDATA_ECC**: ECC fail for corrected ECC error in flash high-cycle data area

It indicates if flash high-cycle data area is concerned by ECC error.

Bit 20 **OBK_ECC**: Single ECC error corrected in flash OB Keys storage area. It indicates the OBK storage concerned by ECC error.

Bits 19:16 Reserved, must be kept at reset value.

Bits 15:0 **ADDR_ECC[15:0]**: ECC error address

When an ECC error occurs (for single correction) during a read operation, ADDR_ECC contains the address that generated the error. It is reset when the flag error is reset.

The flash interface programs the address in this register only when no ECC error flags are set. This means that only the first address that generated an ECC error is saved.

The address in ADDR_ECC is relative to the flash memory area where the error occurred (user flash memory, system flash memory, data area, read-only/OTP area).

7.11.42 FLASH ECC detection register (FLASH_ECCDETR)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x104

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ECCD	Res.	Res.	Res.	Res.	Res.	Res.	OTP_E CC	SYSF _ECC	BK _ECC	EDATA _ECC	OBK _ECC	Res.	Res.	Res.	Res.
rw							r	r	r	r	r				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADDR_ECC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bit 31 **ECCD**: ECC detection

Set by hardware when two ECC error has been detected.

When this bit is set, a NMI is generated.

Cleared by writing 1. Needs to be cleared in order to detect subsequent double ECC errors.

Bits 30:25 Reserved, must be kept at reset value.

Bit 24 **OTP_ECC**: OTP ECC error bit

This bit is set to 1 when double ECC detection occurred during the last read operation from the read-only/ OTP area. The address of the ECC error is available in ADDR_ECC bitfield.

Bit 23 **SYSF_ECC**: ECC fail for double ECC error in system flash memory

It indicates if system flash memory is concerned by ECC error.

Bit 22 **BK_ECC**: ECC fail bank for double ECC error

It indicates which bank is concerned by ECC error

Bit 21 **EDATA_ECC**: ECC fail double ECC error in flash high-cycle data area

It indicates if flash high-cycle data area is concerned by ECC error.

Bit 20 **OBK_ECC**: ECC fail double ECC error in flash OB Keys storage area. It indicates the OBK storage concerned by ECC error.

Bits 19:16 Reserved, must be kept at reset value.

Bits 15:0 **ADDR_ECC[15:0]**: ECC error address

When an ECC error occurs (double detection) during a read operation, the ADDR_ECC contains the address that generated the error. It is reset when the flag error is reset.

The flash interface programs the address in this register only when no ECC error flags are set. This means that only the first address that generated an double ECC error is saved.

The address in ADDR_ECC is relative to the flash memory area where the error occurred (user flash memory, system flash memory, data area, read-only/OTP area).

7.11.43 FLASH ECC data (FLASH_ECCDR)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x108

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
DATA_ECC[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **DATA_ECC[15:0]**: ECC error data

When an double detection ECC error occurs on special areas with 6-bit ECC on 16-bit data (data, read-only/OTP), the failing data is read to this register.

By checking if it is possible to determine whether the failure was on a real data, or due to access to uninitialized memory.

7.11.44 FLASH secure block-based register for Bank2 (FLASH_SECBB2Rx)

These registers (only one for STM32H523/33xx devices, as x = 1) are secure, they can be read and written only by secure access. A nonsecure read/write access is RAZ/WI. The registers can be protected against unprivileged access when SPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x1A0 + 0x004 * (x-1), (x = 1 to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SECBB2[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SECBB2[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **SECBB2[31:0]**: Secure/nonsecure flash Bank2 sector attribute ($y = 0$ to 31)

0: sectors ($32 * (x-1) + y$) in Bank2 is non secure

1: sector ($32 * (x-1) + y$) in Bank2 is secure

7.11.45 FLASH privilege block-based register for Bank2 (FLASH_PRIVBB2Rx)

These registers (only one for STM32H523/33xx devices, as $x = 1$) are privileged, they can be read and written only by a privileged access, and can be protected against nonsecure write access by watermark.

Address offset: $0x1C0 + 0x004 * (x-1)$, ($x = 1$ to 4)

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PRIVBB2[31:16]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PRIVBB2[15:0]															
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:0 **PRIVBB2[31:0]**: Privileged/unprivileged 8-Kbyte flash Bank2 sector attribute ($y = 0$ to 31)

0: sectors ($32 * (x-1) + y$) in Bank2 is unprivileged

1: sector ($32 * (x-1) + y$) in Bank2 is privileged

7.11.46 FLASH security watermark for Bank2 (FLASH_SECWM2R_CUR)

This register is nonsecure, it can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x1E0

Reset value: 0x00XX 00XX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_END[6:0]						
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_STRT[6:0]						
									r	r	r	r	r	r	r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **SECWM2_END[6:0]**: Bank2 security WM end sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **SECWM2_STRT[6:0]**: Bank2 security WM area start sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.47 FLASH security watermark for Bank2 (FLASH_SECWM2R_PRG)

This register is secure, can be read and written only by secure access. A nonsecure read/write access is RAZ/WI. The register can be protected against unprivileged accesses when SPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x1E4

Reset value: 0x00XX 00XX

Bits 0 to 31 are loaded with values from the flash memory at OBL.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_END[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_STRT[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **SECWM2_END[6:0]**: Bank2 security WM area end sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **SECWM2_STRT[6:0]**: Bank2 security WM area start sector

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.48 FLASH write sector group protection for Bank2 (FLASH_WRP2R_CUR)

This register is nonsecure, can be read by both secure and nonsecure access, and protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding option bits.

Address offset: 0x1E8

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WRPSG2[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WRPSG2[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **WRPSG2[31:0]**: Bank2 sector group protection option status byte

Each bit reflects the write protection status of the corresponding group of four consecutive sectors in Bank2 (0: group is write-protected; 1: group is not write-protected)

Bit 0: Group embedding sectors 0 to 3

Bit 1: Group embedding sectors 4 to 7

Bit N: Group embedding sectors $4 \times N$ to $4 \times N + 3$

Bit 31: Group embedding sectors 124 to 127

Note: In STM32H523/33xx devices only bits [7:0] are used. Consider bits [31:8] as reserved, and keep them at reset value.

7.11.49 FLASH write sector group protection for Bank2 (FLASH_WRP2R_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This register is used to program values in corresponding option bits.

Address offset: 0x1EC

Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WRPSG2[31:16]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WRPSG2[15:0]															
r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:0 **WRPSG2[31:0]**: Bank2 sector group protection option status byte

Setting WRPSGn2 bits to 0 write protects the corresponding group of four consecutive sectors in Bank2 (0: group is write-protected; 1: group is not write-protected)

Bit 0: Group embedding sectors 0 to 3

Bit 1: Group embedding sectors 4 to 7

Bit N: Group embedding sectors $4 \times N$ to $4 \times N + 3$

Bit 31: Group embedding sectors 124 to 127

Note: In STM32H523/33xx devices only bits [7:0] are used. Consider bits [31:8] as reserved, and keep them at reset value.

7.11.50 FLASH data sectors configuration Bank2 (FLASH_EDATA2R_CUR)

This register is nonsecure, can be read by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

This read-only register reflects the current values of the corresponding options bits.

Address offset: 0x1F0

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDATA2_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA2_STRT[2:0]		
r													r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **EDATA2_EN**: Bank2 flash high-cycle data enable

0: No flash high-cycle data area

1: Flash high-cycle data is used

Bits 14:3 Reserved, must be kept at reset value.

Bits 2:0 **EDATA2_STRT[2:0]**:

EDATA2_STRT contains the start sectors of the flash high-cycle data area in Bank2. There is no hardware effect to those bits. They shall be managed by ST tools in Flasher.

000: The last sector of Bank2 is reserved for flash high-cycle data

001: The two last sectors of Bank2 are reserved for flash high-cycle data

010: The three last sectors of Bank2 are reserved for flash high-cycle data

...

111: The eight last sectors of Bank2 are reserved for flash high-cycle data

7.11.51 FLASH data sector configuration Bank2 (FLASH_EDATA2R_PRG)

This register is nonsecure. It can be read and written by both secure and nonsecure accesses, and protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register. It is used to program values in corresponding options bits.

Address offset: 0x1F4

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
EDATA2_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA2_STRT[2:0]		
rw													rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **EDATA2_EN**: Bank2 flash high-cycle data enable

0: No flash high-cycle data area

1: Flash high-cycle data is used

Bits 14:3 Reserved, must be kept at reset value.

Bits 2:0 **EDATA2_STRT[2:0]**:

EDATA2_STRT contains the start sectors of the flash high-cycle data area in Bank2. There is no hardware effect to those bits. They shall be managed by ST tools in Flasher.

000: The last sector of Bank2 is reserved for flash high-cycle data.

001: The two last sectors of Bank2 are reserved for flash high-cycle data.

010: The three last sectors of Bank2 are reserved for flash high-cycle data.

...

111: The eight last sectors of Bank2 are reserved for flash high-cycle data.

7.11.52 FLASH HDP Bank2 configuration (FLASH_HDP2R_CUR)

This register is initially loaded with the values of the corresponding option bits. This register is nonsecure. It can be read by both secure and nonsecure accesses, and can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Address offset: 0x1F8

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_END[6:0]						
									r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_STRT[6:0]						
									r	r	r	r	r	r	r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HDP2_END[6:0]**: HDPL barrier end set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **HDP2_STRT[6:0]**: HDPL barrier start set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.11.53 FLASH HDP Bank2 configuration (FLASH_HDP2R_PRG)

This register is nonsecure. It can be read and written only by both secure and nonsecure accesses. This register can be protected against unprivileged access when NSPRIV = 1 in the FLASH_PRIVCFGR register.

Register is accessible only in HDPL1. In HDPL2 or HDPL3 it is WI, RAZ.

This register is used to program values in corresponding option bits.

Address offset: 0x1FC

Reset value: 0xFFFF XXXX

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_END[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_STRT[6:0]						
									rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HDP2_END[6:0]**: HDPL barrier end set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

Bits 15:7 Reserved, must be kept at reset value.

Bits 6:0 **HDP2_STRT[6:0]**: HDPL barrier start set in number of 8-Kbyte sectors

Note: For STM32H523/33xx devices the maximum value is 0x1F, consider bits [6:5] as reserved, and keep them at reset value.

7.12 FLASH register map and reset values

Table 90. Register map and reset value table

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	FLASH_ACR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PRFTEN	Res.	Res.	WRHIGHFREQ [1:0]	LATENCY [3:0]				
	Reset value																								0			0	1	0	0	1	1
0x004	FLASH_NSKEYR	NSKEY[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0
0x008	FLASH_SECKEYR	SECKEY[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00C	FLASH_OPTKEYR	OPTKEY[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x010	FLASH_NSGBKKEYR	NSGBKKEY[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	FLASH_SECOBKKEYR	SECOBKKEY[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018	FLASH_OPSR	CODE_OP [2:0]			Res.	Res.	Res.	Res.	OTP_OP	SYSF_OP	BK_OP	DATA_OP	Res.	ADDR_OP[19:0]																			
	Reset value	X	X	X					X	X	X	X		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x01C	FLASH_OPTCR	SWAP_BANK	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	0																															
0x020	FLASH_NSSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPTCHANGER	OBKWERR	OBKERR	INCERR	STRBERR	PGSERR	WRPERR	EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SDBNE	Res.	Res.
	Reset value									0	0	0	0	0	0	0	0													0		0	0
0x024	FLASH_SECSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OBKWERR	OBKERR	INCERR	STRBERR	PGSERR	WRPERR	EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBNE	Res.	Res.
	Reset value										0	0	0	0	0	0	0													X		X	X
0x028	FLASH_NSCR	BKSEL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OPTCHANGER	OBKWIE	OBKIE	INCERRIE	STRBERRIE	PGSERRIE	WRPERRIE	EOPIE	MER	Res.	Res.	SNB[6:0]						STRT	FW	BER	SER	PG	LOCK	
	Reset value	0									0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	1

Table 90. Register map and reset value table (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0		
0x02C	FLASH_SECCR	BKSEL	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OBKWE	OBKERRIE	INCERRIE	STRBERRIE	PGSERRIE	WRPERRIE	EOPIE	MER	Res.	Res.	SNB[6:0]						STRT	FW	BER	SER	PG	LOCK			
	Reset value	0	0								0	0	0	0	0	0	0	0	0			0	0	0	0	0	0	0	0	0	0	0	0	1	
0x030	FLASH_NSCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLR_OPTCHANGEERR	CLR_OBKWERR	CLR_OBKERR	CLR_INCERR	CLR_STRBERR	CLR_PGSERR	CLR_WRPERR	CLR_EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
	Reset value									0	0	0	0	0	0	0	0	0																	
0x034	FLASH_SECCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CLR_OBKWERR	CLR_OBKERR	CLR_INCERR	CLR_STRBERR	CLR_PGSERR	CLR_WRPERR	CLR_EOP	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.		
	Reset value										0	0	0	0	0	0	0	0																	
0x03C	FLASH_PRIVCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV	SPRIV			
	Reset value																														0	0			
0x040	FLASH_NSOBKCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWAP_OFFSET[8:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALT_SECT_ERASE	ALT_SECT	SWAP_SECT_REQ	LOCK		
	Reset value								1	1	1	1	1	1	1	1	1														0	0	0		
0x044	FLASH_SECOBKCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SWAP_OFFSET[8:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALT_SECT_ERASE	ALT_SECT	SWAP_SECT_REQ	LOCK
	Reset value								1	1	1	1	1	1	1	1	1														0	0	0		
0x048	FLASH_HDPEXTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_EXT[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_EXT[6:0]							
	Reset value										0	0	0	0	0	0	0											0	0	0	0	0	0		
0x04C	Reserved	Reserved																																	
0x050	FLASH_OPTSR_CUR	SWAP_BANK	BOOT_UBE[7:0]								IWDG_STDBY	IWDG_STOP	Res.	Res.	IO_VDDIO2_HSLV	IO_VDD_HSLV	PRODUCT_STATE[7:0]								NRST_STDBY	NRST_STOP	Res.	WWDG_SW	IWDG_SW	BORH_LEN	BOR_LEV[1:0]				
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X		X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X		

Table 90. Register map and reset value table (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x054	FLASH_OPTSR_PRG	SWAP_BANK	Res.	BOOT_UBE[7:0]								IWDG_STDBY	IWDG_STOP	Res.	Res.	IO_VDDIO2_HSLV	IO_VDD_HSLV	PRODUCT_STATE[7:0]								NRST_STDBY	NRST_STOP	Res.	WWDG_SW	IWDG_SW	BORH_EN	BOR_LEV[1:0]		
	Reset value	X		X	X	X	X	X	X	X	X	X	X			X	X	X	X	X	X	X	X	X	X	X	X	X		X	X	X	X	X
0x058	Reserved	Reserved																																
0x05C	Reserved	Reserved																																
0x060	FLASH_NSEPOCHR_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NS_EPOCH[23:0]																							
	Reset value										X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x064	FLASH_NSEPOCHR_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NS_EPOCH[23:0]																							
	Reset value										X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x068	FLASH_SECEPOCHR_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC_EPOCH[23:0]																							
	Reset value										X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x06C	FLASH_SECEPOCHR_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SEC_EPOCH[23:0]																							
	Reset value										X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x070	FLASH_OPTSR2_CUR	TZEN[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	X	X	X	X	X	X	X	X																	X		X	X	X	X	X		
0x074	FLASH_OPTSR2_PRG	TZEN[7:0]								Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	X	X	X	X	X	X	X	X																	X		X	X	X	X	X		
0x080	FLASH_NSBOOTR_CUR	NSBOOTADD[23:0]														NSBOOT_LOCK[7:0]																		
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x084	FLASH_NSBOOTR_PRG	NSBOOTADD[23:0]														NSBOOT_LOCK[7:0]																		
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x088	FLASH_SECBOOTR_CUR	SECBOOTADD[23:0]														SECBOOT_LOCK[7:0]																		
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x08C	FLASH_SECBOOTR_PRG	SECBOOTADD[23:0]														SECBOOT_LOCK[7:0]																		
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x090	FLASH_OTPBLR_CUR	LOCKBL[31:0]																																
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X

Table 90. Register map and reset value table (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x094	FLASH_OTPBLR_PRG	LOCKBL[31:0]																															
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x0A0 + 0x004 * x (x = 1 to 4)	FLASH_SECBB1R_x	SECBB1[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0C0 + 0x004 * x (x = 1 to 4)	FLASH_PRIVBB1R_x	PRIVBB1[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0D0 - 0x0DC	Reserved	Reserved																															
0x0E0	FLASH_SECWM1R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_STRT[6:0]							
	Reset value										X	X	X	X	X	X	X										X	X	X	X	X	X	
0x0E4	FLASH_SECWM1R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_END1[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM1_STRT[6:0]							
	Reset value										X	X	X	X	X	X	X										X	X	X	X	X	X	
0x0E8	FLASH_WRP1R_CUR	WRPSG1[31:0]																															
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x0EC	FLASH_WRP1R_PRG	WRPSG1[31:0]																															
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	
0x0F0	FLASH_EDATA1R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_STRT [2:0]	
	Reset value																	X													X	X	X
0x0F4	FLASH_EDATA1R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA1_STRT [2:0]
	Reset value																	X														X	X
0x0F8	FLASH_HDP1R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_STRT[6:0]							
	Reset value										0	0	0	0	0	0	0										0	0	0	0	0	0	1
0x0FC	FLASH_HDP1R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP1_STRT[6:0]							
	Reset value										X	X	X	X	X	X	0										X	X	X	X	X	X	X
0x100	FLASH_ECCCORR	Res.	ECCC	Res.	Res.	Res.	Res.	ECC0IE	OTP_ECC	SYSF_ECC	BK_ECC	EDATA_ECC	OBK_ECC	Res.	Res.	Res.	Res.	ADDR_ECC[15:0]															
	0x0000 0000		0						0	0	0		0					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Table 90. Register map and reset value table (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x104	FLASH_ECCDETR	ECCD	Res.	Res.	Res.	Res.	Res.	Res.	OTP_ECC_FAIL	SYSE_ECC	BK_ECC	EDATA_ECC	OBK_ECC	Res.	Res.	Res.	Res.	ADDR_ECC[15:0]															
	Reset value	0							0	0	0	0	0					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x108	FLASH_ECCDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DATA_ECC[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1A0 + 0x004 * x (x = 1 to 4)	FLASH_SECBB2R_x	SECBB2[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1A0 + 0x004 * x (x = 1 to 4)	FLASH_PRIVBB2R_x	PRIVBB2[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x1D0 - 0x1DC	Reserved	Reserved																															
0x1E0	FLASH_SECWM2R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_STRT[6:0]							
	Reset value									X	X	X	X	X	X	X	X										X	X	X	X	X	X	X
0x1E4	FLASH_SECWM2R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_END1[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SECWM2_STRT[6:0]						
	Reset value									X	X	X	X	X	X	X	X										X	X	X	X	X	X	X
0x1E8	FLASH_WRP2R_CUR	WRPSG2[31:0]																															
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x1EC	FLASH_WRP2R_PRG	WRPSG2[31:0]																															
	Reset value	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X
0x1F0	FLASH_EDATA2R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA2_EN											EDATA2_STRT [2:0]					
	Reset value																X														X	X	X
0x1F4	FLASH_EDATA2R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	EDATA2_EN											EDATA2_STRT [2:0]					
	Reset value																X														X	X	X
0x1F8	FLASH_HDP2R_CUR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_STRT[6:0]							
	Reset value									X	X	X	X	X	X	X	X									X	X	X	X	X	X	X	X
0x1FC	FLASH_HDP2R_PRG	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_END[6:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HDP2_STRT[6:0]							
	Reset value									X	X	X	X	X	X	X	X									X	X	X	X	X	X	X	X

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

8 Instruction cache (ICACHE)

8.1 ICACHE introduction

The instruction cache (ICACHE) is implemented on the C-AHB code bus of the Cortex[®]-M33 processor to improve performance when fetching instructions and data from internal and external memories.

Some specific features, like dual master ports, hit-under-miss, and critical-word-first refill policy, result in close to zero-wait-state performance in most use cases.

8.2 ICACHE main features

The main features of ICACHE are listed below:

- Bus interface
 - One 32-bit AHB slave port, the execution port (input from Cortex[®]-M33 C-AHB code interface)
 - Two AHB master ports: master1 and master2 ports, with outputs to Fast and Slow buses of main AHB bus matrix, respectively
 - One 32-bit AHB slave port for control (input from AHB peripherals interconnect, for ICACHE registers access)
- Cache access
 - 0 wait-state on hits
 - Hit-under-miss capability: ability to serve processor requests (access to cached data) during an ongoing line refill due to a previous cache miss
 - Dual master access: feature used to decouple the traffic according to targeted memory. For example, the ICACHE assigns fast traffic (addressing flash memories and SRAMs) to the AHB master1 port, and slow traffic (addressing external memories) to the AHB master2 port, thus preventing processor stalls on line refills from external memories. This allows ISR (interrupt service routine) fetching on internal flash memory to take place in parallel with a cache line refill from external memories.
 - Minimal impact on interrupt latency, thanks to dual master
 - Optimal cache line refill thanks to WRAPw bursts of the size of the cache line (32-bit word size, **w**, aligned on cache line size)
 - n-way set-associative default configuration with possibility to configure as 1-way, means direct mapped cache, for applications needing very-low-power consumption profile
- Memory address remap
 - Possibility to remap input address falling into up to four memory regions (used to remap aliased code in external memories to the code region, for execution from the C-AHB code interface)
- Replacement and refill
 - pLRU-t (pseudo-least-recently-used, based on binary tree) replacement policy, algorithm with best complexity/performance balance
 - Critical-word-first refill policy, minimizing processor stalls

- Possibility to configure burst type of AHB memory transaction for remapped regions: INCRw or WRAPw (size w aligned on cache line size)
- Performance counters

The ICACHE implements two performance counters:

 - Hit monitor counter (32-bit)
 - Miss monitor counter (16-bit)
- Error management
 - Possibility to detect an unexpected cacheable write access, to flag an error, and optionally, to raise an interrupt
- TrustZone® security support
- Maintenance operation
 - Cache invalidate: full cache invalidation, fast command, noninterruptible

8.3 ICACHE implementation

Table 91. ICACHE features

Feature	ICACHE
Number of ways	2
Cache size	8 Kbytes
Cache line width	16 bytes
Range granularity of memory regions to be remapped	2 Mbytes
Number of regions to remap	4
Data size of AHB slave interface	32 bits
Data size of AHB fast master1 interface	128 bits
Data size of AHB slow master2 interface	32 bits

8.4 ICACHE functional description

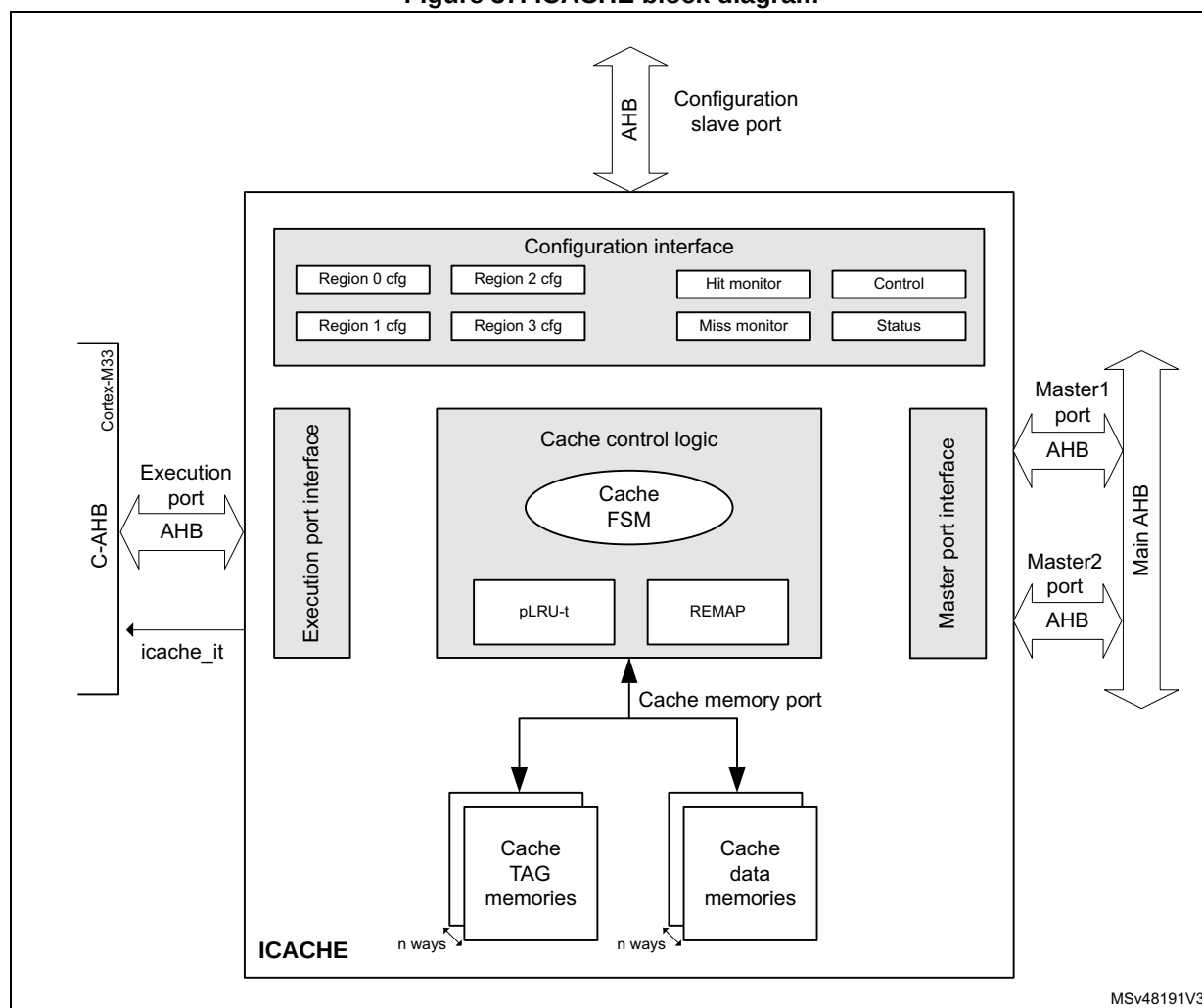
The purpose of this block is to cache instruction fetches or instruction memories loads, coming from the processor. As such, the ICACHE manages only cacheable read transactions, and does not manage cacheable write transactions.

The noncacheable transactions (both read and write) bypass the ICACHE.

For the error management purpose, if a write cacheable transaction is presented (this happens only in case of bad software programming), the ICACHE sets an error flag and, if enabled, raises an interrupt to the processor.

8.4.1 ICACHE block diagram

Figure 37. ICACHE block diagram



8.4.2 ICACHE reset and clocks

The ICACHE is clocked on the Cortex[®]-M33 C-AHB bus clock.

When the ICACHE reset signal is released, a cache invalidate procedure is automatically launched, making the ICACHE busy (ICACHE_SR = 0x0000 0001).

When this procedure is finished:

- the ICACHE is invalidated: “cold cache”, with all cache line valid bits = 0 (ICACHE must be filled up)
- ICACHE_SR = 0x0000 0002 (reflecting the cache is no longer busy)
- the ICACHE is disabled: the EN bit in ICACHE_CR holds its reset state (= 0).

Note: When disabled, the ICACHE is bypassed, except the remapping mechanism that is still functional: slave input requests (remapped or not) are forwarded to the master port(s).

8.4.3 ICACHE TAG memory

The ICACHE TAG memory contains:

- address tags indicating which data are contained in the cache data memories
- validity bits

There is one valid bit per cache line (per way).

The valid bit is set when a cache line is refilled (after a miss).

Valid bits are reset in any of the below cases:

- after the ICACHE reset is released
- when the cache is disabled, by setting EN = 0 in ICACHE_CR (by software)
- when executing an ICACHE invalidate command, by setting CACHEINV = 1 in ICACHE_CR (by software)

When a cacheable transaction is received at the execution input port, its AHB address (HADDR_in) is split into the following fields (see [Table 92](#) for B and W definitions):

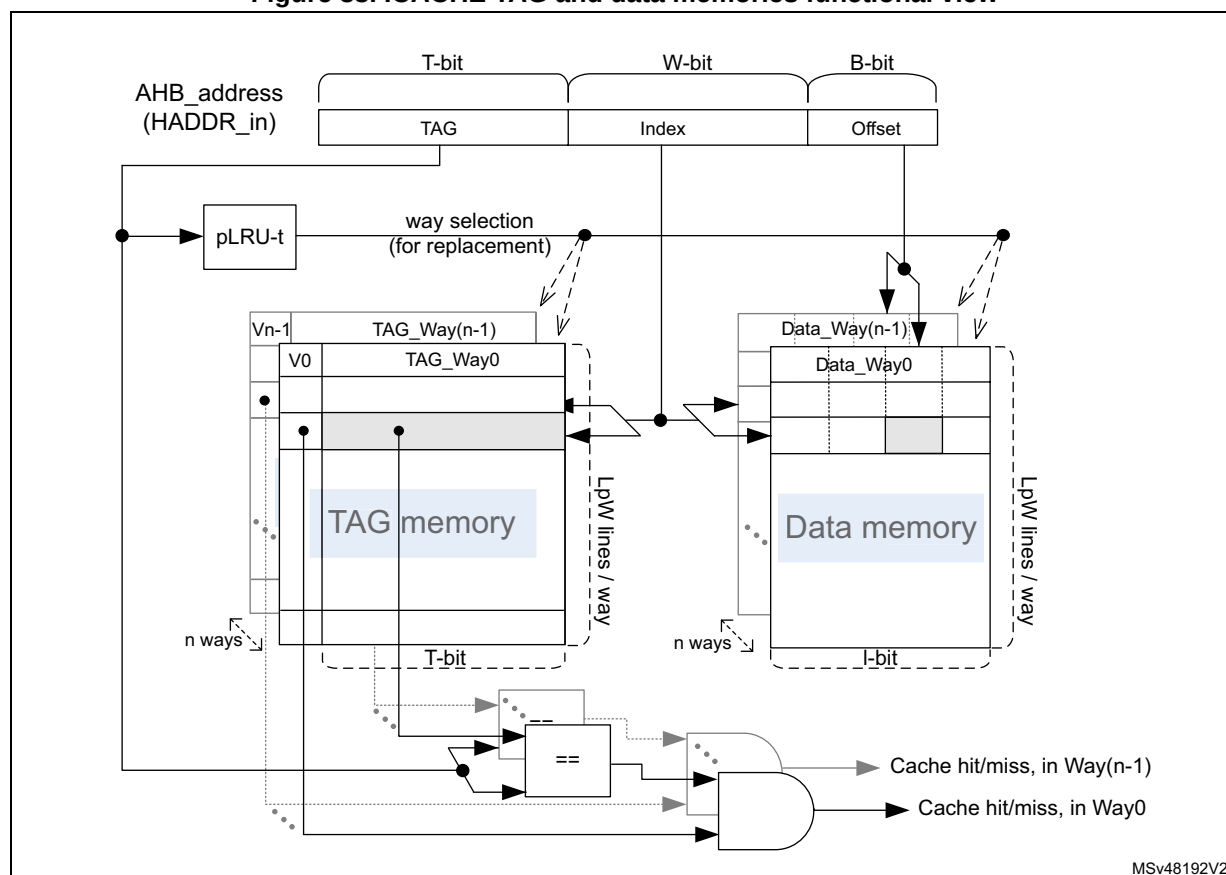
- HADDR_in[B-1:0]: address byte offset, indicates which byte to select inside a cache line.
- HADDR_in[B+W-1:B]: address way index, indicates which cache line to select inside each way.
- HADDR_in[31:B+W]: tag address, to be compared to the TAG memory address to check if the requested data is already available (meaning valid) inside the ICACHE.

The following table gives a summary of the ICACHE main parameters for TAG memory dimensioning. [Figure 38](#) shows the functional view of TAG and data memories, for an n-way set associative ICACHE.

**Table 92. TAG memory dimensioning parameters
for n-way set associative operating mode (default)**

Parameter	Value	Example
Cache size	S Kbytes = s bytes ($s = 1024 \times S$)	8 Kbytes = 8192 bytes
Cache number of ways	n	2
Cache line size	L-byte = l-bit ($l = 8 \times L$)	16-byte = 128-bit
Number of cache lines	$LpW = s / (n \times L)$ lines / way	256 lines / way
Address byte offset size	$B = \log_2(L)$ bit	4-bit
Address way index size	$W = \log_2(LpW)$ bit	8-bit
TAG address size	$T = (32 - W - B)$ bit	20-bit

Figure 38. ICACHE TAG and data memories functional view



MSv48192V2

8.4.4 Direct-mapped ICACHE (1-way cache)

The default configuration (at reset) is an n-way set associative cache (WAYSEL = 1 in ICACHE_CR), but the user can configure the ICACHE as direct mapped by writing WAYSEL = 0 (only possible when the cache is disabled, EN = 0 in ICACHE_CR).

The following table gives a summary of ICACHE main parameters for TAG memory when the direct-mapped cache operating mode is selected.

Table 93. TAG memory dimensioning parameters for direct-mapped cache mode

Parameter	Value	Example
Cache size	S Kbytes = s bytes (s = 1024 x S)	8 Kbytes = 8192 bytes
Cache number of ways	1	1
Cache line size	L-byte = l-bit (l = 8 x L)	16-byte = 128-bit
Number of cache lines	LpW = s / L lines	512 lines
Address byte offset size	B = $\log_2(L)$ bit	4-bit
Address way index size	W = $\log_2(LpW)$ bit	9-bit
TAG address size	T = (32 - W - B) bit	19-bit

All cache operations (such as read, refill, remapping, invalidation) remain the same in the direct-mapped configuration. The only difference is the absence of a replacement algorithm in case of line eviction (as explained in [Section 8.4.8](#)): only one way (the unique one) is possible for any data refill.

8.4.5 ICACHE enable

To activate the ICACHE, the EN bit in ICACHE_CR must be set to 1.

When the ICACHE is disabled, it is bypassed and all transactions are copied from the slave to the master ports in the same clock cycle.

It is recommended to initialize or modify the main memory content (region to be later cached) with the ICACHE disabled, and to enable the ICACHE only when this region remains unchanged (an enabled ICACHE detects cacheable write transactions as errors).

To ensure performance determinism, it is recommended to wait for the end of a potential cache invalidate procedure before enabling the ICACHE. This procedure occurs when the hardware reset signal is released, when CACHEINV is set, or when EN is cleared in ICACHE_CR. During the procedure, BUSYF is set in ICACHE_SR, and once finished, BUSYF is cleared and BSYENDF is set in the same register (raising the ICACHE interrupt if enabled on such a busy end condition).

The software must test BUSYF and/or BSYENDF values before enabling the ICACHE. Else, if the ICACHE is enabled before the end of an invalidate procedure, any cache access (while BUSYF = 1) is treated as noncacheable, and its performance depends on the main memory access time.

The address remapping is performed, whether the ICACHE is enabled or not, if the input transaction address falls into the memory regions defined and enabled in ICACHE_CRRx (see [Figure 39](#)).

The ICACHE is, by default, disabled at boot.

8.4.6 Cacheable and noncacheable traffic

The ICACHE is developed for the Cortex®-M33 core. It is placed on the C-AHB bus, and thus caches the code memory region, ranging from address 0x0000 0000 to 0x1FFF FFFF of the memory map.

To make other memory regions cacheable, the ICACHE supports a memory region remapping feature, used to define up to four external memory regions, whose addresses have an alias in the code region. Addressing these memory regions through their code alias address allows the memory request to be routed to the C-AHB bus, and to be managed by the ICACHE.

Any external memory space physically mapped at an address in the range [0x6000 0000:0xAFFF FFFF] or [0xC000 0000:0xDFFF FFFF] can be aliased with an address in the range [0x0000 0000:0x07FF FFFF] or [0x1000 0000:0x1FFF FFFF].

For a given memory request in the code region, the ICACHE implements the address remapping functionality first. If aliased, it is the remapped address, which is then cached, and, if needed, provided to the master port to address the main AHB bus matrix. The destination physical address does not need further manipulation on the AHB bus.

The remapping functionality is available also for noncacheable traffic, and when the cache is disabled.

Additional details on address remapping are provided in [Section 8.4.7](#).

An incoming memory request to the ICACHE is defined as cacheable according to its AHB transaction memory lookup attribute, as shown in [Table 94](#). This AHB attribute depends on the MPU (memory protection unit) programming for the addressed region.

Table 94. ICACHE cacheability for AHB transaction

AHB lookup attribute	Cacheability
1	Cacheable
0	Noncacheable

In the case of a noncacheable access (read or write), the ICACHE is bypassed. The AHB transaction is propagated unchanged to the master output port, except the transaction address, which may be modified due to the address remapping feature (see [Section 8.4.7](#)).

The bypass, and eventual remap logic, does not increase the latency of the access to the targeted memory.

In the case of a cacheable access, the ICACHE behaves as explained in [Section 8.4.8](#).

Cacheable memory regions are defined and programmed by the user in the MPU responsible for the generation of the AHB attribute signals (HPROT[6:0]) for any transaction addressing a given region.

[Table 95](#) summarizes programmable configurations of various memories.

Table 95. Memory configurations

Memory	Cacheable (MPU programming)	Remapped in the ICACHE (ICACHE_CRRx programming)
Flash	Yes or no	Not required
SRAM	Not recommended	Not required
External memories	Yes	Required
	No	Required if the user wants code in external memories fetched on C-AHB bus (else on S-AHB bus)

8.4.7 Address remapping

The ICACHE allows an alias address to be defined in the code region for up to four external memory regions.

The address remapping is applied on the code alias address, transforming it into the destination external physical address.

The remapping operation is fully software configurable by programming ICACHE_CRRx (x = 0 to 3, number of remapped regions). This programming can be done only when the ICACHE is disabled.

Each region x can be individually enabled with REN in ICACHE_CRRx. Once enabled, the remap operation occurs even if the ICACHE is disabled, or if the transaction is noncacheable.

Remap regions can have different size: each can be programmed in RSIZE of its ICACHE_CRRx. The size of each region is a power of two multiple of range granularity (2 Mbytes), with a minimum of 2 Mbytes, and a maximum of 128 Mbytes.

The address remapping mechanism is based on the matching of an incoming AHB address (HADDR_in) with a given code subregion base-address, and the modification of this address into its (remapped) physical address, as follows:

- HADDR_in belongs to region x if $\text{HADDR_in}[31:RI] = 000:\text{BASEADDR}[28:RI]$, where:
 - 000:BASEADDR is the code subregion base-address programmed in BASEADDR of ICACHE_CRRx.
 - RI defines the number of significant bits to consider. $RI = \log_2(\text{region size})$ with a minimum value of 21 (for a 2-Mbyte region) and a maximum value of 27 (for a 128-Mbyte region).
- If the region x is enabled, the master port output AHB address (HADDR_out) is composed by concatenating the two below parts:
 - REMAPADDR[31:RI] field of ICACHE_CRRx as MSBs
 - HADDR_in[RI-1:0] as LSBs

Figure 39 describes the matching and the output address generation.

Figure 39. ICACHE remapping address mechanism

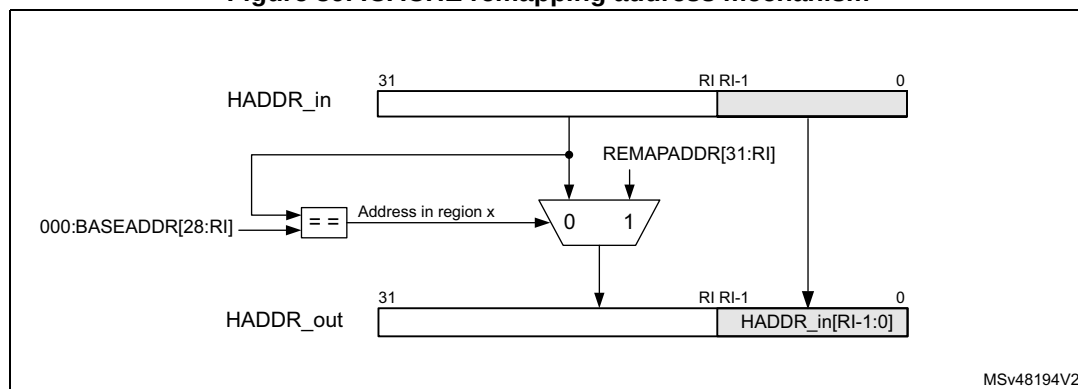


Table 96 summarizes all possible configurations of BASEADDR and REMAPADDR sizes (number of significant MSBs) in ICACHE_CRRx, depending upon RSIZE.

Table 96. ICACHE remap region size, base address, and remap address

Region size (Mbytes)	Base address size (MSBs)	Remap address (MSBs)
2	8	11
4	7	10
8	6	9
16	5	8
32	4	7
64	3	6
128	2	5

Care must be taken while programming BASEADDR and REMAPADDR in ICACHE_CRRx: if the programmed value is bigger than expected (number of MSBs, see [Table 96](#)), the unneeded extra LSBs are ignored.

Typical remapping example: a 128-Mbyte external RAM region physically located in the external address range [0x9000 0000:0x97FF FFFF], remapped in the code section range [0x1800 0000:0x1FFF FFFF]:

- REMAPADDR[31:21] = 0x480
- BASEADDR[28:21] = 0xC0
- HADDR_in[31:27] is compared to 000:BASEADDR[28:27], while HADDR_in and BASEADDR[26:21] are ignored.

If the comparison matches:

- HADDR_out[31:27] gets REMAPADDR[31:27] (in place of HADDR_in[31:27])
- HADDR_out[26:0] gets HADDR_in[26:0]

The software can program the kind of AHB burst generated by the ICACHE master ports on the bus matrix (for cache line refill), by setting HBURST in ICACHE_CRRx with:

- WRAP for remapped external memories accessed through interfaces that can support WRAP burst mode, providing the benefit of the critical-word-first feature performance:
 - WRAP burst size = cache line size
 - WRAP burst start address = word address of the first data requested by the core
- INCR: INCR burst mode for external memories accessed through the interfaces that do not support WRAP burst mode (losing the benefit of critical-word-first feature):
 - INCR burst size = cache line size
 - INCR burst start address = address aligned on the boundary of the cache line containing the requested word.

Note: Coherency is needed when programming the SAU (secure attribution unit) and the MPU (memory protection unit) attributes for both the external regions and their aliased code subregions.

8.4.8 Cacheable accesses

When the ICACHE receives a cacheable transaction from the Cortex®-M33, it checks if the address requested is present in its TAG memory, and if the corresponding cache line is valid.

There are then three alternatives:

- The address is present inside the TAG memory, the cache line is valid: **cache hit**, the data is read from the cache and provided to the processor in the same cycle.
- The address is not present in the TAG memory: **cache miss**, the data is read from the main memory and provided to the processor, and a cache line refill is performed.

The critical-word-first policy ensures minimum wait cycles for the processor, since read data can be provided while the cache still performs a cache line refill (associated latency is the latency of fetching one word from the main memory).

The burst generated on the ICACHE master bus is WRAPw (w being the cache line width, in words) if no address remap occurs. If an address remap occurs, the kind of burst depends upon the HBURST programmed in corresponding ICACHE_CRRx.

The AHB transaction attributes are also propagated to the main AHB bus matrix on the master port selected for the line refill.

- The address is not present in TAG memory, but belongs to the refill burst from the main memory currently ongoing: **cache hit** (hit-under-miss feature).
This happens during cache-line refill. The ICACHE can provide the requested data as soon as data are available at its master interface, thus avoiding a miss (fetching data from the main memory).

In the case of cache refill (due to cache miss), the ICACHE selects which cache line is written with the refill data:

- In direct map (1-way) mode, only one line can be used to store the refill data: the line pointed by the index of the input address.
- In n-way set associative mode, one line among n can be used (the line pointed by the address index, in each of the n ways). The way selection is based on a pLRU-t replacement algorithm that points, for each index, on the way candidate for the next refill.

If the cache line where the refill data must be written is already valid, the targeted cache line must be invalidated first. This is true whatever the direct map or n-way set associative cache mode.

8.4.9 Dual-master cache

The ICACHE implements a dual-port AHB master on the main AHB bus matrix: master1 and master2 ports. This is used to split the traffic going to different destination memories.

The nonremapped traffic goes systematically to master1 port. The remapped traffic can be routed on the master2 port by programming MSTSEL in ICACHE_CRRx (on a region basis).

The code can typically be fetched as follows:

- internal flash memory and internal SRAM on master1 port (Fast bus)
- external flash memory/RAM on master2 port (Slow bus)

For systems not implementing external memories, the traffic to the internal flash memory can be decoupled from the traffic to the internal SRAM (when remapped by the ICACHE). This feature is used to prevent further processor stalls on misses.

Alongside with hit-under-miss, this dual-master feature allows the processor to have an alternative path in case of fetching from different memories.

8.4.10 ICACHE security

The ICACHE implements an Armv8-M TrustZone.

ICACHE configuration registers are protected at system level.

8.4.11 ICACHE maintenance

The software can invalidate the whole content of the ICACHE by programming CACHEINV in the ICACHE_CR register.

When CACHEINV = 1, the ICACHE control logic sets the BUSYF flag in ICACHE_SR and launches the invalidate cache operation, resetting each TAG valid bit to 0 (one valid bit per cache line). CACHEINV is automatically cleared.

Once the invalidate operation is finished (all valid bits reset to 0), the ICACHE automatically clears BUSYF, and sets BSYENDF in the ICACHE_SR register.

If enabled on this flag condition (BSYENDIE = 1 in ICACHE_IER), the ICACHE interrupt is raised. Then, the (empty) cache is available again.

8.4.12 ICACHE performance monitoring

The ICACHE provides the following monitors for performance analysis:

- The 32-bit hit monitor counts the cacheable AHB-transactions on the slave cache port that hits the ICACHE content.
It also takes into account all accesses whose address is present in the TAG memory or in the refill buffer (due to a previous miss, and whose data is coming, or is soon to come, from the cache master port) (see [Section 8.4.8](#)).
- The 16-bit miss monitor counts the cacheable AHB-transactions on the slave cache port that misses the ICACHE content.
It also takes into account all accesses whose address is not present neither in the TAG memory nor in the refill buffer.

Upon reaching their maximum values, these monitors do not wrap over.

Hit and miss monitors can be enabled and reset by software allowing the analysis of specific pieces of code.

The software can perform the following tasks:

- Enable/stop the hit monitor through HITMEN in ICACHE_CR.
- Reset the hit monitor by setting HITMRST in ICACHE_CR.
- Enable/stop the miss monitor through MISSMEN in ICACHE_CR.
- Reset the miss monitor by setting MISSMRST in ICACHE_CR.

To reduce power consumption, these monitors are disabled (stopped) by default.

8.4.13 ICACHE boot

The ICACHE is disabled (EN = 0 in ICACHE_CR) at boot.

The code remapping at boot is not needed for a Cortex®-M33 as it implements the VTOR (vector tables) that allows a boot start address definition different than 0x0.

Once the boot is finished, the ICACHE can be enabled (software setting EN = 1 in ICACHE_CR).

8.5 ICACHE low-power modes

At device level, using the ICACHE reduces the power consumption by fetching instructions from the internal ICACHE most of the time, rather than from the bigger and then more power consuming main memories. This reduction is even higher if the cached main memories are external.

Applications with lower performance profile (in terms of hit ratio) and stringent power consumption constraints may benefit from the lower power consumption of an ICACHE configured as direct mapped. This single-way cache configuration is obtained by programming WAYSEL = 0 in ICACHE_CR (see [Figure 38](#)). The power consumption is reduced by accessing, for each request, only the necessary cut of TAG and data memories. The cache effect still improves fetch performance, even if for most code execution, it is less efficient than with an n-way set associative cache mode.

8.6 ICACHE error management and interrupts

If an unsupported cacheable write request is detected (functional error), the ICACHE generates an error by setting the ERRF flag in ICACHE_SR. An interrupt is generated if the corresponding interrupt enable bit is set (ERRIE = 1 in ICACHE_IER).

The other possible interrupt generation is at the end of a cache invalidation operation. When the cache-busy state is finished, the ICACHE sets the BSYENDF flag in ICACHE_SR. An interrupt is generated if the corresponding interrupt enable bit is set (BSYENDIE = 1 in ICACHE_IER).

All ICACHE interrupt sources raise the same and unique interrupt signal, icache_it, and then use the same interrupt vector.

Table 97. ICACHE interrupts

Interrupt vector	Interrupt event	Event flag	Enable control bit	Interrupt clear method
ICACHE	Functional error	ERRF in ICACHE_SR	ERRIE in ICACHE_IER	Set CERRF to 1 in ICACHE_FCR
	End of busy state (invalidate finished)	BSYENDF in ICACHE_SR	BSYENDIE in ICACHE_IER	Set CBSYENDF to 1 in ICACHE_FCR

The ICACHE also propagates all AHB bus errors (such as security issues, address decoding issues) from the master1 or master2 port back to the execution port.

8.7 ICACHE registers

8.7.1 ICACHE control register (ICACHE_CR)

Address offset: 0x000

Reset value: 0x0000 0004

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MISS MRST	HITM RST	MISS MEN	HITM EN
												rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WAY SEL	CACHE INV	EN
													rw	w	rw

Bits 31:20 Reserved, must be kept at reset value.

Bit 19 **MISSMRST**: miss monitor reset

- 0: release the cache miss monitor reset (needed to enable the counting)
- 1: reset cache miss monitor

Bit 18 **HITMRST**: hit monitor reset

- 0: release the cache miss monitor reset (needed to enable the counting)
- 1: reset cache hit monitor

Bit 17 **MISSMEN**: miss monitor enable

- 0: cache miss monitor switched off. Stopping the monitor does not reset it.
- 1: cache miss monitor enabled

Bit 16 **HITMEN**: hit monitor enable

- 0: cache hit monitor switched off. Stopping the monitor does not reset it.
- 1: cache hit monitor enabled

Bits 15:3 Reserved, must be kept at reset value.

Bit 2 **WAYSEL**: cache associativity mode selection

This bit allows user to choose ICACHE set-associativity. It can be written by software only when cache is disabled (EN = 0).

- 0: direct mapped cache (1-way cache)
- 1: n-way set associative cache (reset value)

Bit 1 **CACHEINV**: cache invalidation

Set by software and cleared by hardware when the BUSYF flag is set (during cache maintenance operation). Writing 0 has no effect.

- 0: no effect
- 1: invalidate entire cache (all cache lines valid bit = 0)

Bit 0 **EN**: enable

- 0: cache disabled
- 1: cache enabled

8.7.2 ICACHE status register (ICACHE_SR)

Address offset: 0x004

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERRF	BSYENDF	BUSYF
													r	r	r

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **ERRF**: cache error flag

- 0: no error
- 1: an error occurred during the operation (cacheable write)

Bit 1 **BSYENDF**: busy end flag

- 0: cache busy
- 1: full invalidate CACHEINV operation finished

Bit 0 **BUSYF**: busy flag

- 0: cache not busy on a CACHEINV operation
- 1: cache executing a full invalidate CACHEINV operation

8.7.3 ICACHE interrupt enable register (ICACHE_IER)

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERRIE	BSYEN DIE	Res.
													rw	rw	

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **ERRIE**: interrupt enable on cache error

Set by software to enable an interrupt generation in case of cache functional error (cacheable write access)

0: interrupt disabled on error

1: interrupt enabled on error

Bit 1 **BSYENDIE**: interrupt enable on busy end

Set by software to enable an interrupt generation at the end of a cache invalidate operation.

0: interrupt disabled on busy end

1: interrupt enabled on busy end

Bit 0 Reserved, must be kept at reset value.

8.7.4 ICACHE flag clear register (ICACHE_FCR)

Address offset: 0x00C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CERRF	CBSY ENDF	Res.
													w	w	

Bits 31:3 Reserved, must be kept at reset value.

Bit 2 **CERRF**: clear cache error flag

Set by software.

0: no effect

1: clears ERRF flag in ICACHE_SR

Bit 1 **CBSYENDF**: clear busy end flag

Set by software.

0: no effect

1: clears BSYENDF flag in ICACHE_SR.

Bit 0 Reserved, must be kept at reset value.

8.7.5 ICACHE hit monitor register (ICACHE_HMONR)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
HITMON[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
HITMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **HITMON[31:0]**: cache hit monitor counter

8.7.6 ICACHE miss monitor register (ICACHE_MMONR)

Address offset: 0x014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MISSMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **MISSMON[15:0]**: cache miss monitor counter

8.7.7 ICACHE region x configuration register (ICACHE_CRRx)

Address offset: 0x020 + 0x4 * x, (x = 0 to 3)

Reset value: 0x0000 0200

Define an alias address in the code region for other regions, making them cacheable.

BASEADDR and REMAPADDR fields are write locked (read only) when EN = 1 in ICACHE_CR.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
HBURST	Res.	Res.	MSTSEL	Res.	REMAPADDR[31:21]										
rw			rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
REN	Res.	Res.	Res.	RSIZE[2:0]				Res.	BASEADDR[28:21]						
rw				rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw

- Bit 31 **HBURST**: output burst type for region x
 0: WRAP
 1: INCR
- Bits 30:29 Reserved, must be kept at reset value.
- Bit 28 **MSTSEL**: AHB cache master selection for region x
 0: no action (master1 selected by default)
 1: master2 selected
- Bit 27 Reserved, must be kept at reset value.
- Bits 26:16 **REMAPADDR[31:21]**: remapped address for region x
 This field replaces the alias address defined by BASEADDR field.
 The only useful bits are [31:RI], where $21 \leq RI \leq 27$ is the number of bits of RSIZE
 (see [Section 8.4.7](#)). If the programmed value has more LSBs, the useless bits are ignored.
- Bit 15 **REN**: enable for region x
 0: disabled
 1: enabled
- Bits 14:12 Reserved, must be kept at reset value.
- Bits 11:9 **RSIZE[2:0]**: size for region x
 000: reserved
 001: 2 Mbytes
 010: 4 Mbytes
 011: 8 Mbytes
 100: 16 Mbytes
 101: 32 Mbytes
 110: 64 Mbytes
 111: 128 Mbytes
- Bit 8 Reserved, must be kept at reset value.
- Bits 7:0 **BASEADDR[28:21]**: base address for region x
 This alias address is replaced by REMAPADDR field.
 The only useful bits are [28:RI], where $21 \leq RI \leq 27$ is the number of bits of RSIZE
 (see [Section 8.4.7](#)). If the programmed value has more LSBs, the useless bits are ignored.

8.7.8 ICACHE register map

Table 98. ICACHE register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	ICACHE_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MISSMRST	HITMRST	MISSMEN	HITMEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WAYSEL	CACHEINV	EN
	Reset value													0	0	0	0														1	0	0
0x004	ICACHE_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERRF	BSYENDF	BUSYF
	Reset value																														0	0	1

Table 98. ICACHE register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x008	ICACHE_IER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ERRIE	BSYENDIE	Res.
	Reset value																														0	0	
0x00C	ICACHE_FCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CERRF	CBSYENDF	Res.
	Reset value																														0	0	
0x010	ICACHE_HMONR	HITMON[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	ICACHE_MMONR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	MISSMON[15:0]																
	Reset value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x018-0x01C	Reserved	Reserved																															
0x020	ICACHE_CRR0	HBURST	Res.	Res.	MSTSEL	Res.	REMAPADDR[31:21]										REN	Res.	Res.	Res.	Res.	RSIZE [2:0]	Res.	BASEADDR[28:21]									
	Reset value	0			0		0	0	0	0	0	0	0	0	0	0	0	0					0	0	1		0	0	0	0	0	0	0
0x024	ICACHE_CRR1	HBURST	Res.	Res.	MSTSEL	Res.	REMAPADDR[31:21]										REN	Res.	Res.	Res.	Res.	RSIZE [2:0]	Res.	BASEADDR[28:21]									
	Reset value	0			0		0	0	0	0	0	0	0	0	0	0	0	0					0	0	1		0	0	0	0	0	0	0
0x028	ICACHE_CRR2	HBURST	Res.	Res.	MSTSEL	Res.	REMAPADDR[31:21]										REN	Res.	Res.	Res.	Res.	RSIZE [2:0]	Res.	BASEADDR[28:21]									
	Reset value	0			0		0	0	0	0	0	0	0	0	0	0	0	0					0	0	1		0	0	0	0	0	0	0
0x02C	ICACHE_CRR3	HBURST	Res.	Res.	MSTSEL	Res.	REMAPADDR[31:21]										REN	Res.	Res.	Res.	Res.	RSIZE [2:0]	Res.	BASEADDR[28:21]									
	Reset value	0			0		0	0	0	0	0	0	0	0	0	0	0	0					0	0	1		0	0	0	0	0	0	0

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

9 Data cache (DCACHE)

9.1 DCACHE introduction

The data cache (DCACHE) is introduced on S-AHB system bus of the Cortex[®]-M33 processor to improve the performance of data traffic to/from external memories.

Some specific features, like hit-under-miss and critical-word-first refill policy, optimize performance on external memories data accesses.

9.2 DCACHE main features

The main features of DCACHE are described below:

- Bus interface
 - One 32-bit AHB slave port, the system port (input from Cortex[®]-M33 S-AHB system interface)
 - One 32-bit AHB master port (output to main AHB bus matrix)
 - One 32-bit AHB slave port for control (input from AHB peripherals interconnect, for access to DCACHE registers)
- Cache access
 - 0 wait-state on hits
 - Hit-under-miss capability: ability to serve processor requests (access to cached data) during an ongoing line refill due to a previous cache miss
 - Optimized cache line refill thanks to WRAP bursts of the size of the cache line (such as WRAP4 for 128-bit cache line)
 - 2-way set-associative
 - Supports both write-back and write-through policies (selectable with AHB bufferable attribute)
 - Read and write-back always allocate
 - Write-through always non-allocate (write-around)
 - Supports byte, half-word, and word writes
- Replacement and refill
 - pLRU-t replacement policy (pseudo-least-recently-used, based on binary tree), algorithm with best complexity/performance balance
 - Critical-word-first refill policy for read transactions, minimizing processor stalls
 - Possibility to configure burst type of all AHB memory transactions: INCRw or WRAPw (size w aligned on cache line size)
- Performance counters

The DCACHE implements four performance counters:

 - Two hit-monitor counters (32-bit): number of read hits, number of write hits
 - Two miss-monitor counters (16-bit): number of read misses, number of write misses

- Error management
 - Possibility to detect error for master port request initiated by DCACHE itself (a cache line written back into main memory, because of an eviction or a clean operation), to flag this error, and optionally to raise an interrupt
- TrustZone security support
- Maintenance operations
 - Cache invalidate: full cache invalidation, fast command, non-interruptible
 - Cache invalidate range: invalidates cache lines whose address belongs to defined range, background task, interruptible
 - Cache clean range: cleans cache lines (if dirty bit = 1, write back line, then clear dirty bit) whose address belongs to defined range, background task, interruptible
 - Cache clean and invalidate range: cleans and invalidates cache lines (if dirty bit = 1, write back line, then invalidate it) whose address belongs to defined range, background task, interruptible

9.3 DCACHE implementation

The DCACHE1 is placed on Cortex[®]-M33 S-AHB bus, and caches only the external RAM memory region (OCTOSPI and FMC), in the address range [0x6000 0000:0x9FFF FFFF] and [0xC000 0000:0xDFFF FFFF] of the memory map.

Indeed, by placing a bus matrix demultiplexing node in front of the DCACHE1, S-AHB bus memory requests addressing SRAM region or peripherals region (respectively in ranges [0x2000 0000:0x3FFF FFFF] and [0x4000 0000:0x5FFF FFFF]) are routed directly to the main AHB bus matrix, and the DCACHE1 is bypassed.

Table 99. DCACHE features

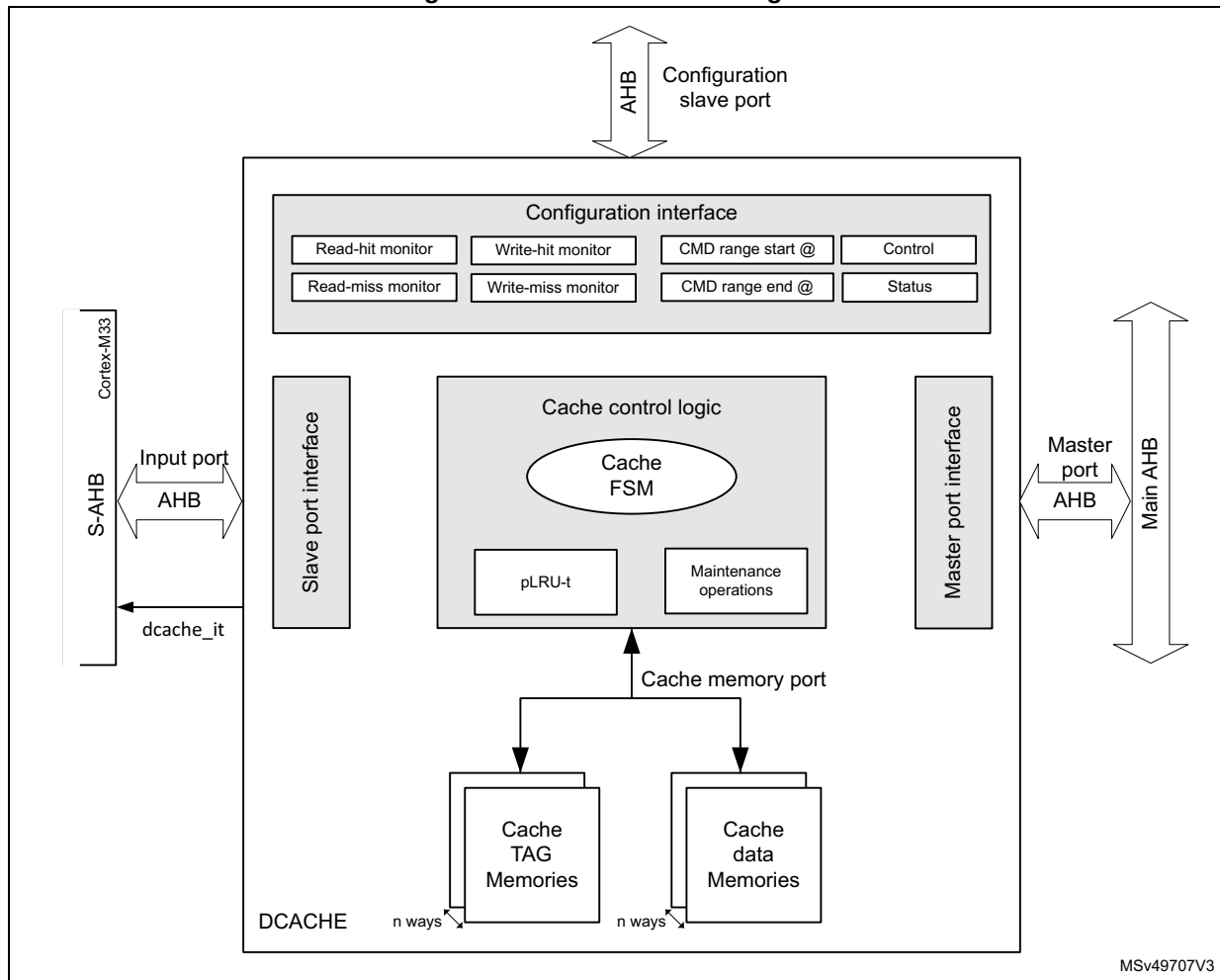
Features	DCACHE1
Number of ways	2
Cache size	4 Kbytes
Cache line width	16 bytes
Data size of AHB Master interface	32 bits

9.4 DCACHE functional description

The purpose of the data cache is to cache external memory data loads and stores, coming from the processor. These accesses include the instruction fetches that may occur at an external memory address. The DCACHE manages both read and write transactions.

9.4.1 DCACHE block diagram

Figure 40. DCACHE block diagram



9.4.2 DCACHE reset and clocks

The DCACHE is clocked on the Cortex[®]-M33 S-AHB bus clock.

When the DCACHE reset signal is released, a cache invalidate procedure is automatically launched, making the DCACHE busy (DCACHE_SR = 0x0000 0001).

When this procedure is finished:

- The DCACHE is invalidated: cold cache, with all cache line valid, dirty, and privilege bits = 0 (DCACHE must be filled up)
- DCACHE_SR = 0x0000 0002 (reflecting the cache is no longer busy)
- The DCACHE is disabled: the EN bit in DCACHE_CR holds its reset state (= 0).

Note: When disabled, the DCACHE is bypassed: slave input requests are just forwarded to the master port.

9.4.3 DCACHE TAG memory

The DCACHE TAG memory contains:

- address tags that indicate which data are contained in the cache data memories
- validity bits
- dirty bits
- privilege bits

There is one valid bit, one dirty bit, and one privilege bit per cache line (per way).

The valid bit enables/disables access to the data cache line: if the line is not valid, the data access (read or write) is performed in the main memory.

The valid bit is set when the cache line is written (refilled by either a read miss or a write-back miss).

Valid bits are reset in any of the below cases:

- after the DCACHE reset is released
- when the cache is disabled, by setting EN = 0 in DCACHE_CR (by software)
- when executing one of the DCACHE invalidate commands, setting by software CACHEINV = 0, or CACHECMD = 0b010 or 0b011 in DCACHE_CR (see [Section 9.4.8](#)).

The dirty bit indicates that the cache line has up-to-date values with respect to the main memory content (the cache has last right value, the main memory is not up to date).

The dirty bit is set when the cache line is written by a slave port write transaction (only in case of an access with write-back attribute).

Dirty bits are reset in any of the below cases:

- after the DCACHE reset is released
- when a line refill is performed on a read miss (on a write-back miss, the refilled cache line is modified by the written data, and dirty bit = 1)
- when the cache invalidation is performed
- when executing one of the DCACHE clean operations (cache line written back to the main memory), setting by software CACHECMD = 0b001 or 0b011 in DCACHE_CR (see [Section 9.4.8](#)).

The privilege bit indicates if the data is managed by a privileged entity. It is assigned according to the value of AHB privileged attribute at the input slave port, for the first access to this line (it is written only during the line refill, on read miss or write-back miss).

The privilege bit holds the same polarity as the privileged attribute: 1 for privileged access, 0 for unprivileged access.

Privilege bits are reset when the cache is invalidated, and after the DCACHE reset is released.

When a cacheable transaction is received at input slave port, its AHB address (HADDR_in) is split into the following fields (see the table below for B and W values):

- HADDR_in[B-1:0]: address byte offset, indicates which byte to select within a cache line.
- HADDR_in[B+W-1:B]: address way index, indicates which cache line to select inside each way.
- HADDR_in[31:B+W]: tag address, to be compared to TAG memory address to check if the requested data is already available (meaning valid) within the DCACHE

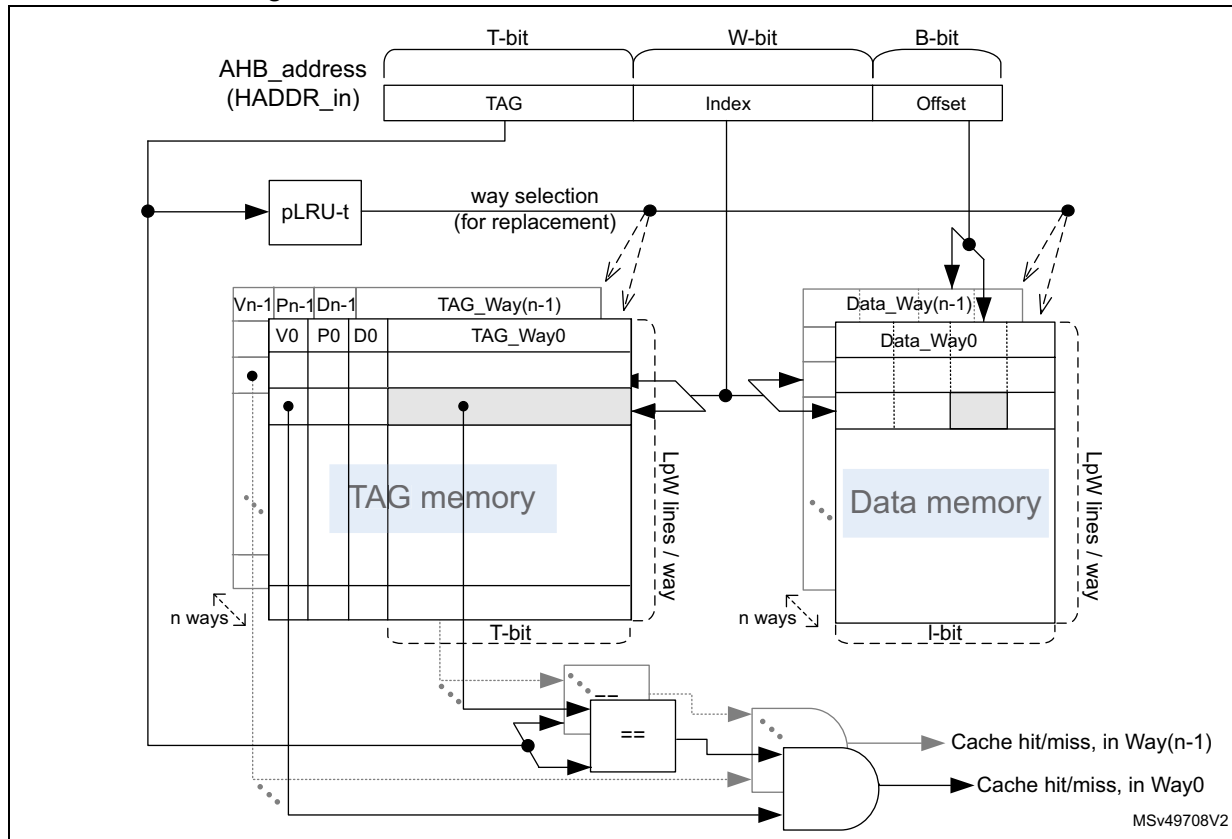
[Table 100](#) gives DCACHE main parameters for TAG memory dimensioning.

[Figure 41](#) shows the functional view of TAG and data memories, for an n-way set associative DCACHE.

Table 100. TAG memory dimensioning parameters

Parameter	Value	Example
Cache size	$S \text{ Kbytes} = s \text{ bytes} (s = 1024 \times S)$	4 Kbytes = 4096 bytes
Cache number of ways	n	2
Cache line size	$L\text{-byte} = l\text{-bit} (l = 8 \times L)$	16-byte = 128-bit
Number of cache lines (per way)	$LpW = s / (n \times L) \text{ lines/way}$	128 lines/way
Address byte offset size	$B = \log_2(L) \text{ bit}$	4-bit
Address way index size	$W = \log_2(LpW) \text{ bit}$	7-bit
TAG address size	$T = (32 - W - B) \text{ bit}$	21-bit

Figure 41. DCACHE TAG and data memories functional view



9.4.4 DCACHE enable

To activate the DCACHE functioning, the EN bit must be set in DCACHE_CR control register.

When DCACHE is disabled, it is bypassed and all transactions are copied from slave port to master port in the same clock cycle, and no comparison is performed with TAG address.

DCACHE is by default disabled at boot.

9.4.5 Cacheable and noncacheable traffic

DCACHE is developed for a Cortex[®]-M33 core and caches the memory regions addressed by the AHB bus connected to it.

In addition, the AHB bus traffic to the memory regions can be cacheable or noncacheable. An incoming memory request to DCACHE is defined as cacheable according to its AHB transaction memory lookup attribute.

In case of write transaction, the DCACHE write policy is defined as write-through or write-back according to its AHB transaction memory bufferable attribute (see [Table 101](#)).

These AHB attributes depend on the memory protection unit (MPU) programming for the addressed region.

Table 101. DCACHE cacheability for AHB transaction

AHB lookup attribute	AHB bufferable attribute	Cacheability
0	x	Read and write: non cacheable
1	0	Read: cacheable Write: (cacheable) write-through
1	1	Read: cacheable Write: (cacheable) write-back

In case of noncacheable access, the DCACHE is bypassed, meaning that the AHB transaction is propagated unchanged to the master output port.

The bypass does not increase the latency of the access to the targeted memory.

In case of cacheable access, the DCACHE behaves as explained in [Section 9.4.6](#).

Cacheable memory regions are defined and programmed by the user in the MPU, responsible for the generation of the AHB attribute signals for any transaction addressing a given region.

9.4.6 Cacheable accesses

When the DCACHE receives a cacheable transaction from Cortex[®]-M33 on its slave port, the DCACHE checks if the address requested is present in its TAG memory and if the corresponding cache line is valid.

For **read** transaction, there are three alternatives:

- The address is present inside the TAG memory, the cache line is valid: **cache read hit**, the data is read from cache and provided to the processor in the same cycle.
- The address is not present in the TAG memory: **cache read miss**, the data is read from the main memory and provided to the processor, and a cache line refill is performed.

The critical-word-first refill policy insures minimum wait cycles for the processor, since read data can be provided while cache is still performing cache line refill (associated latency is the latency of fetching one word from main memory).

The kind of burst generated on the DCACHE master bus depends on HBURST bit in DCACHE_CR: either INCRw or WRAPw (w being the cache line width, in words).

The AHB transaction attributes are also propagated from the slave input (missing) request to the master output refill request.

- The address is not present in the TAG memory but belongs to the refill burst from main memory that is currently ongoing: **cache read hit** as well (hit-under-miss feature). Whatever the line refill is due to a read or write (missing) transaction, the DCACHE can provide the requested read data as soon as the data is available at its master interface, thus avoiding a miss (with data fetch from main memory).

For **write-back** transaction (write transaction, with write-back bufferable attribute), there are three alternatives as well:

- The address is present inside the TAG memory, cache line is valid: **cache write-back hit**, the data is written in cache.
- The address is not present in the TAG memory (or the cache line is not valid): **cache write-back miss**.

First, a line allocation is performed by reading the entire cache line data from main memory. The kind of burst generated on the DCACHE master bus for this line refill depends on HBURST bit in DCACHE_CR: either INCRw or WRAPw (w being the cache line width, in words), and the AHB transaction attributes are propagated from the slave port initial request.

Once the refilled line is written in the DCACHE, the initial data provided on the slave port is written in this DCACHE line (it overwrites the data part of the cache line that was refilled just before).

- The address is not present in the TAG memory but belongs to the refill burst from main memory that is currently ongoing: **cache write-back hit** as well (hit-under-miss feature).

Whatever the line refill is due to a read or write (missing) transaction, the DCACHE can write incoming data directly inside the refilled line, thus avoiding a miss (with refill from main memory).

For **write-through** transaction (write transaction, with write-through bufferable attribute), only two alternatives exist:

- The address is present inside the TAG memory, cache line is valid: **cache write-through hit**, the data is written both in cache and in main memory (through master port).
- The address is not present in the TAG memory (or the cache line is not valid): **cache write-through miss**, the data incoming at slave port is written only in main memory (unlike the write-back miss, there is no line allocation and data written in cache).

In case of cache refill (due to cache miss), the DCACHE selects which cache line is written with the refill data: as a 2-way set associative cache, one line among two can be used (line pointed by the address index, in each of the two ways). The way selection is based on a pLRU-t replacement algorithm that points, for each index, on the way candidate for the next refill.

If the cache line where the refill data must be written is already valid, the targeted cache line must be evicted first:

- If the dirty tag of this line equals 0 (clean data), the line is simply invalidated.
- If it equals 1 (dirty data), the line must be written back in the main memory.

The DCACHE generates a burst write transaction on its master port, with burst type set to INCRw (w being the cache line width, in words), and with AHB memory transaction attribute signals set as below:

- data (not instruction)
- privileged = TAG privilege bit
- write-back (even if it does not care)
- normal memory
- cacheable
- allocate (even if it does not care)

- nonshareable

These AHB attributes cannot be propagated from the slave port (as it is the case for all other transactions emitted by the DCACHE) because the evicting transaction has no relation with the initial missing transaction. Setting of the AHB attributes is fixed, except for the privileged bit that is copied from the TAG privilege bit of the evicted line.

9.4.7 DCACHE security

The DCACHE implements an Armv8-M TrustZone.

DCACHE configuration registers are protected at system level.

9.4.8 DCACHE maintenance

The DCACHE features several maintenance operations that the software can programmed in DCACHE_CR control register:

- **Full invalidate:** invalidates the whole cache, non interruptible task.

The software can invalidate the whole DCACHE content by programming CACHEINV in DCACHE_CR.

When CACHEINV = 1, the DCACHE control logic sets BUSYF flag in DCACHE_SR status register, and performs the operation of cache invalidation, resetting each TAG valid bit to 0 (one valid bit per cache line). Each dirty and privilege bits are also reset to 0 during cache invalidation to prevent unknown values at next cache line validation. CACHEINV is automatically cleared.

Once the full invalidate operation is finished, the DCACHE automatically clears BUSYF flag, and sets BSYENDF in DCACHE_SR.

If enabled on this flag condition (BSYENDIE = 1 in DCACHE_IER), the DCACHE interrupt is raised. Then, the (empty) cache is available again.

This full invalidate operation is not interruptible, meaning that the cache does not treat any cacheable request while BUSYF = 1. However, non-cacheable traffic is treated (since the request address is not compared to TAG ones), the DCACHE being bypassed in the same clock cycle (same behavior as when the DCACHE is disabled).

- **Invalidate range:** invalidates a certain range of addresses in the cache, background task (interruptible).

The software can invalidate a given data region in the DCACHE by programming STARTCMD = 1 and CACHECMD = 0b010 in DCACHE_CR, after the address range was programmed into DCACHE_CMDRSADDR (range start address) and DCACHE_CMDREADADDR (range end address).

The DCACHE control logic then parses the whole TAG memory. If the read line address (TAG address + line index) falls in the programmed address range ($\text{DCACHE_CMDRSADDR} \leq \text{Line Addr} \leq \text{DCACHE_CMDREADADDR}$), the

corresponding cache line is invalidated (line TAG bits cleared, valid bit = dirty bit = privilege bit = 0).

When STARTCMD is set, the DCACHE control logic sets BUSYCMD in DCACHE_SR and launches the invalidate range operation. STARTCMD is also automatically cleared.

Once the operation is finished (all TAG memory parsed), the DCACHE automatically clears BUSYCMD and sets CMDEND in DCACHE_SR.

If enabled on this flag condition (CMDENDIE = 1 in DCACHE_IER), the DCACHE interrupt is raised.

During this invalidate range operation, the DCACHE is interruptible, meaning it can accept new incoming requests that take higher priority than the invalidation process. The TAG memory is accessed for invalidate range operation only if not already accessed by an external cache request. This implies that invalidate range execution is usually not performed in one go, but can be interrupted.

- **Clean range:** cleans a certain range of addresses in the cache, background task (interruptible).

Cleaning a cache line means making sure that the main memory content is up-to-date with the data, which may have been modified in the cache. The clean operation consists in performing the write-back in the main memory of the cache lines that are tagged as “dirty” (the ones with TAG dirty bit set).

The software can clean a given data region in DCACHE by programming STARTCMD = 1, and CACHEDCMD = 0b001 in DCACHE_CR, after the address range was programmed into DCACHE_CMDRSADDR (range start address) and DCACHE_CMDREADADDR (range end address).

The DCACHE control logic then parses the whole TAG memory. If the read line address (TAG address + line index) falls in the programmed address range ($\text{DCACHE_CMDRSADDR} \leq \text{Line Addr} \leq \text{DCACHE_CMDREADADDR}$), and the corresponding line is dirty, this line is cleaned, meaning the whole cache line is written-back in the memory through the DCACHE master port, and its TAG dirty bit is cleared.

When STARTCMD is set, the DCACHE control logic sets BUSYCMD in DCACHE_SR and launches the clean range operation. STARTCMD is also automatically cleared.

Once the operation is finished (all TAG memory parsed), the DCACHE automatically clears BUSYCMD and sets CMDEND in DCACHE_SR.

If enabled on this flag condition (CMDENDIE = 1 in DCACHE_IER), the DCACHE interrupt is raised.

During this clean range operation, the DCACHE is interruptible, meaning it can accept new incoming requests that take higher priority than the cleaning process. The TAG memory is accessed for clean range operation only if not already accessed by an external cache request. This implies that clean range execution is usually not performed in one go, but can be interrupted.

It is under the software responsibility that no bus initiator attempts to change the content of the region being cleaned until clean range is completed. For that, the software can take advantage of BUSYCMD flag in DCACHE_SR, and can poll this flag to prevent any spurious access to the area being cleaned.

Alternatively it can also rely on the command end flag (CMDENDF) or on the DCACHE interrupt to detect the end of the clean range execution.

- **Clean and invalidate range:** cleans and invalidates a certain range of addresses in the cache, background task (interruptible).

This operation cleans the “dirty” cache lines that belong to the operation address range (the same as the clean range operation), and also invalidates all the (valid) cache lines that belong to this address range (whether they are dirty or not).

The software can launch this clean and invalidate range operation, by programming STARTCMD = 1, and CACHECMD = 0b011 in DCACHE_CR, after the address range was programmed into DCACHE_CMDRSADDRR (range start address) and DCACHE_CMDREADDRR (range end address).

This sets and clears the same flags, and potentially the same interrupt as invalidate range or clean range operations.

9.4.9 DCACHE performance monitoring

The DCACHE provides the following monitors for performance analysis:

- The two 32-bit read-hit and write-hit monitors count the AHB transactions at the DCACHE input (slave port) that do not generate a transaction on the DCACHE output (master port).
These monitors also take into account all accesses whose address is present in the TAG memory, or in the refill buffer (due to a previous miss, and whose data is coming, or is soon to come, from the cache master port) (see [Section 9.4.6](#)).
- The two 16-bit read-miss and write-miss monitors count the AHB transactions at the DCACHE input (slave port) that generate a transaction on the DCACHE output (master port).
These monitors also take into account all accesses whose address is not present neither in the TAG memory, nor in the refill buffer.

Upon reaching their maximum values, the monitors do not wrap over.

The software can perform the following tasks:

- Enable/stop the read (write) hit monitor, through R(W)HITMEN in DCACHE_CR.
- Reset the read (write) hit monitor, by setting R(W)HITMRST in DCACHE_CR.
- Enable/stop the read (write) miss monitor, through R(W)MISSMEN in DCACHE_CR.
- Reset the read (write) miss monitor, by setting R(W)MISSMRST in DCACHE_CR.

To reduce power consumption, these monitors are disabled (stopped) by default.

9.4.10 DCACHE boot

The DCACHE is disabled (EN = 0 in DCACHE_CR) at boot.

Once the boot is finished, the DCACHE can be enabled (software setting EN = 1 in DCACHE_CR).

9.5 DCACHE low-power modes

At product level, the DCACHE reduces the power consumption by loading/storing data from/to the internal DCACHE most of the time, rather than from the bigger and then more power consuming main memories. This reduction is even much higher, since the cached main memories are external.

9.6 DCACHE error management and interrupts

A transaction initiated on the DCACHE master port may return an error (a write attempt into a read-only memory, for instance). If the master port request was propagated from a slave port request, the error is propagated back to the slave port. If ever the master port request is initiated by the DCACHE itself (a cache line is written back into the main memory because of an eviction or a clean operation), the DCACHE receives this functional error and flags it internally by setting the ERRF flag in DCACHE_SR.

In such a case, an interrupt is generated if the corresponding interrupt enable bit is set (ERRIE = 1 in DCACHE_IER).

Another case of potential interrupt generation is at the end of a full invalidate operation: when the cache busy state is finished, the DCACHE sets BSYENDF flag in DCACHE_SR.

An interrupt is then generated if the corresponding interrupt enable bit is set (BSYENDIE = 1 in DCACHE_IER).

Last case is at the end of invalidate and/or clean range operations: when the command busy state is finished, the DCACHE sets CMDENDF flag in DCACHE_SR.

An interrupt is also generated if the corresponding interrupt enable bit is set (CMDENDIE = 1 in DCACHE_IER).

All DCACHE interrupt sources raise the same and unique interrupt signal, `dcache_it`, and then use the same interrupt vector.

Table 102. DCACHE interrupts

Interrupt vector	Interrupt event	Event flag	Enable control bit	Interrupt clear method
DCACHE	Functional error	ERRF in DCACHE_SR	ERRIE in DCACHE_IER	Set CERRF to 1 in DCACHE_FCR
	End of busy state (full invalidate finished)	BSYENDF in DCACHE_SR	BSYENDIE in DCACHE_IER	Set CBSYENDF to 1 in DCACHE_FCR
	End of cache operations (address range based)	CMDENDF in DCACHE_SR	CMDENDIE in DCACHE_IER	Set CCMDENDF to 1 in DCACHE_FCR

The DCACHE also propagates all AHB bus errors (such as security issues, address decoding issues) from master port back to the S-AHB slave port.

9.7 DCACHE registers

9.7.1 DCACHE control register (DCACHE_CR)

Address offset: 0x000

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
HBURST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WMISSMRST	WHITMRST	WMISSMEN	WHITMEN	RMISSMRST	RHITMRST	RMISSMEN	RHITMEN
rw								rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	START CMD	CACHECMD[2:0]			Res.	Res.	Res.	Res.	Res.	Res.	CACHE INV	EN
				w	rw	rw	rw							w	rw

Bit 31 **HBURST**: output burst type for cache master port read accesses

Can be set by software, only when EN = 0.

Master port write accesses are always done in INCR burst type.

0: WRAP

1: INCR

Bits 30:24 Reserved, must be kept at reset value.

Bit 23 **WMISSMRST**: write-miss monitor reset

0: release the cache miss monitor reset (needed to enable the counting)

1: reset cache write-miss monitor

Bit 22 **WHITMRST**: write-hit monitor reset

0: release the cache miss monitor reset (needed to enable the counting)

1: reset cache write-hit monitor

Bit 21 **WMISSMEN**: write-miss monitor enable

0: cache write-miss monitor switched off. Stopping the monitor does not reset it.

1: cache write-miss monitor enabled

Bit 20 **WHITMEN**: write-hit monitor enable

0: cache write-hit monitor switched off. Stopping the monitor does not reset it.

1: cache write-hit monitor enabled

Bit 19 **RMISSMRST**: read-miss monitor reset

0: release the cache miss monitor reset (needed to enable the counting)

1: reset cache read-miss monitor

Bit 18 **RHITMRST**: read-hit monitor reset

0: release the cache miss monitor reset (needed to enable the counting)

1: reset cache read-hit monitor

Bit 17 **RMISSMEN**: read-miss monitor enable

0: cache read-miss monitor switched off. Stopping the monitor does not reset it.

1: cache read-miss monitor enabled

Bit 16 **RHITMEN**: read-hit monitor enable

0: cache read-hit monitor switched off. Stopping the monitor does not reset it.

1: cache read-hit monitor enabled

Bits 15:12 Reserved, must be kept at reset value.

Bit 11 **STARTCMD**: starts maintenance command (maintenance operation defined in CACHECMD).

Can be set by software, only when EN = 1, BUSYCMD = 0, BUSYF = 0 and
CACHECMD = 0b001, 0b010 or 0b011.

Cleared by hardware when the BUSYCMD flag is set (during cache maintenance operation). Writing 0 has no effect.

0: command operation (cache maintenance) finished

1: start maintenance command (cache maintenance)

Bits 10:8 **CACHECMD[2:0]**: cache command maintenance operation (cleans and/or invalidates an address range)

Can be set and cleared by software, only when no maintenance command is ongoing (BUSYCMD = 0).

000: no operation

001: clean range

010: invalidate range

011: clean and invalidate range

others: reserved

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **CACHEINV**: full cache invalidation

Can be set by software, only when EN = 1.

Cleared by hardware when the BUSYF flag is set (during full cache invalidation operation). Writing 0 has no effect.

0: no effect

1: invalidate entire cache (all cache lines valid bit = 0)

Bit 0 **EN**: enable

0: cache disabled

1: cache enabled

9.7.2 DCACHE status register (DCACHE_SR)

Address offset: 0x004

Reset value: 0x0000 0001

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDE NDF	BUSYC MDF	ERRF	BSYEN DF	BUSYF
											r	r	r	r	r

Bits 31:5 Reserved, must be kept at reset value.

Bit 4 **CMDENDF**: command end flag

Cleared by writing DCACHE_FCR.CCMDENDF = 1.

0: cache busy or in idle

1: CACHECMD command finished

- Bit 3 **BUSYCMDIF**: command busy flag
 0: cache not busy on a CACHECMD command
 1: cache busy on a CACHECMD command (clean and/or invalidate an address range)
- Bit 2 **ERRF**: cache error flag
 Cleared by writing DCACHE_FCR.CERRF = 1.
 0: no error
 1: an error occurred during the operation (eviction or clean operation write-back error).
- Bit 1 **BSYENDF**: full invalidate busy end flag
 Cleared by writing DCACHE_FCR.CBSYENDF = 1.
 0: cache busy or in idle
 1: full invalidate CACHEINV operation finished
- Bit 0 **BUSYF**: full invalidate busy flag
 0: cache not busy on a CACHEINV operation
 1: cache executing a full invalidate CACHEINV operation

9.7.3 DCACHE interrupt enable register (DCACHE_IER)

Address offset: 0x008

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDE NDIE	Res.	ERRIE	BSYEN DIE	Res.
											rw		rw	rw	

Bits 31:5 Reserved, must be kept at reset value.

- Bit 4 **CMDENDIE**: interrupt enable on command end
 Set by software to enable an interrupt generation at the end of a cache command (clean and/or invalidate an address range)
 0: interrupt disabled on command end
 1: interrupt enabled on command end
- Bit 3 Reserved, must be kept at reset value.
- Bit 2 **ERRIE**: interrupt enable on cache error
 Set by software to enable an interrupt generation in case of cache functional error (eviction or clean operation write-back error)
 0: interrupt disabled on error
 1: interrupt enabled on error
- Bit 1 **BSYENDIE**: interrupt enable on busy end
 Set by SW to enable an interrupt generation at the end of a cache full invalidate operation.
 0: Interrupt disabled on busy end
 1: Interrupt enabled on busy end
- Bit 0 Reserved, must be kept at reset value.

9.7.4 DCACHE flag clear register (DCACHE_FCR)

Address offset: 0x00C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CCMD ENDF	Res.	CERRF	CBSYE NDF	Res.
											w		w	w	

Bits 31:5 Reserved, must be kept at reset value.

Bit 4 **CCMDENDF**: clear command end flag

Set by software.

0: no effect

1: clears CMDENDF flag in DCACHE_SR

Bit 3 Reserved, must be kept at reset value.

Bit 2 **CERRF**: clear cache error flag

Set by software.

0: no effect

1: clears ERRF flag in DCACHE_SR

Bit 1 **CBSYENDF**: clear full invalidate busy end flag

Set by software.

0: no effect

1: clears BSYENDF flag in DCACHE_SR

Bit 0 Reserved, must be kept at reset value.

9.7.5 DCACHE read-hit monitor register (DCACHE_RHMONR)

Address offset: 0x010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RHITMON[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RHITMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **RHITMON[31:0]**: cache read-hit monitor counter

9.7.6 DCACHE read-miss monitor register (DCACHE_RMMONR)

Address offset: 0x014

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RMISSMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **RMISSMON[15:0]**: cache read-miss monitor counter

9.7.7 DCACHE write-hit monitor register (DCACHE_WHMONR)

Address offset: 0x020

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WHITMON[31:16]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WHITMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:0 **WHITMON[31:0]**: cache write-hit monitor counter

9.7.8 DCACHE write-miss monitor register (DCACHE_WMMONR)

Address offset: 0x024

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WMISSMON[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:0 **WMISSMON[15:0]**: cache write-miss monitor counter

9.7.9 DCACHE command range start address register (DCACHE_CMDRSADDRR)

Address offset: 0x028

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CMDSTARTADDR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CMDSTARTADDR[15:4]												Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

Bits 31:4 **CMDSTARTADDR[31:4]**: start address of range to which the cache maintenance command specified in DCACHE_CR.CACHECMD field applies

This register must be set before DCACHE_CR.CACHECMD is written.

Bits 3:0 Reserved, must be kept at reset value.

9.7.10 DCACHE command range end address register (DCACHE_CMDREADDRR)

Address offset: 0x02C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CMDENDADDR[31:16]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
CMDENDADDR[15:4]												Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw				

Bits 31:4 **CMDENDADDR[31:4]**: end address of range to which the cache maintenance command specified in DCACHE_CR.CACHECMD field applies

This register must be set before DCACHE_CR.CACHECMD is written.

Bits 3:0 Reserved, must be kept at reset value.

9.7.11 DCACHE register map

Table 103. DCACHE register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	DCACHE_CR	HBURST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WMISSMRST	WHITMRST	WMISSMEN	WHITMEN	RMISSMRST	RHITMRST	RMISSMEN	RHITMEN	Res.	Res.	Res.	Res.	STARTCMD	CACHECMD [2:0]				Res.	Res.	Res.	Res.	Res.	CACHEINV	EN
	Reset value	0								0	0	0	0	0	0	0	0					0	0	0	0						0	0	

Table 103. DCACHE register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x004	DCACHE_SR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDENDF	BUSYCMDF	ERRF	BSYENDF	BUSYF
	Reset value																											0	0	0	0	1	
0x008	DCACHE_IER	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CMDENDIE	Res.	ERRIE	BSYENDIE	Res.
	Reset value																											0		0	0		
0x00C	DCACHE_FCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CCMDENDF	Res.	CERRF	CBSYENDF	Res.
	Reset value																											0		0	0		
0x010	DCACHE_RHMONR	RHITMON[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x014	DCACHE_RMONR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RMISSMON[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x020	DCACHE_WHMONR	WHITMON[31:0]																															
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x024	DCACHE_WMMONR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WMISSMON[15:0]															
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x028	DCACHE_CMDRSADDRR	CMDSTARTADDR[31:4]																												Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			Res.	Res.	Res.
0x02C	DCACHE_CMDREADRR	CMDENDADDR[31:4]																												Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				Res.	

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

10 Power control (PWR)

10.1 PWR introduction

The power controller manages the device power supplies and power modes transitions.

10.2 PWR main features

The power controller (PWR) main features are:

- Power supplies and supply domains
 - Core domain (V_{CORE})
 - V_{DD} domain
 - Backup domain (V_{SW})
 - Analog domain (V_{DDA})
 - Supply for the SMPS power stage (available on SMPS packages)^(a)
 - V_{DDIO2} supply for ten I/Os (PD6, PD7, PG9:14, PB8, PB9)
 - V_{DDUSB} for USB transceiver
- System supply voltage regulation
 - SMPS step-down converter (STM32H563/573xx devices only)
 - Linear voltage regulator (LDO)
- Power supply supervision
 - POR/PDR monitor
 - BOR monitor
 - PVD monitor
 - AVD monitor
 - Out of functional range temperature monitor
 - Out of functional range Backup domain voltage monitor
- Power management
 - Operating modes
 - Voltage scaling control
 - Low-power modes
- V_{BAT} battery charging
- TrustZone security and privileged protection

a. STM32H563/573xxx devices only.

10.3 PWR pins and internal signals

Table 104. PWR input/output pins

Pin name	Signal type	Description
VDD	Supply	Main supply
GND	Supply	Main ground
VDDA	Supply	Analog peripherals supply
VSSA	Supply	Analog peripherals ground
VDDIO2	Supply	Independent I/O supply
VDDUSB	Supply	USB supply
VCAP	Supply	Logic supply (V_{CORE})
VBAT	Supply	Backup domain supply
VDDSMPS ⁽¹⁾	Supply	SMPS supply
VSSSMPS ⁽¹⁾	Supply	SMPS ground
VLXSMPS ⁽¹⁾	Supply	SMPS output
VREF+	Supply	ADC/DAC high reference voltage
VREF-	Supply	ADC/DAC low reference voltage
WKUPx (x = 1 to 8)	Input	Wake-up pins
CSLEEP	Output	MCU in Sleep mode
CDSTOP	Output	CPU in Stop modes

1. STM32H563/573xx devices only.

Table 105. PWR internal input/output signals

Internal signal name	Signal type	Description
WKUPx (x = 1 to 8)	Input	Wake-up event source

10.4 PWR power supplies and supply domains

Figure 42. Power supply with SMPS (STM32H563/573xx devices only)

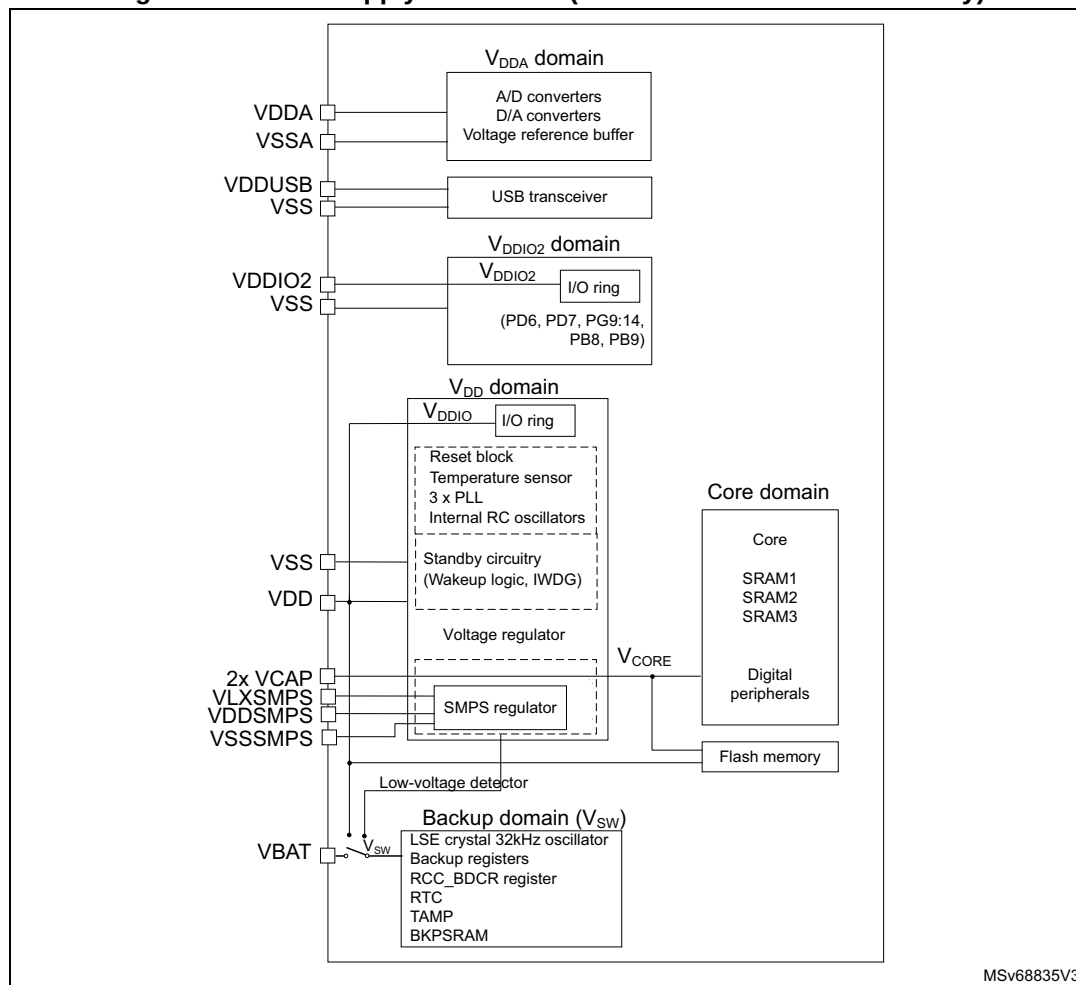


Figure 43. Power supply with LDO

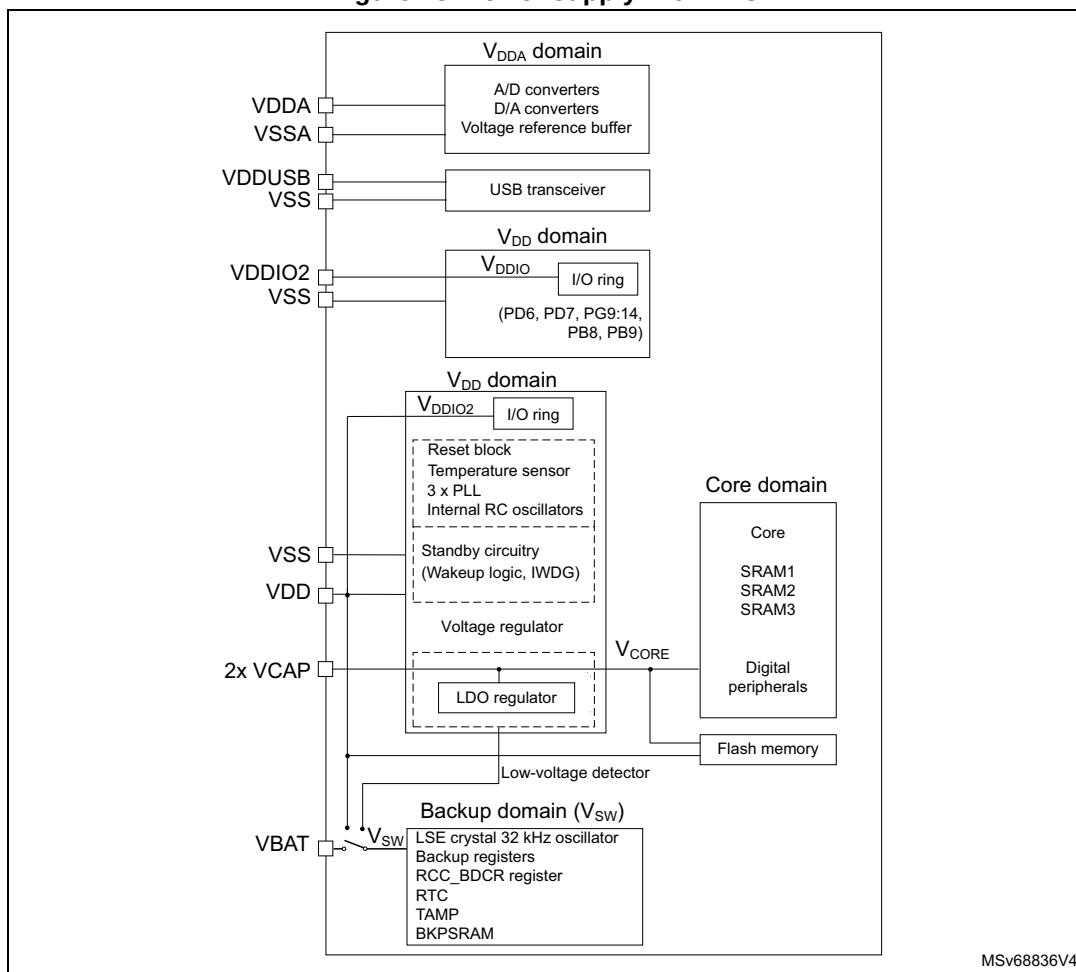
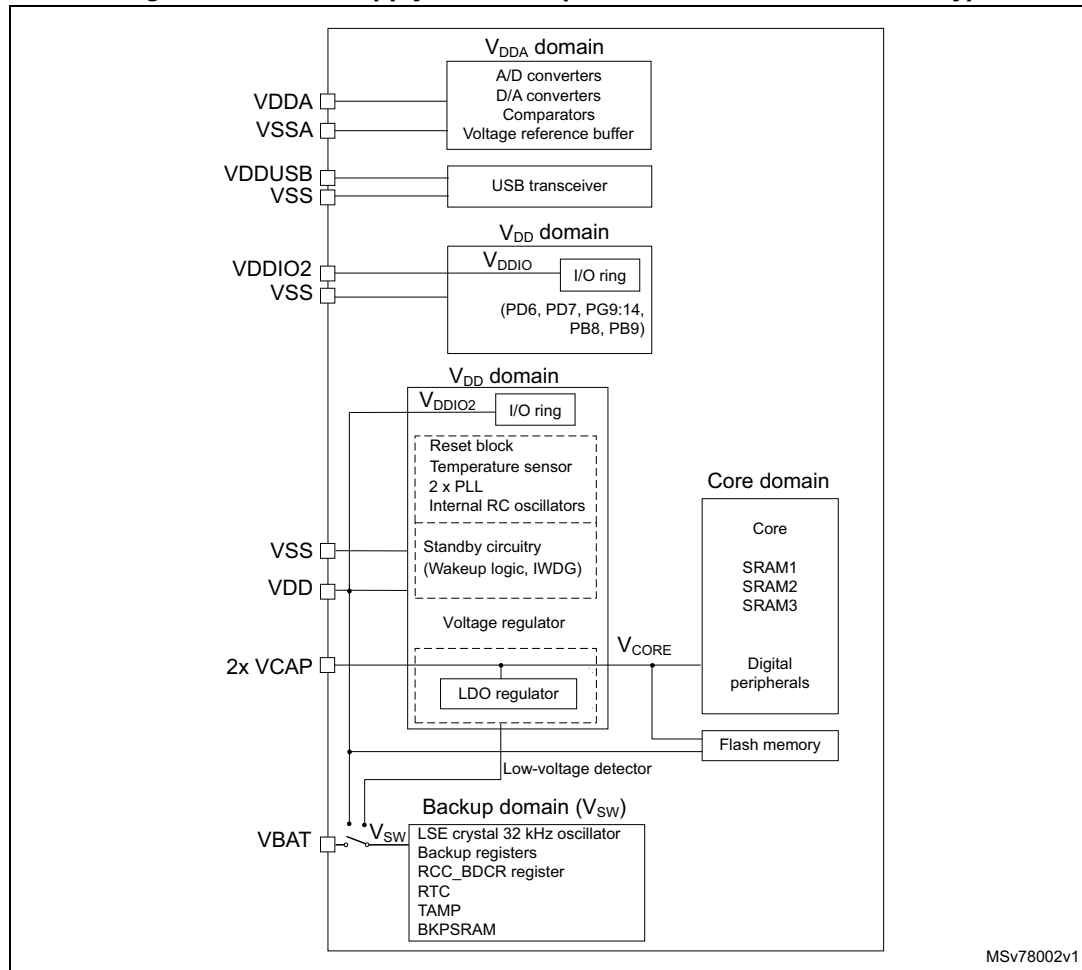


Figure 44. Power supply with LDO (STM32H543/553xx devices only)



MSv78002v1

10.4.1 External power supplies

The devices require a 1.71 to 3.6 V V_{DD} operating voltage supply. Several independent supplies can be provided for specific peripherals, but must not be provided without a valid operating supply on the VDD pin:

- $V_{DD} = 1.71$ to 3.6 V
 V_{DD} is the external power supply for the I/Os, the internal regulator, and the system analog (such as reset, power management, and internal clocks). It is provided externally through the VDD pins.
- $V_{DDA} = 1.62$ V (ADCs, COMP^(a)) or 1.8 V (DAC) / 2.1 (VREFBUF) to 3.6 V
 V_{DDA} is the external analog power supply for A/D converters, D/A converters, and voltage reference buffer. The V_{DDA} voltage level is independent from the V_{DD} voltage, and must preferably be connected to V_{DD} when these peripherals are not used.
- $V_{DDSMPS} = 1.71$ to 3.6 V
 V_{DDSMPS} is the external power supply for the SMPS step-down converter. It is provided

a. Available on STM32H543/553xx devices only.

externally through the VDDSMPS supply pin, and must be connected to the same supply as the VDD pin.

- V_{LXSMPS} is the switched SMPS step-down converter output.

Note: The SMPS power supply pins are available only on specific packages, with SMPS step-down converter option.

- $V_{DDUSB} = 3.0$ to 3.6 V

V_{DDUSB} is the external independent power supply for USB transceivers. The V_{DDUSB} voltage level is independent from the V_{DD} voltage and must preferably be connected to V_{DD} when the USB is not used.

- $V_{DDIO2} = 1.08$ to 3.6 V

V_{DDIO2} is the external power supply for ten I/Os (PD6, PD7, PG9:14, PB8, PB9). The V_{DDIO2} voltage level is independent from the V_{DD} voltage and must preferably be connected to V_{DD} when those pins are not used.

- $V_{CAP} = 1.0$ to 1.35 V: digital core domain supply

This power supply is independent from all the other power supplies:

- When the voltage regulator is enabled, V_{CORE} is delivered by the internal voltage regulator.
- When the voltage regulator is disabled, V_{CORE} is delivered by an external power supply through VCAP pin, or by the SMPS

- $V_{BAT} = 1.2$ to 3.6 V

V_{BAT} is the power supply when V_{DD} is not present (through power switch) for RTC, external clock 32 kHz oscillator, backup registers, and, optionally, backup SRAM.

- V_{REF-} , V_{REF+}

V_{REF+} is the input reference voltage for ADCs and DACs. It is also the output of the internal voltage reference buffer when enabled.

V_{REF+} can be grounded when ADC and DAC are not active.

The internal voltage reference buffer supports four output voltages, configured with VRS bit in the VREFBUF_CSR register:

- $V_{REF+} \sim 1.8$ V. (requires $V_{DDA} \geq 2.1$ V)
- $V_{REF+} \sim 2.048$ V (requires $V_{DDA} \geq 2.4$ V)
- $V_{REF+} \sim 2.5$ V (requires $V_{DDA} \geq 2.8$ V)

V_{REF-} and V_{REF+} pins are not available on all packages. When not available, they are bonded to VSSA and VDDA, respectively.

When the V_{REF+} is double-bonded with VDDA in a package, the internal voltage reference buffer is not available and must be kept disabled.

V_{REF-} must always be equal to V_{SSA} .

10.4.2 Internal regulators

The devices embed two regulators: one LDO and one SMPS^(a) to provide the V_{CORE} supply for digital peripherals, SRAMs (except BKPSRAM), and embedded flash memory.

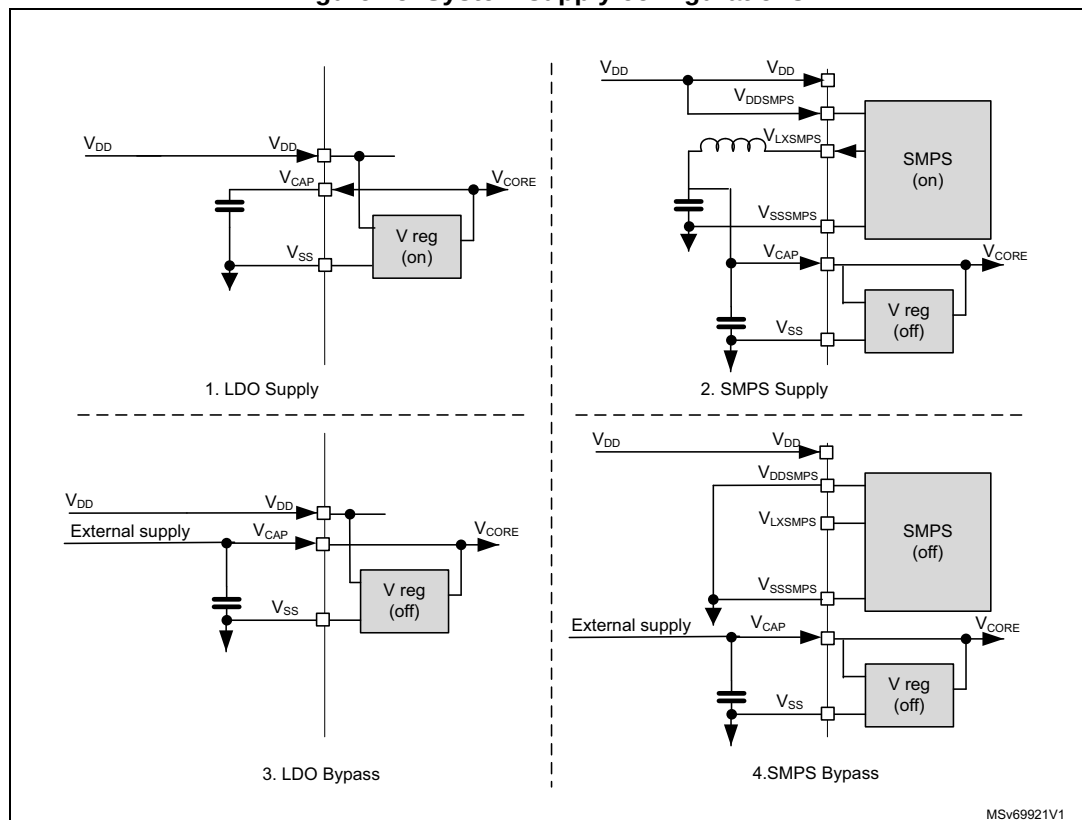
a. STM32H563/573xx devices only.

The SMPS generates this voltage on VCAP (two pins), with a total external capacitor of 10 μF (typical), and requires an external coil of 2.2 μH (typical). The LDO generates this voltage on VCAP, with a total external capacitance of 4.7 μF (typical).

Both regulators can provide four different voltages (voltage scaling), and can operate in Stop modes.

Depending on the package configuration (SMPS or LDO), the regulator is selected by hardware. SMPS and LDO regulator are exclusively selected. The supply configurations shown in [Figure 45](#) are supported for the V_{CORE} domain supply. Both regulators can also be bypassed to allow an external power supply to V_{CORE} .

Figure 45. System supply configurations



Startup with V_{CORE} provided from an external supply (Bypass)

- LDO configuration**
 For devices in LDO packages, when operating in Bypass mode, the supply must first settle at its default level ($\geq 1.1 \text{ V}$), before reaching the POR threshold. Since the LDO is enabled by default after power-up, the external voltage must remain above 1.1 V until the LDO is disabled by software.
- SMPS configuration**
 For devices in SMPS packages, when operating in Bypass mode, the external supply ramp slope must be higher than 3.5 mV/ μs .

Note: Once the internal regulator, either the LDO or the SMPS, is disabled, the external voltage can be adjusted according to the application requirements. See the General operating

conditions section of the datasheet for details about the V_{CORE} level and the maximum operating frequency.

The application must also configure PWR_VOSCR according to the externally provided core voltage level and the related performance.

To adjust the VOS level, the software must select sequentially the intermediate levels.

- When increasing the performance:
 - First, voltage scaling must be incremented (for example when changing from VOS3 to VOS0, lower levels must be selected in the VOS[1:0] bits: VOS2, VOS1, and then VOS0).
 - The external voltage can be increased
 - The system frequency can be increased
- When decreasing the performance:
 - The system frequency must be decreased
 - The external voltage must be decreased
 - The voltage scaling can be decremented (for example when changing from VOS1 to VOS3, lower levels must be selected in the VOS[1:0] bits: VOS2, and then VOS3)

10.4.3 Power-up and power-down power sequences

During power-up and power-down phases, the following power sequence requirements must be respected:

- When V_{DD} is below 1 V, other power supplies (V_{DDA}, V_{DDIO2}, V_{DDUSB}) must remain below V_{DD} + 300 mV.
- When V_{DD} is above 1 V, all power supplies are independent.

During the power-down phase, V_{DD} can temporarily become lower than other supplies only if the energy provided to the MCU remains below 1 mJ. This allows external decoupling capacitors to be discharged with different time constants during the power-down transient phase.

10.4.4 Independent analog peripherals supply

To improve ADC and DAC conversion accuracy and to extend the supply flexibility, the analog peripherals have an independent power supply that can be separately filtered and shielded from noise on the PCB:

- The voltage supply input of the analog peripherals is available on a separate V_{DDA} pin.
- An isolated supply ground connection is provided on the V_{SSA} pin.

The V_{DDA} supply voltage can be different from V_{DD}. The presence of V_{DDA} must be checked before enabling any of the analog peripherals supplied by V_{DDA} (A/D converter, D/A converter, voltage reference buffer).

Power supply level monitoring is available on V_{DDA} via AVDO bit in PWR_VMSR register.

When a single supply is used, V_{DDA} can be externally connected to V_{DD} through the external filtering circuit to ensure a noise-free V_{DDA} reference voltage.

Analog switch voltage booster

By setting the BOOSTE and AVD_READY bits in the PWR_PMCR register, the application can reduce the total harmonic distortion (THD) of the I/O analog switches when the supply voltage is below 2.7 V, ensuring the same performance as with the full voltage range.

To avoid unnecessary current consumption caused by booster activation when $V_{DDA} < 2.7$ V and $V_{DD} > 2.7$ V, V_{DD} can be selected as the supply voltage for the analog switches by keeping the AVD_READY bit cleared to 0 in the PWR_PMCR register.

Table 106. Analog switched recommended configuration

VDDA	VDD	BOOSTE	AVD_READY	Analog switch supply voltage
≤ 2.7 V	≤ 2.7 V	1	1	Booster
	> 2.7 V	0	0	VDD
> 2.7 V	Not applicable	0	1	VDDA

ADC and DAC reference voltage

To ensure a better accuracy on low-voltage inputs and outputs, the user can connect to VREF+ pin, a separate reference voltage lower than V_{DDA} . V_{REF+} is the highest voltage, represented by the full scale value, for an analog input (ADC) or output (DAC) signal.

V_{REF+} can be provided either by an external reference or by an internal buffered voltage reference (VREFBUF). The internal voltage reference can output a configurable voltage, namely 1.8, 2.048 or 2.4 V. The internal voltage reference can also provide the voltage to external components through VREF+ pin. Refer to the device datasheet and to [Section 31: Voltage reference buffer \(VREFBUF\)](#) for further information.

Note: The VREF+ and VREF- pins are not available on all packages (internally connected, respectively, to VDDA and VSSA). Do not enable the internal voltage reference buffer when an external power supply is applied to the VREF+ pin.

10.4.5 Independent I/O supply rail

Some I/Os (PD6, PD7, PG9:14, PB8, PB9) are supplied from a separate supply rail. The power supply for this rail can range from 1.08 to 3.6 V, and is provided externally through the VDDIO2 pin. The V_{DDIO2} voltage level is completely independent from V_{DD} or V_{DDA} . The VDDIO2 pin is available only for some packages. Refer to the pinout diagrams or tables in the related device datasheet(s) for the I/O list(s).

Power supply level monitoring is available on VDDIO2 via VDDIO2RDY bit in PWR_VMSR register.

10.4.6 Independent USB transceivers supply

The USB transceivers are supplied from a separate VDDUSB power supply pin. V_{DDUSB} range is from 3.0 to 3.6 V, and is completely independent from V_{DD} or V_{DDA} .

Power supply level monitoring is available on VDDUSB via the USB33RDY bit in PWR_VMSR register.

Before setting USB33SV bit in PWR USB supply control register (PWR_USBSCR), check that V_{DDUSB} is available by monitoring USB33RDY bit in PWR voltage monitor status

register (PWR_VMSR). The $V_{DD33USB}$ supply level detector must be enabled through USB33DEN bit in PWR USB supply control register (PWR_USBSCR).

Setting USB33SV bit is mandatory to use the USB peripheral. It is used to validate the V_{DDUSB} supply for electrical and logical isolation purposes.

10.4.7 Backup domain

To retain the content of the backup registers and to supply the RTC function when V_{DD} is turned off, the VBAT pin can be connected to an optional backup voltage supplied by a battery or by another source.

The VBAT pin powers the RTC unit, the LSE oscillator, PI8^(a), and PC13 to PC15 I/Os, allowing the RTC to operate even when the main power supply is turned off. The backup SRAM is optionally powered by VBAT pin when the BREN bit is set in the [PWR Backup domain control register \(PWR_BDCR\)](#). The switch to the V_{BAT} supply is controlled by the power-down reset embedded in the Reset block.

Warning: During $t_{RSTTEMPO}$ (temporization at V_{DD} startup) or after a PDR has been detected, the power switch between V_{BAT} and V_{DD} remains connected to V_{BAT} . During the startup phase, if V_{DD} is established in less than $t_{RSTTEMPO}$ (refer to the datasheet for its value) and $V_{DD} > V_{BAT} + 0.6\text{ V}$, a current may be injected into V_{BAT} through an internal diode connected between V_{DD} and the power switch (V_{BAT}). If the power supply/battery connected to the VBAT pin cannot support this current injection, it is strongly recommended to connect an external low-drop diode between this power supply and the VBAT pin.

If no external battery is used in the application, it is recommended to connect V_{BAT} externally to V_{DD} with a 100 nF external ceramic decoupling capacitor.

When the Backup domain is supplied by V_{BAT} (analog switch connected to V_{BAT}), the following pins are available:

- PC13, PI8^(b), PC14, and PC15, which can be configured by RTC or LSE (refer to [Section 51.3: RTC functional description](#))
- PC13, PI8^(b), PA0, PA1, and PA2 when they are configured by TAMP peripheral as tamper pins.

Note: The analog switch can transfer only a limited amount of current, hence the use of GPIOs PC13 to PC15 and PI8^(b) in output mode is restricted: the frequency must be limited (check the datasheet) with a maximum load of 30 pF, and these I/Os cannot be used as a current source (for example to drive a LED).

a. Not available on all devices, check the datasheet.

b. Not available on all devices, check the datasheet.

Backup domain access

After a system reset, the Backup domain (RCC Backup domain control register RCC_BDCR, RTC registers, TAMP registers, backup registers, and backup SRAM) is protected against possible unwanted write accesses. To enable access to the Backup domain, set the DBP bit in the *PWR Backup domain control register (PWR_BDCR)* to enable access to the Backup domain.

Backup RAM

The Backup domain includes up to 4 Kbytes of backup RAM accessible in 32-, 16-, or 8-bit data mode. The backup RAM is supplied from the backup regulator in the Backup domain. When the backup regulator is enabled through BREN bit in the PWR_BDCR, the backup RAM content is retained in Standby and/or VBAT mode (it can be considered as an internal EEPROM if VBAT is always present).

The backup regulator can be ON or OFF, depending on if the application needs the backup RAM function in Standby or VBAT modes.

The backup RAM is read protected and mass erased when a tamper event occurs, this is to prevent confidential data (such as a cryptographic private key) from being accessed.

The backup RAM can be erased in the following ways:

- through the flash interface after a full product state regression
- after a tamper event
- after a Backup domain reset

V_{BAT} battery charging

When V_{DD} is present, it is possible to charge the external battery on VBAT through an internal resistance.

The V_{BAT} charging is done either through a 5 or a 1.5 kΩ resistor, depending upon the VBRS bit value in the PWR_BDCR register.

The battery charging is enabled by setting VBE bit in the PWR_BDCR register. It is automatically disabled in V_{BAT} mode.

10.5 PWR system supply voltage regulation

10.5.1 SMPS and LDO embedded regulators

The devices embed two internal regulators^(a), exclusively enabled by hardware on power-on reset, depending upon package configuration. To supply the V_{CORE} from an external source, it is possible to disable the regulator by setting the BYPASS bit in the PWR_SCCR register.

The BYPASS bit is written once after power-on reset. Written-once mechanism locks the register and any further write access is ignored. The system must be power cycled before writing a new value.

a. STM32H563/573xx devices only.

When V_{CORE} is supplied from an external source the externally applied voltage level must be reflected in the VOSx bits in the PWR_VOSCR register refer to [Startup with V_{CORE} provided from an external supply \(Bypass\)](#).

Both regulators can provide four different voltages (voltage scaling) and can operate in Stop modes.

10.5.2 V_{CORE} supply versus reset, voltage scaling, and low-power modes

After reset, the V_{CORE} is in VOS3.

When exiting the Stop or Standby mode, the voltage range is the VOS3.

10.5.3 Embedded voltage regulator operating modes

There are three different power modes: Run, Stop, and Standby modes.

Run mode

The voltage regulator provides full power to the V_{CORE} domain (core, memories, and digital peripherals). The regulator output voltage can be scaled by software to different voltage levels (VOS0, VOS1, VOS2, and VOS3) that are configured through the VOS bits in the PWR voltage scaling control register (PWR_VOSCR).

The VOS voltage scaling allows optimization of the power consumption when the system is clocked below the maximum frequency. By default, VOS3 is selected after system reset.

VOSx bits can be changed on-the-fly to adapt to the required system performance.

Stop mode

The voltage regulator supplies the V_{CORE} domain to retain the content of registers and internal memories. The regulator mode is selected through the SVOS bits in the PWR power mode control register (PWR_PMCR).

Stop mode power consumption can be further reduced using SVO4 (lower voltage level than VOS3) and even further with SVOS5.

Standby mode

The regulator is OFF and the V_{CORE} domains are powered down. The content of the registers and memories is lost except for the Standby circuitry and the Backup domain.

10.6 PWR power supply and temperature supervision

Power supply level monitoring is available on the following supplies:

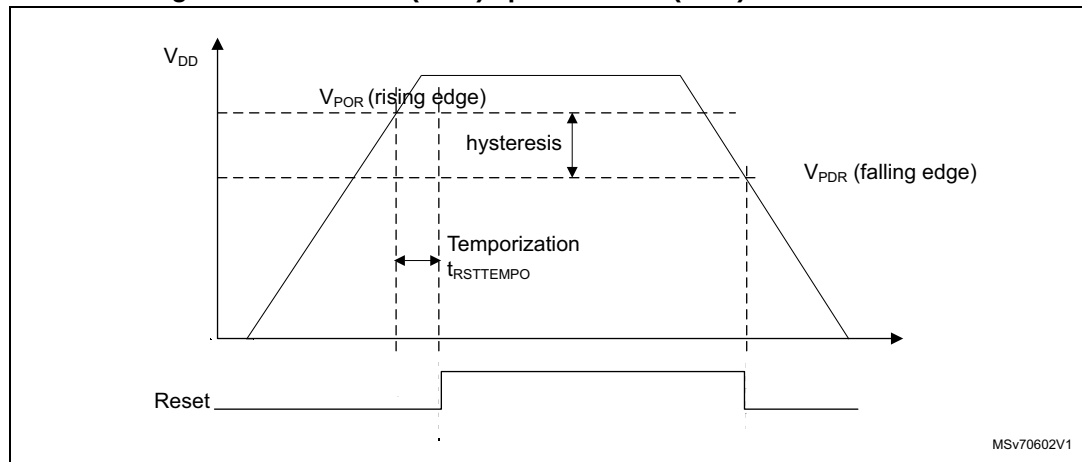
- V_{DD} via POR/PDR (see [Section 10.6.1](#)), BOR (see [Section 10.6.2](#)), and PVD monitor (see [Section 10.6.3](#))
- V_{DDA} via AVD monitor see ([Section 10.6.4](#))
- V_{BAT} via V_{BAT} threshold (see [Section 10.6.6](#))
- V_{DDIO2} via VDDIO2RDY bit (see [Section 10.11.14](#))
- Temperature monitoring (see [Section 10.6.7](#))

10.6.1 Power-on reset (POR)/power-down reset (PDR)

The system has an integrated POR/PDR circuitry that ensures proper startup operation.

The system remains in reset mode when V_{DD} is below a specified VPOR threshold, without the need for an external reset circuit. Once the supply level is above the VPOR threshold, the system is taken out of reset (see [Figure 46](#)). For more details concerning the reset thresholds refer to the electrical characteristics section of the datasheets.

Figure 46. Power-on (POR) / power-down (PDR) reset waveform



1. For thresholds and hysteresis values refer to the datasheets.

10.6.2 Brownout reset (BOR)

During power-on, the brownout reset (BOR) keeps the system under reset until the V_{DD} supply voltage reaches the specified V_{BOR} threshold.

The V_{BOR} threshold is configured through system option bytes:

- BOR OFF (BORH_EN = 0)
- BOR level 1 (V_{BOR1})
- BOR level 2 (V_{BOR2})
- BOR level 3 (V_{BOR3})

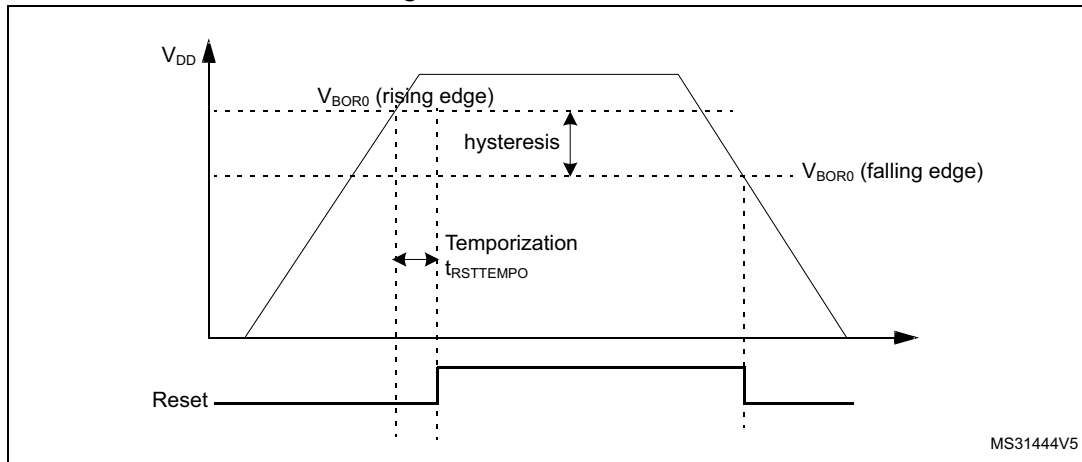
By default, BOR is OFF, it can be enabled by setting BORH_EN option bit.

For more details on the brownout reset thresholds, refer to the section “*Electrical characteristics*” of the product datasheets.

A system reset is generated when the BOR is enabled and V_{DD} supply voltage drops below the selected V_{BOR} threshold.

BOR can be disabled by programming the BORH_EN option bit to 0. To disable the BOR function, V_{DD} must have been higher than the POR threshold to start the system option byte programming sequence. Once BOR is disabled, the power-down is monitored by the PDR.

Figure 47. BOR thresholds



MS31444V5

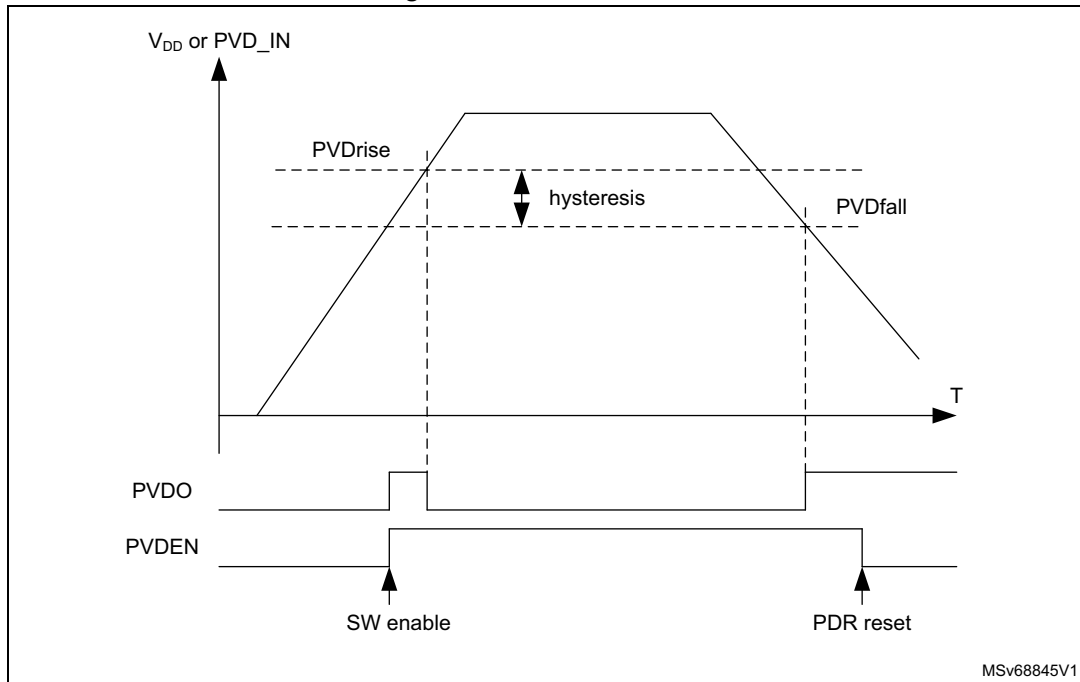
10.6.3 Programmable voltage detector (PVD)

The PVD can be used to monitor the V_{DD} power supply by comparing it to a threshold selected by the PLS[2:0] bits in the [PWR voltage monitor control register \(PWR_VMCR\)](#). The PVD can also be used to monitor a voltage level on the PVD_IN pin. In this case PVD_IN voltage is compared to the internal VREFINT level.

The PVD is enabled by setting the PVDE bit in [PWR voltage monitor control register \(PWR_VMCR\)](#).

A PVDO flag is available in the [PWR voltage monitor status register \(PWR_VMSR\)](#) to indicate if V_{DD} or PVD_IN voltage is higher or lower than the PVD threshold. This event is internally connected to the EXTI and can generate an interrupt, provided it has been enabled through the EXTI registers. The rising/falling edge sensitivity of the EXTI line must be configured according to PVD output behavior. As an example, if the EXTI line is configured to rising edge sensitivity, the interrupt is generated when V_{DD} or PVD_IN voltage drops below the PVD threshold. The service routine can then start an emergency shutdown.

Figure 48. PVD thresholds



1. For thresholds and hysteresis values, refer to the datasheets.

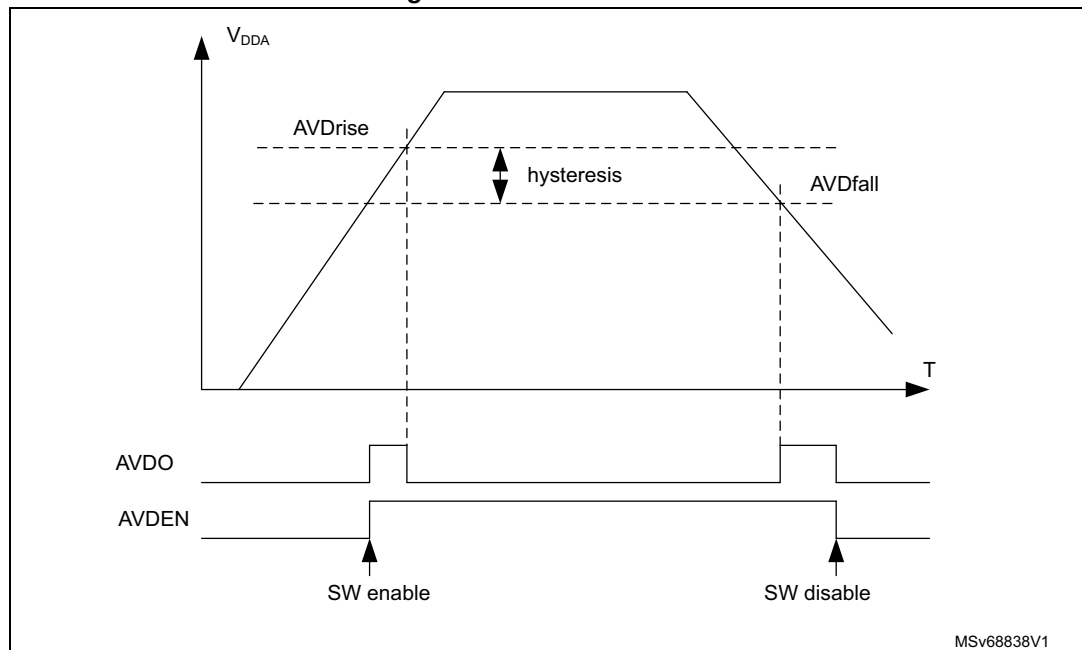
10.6.4 Analog voltage detector (AVD)

The AVD can be used to monitor the V_{DDA} supply by comparing it to a threshold selected by the $ALS[1:0]$ bits in the [PWR voltage monitor control register \(PWR_VMCR\)](#).

The AVD is enabled by setting the $AVDEN$ bit in [PWR voltage monitor control register \(PWR_VMCR\)](#).

An $AVDO$ flag is available in the [PWR voltage monitor status register \(PWR_VMSR\)](#) to indicate whether V_{DDA} is higher or lower than the AVD threshold. This event is internally connected to the EXTI and can generate an interrupt if enabled through the EXTI registers. The $AVDO$ interrupt can be generated when V_{DDA} drops below the AVD threshold and/or when V_{DDA} rises above the AVD threshold, depending on EXTI rising/falling edge configuration. As an example, the service routine can indicate when the V_{DDA} supply drops below a minimum level.

Figure 49. AVD thresholds



1. For thresholds and hysteresis values, refer to the datasheets.

10.6.5 V_{DDIO2} voltage monitor (IO2VM)

The IO2VM monitors the independent supply voltage V_{DDIO2} to ensure that the peripheral is in its functional supply range. The VDDIO2RDY flag (see [PWR voltage monitor control register \(PWR_VMCR\)](#)) indicates whether a valid supply is present or not.

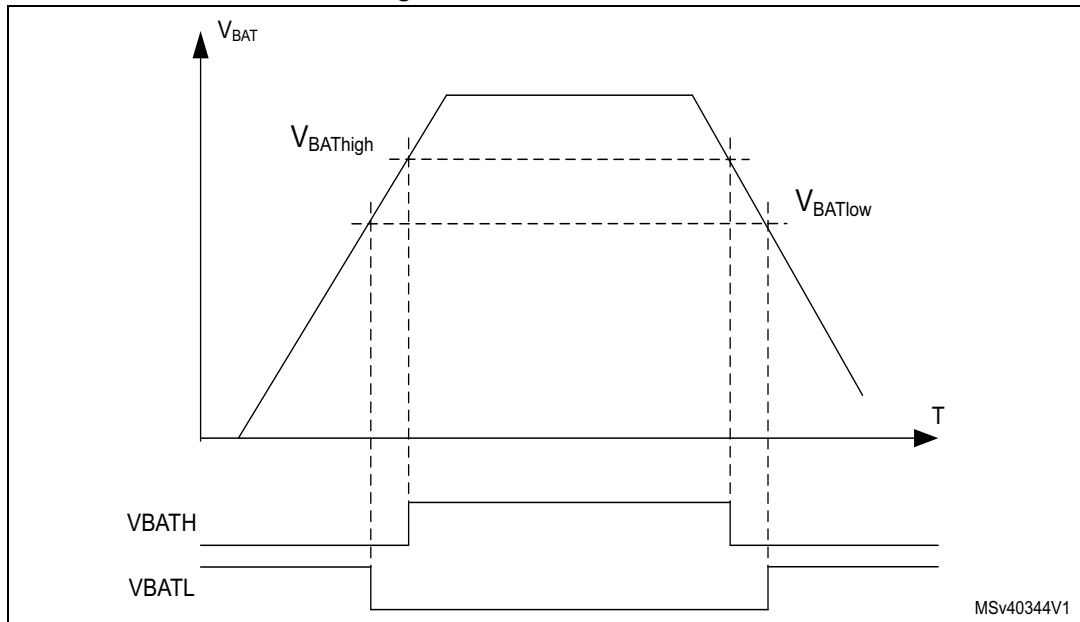
10.6.6 Backup domain voltage monitoring

In VBAT mode, the battery voltage supply (backup domain) can be monitored by comparing it with two threshold levels: $V_{BAThigh}$ and V_{BATlow} . The VBAT supply monitoring can be enabled/disabled via MONEN bit in [PWR Backup domain control register \(PWR_BDCR\)](#). When it is enabled, the battery voltage thresholds increase power consumption.

If the Backup domain voltage monitoring internal tamper is enabled in the TAMP peripheral (ITAMP1E = 1 in the TAMP_CR1 register), a tamper event is generated when the battery voltage is above the functional range.

Note: The Backup domain voltage is V_{DD} when present, V_{BAT} otherwise.

Figure 50. VBAT thresholds



1. For thresholds and hysteresis values, refer to the datasheets.

10.6.7 Temperature monitoring

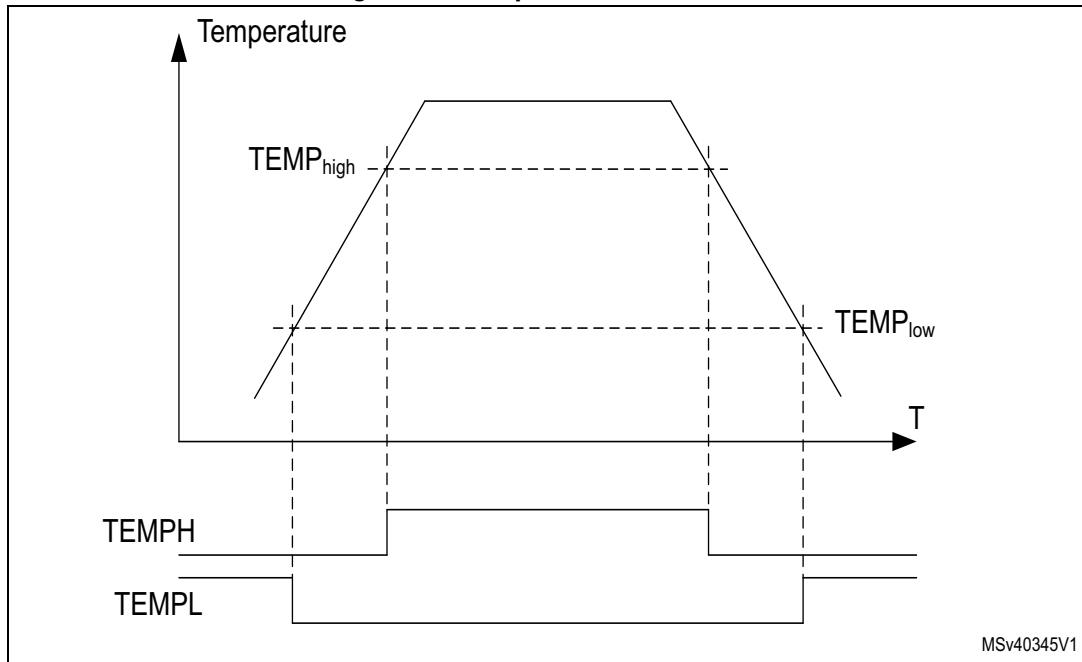
A dedicated temperature sensor cell is embedded in the power control. The junction temperature can be monitored by comparing it with two threshold levels, $TEMP_{HIGH}$ and $TEMP_{LOW}$. $TEMPH$ and $TEMP_L$ flags in the [PWR Backup domain status register \(PWR_BDSR\)](#), which indicates whether the device temperature is higher or lower than the threshold. The temperature monitoring can be enabled/disabled via $MONEN$ bit in [PWR Backup domain control register \(PWR_BDCR\)](#).

When enabled, the temperature thresholds increase power consumption. As an example the levels may be used to trigger a routine to perform temperature control tasks.

If the temperature monitoring internal tamper is enabled in the TAMP peripheral ($ITAMP2E = 1$ in the $TAMP_CR1$ register), a tamper event is generated when the temperature is above or below the functional range.

$TEMPH$ and $TEMP_L$ wake-up interrupts are available on the RTC tamper signals (see [Section 52: Tamper and backup registers \(TAMP\)](#)).

Figure 51. Temperature thresholds



MSv40345V1

1. For thresholds and hysteresis values, refer to the datasheets.

10.7 PWR management

10.7.1 Voltage scaling

The voltage regulator supporting voltage scaling with the following features:

- Run mode voltage scaling
 - VOS0: scale 0
 - VOS1: scale 1
 - VOS2: scale 2
 - VOS3: scale 3
- Stop mode voltage scaling
 - SVOS3: scale 3
 - SVOS4: scale 4
 - SVOS5: scale 5

For more details on voltage scaling values, refer to the product datasheets.

After reset, the system starts on the lowest Run mode voltage scaling (VOS3). The voltage scaling can be changed on-the-fly by software by programming VOS bits in PWR_VOSCR register, according to the required system performance. When exiting from the Stop mode or Standby mode, the Run mode voltage scaling is reset to the default VOS3 value.

Before entering Stop mode, the software must preselect the SVOS level in the PWR_PMCR registers. The Stop mode voltage scaling for SVOS4 and SVOS5 also sets the voltage regulator in Low-power mode, to further reduce power consumption.

10.7.2 Power management examples

Figure 52. Dynamic voltage scaling in Run mode

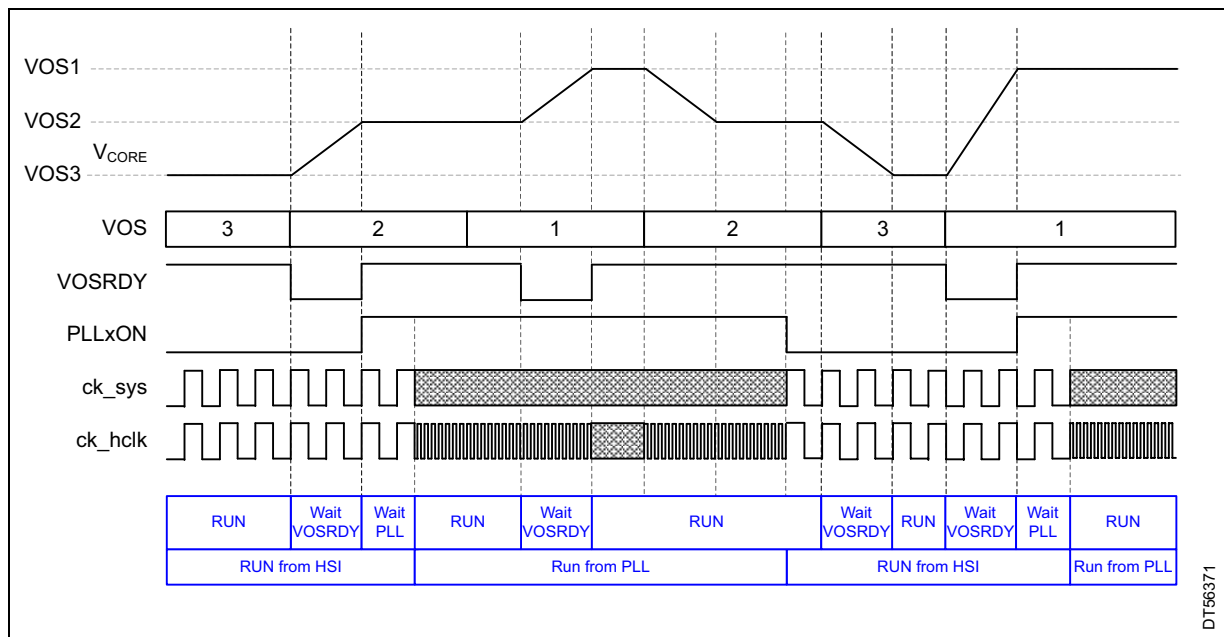


Figure 52 illustrates the following system operation sequence example:

- After reset, the system starts from HSI with VOS3.
- The system performance is increased to a medium-speed clock from the PLL with voltage scaling VOS2. To do this:
 - Program the voltage scaling to VOS2.
 - Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency by enabling the PLL.
 - Once the PLL is locked, switch the system clock.
- The system performance is increased to high-speed clock from the PLL with voltage scaling VOS1. To do this:
 - Program the voltage scaling to VOS1.
 - Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency.
- The system performance is reduced to a medium-speed clock with voltage scaling VOS2. To do this:
 - First decrease the system frequency.
 - Then decrease the voltage scaling to VOS2.
- The next step is to reduce the system performance to the HSI clock with voltage scaling VOS3. To do this:
 - Switch the clock to HSI.
 - Disable the PLL.
 - Decrease the voltage scaling to VOS3.

6. The system performance can then be increased to high-speed clock from the PLL. To do this:
 - a) Program the voltage scaling to VOS1.
 - b) Once the V_{CORE} supply has reached the required level indicated by VOSRDY, increase the clock frequency by enabling the PLL.
 - c) Once the PLL is locked, switch the system clock.

When the system performance (clock frequency) is changed, VOS must be set accordingly, otherwise the system can be unreliable.

10.8 Power modes

By default, the microcontroller is in Run mode after a system or a power reset. Several low-power modes are available to reduce consumption when there is no need to keep the CPU running, for example when waiting for an external event. The user can select the mode that gives the best compromise between low-power consumption, short startup time, and wake-up sources.

The device features the following low-power modes:

- Sleep mode
CPU clock off, peripherals such as NVIC and SysTick can run and wake-up the CPU when an interrupt or an event occurs. Refer to [Section 10.8.4](#).
- Stop mode
Achieves the lowest power consumption, while retaining the content of SRAM and registers. All clocks in the core domain are stopped. The PLL, the HSE crystal oscillators, HSI (except if HSIKERON is set), HSI48 and CSI RC (except if CSIKERON is set) are disabled. The LSE or LSI is still running.
The RTC can remain active (Stop mode with RTC, Stop mode without RTC).
The system clock when exiting from Stop mode can be either HSI up to 64 MHz or CSI, depending on software configuration.
Refer to [Section 10.8.5](#).
- Standby mode
This mode achieves the lowest power consumption with BOR. The internal regulator is switched off so that the core domain is powered off. The PLL, the HSI RC, HSI48, the CSI RC, and the HSE crystal oscillators are also switched off.
The RTC can remain active (Standby mode with RTC, Standby mode without RTC).
The brownout reset (BOR) remains active.
The state of the I/O (except I/Os used by standby mode) during Standby mode can be retained.
After entering Standby mode, SRAMs and register contents are lost except for registers and backup SRAM in the Backup domain and Standby circuitry.
The device exits Standby mode when an external reset (NRST pin), an IWDG reset, WKUP pin event (configurable rising or falling edge), an RTC event occurs (alarm, periodic wake-up, timestamp), or a tamper detection. The tamper detection can be

raised either due to external pins or due to an internal failure detection.

The system clock after wake-up is HSI at 32 MHz.

Refer to [Section 10.8.6](#).

[Table 107](#) shows the power modes overview.

Table 107. Low-power mode summary

Mode name	Entry	Wake-up source ⁽¹⁾	Wake-up system clock	Effect on clocks	Voltage regulators
Sleep (Sleep-now or Sleep-on-exit)	WFI or Return from ISR	Any interrupt	Same as before entering Sleep mode	– CPU clock OFF – No effect on other clocks or analog clock sources	VOS3, VOS2, VOS1, or VOS0
	WFE	Wake-up event			
Stop	LPMS = 0 + SLEEPDEEP bit + WFI or Return from ISR or WFE	– Any EXTI line (configured in the EXTI registers) – Specific peripherals events ⁽²⁾	– CSI when STOPWUCK = 1 in RCC_CFGR – HSI with the frequency before entering Stop mode, up to 64 MHz, when STOPWUCK = 0	– All clocks OFF except LSI and LSE – HSI or CSI can be enabled temporarily when requested by software	SVOS3, SVOS4, or SVOS5
Standby	LPMS = 1 + SLEEPDEEP bit + WFI or Return from ISR or WFE	WKUP pin edge, RTC event, IWDG reset, external reset in NRST pin	HSI clock at 32 MHz	All clocks OFF except LSI and LSE	OFF

1. Refer to [Table 108](#).

2. Peripherals able to wake-up the system from Stop mode (possible only when SVOS3 is selected before entering Stop mode).

Table 108. Functionalities depending on the working mode⁽¹⁾

Peripheral	Run	Sleep	Stop				Standby		VBAT
			Available	Wake-up capability			Available	Wake-up capability	
				SVOS3	SVOS4	SVOS5			
CPU	Y	-	-	-	-	-	-	-	-
Flash memory	O	O	(2)	-	-	-	-	-	-
SRAM1	Y ⁽³⁾	Y ⁽³⁾	O ⁽⁴⁾	-	-	-	-	-	-
SRAM2	Y ⁽³⁾	Y ⁽³⁾	O ⁽⁴⁾	-	-	-	-	-	-
SRAM3	Y ⁽³⁾	Y ⁽³⁾	O ⁽⁴⁾	-	-	-	-	-	-
BKPSRAM	O	O	O	-	-	-	O	-	O
FMC	O	O	-	-	-	-	-	-	-
OCTOSPI1	O	O	-	-	-	-	-	-	-
Backup registers	Y	Y	Y	-	-	-	Y	-	Y
Brownout reset (BOR)	Y	Y	Y	-	-	-	Y	-	-

Table 108. Functionalities depending on the working mode⁽¹⁾ (continued)

Peripheral	Run	Sleep	Stop				Standby		VBAT
			Available	Wake-up capability			Available	Wake-up capability	
				SVOS3	SVOS4	SVOS5			
Programmable voltage detector (PVD)	O	O	O	O	O	O	-	-	-
Analog voltage detector (AVD)	O	O	O	O	O	O	-	-	-
GPDMA	O	O	-	-	-	-	-	-	-
High-speed internal (HSI)	O	O	-	-	-	-	-	-	-
Oscillator HSI48	O	O	-	-	-	-	-	-	-
High-speed external (HSE)	O	O	-	-	-	-	-	-	-
Low-speed internal (LSI)	O	O	O	O	-	-	O	-	-
Low-speed external (LSE)	O	O	O	-	-	-	O	-	O
Low-power RC oscillator (CSI)	O	O	-	-	-	-	-	-	-
Clock security system (CSS)	O	O	-	-	-	-	-	-	-
Clock security system on LSE	O	O	O	O	O ⁽⁵⁾	O ⁽⁵⁾	O	O	O
Backup domain voltage and temperature monitoring	O	O	O	O	O ⁽⁵⁾	O ⁽⁵⁾	O	O	O
RTC/TAMP	O	O	O	O	O	O	O	O	O
Number of TAMP tamper pins	8	8	8	-	-	-	4	-	2
USB FS, UCPD	O	O	O	O	-	-	-	-	-
USARTx	O	O	O	O	-	-	-	-	-
Low-power UART (LPUART)	O	O	O	O	-	-	-	-	-
I2Cx	O	O	O	O	-	-	-	-	-
I3Cx	O	O	O	O	-	-	-	-	-
HDMI_CEC	O	O	O	O	-	-	-	-	-
SPIx	O	O	O	O	-	-	-	-	-
FDCANx	O	O	-	-	-	-	-	-	-
SDMMCx	O	O	-	-	-	-	-	-	-
Ethernet	O	O	O	O	-	-	-	-	-
PLAY	O	O	O	O	-	-	-	-	-
SAIx	O	O	-	-	-	-	-	-	-
ADCx (x = 1, 2, 3)	O	O	-	-	-	-	-	-	-
DAC1 (2 converters)	O	O	O	-	-	-	-	-	-
COMPx (x = 1, 2)	O	O	O	O	-	-	-	-	-
VREFBUF	O	O	O	-	-	-	-	-	-
Temperature sensor (DTS)	O	O	O	O	-	-	-	-	-

Table 108. Functionalities depending on the working mode⁽¹⁾ (continued)

Peripheral	Run	Sleep	Stop				Standby		VBAT
			Available	Wake-up capability			Available	Wake-up capability	
				SVOS3	SVOS4	SVOS5			
Timers (TIMx)	O	O	-	-	-	-	-	-	-
Low-power timer LPTIMx	O	O	O	O	-	-	-	-	-
Independent watchdog (IWDG)	O	O	O	O	O ⁽⁵⁾	O ⁽⁵⁾	O	O	-
Window watchdog (WWDG)	O	O	-	-	-	-	-	-	-
SysTick timer (SYSTICK)	O	O	O	-	-	-	-	-	-
Digital camera interface (DCMI)	O	O	-	-	-	-	-	-	-
Parallel synchronous slave interface (PSSI)	O	O	-	-	-	-	-	-	-
CORDIC coprocessor (CORDIC)	O	O	-	-	-	-	-	-	-
Coupling and chaining bridge (CCB)	O	O	-	-	-	-	-	-	-
Filter mathematical accelerator (FMAC)	O	O	-	-	-	-	-	-	-
Random number generator (RNG)	O	O	-	-	-	-	-	-	-
AES and secure AES (AES, SAES)	O	O	-	-	-	-	-	-	-
Public key accelerator (PKA)	O	O	-	-	-	-	-	-	-
On-the-fly decryption (OTFDEC)	O	O	-	-	-	-	-	-	-
HASH accelerator	O	O	-	-	-	-	-	-	-
CRC calculation unit	O	O	-	-	-	-	-	-	-
GPIOs	O	O	-	-	-	-	O ⁽⁶⁾	O ⁽⁷⁾	-
EXTI	O	O	O	O	O	O	-	-	-

1. Y = yes (enabled). O = optional (disabled by default, can be enabled by software). - = not available.
HSI or CSI are available as kernel clock for peripherals only in SVOS3.
2. The memory can be configured in Low-power mode. By default, it is not in Low-power mode during Stop (SVOS3, SVOS4).
3. The SRAM clock can be gated on or off independently. By default clock is enabled in Run and Sleep modes.
4. The SRAMs can be powered on or off independently. By default, they are not in Power-off mode during Stop.
5. Wake-up with internal tamper.
6. GPIOs state can be retained during Standby mode. By default GPIOs states are not retained.
7. 8 pins are capable of wake-up from Standby mode: PA0, PA2, PB7, PC1, PC13, PD2, PD3 and PI8 (not available on all devices, check the datasheet).

In addition, the power consumption in Run mode can be reduced by slowing down the system clocks, configuring voltage scaling to lower power ranges, and by gating the clocks to the APB and AHB peripherals when they are not used.

Debug mode

By default, the debug connection is lost if the application puts the MCU in Stop or Standby mode while the debug features are used. This is due to the fact that the Cortex-M33 core is no longer clocked.

However, by setting some configuration bits in the DBGMCU control registers, the software can be debugged even when using the low-power modes extensively. For more details, refer to [Section 66.2.5: Debug and low-power modes](#).

10.8.1 Slowing down system clocks

In Run mode, the speed of the system clocks (SYSCLK, HCLK, PCLK) can be reduced by programming the prescaler registers. These prescalers can also be used to slow down the peripherals before entering the Sleep mode.

For more details, refer to [Section 11: Reset and clock control \(RCC\)](#).

10.8.2 Peripheral clock gating

In Run mode, the HCLK and PCLK for individual peripherals and memories can be stopped at any time to reduce the power consumption.

To further reduce the power consumption in Sleep mode, the peripheral clocks can be disabled before executing the WFI or WFE instructions.

The peripheral clock gating is controlled by the RCC_AHBxENR and RCC_APBxENR registers.

Disabling the peripherals clocks in Sleep mode can be performed automatically by resetting the corresponding bit in the RCC_AHBxLPENR and RCC_APBxLPENR registers.

10.8.3 Low-power modes

Entering a low-power mode

The MCU enters in low-power modes by executing the WFI (wait for interrupt), or WFE (wait for event) instructions, or when the SLEEPONEXIT bit in the Cortex-M33 system control register is set on Return from ISR.

Entering into a low-power mode through WFI or WFE is executed only if no interrupt is pending or no event is pending.

Exiting a low-power mode

The MCU exits the Sleep or Stop mode according to how the low-power mode was entered:

- If the WFI instruction or Return from ISR was used to enter the low-power mode, any peripheral interrupt acknowledged by the NVIC can wake up the device.
- If the WFE instruction is used to enter the low-power mode, the MCU exits the low-power mode as soon as an event occurs. The wake-up event can be generated either by:
 - an NVIC IRQ interrupt:
 - > When SEVONPEND = 0 in the Cortex-M33 system control register
By enabling an interrupt in the peripheral control register and in the NVIC. When the MCU resumes from WFE, the peripheral interrupt pending bit and the NVIC peripheral IRQ channel pending bit (in the NVIC interrupt clear pending register) must be cleared. Only NVIC interrupts with high enough priority can wake up and interrupt the MCU.
 - > When SEVONPEND = 1 in the Cortex-M33 system control register
By enabling an interrupt in the peripheral control register and optionally in the

NVIC. When the MCU resumes from WFE, the peripheral interrupt pending bit and when enabled the NVIC peripheral IRQ channel pending bit (in the NVIC interrupt clear pending register) must be cleared. All NVIC interrupts wake up the MCU, even the disabled ones. Only enabled NVIC interrupts with high enough priority can wake up and interrupt the MCU.

- an event:
 - > Configuring an EXTI line in event mode. When the CPU resumes from WFE, it is not necessary to clear the EXTI peripheral interrupt pending bit or the NVIC IRQ channel pending bit, as the pending bits corresponding to the event line are not set. It may be necessary to clear the interrupt flag in the peripheral.

The MCU exits Standby mode through an external reset (NRST pin), an IWDG reset, a rising edge on one of the enabled WKUPx pins or a RTC/TAMP event (see [Figure 685: RTC block diagram](#)).

After waking up from Standby mode, the program execution restarts in the same way as after a reset (boot pin sampling, option bytes loading, reset vector is fetched).

Caution: When the device is in Stop mode, a peripheral interrupt powers on an internal oscillator. The corresponding NVIC interrupt channel must be enabled to allow the interrupt to exit the device from Stop mode. It is not allowed to disable a peripheral interrupt by disabling only the NVIC channel while keeping the peripheral interrupt enable, as the device could remain in Stop mode with clock ON.

10.8.4 Sleep mode

I/O states in Sleep mode

In Sleep mode, all I/O pins keep the same state as in Run mode.

Entering the Sleep mode

The MCU enters the Sleep mode as described in [Entering a low-power mode](#), when the SLEEPDEEP bit in the Cortex-M33 system control register is cleared (see the table below for details on how to enter the Sleep mode).

Exiting the Sleep mode

The MCU exits the Sleep mode as described in [Exiting a low-power mode](#) (see the table below for details on how to exit the Sleep mode).

Table 109. Sleep mode

Sleep mode	Description
Mode entry	WFI (wait for interrupt) or WFE (wait for event) while: – SLEEPDEEP = 0 – No interrupt (for WFI) or event (for WFE) pending Refer to the Cortex-M33 system control register.
	On return from ISR while: – SLEEPDEEP = 0 and – SLEEPONEXIT = 1 – No interrupt pending Refer to the Cortex-M33 system control register.
Mode exit	If WFI or Return from ISR was used for entry Interrupt (see Table 152: STM32H562/563/573xx vector table) If WFE was used for entry and SEVONPEND = 0: Wake-up event (see Section 18.4: EXTI functional description) If WFE was used for entry and SEVONPEND = 1: Interrupt even when disabled in NVIC (see Table 152: STM32H562/563/573xx vector table) or wake-up event (see Section 18.4: EXTI functional description)
Wake-up latency	None

10.8.5 Stop mode

The Stop mode is based on the Cortex-M33 Deepsleep mode combined with the peripheral clock gating. The voltage regulator is configured by SVOSx bits (the selected SVOS4 and SVOS5 levels add an additional startup delay when exiting from system Stop mode). In Stop mode, all clocks in the core domain are stopped. The PLL, HSI, HSI48, CSI and HSE oscillators are disabled.

It is possible to keep the HSI or CSI clock enabled during Stop mode, to be quickly available as kernel clock for peripherals.

All SRAMs and register contents are preserved, but the SRAMs can be totally or partially switched off to further reduced consumption. The user can select which memory is discarded during Stop mode by means of xxSO bits in PWR_PMCR registers.

Table 110. Memory shut-off block selection

Selection bit	Shut-off block in Stop mode	STM32H523/533xx	STM32H562/563/573xx	STM32H543/553xx
SRAM1SO	AHB SRAM1	X	X	X
SRAM2_48SO	AHB SRAM2 48-Kbyte	X	X	X
SRAM2_16SO	AHB SRAM2 16-Kbyte	-	X	-
SRAM2_16HSO	AHB SRAM2 16-Kbyte	X	-	X
SRAM2_16LSO	AHB SRAM2 16-Kbyte	X	-	X
SRAM3SO	AHB SRAM3	X	-	X
ETHERNETSO	ETHERNET RAM	-	X	X

The BOR is always available in Stop mode.

I/O states in Stop mode

In the Stop mode, all I/O pins keep the same state as in the Run mode.

Entering the Stop mode

The MCU enters the Stop mode as described in [Entering a low-power mode](#), when the SLEEPDEEP bit in the Cortex-M33 system control register is set (see [Table 111](#) for details on how to enter the Stop mode).

If the flash memory programming is ongoing, the Stop mode entry is delayed until the memory access is finished.

If an access to the APB domain is ongoing, the Stop mode entry is delayed until the APB access is finished.

In Stop mode, the following features can be selected by programming the individual control bits:

- The independent watchdog (IWDG) is started by writing to its key register or by hardware option. Once started, it can be stopped only by a reset (see [Section 49.4: IWDG functional description](#)).
- The real-time clock (RTC) is configured by the RTCEN bit in the [RCC Backup domain control register \(RCC_BDCR\)](#).
- The internal RC oscillator LSI clock is configured by the LSION bit in RCC_BCDR.
- The external 32.768 kHz oscillator (LSE) is configured by the LSEON bit in RCC_BCDR.

The AVD and the PVD can be used in Stop mode. If they are not needed, they must be disabled by software to save their power consumptions.

The ADCx ($x = 1, 2, 3^{(a)}$), the DAC1 (two channels), the temperature sensor, and the VREFBUF can consume power during the Stop mode, unless they are disabled before entering this mode.

Exiting the Stop mode

The MCU exits Stop mode by enabling an EXTI interrupt or event depending on how the low-power mode was entered. Some peripherals are able to wake up the system (refer to [Table 157: EXTI line connections](#)) from Stop mode, this is only possible when SVOS3 is selected before entering Stop mode.

Note: When wake-up from Stop with peripherals is needed, SVOS3 must be selected.

When exiting Stop mode by issuing an interrupt or a wake-up event, CSI is selected as system clock if bit STOPWUCK is set in RCC clock configuration register 1 (RCC_CFGR). The HSI oscillator is selected as system clock if STOPWUCK is cleared. The wake-up time is shorter when CSI is selected as wake-up system clock. The HSI selection allows a wake-up at higher frequency (up to 64 MHz).

The MCU exits Stop mode by enabling an EXTI interrupt or event depending on how the low-power mode was entered.

a. Only available on STM32H543/553xx devices.

When exiting the Stop mode, the MCU is in Run mode, VOS3.

Table 111. Stop mode

Stop mode	Description
Mode entry	WFI (wait for interrupt) or WFE (wait for event) while: – SLEEPDEEP bit is set in Cortex-M33 system control register – No interrupt (for WFI) or event (for WFE) pending – LPMS = 0 in PWR_PMCR
	On Return from ISR while: – SLEEPDEEP bit is set in Cortex-M33 system control register – SLEEPONEXIT = 1 – No interrupt pending – LPMS = 0 in PWR_PMCR
	<i>Note: To enter Stop mode, all EXTI line pending bits (in the EXTI rising edge pending register 2 (EXTI_RPR2)), and the peripheral flags generating wake-up interrupts must be cleared. Otherwise, the Stop mode entry procedure is ignored and the program execution continues.</i>
Mode exit	If WFI or Return from ISR was used for entry: - any EXTI line configured in interrupt mode (the corresponding EXTI interrupt vector must be enabled in the NVIC). The interrupt source can be external interrupts or peripherals with wake-up capability (see Table 152: STM32H562/563/573xx vector table). - any peripheral interrupt occurring when the AHB/APB clocks are present due to an autonomous peripheral clock request (the peripheral vector must be enabled in the NVIC)
	If WFE was used for entry and SEVONPEND = 0: - any EXTI line configured in event mode (see Section 18.4: EXTI functional description).
	If WFE was used for entry and SEVONPEND = 1: - any EXTI line configured in interrupt mode (even if the corresponding EXTI interrupt vector is disabled in the NVIC). The interrupt source can be external interrupts or peripherals with wake-up capability (see Table 152: STM32H562/563/573xx vector table). - any EXTI line is configured in event mode (see Section 18.4: EXTI functional description)
Wake-up latency	<i>Note: All peripheral clocks must be enabled to allow this peripheral to generate a wake-up from Stop interrupt ([PERIPH]EN and [PERIPH]LPEN bits must be set in the RCC, and a functional independent clock must be selected).</i> Longest wake-up time between: HSI or CSI wake-up time and flash memory wake-up time from Stop mode.

10.8.6 Standby mode

The lowest power mode in which the BOR is active is the Standby mode. It is based on the Cortex-M33 Deepsleep mode, with the voltage regulators disabled. The PLL, HSI, HSI48, CSI, and HSE oscillators are also switched off.

The SRAMs and register contents are lost except for registers in the Backup domain and Standby circuitry (see [Figure 42](#) and [Figure 43](#)).

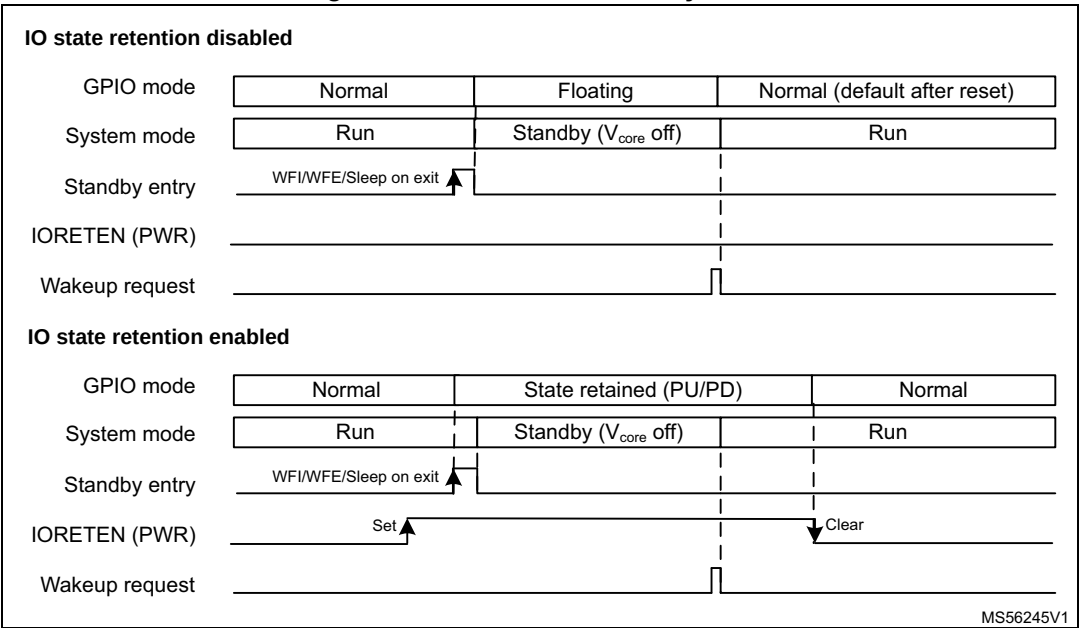
The BOR is always available in Standby mode.

I/O states in Standby mode

In the Standby mode, the I/Os are by default in floating state. If the IORETEN bit in the PWR_IOPRETR register is set, the I/Os output state is retained. I/O retention mode is enabled for all I/Os except those supporting the standby functionality and JTAG I/Os (PA13, PA14, PA15, and PB4). When entering into Standby mode, the state of the output is sampled, and pull-up or pull-down resistor are set to maintain the I/O output during Standby mode.

If the JTAGIORETEN bit in the PWR_IOPRETR register is set, the I/Os output state is retained. I/O retention mode is enabled for PA13, PA14, PA15, and PB4 (default JTAG pull-up/pull-down after wake-up is not enabled).

Figure 53. I/O states in Standby mode



After wake-up from Standby mode, as long as IORETEN (or JTAGIORETEN for JTAG I/Os) is set, the retained stated (pull-up/pull-down) remains applied.

The GPIO pin state before standby can be identified with GPIO_IDR register (when both GPIO port clock and input buffer are enabled).

The application can release the I/O state (clear the retained Pull-up/Pull-down) by clearing the IORETEN (or JTAGIORETEN for JTAG IOs) bit, before or after reconfiguring the GPIOs and related peripherals. The GPIOs can then be configured in a known state before releasing the retained state.

The RTC outputs on PC13 and PI8^(a) are functional in Standby mode. PC14 and PC15 used for LSE are also functional. Eight wake-up (WKUPx, x = 1 to 8) and four RTC tamper pins are available.

a. Not available on all devices, check the datasheet.

Entering Standby mode

The MCU enters the Standby mode as described in [Entering a low-power mode](#), when the SLEEPDEEP bit in the Cortex-M33 system control register is set (see [Table 112](#) for details on how to enter Standby mode).

In Standby mode, the following features can be selected by programming individual control bits:

- The independent watchdog (IWDG) is started by writing to its Key register or by hardware option. Once started, it can be stopped only by a reset (see [Section 49.4: IWDG functional description](#)).
- The real-time clock (RTC) is configured by the RTCEN bit in [RCC Backup domain control register \(RCC_BDCR\)](#).
- The internal RC oscillator LSI clock, is configured by the LSION bit in [RCC_BDCR](#).
- The external 32.768 kHz oscillator (LSE) is configured by the LSEON bit in [RCC_BDCR](#).
- The I/Os retention is configured by the IORETEN bit in the [PWR_IOPRETR](#) register.

Exiting Standby mode

The MCU exits the Standby mode as described in [Exiting a low-power mode](#). The SBF status flag in the [PWR status register \(PWR_PMSR\)](#) indicates that the MCU was in Standby mode. All registers are reset after wake-up from Standby except for [PWR Backup domain control register \(PWR_BDCR\)](#) and [PWR I/O retention register \(PWR_IOPRETR\)](#) (see [Table 112](#) for more details on how to exit Standby mode).

When exiting Standby mode, I/Os output state that were retained during Standby through IORETEN bit, keep this configuration upon exiting Standby mode until the IORETEN bit in [PWR_IOPRETR](#) register is cleared. Once IORETEN is cleared, the I/Os are configured to their reset values, or to the pull-up/pull-down state according to the [GPIOx_PUPDR](#) registers.

For I/Os, with a pull-up or pull-down predefined after reset (some JTAG/SWD I/Os), in case those pull-up or pull-down are different from the retained values during Standby, both a pull-down and pull-up are applied until IORETEN is cleared, releasing the retained value.

Also in case the [GPIOx_PUPDR](#) values programmed after exiting from Standby are different from the retained values during Standby, both a pull-down and pull-up are applied until IORETEN is cleared, releasing the retained value.

Table 112. Standby mode

Standby mode	Description
Mode entry	WFI (wait for interrupt) or WFE (wait for event) while: – SLEEPDEEP bit is set in Cortex-M33 system control register – No interrupt (for WFI) or event (for WFE) pending – LPMS = 1 in PWR_PMCR – WUFx bits cleared in PWR_WUSR
	On Return from ISR while: – SLEEPDEEP bit is set in Cortex-M33 system control register – SLEEPONEXIT = 1 – No interrupt pending – LPMS = 1 in PWR_PMCR – WUFx bits cleared in PWR_WUSR – RTC/TAMP flags corresponding to the chosen wake-up source, cleared
Mode exit	WKUPx pin edge, RTC event, external Reset in NRST pin, IWDG Reset, BOR reset
Wake-up latency	Reset phase

10.8.7 Power modes output pins

In order to help the debug, three signals are available as device pins alternate functions:

- **CSLEEP**
When set, CSLEEP indicates that the system is in Sleep mode: WFI or WFE has been executed.
When cleared, CSLEEP indicates that the system is in Run mode.
- **CDSTOP**
When set, CDSTOP indicates that the system is in CStop mode, meaning that the following conditions are fulfilled:
 - WFI or WFE has been executed with CPU SLEEPDEEP = 1.
 - No AHB/APB clock is running.
When cleared, CDSTOP indicates that the system is not in CStop mode: AHB/APB clocks are running.

The table below explains the MCU power mode depending on these signals states.

Table 113. Power modes output states versus MCU power modes

CSLEEP	CSTOP	MCU power modes ⁽¹⁾
0	0	Run mode
1	0	Sleep mode or Stop mode, with AHB/APB clocks running
1	1	Stop mode

1. CSLEEP and CDSTOP are generated in core domain, consequently they are not driven in Standby mode.

10.9 PWR security and privileged protection

10.9.1 PWR security protection

When the TrustZone security is activated by the TZEN option byte in the flash memory option byte configuration register, some PWR register fields can be secured against nonsecure access.

The PWR TrustZone security allows the following features to be secured through the PWR_SECCFGR register:

- Low-power mode
- Wake-up (WKUP) pins
- Voltage detection and monitoring
- VBAT mode
- I/Os retention configuration

Other PWR configuration bits are secure when:

- The system clock selection is secure in RCC: the voltage scaling (VOS) configuration is secure.
- The UCPD1 is secure in the GTZC: the PWR_UCPDR register is secure.

[Table 114](#) gives a summary of the PWR secured bits following the security configuration bit in PWR_SECCFGR.

A nonsecure access to a secure-protected register bit is denied:

- The secured bits are not written (WI) with a nonsecure write access.
- The secured bits are read as 0 (RAZ) with a nonsecure read access.

A nonsecure write access to PWR_SECCFGR is WI and generates an illegal access event and an interrupt if enabled in the GTZC. It can be read with a nonsecure read access.

When the TrustZone security is disabled (TZEN = 0xC3), PWR_SECCFGR is RAZ/WI and all other registers are nonsecure.

Table 114. PWR security configuration summary

Secure configuration register	Security configuration bit	Register name	Secured bits	Nonsecure access on secure bits
PWR_SECCFGR	Not applicable ⁽¹⁾	PWR_SECCFGR	All bits	Read OK. WI and illegal access event
PWR_SECCFGR	At least one bit is set	PWR_PRIVCFGR	SPRIV	Read OK. WI
PWR_SECCFGR	LPMSEC	PWR_PMCR	All bits	WI
PWR_SECCFGR	VUSBSEC	PWR_USBSCR	All bits	WI
PWR_SECCFGR	VBSEC	PWR_BDCR	All bits	WI
		PWR_DBPCR	All bits	WI
PWR_SECCFGR	RETSEC	PWR_IORETR	All bits	WI

Table 114. PWR security configuration summary (continued)

Secure configuration register	Security configuration bit	Register name	Secured bits	Nonsecure access on secure bits
PWR_SECCFGR	WUPxSEC (x = 1 to 8)	PWR_WUCR	WUPENx	RAZ/WI
			WUPPx	RAZ/WI
			WUPPUPD	RAZ/WI
		PWR_WUSCR	CWUFx	WI
GTZC_TZSC_SECCFGR	UCPD1SEC	PWR_UCPDR	All bits	RAZ/WI
RCC_SECCFGR	SYSCLKSEC	PWR_VOSCR	VOS[1:0]	RAZ/WI
RCC_SECCFGR	SCMSEC	PWR_SCCR	All bits	WI
RCC_SECCFGR	SCMSEC	PWR_VMCR	All bits	WI

1. PWR_SECCFGR is always secure.

10.9.2 PWR privileged protection

By default, after a reset, all registers can be read or written with both privileged and unprivileged accesses, except PWR_PRIVCFGR, which can be written only with privileged access. PWR_PRIVCFGR can be read by secure and non secure, privileged and unprivileged accesses.

The SPRIV bit in PWR_PRIVCFGR can be written only with secure privileged access. This bit configures the privileged access of all PWR secure functions (defined by PWR_SECCFGR, GTZC, RCC, or GPIO, as shown in [Table 114](#)).

When the SPRIV bit is set in PWR_PRIVCFGR:

- The PWR secure bits can be written only with privileged access, including PWR_SECCFGR.
- The PWR secure bits can be read only with privileged access except PWR_SECCFGR and PWR_PRIVCFGR that can be read by privileged or unprivileged access.
- An unprivileged access to a privileged PWR bit or register is discarded: the bits are read as 0 and the write to these bits is ignored (RAZ/WI).

The NSPRIV bit of PWR_PRIVCFGR can be written only with privileged access, secure or nonsecure. It configures the privileged access of all PWR securable functions configured as nonsecure (defined by PWR_SECCFGR, GTZC, RCC, or GPIO, see [Table 114](#)).

When the NSPRIV bit is set in PWR_PRIVCFGR:

- The PWR securable bits configured as nonsecure can be written only with privileged access.
- The PWR securable bits configured as nonsecure can be read only with privileged access except PWR_PRIVCFGR, which can be read by privileged or unprivileged accesses.
- The VOSRDY and BOOSTRDY bits in PWR_VOSR, PWR_SR, PWR_SVMSR, PWR_BDSR and PWR_WUSR can be read with privileged or unprivileged accesses.
- An unprivileged access to a privileged PWR bit or register is discarded: the bits are read as 0, and the write to these bits is ignored (RAZ/WI).

10.10 PWR interrupts

[Table 115](#) gives a summary of the interrupt sources and the way to control them.

Table 115. PWR interrupt requests

Interrupt vector	Interrupt event	Event flag	Enable control bit	Interrupt clear method	Exit Sleep, Stop modes	Exit Standby modes
PVD/AVD output	Programmable voltage detector through EXTI line 16	PVDO/AVDO	EXTI line 16 enabled	Write EXTI PIF16 = 1	Yes	No

10.11 PWR registers

The PWR registers can be accessed in word, half-word and byte format, unless otherwise specified.

10.11.1 PWR power mode control register (PWR_PMCR)

This register is protected against nonsecure access when LPMSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when LPMSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when LPMSEC = 0 and NSPRIV = 1.

Address offset: 0x000

Reset value: 0x0000 000C

STM32H562/563/573xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	SRAM1 SO	SRAM2 _48SO	SRAM2 _16SO	SRAM3 SO	Res.	Res.	Res.	Res.	Res.	Res.	ETHER NETSO
					rw	rw	rw	rw							rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	AVD_ READY	BOOST E	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	SVOS[1:0]		Res.	LPMS
		rw	rw			rw		rw				rw	rw		rw

Bits 31:27 Reserved, must be kept at reset value.

Bit 26 **SRAM1SO**: AHB SRAM1 shut-off in Stop mode

0: AHB RAM1 content is kept in Stop mode.

1: AHB RAM1 content is lost in Stop mode.

Bit 25 **SRAM2_48SO**: AHB SRAM2 48-Kbyte shut-off in Stop mode.

0: AHB RAM2 48-Kbyte content is kept in Stop mode.

1: AHB RAM2 48-Kbyte content is lost in Stop mode.

Bit 24 **SRAM2_16SO**: AHB SRAM2 16-Kbyte shut-off in Stop mode.

0: AHB RAM2 16-Kbyte content is kept in Stop mode.

1: AHB RAM2 16-Kbyte content is lost in Stop mode.

Bit 23 **SRAM3SO**: AHB SRAM3 shut-off in Stop mode.

0: AHB RAM3 content is kept in Stop mode.

1: AHB RAM3 content is lost in Stop mode.

Bits 22:17 Reserved, must be kept at reset value.

Bit 16 **ETHERNETSO**: ETHERNET RAM shut-off in Stop mode.

0: ETHERNET RAM content is kept in Stop mode.

1: ETHERNET RAM content is lost in Stop mode.

Bits 15:14 Reserved, must be kept at reset value.

Bit 13 **AVD_READY**: analog voltage ready

This bit is used only when the analog switch boost must be enabled (see BOOSTE bit).
It must be set by software when the expected V_{DDA} analog supply level is available.
The correct analog supply level is indicated by the AVDO bit (PWR_VMSR register) after setting the AVDEN bit (PWR_VMCR register) and selecting the supply level to be monitored (ALS bits).
0: peripheral analog voltage V_{DDA} not ready (default)
1: peripheral analog voltage V_{DDA} ready.

Bit 12 **BOOSTE**: analog switch V_{BOOST} control

This bit enables the booster to guarantee the analog switch AC performance when the V_{DD} supply voltage is below 2.7 V (reduction of the total harmonic distortion to have the same switch performance over the full supply voltage range) The V_{DD} supply voltage can be monitored through the PVD and the PLS bits.
0: booster disabled (default)
1: booster enabled if analog voltage ready (AVD_READY = 1)

Bits 11:10 Reserved, must be kept at reset value.

Bit 9 **FLPS**: flash memory low-power mode in Stop mode

This bit is used to obtain the best trade-off between low-power consumption and restart time when exiting from Stop mode.
When it is set, the flash memory enters low-power mode when the system is in Stop mode.
0: flash memory remains in normal mode when the system enters Stop mode (quick restart time).
1: flash memory enters low-power mode when the system enters Stop mode (low-power consumption).

Note: When system enters Stop mode with SVOS5 enabled, flash memory is automatically forced in low-power mode.

Bit 8 Reserved, must be kept at reset value.

Bit 7 **CSSF**: clear Standby and Stop flags (always read as 0)

This bit is cleared to 0 by hardware.
0: no effect
1: STOPF and SBF flags cleared

Bits 6:4 Reserved, must be kept at reset value.

Bits 3:2 **SVOS[1:0]**: system Stop mode voltage scaling selection

These bits control the V_{CORE} voltage level in system Stop mode, to obtain the best trade-off between power consumption and performance.
00: reserved
01: SVOS5 scale 5
10: SVOS4 scale 4
11: SVOS3 scale 3 (default)

Bit 1 Reserved, must be kept at reset value.

Bit 0 **LPMS**: low-power mode selection

This bit defines the DeepSleep mode.
0: keeps Stop mode when entering DeepSleep.
1: allows Standby mode when entering DeepSleep.

10.11.2 PWR power mode control register [alternate] (PWR_PMCR)

This register is protected against nonsecure access when LPMSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when LPMSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when LPMSEC = 0 and NSPRIV = 1.

Address offset: 0x000

Reset value: 0x0000 000C

STM32H523/533xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SRAM1 SO	SRAM2 _48SO	SRAM2 _16HSO	SRAM2 _16LSO	SRAM3 SO	Res.	Res.	Res.	Res.	Res.	Res.	Res.
				rw	rw	rw	rw	rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	AVD READY	BOOST E	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	SVOS[1:0]		Res.	LPMS
		rw	rw			rw		rw				rw	rw		rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **SRAM1SO**: AHB SRAM1 shut-off in Stop mode

- 0: AHB RAM1 content is kept in Stop mode.
- 1: AHB RAM1 content is lost in Stop mode.

Bit 26 **SRAM2_48SO**: AHB SRAM2 48-Kbyte shut-off in Stop mode.

- 0: AHB RAM2 48-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 48-Kbyte content is lost in Stop mode.

Bit 25 **SRAM2_16HSO**: AHB SRAM2 high 16-Kbyte shut-off in Stop mode.

- 0: AHB RAM2 high 16-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 high 16-Kbyte content is lost in Stop mode.

Bit 24 **SRAM2_16LSO**: AHB SRAM2 low 16-Kbyte shut-off in Stop mode.

- 0: AHB RAM2 low 16-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 low 16-Kbyte content is lost in Stop mode.

Bit 23 **SRAM3SO**: AHB SRAM3 shut-off in Stop mode.

- 0: AHB RAM3 content is kept in Stop mode.
- 1: AHB RAM3 content is lost in Stop mode.

Bits 22:14 Reserved, must be kept at reset value.

Bit 13 **AVD_READY**: analog voltage ready

This bit is used only when the analog switch boost must be enabled (see BOOSTE bit).

It must be set by software when the expected V_{DDA} analog supply level is available.

The correct analog supply level is indicated by the AVDO bit (PWR_VMSR register) after setting the AVDEN bit (PWR_VMCR register) and selecting the supply level to be monitored (ALS bits).

- 0: peripheral analog voltage V_{DDA} not ready (default)
- 1: peripheral analog voltage V_{DDA} ready.

Bit 12 **BOOSTE**: analog switch V_{BOOST} control

This bit enables the booster to guarantee the analog switch AC performance when the V_{DD} supply voltage is below 2.7 V (reduction of the total harmonic distortion to have the same switch performance over the full supply voltage range) The V_{DD} supply voltage can be monitored through the PVD and the PLS bits.

0: booster disabled (default)

1: booster enabled if analog voltage ready ($\text{AVD_READY} = 1$)

Bits 11:10 Reserved, must be kept at reset value.

Bit 9 **FLPS**: flash memory low-power mode in Stop mode

This bit is used to obtain the best trade-off between low-power consumption and restart time when exiting from Stop mode.

When it is set, the flash memory enters low-power mode when the system is in Stop mode.

0: flash memory remains in normal mode when the system enters Stop mode (quick restart time).

1: flash memory enters low-power mode when the system enters Stop mode (low-power consumption).

Note: When system enters Stop mode with SVOS5 enabled, flash memory is automatically forced in low-power mode.

Bit 8 Reserved, must be kept at reset value.

Bit 7 **CSSF**: clear Standby and Stop flags (always read as 0)

This bit is cleared to 0 by hardware.

0: no effect

1: STOPF and SBF flags cleared

Bits 6:4 Reserved, must be kept at reset value.

Bits 3:2 **SVOS[1:0]**: system Stop mode voltage scaling selection

These bits control the V_{CORE} voltage level in system Stop mode, to obtain the best trade-off between power consumption and performance.

00: reserved

01: SVOS5 scale 5

10: SVOS4 scale 4

11: SVOS3 scale 3 (default)

Bit 1 Reserved, must be kept at reset value.

Bit 0 **LPMS**: low-power mode selection

This bit defines the Deepsleep mode.

0: keeps Stop mode when entering DeepSleep.

1: allows Standby mode when entering DeepSleep.

10.11.3 PWR power mode control register [alternate] (PWR_PMCR)

This register is protected against nonsecure access when LPMSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when LPMSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when LPMSEC = 0 and NSPRIV = 1.

Address offset: 0x000

Reset value: 0x0000 000C

STM32H543/553xx devices only

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	SRAM1 SO	SRAM2 _48SO	SRAM2 _16HSO	SRAM2 _16LSO	SRAM3 SO	Res.	Res.	Res.	Res.	Res.	Res.	ETHER NETSO
				rw	rw	rw	rw	rw							rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	AVD READY	BOOST E	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	SVOS[1:0]		Res.	LPMS
		rw	rw			rw		rw				rw	rw		rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **SRAM1SO**: AHB SRAM1 shut-off in Stop mode

- 0: AHB RAM1 content is kept in Stop mode.
- 1: AHB RAM1 content is lost in Stop mode.

Bit 26 **SRAM2_48SO**: AHB SRAM2 48-Kbyte shut-off in Stop mode

- 0: AHB RAM2 48-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 48-Kbyte content is lost in Stop mode.

Bit 25 **SRAM2_16HSO**: AHB SRAM2 high 16-Kbyte shut-off in Stop mode

- 0: AHB RAM2 high 16-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 high 16-Kbyte content is lost in Stop mode.

Bit 24 **SRAM2_16LSO**: AHB SRAM2 low 16-Kbyte shut-off in Stop mode.

- 0: AHB RAM2 low 16-Kbyte content is kept in Stop mode.
- 1: AHB RAM2 low 16-Kbyte content is lost in Stop mode.

Bit 23 **SRAM3SO**: AHB SRAM3 shut-off in Stop mode.

- 0: AHB RAM3 content is kept in Stop mode.
- 1: AHB RAM3 content is lost in Stop mode.

Bits 22:17 Reserved, must be kept at reset value.

Bit 16 **ETHERNETSO**: ETHERNET RAM shut-off in Stop mode

- 0: ETHERNET RAM content is kept in Stop mode.
- 1: ETHERNET RAM content is lost in Stop mode.

Bits 15:14 Reserved, must be kept at reset value.

Bit 13 **AVD_READY**: analog voltage ready

This bit is used only when the analog switch boost must be enabled (see BOOSTE bit).

It must be set by software when the expected V_{DDA} analog supply level is available.

The correct analog supply level is indicated by the AVDO bit (PWR_VMSR register) after setting the AVDEN bit (PWR_VMCR register) and selecting the supply level to be monitored (ALS bits).

0: peripheral analog voltage V_{DDA} not ready (default)

1: peripheral analog voltage V_{DDA} ready.

Bit 12 **BOOSTE**: analog switch V_{BOOST} control

This bit enables the booster to guarantee the analog switch AC performance when the V_{DD} supply voltage is below 2.7 V (reduction of the total harmonic distortion to have the same switch performance over the full supply voltage range) The V_{DD} supply voltage can be monitored through the PVD and the PLS bits.

0: booster disabled (default)

1: booster enabled if analog voltage ready (AVD_READY = 1)

Bits 11:10 Reserved, must be kept at reset value.

Bit 9 **FLPS**: flash memory low-power mode in Stop mode

This bit is used to obtain the best trade-off between low-power consumption and restart time when exiting from Stop mode.

When it is set, the flash memory enters low-power mode when the system is in Stop mode.

0: flash memory remains in normal mode when the system enters Stop mode (quick restart time).

1: flash memory enters low-power mode when the system enters Stop mode (low-power consumption).

Note: When system enters Stop mode with SVOS5 enabled, flash memory is automatically forced in low-power mode.

Bit 8 Reserved, must be kept at reset value.

Bit 7 **CSSF**: clear Standby and Stop flags (always read as 0)

This bit is cleared to 0 by hardware.

0: no effect

1: STOPF and SBF flags cleared

Bits 6:4 Reserved, must be kept at reset value.

Bits 3:2 **SVOS[1:0]**: system Stop mode voltage scaling selection

These bits control the V_{CORE} voltage level in system Stop mode, to obtain the best trade-off between power consumption and performance.

00: reserved

01: SVOS5 scale 5

10: SVOS4 scale 4

11: SVOS3 scale 3 (default)

Bit 1 Reserved, must be kept at reset value.

Bit 0 **LPMS**: low-power mode selection

This bit defines the DeepSleep mode.

0: keeps Stop mode when entering DeepSleep.

1: allows Standby mode when entering DeepSleep.

10.11.4 PWR status register (PWR_PMSR)

Address offset: 0x004

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SBF	STOPF	Res.	Res.	Res.	Res.	Res.
									r	r					

Bits 31:7 Reserved, must be kept at reset value.

Bit 6 **SBF**: System standby flag

This bit is set by hardware and cleared only by a POR or by setting the CSSF bit.

0: system has not been in Standby mode.

1: system has been in Standby mode.

Bit 5 **STOPF**: Stop flag

This bit is set by hardware and cleared only by any reset or by setting the CSSF bit.

0: system has not been in Stop mode.

1: system has been in Stop mode.

Bits 4:0 Reserved, must be kept at reset value.

10.11.5 PWR voltage scaling control register (PWR_VOSCR)

Some register fields are protected against nonsecure access, depending on RCC_SECCFGR register. These fields can be protected against unprivileged access, depending on PWR_PRIVCFGR register configuration.

Address offset: 0x0010

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VOS[1:0]		Res.	Res.	Res.	Res.
										rw	rw				

Bits 31:6 Reserved, must be kept at reset value.

Bits 5:4 **VOS[1:0]**: voltage scaling selection according to performance

These bits control the V_{CORE} voltage level and allow to obtain the best trade-off between power consumption and performance:

- In bypass mode, these bits must also be set according to the external provided core voltage level and related performance.
- When increasing the performance, the voltage scaling must be changed before increasing the system frequency.
- When decreasing performance, the system frequency must first be decreased before changing the voltage scaling.

00: scale 3 (default)

01: scale 2

10: scale 1

11: scale 0

Bits 3:0 Reserved, must be kept at reset value.

10.11.6 PWR voltage scaling status register (PWR_VOSSR)

Address offset: 0x0014

Reset value: 0x0000 2008

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ACTVOS[1:0]		ACTVOS RDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VOS RDY	Res.	Res.	Res.
r	r	r										r			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:14 **ACTVOS[1:0]**: voltage output scaling currently applied to V_{CORE}

This bit provides the last VOS value.

00: VOS3 (lowest power)

01: VOS2

10: VOS1

11: VOS0 (highest frequency)

Bit 13 **ACTVOSRDY**: Voltage level ready for currently used VOS

0: V_{CORE} is above or below the current voltage scaling provided by ACTVOS[1:0]

1: V_{CORE} is equal to the current voltage scaling provided by ACTVOS[1:0]

Bits 12:4 Reserved, must be kept at reset value.

Bit 3 **VOSRDY**: Ready bit for V_{CORE} voltage scaling output selection.

0: Not ready, voltage level below VOS selected level

1: Ready, voltage level at or above VOS selected level

Note: The VOSRDY flag must be used only when switching from low to high-voltage scale (like switching from VOS3 to VOS0).

Bits 2:0 Reserved, must be kept at reset value.

10.11.7 PWR Backup domain control register (PWR_BDCR)

This register is protected against nonsecure access when VBSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when VBSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when VBSEC = 0 and NSPRIV = 1.

This register is not reset by wake-up from Standby mode, RESET signal and VDD POR. It is reset only by V_{SW} POR and VSWRST reset. This register must not be accessed when VSWRST bit in RCC_BDCR register resets the V_{SW} domain.

MONEN and BREN bits must not be accessed when VSWRST bit in RCC_BDCR register resets the V_{SW} domain. After reset, MONEN and BREN of this register are write-protected. Prior to modifying their content, the DBP bit in PWR_DBPCR register must be set to disable the write protection.

Address offset: 0x20

Power-on reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	VBRS	VBE	Res.	Res.	Res.	Res.	Res.	Res.	MONEN	BREN
						rw	rw							rw	rw

Bits 31:10 Reserved, must be kept at reset value.

- Bit 9 **VBRS**: V_{BAT} charging resistor selection
- 0: Charge V_{BAT} through a 5 kΩ resistor.
 - 1: Charge V_{BAT} through a 1.5 kΩ resistor.

- Bit 8 **VBE**: V_{BAT} charging enable
- 0: V_{BAT} battery charging disabled.
 - 1: V_{BAT} battery charging enabled.

Note: Reset only by POR.

Bits 7:2 Reserved, must be kept at reset value.

- Bit 1 **MONEN**: Backup domain voltage and temperature monitoring enable
- 0: Backup domain voltage and temperature monitoring disabled
 - 1: Backup domain voltage and temperature monitoring enabled

- Bit 0 **BREN**: Backup RAM retention in Standby and V_{BAT} modes

When this bit set, the backup regulator (used to maintain the backup RAM content in Standby and V_{BAT} modes) is enabled.

If BREN is cleared, the backup regulator is switched off. The backup RAM can still be used in Run and Stop modes. However its content is lost in Standby and V_{BAT} modes.

If BREN is set, the application must wait till the backup regulator ready flag (BRRDY) is set to indicate that the data written into the SRAM is maintained in Standby and V_{BAT} modes.

- 0: Backup RAM content lost in Standby and V_{BAT} modes.
- 1: Backup RAM content preserved in Standby and V_{BAT} modes

10.11.8 PWR Backup domain control register (PWR_DBPCR)

This register is protected against nonsecure access when VBSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when VBSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when VBSEC = 0 and NSPRIV = 1.

Address offset: 0x24

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	DBP
															rw

Bits 31:1 Reserved, must be kept at reset value.

Bit 0 **DBP**: Disable Backup domain write protection

In reset state, all registers and SRAM in Backup domain are protected against parasitic write access. This bit must be set to enable write access to these registers.

0: Write access to Backup domain disabled

1: Write access to Backup domain enabled

10.11.9 PWR Backup domain status register (PWR_BDSR)

This register is not reset by wake-up from Standby mode, RESET signal and VDD POR. It is reset only by V_{SW} POR and VSWRST reset.

Address offset: 0x028

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TEMPH	TEMPL	VBATH	VBATL	Res.	Res.	Res.	BRRDY
								r	r	r	r				r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **TEMPH**: temperature level monitoring versus high threshold

0: temperature below high threshold level

1: temperature equal or above high threshold level

Bit 22 **TEMPL**: temperature level monitoring versus low threshold

0: temperature above low threshold level

1: temperature equal or below low threshold level

Bit 21 **VBATH**: V_{BAT} level monitoring versus high threshold

0: V_{BAT} level below high threshold level

1: V_{BAT} level equal or above high threshold level

Bit 20 **VBATL**: V_{BAT} level monitoring versus low threshold

0: V_{BAT} level above low threshold level

1: V_{BAT} level equal or below low threshold level

Bits 19:17 Reserved, must be kept at reset value.

Bit 16 **BRRDY**: backup regulator ready

This bit is set by hardware to indicate that the backup regulator is ready.

0: backup regulator not ready

1: backup regulator ready

Bits 15:0 Reserved, must be kept at reset value.

10.11.10 PWR USB Type-C power delivery register (PWR_UCPDR)

This register is protected against nonsecure access when UCPD1SEC = 1 in the TZSC_SECCFGR register, and against unprivileged access when UCPD1SEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when UCPD1SEC = 0 and NSPRIV = 1.

Address offset: 0x02C

Reset value: 0x0000 0000

Not affected by exit Standby mode.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD_STBY	UCPD_DBDIS
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **UCPD_STBY**: USB Type-c and Power delivery Standby mode

When set, this bit is used to memorize the UCPD configuration in Standby mode. This bit must be written to 1 just before entering Standby mode when using UCPD, and it must be written to 0 after exiting the standby mode and before writing any UCPD register.

Bit 0 **UCPD_DBDIS**: USB Type-C and power delivery dead battery disable

After exiting reset, the USB Type-C “dead battery” behavior is enabled, which may have a pull-down effect on CC1 and CC2 pins. It is recommended to disable it in all case, either to stop this pull-down or to hand over control to the UCPD (which should therefore be initialized before doing the disable).

0: Enable USB Type-C dead battery pull-down behavior on UCPDx_CC1 and UCPDx_CC2 pins.

1: Disable USB Type-C dead battery pull-down behavior on UCPDx_CC1 and UCPDx_CC2 pins.

10.11.11 PWR supply configuration control register (PWR_SCCR)

This register is protected against nonsecure access when SCMSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when SCMSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when SCMSEC = 0 and NSPRIV = 1.

Address offset: 0x030

Reset value: 0x0000 0X00 (STM32H562/563/573xx devices)

Reset value: 0x0000 0100 (STM32H523/533/543/553xx devices)

The reset value of this register changes according to the package. Bits 9 and 8 indicate the power configuration. Their values are exclusive.

Reset by POR only, not reset by wake-up from Standby mode and RESET pad. The BYPASS bit of this register is written once after POR. Written-once mechanism locks the register and any further write access is ignored. The system must be power cycled before writing a new value.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	SMPS EN	LDO EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS
						r	r								rwo

Bits 31:10 Reserved, must be kept at reset value.

Bit 9 **SMPS**EN: SMPS enable

The value is set by hardware when the package uses the SMPS regulator.

Note: Check the actual availability of this bit on the datasheet. If not available, consider it as reserved, and keep it at reset value.

Bit 8 **LDO**EN: LDO enable

The value is set by hardware when the package uses the LDO regulator.

Bits 7:1 Reserved, must be kept at reset value.

Bit 0 **BYPASS**: power management unit bypass

0: Power management unit normal operation. Use the internal regulator.

1: Power management unit bypassed. Use the external power (voltage monitoring still active)

10.11.12 PWR voltage monitor control register (PWR_VMCR)

This register is protected against nonsecure access when SCMSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when SCMSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when SCMSEC = 0 and NSPRIV = 1.

The PVDE and PLS bits are protected by lock mechanism. The lock control is in the SBS module controlled by PVDL bit in SBS_CFGR2 register. By default, the PVDE and PLS are unlocked.

Address offset: 0x034

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	ALS[1:0]		AVDEN	Res.	Res.	Res.	Res.	PLS[2:0]		PVDE	
					rw	rw	rw					rw	rw	rw	rw

Bits 31:11 Reserved, must be kept at reset value.

Bits 10:9 **ALS[1:0]**: analog voltage detector (AVD) level selection

These bits select the analog voltage detector (AVD) thresholds.

00: AVD level0 ($V_{AVD0} \sim 1.7$ V)

01: AVD level1 ($V_{AVD1} \sim 2.1$ V)

10: AVD level2 ($V_{AVD2} \sim 2.5$ V)

11: AVD level3 ($V_{AVD3} \sim 2.8$ V)

Bit 8 **AVDEN**: peripheral voltage monitor on V_{DDA} enable

0: peripheral voltage monitor on V_{DDA} disabled

1: peripheral voltage monitor on V_{DDA} enabled

Bits 7:4 Reserved, must be kept at reset value.

Bits 3:1 **PLS[2:0]**: programmable voltage detector (PVD) level selection

These bits select the programmable voltage detector (PVD) thresholds.

000: PVD level0 ($VPVD0 \sim 1.95$ V)

001: PVD level1 ($VPVD1 \sim 2.10$ V)

010: PVD level2 ($VPVD2 \sim 2.25$ V)

011: PVD level3 ($VPVD3 \sim 2.40$ V)

100: PVD level4 ($VPVD4 \sim 2.55$ V)

101: PVD level5 ($VPVD5 \sim 2.70$ V)

110: PVD level6 ($VPVD6 \sim 2.85$ V)

111: PVD_IN pin

Bit 0 **PVDE**: PVD enable

0: PVD disabled

1: PVD enabled

10.11.13 PWR USB supply control register (PWR_USBSCR)

This register is protected against nonsecure access when $VUSBSEC = 1$ in the PWR_SECCFGR register, and against unprivileged access when $VUSBSEC = 1$ and $SPRIV = 1$ in the PWR_PRIVCFGR register, or when $VUSBSEC = 0$ and $NSPRIV = 1$.

Address offset: 0x038

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	USB33 SV	USB33 DEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
						rw	rw								
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:26 Reserved, must be kept at reset value.

Bit 25 **USB33SV**: independent USB supply valid

This bit is used to validate the V_{DDUSB} supply for electrical and logical isolation purpose.

Setting this bit is mandatory to use the USBFS peripheral. If V_{DDUSB} is not always present in the application, the V_{DDUSB} voltage monitor can be used to determine whether this supply is ready or not.

0: V_{DDUSB} is not present. Logical and electrical isolation is applied to ignore this supply.

1: V_{DDUSB} is valid.

Bit 24 **USB33DEN**: V_{DDUSB} voltage level detector enable

0: V_{DDUSB} voltage level detector disabled

1: V_{DDUSB} voltage level detector enabled

Bits 23:0 Reserved, must be kept at reset value.

10.11.14 PWR voltage monitor status register (PWR_VMSR)

Address offset: 0x03C

Reset value: 0x00X0 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USB33 RDY	Res.	PVDO	Res.	VDDIO2 RDY	AVDO	Res.	Res.	Res.
							r		r		r	r			
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **USB33RDY**: V_{DDUSB} ready

0: V_{DDUSB} is below the threshold of the V_{DDUSB} voltage monitor.

1: V_{DDUSB} is equal or above the threshold of the V_{DDUSB} voltage monitor.

Bit 23 Reserved, must be kept at reset value.

Bit 22 **PVDO**: programmable voltage detect output

This bit is set and cleared by hardware. It is valid only if the PVD has been enabled by the PVDE bit.

0: V_{DD} is equal or higher than the PVD threshold selected through the PLS[2:0] bits.

1: V_{DD} is lower than the PVD threshold selected through the PLS[2:0] bits.

Note: Since the PVD is disabled in Standby mode, this bit is equal to 0 after Standby or reset until the PVDE bit is set.

Bit 21 Reserved, must be kept at reset value.

Bit 20 **VDDIO2RDY**: voltage detector output on V_{DDIO2}

This bit is set and cleared by hardware.

0: V_{DDIO2} is below 1.2 V.

1: V_{DDIO2} is above or equal to 1.2 V.

Bit 19 **AVDO**: analog voltage detector output on V_{DDA}

This bit is set and cleared by hardware. It is valid only if AVD on VDDA is enabled by the AVDEN bit.

0: V_{DDA} is equal or higher than the AVD threshold selected with the ALS[2:0] bits.

1: V_{DDA} is lower than the AVD threshold selected with the ALS[2:0] bits.

Note: Since the AVD is disabled in Standby mode, this bit is equal to 0 after standby or reset until the AVDEN bit is set.

Bits 18:0 Reserved, must be kept at reset value.

10.11.15 PWR wake-up status clear register (PWR_WUSCR)

Each register bit CWUF $_x$ ($x = 1$ to 8) is protected against nonsecure access when WUPxSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when WUPxSEC = 1 in PWR_SECCFGR and SPRIV = 1 in PWR_PRIVCFGR, or when WUPxSEC = 0 and NSPRIV = 1.

Address offset: 0x040

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CWUF 8	CWUF 7	CWUF 6	CWUF 5	CWUF 4	CWUF 3	CWUF 2	CWUF 1
								w	w	w	w	w	w	w	w

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **CWUF $_x$** : clear wake-up pin flag for WUF $_x$ ($x = 8$ to 1)

These bits are always read as 0.

0: no effect

1: writing 1 clears the WUF $_x$ wake-up pin flag (bit is cleared to 0 by hardware).

10.11.16 PWR wake-up status register (PWR_WUSR)

Address offset: 0x044

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WUF8	WUF7	WUF6	WUF5	WUF4	WUF3	WUF2	WUF1
								r	r	r	r	r	r	r	r

Bits 31:8 Reserved, must be kept at reset value.

Bits 7:0 **WUFx**: wake-up pin WUFx flag (x = 8 to 1)

This bit is set by hardware and cleared only by a RESET pin or by setting the CWUFx bit in PWR_WUSCR register.

0: no wake-up event occurred.

1: wake-up event received from WUFx pin.

10.11.17 PWR wake-up configuration register (PWR_WUCR)

Each WUPPUPDx (x = 1 to 8), WUPPx (x = 1 to 8), and WUPENx (x = 1 to 8) bit is protected against nonsecure access when WUPxSEC = 1 (x = 1 to 8) in the PWR_SECCFGR register. Each WUPENx bit is protected against unprivileged access when WUPxSEC = 1 in PWR_SECCFGR and SPRIV = 1 in PWR_PRIVCFGR, or when WUPxSEC = 0 and NSPRIV = 1.

Address offset: 0x048

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
WUPPUPD8[1:0]		WUPPUPD7[1:0]		WUPPUPD6[1:0]		WUPPUPD5[1:0]		WUPPUPD4[1:0]		WUPPUPD3[1:0]		WUPPUPD2[1:0]		WUPPUPD1[1:0]	
rw	rw	rw	rw	rw	rw	rw	w	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
WUP P8	WUP P7	WUP P6	WUP P5	WUP P4	WUP P3	WUP P2	WUP P1	WUP EN8	WUP EN7	WUP EN6	WUP EN5	WUP EN4	WUP EN3	WUPE N2	WUP EN1
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 **WUPPUPDx[1:0]**: wake-up pin pull configuration for WKUPx (x = 8 to 1)

These bits define the I/O pad pull configuration used when WUPENx = 1. The associated GPIO port pull configuration must be set to the same value or to 00. The wake-up pin pull configuration is kept in Standby mode.

00: no pull-up

01: pull-up

10: pull-down

11: reserved

Bits 15:8 **WUPPx**: wake-up pin polarity bit for WUPx (x = 8 to 1)

These bits define the polarity used for event detection on WUPx external wake-up pin.

0: detection on high level (rising edge)

1: detection on low level (falling edge)

Bits 7:0 **WUPENx**: enable wake-up pin WUPx (x = 8 to 1)

These bits are set and cleared by software.

0: an event on WUPx pin does not wake-up the system from Standby mode.

1: a rising or falling edge on WUPx pin wakes up the system from Standby mode.

Note: An additional wake-up event is detected if WUPx pin is enabled (by setting the WUPENx bit) when WUPx pin level is already high when WUPPx selects rising edge, or low when WUPPx selects falling edge.

10.11.18 PWR I/O retention register (PWR_IORETR)

This register is protected against nonsecure access when RETSEC = 1 in the PWR_SECCFGR register, and against unprivileged access when RETBSEC = 1 and SPRIV = 1 in the PWR_PRIVCFGR register, or when RETSEC = 0 and NSPRIV = 1.

Address offset: 0x050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JTAGIO RETEN
															rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IORET EN
															rw

Bits 31:17 Reserved, must be kept at reset value.

Bit 16 **JTAGIORETEN**: IO retention enable for JTAG IOs

0: IO Retention mode is disabled.

1: IO Retention mode is enabling for PA13, PA14, PA15, and PB4.

when entering into standby mode, the output is sampled, and apply to the output IO during the standby power mode

Bits 15:1 Reserved, must be kept at reset value.

Bit 0 **IORETEN**: IO retention enable

0: IO Retention mode is disabled.

1: IO Retention mode is enabling for all IO except the IO support the standby functionality and PA13, PA14, PA15, and PB4.

When entering into standby mode, the output is sampled, and apply to the output IO during the standby power mode.

Note: The IO state is not retained if the DBG_STANDBY bit is set in DBGMCU_CR register.

10.11.19 PWR security configuration register (PWR_SECCFGR)

This register can be written only when the access is secure. It can be read by secure or nonsecure access. The register is write-protected against unprivileged write access when SPRIV = 1 in the PWR_PRIVCFGR.

Address offset: 0x100

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
VUSB SEC	VB SEC	SCM SEC	LPM SEC	RET SEC	Res.	Res.	Res.	WUP8 SEC	WUP7 SEC	WUP6 SEC	WUP5 SEC	WUP4 SEC	WUP3 SEC	WUP2 SEC	WUP1 SEC
rw	rw	rw	rw	rw				rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:16 Reserved, must be kept at reset value.

Bit 15 **VUSBSEC**: voltage USB secure protection

0: PWR_USBSCR can be read and written with secure or nonsecure access.

1: PWR_USBSCR can be read and written only with secure access..

Bit 14 **VBSEC**: Backup domain secure protection

0: PWR_BDCR, PWR_DBPCR can be read and written with secure or nonsecure access.

1: PWR_BDCR, PWR_DBPCR can be read and written only with secure access.

Bit 13 **SCMSEC**: supply configuration and monitoring secure protection.

0: PWR_SCCR and PWR_VMCR can be read and written with secure or nonsecure access.

1: PWR_SCCR and PWR_VMCR can be read and written only with secure access.

Bit 12 **LPMSEC**: low-power modes secure protection

0: PWR_PMCR can be read and written with secure or nonsecure access.

1: PWR_PMCR can be read and written only with secure access.

Bit 11 **RETSEC**: retention secure protection

0: PWR_IORETR can be read and written with secure or nonsecure access.

1: PWR_IORETR can be read and written only with secure access.

Bits 10:8 Reserved, must be kept at reset value.

Bits 7:0 **WUPxSEC**: WUPx secure protection (x = 8 to 1)

0: The bits related to the WKUPx wake-up pin in PWR_WUSCR and PWR_WUCR can be read and written with secure or nonsecure access.

1: The bits related to the WKUPx wake-up pin in PWR_WUSCR and PWR_WUCR can be read and written only with secure access.

10.11.20 PWR privilege configuration register (PWR_PRIVCFGR)

This register can be written only when the access is privileged. It can be read by privileged or unprivileged access.

Address offset: 0x104

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV	SPRIV
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 **NSPRIV**: PWR nonsecure functions privilege configuration

Set and reset by software. This bit can be written only by privileged access, secure or nonsecure.

0: Read and write to PWR nonsecure functions can be done by privileged or unprivileged access.

1: Read and write to PWR nonsecure functions can be done by privileged access only.

Bit 0 **SPRIV**: PWR secure functions privilege configuration

Set and reset by software. This bit can be written only by a secure privileged access.

0: Read and write to PWR secure functions can be done by privileged or unprivileged access.

1: Read and write to PWR secure functions can be done by privileged access only.

10.11.21 PWR register map

Table 116. PWR register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	PWR_PMC ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	SRAM1SO	SRAM2_48SO	SRAM2_16SO	SRAM3SO	Res.	Res.	Res.	Res.	Res.	Res.	ETHERNETSO	Res.	Res.	Res.	AVD_READY	BOOSTE	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	svos[1:0]		Res.	LPMS
	Reset value						0	0	0	0							0			0	0	Res.		0		0				1	1	0		
0x00	PWR_PMC ⁽²⁾	Res.	Res.	Res.	Res.	SRAM1SO	SRAM2_48SO	SRAM2_16HSO	SRAM2_16LO	SRAM3SO	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	AVD_READY	BOOSTE	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	svos[1:0]		Res.	LPMS
	Reset value						0	0	0	0										0	0	Res.		0		0				1	1	0		
0x00	PWR_PMC ⁽³⁾	Res.	Res.	Res.	Res.	SRAM1SO	SRAM2_48SO	SRAM2_16HSO	SRAM2_16LO	SRAM3SO	Res.	Res.	Res.	Res.	Res.	Res.	ETHERNETSO	Res.	Res.	Res.	AVD_READY	BOOSTE	Res.	Res.	FLPS	Res.	CSSF	Res.	Res.	Res.	svos[1:0]		Res.	LPMS
	Reset value						0	0	0	0							0			0	0	Res.		0		0				1	1	0		
0x04	PWR_PMSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SBF	STOPF		Res.	Res.	Res.	Res.	
	Reset value																				0					0	STOPF		Res.	Res.	Res.	Res.		
0x008 to 0x00F	Reserved	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
0x10	PWR_VOSCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VOS [1:0]	VOS [1:0]		Res.	Res.	Res.	Res.	
	Reset value																										0	0	Res.	Res.	Res.	Res.	Res.	
0x14	PWR_VOSSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ACTVOS [1:0]	ACTVOSRDY		Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VOSSRDY		Res.	Res.	
	Reset value																	0	0	1									1					
0x018 to 0x01F	Reserved																																	
0x20	PWR_BDCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																							0	0									
0x24	PWR_DBPCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																							0	0								0	
0x28	PWR_BDSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TEMPH	TEMPL	VBATH	VBATL	Res.	Res.	Res.	Res.	BRDY	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value								0	0	0	0					0																	
0x2C	PWR_UCPDR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value																															0	UCPD_STBY	
																																UCPD_DBDIS		

Table 116. PWR register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x30	PWR_SCCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	SMPSEN ⁽⁴⁾	LDOEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	BYPASS
	Reset value																							X	X								0
0x34	PWR_VMCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ALS [1:0]	AVDEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLS [2:0]	PVDE
	Reset value																						0	0	0							0	0
0x38	PWR_USBSCR	Res.	Res.	Res.	Res.	Res.	Res.	USB33SV	USB33DEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value							0	0																								
0x3C	PWR_VMSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USB33RDY	Res.	PVDD	Res.	VDDIO2RDY	AVDD	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								0		0		X	0																			
0x40	PWR_WUSCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																									0	0	0	0	0	0	0	0
0x44	PWR_WUSR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																																
0x48	PWR_WUCR	WUPPUPD8 [1:0]	WUPPUPD7 [1:0]	WUPPUPD6 [1:0]	WUPPUPD5 [1:0]	WUPPUPD4 [1:0]	WUPPUPD3 [1:0]	WUPPUPD2 [1:0]	WUPPUPD1 [1:0]	WUPP8	WUPP7	WUPP6	WUPP5	WUPP4	WUPP3	WUPP2	WUPP1	WUPEN8	WUPEN7	WUPEN6	WUPEN5	WUPEN4	WUPEN3	WUPEN2	WUPEN1	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04C	Reserved																																
0x50	PWR_IORETR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	JTAGIORETR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	IORETR
	Reset value																0																0
0x054 to 0x0FF	Reserved																																
0x100	PWR_SECCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VUSBSEC	VBSEC	SCMSEC	LPMSEC	RETSEC	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																0	0	0	0	0							0	0	0	0	0	0
0x104	PWR_PRIVCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV
	Reset value																															0	0

1. STM32H562/572/573xx devices only.
2. STM32H523/533xx devices only.
3. STM32H543/553xx devices only.
4. This bit is reserved on STM32H523/533/543/553xx devices, keep it at reset value.

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

11 Reset and clock control (RCC)

11.1 RCC introduction

The reset and clock control (RCC) manages the different resets, and generates the clocks for the bus and peripherals.

11.2 RCC pins and internal signals

[Table 117](#) lists the RCC inputs and output signals connected to package pins or balls.

Table 117. RCC input/output signals connected to package pins or balls

Signal name	Signal type	Description
NRST	I/O	System reset, can be used to provide reset to external devices
OSC32_IN	I	32 kHz oscillator input
OSC32_OUT	O	32 kHz oscillator output
OSC_IN	I	System oscillator input
OSC_OUT	O	System oscillator output
MCO	O	Output clock for external devices
LSCO	O	Low-speed output clock for external devices
AUDIOCLK	I	External kernel clock input for SAI1, SAI2, I2S1, I2S2, and I2S3

11.3 RCC reset functional description

There are three types of reset:

- a system reset
- a power reset
- a Backup domain reset

11.3.1 Power reset

A power reset is generated when one of the following events occurs:

- a brownout reset (BOR)
- when exiting Standby mode

A brownout reset, including power-on or power-down reset (POR/PDR), sets all registers to their reset values except the ones in the Backup domain.

When exiting Standby mode, all registers in the core domain are set to their reset value. Registers outside the core domain (RTC, WKUP, IWDG, and GPIO pullup/pulldown configuration during Standby and Standby mode exit) are not impacted.

11.3.2 System reset

A system reset sets all registers to their reset values except the reset flags in [RCC reset status register \(RCC_RSR\)](#) and the registers in the Backup domain.

A system reset is generated when one of the following events occurs:

- a low level on the NRST pin (external reset)
- a window watchdog event (WWDG reset)
- an independent watchdog event (IWDG reset)
- a software reset (SW reset) (see [Software reset](#))
- a low-power mode security reset (see [Low-power mode security reset](#))
- a brownout reset

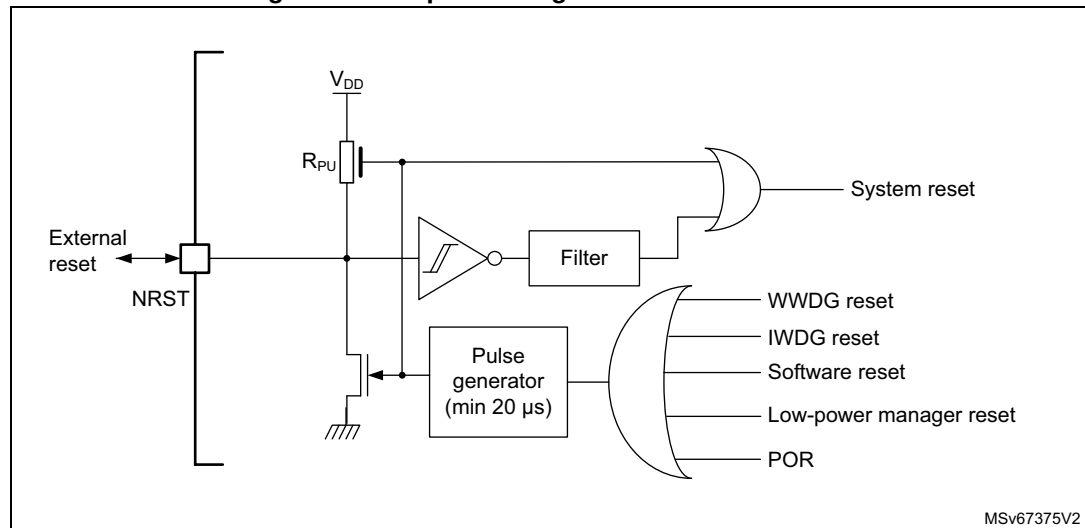
The reset source can be identified by checking the reset flags in [RCC reset status register \(RCC_RSR\)](#).

These sources act on the NRST pin and this pin is always kept low during the delay phase. The reset service routine vector is selected depending on product state, on Boot option bytes or on both.

The system reset signal provided to the device is output on the NRST pin. The pulse generator guarantees a minimum reset pulse duration of 20 μ s for each internal reset source. In case of an external reset, the reset pulse is generated while the NRST pin is asserted low.

In case on an internal reset, the internal pull-up R_{PU} is deactivated to save the power consumption through the pull-up resistor.

Figure 54. Simplified diagram of the reset circuit



Software reset

The SYSRESETREQ bit in Cortex-M33 application interrupt and reset control register must be set to force a software reset on the device.

Low-power mode security reset

To avoid that critical applications mistakenly enter a low-power mode, the following low-power mode security resets are available. If enabled in option bytes, the resets are generated in any of the following conditions:

- Entering Standby mode: this type of reset is enabled by resetting nRST_STDBY bit in user option bytes. In this case, whenever a Standby mode entry sequence is successfully executed, the device is reset instead of entering Standby mode.
- Entering Stop mode: this type of reset is enabled by resetting nRST_STOP bit in user option bytes. In this case, whenever a Stop mode entry sequence is successfully executed, the device is reset instead of entering Stop mode.

For further information on the user option bytes, refer to [Section 7.4.1: Option bytes description](#).

11.3.3 Backup domain reset

The Backup domain has two specific resets, generated after one of the following events:

- a software reset, triggered by setting the VSWRST bit in the [RCC Backup domain control register \(RCC_BDCR\)](#). Write access to this domain must be enabled before setting VSWRST bit to perform the reset.
- a V_{DD} or V_{BAT} power on, if both supplies have previously been powered off

A Backup domain reset affects the LSE oscillator, the RTC, the backup registers, the backup SRAM, and the RCC_BDCR register.

11.3.4 Reset source identification

The application can identify the reset source by checking the reset flags in the RCC_RSR register. The software can reset the flags by setting the RMVF bit.

[Table 118](#) shows how the status bits of the RCC_RSR register behave according to the situation that generated the reset. For example, when an IWDG timeout occurs, if the CPU is reading the RCC_RSR register during the boot phase, both PINRSTF and IWDGRSTF bits are set, indicating that the IWDG also generated a pin reset.

Table 118. Reset source identification (RCC_RSR)⁽¹⁾

Reset		LPWRRSTF	WWDGRSTF	IWDGRSTF	SFTRSTF	BORRSTF	PINRSTF
1	Power-on reset	0	0	0	0	1	1
2	Pin/pad reset	0	0	0	0	0	1
3	Brownout (low or high) reset	0	0	0	0	1	1
4	System reset generated by CPU	0	0	0	1	0	1
5	WWDG reset	0	1	0	0	0	1
6	IWDG reset	0	0	1	0	0	1
7	Illegal stop entry reset	1	0	0	0	0	1

1. Gray cells highlight the register bits that are set.

11.4 RCC clocks functional description

Four different clock sources can be used to drive the system clock (SYSCLK):

- HSI: high-speed internal up to 64 MHz RC oscillator clock
- CSI: low power internal RC oscillator clock
- HSE: high-speed external crystal or clock, from 4 to 50 MHz
- PLL1 clock

The HSI is used as system clock source after startup from reset, configured at 32 MHz.

The device has the following additional clock sources:

- LSI: 32 kHz low-speed internal RC that drives the independent watchdog and optionally the RTC used for auto-wake-up from Stop and Standby modes
- LSE: 32.768 kHz low-speed external crystal or clock that optionally drives the real-time clock (rtc_ck)
- HSI48: internal 48 MHz RC that potentially drives the USB FS and the RNG
- PLL2 and PLL3^(a) clocks

Each clock source can be switched on or off independently when it is not used, to optimize power consumption.

Several prescalers can be used to configure the AHB frequency, the APB1 and APB2 domains. The maximum frequency of the AHB and APB domains is 250 MHz.

The peripheral clocks are derived from their bus clock (HCLK, PCLK1, PCLK2 or PCLK3), except those that receive an independent kernel clock. This kernel clock can be selected by software between several sources thanks to RCC_CCIPRx registers (x = 1,2,3,4,5).

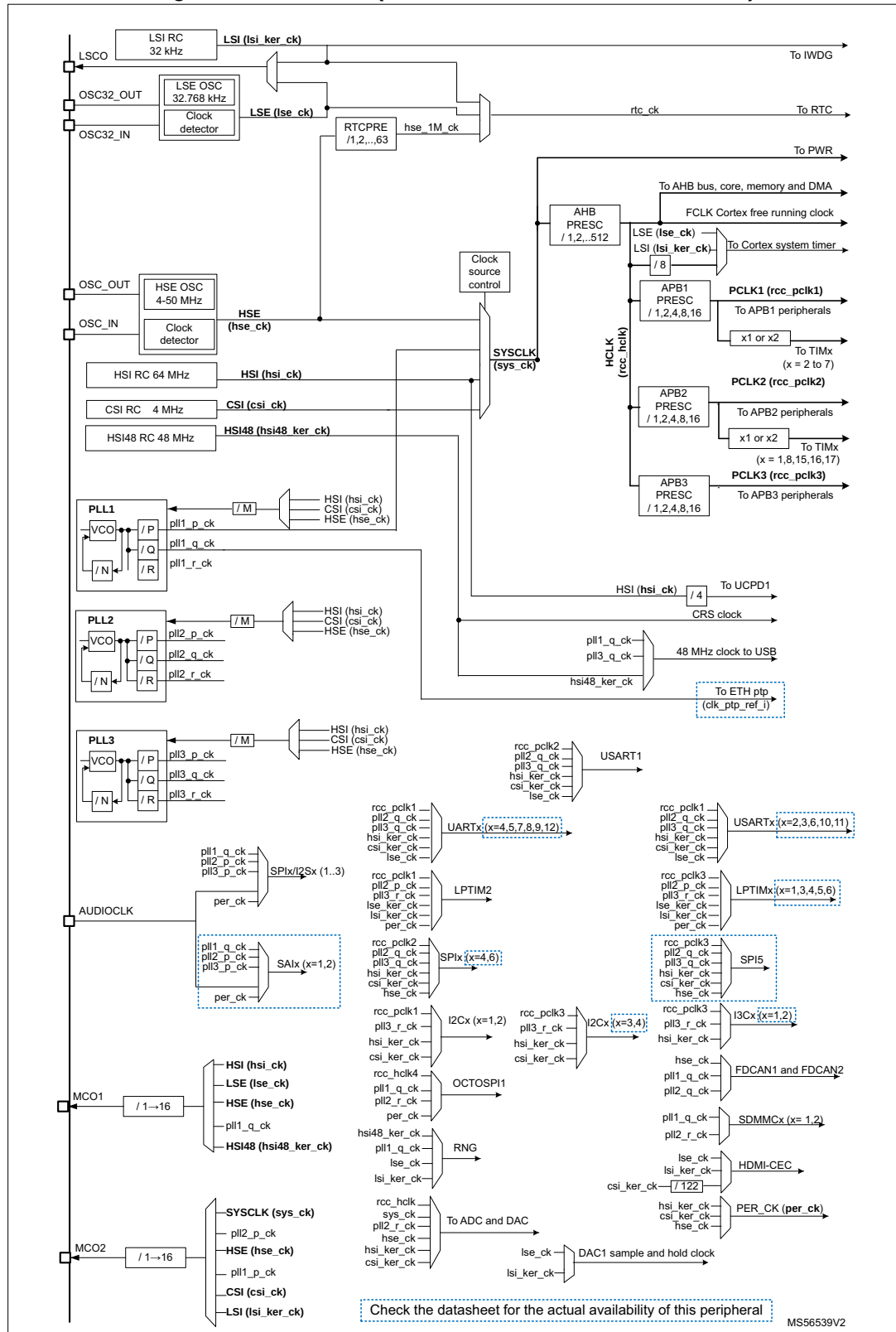
In addition, the RTC kernel clock is selected by software in RCC_BDCR. The IWDG clock is always the LSI 32 kHz clock.

The RCC feeds the Cortex system timer (SysTick) external clock with the AHB clock (HCLK) divided by eight, or LSE or LSI. The SysTick can work either with this clock or directly with the Cortex clock (HCLK), configurable in the SysTick control and status register.

FCLK acts as Cortex-M33 free-running clock.

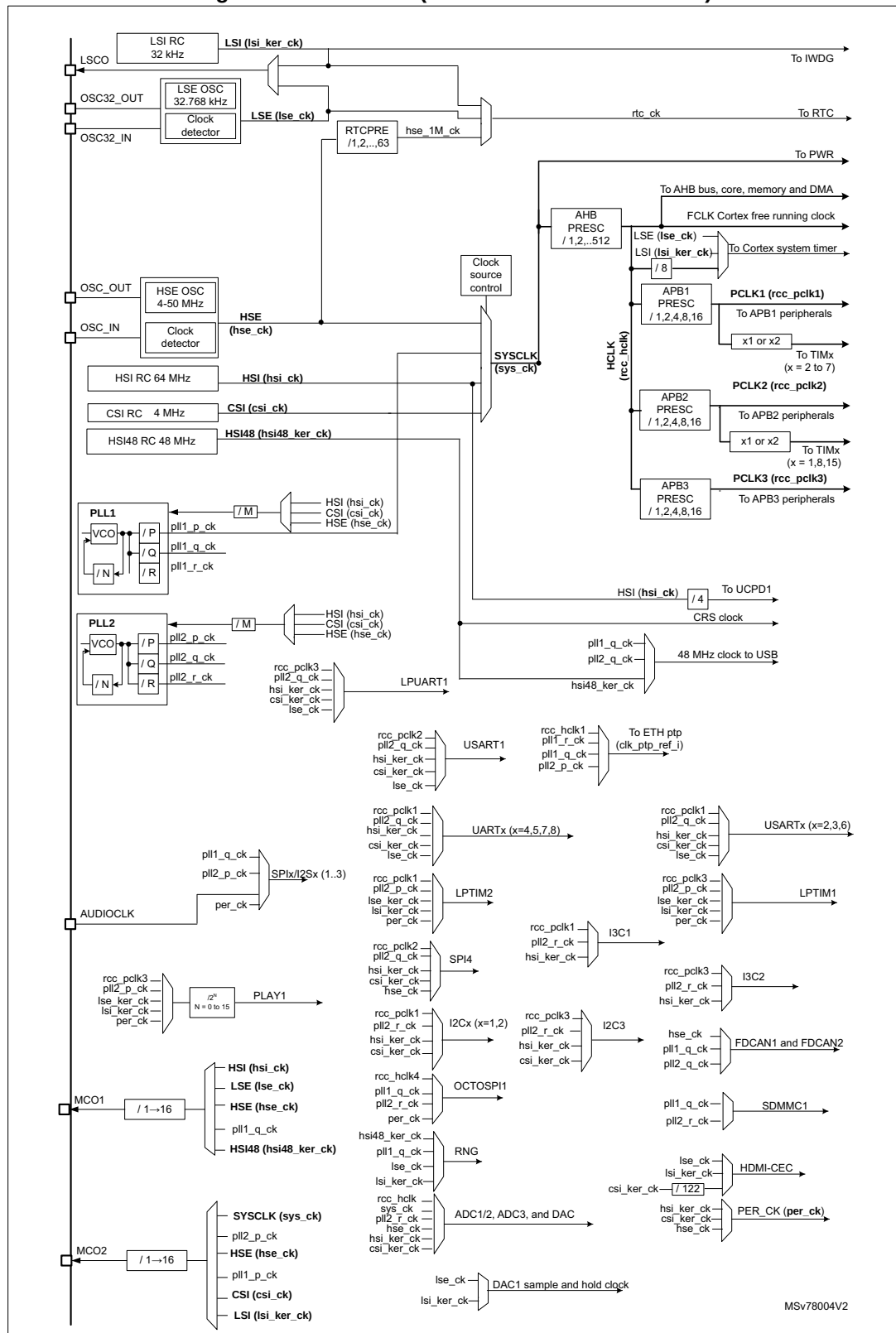
a. Not available on STM32H543/553xx devices.

Figure 55. Clock tree (STM32H523/533/56x/573xx devices)



1. For details on the internal/external clock source, refer to the electrical characteristics in the datasheet.

Figure 56. Clock tree (STM32H543/553xx devices)

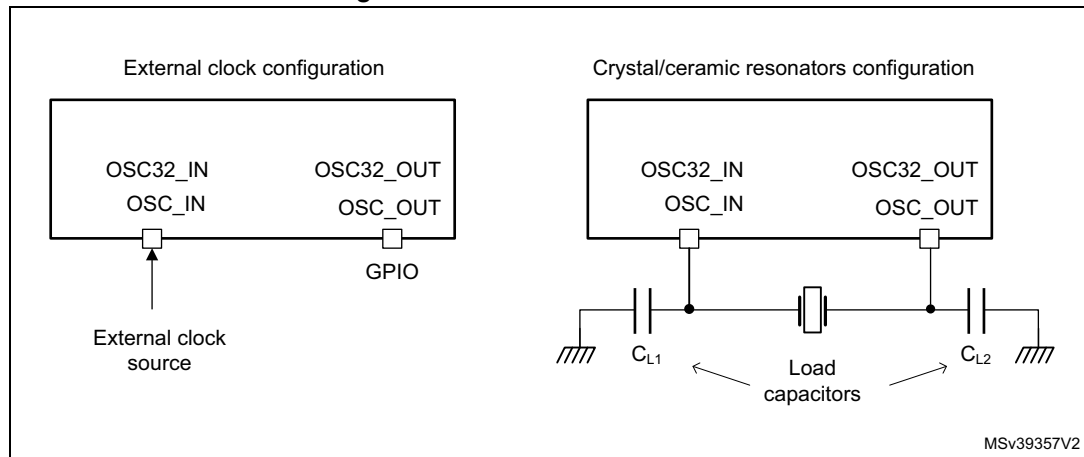


1. For details on the internal/external clock source, refer to the electrical characteristics in the datasheet.

11.4.1 HSE clock

The HSE block can generate a clock from an external crystal/ceramic resonator, or from an external clock source.

Figure 57. HSE/LSE clock sources



External clock source (HSE bypass)

In this mode, an external clock source must be provided to OSC_IN pin. The external clock can be low swing (analog) or digital. If this clock is directly used by a peripheral, the duty cycle requirement is defined by the peripheral and the application (refer to datasheet for more details).

In case of an analog clock (low swing) the HSEBYP and HSEON bits must be set to 1 in the [RCC clock control register \(RCC_CR\)](#).

In case of a digital clock, the HSEBYP and the HSEEXT bits must be set to 1 followed by setting the HSEON bit to 1 in the [RCC clock control register \(RCC_CR\)](#).

External crystal/ceramic resonator

The oscillator is enabled by setting the HSEBYP bit to 0 and HSEON bit to 1.

The HSE can be used when the product requires a very accurate high-speed clock.

The associated hardware configuration is shown in [Figure 57](#). The resonator and the load capacitors must be placed as close as possible to the oscillator pins to minimize output distortion and startup stabilization time. The loading capacitance values must be adjusted according to the selected crystal or ceramic resonator. Refer to the electrical characteristics section of the datasheet for more details.

The HSERDY flag of the [RCC clock control register \(RCC_CR\)](#) indicates whether the HSE oscillator is stable or not. At startup, the **hse_ck** clock is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

The HSE can be switched ON and OFF through the HSEON bit. Note that the HSE cannot be switched OFF if one of the following two conditions is met:

- the HSE is used directly (via software mux) as system clock
- the HSE is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSEON bit to 0.

The HSE is automatically disabled by hardware, when the system enters Stop or Standby mode.

In addition, the HSE clock can be driven to the MCO1 and MCO2 outputs and used as clock source for other application components.

11.4.2 HSI clock

The HSI block provides the default clock to the product.

The HSI is a high-speed internal RC oscillator that can be used directly as system clock, peripheral clock, or as PLL input. A predivider allows the application to select an HSI output frequency of 8, 16, 32 or 64 MHz. This predivider is controlled by the HSIDIV.

The HSI advantages are the following:

- low-cost clock source, as no external crystal is required
- faster startup time than HSE (a few microseconds)

The HSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The HSI can be switched ON and OFF using the HSION bit. Note that the HSI cannot be switched OFF if one of the two conditions is met:

- the HSI is used directly (via software mux) as system clock
- the HSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the HSION bit to 0. Note that the HSIDIV cannot be changed if the HSI is selected as reference clock for at least one enabled PLL (PLLxON bit set to 1). In that case the hardware does not update the HSIDIV with the new value. However it is possible to change the HSIDIV if the HSI is used directly as system clock.

The HSIRDY flag indicates if the HSI is stable or not. At startup, the HSI output clock is not released until this bit is set by hardware.

The HSI clock can also be used as a backup source (auxiliary clock) if the HSE fails (refer to [Section 11.4.10: Clock security system \(CSS\)](#)). The HSI can be disabled or not when the system enters Stop mode.

In addition, the HSI clock can be driven to the MCO1 output and used as clock source for other application components.

Care must be taken when the HSI is used as kernel clock for communication peripherals, the application must take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available
- the frequency accuracy.

Note: The HSI can remain enabled when the system is in Stop mode.

HSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations. That is why each device is factory calibrated by STMicroelectronics to achieve an accuracy of ACCHSI (refer to the product datasheet for more information).

After a power-on reset, the factory calibration value is loaded in the HSICAL[11:0] bits. If the application is subject to voltage or temperature variations, this may affect the RC oscillator frequency. The user application can trim the HSI frequency using the HSITRIM[6:0] bits.

Note: HSICAL[11:0] and HSITRIM[6:0] are located in the [RCC CSI calibration register \(RCC_CSICFGR\)](#).

11.4.3 CSI oscillator

The CSI is a low-power RC oscillator that can be used directly as system clock, peripheral clock, or PLL input.

The CSI advantages are the following:

- low-cost clock source since no external crystal is required
- faster startup time than HSE (a few microseconds)
- very low-power consumption,

The CSI provides a clock frequency of about 4 MHz, while the HSI is able to provide a clock up to 64 MHz.

CSI frequency, even with frequency calibration, is less accurate than an external crystal oscillator or ceramic resonator.

The CSI can be switched ON and OFF through the CSION bit. The CSIRDY flag indicates whether the CSI is stable or not. At startup, the CSI output clock is not released until this bit is set by hardware.

The CSI cannot be switched OFF if one of the two conditions is met:

- The CSI is used directly (via software mux) as system clock.
- The CSI is selected as reference clock for PLL1, with PLL1 enabled and selected to provide the system clock (via software mux).

In that case the hardware does not allow programming the CSION bit to 0.

The CSI can be disabled or not when the system enters Stop mode.

In addition, the CSI clock can be driven to the MCO2 output and used as clock source for other application components.

Even if the CSI settling time is faster than the HSI, care must be taken when the CSI is used as kernel clock for communication peripherals: the application must take into account the following parameters:

- the time interval between the moment where the peripheral generates a kernel clock request and the moment where the clock is really available,
- the frequency precision.

Note: CSION and CSIRDY bits are located in the [RCC clock control register \(RCC_CR\)](#).

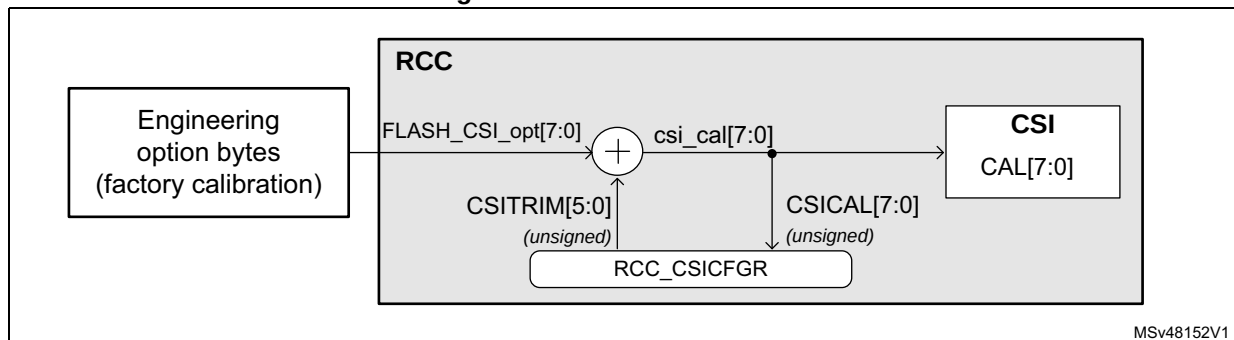
CSI calibration

RC oscillator frequencies can vary from one chip to another due to manufacturing process variations, this is why each device is factory calibrated to achieve ACC_{CSI} accuracy (refer to the product datasheet for more information). After reset, the factory calibration value is loaded in the CSICAL[7:0] bits.

Voltage and/or temperature variations affect the RC oscillator frequency. The user application can trim the CSI frequency using the CSITRIM[5:0] bits.

Note: Bits CSICAL[7:0] and CSITRIM[5:0] are located into the [RCC CSI calibration register \(RCC_CSICFGR\)](#).

Figure 58. CSI calibration flow



11.4.4 HSI48 clock

The HSI48 clock signal is generated from an internal 48 MHz RC oscillator, and can be used directly for USB and for random number generator (RNG).

The internal 48 MHz RC oscillator is mainly dedicated to provide a high-precision clock to the USB peripheral by means of a special clock recovery system (CRS) circuitry. The CRS can use the USB SOF signal, the LSE, or an external signal to automatically and quickly adjust the oscillator frequency on-the-fly. It is disabled as soon as the system enters Stop or Standby mode. When the CRS is not used, the HSI48 RC oscillator runs on its default frequency, subject to manufacturing process variations.

For more details on how to configure and use the CRS peripheral, refer to [Section 12: Clock recovery system \(CRS\)](#).

The HSI48RDY flag in the RCC_CR register indicates whether the HSI48 RC oscillator is stable or not. At startup, the HSI48 RC oscillator output clock is not released until this bit is set by hardware.

The HSI48 can be switched on and off using the HSI48ON bit in the RCC_CR register.

11.4.5 PLL description

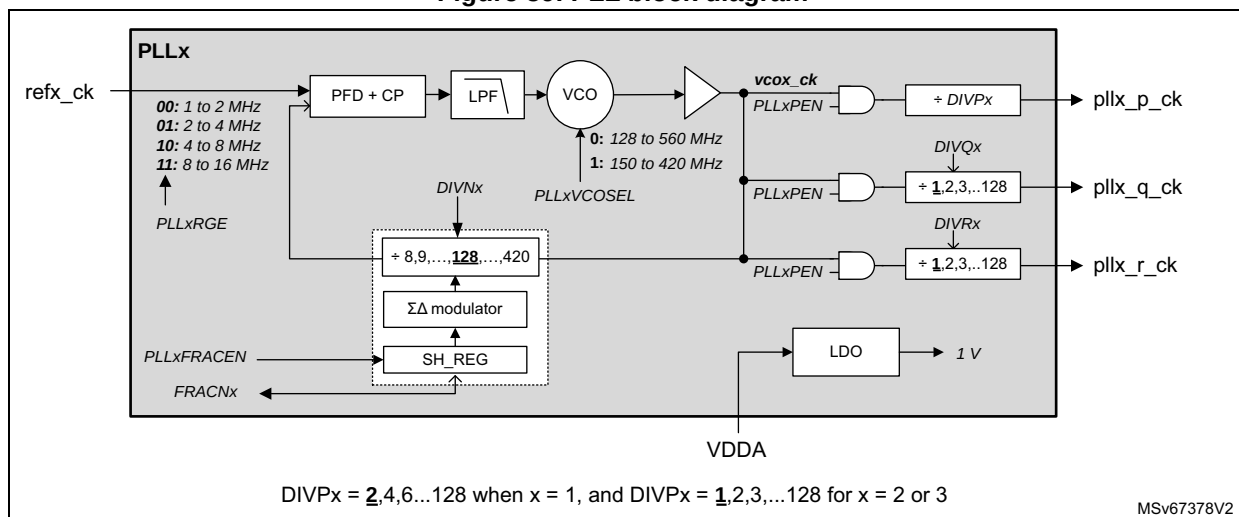
The RCC features three PLLs:

- a main PLL (PLL1), generally used to provide clocks to the CPU and some peripherals
- two dedicated PLLs (PLL2 and PLL3^(a)), used to generate the kernel clock for peripherals

The PLLs integrated into the RCC are completely independent. They offer the following features:

- A VCO supporting two modes:
 - a wide-range
 - a low-range used, for instance, in audio applications
- Input frequency range:
 - 2 to 16 MHz for the VCO in wide-range mode
 - 1 to 2 MHz for the VCO in low-range mode
- Capability to work either in integer or fractional mode
- 13-bit sigma-delta modulator, to fine-tune the VCO frequency by 11 to 0.3 ppm steps
- The sigma-delta modulator can be updated on-the-fly without generating frequency overshoots on PLLs outputs
- Each PLL offers three outputs with post-dividers

Figure 59. PLL block diagram



The PLLs are controlled via RCC_PLLxDIVR, RCC_PLLxFRACR, RCC_PLLCFGR, and RCC_CR registers.

The frequency of the reference clock provided to the PLLs (**refx_ck**) must range from 1 to 16 MHz. The PLLxM dividers of the RCC PLLx clock source selection register (RCC_PLLxCFGR) must be properly programmed to match this condition. In addition, the PLLxRGE[1:0] field of the RCC PLLx clock source selection register (RCC_PLLxCFGR) must be set according to the reference input frequency to optimize performance.

The user application can then configure the VCO. The smaller range (150 to 420 MHz) must be chosen when the reference clock frequency is lower than 2 MHz.

To reduce the power consumption, it is recommended to configure the VCO output to the smaller range.

DIVNx loop divider must be programmed to achieve the expected frequency at VCO output. In addition, the VCO output range must be respected.

a. PLL3 is not available on STM32H543/553xx devices.

The PLLs operate in integer mode when the value of SH_REG bit of the FRACNx shadow register is set to 0. The SH_REG bit is updated with the FRACNx value when PLLxFRACEN bit goes from 0 to 1. The sigma-delta modulator is designed to minimize the jitter impact, while allowing very small frequency steps.

The PLLs can be enabled by setting PLLxON to 1. The PLLxRDY bits indicate that the PLL is ready (locked).

Note: *Before enabling the PLLs, make sure that the reference frequency (**refx_ck**) provided to it is stable, so the hardware does not allow changing PLLxM when the PLLx is ON, and it is also not possible to change PLLSRC when one of the PLLs is ON.*

The hardware prevents writing PLL1ON to 0 if the PLL1 is currently used to deliver the system clock. There are other hardware protections on the clock generators (refer to [HSE clock](#), [HSI clock](#), and [CSI oscillator](#)).

The following PLL parameters cannot be changed once the PLL is enabled: DIVNx, PLLxRGE, PLLxVCOSEL, PLLxP (DIVP), PLLxQ (DIVQ), and PLLxR (DIVR).

To save power, when a PLLx output (pll_x_ck, pll_x_q_ck, or pll_x_r_ck) is not used, disable it by clearing the corresponding enable bit (PLLxPEN, PLLxQEN, or PLLxREN) in the PLLxCFGR register.

If the above rules are not respected, the PLL output frequency is not guaranteed.

Output frequency computation

When the PLL is configured in integer mode (SH_REG = 0), the VCO frequency (F_{VCO}) is given by the following expression:

$$F_{VCO} = F_{REF_CK} \times DIVN$$

$$F_{PLL_y_CK} = (F_{VCO} / (DIVy + 1)) \text{ with } y = P, Q, \text{ or } R$$

When the PLL is configured in fractional mode (SH_REG different from 0), the DIVN divider must be initialized before enabling the PLLs. However, it is possible to change the value of FRACNx on-the-fly without disturbing the PLL output.

This feature can be used either to generate a specific frequency from any crystal value with a good accuracy, or to fine-tune the frequency on-the-fly.

For each PLL, the VCO frequency is given by the following formula:

$$F_{VCO} = F_{ref_ck} \times \left(DIVN + \frac{FRACN}{2^{(13)}} \right)$$

Note: *For PLL1, DIVP can only take odd values.*

The PLLs are disabled by hardware when:

- the system enters Stop or Standby mode
- an HSE failure occurs when HSE or PLL (clocked by HSE) are used as system clock

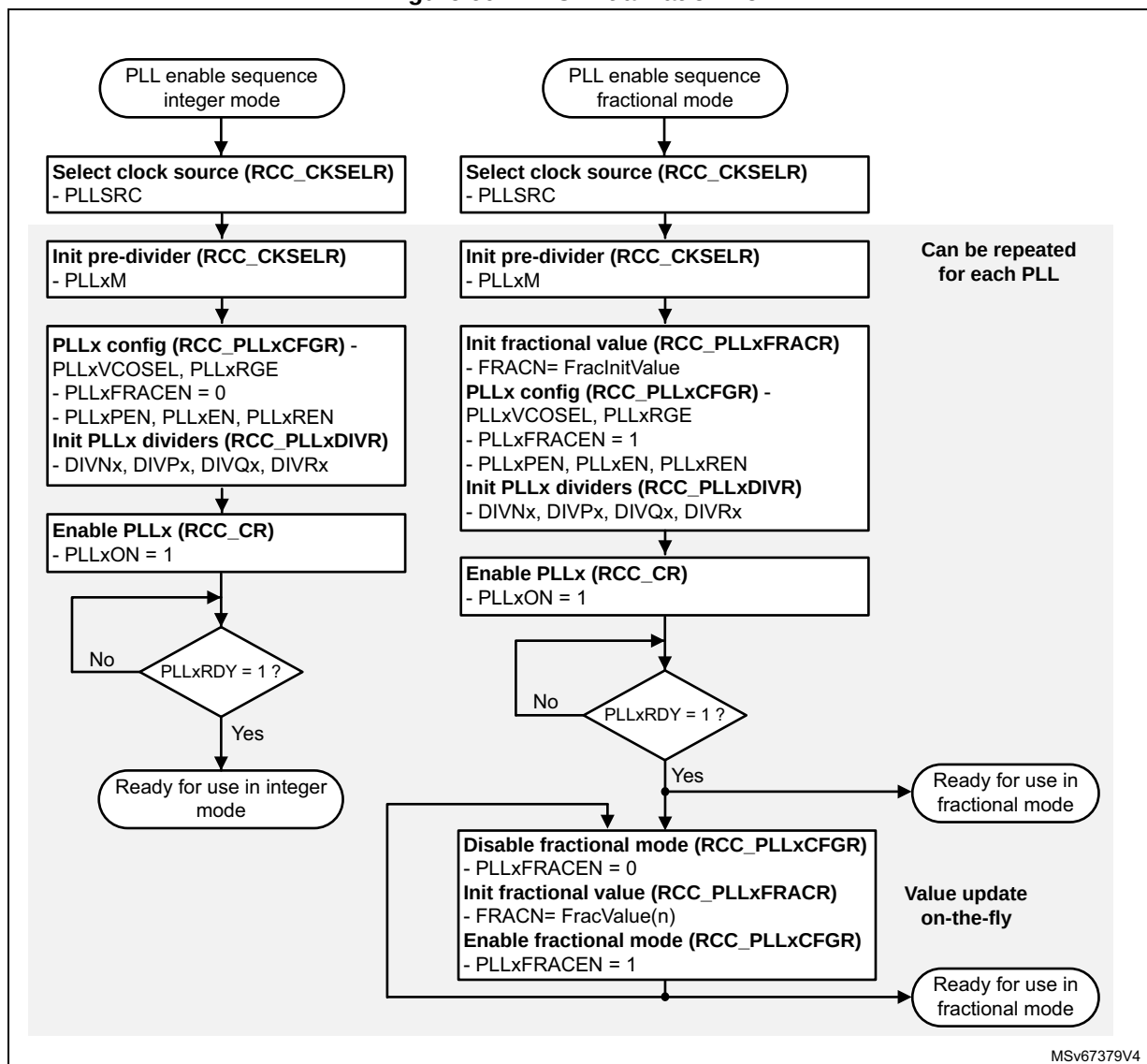
PLL initialization phase

[Figure 60](#) shows the recommended PLL initialization sequence in integer and fractional mode. The PLLx are supposed to be disabled at the start of the initialization sequence:

1. Initialize the PLLs registers according to the required frequency.
 - Set PLLXFRACEN of RCC PLLx clock source selection register (RCC_PLLXCFGR) to 0 for integer mode.
 - For fractional mode, set FRACN to the required initial value (FracInitValue) and then set PLLXFRACEN to 1.
2. Once the PLLXON bit is set to 1, the user application must wait until PLLXRDY bit is set to 1. If the PLLx is in fractional mode, the PLLXFRACEN bit must not be set back to 0 as long as PLLXRDY = 0.
3. Once the PLLXRDY bit is set to 1, the PLLx is ready to be used.
4. If the application intends to tune the PLLx frequency on-the-fly (possible only in fractional mode), then:
 - a) PLLXFRACEN must be set to 0. When PLLXFRACEN = 0, the sigma-delta modulator is still operating with the value latched into SH_REG. The application must wait for three clock periods of refx_ck (PLLXFRACEN bit propagation delay).
 - b) A new value must be uploaded into PLLXFRACR (FracValue(n)).
 - c) PLLXFRACEN must be set to 1, to latch the content of PLLXFRACR into its shadow register. The new value is considered after three clock periods of refx_ck (PLLXFRACEN bit propagation delay).

Note: When the PLLXRDY goes to 1 the difference between the PLLx output frequency and the target value is lower than $\pm 2\%$.

Figure 60. PLLs initialization flow



11.4.6 LSE clock

The LSE can generate a clock from an external crystal/ceramic resonator, or from an external user clock.

External clock source (LSE bypass)

In this mode, an external clock source must be provided to OSC32_IN pin (see [Figure 57](#)). The input clock can have a frequency up to 1 MHz, and be low swing (analog) or digital. A duty cycle close to 50% is recommended.

In case of an analog clock (low swing), the LSEBYP and LSEON bits must be set to 1 ([RCC Backup domain control register \(RCC_BDCR\)](#)).

In case of a digital clock, the LSEBYP and the LSEEXT bits must be set to 1 followed by setting the LSEON bit to 1 ([RCC Backup domain control register \(RCC_BDCR\)](#)). If the RTC

is used, the LSE bypass must not be configured in digital mode, but in low swing analog mode (default value after reset).

External crystal/ceramic resonator (LSE crystal)

The LSE clock is generated from a 32.768 kHz crystal or ceramic resonator. It has the advantage to provide a low-power, highly accurate clock source to the real-time clock (RTC) for clock/calendar or other timing functions.

The LSERDY flag of the [RCC Backup domain control register \(RCC_BDCR\)](#) indicates whether the LSE crystal is stable or not. At startup, the LSE crystal output clock signal is not released until this bit is set by hardware. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

The LSE oscillator is switched ON and OFF using the LSEON bit. The LSE remains enabled when the system enters Stop or Standby mode.

In addition, the LSE clock can be driven to the MCO1 output and used as clock source for other application components.

The LSE also offers a programmable driving capability (LSEDRV[1:0]), which can be used to modulate the amplifier driving capability. This driving capability is chosen according to the external crystal/ceramic component requirement to ensure a stable oscillation.

The driving capability must be set before enabling the LSE oscillator.

11.4.7 LSI clock

The LSI acts as a low-power clock source that can be kept running when the system is in Stop or Standby mode for the independent watchdog (IWDG) and auto-wake-up unit (AWU).

The clock frequency is around 32 kHz. For more details, refer to the electrical characteristics section of the datasheet.

The LSI can be switched ON and OFF using the LSION bit. The LSIRDY flag indicates whether the LSI oscillator is stable or not. If an independent watchdog is started either by hardware or software, the LSI is forced ON and cannot be disabled.

The LSI remains enabled when the system enters Stop or Standby mode.

At LSI startup, the clock is not provided until the hardware sets the LSIRDY bit. An interrupt can be generated if enabled in the [RCC clock source interrupt enable register \(RCC_CIER\)](#).

In addition, the LSI clock can be driven to the MCO2 output, and used as a clock source for other application components.

Note: Bits LSION and LSIRDY are located in the [RCC Backup domain control register \(RCC_BDCR\)](#).

11.4.8 System clock (SYSCLK) selection

Four different clock sources can be used to drive the system clock (SYSCLK):

- HSI oscillator
- CSI oscillator
- HSE oscillator
- PLL

The system clock maximum frequency is 250 MHz. After a system reset (or after leaving Standby mode), the HSI oscillator, at 32 MHz, is selected as system clock. When a clock source is used directly or through the PLL as a system clock, it is not possible to stop it.

A switch from one clock source to another occurs only if the target clock source is ready (clock stable after startup delay or PLL locked). If a clock source not yet ready is selected, the switch occurs when the clock source becomes ready. Status bits in the [RCC clock control register \(RCC_CR\)](#) indicate which clocks are ready, and which clock is currently used as a system clock.

11.4.9 Handling clock generators in stop and standby modes

When the whole system enters Stop mode, all the clocks (system and kernel clocks) are stopped, as well as the following clock sources:

- CSI, HSI (depending on HSIKERON and CSIKERON bits)
- HSE
- PLL1, PLL2, and PLL3^(a)
- HSI48

The content of the RCC registers is not altered except for PLL1ON, PLL2ON, PLL3ON^(a), HSEON, and HSI48ON, set to 0.

Exiting Stop mode

When the system exits this mode via a wake-up event, the application can select which oscillator (HSI and/or CSI) is used to restart. The STOPWUCK bit selects the oscillator used as system clock. The STOPKERWUCK bit selects the oscillator used as kernel clock for peripherals. The STOPKERWUCK bit is useful if after a system Stop, a peripheral needs a kernel clock generated by an oscillator different from the one used for the system clock.

These bits belong to the [RCC clock configuration register1 \(RCC_CFGR1\)](#)

Table 119. STOPWUCK and STOPKERWUCK description

STOPWUCK	STOPKERWUCK	Activated oscillator(s) when the system exits Stop mode	Distributed clocks when the system exits Stop mode	
			System clock	Kernel clock
0	0	HSI	HSI	HSI
	1	HSI and CSI		HSI and/or CSI
1	0		CSI	CSI
	1	CSI		

a. PLL3 is not available on STM32H543/553xx devices.

During Stop mode

There are two specific cases where the HSI or CSI can be enabled during this mode.

- When a dedicated peripheral requests the kernel clock the peripheral receives the HSI or CSI according to the kernel clock source selected for this peripheral (via CKPERSEL[1:0]).
- When the HSIKERON or CSIKERON bits of the [RCC clock control register \(RCC_CR\)](#) are set, the HSI and CSI are kept running, but the outputs are gated. The clock is then available immediately when the system exits Stop mode, or when a peripheral requests the kernel clock (see [Table 117](#) for details).

Table 120. HSIKERON and CSIKERON behavior

HSIKERON (CSIKERON)	HSI (CSI) state during Stop mode	HSI (CSI) state setting time
0	OFF	$t_{su(HSI)} t_{su(CSI)}$ ⁽¹⁾
1	Running and gated	Immediate

1. $t_{su(HSI)}$ and $t_{su(CSI)}$ are the startup times of, respectively, the HSI and CSI oscillators (refer to the product datasheet for the values of these parameters).

When the microcontroller exists system standby mode, the HSI is selected as system and kernel clock. The RCC registers are reset to their initial values except for the RCC_RSR and RCC_BDCR registers.

Note: The HSI and CSI outputs provide two clock paths:

- one path for the system clock (hsi_ck or csi_ck)
- one path for the peripheral kernel clock (hsi_ker_ck or csi_ker_ck).

When a peripheral requests the kernel clock in system stop mode, only the path providing the hsi_ker_ck or csi_ker_ck is activated.

11.4.10 Clock security system (CSS)

Clock security system on HSE

The clock security system can be enabled by software via the HSECSSON bit, which can be enabled even when the HSEON is set to 0.

The CSS on HSE is enabled by the hardware when the HSE is enabled and ready, and HSECSSON set to 1.

The CSS on HSE is disabled when the HSE is disabled. As a result, this function does not work when the system is in Stop mode.

It is not possible to clear directly the HSECSSON bit by software.

The HSECSSON bit is cleared by hardware when a system reset occurs or when the system enters Standby mode.

If a failure is detected on the HSE clock, the system automatically switches to the HSI or CSI, depending on STOPWUCK bit configuration in [RCC clock configuration register1 \(RCC_CFGR1\)](#), to provide a safe clock. The HSE is then automatically disabled, a clock failure event is sent to the break inputs of the advanced-control timers (TIM1, TIM8), and an NMI is automatically generated to inform the application about the failure, allowing the MCU

to perform rescue operations. If the HSE output was used as clock source for PLLs when the failure occurred, the PLLs are also disabled.

If an HSE clock failure occurs when the CSS is enabled, the CSS generates an interrupt that causes the automatic generation of an NMI. The HSECSSF flag in [RCC clock source interrupt flag register \(RCC_CIFR\)](#) is set to 1 to allow the application to identify the failure source. The NMI routine is executed indefinitely until the HSECSSF bit is cleared. As a consequence, the application must clear the HSECSSF flag in the NMI ISR by setting the HSECSSC bit in the [RCC clock source interrupt clear register \(RCC_CICR\)](#).

Clock security system on LSE

A clock security system on LSE can be activated by software writing the LSECSSON bit in the [RCC Backup domain control register \(RCC_BDCR\)](#). This bit can be disabled only by a hardware or RTC software reset, or after a failure detection on LSE. LSECSSON must be written after LSE is enabled (LSEON enabled) and ready (LSERDY set by hardware), and after the RTC clock has been selected by RTCSEL.

The CSS operates in all modes, including VBAT. It works also under system reset (excluding power-on reset). The system detects when the LSE disappears, or if it is over frequency.

If a failure is detected on the external 32 kHz oscillator, the LSE clock is no longer supplied to the RTC, but no hardware action is made to the registers.

The CSS on LSE detection event is connected to the internal tamper 3 of the TAMP peripheral. The internal tamper 3 must be enabled (ITAMP3E = 1 in TAMP_CR1 register) and the associated interrupt enabled (ITAMP3IE in TAMP_IER) to wake up from the low-power modes. This erases also the TAMP backup registers and backup SRAM unless the ITAMP3NOER = 1 in the TAMP_CR3 (see [Section 52: Tamper and backup registers \(TAMP\)](#) for more details).

In case of CSS on LSE detection event (LSECSSD = 1 in the RCC_BDCR), the software must disable the LSECSSON bit, stop the defective 32 kHz oscillator (disabling LSEON), and change the RTC clock source (no clock or LSI or HSE, with RTCSEL), or take actions to secure the application.

11.4.11 Clock output generation (MCO1/MCO2)

Two microcontroller clock output pins (MCO1 and MCO2) are available. A clock source can be selected for each output. The selected clock can be divided thanks to configurable prescaler (refer to [Figure 55](#) for additional information on signal selection).

MCO1 and MCO2 outputs are controlled via MCO1PRE[3:0], MCO1[2:0], MCO2PRE[3:0], and MCO2[2:0], located in the [RCC clock configuration register1 \(RCC_CFGR1\)](#).

The GPIO port corresponding to each MCO pin must be programmed in alternate function mode.

The clock provided to the MCOs outputs must not exceed the maximum pin speed (refer to the product datasheet for information on the supported speed).

Another output (LSCO) allows one of the low-speed clocks (LSI, LSE) to be output onto the external LSCO pin. This output is available in Stop mode, not available in Standby and VBAT modes. The selection is controlled by the LSCOSEL bit, and enabled by the LSCOEN bit in the [RCC Backup domain control register \(RCC_BDCR\)](#).

The MCO clock output requires the corresponding alternate function selected on the MCO pin. The LSCO pin must be left in default POR state.

11.4.12 Kernel clock selection

Some peripherals are designed to work with two different clock domains that operate asynchronously:

- a clock domain synchronous with the register and bus interface (**ckg_bus_perx** clock)
- a clock domain generally synchronous with the peripheral (kernel clock)

The benefit of having peripherals supporting these two clock domains is that the user application has more freedom to choose optimized clock frequency for the CPU, bus matrix and for the kernel part of the peripheral. The user application can thus change the bus frequency without reprogramming the peripherals. As an example, an ongoing transfer with UART is not disturbed if its APB clock is changed on-the-fly.

[Table 121](#) shows the kernel clock that the RCC can deliver to the peripherals. Each row represents a multiplexer and the peripherals connected to its output.

Table 121. Kernel clock distribution overview

Peripherals ⁽¹⁾	Clock multiplexer control bits	pll1_q_ck	pll1_r_ck ⁽²⁾	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck ⁽³⁾	pll3_q_ck ⁽³⁾	pll3_r_ck ⁽³⁾	sys_ck	bus clocks ⁽⁴⁾	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsi_ck	per_ck ⁽⁵⁾	AUDIOCLK	Disabled
OCTOSPI	OSPISEL	1	-	-	-	2	-	-	-	-	0	-	-	-	-	-	-	3	-	-
SDMMC1 ⁽⁶⁾	SDMMC1SEL	0	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
SDMMC2 ⁽⁶⁾	SDMMC2SEL	0	-	-	-	1	-	-	-	-	-	-	-	-	-	-	-	-	-	-
PLAY1	PLAY1SEL	-	-	1	-	-	-	-	-	-	0	-	-	-	-	3	4	5	-	-
FDCAN	FDCANSEL	1	-	-	2	-	-	-	-	-	-	0	-	-	-	-	-	-	-	3
CEC	CECSEL	-	-	-	-	-	-	-	-	-	-	-	-	2 ⁽⁷⁾	-	0	1	-	-	-
I2C1	I2C1SEL	-	-	-	-	1 ⁽²⁾	-	-	1 ⁽³⁾	-	0	-	2	3	-	-	-	-	-	-
I2C2	I2C2SEL	-	-	-	-	1 ⁽²⁾	-	-	1 ⁽³⁾	-	0	-	2	3	-	-	-	-	-	-
I2C3	I2C3SEL	-	-	-	-	1 ⁽²⁾	-	-	1 ⁽³⁾	-	0	-	2	3	-	-	-	-	-	-
I2C4	I2C4SEL	-	-	-	-	-	-	-	1	-	0	-	2	3	-	-	-	-	-	-
I3C1	I3C1SEL	-	-	-	-	1 ⁽²⁾	-	-	1 ⁽³⁾	-	0	-	2	-	-	-	-	-	-	3
I3C2	I3C2SEL	-	-	-	-	1 ⁽²⁾	-	-	1 ⁽³⁾	-	0	-	2	-	-	-	-	-	-	3
LPTIM1	LPTIM1SEL	-	-	1	-	-	-	-	2 ⁽³⁾	-	0	-	-	-	-	3	4	5	-	-
LPTIM2	LPTIM2SEL	-	-	1	-	-	-	-	2 ⁽³⁾	-	0	-	-	-	-	3	4	5	-	-
LPTIM3	LPTIM3SEL	-	-	1	-	-	-	-	2	-	0	-	-	-	-	3	4	5	-	-
LPTIM4	LPTIM4SEL	-	-	1	-	-	-	-	2	-	0	-	-	-	-	3	4	5	-	-
LPTIM5	LPTIM5SEL	-	-	1	-	-	-	-	2	-	0	-	-	-	-	3	4	5	-	-
LPTIM6	LPTIM6SEL	-	-	1	-	-	-	-	2	-	0	-	-	-	-	3	4	5	-	-
TIM[8:1],	-	-	-	-	-	-	-	-	-	-	x	-	-	-	-	-	-	-	-	-
TIM[17:12]	-	-	-	-	-	-	-	-	-	-	x	-	-	-	-	-	-	-	-	-
TIM12/15 & LPTIM2	TIMICSEL	-	-	-	-	-	-	-	-	-	-	-	1	1	-	-	-	-	-	0
RNG	RNGSEL	1	-	-	-	-	-	-	-	-	-	-	-	-	0	2	3	-	-	-
SAI1	SAI1SEL	0	-	1	-	-	2	-	-	-	-	-	-	-	-	-	-	4	3	-
SAI2	SAI2SEL	0	-	1	-	-	2	-	-	-	-	-	-	-	-	-	-	4	3	-

Table 121. Kernel clock distribution overview (continued)

Peripherals ⁽¹⁾	Clock multiplexer control bits	pll1_q_ck	pll1_r_ck ⁽²⁾	pll2_p_ck	pll2_q_ck	pll2_r_ck	pll3_p_ck ⁽³⁾	pll3_q_ck ⁽³⁾	pll3_r_ck ⁽³⁾	sys_ck	bus clocks ⁽⁴⁾	hse_ck	hsi_ker_ck	csi_ker_ck	hsi48_ck	lse_ck	lsi_ck	per_ck ⁽⁵⁾	AUDIOCLK	Disabled
SPI/I2S1	SPI1SEL	0	-	1	-	-	2 ⁽³⁾	-	-	-	-	-	-	-	-	-	-	4	3	-
SPI/I2S2	SPI2SEL	0	-	1	-	-	2 ⁽³⁾	-	-	-	-	-	-	-	-	-	-	4	3	-
SPI/I2S3	SPI3SEL	0	-	1	-	-	2 ⁽³⁾	-	-	-	-	-	-	-	-	-	-	4	3	-
SPI4	SPI4SEL	-	-	-	1	-	-	2 ⁽³⁾	-	-	0	5	3	4	-	-	-	-	-	-
SPI5	SPI5SEL	-	-	-	1	-	-	2	-	-	0	5	3	4	-	-	-	-	-	-
SPI6	SPI6SEL	-	-	-	1	-	-	2	-	-	0	5	3	4	-	-	-	-	-	-
USARTx (x=1,2,3,6,10,11)	USARTxSEL	-	-	-	1	-	-	2 ⁽³⁾	-	-	0	-	3	4	-	5	-	-	-	-
UARTx	UARTxSEL	-	-	-	1	-	-	2 ⁽³⁾	-	-	0	-	3	4	-	5	-	-	-	-
LPUART1	LPUART1SEL	-	-	-	1	-	-	2 ⁽³⁾	-	-	0	-	3	4	-	5	-	-	-	-
USB	USBSEL	1	-	-	2 ⁽²⁾	-	-	2 ⁽³⁾	-	-	-	-	-	-	3	-	-	-	-	0
ADCDAC ⁽⁶⁾	ADCDACSEL	-	-	-	-	2	-	-	-	1	0	3	4	5	-	-	-	-	-	-
DAC	DACSEL	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0	1	-	-	-
UCPD1	-	-	-	-	-	-	-	-	-	-	-	-	x ⁽⁹⁾	-	-	-	-	-	-	-
ETH CLK ⁽²⁾	ETHCLKSEL	1	-	-	-	-	-	-	-	-	-	0	-	-	-	-	-	-	-	-
ETH (ptp) ⁽²⁾	ETHPTPCLKSEL	2	1	3	-	-	-	-	-	-	0	-	-	-	-	-	-	-	-	-
ETH(ptp)	-	x	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
RTC/AWU	RTCSEL	-	-	-	-	-	-	-	-	-	-	3 ₍₁₀₎	-	-	-	1	2	-	-	0
All	CKPERSEL	-	-	-	-	-	-	-	-	-	-	2	0	1	-	-	-	-	-	3

- For the actual availability of the peripheral refer to the device datasheet.
- Available only on STM32H543/553xx devices.
- Not available on STM32H543/553xx devices.
- The bus (APB or AHB) clocks are the bus interface clocks to whom the peripherals are connected.
- The per_ck clock can be hse_ck, hsi_ker_ck, or csi_ker_ck, according to CKPERSEL selection.
- With a duty cycle close to 50%, meaning that DIV[P/Q/R]x, values must be even. For SDMMCx the duty cycle must be 50% when supporting DDR.
- Clock CSI divided by 122.
- With a duty cycle close to 50%, meaning that DIV[P/Q/R]x, values must be even. For ADC the duty cycle must be 50% when supporting DDR.
- Clock HSI divided by 4.
- Clock HSE divided by RTCPRE.

To reduce the number of switches, some peripherals share the same kernel clock source, but all have their dedicated enable signal.

Peripherals dedicated to audio applications^(a)

The audio peripherals generally need specific accurate frequencies, the kernel clock of the SAI or SPI/I2S can be generated by:

- PLL1 when the amount of active PLLs must be reduced (for SAI1 and SPI1 to 3)
- APB2 peripheral clock (for SPI4 and 5)
- APB3 peripheral clock (for SPI6)
- PLL2 or PLL3^(b) for optimal flexibility in frequency generation
- HSE, HSI, or CSI when the current consumption is critical
- AUDIOCLK when an external clock reference must be used

Peripherals dedicated to control and data transfer

Peripherals such as SPIs, I2Cs, UARTs do not need a specific kernel clock frequency but a clock fast enough to generate the correct baud rate, or the required bit clock on the serial interface. For that purpose the source can be selected among the following ones:

- PLL1 when the amount of active PLLs must be reduced
- PLL2 or PLL3^(b) if better flexibility is required. As an example, this solution allows changing the frequency bus via PLL1 without affecting the speed of some serial interfaces.
- HSI or CSI for low-power use-cases or when the peripheral must quickly wake up from Stop mode (such as UART or I2C)

Note: UARTs also need the LSE clock when high baud rates are not required.

The OCTOSPI and SDMMC1/2^(a) can also use a clock different from the bus interface one for more flexibility.

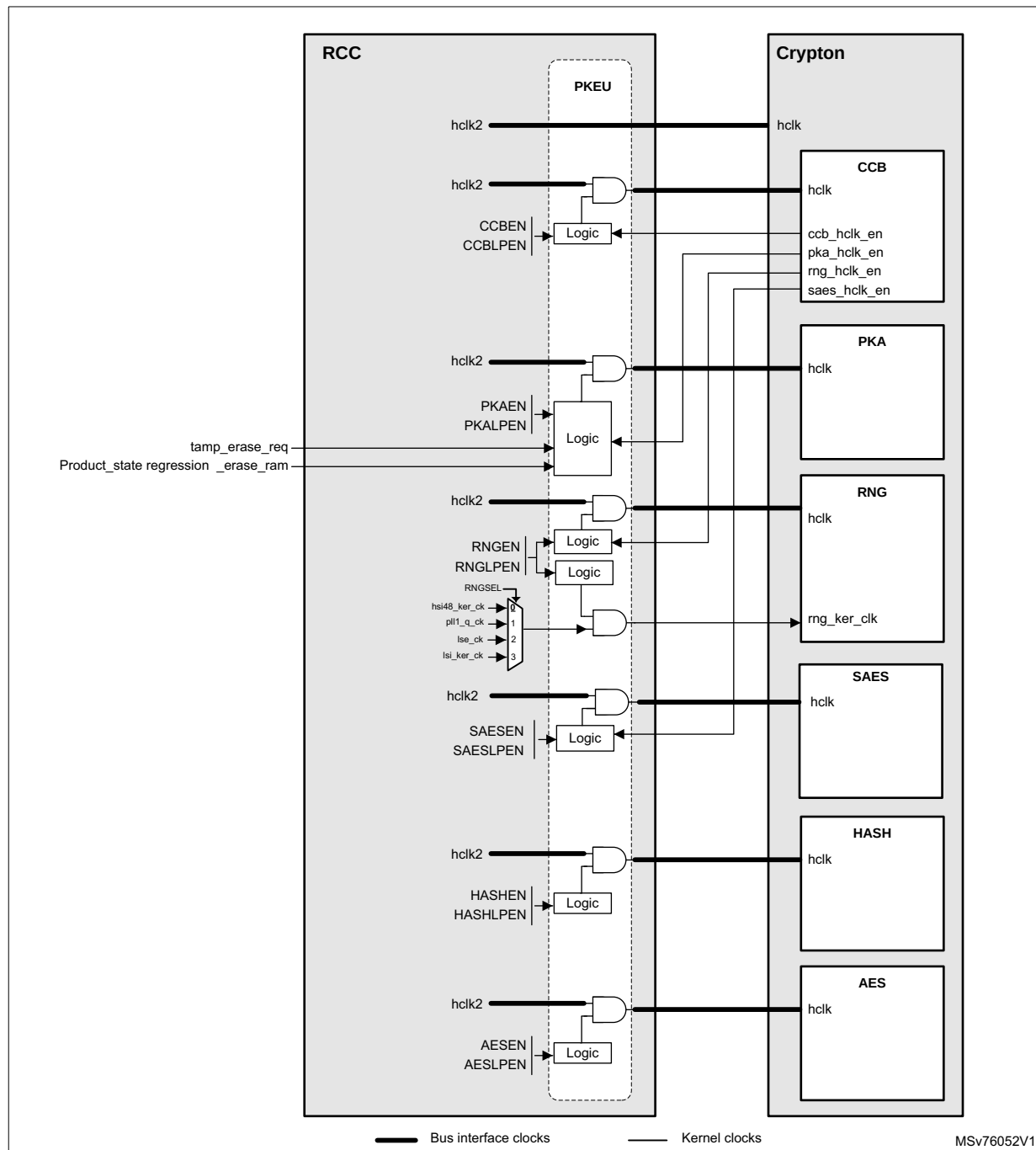
a. Refer to the device datasheet for the availability of the peripherals.

b. PLL3 is not available on STM32H543/553xx devices.

Clock distribution for CCB subsystem

Available only on STM32H543/553xx devices.

Figure 61. Clock distribution for CCB subsystem



RTC/AWU clock

The **rtc_ck** clock source can be one of the following:

- the **hse_1M_ck** (**hse_ck** divided by a programmable prescaler)
- the **lse_ck**
- the **lsi_ck** clock

The source clock is selected by programming the RTCSEL[1:0] bits in the [RCC Backup domain control register \(RCC_BDCR\)](#) and the RTCPRE[5:0] bits in the [RCC clock configuration register1 \(RCC_CFGR1\)](#).

This selection cannot be modified without resetting the Backup domain.

If the LSE is selected as RTC clock, the RTC works normally even if the backup or the V_{DD} supply disappears.

The LSE clock is in the Backup domain, whereas the other oscillators are not. As a consequence:

- If LSE is selected as RTC clock, the RTC continues working even if the V_{DD} supply is switched OFF, provided the V_{BAT} supply is maintained.
- If LSI is selected as the RTC clock, the AWU state is not guaranteed if the V_{DD} supply is powered off.
- If the HSE clock is used as RTC clock, the RTC state is not guaranteed if the V_{DD} supply is powered off or if the V_{CORE} supply is powered off.

The **rtc_ck** clock is enabled through RTCEN bit located in the [RCC Backup domain control register \(RCC_BDCR\)](#).

The RTC bus interface clock (APB clock) is enabled through RTCAPBEN and RTCAPBLPEN bits located in RCC_APB3ENR/LPENR registers.

Note: To read the RTC calendar register when the APB clock frequency is less than seven times the RTC clock frequency ($F_{APB} < 7 \times F_{RTCLK}$), the software must read the calendar time and date registers twice. The data are correct if the second read access to RTC_TR gives the same result than the first one. Otherwise a third read access must be performed.

Watchdog clocks

The RCC provides the clock for the two watchdog blocks available on the circuit. The independent watchdog (IWDG) is connected to the LSI. The window watchdog (WWDG) is connected to the APB clock.

If an independent watchdog is started by either hardware option or software access, the LSI is forced ON and cannot be disabled. After the LSI oscillator setup delay, the clock is provided to the IWDG.

Clock frequency measurement using TIMx

Most of the clock source generator frequencies can be measured by means of the input capture of TIMx.

- Calibrating the HSI or CSI with the LSE:

The primary purpose of having the LSE connected to a TIMx input capture is to be able to accurately measure the HSI or CSI. This requires to use the HSI or CSI as system clock source either directly or via PLL1. The number of system clock counts between consecutive edges of the LSE signal gives a measurement of the internal clock period. Taking advantage of the high precision of LSE crystals (typically a few tens of ppm) we

can determine the internal clock frequency with the same resolution, and trim the source to compensate for manufacturing-process and/or temperature- and voltage-related frequency deviations.

The basic concept consists in providing a relative measurement (e.g. HSI/LSE ratio). The precision is therefore tightly linked to the ratio between the two clock sources. The greater the ratio is, the more accurate the measurement is.

The HSI and CSI oscillators have dedicated user-accessible calibration bits for this purpose (see [RCC CSI calibration register \(RCC_CSICFGR\)](#)). When HSI or CSI is used via the PLLx, the system clock can also be fine-tuned by using the fractional divider of the PLLs.

- Calibrating the LSI with the HSI:

The LSI frequency can also be measured: this is useful for applications that do not have a crystal. The ultra-low-power LSI oscillator has a large manufacturing process deviation. The LSI clock frequency can be measured using the more precise HSI clock source. Using this measurement, a more accurate RTC time base timeouts (when LSI is used as the RTC clock source) and/or an IWDG timeout with an acceptable accuracy can be obtained.

11.4.13 RTC and TAMP clock

The RTCCLK clock source is used by RTC and TAMP, and can be either the HSE / 32, LSE or LSI clock. It is selected by programming the RTCSEL[1:0] bits in the [RCC Backup domain control register \(RCC_BDCR\)](#). This selection cannot be modified without resetting the Backup domain. The system must always be configured so as to get a PCLK frequency greater than or equal to the RTCCLK frequency for a proper operation of the RTC. The TAMP does not require any kernel clock if only the backup registers are used, with tamper in edge detection mode. All other tamper detection modes require a kernel clock (refer to [Section 52: Tamper and backup registers \(TAMP\)](#) for more details).

The LSE is in the Backup domain, whereas the HSE and LSI clocks are not. Consequently:

- If LSE is selected as RTC and TAMP clock, these peripherals continue to work even if the V_{DD} supply is switched off, provided the V_{BAT} supply is maintained.
- If the HSE clock divided by a prescaler is used as the RTC or TAMP clock, the RTC state is not guaranteed if the V_{DD} supply is powered off or if the internal voltage regulator is powered off (removing power from the core domain). Depending on the TAMP configuration, this one can remain functional if used in a mode that does not need any kernel clock.
- If the LSI is used as the RTC or TAMP clock, the RTC state is not guaranteed if the V_{DD} supply is powered off. Depending on the TAMP configuration, this one can remain functional if used in a mode that does not need any kernel clock.

When the RTC and TAMP clock is LSE or LSI, the RTC remains clocked and functional under system reset.

11.4.14 Timer clock

The timer clock frequencies are automatically defined by hardware.

There are two cases:

- If the APB prescaler equals 1, the timer clock frequencies are set to the APB domain frequency.
- Otherwise, they are set to twice ($\times 2$) the APB domain frequency.

11.4.15 Watchdog clock

If the independent watchdog (IWDG) is started by either hardware option or software access, the LSI oscillator is forced on and cannot be disabled. After the LSI oscillator temporization, the LSI 32 kHz clock is provided to the IWDG.

11.4.16 Peripherals clock gating and autonomous mode

Peripherals clock gating in Run mode

Each peripheral clock can be enabled by the corresponding EN bit in the RCC_AHBxENR and RCC_APBxENR registers.

When the peripheral clock is not active, read or write accesses to the peripheral registers are not supported.

The enable bit has a synchronization mechanism to create a glitch-free clock for the peripheral. After the enable bit is set, there the clock is active after 2 cycles of the peripheral bus clock.

Caution: Just after enabling the clock for a peripheral, the software must wait for these 2 clock cycles before accessing the peripheral registers.

Peripherals clock gating in Sleep mode

When a peripheral is enabled, its clock can be automatically gated off when the device is in Sleep mode, by clearing the peripheral LPEN bit in the RCC_AHBxLPENR and RCC_APBxLPENR registers. Both EN and LPEN bit of the peripheral must be set to keep the clock on in Sleep mode.

11.5 RCC security and privilege functional description

11.5.1 RCC TrustZone security protection modes

When the TrustZone security is activated by the TZEN option byte in the Flash option byte configuration register, the RCC is able to secure RCC configuration and status bits from being modified by nonsecure accesses.

This is configured through the [RCC secure configuration register \(RCC_SECCFGR\)](#) to prevent nonsecure access to read or modify the following features:

- HSE, HSE-CSS, HSI, CSI, LSI, LSE, LSE-CSS, LSCO, HSI48 configuration and status bits
- PLL1, PLL2, PLL3, AHB and APB prescalers configuration and status bits
- system clock (SYSCLK) and ICLK source clock selection and status bits
- MCO clock output configuration and STOPWUCK and STOPKERWUCK bit
- Remove reset flag RMVF configuration

If SPRIV is set in the [RCC privilege configuration register \(RCC_PRIVCFGR\)](#), the RCC_SECCFGR register can be written only by secure and privileged access. If SPRIV is cleared in RCC_PRIVCFGR, RCC_SECCFGR can be written only by secure access, privileged or unprivileged.

RCC_SECCFGR can be read by secure, nonsecure, privileged and unprivileged access.

When a peripheral is configured as secure, its related clock, reset, clock source selection and clock enable during low-power modes control bits, are also secure in the RCC_AHBxENR, RCC_APBxENR, RCC_CCIPRx and RCC_BDCR registers.

Note: When at least one feature in RTC or TAMP is secure, the RTCPRE[5:0] in RCC_CFGR1, as well as RTCEN and RTCSEL[1:0] in RCC_BDCR, become secure.

A peripheral is secure when:

- For securable peripherals by TZSC (see [Section 5.4.3: TrustZone security controller \(TZSC\)](#)), the SEC security bit corresponding to this peripheral is set in the GTZC TZSC secure configuration registers.
- For TrustZone®-aware peripherals, a security feature of this peripheral is enabled through its dedicated bits.

[Table 122](#) summarizes the RCC secured bits following the security configuration bit in the RCC_SECCFGR register.

When one security configuration bit is set, some configuration and status bits are secured. The RCC registers may contain secure and nonsecure bits:

- Secured bits: read and write operations are only allowed by a secure access. Nonsecure read returns 0 and write accesses are ignored. No illegal access event is generated.
- Nonsecure bits: no restriction. Read and write operations are allowed by both secure and nonsecure accesses.
- A nonsecure write access to RCC_SECCFGR is ignored and generates an illegal access event. An illegal access interrupt is generated if the RCC illegal access interrupt is enabled in the GTZC TZIC registers. RCC_SECCFGR can be read by secure or nonsecure access.

When the TrustZone security is disabled (TZEN = 0xC3), all registers are nonsecure. RCC_SECCFGR write accesses are ignored.

Table 122. RCC security configuration summary

Configuration bit in RCC_SECCFGR	Secured bits	Corresponding register
HSISEC	HSION, HSIKERON, HSIRDY	RCC_CR
	HSICAL[11:0], HSITRIM[6:0]	RCC_HSI CFGR
	HSIRDYIE	RCC_CIER
	HSIRDYIF	RCC_CIFR
	HSIRDYC	RCC_CICR
HSESEC	HSEON, HSERDY, HSEBYP, HSECSSON, HSEEXT	RCC_CR
	HSERDYIE, HSECSSF	RCC_CIER
	HSERDYIF, HSECSSF	RCC_CIFR
	HSERDYC, HSECSSC	RCC_CICR

Table 122. RCC security configuration summary (continued)

Configuration bit in RCC_SECCFGR	Secured bits	Corresponding register
CSISEC	CSISON, CSIKERON, CSISRDY	RCC_CR
	CSICAL[7:0], CSITRIM[5:0]	RCC_CSICFGR
	CSISRDYIE	RCC_CIER
	CSISRDYIF	RCC_CIFR
	CSISRDYIC	RCC_CICR
LSISEC	LSION, LSIRDY, LSIPREDIV, LSCOSEL, LSCOEN	RCC_BDCR
	LSIRDYIE	RCC_CIER
	LSIRDYIF	RCC_CIFR
	LSIRDYC	RCC_CICR
LSESEC	LSECSSON, LSECSSD, LSEDRV[1:0], LSEBYP, LSERDY, LSEON, LSEEXT, LSCOSEL, LSCOEN	RCC_BDCR
	LSERDYIE	RCC_CIER
	LSERDYF	RCC_CIFR
	LSERDYC	RCC_CICR
SYSCLKSEC	SW[1:0], SWS[1:0], STOPWUCK, STOPKERWUCK, MCO1SEL[3:0], MCO1PRE[2:0], MCO2SEL[3:0], MCO2PRE[2:0]	RCC_CFGR1
	SYSTICKSEL[1:0]	RCC_CCIPR4
	VOS[1:0]	PWR_VOSR
	TIMPRE	RCC_CFGR1
PRESCSEC	HPRE[3:0], PPRE1[2:0], PPRE2[2:0], PPRE3[2:0]	RCC_CFGR2
PLL1SEC	PLL1SRC[1:0], PLL1RGE[1:0], PLL1FRACEN, PLL1M[3:0], PLL1VCOSEL, PLL1PEN, PLL1QEN, PLL1REN	RCC_PLL1CFGR
	PLL1N[8:0], PLL1P[6:0], PLL1Q[6:0], PLL1R[6:0]	RCC_PLL1DIVR
	PLL1FRACN[12:0]	RCC_PLL1FRACR
	PLL1RDY, PLL1ON	RCC_CR
	PLL1RDYIE	RCC_CIER
	PLL1RDYF	RCC_CIFR
	PLL1RDYC	RCC_CICR

Table 122. RCC security configuration summary (continued)

Configuration bit in RCC_SECCFGR	Secured bits	Corresponding register
PLL2SEC	PLL2SRC[1:0], PLL2RGE[1:0], PLL2FRACEN, PLL2M[3:0], PLL2PEN, PLL2QEN, PLL2REN, PLL2VCOSEL	RCC_PLL2CFGR
	PLL2N[8:0], PLL2P[6:0], PLL2Q[6:0], PLL2R[6:0]	RCC_PLL2DIVR
	PLL2FRACN[12:0]	RCC_PLL2FRACR
	PLL2RDY, PLL2ON	RCC_CR
	PLL2RDYIE	RCC_CIER
	PLL2RDYF	RCC_CIFR
	PLL2RDYC	RCC_CICR
PLL3SEC ⁽¹⁾	PLL3SRC[1:0], PLL3RGE[1:0], PLL3FRACEN, PLL3M[3:0], PLL3PEN, PLL3QEN, PLL3REN, PLL3VCOSEL	RCC_PLL3CFGR
	PLL3N[8:0], PLL3P[6:0], PLL3Q[6:0], PLL3R[6:0]	RCC_PLL3DIVR
	PLL3FRACN[12:0]	RCC_PLL3FRACR
	PLL3RDY, PLL3ON	RCC_CR
	PLL3RDYIE	RCC_CIER
	PLL3RDYF	RCC_CIFR
	PLL3RDYC	RCC_CICR
HSI48SEC ⁽²⁾	HSI48ON, HSI48RDY	RCC_CR
	HSI48CAL[9:0]	RCC_CRRCR
	HSI48RDYIE	RCC_CIER
	HSI48RDYF	RCC_CIFR
	HSI48RDYC	RCC_CICR
IPKERSECCFG	CKERPSEL[1:0]	RCC_CCIPR5
RMVFSEC	RMVF	RCC_CSR

1. PLL3SEC registers are not available on STM32H543/553xx devices.

2. TRIM field of the HSI48 is located in CRS peripheral. Be sure to secure it using CRSSEC bit in GTZC1 TZSC secure configuration register 1.

11.5.2 RCC privilege protection modes

By default, after reset, all RCC registers can be read or written with both privileged and unprivileged access except *RCC privilege configuration register (RCC_PRIVCFGR)* that can be written with privileged access only. RCC_PRIVCFGR can be read by secure and non secure, privileged and unprivileged access.

The SPRIV bit in RCC_PRIVCFGR can be written with secure privileged access only. This bit configures the privileged access of all RCC secure functions (as defined by *RCC secure configuration register (RCC_SECCFGR)* or by the GTZC for securable peripherals, or by the peripheral itself in case of TrustZone-aware peripherals).

When the SPRIV bit is set in RCC_PRIVCFGR:

- Writing the RCC secure bits is possible only with privileged access, including RCC_SECCFGR.
- The RCC secure bits can be read only with privileged access except RCC_SECCFGR and RCC_PRIVCFGR that can be read by privileged or unprivileged access.
- An unprivileged access to a privileged RCC bit or register is discarded: the bits are read as zero and the write to these bits is ignored (RAZ/WI).

The NSPRIV bit in RCC_PRIVCFGR can be written with privileged access only, secure or nonsecure. This bit configures the privileged access of all RCC nonsecure functions (as defined by RCC_SECCFGR, or by the GTZC for securable peripherals, or by the peripheral itself in case of TrustZone-aware peripherals).

When the NSPRIV bit is set in RCC_PRIVCFGR:

- Writing the RCC nonsecure bits is possible only with privileged access.
- The RCC nonsecure bits can be read only with privileged access except RCC_PRIVCFGR that can be read by privileged or unprivileged access.
- An unprivileged access to a privileged RCC bit or register is discarded: the bits are read as zero and the write to these bits is ignored (RAZ/WI).

11.6 RCC low-power modes

- AHB and APB peripheral clocks, including DMA clock, can be disabled by software.
- Sleep mode stops the CPU clock. The memory interface clocks (flash memory, caches and all SRAM interfaces) can be stopped by software during Sleep mode. The AHB to APB bridge clocks are disabled by hardware during Sleep mode when all the clocks of the peripherals connected to them are disabled.
- Stop mode stops all the clocks in the core domain and disable the PLLs, HSI, HSI48, CSI and HSE oscillators. However, HSI or CSI can be switched ON to generate a wake-up interrupt. LSI and LSE remain active in Stop mode.
- Standby mode stops all the clocks in the core domain and disable the PLLs, HSI, HSI48, CSI and HSE oscillators.

The CPU DeepSleep mode can be overridden for debugging by setting the DBG_STOP or DBG_STANDBY bit in the DBGMCU_CR register.

When exiting Stop mode, the system clock is either HSI or CSI, depending on the software configuration of STOPWUCK in the [RCC CPU domain clock configuration register 2 \(RCC_CFGR2\)](#). The frequency (range and user trim) of the HSI is the one configured before entering Stop mode.

The other internal oscillator can be automatically woken up in addition to the one used by the system clock, to avoid waiting for the other oscillator wake-up time when the device is back in Run mode. This is done thanks to STOPKERWUCK in RCC_CFGR1.

When leaving the Standby mode, the system clock is HSI (32 MHz). The user trim is lost.

If a flash memory programming operation is ongoing, Stop or Standby mode entry is delayed until the flash memory interface access is finished. If an access to the APB domain is ongoing, Stop or Standby mode entry is delayed until the APB access is finished.

11.7 RCC interrupts

[Table 123](#) summarizes the interrupt sources and the way to control them.

Table 123. Interrupt sources and control

Interrupt vector	Interrupt event flag	Description	Enable control bits	Interrupt clear method	Exit from Sleep mode	Exit from Stop and Standby modes
RCC	LSIRDYF	LSI ready	LSIRDYIE and LSISEC = 0	Set LSIRDYC to 1	Yes	No
	LSERDYF	LSE ready	LSERDYIE and LSESEC = 0	Set LSERDYC to 1	Yes	No
	HSIDRYF	HSI ready	HSIDRYIE and HSISEC = 0	Set HSIRDYC to 1	Yes	No
	HSERDYF	HSE ready	HSERDYIE and HSESEC = 0	Set HSERDYC to 1	Yes	No
	CSISRDYF	CSIS ready	CSISRDYIE and CSISEC = 0	Set CSISRDYC to 1	Yes	No
	HSI48RDYF	HSI48 ready	HSI48RDYIE and HSI48SEC = 0	Set HSI48RDYC to 1	Yes	No
	PLL1RDYF	PLL1 ready	PLL1RDYIE and PLL1SEC = 0	Set PLL1RDYC to 1	Yes	No
	PLL2RDYF	PLL2 ready	PLL2RDYIE and PLL2SEC = 0	Set PLL2RDYC to 1	Yes	No
	PLL3RDYF ⁽¹⁾	PLL3 ready	PLL3RDYIE and PLL3SEC = 0	Set PLL3DYC to 1	Yes	No
RCC_S ⁽²⁾	LSIRDYF	LSI ready	LSIRDYIE and LSISEC = 1	Set LSIRDYC to 1	Yes	No
	LSERDYF	LSE ready	LSERDYIE and LSESEC = 1	Set LSERDYC to 1	Yes	No
	HSIDRYF	HSI ready	HSIDRYIE and HSISEC = 1	Set HSIRDYC to 1	Yes	No
	HSERDYF	HSE ready	HSERDYIE and HSESEC = 1	Set HSERDYC to 1	Yes	No
	CSISRDYF	CSIS ready	CSISRDYIE and CSISEC = 1	Set CSISRDYC to 1	Yes	No
	HSI48RDYF	HSI48 ready	HSI48RDYIE and HSI48SEC = 1	Set HSI48RDYC to 1	Yes	No
	PLL1RDYF	PLL1 ready	PLL1RDYIE and PLL1SEC = 1	Set PLL1RDYC to 1	Yes	No
	PLL2RDYF	PLL2 ready	PLL2RDYIE and PLL2SEC = 1	Set PLL2RDYC to 1	Yes	No
	PLL3RDYF ⁽¹⁾	PLL3 ready	PLL3RDYIE and PLL3SEC = 1	Set PLL3RDYC to 1	Yes	No

Table 123. Interrupt sources and control (continued)

Interrupt vector	Interrupt event flag	Description	Enable control bits	Interrupt clear method	Exit from Sleep mode	Exit from Stop and Standby modes
TAMP	ITAMP3F ⁽³⁾	LSE CSS failure	LSECSSON and ITAMP3E ⁽³⁾ and ITAMP3IE ⁽³⁾	Set CITAMP3F ⁽³⁾ to 1	Yes	Yes
NMI	HSECSSF	HSE CSS failure	⁽⁴⁾	Set HSECSSC to 1	Yes	No

1. PLL3 is not available on STM32H543/553xx devices.
2. The RCC secure interrupt vector is used only when TrustZone is enabled.
3. The LSE CSS failure event (LSECSSD) is connected to TAMP internal tamper 3. to get the interrupt associated to this event, the internal tamper 3 must be enabled, and the internal tamper 3 interrupt must be enabled. The ITAMP3F, ITAMP3E, ITAMP3IE, and CITAMP3F bits are in the TAMP peripheral.
4. It is not possible to mask this interrupt when the security system feature is enabled (HSECSSON = 1).

11.8 RCC registers

11.8.1 RCC clock control register (RCC_CR)

Address offset: 0x000

Reset value: 0x0000 002B

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	PLL3RDY	PLL3ON	PLL2RDY	PLL2ON	PLL1RDY	PLL1ON	Res.	Res.	Res.	HSEEXT	HSECSN	HSEBYP	HSERDY	HSEON
		r	rw	r	rw	r	rw				rw	rs	rw	r	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	HSI48RDY	HSI48ON	Res.	CSIKERON	CSIRDY	CSION	Res.	Res.	HSIDIVF	HSIDIV[1:0]		HSIKERON	HSIRDY	HSION
		r	rw		rw	r	rw			r	rw	rw	rw	r	rw

Bits 31:30 Reserved, must be kept at reset value.

Bit 29 **PLL3RDY**: PLL3 clock ready flag

Set by hardware to indicate that the PLL3 is locked.

0: PLL3 unlocked (default after reset)

1: PLL3 locked

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 28 **PLL3ON**: PLL3 enable

Set and cleared by software to enable PLL3.

Cleared by hardware when entering Stop or Standby mode.

0: PLL3 OFF (default after reset)

1: PLL3 ON

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 27 **PLL2RDY**: PLL2 clock ready flag

Set by hardware to indicate that the PLL is locked.

0: PLL2 unlocked

1: PLL2 locked

Bit 26 **PLL2ON**: PLL2 enable

Set and cleared by software to enable PLL2.

Cleared by hardware when entering Stop or Standby mode.

0: PLL2 OFF (default after reset)

1: PLL2 ON

Bit 25 **PLL1RDY**: PLL1 clock ready flag

Set by hardware to indicate that the PLL1 is locked.

0: PLL1 unlocked (default after reset)

1: PLL1 locked

Bit 24 PLL1ON: PLL1 enable

Set and cleared by software to enable PLL1.

Cleared by hardware when entering Stop or Standby mode. Note that the hardware prevents writing this bit to 0, if the PLL1 output is used as the system clock.

0: PLL1 OFF (default after reset)

1: PLL1 ON

Bits 23:21 Reserved, must be kept at reset value.

Bit 20 HSEEXT: external high speed clock type in Bypass mode

Set and reset by software to select the external clock type (analog or digital).

The external clock must be enabled with the HSEON bit to be used by the device. The HSEEXT bit can be written only if the HSE oscillator is disabled.

0: HSE in analog mode (default after reset)

1: HSE in digital mode

Bit 19 HSECSSON: HSE clock security system enable

Set by software to enable clock security system on HSE.

This bit is "set only" (disabled by a system reset or when the system enters in Standby mode). When HSECSSON is set, the clock detector is enabled by hardware when the HSE is ready and disabled by hardware if an oscillator failure is detected.

0: CSS on HSE OFF (clock detector OFF) (default after reset)

1: CSS on HSE ON (clock detector ON if the HSE oscillator is stable, OFF if not).

Bit 18 HSEBYP: HSE clock bypass

Set and cleared by software to bypass the oscillator with an external clock. The external clock must be enabled with the HSEON bit to be used by the device.

The HSEBYP bit can be written only if the HSE oscillator is disabled.

0: HSE oscillator not bypassed (default after reset)

1: HSE oscillator bypassed with an external clock

Bit 17 HSERDY: HSE clock ready flag

Set by hardware to indicate that the HSE oscillator is stable.

0: HSE clock is not ready (default after reset)

1: HSE clock is ready

Bit 16 HSEON: HSE clock enable

Set and cleared by software.

Cleared by hardware to stop the HSE when entering Stop or Standby mode.

This bit cannot be cleared if the HSE is used directly (via SW mux) as system clock, or if the HSE is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to 1).

0: HSE is OFF (default after reset)

1: HSE is ON

Bits 15:14 Reserved, must be kept at reset value.

Bit 13 HSI48RDY: HSI48 clock ready flag

Set by hardware to indicate that the HSI48 oscillator is stable.

0: HSI48 clock is not ready (default after reset)

1: HSI48 clock is ready

Bit 12 HSI48ON: HSI48 clock enable

Set by software and cleared by software or by the hardware when the system enters to Stop or Standby mode.

0: HSI48 is OFF (default after reset)

1: HSI48 is ON

Bit 11 Reserved, must be kept at reset value.

Bit 10 **CSIKERON**: CSI clock enable in Stop mode

Set and reset by software to force the CSI to ON, even in Stop mode, to be quickly available as kernel clock for some peripherals. This bit has no effect on the value of CSION.

0: no effect on CSI (default after reset)

1: CSI is forced to ON even in Stop mode

Bit 9 **CSIRDY**: CSI clock ready flag

Set by hardware to indicate that the CSI oscillator is stable. This bit is activated only if the RC is enabled by CSION (it is not activated if the CSI is enabled by CSIKERON or by a peripheral request).

0: CSI clock is not ready (default after reset)

1: CSI clock is ready

Bit 8 **CSION**: CSI clock enable

Set and reset by software to enable/disable CSI clock for system and/or peripheral.

Set by hardware to force the CSI to ON when the system leaves Stop mode, if STOPWUCK = 1 or STOPKERWUCK = 1.

This bit cannot be cleared if the CSI is used directly (via SW mux) as system clock, or if the CSI is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to 1).

0: CSI is OFF (default after reset)

1: CSI is ON

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **HSIDIVF**: HSI divider flag

Set and reset by hardware.

As a write operation to HSIDIV has not an immediate effect on the frequency, this flag indicates the

current status of the HSI divider. HSIDIVF goes immediately to 0 when HSIDIV value is changed, and is set back to 1 when the output frequency matches the value programmed into HSIDIV.

0: new division ratio not yet propagated to hsi_ck, hsi_ker_ck (default after reset)

1: hsi_ck, hsi_ker_ck clock frequency reflects the new HSIDIV value (default register value when the clock setting is completed).

Bits 4:3 **HSIDIV[1:0]**: HSI clock divider

Set and reset by software.

These bits allow selecting a division ratio to configure the wanted HSI clock frequency. The HSIDIV cannot be changed if the HSI is selected as reference clock for at least one enabled PLL (PLLxON bit set to 1). In that case, the new HSIDIV value is ignored..

00: division by 1, hsi_ck, hsi_ker_ck = 64 MHz

01: division by 2, hsi_ck, hsi_ker_ck = 32 MHz (default after reset)

10: division by 4, hsi_ck, hsi_ker_ck = 16 MHz

11: division by 8, hsi_ck, hsi_ker_ck = 8 MHz

Bit 2 **HSIKERON**: HSI clock enable in Stop mode

Set and reset by software to force the HSI to ON, even in Stop mode, to be quickly available as kernel clock for peripherals. This bit has no effect on the value of HSION.

0: no effect on HSI (default after reset)

1: HSI is forced to ON even in Stop mode

Bit 1 **HSIRDY**: HSI clock ready flag

Set by hardware to indicate that the HSI oscillator is stable.

0: HSI clock is not ready (default after reset)

1: HSI clock is ready

Bit 0 **HSION**: HSI clock enable

Set and cleared by software.

Set by hardware to force the HSI to ON when the product leaves Stop mode, if STOPWUCK = 1 or STOPKERWUCK = 1.

Set by hardware to force the HSI to ON when the product leaves Standby mode or in case of a failure of the HSE which is used as the system clock source.

This bit cannot be cleared if the HSI is used directly (via SW mux) as system clock, or if the HSI is selected as reference clock for PLL1 with PLL1 enabled (PLL1ON bit set to 1).

0: HSI is OFF

1: HSI is ON (default after reset)

11.8.2 RCC HSI calibration register (RCC_HSI CFGR)

Address offset: 0x010

Reset value: 0x0040 0XXX

Reset value depends on the flash memory option bytes setting.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSITRIM[6:0]						
									rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	HSICAL[11:0]											
				r	r	r	r	r	r	r	r	r	r	r	r

Bits 31:23 Reserved, must be kept at reset value.

Bits 22:16 **HSITRIM[6:0]**: HSI clock trimming

Set by software to adjust calibration.

HSITRIM field is added to the engineering option bytes loaded during reset phase (FLASH_HSI_OPT) to form the calibration trimming value.

HSICAL = HSITRIM + FLASH_HSI_OPT.

After a change of HSITRIM it takes one system clock cycle before the new HSITRIM value is updated

Note: The reset value of the field is 0x40.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:0 **HSICAL[11:0]**: HSI clock calibration

Set by hardware by option byte loading during system reset nreset. Adjusted by software through trimming bits HSITRIM.

This field represents the sum of engineering option byte calibration and HSITRIM bits values.

11.8.3 RCC clock recovery RC register (RCC_CRRCR)

Address offset: 0x014

Reset value: 0x0000 0XXX

Reset value depends on the flash option bytes setting

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	HSI48CAL[9:0]									
						r	r	r	r	r	r	r	r	r	r

Bits 31:10 Reserved, must be kept at reset value.

Bits 9:0 **HSI48CAL[9:0]**: Internal RC 48 MHz clock calibration

Set by hardware by option-byte loading during system reset NRESET. Read-only.

11.8.4 RCC CSI calibration register (RCC_CSICFGR)

Address offset: 0x018

Reset value: 0x0020 0XXX

Reset value depends on the flash option bytes setting

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSITRIM[5:0]					
										rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSICAL[7:0]							
								rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:22 Reserved, must be kept at reset value.

Bits 21:16 **CSITRIM[5:0]**: CSI clock trimming

Set by software to adjust calibration.

CSITRIM field is added to the engineering option bytes loaded during reset phase (FLASH_CSI_OPT) to form the calibration trimming value.

$CSICAL = CSITRIM + FLASH_CSI_OPT$.

Note: The reset value of the field is 0x20.

Bits 15:8 Reserved, must be kept at reset value.

Bits 7:0 **CSICAL[7:0]**: CSI clock calibration

Set by hardware by option byte loading during system reset NRESET. Adjusted by software through trimming bits CSITRIM.

This field represents the sum of engineering option byte calibration value and CSITRIM bits value.

11.8.5 RCC clock configuration register1 (RCC_CFGR1)

Address offset: 0x01C

Reset value: 0x0000 0000

Access: $0 \leq \text{wait state} \leq 2$; word, half-word and byte access

One or two wait states are inserted only if the access occurs during clock source switch.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MCO2SEL[2:0]			MCO2PRE[3:0]				MCO1SEL[2:0]			MCO1PRE[3:0]				Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
TIMPR E	Res.	RTCPRE[5:0]						STOPK ERWU CK	STOP WUCK	Res.	SWS[1:0]		Res.	SW[1:0]	
rw		rw	rw	rw	rw	rw	rw	rw	rw		r	r		rw	rw

Bits 31:29 **MCO2SEL[2:0]**: microcontroller clock output 2

Set and cleared by software. Clock source selection may generate glitches on MCO2.

It is highly recommended to configure these bits only after reset, before enabling the external oscillators and the PLLs.

000: system clock selected (**sys_ck**) (default after reset)

001: PLL2 oscillator clock selected (**pll2_p_ck**)

010: HSE clock selected (**hse_ck**)

011: PLL1 clock selected (**pll1_p_ck**)

100: CSI clock selected (**csi_ck**)

101: LSI clock selected (**lsi_ck**)

Others: reserved

Bits 28:25 **MCO2PRE[3:0]**: MCO2 prescaler

Set and cleared by software to configure the prescaler of the MCO2. Modification of this prescaler may generate glitches on MCO2. It is highly recommended to change this prescaler only after reset, before enabling the external oscillators and the PLLs.

0000: prescaler disabled (default after reset)

0001: division by 1 (bypass)

0010: division by 2

0011: division by 3

0100: division by 4

...

1111: division by 15

Bits 24:22 **MCO1SEL[2:0]**: Microcontroller clock output 1

Set and cleared by software. Clock source selection may generate glitches on MCO1.

It is highly recommended to configure these bits only after reset, before enabling the external oscillators and the PLLs.

000: HSI clock selected (**hsi_ck**) (default after reset)

001: LSE oscillator clock selected (**lse_ck**)

010: HSE clock selected (**hse_ck**)

011: PLL1 clock selected (**pll1_q_ck**)

100: HSI48 clock selected (**hsi48_ck**)

Others: reserved

Bits 21:18 **MCO1PRE[3:0]**: MCO1 prescaler

Set and cleared by software to configure the prescaler of the MCO1. Modification of this prescaler may generate glitches on MCO1. It is highly recommended to change this prescaler only after reset, before enabling the external oscillators and the PLLs.

0000: prescaler disabled (default after reset)

0001: division by 1 (bypass)

0010: division by 2

0011: division by 3

0100: division by 4

...

1111: division by 15

Bits 17:16 Reserved, must be kept at reset value.

Bit 15 **TIMPRE**: timers clocks prescaler selection

This bit is set and reset by software to control the clock frequency of all the timers connected to APB1 and APB2 domains.

0: The timers kernel clock is equal to **rcc_hclk1** if PPRE1 or PPRE2 corresponds to a division by 1 or 2, else it is equal to $2 \times F_{\text{rcc_pclk1}}$ or $2 \times F_{\text{rcc_pclk2}}$ (default after reset)

1: The timers kernel clock is equal to $2 \times F_{\text{rcc_pclk1}}$ or $2 \times F_{\text{rcc_pclk2}}$ if PPRE1 or PPRE2 corresponds to a division by 1, 2 or 4, else it is equal to $4 \times F_{\text{rcc_pclk1}}$ or $4 \times F_{\text{rcc_pclk2}}$

Bit 14 Reserved, must be kept at reset value.

Bits 13:8 **RTCPRE[5:0]**: HSE division factor for RTC clock

Set and cleared by software to divide the HSE to generate a clock for RTC.

Caution: The software must set these bits correctly to ensure that the clock supplied to the RTC is lower than 1 MHz. These bits must be configured if needed before selecting the RTC clock source.

000000: no clock (default after reset)

000001: no clock

000010: HSE/2

000011: HSE/3

000100: HSE/4

...

111110: HSE/62

111111: HSE/63

Bit 7 **STOPKERWUCK**: kernel clock selection after a wakeup from system Stop

Set and reset by software to select the kernel wakeup clock from system Stop.

0: HSI selected as wakeup clock from system Stop (default after reset)

1: CSI selected as wakeup clock from system Stop

Bit 6 **STOPWUCK**: system clock selection after a wakeup from system Stop

Set and reset by software to select the system wakeup clock from system Stop.

The selected clock is also used as emergency clock for the clock security system (CSS) on HSE. 0: HSI selected as wakeup clock from system Stop (default after reset)

1: CSI selected as wakeup clock from system Stop

Caution: STOPWUCK must not be modified when CSS is enabled (by HSECSSON bit) and the system clock is HSE (SWS = 10) or a switch on HSE is requested (SW = 10).

Bit 5 Reserved, must be kept at reset value.

Bits 4:3 **SWS[1:0]**: system clock switch status

Set and reset by hardware to indicate which clock source is used as system clock. 000: HSI used as system clock (**hsi_ck**) (default after reset).

01: CSI used as system clock (**csi_ck**)

10: HSE used as system clock (**hse_ck**)

11: PLL1 used as system clock (**pll1_p_ck**)

Others: reserved

Bit 2 Reserved, must be kept at reset value.

Bits 1:0 **SW[1:0]**: system clock and trace clock switch

Set and reset by software to select system clock and trace clock sources (**sys_ck**).

Set by hardware to:

-force the selection of the HSI or CSI (depending on STOPWUCK selection) when leaving a system Stop mode

-force the selection of the HSI in case of failure of the HSE when used directly or indirectly as system clock

00: HSI selected as system clock (**hsi_ck**) (default after reset)

01: CSI selected as system clock (**csi_ck**)

10: HSE selected as system clock (**hse_ck**)

11: PLL1 selected as system clock (**pll1_p_ck** for **sys_ck**)

Others: reserved

11.8.6 RCC CPU domain clock configuration register 2 (RCC_CFGR2)

Address offset: 0x020

Reset value: 0x0000 0000

1 or 2 wait states are inserted only if the access occurs during clock source switch.

From 0 to 15 wait states are inserted if the access occurs when the APB or AHB prescalers values update is ongoing.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	APB3D IS	APB2D IS	APB1D IS	AHB4D IS	Res.	AHB2D IS	AHB1D IS
									rw	rw	rw	rw		rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	PPRE3[2:0]			Res.	PPRE2[2:0]			Res.	PPRE1[2:0]			HPRE[3:0]			
	rw	rw	rw		rw	rw	rw		rw	rw	rw	rw	rw	rw	rw

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **APB3DIS**: APB3 clock disable value. Set and cleared by software

This bit can be set to further reduce power consumption, when none of the APB3 peripherals are used and when their clocks are disabled in RCC_APB3ENR. When this bit is set, all the APB3 peripherals clocks are off.

0: APB3 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: APB3 clock disabled

Bit 21 **APB2DIS**: APB2 clock disable value

This bit can be set to further reduce power consumption, when none of the APB2 peripherals are used and when their clocks are disabled in RCC_APB2ENR. When this bit is set, all the APB2 peripherals clocks are off.

0: APB2 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: APB2 clock disabled

Bit 20 **APB1DIS**: APB1 clock disable value

This bit can be set to further reduce power consumption, when none of the APB1 peripherals (except IWDG) are used and when their clocks are disabled in RCC_APB1ENR. When this bit is set, all the APB1 peripherals clocks are off, except for IWDG.

0: APB1 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: APB1 clock disabled

Bit 19 **AHB4DIS**: AHB4 clock disable

This bit can be set to further reduce power consumption, when none of the AHB4 peripherals from RCC_AHB4ENR are used and when their clocks are disabled in RCC_AHB4ENR. When this bit is set, all the AHB4 peripherals clocks from RCC_AHB4ENR are off.

0: AHB4 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: AHB4 clock disabled

Bit 18 Reserved, must be kept at reset value.

Bit 17 **AHB2DIS**: AHB2 clock disable

This bit can be set to further reduce power consumption, when none of the AHB2 peripherals from RCC_AHB2ENR are used and when their clocks are disabled in RCC_AHB2ENR. When this bit is set, all the AHB2 peripherals clocks are off, except for SRAM2 and SRAM3.

0: AHB2 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: AHB2 clock disabled

Bit 16 **AHB1DIS**: AHB1 clock disable

This bit can be set to further reduce power consumption, when none of the AHB1 peripherals from RCC_AHB1ENR are used and when their clocks are disabled in RCC_AHB1ENR. When this bit is set, all the AHB1 peripherals clocks are off, except for FLASH, BKPSRAM, ICACHE, DCACHE1 and SRAM1.

0: AHB1 clock enabled, distributed to peripherals according to their dedicated clock enable control bits

1: AHB1 clock disabled

Bit 15 Reserved, must be kept at reset value.

Bits 14:12 **PPRE3[2:0]**: APB low-speed prescaler (APB3)

Set and reset by software to control APB low-speed clocks division factor.

The clocks are divided with the new prescaler factor from 1 to 16 APB cycles after PPRE3 write.

0xx: $rcc_pclk3 = rcc_hclk1$

100: $rcc_pclk3 = rcc_hclk1 / 2$

101: $rcc_pclk3 = rcc_hclk1 / 4$

110: $rcc_pclk3 = rcc_hclk1 / 8$

111: $rcc_pclk3 = rcc_hclk1 / 16$

Bit 11 Reserved, must be kept at reset value.

Bits 10:8 **PPRE2[2:0]**: APB high-speed prescaler (APB2)

Set and reset by software to control APB high-speed clocks division factor.

The clocks are divided with the new prescaler factor from 1 to 16 APB cycles after PPRE2 write.

0xx: $rcc_pclk2 = rcc_hclk1$

100: $rcc_pclk2 = rcc_hclk1 / 2$

101: $rcc_pclk2 = rcc_hclk1 / 4$

110: $rcc_pclk2 = rcc_hclk1 / 8$

111: $rcc_pclk2 = rcc_hclk1 / 16$

Bit 7 Reserved, must be kept at reset value.

Bits 6:4 **PPRE1[2:0]**: APB low-speed prescaler (APB1)

Set and reset by software to control the division factor of rcc_pclk1 .

The clock is divided by the new prescaler factor from 1 to 16 cycles of rcc_hclk after PPRE write.

0xx: $rcc_pclk1 = rcc_hclk1$ (default after reset)

100: $rcc_pclk1 = rcc_hclk1 / 2$

101: $rcc_pclk1 = rcc_hclk1 / 4$

110: $rcc_pclk1 = rcc_hclk1 / 8$

111: $rcc_pclk1 = rcc_hclk1 / 16$

Bits 3:0 **HPRE[3:0]**: AHB prescaler

Set and reset by software to control the division factor of rcc_hclk . Changing this division ratio has an impact on the frequency of all bus matrix clocks

0xxx: $rcc_hclk = sys_ck$ (default after reset)

1000: $rcc_hclk = sys_ck / 2$

1001: $rcc_hclk = sys_ck / 4$

1010: $rcc_hclk = sys_ck / 8$

1011: $rcc_hclk = sys_ck / 16$

1100: $rcc_hclk = sys_ck / 64$

1101: $rcc_hclk = sys_ck / 128$

1110: $rcc_hclk = sys_ck / 256$

1111: $rcc_hclk = sys_ck / 512$

Caution: Be careful when using the voltage scaling. Due to the propagation delay of the new division factor, after a prescaler factor change, and before lowering the V_{CORE} voltage, this register must be read to check that the new prescaler value has been taken into account.

Depending upon the clock source frequency and the voltage range, the software application must program a correct value in HPRE to make sure that the system frequency does not exceed the maximum frequency.

11.8.7 RCC PLL clock source selection register (RCC_PLL1CFGR)

Address offset: 0x028

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL1REN	PLL1QEN	PLL1PEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	PLL1M[5:0]						Res.	Res.	PLL1VCOSEL	PLL1FRACEN	PLL1RGE[1:0]		PLL1SRC[1:0]	
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bit 18 **PLL1REN**: PLL1 DIVR divider output enable

Set and reset by software to enable the pll1_r_ck output of the PLL1.

To save power, DIVR1EN and DIVR1 bits must be set to 0 when the pll1_r_ck is not used.

This bit can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

0: pll1_r_ck output disabled (default after reset)

1: pll1_r_ck output enabled

Bit 17 **PLL1QEN**: PLL1 DIVQ divider output enable

Set and reset by software to enable the pll1_q_ck output of the PLL1.

To save power, when the pll1_q_ck output of the PLL1 is not used, the pll1_q_ck must be disabled.

This bit can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

0: pll1_q_ck output disabled (default after reset)

1: pll1_q_ck output enabled

Bit 16 **PLL1PEN**: PLL1 DIVP divider output enable

Set and reset by software to enable the pll1_p_ck output of the PLL1.

This bit can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

To save power, when the pll1_p_ck output of the PLL1 is not used, the pll1_p_ck must be disabled.

0: pll1_p_ck output disabled (default after reset)

1: pll1_p_ck output enabled

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **PLL1M[5:0]**: prescaler for PLL1

Set and cleared by software to configure the prescaler of the PLL1.

The hardware does not allow any modification of this prescaler when PLL1 is enabled (PLL1ON = 1 or PLL1RDY = 1).

To save power when PLL1 is not used, the value of PLL1M must be set to 0.

000000: prescaler disabled (default after reset)

000001: division by 1 (bypass)

000010: division by 2

000011: division by 3

...

100000: division by 32

...

111111: division by 63

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **PLL1VCOSEL**: PLL1 VCO selection

Set and reset by software to select the proper VCO frequency range used for PLL1. This bit must be written before enabling the PLL1.

0: wide VCO range: 128 to 560 MHz (default after reset)

1: medium VCO range: 150 to 420 MHz

Bit 4 **PLL1FRACEN**: PLL1 fractional latch enable

Set and reset by software to latch the content of FRACN1 into the sigma-delta modulator.

To latch the FRACN1 value into the sigma-delta modulator, PLL1FRACEN must be set to 0, then set to 1. The transition 0 to 1 transfers the content of FRACN1 into the modulator.

Bits 3:2 **PLL1RGE[1:0]**: PLL1 input frequency range

Set and reset by software to select the proper reference frequency range used for PLL1. This bit must be written before enabling the PLL1.

00: PLL1 input (ref1_ck) clock range frequency between 1 and 2 MHz (default after reset)

01: PLL1 input (ref1_ck) clock range frequency between 2 and 4 MHz

10: PLL1 input (ref1_ck) clock range frequency between 4 and 8 MHz

11: PLL1 input (ref1_ck) clock range frequency between 8 and 16 MHz

Bits 1:0 **PLL1SRC[1:0]**: PLL1M and PLLs clock source selection

Set and reset by software to select the PLL clock source. These bits can be written only when all PLLs are disabled.

To save power, when no PLL is used, the value of PLL1SRC must be set to '00'.

00: no clock send to PLL1M divider and PLLs (default after reset).

01: HSI selected as PLL clock (hsi_ck)

10: CSI selected as PLL clock (csi_ck)

11: HSE selected as PLL clock (hse_ck)

11.8.8 RCC PLL clock source selection register (RCC_PLL2CFGR)

Address offset: 0x02C

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL2REN	PLL2QEN	PLL2PEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	PLL2M[5:0]						Res.	Res.	PLL2VCOSEL	PLL2FRACEN	PLL2RGE[1:0]		PLL2SRC[1:0]	
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

Bit 18 **PLL2REN**: PLL2 DIVR divider output enable

Set and reset by software to enable the pll2_r_ck output of the PLL2.

To save power, DIVR2EN and DIVR2 bits must be set to 0 when the pll2_r_ck is not used.

0: pll2_r_ck output disabled (default after reset)

1: pll2_r_ck output enabled

Bit 17 **PLL2QEN**: PLL2 DIVQ divider output enable

Set and reset by software to enable the pll2_q_ck output of the PLL2.

To save power, when the pll2_q_ck output of the PLL2 is not used, the pll2_q_ck must be disabled.

0: pll2_q_ck output disabled (default after reset)

1: pll2_q_ck output enabled

Bit 16 **PLL2PEN**: PLL2 DIVP divider output enable

Set and reset by software to enable the pll2_p_ck output of the PLL2.

To save power, when the pll2_p_ck output of the PLL2 is not used, the pll2_p_ck must be disabled.

0: pll2_p_ck output disabled (default after reset)

1: pll2_p_ck output enabled

Bits 15:14 Reserved, must be kept at reset value.

Bits 13:8 **PLL2M[5:0]**: prescaler for PLL2

Set and cleared by software to configure the prescaler of the PLL2.

The hardware does not allow any modification of this prescaler when PLL2 is enabled (PLL2ON = 1 or PLL2RDY = 1).

To save power when PLL2 is not used, the value of PLL2M must be set to 0.

000000: prescaler disabled (default after reset)

000001: division by 1 (bypass)

000010: division by 2

000011: division by 3

...

100000: division by 32

...

111111: division by 63

Bits 7:6 Reserved, must be kept at reset value.

Bit 5 **PLL2VCOSEL**: PLL2 VCO selection

Set and reset by software to select the proper VCO frequency range used for PLL2.

This bit must be written before enabling the PLL2.

0: wide VCO range: 128 to 560 MHz (default after reset)

1: medium VCO range 150 to 420 MHz

Bit 4 **PLL2FRACEN**: PLL2 fractional latch enable

Set and reset by software to enable the pll2_p_ck output of the PLL2.

To save power, when the pll2_p_ck output of the PLL2 is not used, the pll2_p_ck must be disabled.

0: pll2_p_ck output disabled (default after reset)

1: pll2_p_ck output enabled

Bits 3:2 **PLL2RGE[1:0]**: PLL2 input frequency range

Set and reset by software to select the proper reference frequency range used for PLL2.

These bits must be written before enabling the PLL2.

00: PLL2 input (ref2_ck) clock range frequency between 1 and 2 MHz (default after reset)

01: PLL2 input (ref2_ck) clock range frequency between 2 and 4 MHz

10: PLL2 input (ref2_ck) clock range frequency between 4 and 8 MHz

11: PLL2 input (ref2_ck) clock range frequency between 8 and 16 MHz

Bits 1:0 **PLL2SRC[1:0]**: PLL2M and PLLs clock source selection

Set and reset by software to select the PLL clock source.

These bits can be written only when all PLLs are disabled.

To save power, when no PLL is used, the value of PLL2SRC must be set to '00'.

00: no clock send to PLL2M divider and PLLs (default after reset)

01: HSI selected as PLL clock (hsi_ck)

10: CSI selected as PLL clock (csi_ck)

11: HSE selected as PLL clock (hse_ck)

11.8.9 RCC PLL clock source selection register (RCC_PLL3CFGR)

Address offset: 0x030

Reset value: 0x0000 0000

Access: no wait state, word, half-word and byte access

Note: The **RCC_PLL3CFGR** register is not available on **STM32H543/553xx** devices

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL3REN	PLL3QEN	PLL3PEN
													rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	PLL3M[5:0]						Res.	Res.	PLL3VCOSEL	PLL3FRACEN	PLL3RGE[1:0]		PLL3SRC[1:0]	
		rw	rw	rw	rw	rw	rw			rw	rw	rw	rw	rw	rw

Bits 31:19 Reserved, must be kept at reset value.

- Bit 18 **PLL3REN**: PLL3 DIVR divider output enable
 Set and reset by software to enable the pll3_r_ck output of the PLL3.
 To save power, DIVR2EN and DIVR2 bits must be set to 0 when the pll3_r_ck is not used.
 0: pll3_r_ck output disabled (default after reset)
 1: pll3_r_ck output enabled
- Bit 17 **PLL3QEN**: PLL3 DIVQ divider output enable
 Set and reset by software to enable the pll3_q_ck output of the PLL3.
 To save power, when the pll3_q_ck output of the PLL3 is not used, the pll3_q_ck must be disabled.
 0: pll3_q_ck output disabled (default after reset)
 1: pll3_q_ck output enabled
- Bit 16 **PLL3PEN**: PLL3 DIVP divider output enable
 Set and reset by software to enable the pll3_p_ck output of the PLL3.
 To save power, when the pll3_p_ck output of the PLL3 is not used, the pll3_p_ck must be disabled.
 0: pll3_p_ck output disabled (default after reset)
 1: pll3_p_ck output enabled
- Bits 15:14 Reserved, must be kept at reset value.
- Bits 13:8 **PLL3M[5:0]**: prescaler for PLL3
 Set and cleared by software to configure the prescaler of the PLL3.
 The hardware does not allow any modification of this prescaler when PLL3 is enabled (PLL3ON = 1 or PLL3RDY = 1).
 To save power when PLL3 is not used, the value of PLL3M must be set to 0.
 000000: prescaler disabled (default after reset)
 000001: division by 1 (bypass)
 000010: division by 2
 000011: division by 3
 ...
 100000: division by 32
 ...
 111111: division by 63
- Bits 7:6 Reserved, must be kept at reset value.
- Bit 5 **PLL3VCOSEL**: PLL3 VCO selection
 Set and reset by software to select the proper VCO frequency range used for PLL3.
 This bit must be written before enabling the PLL3.
 0: wide VCO range 128 to 560 MHz (default after reset)
 1: medium VCO range 150 to 420 MHz
- Bit 4 **PLL3FRACEN**: PLL3 fractional latch enable
 Set and reset by software to latch the content of FRACN3 into the sigma-delta modulator.
 To latch the FRACN3 value into the sigma-delta modulator, PLL3FRACEN must be set to 0, then set to 1. The transition 0 to 1 transfers the content of FRACN3 into the modulator.
- Bits 3:2 **PLL3RGE[1:0]**: PLL3 input frequency range
 Set and reset by software to select the proper reference frequency range used for PLL3.
 This bit must be written before enabling the PLL3.
 00: PLL3 input (ref3_ck) clock range frequency between 1 and 2 MHz (default after reset)
 01: PLL3 input (ref3_ck) clock range frequency between 2 and 4 MHz
 10: PLL3 input (ref3_ck) clock range frequency between 4 and 8 MHz
 11: PLL3 input (ref3_ck) clock range frequency between 8 and 16 MHz

Bits 1:0 **PLL3SRC[1:0]**: PLL3M and PLLs clock source selection

Set and reset by software to select the PLL clock source. These bits can be written only when all PLLs are disabled.

To save power, when no PLL is used, the value of PLL3SRC must be set to '00'.

00: no clock send to PLL3M divider and PLLs (default after reset)

01: HSI selected as PLL clock (hsi_ck)

10: CSI selected as PLL clock (csi_ck)

11: HSE selected as PLL clock (hse_ck)

11.8.10 RCC PLL1 dividers register (RCC_PLL1DIVR)

Address offset: 0x034

Reset value: 0x0101 0280

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	PLL1R[6:0]							Res.	PLL1Q[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL1P[6:0]								PLL1N[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **PLL1R[6:0]**: PLL1 DIVR division factor

Set and reset by software to control the frequency of the pll1_r_ck clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

0000000: pll1_r_ck = vco1_ck / 1

0000001: pll1_r_ck = vco1_ck / 2 (default after reset)

0000010: pll1_r_ck = vco1_ck / 3

0000011: pll1_r_ck = vco1_ck / 4

...

1111111: pll1_r_ck = vco1_ck / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **PLL1Q[6:0]**: PLL1 DIVQ division factor

Set and reset by software to control the frequency of the pll1_q_ck clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

0000000: pll1_q_ck = vco1_ck

0000001: pll1_q_ck = vco1_ck / 2 (default after reset)

0000010: pll1_q_ck = vco1_ck / 3

0000011: pll1_q_ck = vco1_ck / 4

...

1111111: pll1_q_ck = vco1_ck / 128

Bits 15:9 **PLL1P[6:0]**: PLL1 DIVP division factor

Set and reset by software to control the frequency of the pll1_p_ck clock.

These bits can be written only when the PLL1 is disabled (PLL1ON = 0 and PLL1RDY = 0).

Note that odd division factors are not allowed.

0000000: Not allowed

0000001: pll1_p_ck = vco1_ck / 2 (default after reset)

0000010: Not allowed

0000011: pll1_p_ck = vco1_ck / 4

...

1111111: pll1_p_ck = vco1_ck / 128

Bits 8:0 **PLL1N[8:0]**: Multiplication factor for PLL1VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL1ON = 0 and PLL1RDY = 0).

0x003: PLL1N = 4

0x004: PLL1N = 5

0x005: PLL1N = 6

...

0x080: PLL1N = 129 (default after reset)

...

0x1FF: PLL1N = 512

Others: reserved

11.8.11 RCC PLL1 fractional divider register (RCC_PLL1FRACR)

Address offset: 0x038

Reset value: 0x0000 0000

Access: no wait state; word and half-word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL1FRACN[12:0]													Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **PLL1FRACN[12:0]**: fractional part of the multiplication factor for PLL1 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO.
These bits can be written at any time, allowing dynamic fine-tuning of the PLL1 VCO.

Caution: The software must set correctly these bits to insure that the VCO output frequency is in the valid frequency range, that is:

* 128 to 560 MHz if PLL1VCOSEL = 0

* 150 to 420 MHz if PLL1VCOSEL = 1

VCO output frequency = $F_{\text{ref1_ck}} \times (\text{PLL1N} + (\text{PLL1FRACN} / 2^{13}))$, with

* PLL1N between 8 and 420

* PLL1FRACN between 0 and $2^{13} - 1$

* The input frequency $F_{\text{ref1_ck}}$ must be between 1 and 16 MHz.

To change the PLL1FRACN value on-the-fly even if the PLL is enabled, the application must proceed as follows:

* Set the bit PLL1FRACEN to 0

* Write the new fractional value into PLL1FRACN

* Set the bit PLL1FRACEN to 1

Bits 2:0 Reserved, must be kept at reset value.

11.8.12 RCC PLL2 dividers register (RCC_PLL2DIVR)

Address offset: 0x03C

Reset value: 0x0101 0280

Access: no wait state, word, half-word and byte access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	PLL2R[6:0]							Res.	PLL2Q[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL2P[6:0]								PLL2N[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **PLL2R[6:0]**: PLL2 DIVR division factor

Set and reset by software to control the frequency of the pll2_r_ck clock.

These bits can be written only when the PLL1 is disabled (PLL2ON = 0 and PLL2RDY = 0).

0000000: pll2_r_ck = vco2_ck

0000001: pll2_r_ck = vco2_ck / 2 (default after reset)

0000010: pll2_r_ck = vco2_ck / 3

0000011: pll2_r_ck = vco2_ck / 4

...

1111111: pll2_r_ck = vco2_ck / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **PLL2Q[6:0]**: PLL2 DIVQ division factor

Set and reset by software to control the frequency of the pll2_q_ck clock.

These bits can be written only when the PLL2 is disabled (PLL2ON = 0 and PLL2RDY = 0).

0000000: pll2_q_ck = vco2_ck

0000001: pll2_q_ck = vco2_ck / 2 (default after reset)

0000010: pll2_q_ck = vco2_ck / 3

0000011: pll2_q_ck = vco2_ck / 4

...

1111111: pll2_q_ck = vco2_ck / 128

Bits 15:9 **PLL2P[6:0]**: PLL2 DIVP division factor

Set and reset by software to control the frequency of the pll2_p_ck clock.

These bits can be written only when the PLL2 is disabled (PLL2ON = 0 and PLL2RDY = 0).

0000000: pll2_p_ck = vco2_ck

0000001: pll2_p_ck = vco2_ck / 2 (default after reset)

0000010: pll2_p_ck = vco2_ck / 3

0000011: pll2_p_ck = vco2_ck / 4

...

1111111: pll2_p_ck = vco2_ck / 128

Bits 8:0 **PLL2N[8:0]**: Multiplication factor for PLL2VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL2ON = 0 and PLL2RDY = 0).

0x003: PLL2N = 4

0x004: PLL2N = 5

0x005: PLL2N = 6

...

0x080: PLL2N = 129 (default after reset)

...

0x1FF: PLL2N = 512

Others: reserved

11.8.13 RCC PLL2 fractional divider register (RCC_PLL2FRACR)

Address offset: 0x040

Reset value: 0x0000 0000

Access: no wait state; word and half-word access

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL2FRACN[12:0]													Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **PLL2FRACN[12:0]**: fractional part of the multiplication factor for PLL2 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO. These bits can be written at any time, allowing dynamic fine-tuning of the PLL2 VCO.

Caution: The software must set correctly these bits to insure that the VCO output frequency is between its valid frequency range, that is:

* 128 to 560 MHz if PLL2VCOSEL = 0

* 150 to 420 MHz if PLL2VCOSEL = 1

VCO output frequency = Fref2_ck x (PLL2N + (PLL2FRACN / 2¹³)), with

* PLL2N between 8 and 420

* PLL2FRACN can be between 0 and 2¹³⁻¹

* The input frequency Fref2_ck must be between 1 and 16 MHz.

To change the PLL2FRACN value on-the-fly even if the PLL is enabled, the application must proceed as follows:

* Set the bit PLL2FRACEN to 0

* Write the new fractional value into PLL2FRACN

* Set the bit PLL2FRACEN to 1

Bits 2:0 Reserved, must be kept at reset value.

11.8.14 RCC PLL3 dividers register (RCC_PLL3DIVR)

Address offset: 0x044

Reset value: 0x0101 0280

Access: no wait state; word, half-word and byte access

Note: The RCC_PLL3DIVR register is not available on STM32H543/553xx devices

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	PLL3R[6:0]							Res.	PLL3Q[6:0]						
	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL3P[6:0]								PLL3N[8:0]							
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:24 **PLL3R[6:0]**: PLL3 DIVR division factor

Set and reset by software to control the frequency of the **pll3_r_ck** clock.

These bits can be written only when the PLL1 is disabled (PLL3ON = 0 and PLL3RDY = 0).

0000000: **pll3_r_ck** = **vco3_ck**

0000001: **pll3_r_ck** = **vco3_ck** / 2 (default after reset)

0000010: **pll3_r_ck** = **vco3_ck** / 3

0000011: **pll3_r_ck** = **vco3_ck** / 4

...

1111111: **pll3_r_ck** = **vco3_ck** / 128

Bit 23 Reserved, must be kept at reset value.

Bits 22:16 **PLL3Q[6:0]**: PLL3 DIVQ division factor

Set and reset by software to control the frequency of the pll3_q_ck clock.

These bits can be written only when the PLL3 is disabled (PLL3ON = 0 and PLL3RDY = 0).

0000000: pll3_q_ck = vco3_ck

0000001: pll3_q_ck = vco3_ck / 2 (default after reset)

0000010: pll3_q_ck = vco3_ck / 3

0000011: pll3_q_ck = vco3_ck / 4

...

1111111: pll3_q_ck = vco3_ck / 128

Bits 15:9 **PLL3P[6:0]**: PLL3 DIVP division factor

Set and reset by software to control the frequency of the pll3_p_ck clock.

These bits can be written only when the PLL3 is disabled (PLL3ON = 0 and PLL3RDY = 0).

0000000: pll3_p_ck = vco3_ck

0000001: pll3_p_ck = vco3_ck / 2 (default after reset)

0000010: pll3_p_ck = vco3_ck / 3

0000011: pll3_p_ck = vco3_ck / 4

...

1111111: pll3_p_ck = vco3_ck / 128

Bits 8:0 **PLL3N[8:0]**: Multiplication factor for PLL3VCO

Set and reset by software to control the multiplication factor of the VCO.

These bits can be written only when the PLL is disabled (PLL3ON = 0 and PLL3RDY = 0).

0x003: PLL3N = 4

0x004: PLL3N = 5

0x005: PLL3N = 6

...

0x080: PLL3N = 129 (default after reset)

...

0x1FF: PLL3N = 512

Others: reserved

11.8.15 RCC PLL3 fractional divider register (RCC_PLL3FRACR)

Address offset: 0x048

Reset value: 0x0000 0000

Access: no wait state; word and half-word access

Note: The RCC_PLL3FRACR register is not available on STM32H543/553xx devices

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PLL3FRACN[12:0]													Res.	Res.	Res.
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw			

Bits 31:16 Reserved, must be kept at reset value.

Bits 15:3 **PLL3FRACN[12:0]**: fractional part of the multiplication factor for PLL3 VCO

Set and reset by software to control the fractional part of the multiplication factor of the VCO. These bits can be written at any time, allowing dynamic fine-tuning of the PLL3 VCO.

Caution: The software must set correctly these bits to insure that the VCO output frequency is between its valid frequency range, that is:

* 128 to 560 MHz if PLL3VCOSEL = 0

* 150 to 420 MHz if PLL3VCOSEL = 1

VCO output frequency = Fref3_ck x (PLL3N + (PLL3FRACN / 213)), with

* PLL3N between 8 and 420

* PLL3FRACN can be between 0 and 213 - 1

* The input frequency Fref3_ck must be between 1 and 16 MHz.

To change the PLL3FRACN value on-the-fly even if the PLL is enabled, the application must proceed as follows:

* Set the bit PLL3FRACEN to 0

* Write the new fractional value into PLL3FRACN

* Set the bit PLL3FRACEN to 1

Bits 2:0 Reserved, must be kept at reset value.

11.8.16 RCC clock source interrupt enable register (RCC_CIER)

Address offset: 0x050

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL3R DYIE	PLL2R DYIE	PLL1R DYIE	HSI48R DYIE	HSERD YIE	HSIRD YIE	CSIRD YIE	LSERD YIE	LSIRD YIE
							rW	rW	rW	rW	rW	rW	rW	rW	rW

Bits 31:9 Reserved, must be kept at reset value.

Bit 8 **PLL3RDYIE**: PLL3 ready interrupt enable

Set and reset by software to enable/disable interrupt caused by PLL3 lock.

0: PLL3 lock interrupt disabled (default after reset)

1: PLL3 lock interrupt enabled

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **PLL2RDYIE**: PLL2 ready interrupt enable

Set and reset by software to enable/disable interrupt caused by PLL2 lock.

0: PLL2 lock interrupt disabled (default after reset)

1: PLL2 lock interrupt enabled

Bit 6 **PLL1RDYIE**: PLL1 ready interrupt enable

Set and reset by software to enable/disable interrupt caused by PLL1 lock.

0: PLL1 lock interrupt disabled (default after reset)

1: PLL1 lock interrupt enabled

Bit 5 HSI48RDYIE: HSI48 ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the HSI48 oscillator stabilization.

0: HSI48 ready interrupt disabled (default after reset)

1: HSI48 ready interrupt enabled

Bit 4 HSERDYIE: HSE ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the HSE oscillator stabilization.

0: HSE ready interrupt disabled (default after reset)

1: HSE ready interrupt enabled

Bit 3 HSIRDYIE: HSI ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the HSI oscillator stabilization.

0: HSI ready interrupt disabled (default after reset)

1: HSI ready interrupt enabled

Bit 2 CSIRDYIE: CSI ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the CSI oscillator stabilization.

0: CSI ready interrupt disabled (default after reset)

1: CSI ready interrupt enabled

Bit 1 LSERDYIE: LSE ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the LSE oscillator stabilization.

0: LSE ready interrupt disabled (default after reset)

1: LSE ready interrupt enabled

Bit 0 LSIRDYIE: LSI ready interrupt enable

Set and reset by software to enable/disable interrupt caused by the LSI oscillator stabilization.

0: LSI ready interrupt disabled (default after reset)

1: LSI ready interrupt enabled

11.8.17 RCC clock source interrupt flag register (RCC_CIFR)

Address offset: 0x054

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	HSECSSF	Res.	PLL3RDYF	PLL2RDYF	PLL1RDYF	HSI48RDYF	HSERDYF	HSIRDYF	CSIRDYF	LSERDYF	LSIRDYF
					r		r	r	r	r	r	r	r	r	r

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 HSECSSF: HSE clock security system interrupt flag

Reset by software by writing HSECSSC bit.

Set by hardware in case of HSE clock failure.

0: no clock security interrupt caused by HSE clock failure (default after reset)

1: clock security interrupt caused by HSE clock failure

Bit 9 Reserved, must be kept at reset value.

Bit 8 PLL3RDYF: PLL3 ready interrupt flag

Reset by software by writing PLL3RDYC bit.

Set by hardware when the PLL3 locks and PLL3RDYIE is set.

0: no clock ready interrupt caused by PLL3 lock (default after reset)

1: clock ready interrupt caused by PLL3 lock

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 PLL2RDYF: PLL2 ready interrupt flag

Reset by software by writing PLL2RDYC bit.

Set by hardware when the PLL2 locks and PLL2RDYIE is set.

0: no clock ready interrupt caused by PLL2 lock (default after reset)

1: clock ready interrupt caused by PLL2 lock

Bit 6 PLL1RDYF: PLL1 ready interrupt flag

Reset by software by writing PLL1RDYC bit.

Set by hardware when the PLL1 locks and PLL1RDYIE is set.

0: no clock ready interrupt caused by PLL1 lock (default after reset)

1: clock ready interrupt caused by PLL1 lock

Bit 5 HSI48RDYF: HSI48 ready interrupt flag

Reset by software by writing HSI48RDYC bit.

Set by hardware when the HSI48 clock becomes stable and HSI48RDYIE is set.

0: no clock ready interrupt caused by the HSI48 oscillator (default after reset)

1: clock ready interrupt caused by the HSI48 oscillator

Bit 4 HSERDYF: HSE ready interrupt flag

Reset by software by writing HSERDYC bit.

Set by hardware when the HSE clock becomes stable and HSERDYIE is set.

0: no clock ready interrupt caused by the HSE (default after reset)

1: clock ready interrupt caused by the HSE

Bit 3 HSIRDYF: HSI ready interrupt flag

Reset by software by writing HSIRDYC bit.

Set by hardware when the HSI clock becomes stable and HSIRDYIE is set.

0: no clock ready interrupt caused by the HSI (default after reset)

1: clock ready interrupt caused by the HSI

Bit 2 CSIRDYF: CSI ready interrupt flag

Reset by software by writing CSIRDYC bit.

Set by hardware when the CSI clock becomes stable and CSIRDYIE is set.

0: no clock ready interrupt caused by the CSI (default after reset)

1: clock ready interrupt caused by the CSI

Bit 1 LSERDYF: LSE ready interrupt flag

Reset by software by writing LSERDYC bit.

Set by hardware when the LSE clock becomes stable and LSERDYIE is set.

0: no clock ready interrupt caused by the LSE (default after reset)

1: clock ready interrupt caused by the LSE

Bit 0 LSIRDYF: LSI ready interrupt flag

Reset by software by writing LSIRDYC bit.

Set by hardware when the LSI clock becomes stable and LSIRDYIE is set.

0: no clock ready interrupt caused by the LSI (default after reset)

1: clock ready interrupt caused by the LSI

11.8.18 RCC clock source interrupt clear register (RCC_CICR)

Address offset: 0x058

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	HSECS SC	Res.	PLL3R DYC	PLL2R DYC	PLL1R DYC	HSI48R DYC	HSERD YC	HSIRD YC	CSIRD YC	LSERD YC	LSIRD YC
					rc_w1		rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Bits 31:11 Reserved, must be kept at reset value.

Bit 10 **HSECSSC**: HSE clock security system interrupt clear

Set by software to clear HSECSSF.

Reset by hardware when clear done.

0: HSECSSF no effect (default after reset)

1: HSECSSF cleared

Bit 9 Reserved, must be kept at reset value.

Bit 8 **PLL3RDYC**: PLL3 ready interrupt clear

Set by software to clear PLL3RDYF.

Reset by hardware when clear done.

0: PLL3RDYF no effect (default after reset)

1: PLL3RDYF cleared

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **PLL2RDYC**: PLL2 ready interrupt clear

Set by software to clear PLL2RDYF.

Reset by hardware when clear done.

0: PLL2RDYF no effect (default after reset)

1: PLL2RDYF cleared

Bit 6 **PLL1RDYC**: PLL1 ready interrupt clear

Set by software to clear PLL1RDYF.

Reset by hardware when clear done.

0: PLL1RDYF no effect (default after reset)

1: PLL1RDYF cleared

Bit 5 **HSI48RDYC**: HSI48 ready interrupt clear

Set by software to clear HSI48RDYF.

Reset by hardware when clear done.

0: HSI48RDYF no effect (default after reset)

1: HSI48RDYF cleared

Bit 4 **HSERDYC**: HSE ready interrupt clear

Set by software to clear HSERDYF.

Reset by hardware when clear done.

0: HSERDYF no effect (default after reset)

1: HSERDYF cleared

- Bit 3 **HSIRDYC**: HSI ready interrupt clear
 Set by software to clear HSIRDYF.
 Reset by hardware when clear done.
 0: HSIRDYF no effect (default after reset)
 1: HSIRDYF cleared
- Bit 2 **CSIRDYC**: CSI ready interrupt clear
 Set by software to clear CSIRDYF.
 Reset by hardware when clear done.
 0: CSIRDYF no effect (default after reset)
 1: CSIRDYF cleared
- Bit 1 **LSERDYC**: LSE ready interrupt clear
 Set by software to clear LSERDYF.
 Reset by hardware when clear done.
 0: LSERDYF no effect (default after reset)
 1: LSERDYF cleared
- Bit 0 **LSIRDYC**: LSI ready interrupt clear
 Set by software to clear LSIRDYF.
 Reset by hardware when clear done.
 0: LSIRDYF no effect (default after reset)
 1: LSIRDYF cleared

11.8.19 RCC AHB1 reset register (RCC_AHB1RSTR)

Address offset: 0x060

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETHRS T	Res.	RAMC FGRST	Res.
												rw		rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMAC RST	CORDI CRST	Res.	CRCR ST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPDM A2RST	GPDM A1RST
rw	rw		rw											rw	rw

Bits 31:20 Reserved, must be kept at reset value.

- Bit 19 **ETHRST**: ETHRST block reset
 Set and reset by software
 0: does not reset ETHRST block (default after reset)
 1: resets the ETHRST block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 18 Reserved, must be kept at reset value.

- Bit 17 **RAMCFGRST**: RAMCFG block reset
 Set and reset by software.
 0: does not reset RAMCFG block (default after reset)
 1: resets RAMCFG block

Bit 16 Reserved, must be kept at reset value.

Bit 15 **FMACRST**: FMAC block reset

Set and reset by software.

0: does not reset FMAC block (default after reset)

1: resets FMAC block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value..

Bit 14 **CORDICRST**: CORDIC block reset

Set and reset by software.

0: does not reset CORDIC block (default after reset)

1: resets CORDIC block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 Reserved, must be kept at reset value.

Bit 12 **CRCRST**: CRC block reset Set and reset by software.

0: does not reset CRC block (default after reset)

1: resets CRC block

Bits 11:2 Reserved, must be kept at reset value.

Bit 1 **GPDMA2RST**: GPDMA2 block reset

Set and reset by software.

0: does not reset GPDMA2 block (default after reset)

1: resets GPDMA2 block

Bit 0 **GPDMA1RST**: GPDMA1 block reset

Set and reset by software.

0: does not reset GPDMA1 block (default after reset)

1: resets GPDMA1 block

11.8.20 RCC AHB2 peripheral reset register (RCC_AHB2RSTR)

Address offset: 0x064

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	ADC3RST	Res.	Res.	CCBRST	SAESRST	PKARST	RNGRST	HASHRST	AESRST
							rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	DCMI_PSSIRST	DAC1RST	ADCRST	Res.	GPIOIRST	GPIOHRST	GPIOGRST	GPIOFRST	GPIOERST	GPIODRST	GPIOCRST	GPIOBRST	GPIOARST
			rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **ADC3RST**: ADC3 reset

Set and reset by software.

0: does not reset ADC3 (default after reset)

1: resets ADC3

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **CCBRST**: CCB reset

Set and reset by software.

0: does not reset CCB (default after reset)

1: resets CCB

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **SAESRST**: SAES block reset

Set and reset by software.

0: does not reset SAES block (default after reset)

1: resets SAES block

Bit 19 **PKARST**: PKA block reset

Set and reset by software.

0: does not reset PKA block (default after reset)

1: resets PKA block

Bit 18 **RNGRST**: RNG block reset

Set and reset by software.

0: does not reset RNG block (default after reset)

1: resets RNG block

Bit 17 **HASHRST**: HASH block reset

Set and reset by software.

0: does not reset HASH block (default after reset)

1: resets HASH block

Bit 16 **AESRST**: AES block reset

Set and reset by software.

0: does not reset AES block (default after reset)

1: resets AES block

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **DCMI_PSSI**RST: digital camera interface block reset (DCMI or PSSI depending which interface is active)

Set and reset by software.

0: does not reset the DCMI/PSSI block (default after reset)

1: resets the DCMI/PSSI block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

- Bit 11 **DAC1RST**: DAC block reset
Set and reset by software.
0: does not reset DAC block (default after reset)
1: resets DAC block
- Bit 10 **ADCRST**: ADC1 and 2 blocks reset
Set and reset by software.
0: does not reset ADC1 and 2 blocks (default after reset)
1: resets ADC1 and 2 blocks
- Bit 9 Reserved, must be kept at reset value.
- Bit 8 **GPIORST**: GPIOI block reset
Set and reset by software.
0: does not reset the GPIOI block (default after reset)
1: resets the GPIOI block
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.
- Bit 7 **GPIOHRST**: GPIOH block reset
Set and reset by software.
0: does not reset the GPIOH block (default after reset)
1: resets the GPIOH block
- Bit 6 **GPIOGRST**: GPIOG block reset
Set and reset by software.
0: does not reset the GPIOG block (default after reset)
1: resets the GPIOG block
- Bit 5 **GPIOFRST**: GPIOF block reset
Set and reset by software.
0: does not reset the GPIOF block (default after reset)
1: resets the GPIOF block
- Bit 4 **GPIOERST**: GPIOE block reset
Set and reset by software.
0: does not reset the GPIOE block (default after reset)
1: resets the GPIOE block
- Bit 3 **GPIODRST**: GPIOD block reset
Set and reset by software.
0: does not reset the GPIOD block (default after reset)
1: resets the GPIOD block
- Bit 2 **GPIOCRST**: GPIOC block reset
Set and reset by software.
0: does not reset the GPIOC block (default after reset)
1: resets the GPIOC block
- Bit 1 **GPIOBRST**: GPIOB block reset
Set and reset by software.
0: does not reset the GPIOB block (default after reset)
1: resets the GPIOB block

Bit 0 **GPIOARST**: GPIOA block reset
 Set and reset by software.
 0: does not reset the GPIOA block (default after reset)
 1: resets the GPIOA block

11.8.21 RCC AHB4 peripheral reset register (RCC_AHB4RSTR)

Address offset: 0x06C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OCTOSPI1RST	Res.	Res.	Res.	FMCRST
											rw				rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	SDMMC2RST	SDMMC1RST	Res.	Res.	Res.	OTFDEC1RST	Res.	Res.	Res.	Res.	Res.	Res.	Res.
			rw	rw				rw							

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **OCTOSPI1RST**: OCTOSPI1 block reset
 Set and reset by software.
 0: does not reset OCTOSPI1 block (default after reset)
 1: resets OCTOSPI1 block

Bits 19:17 Reserved, must be kept at reset value.

Bit 16 **FMCRST**: FMC block reset
 Set and reset by software.
 0: does not reset FMC block (default after reset)
 1: resets FMC block

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **SDMMC2RST**: SDMMC2 and SDMMC2 delay blocks reset
 Set and reset by software.
 0: does not reset SDMMC2 and SDMMC2 delay blocks (default after reset)
 1: resets SDMMC2 and SDMMC2 delay blocks

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 **SDMMC1RST**: SDMMC1 and SDMMC1 delay blocks reset
 Set and reset by software.
 0: does not reset SDMMC1 and SDMMC1 delay blocks (default after reset)
 1: resets SDMMC1 and SDMMC1 delay blocks

Bits 10:8 Reserved, must be kept at reset value.

Bit 7 **OTFDEC1RST**: OTFDEC1 block reset
 Set and reset by software.
 0: does not reset OTFDEC1 block (default after reset)
 1: resets OTFDEC1 block

Bits 6:0 Reserved, must be kept at reset value.

11.8.22 RCC APB1 peripheral low reset register (RCC_APB1LRSTR)

Address offset: 0x074

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8 RST	UART7 RST	Res.	CECRST	USART 11RST	USART 10RST	USART 6RST	CRSRST	I3C1RS T	I2C2RS T	I2C1RS T	UART5 RST	UART4 RST	USART 3RST	USART 2RST	COMP RST
rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3R ST	SPI2R ST	Res.	Res.	Res.	Res.	Res.	TIM14 RST	TIM13 RST	TIM12 RST	TIM7R ST	TIM6R ST	TIM5R ST	TIM4R ST	TIM3R ST	TIM2R ST
rw	rw						rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **UART8RST**: UART8 block reset

Set and reset by software.

0: does not reset the UART8 block (default after reset)

1: resets the UART8 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 30 **UART7RST**: UART7 block reset

Set and reset by software.

0: does not reset the UART7 block (default after reset)

1: resets the UART7 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 29 Reserved, must be kept at reset value.

Bit 28 **CECRST**: HDMI-CEC block reset

Set and reset by software.

0: does not reset the HDMI-CEC block (default after reset)

1: resets the HDMI-CEC block

Bit 27 **USART11RST**: USART11 block reset

Set and reset by software.

0: does not reset the USART11 block (default after reset)

1: resets the USART11 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 26 **USART10RST**: USART10 block reset

Set and reset by software.

0: does not reset the USART10 block (default after reset)

1: resets the USART10 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

- Bit 25 **USART6RST**: USART6 block reset
Set and reset by software.
0: does not reset the USART6 block (default after reset)
1: resets the USART6 block
- Bit 24 **CRSRST**: CRS block reset
Set and reset by software.
0: does not reset the CRS block (default after reset)
1: resets the CRS block
- Bit 23 **I3C1RST**: I3C1 block reset
Set and reset by software.
0: does not reset the I3C1 block (default after reset)
1: resets the I3C1 block
- Bit 22 **I2C2RST**: I2C2 block reset
Set and reset by software.
0: does not reset the I2C2 block (default after reset)
1: resets the I2C2 block
- Bit 21 **I2C1RST**: I2C1 block reset
Set and reset by software.
0: does not reset the I2C1 block (default after reset)
1: resets the I2C1 block
- Bit 20 **UART5RST**: UART5 block reset
Set and reset by software.
0: does not reset the UART5 block (default after reset)
1: resets the UART5 block
- Bit 19 **UART4RST**: UART4 block reset
Set and reset by software.
0: does not reset the UART4 block (default after reset)
1: resets the UART4 block
- Bit 18 **USART3RST**: USART3 block reset
Set and reset by software.
0: does not reset the USART3 block (default after reset)
1: resets the USART3 block
- Bit 17 **USART2RST**: USART2 block reset
Set and reset by software.
0: does not reset the USART2 block (default after reset)
1: resets the USART2 block
- Bit 16 **COMPRST**: COMP block reset
Set and reset by software.
0: does not reset the COMP block (default after reset)
1: resets the COMP block
- Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.*
- Bit 15 **SPI3RST**: SPI3 block reset
Set and reset by software.
0: does not reset the SPI3 block (default after reset)
1: resets the SPI3 block

Bit 14 **SPI2RST**: SPI2 block reset

Set and reset by software.

0: does not reset the SPI2 block (default after reset)

1: resets the SPI2 block

Bits 13:9 Reserved, must be kept at reset value.

Bit 8 **TIM14RST**: TIM14 block reset

Set and reset by software.

0: does not reset the TIM14 block (default after reset)

1: resets the TIM14 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **TIM13RST**: TIM13 block reset

Set and reset by software.

0: does not reset the TIM13 block (default after reset)

1: resets the TIM13 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 6 **TIM12RST**: TIM12 block reset

Set and reset by software.

0: does not reset the TIM12 block (default after reset)

1: resets the TIM12 block

Bit 5 **TIM7RST**: TIM7 block reset

Set and reset by software.

0: does not reset the TIM7 block (default after reset)

1: resets the TIM7 block

Bit 4 **TIM6RST**: TIM6 block reset

Set and reset by software.

0: does not reset the TIM6 block (default after reset)

1: resets the TIM6 block

Bit 3 **TIM5RST**: TIM5 block reset

Set and reset by software.

0: does not reset the TIM5 block (default after reset)

1: resets the TIM5 block

Bit 2 **TIM4RST**: TIM4 block reset

Set and reset by software.

0: does not reset the TIM4 block (default after reset)

1: resets the TIM4 block

Bit 1 **TIM3RST**: TIM3 block reset

Set and reset by software.

0: does not reset the TIM3 block (default after reset)

1: resets the TIM3 block

Bit 0 **TIM2RST**: TIM2 block reset

Set and reset by software.

0: does not reset the TIM2 block (default after reset)

1: resets the TIM2 block

11.8.23 RCC APB1 peripheral high reset register (RCC_APB1HRSTR)

Address offset: 0x078

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD1 RST	Res.	Res.	Res.	Res.	Res.	Res.	Res.
								rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	FDCAN RST	Res.	Res.	Res.	LPTIM2 RST	Res.	DTSRST T	Res.	UART1 2RST	UART9 RST
						rw				rw		rw		rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **UCPD1RST**: UCPD1 block reset

Set and reset by software.

0: does not reset the UCPD1 block (default after reset)

1: resets the UCPD1 block

Bits 22:10 Reserved, must be kept at reset value.

Bit 9 **FDCANRST**: FDCAN1 and FDCAN2 blocks reset

Set and reset by software.

0: does not reset the FDCAN1 and FDCAN2 blocks (default after reset)

1: resets the FDCAN1 and FDCAN2 blocks

Bits 8:6 Reserved, must be kept at reset value.

Bit 5 **LPTIM2RST**: LPTIM2 block reset

Set and reset by software.

0: does not reset the LPTIM2 block (default after reset)

1: resets the LPTIM2 block

Bit 4 Reserved, must be kept at reset value.

Bit 3 **DTSRST**: DTS block reset

Set and reset by software.

0: does not reset the DTS block (default after reset)

1: resets the DTS block

Bit 2 Reserved, must be kept at reset value.

Bit 1 **UART12RST**: UART12 block reset

Set and reset by software.

0: does not reset the UART12 block (default after reset)

1: resets the UART12 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 0 **UART9RST**: UART9 block reset

Set and reset by software.

0: does not reset UART9 block (default after reset)

1: resets UART9 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

11.8.24 RCC APB2 peripheral reset register (RCC_APB2RSTR)

Address offset: 0x07C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBRST	Res.	SAI2RST	SAI1RST	SPI6RST	SPI4RST	TIM17RST	TIM16RST	TIM15RST
							rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART1RST	TIM8RST	SPI1RST	TIM1RST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw	rw	rw	rw											

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **USBRST**: USB block reset

Set and reset by software.

0: does not reset the USB block (default after reset)

1: resets the USB block

Bit 23 Reserved, must be kept at reset value.

Bit 22 **SAI2RST**: SAI2 block reset

Set and reset by software.

0: does not reset the SAI2 block (default after reset)

1: resets the SAI2 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 21 **SAI1RST**: SAI1 block reset

Set and reset by software.

0: does not reset the SAI1 (default after reset)

1: resets the SAI1

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **SPI6RST**: SPI6 block reset

Set and reset by software.

0: does not reset the SPI6 block (default after reset)

1: resets the SPI6 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 19 **SPI4RST**: SPI4 block reset

Set and reset by software.

0: does not reset the SPI4 block (default after reset)

1: resets the SPI4 block

Bit 18 **TIM17RST**: TIM17 block reset

Set and reset by software.

0: does not reset the TIM17 block (default after reset)

1: resets the TIM17 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 17 **TIM16RST**: TIM16 block reset

Set and reset by software.

0: does not reset the TIM16 block (default after reset)

1: resets the TIM16 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 16 **TIM15RST**: TIM15 block reset

Set and reset by software.

0: does not reset the TIM15 block (default after reset)

1: resets the TIM15 block

Bit 15 Reserved, must be kept at reset value.

Bit 14 **USART1RST**: USART1 block reset

Set and reset by software.

0: does not reset the USART1 block (default after reset)

1: resets the USART1 block

Bit 13 **TIM8RST**: TIM8 block reset

Set and reset by software.

0: does not reset the TIM8 block (default after reset)

1: resets the TIM8 block

Bit 12 **SPI1RST**: SPI1 block reset

Set and reset by software.

0: does not reset the SPI1 block (default after reset)

1: resets the SPI1 block

Bit 11 **TIM1RST**: TIM1 block reset

Set and reset by software.

0: does not reset the TIM1 block (default after reset)

1: resets the TIM1 block

Bits 10:0 Reserved, must be kept at reset value.

11.8.25 RCC APB3 peripheral reset register (RCC_APB3RSTR)

Address offset: 0x080

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1 RST	PLAY1 POR	VREFR ST	Res.	Res.	Res.	Res.
									rw	rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIM6 RST	LPTIM5 RST	LPTIM4 RST	LPTIM3 RST	LPTIM1 RST	Res.	I3C2RS T	I2C4RS T	I2C3RS T	LPUAR T1RST	SPI5R ST	Res.	Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw					

Bits 31:23 Reserved, must be kept at reset value.

Bit 22 **PLAY1RST**: PLAY1 block reset

Set and reset by software. This bit does not reset the user configuration registers and the matrix

0: does not reset the PLAY1 block (default after reset)

1: resets the PLAY1 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 21 **PLAY1POR**: PLAY1 block power-on-reset

Set and reset by software. This bit generates a global reset including the user configuration registers and the matrix

0: does not reset the PLAY1 block (default after reset)

1: resets the PLAY1 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **VREFRST**: VREFBUF block reset

Set and reset by software.

0: does not reset the VREFBUF block (default after reset)

1: resets the VREFBUF block

Bits 19:16 Reserved, must be kept at reset value.

Bit 15 **LPTIM6RST**: LPTIM6 block reset

Set and reset by software.

0: does not reset the LPTIM6 block (default after reset)

1: resets the LPTIM6 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 14 LPTIM5RST: LPTIM5 block reset

Set and reset by software.

0: does not reset the LPTIM5 block (default after reset)

1: resets the LPTIM5 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 LPTIM4RST: LPTIM4 block reset

Set and reset by software.

0: does not reset the LPTIM4 block (default after reset)

1: resets the LPTIM4 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 12 LPTIM3RST: LPTIM3 block reset

Set and reset by software.

0: does not reset the LPTIM3 block (default after reset)

1: resets the LPTIM3 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 LPTIM1RST: LPTIM1 block reset

Set and reset by software.

0: does not reset the LPTIM1 block (default after reset)

1: resets the LPTIM1 block

Bit 10 Reserved, must be kept at reset value.

Bit 9 I3C2RST: I3C2 block reset

Set and reset by software.

0: does not reset the I3C2 block (default after reset)

1: resets the I3C2 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value..

Bit 8 I2C4RST: I2C4 block reset

Set and reset by software.

0: does not reset the I2C4 block (default after reset)

1: resets the I2C4 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 I2C3RST: I2C3 block reset

Set and reset by software.

0: does not reset the I2C3 block (default after reset)

1: resets the I2C3 block

Bit 6 LPUART1RST: LPUART1 block reset

Set and reset by software.

0: does not reset the LPUART1 block (default after reset)

1: resets the LPUART1 block

Bit 5 **SPI5RST**: SPI5 block reset

Set and reset by software.

0: does not reset the SPI5 block (default after reset)

1: resets the SPI5 block

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 4:0 Reserved, must be kept at reset value.

11.8.26 RCC AHB1 peripherals clock register (RCC_AHB1ENR)

Address offset: 0x088

Reset value: 0xD000 0100

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM1 EN	DCAC HEEN	Res.	BKPR AMEN	Res.	Res.	Res.	TZSC1 EN	Res.	Res.	ETHRX EN	ETHTX EN	ETHEN	ETHCK EN	RAMC FGEN	Res.
rw	rw		rw				rw			rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMACE N	CORDI CEN	Res.	CRCE N	Res.	Res.	Res.	FLITFE N	Res.	Res.	Res.	Res.	Res.	Res.	GPDM A2EN	GPDM A1EN
rw	rw		rw				rw							rw	rw

Bit 31 **SRAM1EN**: SRAM1 clock enable

Set and reset by software.

0: SRAM1 clock disabled

1: SRAM1 clock enabled (default after reset)

Bit 30 **DCACHEEN**: DCACHE clock enable

Set and reset by software

0: DCACHE peripheral clock disabled (default after reset)

1: DCACHE peripheral clock enabled

Bit 29 Reserved, must be kept at reset value.

Bit 28 **BKPRAMEN**: BKPRAM clock enable

Set and reset by software

0: BKPRAM peripheral clock disabled (default after reset)

1: BKPRAM peripheral clock enabled

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **TZSC1EN**: TZSC1 clock enable

Set and reset by software

0: TZSC1 peripheral clock disabled (default after reset)

1: TZSC1 peripheral clock enabled

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 ETHRXEN: ETHRX clock enable

Set and reset by software

0: ETHRX clock disabled (default after reset)

1: ETHRX clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 ETHTXEN: ETHTX clock enable

Set and reset by software

0: ETHTX clock disabled (default after reset)

1: ETHTX clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 19 ETHER: ETH clock enable

Set and reset by software

0: ETH peripheral clock disabled (default after reset)

1: ETH peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 18 ETHCKEN: ETH internal clock enable

Set and reset by software.

0: ETH internal clock disabled (default after reset)

1: ETH internal clock enabled

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 17 RAMCFGEN: RAMCFG clock enable

Set and reset by software.

0: RAMCFG peripheral clock disabled (default after reset)

1: RAMCFG peripheral clock enabled

Bit 16 Reserved, must be kept at reset value.**Bit 15 FMACEN:** FMAC clock enable

Set and reset by software.

0: FMAC peripheral clock disabled (default after reset)

1: FMAC peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 14 CORDICEN: CORDIC clock enable

Set and reset by software.

0: CORDIC peripheral clock disabled (default after reset)

1: CORDIC peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 Reserved, must be kept at reset value.

Bit 12 **CRCEN**: CRC clock enable

Set and reset by software.

0: CRC peripheral clock disabled (default after reset)

1: CRC peripheral clock enabled

Bits 11:9 Reserved, must be kept at reset value.

Bit 8 **FLITFEN**: Flash interface clock enable

Set and reset by software.

0: FLASH interface clock disabled

1: FLASH interface clock enabled (default after reset)

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **GPDMA2EN**: GPDMA2 clock enable

Set and reset by software.

0: GPDMA2 peripheral clock disabled (default after reset)

1: GPDMA2 peripheral clock enabled

Bit 0 **GPDMA1EN**: GPDMA1 clock enable

Set and reset by software.

0: GPDMA1 peripheral clock disabled (default after reset)

1: GPDMA1 peripheral clock enabled

11.8.27 RCC AHB2 peripheral clock register (RCC_AHB2ENR)

Address offset: 0x08C

Reset value: 0xC000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3 EN	SRAM2 EN	Res.	Res.	Res.	Res.	Res.	ADC3E N	Res.	Res.	CCBEN	SAESE N	PKAEN	RNGE N	HASHE N	AESEN
rw	rw						rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	DCMI PSSI EN	DAC1E N	ADCEN	Res.	GPIOIE N	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **SRAM3EN**: SRAM3 clock enable

Set and reset by software.

0: SRAM3 clock disabled

1: SRAM3 clock enabled (default after reset)

Bit 30 **SRAM2EN**: SRAM2 clock enable

Set and reset by software.

0: SRAM2 clock disabled

1: SRAM2 clock enabled (default after reset)

Bits 29:25 Reserved, must be kept at reset value.

Bit 24 **ADC3EN**: ADC3 clock enable

Set and reset by software.

0: ADC3 clock disabled (default after reset)

1: ADC3 clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **CCBEN**: CCB clock enable

Set and reset by software.

0: CCB clock disabled (default after reset)

1: CCB clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **SAESEN**: SAES clock enable

Set and reset by software.

0: SAES peripheral clock disabled (default after reset)

1: SAES peripheral clock enabled

Bit 19 **PKAEN**: PKA clock enable

Set and reset by software.

0: PKA peripheral clock disabled (default after reset)

1: PKA peripheral clock enabled

Bit 18 **RNGEN**: RNG clock enable

Set and reset by software.

0: RNG peripheral clock disabled (default after reset)

1: RNG peripheral clock enabled

Bit 17 **HASHEN**: HASH clock enable

Set and reset by software.

0: HASH peripheral clock disabled (default after reset)

1: HASH peripheral clock enabled

Bit 16 **AESEN**: AES clock enable

Set and reset by software.

0: AES peripheral clock disabled (default after reset)

1: AES peripheral clock enabled

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **DCMI_PSSIEN**: digital camera interface clock enable (DCMI or PSSI depending which interface is active)

Set and reset by software.

0: DCMI/PSSI peripheral clock disabled (default after reset)

1: DCMI/PSSI peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 **DAC1EN**: DAC clock enable

Set and reset by software.

0: DAC peripheral clock disabled (default after reset)

1: DAC peripheral clock enabled

Bit 10 **ADCEN**: ADC1 and 2 peripherals clock enable

Set and reset by software.

0: ADC1 and 2 peripherals clock disabled (default after reset)

1: ADC1 and 2 peripherals clock enabled

Bit 9 Reserved, must be kept at reset value.

Bit 8 **GPIOIEN**: GPIOI clock enable

Set and reset by software.

0: GPIOI peripheral clock disabled (default after reset)

1: GPIOI peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **GPIOHEN**: GPIOH clock enable

Set and reset by software.

0: GPIOH peripheral clock disabled (default after reset)

1: GPIOH peripheral clock enabled

Bit 6 **GPIOGEN**: GPIOG clock enable

Set and reset by software.

0: GPIOG peripheral clock disabled (default after reset)

1: GPIOG peripheral clock enabled

Bit 5 **GPIOFEN**: GPIOF clock enable

Set and reset by software.

0: GPIOF peripheral clock disabled (default after reset)

1: GPIOF peripheral clock enabled

Bit 4 **GPIOEEN**: GPIOE clock enable

Set and reset by software.

0: GPIOE peripheral clock disabled (default after reset)

1: GPIOE peripheral clock enabled

Bit 3 **GPIODEN**: GPIOD clock enable

Set and reset by software.

0: GPIOD peripheral clock disabled (default after reset)

1: GPIOD peripheral clock enabled

Bit 2 **GPIOCEN**: GPIOC clock enable

Set and reset by software.

0: GPIOC peripheral clock disabled (default after reset)

1: GPIOC peripheral clock enabled

Bit 1 **GPIOBEN**: GPIOB clock enable

Set and reset by software.

0: GPIOB peripheral clock disabled (default after reset)

1: GPIOB peripheral clock enabled

- Bit 0 **GPIOAEN**: GPIOA clock enable
 Set and reset by software.
 0: GPIOA peripheral clock disabled (default after reset)
 1: GPIOA peripheral clock enabled

11.8.28 RCC AHB4 peripheral clock register (RCC_AHB4ENR)

Address offset: 0x094

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OCTOSPI1EN	Res.	Res.	Res.	FMCE N
											rw				rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	SDMMC2EN	SDMMC1EN	Res.	Res.	Res.	OTFDEC1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.
			rw	rw				rw							

Bits 31:21 Reserved, must be kept at reset value.

- Bit 20 **OCTOSPI1EN**: OCTOSPI1 clock enable
 Set and reset by software.
 0: OCTOSPI1 peripheral clock disabled (default after reset)
 1: OCTOSPI1 peripheral clock enabled

Bits 19:17 Reserved, must be kept at reset value.

- Bit 16 **FMCE N**: FMC clock enable
 Set and reset by software.
 0: FMC peripheral clock disabled (default after reset)
 1: FMC peripheral clock enabled

Bits 15:13 Reserved, must be kept at reset value.

- Bit 12 **SDMMC2EN**: SDMMC2 and SDMMC2 delay peripheral clock enable
 Set and reset by software.
 0: SDMMC2 and SDMMC2 delay peripherals clock disabled (default after reset)
 1: SDMMC2 and SDMMC2 delay peripherals clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

- Bit 11 **SDMMC1EN**: SDMMC1 and SDMMC1 delay peripheral clock enable reset
 0: SDMMC1 and SDMMC1 delay peripherals clock disabled (default after reset)
 1: SDMMC1 and SDMMC1 delay peripherals clock enabled

Bits 10:8 Reserved, must be kept at reset value.

- Bit 7 **OTFDEC1EN**: OTFDEC1 clock enable
 Set and reset by software.
 0: OTFDEC1 peripheral clock disabled (default after reset)
 1: OTFDEC1 peripheral clock enabled

Bits 6:0 Reserved, must be kept at reset value.

11.8.29 RCC APB1 peripheral clock register (RCC_APB1LENR)

Address offset: 0x09C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8 EN	UART7 EN	Res.	CECEN	USART 11EN	USART 10EN	USART 6EN	CRSEN	I3C1EN	I2C2EN	I2C1EN	UART5 EN	UART4 EN	USART 3EN	USART 2EN	COMP EN
rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3E N	SPI2E N	Res.	Res.	WWDG EN	Res.	Res.	TIM14E N	TIM13E N	TIM12E N	TIM7E N	TIM6E N	TIM5E N	TIM4E N	TIM3E N	TIM2E N
rw	rw			rw			rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **UART8EN**: UART8 clock enable

Set and reset by software.

0: UART8 peripheral clock disabled (default after reset)

1: UART8 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 30 **UART7EN**: UART7 clock enable

Set and reset by software.

0: UART7 peripheral clock disabled (default after reset)

1: UART7 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 29 Reserved, must be kept at reset value.

Bit 28 **CECEN**: HDMI-CEC clock enable

Set and reset by software.

0: HDMI-CEC peripheral clock disabled (default after reset)

1: HDMI-CEC peripheral clock enabled

Bit 27 **USART11EN**: USART11 clock enable

0: USART11 peripheral clock disabled (default after reset)

1: USART11 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 26 **USART10EN**: USART10 clock enable

Set and reset by software.

0: USART10 peripheral clock disabled (default after reset)

1: USART10 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

- Bit 25 **USART6EN**: USART6 clock enable
Set and reset by software.
0: USART6 peripheral clock disabled (default after reset)
1: USART6 peripheral clock enabled
- Bit 24 **CRSEN**: CRS clock enable
Set and reset by software.
0: CRS peripheral clock disabled (default after reset)
1: CRS peripheral clock enabled
- Bit 23 **I3C1EN**: I3C1 clock enable
Set and reset by software.
0: I3C1 peripheral clock disabled (default after reset)
1: I3C1 peripheral clock enabled
- Bit 22 **I2C2EN**: I2C2 clock enable
Set and reset by software.
0: I2C2 peripheral clock disabled (default after reset)
1: I2C2 peripheral clock enabled
- Bit 21 **I2C1EN**: I2C1 clock enable
Set and reset by software.
0: I2C1 peripheral clock disabled (default after reset)
1: I2C1 peripheral clock enabled
- Bit 20 **UART5EN**: UART5 clock enable
Set and reset by software.
0: UART5 peripheral clock disabled (default after reset)
1: UART5 peripheral clock enabled
- Bit 19 **UART4EN**: UART4 clock enable
Set and reset by software.
0: UART4 peripheral clock disabled (default after reset)
1: UART4 peripheral clock enabled
- Bit 18 **USART3EN**: USART3 clock enable
Set and reset by software.
0: USART3 peripheral clock disabled (default after reset)
1: USART3 peripheral clock enabled
- Bit 17 **USART2EN**: USART2 clock enable
Set and reset by software.
0: USART2 peripheral clock disabled (default after reset)
1: USART2 peripheral clock enabled
- Bit 16 **COMPEN**: COMP clock enable
Set and reset by software.
0: COMP peripheral clock disabled (default after reset)
1: COMP peripheral clock enabled
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.
- Bit 15 **SPI3EN**: SPI3 clock enable
Set and reset by software.
0: SPI3 peripheral clock disabled (default after reset)
1: SPI3 peripheral clock enabled

Bit 14 **SPI2EN**: SPI2 clock enable

Set and reset by software.

0: SPI2 peripheral clock disabled (default after reset)

1: SPI2 peripheral clock enabled

Bits 13:12 Reserved, must be kept at reset value.

Bit 11 **WWDGEN**: WWDG clock enable

Set and reset by software.

0: WWDG peripheral clock disabled (default after reset)

1: WWDG peripheral clock enabled

Bits 10:9 Reserved, must be kept at reset value.

Bit 8 **TIM14EN**: TIM14 clock enable

Set and reset by software.

0: TIM14 peripheral clock disabled (default after reset)

1: TIM14 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **TIM13EN**: TIM13 clock enable

Set and reset by software.

0: TIM13 peripheral clock disabled (default after reset)

1: TIM13 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 6 **TIM12EN**: TIM12 clock enable

Set and reset by software.

0: TIM12 peripheral clock disabled (default after reset)

1: TIM12 peripheral clock enabled

Bit 5 **TIM7EN**: TIM7 clock enable

Set and reset by software.

0: TIM7 peripheral clock disabled (default after reset)

1: TIM7 peripheral clock enabled

Bit 4 **TIM6EN**: TIM6 clock enable

Set and reset by software.

0: TIM6 peripheral clock disabled (default after reset)

1: TIM6 peripheral clock enabled

Bit 3 **TIM5EN**: TIM5 clock enable

Set and reset by software.

0: TIM5 peripheral clock disabled (default after reset)

1: TIM5 peripheral clock enabled

Bit 2 **TIM4EN**: TIM4 clock enable

Set and reset by software.

0: TIM4 peripheral clock disabled (default after reset)

1: TIM4 peripheral clock enabled

Bit 1 **TIM3EN**: TIM3 clock enable

Set and reset by software.

0: TIM3 peripheral clock disabled (default after reset)

1: TIM3 peripheral clock enabled

Bit 0 **TIM2EN**: TIM2 clock enable

Set and reset by software.

0: TIM2 peripheral clock disabled (default after reset)

1: TIM2 peripheral clock enabled

11.8.30 RCC APB1 peripheral clock register (RCC_APB1HENR)

Address offset: 0x0A0

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD1 EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.
								rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	FDCAN EN	Res.	Res.	Res.	LPTIM2 EN	Res.	DTSEN	Res.	UART1 2EN	UART9 EN
						rw				rw		rw		rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **UCPD1EN**: UCPD1 clock enable

Set and reset by software.

0: UCPD peripheral clock disabled (default after reset)

1: UCPD peripheral clock enabled

Bits 22:10 Reserved, must be kept at reset value.

Bit 9 **FDCANEN**: FDCAN1 and FDCAN2 peripheral clock enable

Set and reset by software.

0: FDCAN1 and FDCAN2 peripheral clock disabled (default after reset)

1: FDCAN1 and FDCAN2 peripheral clock enabled

Bits 8:6 Reserved, must be kept at reset value.

Bit 5 **LPTIM2EN**: LPTIM2 clock enable

Set and reset by software.

0: LPTIM2 peripheral clock disabled (default after reset)

1: LPTIM2 peripheral clock enabled

Bit 4 Reserved, must be kept at reset value.

Bit 3 **DTSEN**: DTS clock enable

Set and reset by software.

0: DTS peripheral clock disabled (default after reset)

1: DTS peripheral clock enabled

Bit 2 Reserved, must be kept at reset value.

Bit 1 **UART12EN**: UART12 clock enable

Set and reset by software.

0: UART12 peripheral clock disabled (default after reset)

1: UART12 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 0 **UART9EN**: UART9 clock enable

Set and reset by software.

0: UART9 peripheral clock disabled (default after reset)

1: resets UART9 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

11.8.31 RCC APB2 peripheral clock register (RCC_APB2ENR)

Address offset: 0x0A4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBEN	Res.	SAI2EN	SAI1EN	SPI6EN	SPI4EN	TIM17EN	TIM16EN	TIM15EN
							rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART1EN	TIM8EN	SPI1EN	TIM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw	rw	rw	rw											

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **USBEN**: USB clock enable

Set and reset by software.

0: USB peripheral clock disabled (default after reset)

1: USB peripheral clock enabled

Bit 23 Reserved, must be kept at reset value.

Bit 22 **SAI2EN**: SAI2 clock enable

Set and cleared by software.

0: SAI2 clock disabled

1: SAI2 clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 21 **SAI1EN**: SAI1 clock enable

Set and reset by software.

0: SAI1 peripheral clock disabled (default after reset)

1: SAI1 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **SPI6EN**: SPI6 clock enable

Set and reset by software.

0: SPI6 peripheral clock disabled (default after reset)

1: SPI6 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value..

Bit 19 **SPI4EN**: SPI4 clock enable

Set and reset by software.

0: SPI4 peripheral clock disabled (default after reset)

1: SPI4 peripheral clock enabled

Bit 18 **TIM17EN**: TIM17 clock enable

Set and reset by software.

0: TIM17 peripheral clock disabled (default after reset)

1: TIM17 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 17 **TIM16EN**: TIM16 clock enable

Set and reset by software.

0: TIM16 peripheral clock disabled (default after reset)

1: TIM16 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 16 **TIM15EN**: TIM15 clock enable

Set and reset by software.

0: TIM15 peripheral clock disabled (default after reset)

1: TIM15 peripheral clock enabled

Bit 15 Reserved, must be kept at reset value.

Bit 14 **USART1EN**: USART1 clock enable

Set and reset by software.

0: USART1 peripheral clock disabled (default after reset)

1: USART1 peripheral clock enabled

Bit 13 **TIM8EN**: TIM8 clock enable

Set and reset by software.

0: TIM8 peripheral clock disabled (default after reset)

1: TIM8 peripheral clock enabled

Bit 12 **SPI1EN**: SPI1 clock enable

Set and reset by software.

0: SPI1 peripheral clock disabled (default after reset)

1: SPI1 peripheral clock enabled

Bit 11 **TIM1EN**: TIM1 clock enable

Set and reset by software.

0: TIM1 peripheral clock disabled (default after reset)

1: TIM1 peripheral clock enabled

Bits 10:0 Reserved, must be kept at reset value.

11.8.32 RCC APB3 peripheral clock register (RCC_APB3ENR)

Address offset: 0x0A8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1 EN	PLAY1 APBEN	RTCAP BEN	VREFB UFEN	Res.	Res.	Res.	Res.
								rw	rw	rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIM6 EN	LPTIM5 EN	LPTIM4 EN	LPTIM3 EN	LPTIM1 EN	Res.	I3C2EN	I2C4EN	I2C3EN	LPUAR T1EN	SPI5E N	Res.	Res.	Res.	SBSSEN	Res.
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw				rw	

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **PLAY1EN**: PLAY1 clock enable

Set and reset by software.

0: PLAY1 clock disabled (default after reset)

1: PLAY1 clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 22 **PLAY1APBEN**: Clock enable for PLAY1 configuration

Set and reset by software.

0: PLAY1 clock disabled (default after reset)

1: PLAY1 clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 21 **RTCAPBEN**: RTC APB interface clock enable

Set and reset by software.

0: RTC APB interface clock disabled (default after reset)

1: RTC APB interface clock enabled

Bit 20 **VREFBUFEN**: VREFBUF clock enable

Set and reset by software.

0: VREFBUF peripheral clock disabled (default after reset)

1: VREFBUF peripheral clock enabled

Bits 19:16 Reserved, must be kept at reset value.

Bit 15 **LPTIM6EN**: LPTIM6 clock enable

Set and reset by software.

0: LPTIM6 peripheral clock disabled (default after reset)

1: LPTIM6 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 14 LPTIM5EN: LPTIM5 clock enable

Set and reset by software.

0: LPTIM5 peripheral clock disabled (default after reset)

1: LPTIM5 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 LPTIM4EN: LPTIM4 clock enable

Set and reset by software.

0: LPTIM4 peripheral clock disabled (default after reset)

1: LPTIM4 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 12 LPTIM3EN: LPTIM3 clock enable

Set and reset by software.

0: LPTIM3 peripheral clock disabled (default after reset)

1: LPTIM3 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 LPTIM1EN: LPTIM1 clock enable

Set and reset by software.

0: LPTIM1 peripheral clock disabled (default after reset)

1: LPTIM1 peripheral clock enabled

Bit 10 Reserved, must be kept at reset value.

Bit 9 I3C2EN: I3C2 clock enable

Set and reset by software.

0: I3C2 peripheral clock disabled (default after reset)

1: I3C2 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 8 I2C4EN: I2C4 clock enable

Set and reset by software.

0: I2C4 peripheral clock disabled (default after reset)

1: I2C4 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 I2C3EN: I2C3 clock enable

Set and reset by software.

0: I2C3 peripheral clock disabled (default after reset)

1: I2C3 peripheral clock enabled

Bit 6 LPUART1EN: LPUART1 clock enable

Set and reset by software.

0: LPUART1 peripheral clock disabled (default after reset)

1: LPUART1 peripheral clock enabled

Bit 5 **SPI5EN**: SPI5 clock enable

Set and reset by software.

0: SPI5 peripheral clock disabled (default after reset)

1: SPI5 peripheral clock enabled

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **SBSSEN**: SBS clock enable

Set and reset by software.

0: SBS peripheral clock disabled (default after reset)

1: SBS peripheral clock enabled

Bit 0 Reserved, must be kept at reset value.

11.8.33 RCC AHB1 sleep clock register (RCC_AHB1LPENR)

Address offset: 0x0B0

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0xF13A D103 (for STM32H562/563/573xx)

Reset value: 0xF102 1103 (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM1 LPEN	DCACHELPE N	ICACHELPE N	BKPRAMLPE N	Res.	Res.	Res.	TZSC1 LPEN	Res.	Res.	ETHRX LPEN	ETHTX LPEN	ETHLP EN	ETHCK LPEN	RAMC FGLPE N	Res.
rw	rw	rw	rw				rw			rw	rw	rw	rw	rw	
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FMACL PEN	CORCLPE N	Res.	CRCLPE N	Res.	Res.	Res.	FLITFL PEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
rw	rw		rw				rw								

Bit 31 **SRAM1LPEN**: SRAM1 clock enable during Sleep mode

Set and reset by software

0: SRAM1 peripheral clock disabled during Sleep mode

1: SRAM1 peripheral clock enabled during Sleep mode (default after reset)

Bit 30 **DCACHELPEN**: DCACHE clock enable during Sleep mode

Set and reset by software

0: DCACHE peripheral clock disabled during Sleep mode

1: DCACHE peripheral clock enabled during Sleep mode (default after reset)

Bit 29 **ICACHELPEN**: ICACHE clock enable during Sleep mode

Set and reset by software

0: ICACHE peripheral clock disabled during Sleep mode

1: ICACHE peripheral clock enabled during Sleep mode (default after reset)

Bit 28 **BKPRAMLPE**: BKPRAM clock enable during Sleep mode

Set and reset by software

0: BKPRAM peripheral clock disabled during Sleep mode

1: BKPRAM peripheral clock enabled during Sleep mode (default after reset)

Bits 27:25 Reserved, must be kept at reset value.

Bit 24 **TZSC1LPEN**: TZSC1 clock enable during Sleep mode

Set and reset by software

0: TZSC1 peripheral clock disabled during Sleep mode

1: TZSC1 peripheral clock enabled during Sleep mode (default after reset)

Bits 23:22 Reserved, must be kept at reset value.

Bit 21 **ETHRXLPEN**: ETHRX clock enable during Sleep mode

Set and reset by software

0: ETHRX clock disabled during Sleep mode

1: ETHRX clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **ETHTXLPEN**: ETHTX clock enable during Sleep mode

Set and reset by software

0: ETHTX clock disabled during Sleep mode

1: ETHTX clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 19 **ETHLPEN**: ETH clock enable during Sleep mode

Set and reset by software

0: ETH peripheral clock disabled during Sleep mode

1: ETH peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 18 **ETHCKLPEN**: ETH internal clock enable during Sleep mode

Set and reset by software

0: ETH internal clock disabled during Sleep mode

1: ETH internal clock enabled during Sleep mode (default after reset)

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 17 **RAMCFGLPEN**: RAMCFG clock enable during Sleep mode

Set and reset by software.

0: RAMCFG peripheral clock disabled during Sleep mode

1: RAMCFG peripheral clock enabled during Sleep mode (default after reset)

Bit 16 Reserved, must be kept at reset value.

Bit 15 **FMACLPEN**: FMAC clock enable during Sleep mode

Set and reset by software.

0: FMAC peripheral clock disabled during Sleep mode

1: FMAC peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 14 **CORDICLPEN**: CORDIC clock enable during Sleep mode

Set and reset by software.

0: CORDIC peripheral clock disabled during Sleep mode

1: CORDIC peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 Reserved, must be kept at reset value.

Bit 12 **CRCLPEN**: CRC clock enable during Sleep mode

Set and reset by software.

0: CRC peripheral clock disabled during Sleep mode

1: CRC peripheral clock enabled during Sleep mode (default after reset)

Bits 11:9 Reserved, must be kept at reset value.

Bit 8 **FLITFLPEN**: Flash interface (FLITF) clock enable during Sleep mode

Set and reset by software.

0: FLITF peripheral clock disabled during Sleep mode

1: FLITF peripheral clock enabled during Sleep mode (default after reset)

Bits 7:2 Reserved, must be kept at reset value.

Bit 1 **GPDMA2LPEN**: GPDMA2 clock enable during Sleep mode

Set and reset by software.

0: GPDMA2 peripheral clock disabled during Sleep mode

1: GPDMA2 peripheral clock enabled during Sleep mode (default after reset)

Bit 0 **GPDMA1LPEN**: GPDMA1 clock enable during Sleep mode

Set and reset by software.

0: GPDMA1 peripheral clock disabled during Sleep mode

1: GPDMA1 peripheral clock enabled during Sleep mode (default after reset)

11.8.34 RCC AHB2 sleep clock register (RCC_AHB2LPENR)

Address offset: 0x0B4

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0xC01F 1DFF (for STM32H562/563/573xx)

Reset value: 0xC01F 1CFF (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SRAM3 LPEN	SRAM2 LPEN	Res.	Res.	Res.	Res.	Res.	ADC3L PEN	Res.	Res.	CCBLP EN	SAESL PEN	PKALP EN	RNGLP EN	HASHL PEN	AESLP EN
rw	rw						rw			rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	DCMI PSSILP EN	DAC1L PEN	ADCLP EN	Res.	GPIOIL PEN	GPIOH LPEN	GPIOG LPEN	GPIOF LPEN	GPIOE LPEN	GPIOD LPEN	GPIOC LPEN	GPIOB LPEN	GPIOA LPEN
			rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw

- Bit 31 **SRAM3LPEN**: SRAM3 clock enable during Sleep mode
Set and reset by software.
0: SRAM3 peripheral clock disabled during Sleep mode
1: SRAM3 peripheral clock enabled during Sleep mode (default after reset)
- Bit 30 **SRAM2LPEN**: SRAM2 clock enable during Sleep mode
Set and reset by software.
0: SRAM2 peripheral clock disabled during Sleep mode
1: SRAM2 peripheral clock enabled during Sleep mode (default after reset)
- Bits 29:25 Reserved, must be kept at reset value.
- Bit 24 **ADC3LPEN**: ADC3 peripheral clock enable during Sleep mode
Set and reset by software.
0: ADC3 peripheral clock disabled during Sleep mode
1: ADC3 peripheral clock enabled during Sleep mode (default after reset)
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.
- Bits 23:22 Reserved, must be kept at reset value.
- Bit 21 **CCBLPEN**: CCB clock enable during Sleep mode
Set and reset by software.
0: CCB peripheral clock disabled during Sleep mode
1: CCB peripheral clock enabled during Sleep mode (default after reset)
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.
- Bit 20 **SAESLPEN**: SAES clock enable during Sleep mode
Set and reset by software.
0: SAES peripheral clock disabled during Sleep mode
1: SAES peripheral clock enabled during Sleep mode (default after reset)
- Bit 19 **PKALPEN**: PKA clock enable during Sleep mode
Set and reset by software.
0: PKA peripheral clock disabled during Sleep mode
1: PKA peripheral clock enabled during Sleep mode (default after reset)
- Bit 18 **RNGLPEN**: RNG clock enable during Sleep mode
Set and reset by software.
0: RNG peripheral clock disabled during Sleep mode
1: RNG peripheral clock enabled during Sleep mode (default after reset)
- Bit 17 **HASHLPEN**: HASH clock enable during Sleep mode
Set and reset by software.
0: HASH peripheral clock disabled during Sleep mode
1: HASH peripheral clock enabled during Sleep mode (default after reset)
- Bit 16 **AESLPEN**: AES clock enable during Sleep mode
Set and reset by software.
0: AES peripheral clock disabled during Sleep mode
1: AES peripheral clock enabled during Sleep mode (default after reset)
- Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **DCMI_PSSILPEN**: digital camera interface clock enable during Sleep mode (DCMI or PSSI depending which interface is active)

Set and reset by software.

0: DCMI/PSSI peripheral clock disabled during Sleep mode

1: DCMI/PSSI peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 **DAC1LPEN**: DAC clock enable during Sleep mode

Set and reset by software.

0: DAC peripheral clock disabled during Sleep mode

1: DAC peripheral clock enabled during Sleep mode (default after reset)

Bit 10 **ADCLPEN**: ADC1 and 2 peripherals clock enable during Sleep mode

Set and reset by software.

0: ADC1 and 2 peripherals clock disabled during Sleep mode

1: ADC1 and 2 peripherals clock enabled during Sleep mode (default after reset)

Bit 9 Reserved, must be kept at reset value.

Bit 8 **GPIOLPEN**: GPIOI clock enable during Sleep mode

Set and reset by software.

0: GPIOI peripheral clock disabled during Sleep mode

1: GPIOI peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **GPIOHPEN**: GPIOH clock enable during Sleep mode

Set and reset by software.

0: GPIOH peripheral clock disabled during Sleep mode

1: GPIOH peripheral clock enabled during Sleep mode (default after reset)

Bit 6 **GPIOGLPEN**: GPIOG clock enable during Sleep mode

Set and reset by software.

0: GPIOG peripheral clock disabled during Sleep mode

1: GPIOG peripheral clock enabled during Sleep mode (default after reset)

Bit 5 **GPIOFLPEN**: GPIOF clock enable during Sleep mode

Set and reset by software.

0: GPIOF peripheral clock disabled during Sleep mode

1: GPIOF peripheral clock enabled during Sleep mode (default after reset)

Bit 4 **GPIOELPEN**: GPIOE clock enable during Sleep mode

Set and reset by software.

0: GPIOE peripheral clock disabled during Sleep mode

1: GPIOE peripheral clock enabled during Sleep mode (default after reset)

Bit 3 **GPIODLPEN**: GPIOD clock enable during Sleep mode

Set and reset by software.

0: GPIOD peripheral clock disabled during Sleep mode

1: GPIOD peripheral clock enabled during Sleep mode (default after reset)

Bit 2 **GPIOCLPEN**: GPIOC clock enable during Sleep mode

Set and reset by software.

0: GPIOC peripheral clock disabled during Sleep mode

1: GPIOC peripheral clock enabled during Sleep mode (default after reset)

Bit 1 **GPIOBLPEN**: GPIOB clock enable during Sleep mode

Set and reset by software.

0: GPIOB peripheral clock disabled during Sleep mode

1: GPIOB peripheral clock enabled during Sleep mode (default after reset)

Bit 0 **GPIOALPEN**: GPIOA clock enable during Sleep mode

Set and reset by software.

0: GPIOA peripheral clock disabled during Sleep mode

1: GPIOA peripheral clock enabled during Sleep mode (default after reset)

11.8.35 RCC AHB4 sleep clock register (RCC_AHB4LPENR)

Address offset: 0x0BC

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0x0011 1880 (for STM32H562/563/573xx)

Reset value: 0x0011 0880 (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OCTOSPI1LPEN	Res.	Res.	Res.	FMCLPEN
											rw				rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	SDMMC2LPEN	SDMMC1LPEN	Res.	Res.	Res.	OTFDEC1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.
			rw	rw				rw							

Bits 31:21 Reserved, must be kept at reset value.

Bit 20 **OCTOSPI1LPEN**: OCTOSPI1 clock enable during Sleep mode

Set and reset by software.

0: OCTOSPI1 peripheral clock disabled during Sleep mode

1: OCTOSPI1 peripheral clock enabled during Sleep mode (default after reset)

Bits 19:17 Reserved, must be kept at reset value.

Bit 16 **FMCLPEN**: FMC clock enable during Sleep mode

Set and reset by software.

0: FMC peripheral clock disabled during Sleep mode

1: FMC peripheral clock enabled during Sleep mode (default after reset)

Bits 15:13 Reserved, must be kept at reset value.

Bit 12 **SDMMC2LPEN**: SDMMC2 and SDMMC2 delay peripheral clock enable during Sleep mode

Set and reset by software.

0: SDMMC2 and SDMMC2 delay peripherals clock disabled during Sleep mode

1: SDMMC2 and SDMMC2 delay peripherals clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 **SDMMC1LPEN**: SDMMC1 and SDMMC1 delay peripheral clock enable during Sleep mode

Set and reset by software

0: SDMMC1 and SDMMC1 delay peripherals clock disabled during Sleep mode

1: SDMMC1 and SDMMC1 delay peripherals clock enabled during Sleep mode (default after reset)

Bits 10:8 Reserved, must be kept at reset value.

Bit 7 **OTFDEC1LPEN**: OTFDEC1 clock enable during Sleep mode

Set and reset by software.

0: OTFDEC1 peripheral clock disabled during Sleep mode

1: OTFDEC1 peripheral clock enabled during Sleep mode (default after reset)

Bits 6:0 Reserved, must be kept at reset value.

11.8.36 RCC APB1 sleep clock register (RCC_APB1LLPENR)

Address offset: 0x0C4

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0xDFFE C9FF (for STM32H562/563/573xx)

Reset value: 0x13FE C87F (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
UART8 LPEN	UART7 LPEN	Res.	CECLP EN	USART 11LPE N	USART 10LPE N	USART 6LPEN	CRSLP EN	I3C1LP EN	I2C2LP EN	I2C1LP EN	UART5 LPEN	UART4 LPEN	USART 3LPEN	USART 2LPEN	COMP LPEN
rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI3LP EN	SPI2LP EN	Res.	Res.	WWDG LPEN	Res.	Res.	TIM14L PEN	TIM13L PEN	TIM12L PEN	TIM7LP EN	TIM6LP EN	TIM5LP EN	TIM4LP EN	TIM3LP EN	TIM2LP EN
rw	rw			rw			rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **UART8LPEN**: UART8 clock enable during Sleep mode

Set and reset by software.

0: UART8 peripheral clock disabled during Sleep mode

1: UART8 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 30 **UART7LPEN**: UART7 clock enable during Sleep mode

Set and reset by software.

0: UART7 peripheral clock disabled during Sleep mode

1: UART7 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 29 Reserved, must be kept at reset value.

Bit 28 **CECLPEN**: HDMI-CEC clock enable during Sleep mode

Set and reset by software.

0: HDMI-CEC peripheral clock disabled during Sleep mode

1: HDMI-CEC peripheral clock enabled during Sleep mode (default after reset)

Bit 27 **USART11LPEN**: USART11 clock enable during Sleep mode

Set and reset by software.

0: USART11 peripheral clock disabled during Sleep mode

1: USART11 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 26 **USART10LPEN**: USART10 clock enable during Sleep mode

Set and reset by software.

0: USART10 peripheral clock disabled during Sleep mode

1: USART10 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 25 **USART6LPEN**: USART6 clock enable during Sleep mode

Set and reset by software.

0: USART6 peripheral clock disabled during Sleep mode

1: USART6 peripheral clock enabled during Sleep mode (default after reset)

Bit 24 **CRSLPEN**: CRS clock enable during Sleep mode

Set and reset by software.

0: CRS peripheral clock disabled during Sleep mode

1: CRS peripheral clock enabled during Sleep mode (default after reset)

Bit 23 **I3C1LPEN**: I3C1 clock enable during Sleep mode

Set and reset by software.

0: I3C1 peripheral clock disabled during Sleep mode

1: I3C1 peripheral clock enabled during Sleep mode (default after reset)

Bit 22 **I2C2LPEN**: I2C2 clock enable during Sleep mode

Set and reset by software.

0: I2C2 peripheral clock disabled during Sleep mode

1: I2C2 peripheral clock enabled during Sleep mode (default after reset)

Bit 21 **I2C1LPEN**: I2C1 clock enable during Sleep mode

Set and reset by software.

0: I2C1 peripheral clock disabled during Sleep mode

1: I2C1 peripheral clock enabled during Sleep mode (default after reset)

- Bit 20 **UART5LPEN**: UART5 clock enable during Sleep mode
Set and reset by software.
0: UART5 peripheral clock disabled during Sleep mode
1: UART5 peripheral clock enabled during Sleep mode (default after reset)
- Bit 19 **UART4LPEN**: UART4 clock enable during Sleep mode
Set and reset by software.
0: UART4 peripheral clock disabled during Sleep mode
1: UART4 peripheral clock enabled during Sleep mode (default after reset)
- Bit 18 **USART3LPEN**: USART3 clock enable during Sleep mode
Set and reset by software.
0: USART3 peripheral clock disabled during Sleep mode
1: USART3 peripheral clock enabled during Sleep mode (default after reset)
- Bit 17 **USART2LPEN**: USART2 clock enable during Sleep mode
Set and reset by software.
0: USART2 peripheral clock disabled during Sleep mode
1: USART2 peripheral clock enabled during Sleep mode (default after reset)
- Bit 16 **COMPLPEN**: COMP clock enable during Sleep mode
Set and reset by software.
0: COMP peripheral clock disabled during Sleep mode
1: COMP peripheral clock enabled during Sleep mode (default after reset)
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.
- Bit 15 **SPI3LPEN**: SPI3 clock enable during Sleep mode
Set and reset by software.
0: SPI3 peripheral clock disabled during Sleep mode
1: SPI3 peripheral clock enabled during Sleep mode (default after reset)
- Bit 14 **SPI2LPEN**: SPI2 clock enable during Sleep mode
Set and reset by software.
0: SPI2 peripheral clock disabled during Sleep mode
1: SPI2 peripheral clock enabled during Sleep mode (default after reset)
- Bits 13:12 Reserved, must be kept at reset value.
- Bit 11 **WWDGLPEN**: WWDG clock enable during Sleep mode
Set and reset by software.
0: WWDG peripheral clock disabled during Sleep mode
1: WWDG peripheral clock enabled during Sleep mode (default after reset)
- Bits 10:9 Reserved, must be kept at reset value.
- Bit 8 **TIM14LPEN**: TIM14 clock enable during Sleep mode
Set and reset by software.
0: TIM14 peripheral clock disabled during Sleep mode
1: TIM14 peripheral clock enabled during Sleep mode (default after reset)
Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **TIM13LPEN**: TIM13 clock enable during Sleep mode

Set and reset by software.

0: TIM13 peripheral clock disabled during Sleep mode

1: TIM13 peripheral clock enabled during Sleep mode (default after reset)

Note: For the actual availability of this bit refer to the datasheet. If not present, consider it as reserved, and keep it at reset value.

Bit 6 **TIM12LPEN**: TIM12 clock enable during Sleep mode

Set and reset by software.

0: TIM12 peripheral clock disabled during Sleep mode

1: TIM12 peripheral clock enabled during Sleep mode (default after reset)

Bit 5 **TIM7LPEN**: TIM7 clock enable during Sleep mode

Set and reset by software.

0: TIM7 peripheral clock disabled during Sleep mode

1: TIM7 peripheral clock enabled during Sleep mode (default after reset)

Bit 4 **TIM6LPEN**: TIM6 clock enable during Sleep mode

Set and reset by software.

0: TIM6 peripheral clock disabled during Sleep mode

1: TIM6 peripheral clock enabled during Sleep mode (default after reset)

Bit 3 **TIM5LPEN**: TIM5 clock enable during Sleep mode

Set and reset by software.

0: TIM5 peripheral clock disabled during Sleep mode

1: TIM5 peripheral clock enabled during Sleep mode (default after reset)

Bit 2 **TIM4LPEN**: TIM4 clock enable during Sleep mode

Set and reset by software.

0: TIM4 peripheral clock disabled during Sleep mode

1: TIM4 peripheral clock enabled during Sleep mode (default after reset)

Bit 1 **TIM3LPEN**: TIM3 clock enable during Sleep mode

Set and reset by software.

0: TIM3 peripheral clock disabled during Sleep mode

1: TIM3 peripheral clock enabled during Sleep mode (default after reset)

Bit 0 **TIM2LPEN**: TIM2 clock enable during Sleep mode

Set and reset by software.

0: TIM2 peripheral clock disabled during Sleep mode

1: TIM2 peripheral clock enabled during Sleep mode (default after reset)

11.8.37 RCC APB1 sleep clock register (RCC_APB1HLPENR)

Address offset: 0x0C8

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0x4080 022B (for STM32H562/563/573xx)

Reset value: 0x4080 0228 (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD1 LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.
								rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	FDCAN LPEN	Res.	Res.	Res.	LPTIM2 LPEN	Res.	DTSLP EN	Res.	UART1 2LPEN	UART9 LPEN
						rw				rw		rw		rw	rw

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **UCPD1LPEN**: UCPD1 clock enable during Sleep mode

Set and reset by software.

0: UCPD peripheral clock disabled during Sleep mode

1: UCPD peripheral clock enabled during Sleep mode (default after reset)

Bits 22:10 Reserved, must be kept at reset value.

Bit 9 **FDCANLPEN**: FDCAN1 and FDCAN2 peripheral clock enable during Sleep mode

Set and reset by software.

0: FDCAN1 and FDCAN2 peripheral clock disabled during Sleep mode

1: FDCAN1 and FDCAN2 peripheral clock enabled during Sleep mode (default after reset)

Bits 8:6 Reserved, must be kept at reset value.

Bit 5 **LPTIM2LPEN**: LPTIM2 clock enable during Sleep mode

Set and reset by software.

0: LPTIM2 peripheral clock disabled during Sleep mode

1: LPTIM2 peripheral clock enabled during Sleep mode (default after reset)

Bit 4 Reserved, must be kept at reset value.

Bit 3 **DTSLPEN**: DTS clock enable during Sleep mode

Set and reset by software.

0: DTS peripheral clock disabled during Sleep mode

1: DTS peripheral clock enabled during Sleep mode (default after reset)

Bit 2 Reserved, must be kept at reset value.

Bit 1 **UART12LPEN**: UART12 clock enable during Sleep mode

Set and reset by software.

0: UART12 peripheral clock disabled during Sleep mode

1: UART12 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 0 **UART9LPEN**: UART9 clock enable during Sleep mode

Set and reset by software.

0: UART9 peripheral clock disabled during Sleep mode

1: resets UART9 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.e.

11.8.38 RCC APB2 sleep clock register (RCC_APB2LPENR)

Address offset: 0x0CC

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0x017F 7800 (for STM32H562/563/573xx)

Reset value: 0x0109 7800 (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBLP EN	Res.	SAI2LP EN	SAI1LP EN	SPI6LP EN	SPI4LP EN	TIM17L PEN	TIM16L PEN	TIM15L PEN
							rw		rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	USART 1LPEN	TIM8LP EN	SPI11LP EN	TIM11LP EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	rw	rw	rw	rw											

Bits 31:25 Reserved, must be kept at reset value.

Bit 24 **USBLPEN**: USB clock enable during Sleep mode

Set and reset by software.

0: USB peripheral clock disabled during Sleep mode

1: USB peripheral clock enabled during Sleep mode (default after reset)

Bit 23 Reserved, must be kept at reset value.

Bit 22 **SAI2LPEN**: SAI2 clock enable during Sleep mode

Set and reset by software.

0: SAI2 peripheral clock disabled during Sleep mode

1: SAI2 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 21 **SAI1LPEN**: SAI1 clock enable during Sleep mode

Set and reset by software.

0: SAI1 peripheral clock disabled during Sleep mode

1: SAI1 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 20 **SPI6LPEN**: SPI6 clock enable during Sleep mode

Set and reset by software.

0: SPI6 peripheral clock disabled during Sleep mode

1: SPI6 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 19 **SPI4LPEN**: SPI4 clock enable during Sleep mode

Set and reset by software.

0: SPI4 peripheral clock disabled during Sleep mode

1: SPI4 peripheral clock enabled during Sleep mode (default after reset)

Bit 18 **TIM17LPEN**: TIM17 clock enable during Sleep mode

Set and reset by software.

0: TIM17 peripheral clock disabled during Sleep mode

1: TIM17 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value

Bit 17 **TIM16LPEN**: TIM16 clock enable during Sleep mode

Set and reset by software.

0: TIM16 peripheral clock disabled during Sleep mode

1: TIM16 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 16 **TIM15LPEN**: TIM15 clock enable during Sleep mode

Set and reset by software.

0: TIM15 peripheral clock disabled during Sleep mode

1: TIM15 peripheral clock enabled during Sleep mode (default after reset)

Bit 15 Reserved, must be kept at reset value.

Bit 14 **USART1LPEN**: USART1 clock enable during Sleep mode

Set and reset by software.

0: USART1 peripheral clock disabled during Sleep mode

1: USART1 peripheral clock enabled during Sleep mode (default after reset)

Bit 13 **TIM8LPEN**: TIM8 clock enable during Sleep mode

Set and reset by software.

0: TIM8 peripheral clock disabled during Sleep mode

1: TIM8 peripheral clock enabled during Sleep mode (default after reset)

Bit 12 **SPI1LPEN**: SPI1 clock enable during Sleep mode

Set and reset by software.

0: SPI1 peripheral clock disabled during Sleep mode

1: SPI1 peripheral clock enabled during Sleep mode (default after reset)

Bit 11 **TIM1LPEN**: TIM1 clock enable during Sleep mode

Set and reset by software.

0: TIM1 peripheral clock disabled during Sleep mode

1: TIM1 peripheral clock enabled during Sleep mode (default after reset)

Bits 10:0 Reserved, must be kept at reset value.

11.8.39 RCC APB3 sleep clock register (RCC_APB3LPENR)

Address offset: 0x0D0

Reset value: 0xFFFF FFFF (for STM32H543/553xx)

Reset value: 0x0030 F9E2 (for STM32H562/563/573xx)

Reset value: 0x0030 FAE2 (for STM32H523/533xx)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1 LPEN	Res.	RTCAP BLPEN	VREFL PEN	Res.	Res.	Res.	Res.
								rw		rw	rw				
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
LPTIM6 LPEN	LPTIM5 LPEN	LPTIM4 LPEN	LPTIM3 LPEN	LPTIM1 LPEN	Res.	I3C2LP EN	I2C4LP EN	I2C3LP EN	LPUAR T1LPE N	SPI5LP EN	Res.	Res.	Res.	SBSLP EN	Res.
rw	rw	rw	rw	rw		rw	rw	rw	rw	rw				rw	

Bits 31:24 Reserved, must be kept at reset value.

Bit 23 **PLAY1LPEN**: PLAY1 clock enable during Sleep mode

Set and reset by software.

0: PLAY1 clock disabled during Sleep mode

1: PLAY1 clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 22 Reserved, must be kept at reset value.

Bit 21 **RTCAPBLPEN**: RTC APB interface clock enable during Sleep mode

Set and reset by software.

0: RTC APB interface clock disabled during Sleep mode

1: RTC APB interface clock enabled during Sleep mode (default after reset)

Bit 20 **VREFLPEN**: VREFBUF clock enable during Sleep mode

Set and reset by software.

0: VREFBUF peripheral clock disabled during Sleep mode

1: VREFBUF peripheral clock enabled during Sleep mode (default after reset)

Bits 19:16 Reserved, must be kept at reset value.

Bit 15 **LPTIM6LPEN**: LPTIM6 clock enable during Sleep mode

Set and reset by software.

0: LPTIM6 peripheral clock disabled during Sleep mode

1: LPTIM6 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 14 **LPTIM5LPEN**: LPTIM5 clock enable during Sleep mode

Set and reset by software.

0: LPTIM5 peripheral clock disabled during Sleep mode

1: LPTIM5 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 13 **LPTIM4LPEN**: LPTIM4 clock enable during Sleep mode

Set and reset by software.

0: LPTIM4 peripheral clock disabled during Sleep mode

1: LPTIM4 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 12 **LPTIM3LPEN**: LPTIM3 clock enable during Sleep mode

Set and reset by software.

0: LPTIM3 peripheral clock disabled during Sleep mode

1: LPTIM3 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 11 **LPTIM1LPEN**: LPTIM1 clock enable during Sleep mode

Set and reset by software.

0: LPTIM1 peripheral clock disabled during Sleep mode

1: LPTIM1 peripheral clock enabled during Sleep mode (default after reset)

Bit 10 Reserved, must be kept at reset value.

Bit 9 **I3C2LPEN**: I3C2 clock enable during Sleep mode

Set and reset by software.

0: I3C2 peripheral clock disabled during Sleep mode

1: I3C2 peripheral clock enabled during Sleep mode (default after reset)

Bit 8 **I2C4LPEN**: I2C4 clock enable during Sleep mode

Set and reset by software.

0: I2C4 peripheral clock disabled during Sleep mode

1: I2C4 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 7 **I2C3LPEN**: I2C3 clock enable during Sleep mode

Set and reset by software.

0: I2C3 peripheral clock disabled during Sleep mode

1: I2C3 peripheral clock enabled during Sleep mode (default after reset)

Bit 6 **LPUART1LPEN**: LPUART1 clock enable during Sleep mode

Set and reset by software.

0: LPUART1 peripheral clock disabled during Sleep mode

1: LPUART1 peripheral clock enabled during Sleep mode (default after reset)

Bit 5 **SPI5LPEN**: SPI5 clock enable during Sleep mode

Set and reset by software.

0: SPI5 peripheral clock disabled during Sleep mode

1: SPI5 peripheral clock enabled during Sleep mode (default after reset)

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 4:2 Reserved, must be kept at reset value.

Bit 1 **SBSLPEN**: SBS clock enable during Sleep mode

Set and reset by software.

0: SBS peripheral clock disabled during Sleep mode

1: SBS peripheral clock enabled during Sleep mode (default after reset)

Bit 0 Reserved, must be kept at reset value.

11.8.40 RCC kernel clock configuration register (RCC_CCIPR1)

Address offset: 0x0D8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIMICSEL	Res.	USART10SEL[2:0]			UART9SEL[2:0]			UART8SEL[2:0]			UART7SEL[2:0]			USART6SEL[2:1]	
rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
USART6SEL[0]	UART5SEL[2:0]			UART4SEL[2:0]			USART3SEL[2:0]			USART2SEL[2:0]			USART1SEL[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **TIMICSEL**: TIM12, TIM15 and LPTIM2 input capture source selection

Set and reset by software.

0: No internal clock available for timers input capture (default after reset)

1: hsi_ker_ck/1024, hsi_ker_ck/8 and csi_ker_ck/128 selected for timers input capture

Bit 30 Reserved, must be kept at reset value.

Bits 29:27 **USART10SEL[2:0]**: USART10 kernel clock source selection

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010: pll3_q_ck selected as kernel clock

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 26:24 **UART9SEL[2:0]**: UART9 kernel clock source selection

- 000: rcc_pclk1 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010: pll3_q_ck selected as kernel clock
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: lse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 23:21 **UART8SEL[2:0]**: UART8 kernel clock source selection

- 000: rcc_pclk1 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010:
 - pll3_q_ck selected as kernel clock for STM32H562/563/573xx devices
 - reserved for STM32H543/553xx devices
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: lse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bits 20:18 **UART7SEL[2:0]**: UART7 kernel clock source selection

- 000: rcc_pclk1 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010:
 - pll3_q_ck selected as kernel clock for STM32H562/563/573xx devices
 - reserved for STM32H543/553xx devices
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: lse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bits 17:15 **USART6SEL[2:0]**: USART6 kernel clock source selection

- 000: rcc_pclk1 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010:
 - pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices
 - reserved for STM32H543/553xx devices
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: lse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bits 14:12 **UART5SEL[2:0]**: UART5 kernel clock source selection

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

- pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 11:9 **UART4SEL[2:0]**: UART4 kernel clock source selection

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

- pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 8:6 **USART3SEL[2:0]**: USART3 kernel clock source selection

Set and reset by software.

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

- pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 5:3 **USART2SEL[2:0]**: USART2 kernel clock source selection

Set and reset by software.

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

- pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 2:0 **USART1SEL[2:0]**: USART1 kernel clock source selection

Set and reset by software.

000: rcc_pclk2 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

– pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

– reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

11.8.41 RCC kernel clock configuration register (RCC_CCIPR2)

Address offset: 0x0DC

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	LPTIM6SEL[2:0]			Res.	LPTIM5SEL[2:0]			Res.	LPTIM4SEL[2:0]			Res.	LPTIM3SEL[2:0]		
	rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	LPTIM2SEL[2:0]			Res.	LPTIM1SEL[2:0]			Res.	UART12SEL[2:0]			Res.	USART11SEL[2:0]		
	rw	rw	rw		rw	rw	rw		rw	rw	rw		rw	rw	rw

Bit 31 Reserved, must be kept at reset value.

Bits 30:28 **LPTIM6SEL[2:0]**: LPTIM6 kernel clock source selection

000: rcc_pclk3 selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

010: pll3_r_ck selected as kernel clock

011: lse_ck selected as kernel clock

100: lsi_ck selected as kernel clock

101: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 27 Reserved, must be kept at reset value.

Bits 26:24 **LPTIM5SEL[2:0]**: LPTIM5 kernel clock source selection

000: rcc_pclk3 selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

010: pll3_r_ck selected as kernel clock

011: lse_ck selected as kernel clock

100: lsi_ck selected as kernel clock

101: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 23 Reserved, must be kept at reset value.

Bits 22:20 **LPTIM4SEL[2:0]**: LPTIM4 kernel clock source selection

- 000: rcc_pclk3 selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010: pll3_r_ck selected as kernel clock
- 011: lse_ck selected as kernel clock
- 100: lsi_ck selected as kernel clock
- 101: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 19 Reserved, must be kept at reset value.

Bits 18:16 **LPTIM3SEL[2:0]**: LPTIM3 kernel clock source selection

- 000: rcc_pclk3 selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010: pll3_r_ck selected as kernel clock
- 011: lse_ck selected as kernel clock
- 100: lsi_ck selected as kernel clock
- 101: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 15 Reserved, must be kept at reset value.

Bits 14:12 **LPTIM2SEL[2:0]**: LPTIM2 kernel clock source selection

- 000: rcc_pclk1 selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010:
 - pll3_r_ck selected as kernel clock for STM32H523/533/562/563/573xx devices
 - reserved for STM32H543/553xx devices
- 011: lse_ck selected as kernel clock
- 100: lsi_ck selected as kernel clock
- 101: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bit 11 Reserved, must be kept at reset value.

Bits 10:8 **LPTIM1SEL[2:0]**: LPTIM1 kernel clock source selection

- 000: rcc_pclk3 selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010:
 - pll3_r_ck selected as kernel clock for STM32H523/533/562/563/573xx devices
 - reserved for STM32H543/553xx devices
- 011: lse_ck selected as kernel clock
- 100: lsi_ck selected as kernel clock
- 101: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bit 7 Reserved, must be kept at reset value.

Bits 6:4 **UART12SEL[2:0]**: UART12 kernel clock source selection

Set and reset by software.

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010: pll3_q_ck selected as kernel clock

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 3 Reserved, must be kept at reset value.

Bits 2:0 **USART11SEL[2:0]**: USART11 kernel clock source selection

Set and reset by software.

000: rcc_pclk1 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010: pll3_q_ck selected as kernel clock

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: lse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

11.8.42 RCC kernel clock configuration register (RCC_CCIPR3)

Address offset: 0x0E0

Reset value: 0x0000 0000 (STM32H523/533/6x/573xx devices)

Reset value: 0x0010 0000 (STM32H543/553xx devices)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
PLAY1SEL[2:0]			Res.	Res.	LPUART1SEL[2:0]			PLAY1DIVSEL[3:0]				Res.	Res.	SPI6SEL[2:1]	
rw	rw	rw			rw	rw	rw	rw	rw	rw	rw			rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
SPI6SEL[0]	SPI5SEL[2:0]			SPI4SEL[2:0]			SPI3SEL[2:0]			SPI2SEL[2:0]			SPI1SEL[2:0]		
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:29 **PLAY1SEL[2:0]**: PLAY1 kernel clock source selection

000: rcc_pclk3 selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

011: lse_ck selected as kernel clock

100: lsi_ck selected as kernel clock

101: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 28:27 Reserved, must be kept at reset value.

Bits 26:24 **LPUART1SEL[2:0]**: LPUART1 kernel clock source selection

- 000: rcc_pclk3 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010: pll3_q_ck selected as kernel clock
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: lse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Bits 23:20 **PLAY1DIVSEL[3:0]**: 2^N divider with $N = 0$ to 15

- 0000: $N = 0$, PLAY1DIV_1
- 0001: $N = 1$, PLAY1DIV_2 (default after reset)
- 0010: $N = 2$, PLAY1DIV_4
- 0011: $N = 3$, PLAY1DIV_8
- ...
- 1111: $N = 15$, PLAY1DIV_32768

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 19:18 Reserved, must be kept at reset value.

Bits 17:15 **SPI6SEL[2:0]**: SPI6 kernel clock source selection

- 000: rcc_pclk2 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010: pll3_q_ck selected as kernel clock
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: hse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 14:12 **SPI5SEL[2:0]**: SPI5 kernel clock source selection

- 000: rcc_pclk3 selected as kernel clock (default after reset)
- 001: pll2_q_ck selected as kernel clock
- 010: pll3_q_ck selected as kernel clock
- 011: hsi_ker_ck selected as kernel clock
- 100: csi_ker_ck selected as kernel clock
- 101: hse_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 11:9 SPI4SEL[2:0]: SPI4 kernel clock source selection

000: rcc_pclk2 selected as kernel clock (default after reset)

001: pll2_q_ck selected as kernel clock

010:

- pll3_q_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: hsi_ker_ck selected as kernel clock

100: csi_ker_ck selected as kernel clock

101: hse_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 8:6 SPI3SEL[2:0]: SPI3 kernel clock source selection

Set and reset by software.

000: pll1_q_ck selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

010:

- pll3_p_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: AUDIOCLK selected as kernel clock

100: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 5:3 SPI2SEL[2:0]: SPI2 kernel clock source selection

Set and reset by software.

000: pll1_q_ck selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

010:

- pll3_p_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: AUDIOCLK selected as kernel clock

100: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

Bits 2:0 SPI1SEL[2:0]: SPI1 kernel clock source selection

Set and reset by software.

000: pll1_q_ck selected as kernel clock (default after reset)

001: pll2_p_ck selected as kernel clock

010:

- pll3_p_ck selected as kernel clock for STM32H523/533/562/563/573xx devices

- reserved for STM32H543/553xx devices

011: AUDIOCLK selected as kernel clock

100: per_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

11.8.43 RCC kernel clock configuration register (RCC_CCIPR4)

Address offset: 0x0E4

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ETHPTPDIV[3:0]				I3C2SEL[1:0]		I3C1SEL[1:0]		I2C4SEL[1:0]		I2C3SEL[1:0]		I2C2SEL[1:0]		I2C1SEL[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	ETHCLKSEL	SDMMCC2SEL	SDMMCC1SEL	USBSEL[1:0]		SYSTICKSEL[1:0]		OCTOSPI1SEL[1:0]	
							rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 **ETHPTPDIV[3:0]**: ETH PTP clock division

This bitfield selects the division factor of the PTP clock.

0000: Divided by 1 (default after reset)

0001: Divided by 2

...

1110: Divided by 15

1111: Divided by 16

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bits 27:26 **I3C2SEL[1:0]**: I3C2 kernel clock source selection

00: rcc_pclk3 selected as kernel clock (default after reset)

01:

– pll2_r_ck selected as kernel clock for STM32H543/553xx devices

– pll3_r_ck selected as kernel clock for the other STM32H5 series devices

10: hsi_ker_ck selected as kernel clock

11: no clock selected

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 25:24 **I3C1SEL[1:0]**: I3C1 kernel clock source selection

00: rcc_pclk1 selected as kernel clock (default after reset)

01:

– pll2_r_ck selected as kernel clock for STM32H543/553xx devices

– pll3_r_ck selected as kernel clock for the other STM32H5 series devices

10: hsi_ker_ck selected as kernel clock

11: no clock selected

Bits 23:22 **I2C4SEL[1:0]**: I2C4 kernel clock source selection

00: rcc_pclk3 selected as kernel clock (default after reset)

01: pll3_r_ck selected as kernel clock

10: hsi_ker_ck selected as kernel clock

11: csi_ker_ck selected as kernel clock

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 21:20 **I2C3SEL[1:0]**: I2C3 kernel clock source selection

00: rcc_pclk3 selected as kernel clock (default after reset)

01:

- pll2_r_ck selected as kernel clock for STM32H543/553xx devices
- pll3_r_ck selected as kernel clock for the other STM32H5 series devices

10: hsi_ker_ck selected as kernel clock

11: csi_ker_ck selected as kernel clock

Bits 19:18 **I2C2SEL[1:0]**: I2C2 kernel clock source selection

00: rcc_pclk1 selected as kernel clock (default after reset)

01:

- pll2_r_ck selected as kernel clock for STM32H543/553xx devices
- pll3_r_ck selected as kernel clock for the other STM32H5 series devices

10: hsi_ker_ck selected as kernel clock

11: csi_ker_ck selected as kernel clock

Bits 17:16 **I2C1SEL[1:0]**: I2C1 kernel clock source selection

00: rcc_pclk1 selected as kernel clock (default after reset)

01:

- pll2_r_ck selected as kernel clock for STM32H543/553xx devices
- pll3_r_ck selected as kernel clock for the other STM32H5 series devices

10: hsi_ker_ck selected as kernel clock

11: csi_ker_ck selected as kernel clock

Bits 15:9 Reserved, must be kept at reset value.

Bit 8 **ETHCLKSEL**: ETH clock source selection

This bit selects the reference clock of the PAD ETH_CLK

0: hse_ck (default after reset)

1: pll1_q_ck

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 7 **SDMMC2SEL**: SDMMC2 kernel clock source selection

0: pll1_q_ck selected as kernel clock (default after reset)

1: pll2_r_ck selected as kernel clock

Note: This bit is only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bit 6 **SDMMC1SEL**: SDMMC1 kernel clock source selection

0: pll1_q_ck selected as kernel clock (default after reset)

1: pll2_r_ck selected as kernel clock

Bits 5:4 **USBSEL[1:0]**: USB kernel clock source selection

00: no clock is selected as kernel clock (default after reset)

01: pll1_q_ck selected as kernel clock

10:

- pll2_q_ck selected as kernel clock for STM32H543/553xx devices
- pll3_q_ck selected as kernel clock for the other STM32H5 series devices

11: hsi48_ker_ck selected as kernel clock

Bits 3:2 **SYSTICKSEL[1:0]**: SYSTICK clock source selection

00: rcc_hclk/8 selected as clock source (default after reset)

01: lsi_ker_ck[1] selected as clock source

10: lse_ck[1] selected as clock source

11: reserved, the kernel clock is disabled

Note: rcc_hclk frequency must be four times higher than lsi_ker_ck/lse_ck, that is, period (LSI/LSE) $\geq 4 \times$ period (HCLK).

Bits 1:0 **OCTOSPI1SEL[1:0]**: OCTOSPI1 kernel clock source selection

Set and reset by software.

00: rcc_hclk4 selected as kernel clock (default after reset)

01: pll1_q_ck selected as kernel clock

10: pll2_r_ck selected as kernel clock

11: per_ck selected as kernel clock

11.8.44 RCC kernel clock configuration register (RCC_CCIPR5)

Address offset: 0x0E8

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
CKPERSEL[1:0]		Res.	Res.	Res.	Res.	Res.	Res.	ETHREFCLKSEL	Res.	SAI2SEL[2:0]			SAI1SEL[2:0]		
rw	rw							rw		rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	ETHTPCLKSEL[1:0]		FDCANSEL[1:0]		CECSEL[1:0]		RNGSEL[1:0]		DACSEL	ADCDACSEL[2:0]		
				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:30 **CKPERSEL[1:0]**: per_ck clock source selection

00: hsi_ker_ck selected as kernel clock (default after reset)

01: csi_ker_ck selected as kernel clock

10: hse_ck selected as kernel clock

11: reserved, the per_ck clock is disabled

Bits 29:24 Reserved, must be kept at reset value.

Bit 23 **ETHREFCLKSEL**: ETH RMII reference kernel clock source selection

This bit selects the reference clock of the ETH for RMII mode. It must be set to 0 in any other mode than RMII.

0: ETH_RMII_REF_CLK (default after reset)

1: eth_clk_fb

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 22 Reserved, must be kept at reset value.

Bits 21:19 **SAI2SEL[2:0]**: SAI2 kernel clock source selection

- 000: pll1_q_ck selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010: pll3_p_ck selected as kernel clock
- 011: AUDIOCLK selected as kernel clock
- 100: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 18:16 **SAI1SEL[2:0]**: SAI1 kernel clock source selection

- 000: pll1_q_ck selected as kernel clock (default after reset)
- 001: pll2_p_ck selected as kernel clock
- 010: pll3_p_ck selected as kernel clock
- 011: AUDIOCLK selected as kernel clock
- 100: per_ck selected as kernel clock
- Others: reserved, the kernel clock is disabled

Note: These bits are only available on some devices in the STM32H5 series. Refer to the device datasheet for availability of its associated peripheral. If not present, consider this bit as reserved and keep it at reset value.

Bits 15:12 Reserved, must be kept at reset value.

Bits 11:10 **ETHPTPCLKSEL[1:0]**: ETH PTP kernel clock source selection

- Set and reset by software.
- 00: rcc_hclk1 selected as kernel clock (default after reset)
- 01: pll1_r_ck selected as kernel clock
- 10: pll1_q_ck selected as kernel clock
- 11: pll2_p_ck selected as kernel clock

Note: This bit is only available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bits 9:8 **FDCANSEL[1:0]**: FDCAN1 and FDCAN2 kernel clock source selection

- 00: hse_ck selected as kernel clock (default after reset)
- 01: pll1_q_ck selected as kernel clock
- 10: pll2_q_ck selected as kernel clock
- 11: reserved, the kernel clock is disabled

Bits 7:6 **CECSEL[1:0]**: HSMI-CEC kernel clock source selection

- 00: lse_ck selected as kernel clock (default after reset)
- 01: lsi_ker_ck selected as kernel clock
- 10: csi_ker_ck/122 selected as kernel clock
- 11: reserved, the kernel clock is disabled

Bits 5:4 **RNGSEL[1:0]**: RNG kernel clock source selection

- 00: hsi48_ker_ck selected as kernel clock (default after reset)
- 01: pll1_q_ck selected as kernel clock
- 10: lse_ck selected as kernel clock
- 11: lsi_ker_ck selected as kernel clock

Bit 3 **DACSEL**: DAC sample and hold clock

- 0: dac_hold_ck selected as kernel clock (default after reset)
- 1: dac_hold_ck selected as kernel clock

Bits 2:0 **ADCDACSEL[2:0]**: ADC and DAC kernel clock source selection

000: rcc_hclk selected as kernel clock (default after reset)

001: sys_ck selected as kernel clock

010: pll2_r_ck selected as kernel clock

011: hse_ck selected as kernel clock

100: hsi_ker_ck selected as kernel clock

101: csi_ker_ck selected as kernel clock

Others: reserved, the kernel clock is disabled

11.8.45 RCC Backup domain control register (RCC_BDCR)

Address offset: 0x0F0

Reset value: 0x0000 0000

Reset value: 0x0000 0080 (for STM32H543/553xx devices)

Reset by Backup domain reset.

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access.

Wait states are inserted in case of successive accesses to this register.

Note: After a system reset, this register is write-protected (except bits 27:24 and bit 16). To modify the backup domain bits, the DBP bit in the [PWR Backup domain control register \(PWR_BDCR\)](#) must be set to 1. All bits (except bits 27:24 and bit 16) are reset only after a Backup domain reset (see [Section 11.3.3](#)). Other resets do not have effect on these bits.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	LSIRD Y	LSION	LSCOS EL	LSCOE N	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VSWR ST
				rw	rw	rw	rw								rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RTCEN	Res.	Res.	Res.	Res.	Res.	RTCSEL[1:0]		LSEEXT	LSECS SD	LSECS SON	LSEDRV[1:0]		LSEBY P	LSERD Y	LSEON
rw						rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:28 Reserved, must be kept at reset value.

Bit 27 **LSIRDY**: LSI oscillator ready

Set and cleared by hardware to indicate when the LSI oscillator is stable. After the LSION bit is cleared, LSIRDY goes low after three internal low-speed oscillator clock cycles.

This bit is set when the LSI is used by IWDG or RTC, even if LSION = 0.

0: LSI oscillator not ready

1: LSI oscillator ready

Bit 26 **LSION**: LSI oscillator enable

Set and cleared by software.

0: LSI oscillator off

1: LSI oscillator on

Bit 25 **LSCOSEL**: Low-speed clock output selection

Set and cleared by software.

0: LSI clock selected

1: LSE clock selected

- Bit 24 **LSCOEN**: Low-speed clock output (LSCO) enable
Set and cleared by software.
0: LSCO output disabled
1: LSCO output enabled
- Bits 23:17 Reserved, must be kept at reset value.
- Bit 16 **VSWRST**: VSwitch domain software reset
Set and reset by software.
0: reset not activated (default after Backup domain reset)
1: resets the entire VSW domain
- Bit 15 **RTCEN**: RTC clock enable
Set and reset by software.
0: rtc_ck disabled (default after Backup domain reset)
1: rtc_ck enabled
- Bits 14:10 Reserved, must be kept at reset value.
- Bits 9:8 **RTCSEL[1:0]**: RTC clock source selection
Set by software to select the clock source for the RTC.
These bits can be written only one time (except in case of failure detection on LSE).
These bits must be written before LSECSSON is enabled.
The VSWRST bit can be used to reset them, then it can be written one time again.
If HSE is selected as RTC clock, this clock is lost when the system is in Stop mode or in case of a pin reset (NRST).
00: no clock (default after Backup domain reset)
01: LSE selected as RTC clock
10: LSI selected as RTC clock
11: HSE divided by RTCPRE value selected as RTC clock
- Bit 7 **LSEEXT**: low-speed external clock type in bypass mode
Set and reset by software to select the external clock type (analog or digital).
The external clock must be enabled with the LSEON bit, to be used by the device.
The LSEEXT bit can be written only if the LSE oscillator is disabled.
0: LSE in analog mode (default after Backup domain reset)
1: LSE in digital mode
Note: The default value after a Backup domain reset is 1 for the STM32H543/553xx.
- Bit 6 **LSECSSD**: LSE clock security system failure detection
Set by hardware to indicate when a failure has been detected by the clock security system on the external 32 kHz oscillator.
0: no failure detected on 32 kHz oscillator (default after Backup domain reset)
1: failure detected on 32 kHz oscillator
- Bit 5 **LSECSSON**: LSE clock security system enable
Set by software to enable the clock security system on 32 kHz oscillator.
LSECSSON must be enabled after LSE is enabled (LSEON enabled) and ready (LSERDY set by hardware) and after RTCSEL is selected.
Once enabled, this bit cannot be disabled, except after a LSE failure detection (LSECSSD = 1). In that case the software must disable LSECSSON.
0: CSS on 32 kHz oscillator OFF (default after Backup domain reset)
1: CSS on 32 kHz oscillator ON

Bits 4:3 **LSEDRV[1:0]**: LSE oscillator driving capability

Set by software to select the driving capability of the LSE oscillator.

These bit can be written only if LSE oscillator is disabled (LSEON = 0 and LSERDY = 0).

00: lowest drive (default after Backup domain reset)

01: medium-low drive

10: medium-high drive

11: highest drive

Bit 2 **LSEBYP**: LSE oscillator bypass

Set and reset by software to bypass oscillator in debug mode. This bit must not be written when the LSE is enabled (by LSEON) or ready (LSERDY = 1)

0: LSE oscillator not bypassed (default after Backup domain reset)

1: LSE oscillator bypassed

Bit 1 **LSERDY**: LSE oscillator ready

Set and reset by hardware to indicate when the LSE is stable.

This bit needs 6 cycles of lse_ck clock to fall down after LSEON has been set to 0.

0: LSE oscillator not ready (default after Backup domain reset)

1: LSE oscillator ready

Bit 0 **LSEON**: LSE oscillator enabled

Set and reset by software.

0: LSE oscillator OFF (default after Backup domain reset)

1: LSE oscillator ON

11.8.46 RCC reset status register (RCC_RSR)

Address offset: 0x0F4

Reset value: 0x0C00 0000

Reset by power-on reset only.

Access: $0 \leq \text{wait state} \leq 3$, word, half-word and byte access.

Wait states are inserted in case of successive accesses to this register.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
LPWR RSTF	WWDG RSTF	IWDG RSTF	SFTR STF	BORR STF	PINRS TF	Res.	Res.	RMVF	Res.	Res.	Res.	Res.	Res.	Res.	Res.
rw	rw	rw	rw	rw	rw			rw							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.

Bit 31 **LPWRRSTF**: Low-power reset flag

Set by hardware when a reset occurs due to Stop or Standby mode entry, whereas the corresponding nRST_STOP, nRST_STBY option bit is cleared.

Cleared by writing to the RMVF bit.

0: No illegal low-power mode reset occurred

1: Illegal low-power mode reset occurred

- Bit 30 **WWDGRSTF**: window watchdog reset flag
 Reset by software by writing the RMVF bit.
 Set by hardware when a window watchdog reset occurs.
 0: no window watchdog reset occurred from WWDG (default after power-on reset)
 1: window watchdog reset occurred from WWDG
- Bit 29 **IWDGRSTF**: independent watchdog reset flag
 Reset by software by writing the RMVF bit.
 Set by hardware when an independent watchdog reset occurs.
 0: no independent watchdog reset occurred (default after power-on reset)
 1: independent watchdog reset occurred
- Bit 28 **SFTRSTF**: system reset from CPU reset flag
 Reset by software by writing the RMVF bit.
 Set by hardware when the system reset is due to CPU. The CPU can generate a system reset by writing SYSRESETREQ bit of AIRCR register of the core M33.
 0: no CPU software reset occurred (default after power-on reset)
 1: a system reset has been generated by the CPU
- Bit 27 **BORRSTF**: BOR reset flag
 Reset by software by writing the RMVF bit.
 Set by hardware when a BOR reset occurs (pwr_bor_rst).
 0: no BOR reset occurred
 1: BOR reset occurred (default after power-on reset)
- Bit 26 **PINRSTF**: pin reset flag (NRST)
 Reset by software by writing the RMVF bit.
 Set by hardware when a reset from pin occurs.
 0: no reset from pin occurred
 1: reset from pin occurred (default after power-on reset)
- Bits 25:24 Reserved, must be kept at reset value.
- Bit 23 **RMVF**: remove reset flag
 Set and reset by software to reset the value of the reset flags.
 0: reset of the reset flags not activated (default after power-on reset)
 1: resets the value of the reset flags
- Bits 22:0 Reserved, must be kept at reset value.

11.8.47 RCC secure configuration register (RCC_SECCFGR)

Address offset: 0x110

Reset value: 0x0000 0000

Access: no wait state; word, half-word and byte access

When the system is secure (TZEN = 0xB4), this register can be written only by a secure privileged access if SPRIV = 1, and by a secure privileged or unprivileged access if SPRIV = 0. A non-secure write access generates an illegal access event and data is not written. This register can be read by secure or non-secure, privilege or unprivileged access.

When the system is not secure (TZEN = 0xC3), this register is read as 0 and the register write is ignored.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	CKPER SELSEC C	RMVFS EC	HSI48S EC	Res.	PLL3S EC	PLL2S EC	PLL1S EC	PRESC SEC	SYSCLK SEC	LSESEC C	LSISE C	CSISE C	HSESEC C	HSISE C
		rw	rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:14 Reserved, must be kept at reset value.

Bit 13 **CKPERSELSEC**: per_ck selection security

Set and reset by software.

0: non secure

1: secure

Bit 12 **RMVFSEC**: Remove reset flag security

Set and reset by software.

0: non secure

1: secure

Bit 11 **HSI48SEC**: HSI48 clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 10 Reserved, must be kept at reset value.

Bit 9 **PLL3SEC**: PLL3 clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Note: This bit is not available on STM32H543/553xx devices. If not present, consider this bit as reserved, and keep its bits at reset value.

Bit 8 **PLL2SEC**: PLL2 clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 7 **PLL1SEC**: PLL1 clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 6 **PRESCSEC**: AHBx/APBx prescaler configuration bits security

Set and reset by software.

0: non secure

1: secure

Bit 5 **SYCLKSEC**: SYCLK clock selection, STOPWUCK bit, clock output on MCO configuration security

Set and reset by software.

0: non secure

1: secure

Bit 4 **LSESEC**: LSE clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 3 **LSISEC**: LSI clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 2 **CSISEC**: CSI clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

Bit 1 **HSESEC**: HSE clock configuration bits, status bits and HSE_CSS security

Set and reset by software.

0: non secure

1: secure

Bit 0 **HSISEC**: HSI clock configuration and status bits security

Set and reset by software.

0: non secure

1: secure

11.8.48 RCC privilege configuration register (RCC_PRIVCFGR)

Address offset: 0x114

Reset value: 0x0000 0000

Access: no wait state; word, half-word and byte access

This register can be written only by a privileged access. It can be read by privileged or unprivileged access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV	SPRIV
														rw	rw

Bits 31:2 Reserved, must be kept at reset value.

Bit 1 NSPRIV: RCC non-secure functions privilege configuration

Set and reset by software. This bit can be written only by privileged access, secure or non-secure.

0: Read and write to RCC non-secure functions can be done by privileged or unprivileged access.

1: Read and write to RCC non-secure functions can be done by privileged access only

Bit 0 SPRIV: RCC secure functions privilege configuration

Set and reset by software. This bit can be written only by a secure privileged access.

0: Read and write to RCC secure functions can be done by privileged or unprivileged access.

1: Read and write to RCC secure functions can be done by privileged access only

11.9 RCC register map

Table 124. RCC register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x000	RCC_CR	Res.	Res.	PLL3RDY ⁽¹⁾	PLL3ON ⁽¹⁾	PLL2RDY	PLL2ON	PLL1RDY	PLL1ON	Res.	Res.	Res.	HSEEXT	HSECSSON	HSEBYP	HSERDY	HSEON	Res.	Res.	HSI48RDY	HSI48ON	Res.	CSIKERON	CSIRDY	CSION	Res.	Res.	Res.	HSIDIVF	HSIDIV	[1:0]	HSIKERON	HSIRDY	HSION
	Reset value			0	0	0	0	0	0				0	0	0	0	0			0	0		0	0	0				1	0	1	0	1	1
0x004-0x00C	Reserved																																	
0x010	RCC_HSICFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSITRIM[6:0]						Res.	Res.	Res.	Res.	HSICAL[11:0]													
	Reset value										1	0	0	0	0	0	0						X	X	X	X	X	X	X	X	X	X	X	X
0x014	RCC_CRRCR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	HSI48CAL[9:0]												
	Reset value																								X	X	X	X	X	X	X	X	X	
0x018	RCC_CSICFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSITRIM[5:0]						Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CSICAL[7:0]							
	Reset value											1	0	0	0	0	0										X	X	X	X	X	X	X	X
0x01C	RCC_CFGR1	MCO2SEL[2:0]			MCO2PRE[3:0]				MCO1SEL[2:0]			MCO1PRE[3:0]				Res.	Res.	RTCPRE[5:0]					STOPKERWUCK	STOPWUCK	Res.	SWS	[1:0]	Res.	SW	[1:0]				
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0			0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x020	RCC_CFGR2	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	APB3DIS	APB2DIS	APB1DIS	AHB4DIS	Res.	AHB2DIS	AHB1DIS	Res.	PPRE3	[2:0]	Res.	PPRE2	[2:0]	Res.	PPRE1	[2:0]	Res.	HPRE	[3:0]					
	Reset value										0	0	0	0		0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x024	Reserved																																	
0x028	RCC_PLL1CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL1M[5:0]					Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																			0	0	0	0	0	0			0	0	0	0	0	0	
0x02C	RCC_PLL2CFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL2M[5:0]					Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																			0	0	0	0	0	0			0	0	0	0	0	0	
0x030	RCC_PLL3CFGR ⁽¹⁾	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLL3M[5:0]					Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value																			0	0	0	0	0	0			0	0	0	0	0	0	
0x034	RCC_PLL1DIVR	Res.	PLL1R[6:0]						Res.	PLL1Q[6:0]						PLL1P[6:0]						PLL1N[8:0]												
	Reset value		0	0	0	0	0	0	1		0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0

Table 124. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x038	RCC_PLL1FRACR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PLL1FRACN[12:0]												Res	Res	Res	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x03C	RCC_PLL2DIVR	Res	PLL2R[6:0]						Res	PLL2Q[6:0]						PLL2P[6:0]						PLL2N[8:0]											
	Reset value		0	0	0	0	0	0	0	1		0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0
0x040	RCC_PLL2FRACR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PLL2FRACN[12:0]												Res	Res	Res	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x044	RCC_PLL3DIVR ⁽¹⁾	Res	PLL3R[6:0]						Res	PLL3Q[6:0]						PLL3P[6:0]						PLL3N[8:0]											
	Reset value		0	0	0	0	0	0	1		0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x048	RCC_PLL3FRACR ⁽¹⁾	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	PLL3FRACN[12:0]												Res	Res	Res	
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0			
0x050	RCC_CIER	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x054	RCC_CIFR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x058	RCC_CICR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x060	RCC_AHB1RSTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value												0	ETHRST ⁽²⁾	0	RAMCFGIRST	0	FMACRST ⁽²⁾	0	CORDICRST ⁽²⁾	0	CRCRST	0	DCMI	PSSIRST ⁽²⁾	0	DAC1RST	0	ADCRST	0	GPIO1RST ⁽²⁾	0	GPDMA2RST
0x064	RCC_AHB2RSTR	Res	Res	Res	Res	Res	Res	Res	ADC3RST ⁽²⁾	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value							0			0	CCBRST ⁽²⁾	0	SAESRST	0	PKARST	0	RNGRST	0	HASHRST	0	AESRST	0	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
0x06C	RCC_AHB4RSTR	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res
	Reset value												0	OCTOSPI1RST	0	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res	Res

Table 124. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x074	RCC_APB1LRSTR	UART8RST ⁽²⁾	UART7RST ⁽²⁾	Res.	CECRST	USART11 ⁽²⁾	USART10RST ⁽²⁾	USART6RST	CRSRST	I3C1RST	I2C2RST	I2C1RST	UART5RST	UART4RST	USART3RST	USART2RST	COMRST ⁽²⁾	SPI3RST	SPI2RST	Res.	Res.	Res.	Res.	Res.	Res.	TIM14RST ⁽²⁾	TIM13RST ⁽²⁾	TIM12RST	TIM7RST	TIM6RST	TIM5RST	TIM4RST	TIM3RST	TIM2RST
	Reset value	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						0	0	0	0	0	0	0	0	0
0x078	RCC_APB1HRSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD1RST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANRST	Res.	Res.	Res.	Res.	LPTIM2RST	DTSRST	Res.	UART12RST ⁽²⁾	UART9RST ⁽²⁾	
	Reset value									0														0					0			0	0	0
0x07C	RCC_APB2RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBRST	Res.	SAI2RST ⁽²⁾	SAI1RST ⁽²⁾	SPI6RST ⁽²⁾	SPI4RST	TIM17RST ⁽²⁾	TIM16RST ⁽²⁾	TIM15RST	Res.	USART1RST	TIM8RST	SPI1RST	TIM1RST	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value								0		0	0	0	0	0	0	0		0	0	0	0												
0x080	RCC_APB3RSTR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1RST ⁽²⁾	PLAY1POR ⁽²⁾	Res.	VREFRST	Res.	Res.	Res.	Res.	LPTIM6RST ⁽²⁾	LPTIM5RST ⁽²⁾	LPTIM4RST ⁽²⁾	LPTIM3RST ⁽²⁾	LPTIM1RST	Res.	Res.	I3C2RST ⁽²⁾	I2C4RST ⁽²⁾	I2C3RST	LPUART1RST	SPI5RST ⁽²⁾	Res.	Res.	Res.	Res.	
	Reset value									0	0		0	0				0	0	0	0	0			0	0	0	0	0					
0x088	RCC_AHB1ENR	SRAM1EN	DCACHEEN	Res.	BKPRAMEN	Res.	Res.	Res.	TZSC1EN	Res.	Res.	ETHRXEN ⁽²⁾	ETHTXEN ⁽²⁾	ETHEN ⁽²⁾	ETHCKEN ⁽³⁾	RAMCFGEN	Res.	FMACEN ⁽²⁾	CORDICEN ⁽²⁾	Res.	CRCEEN	Res.	Res.	Res.	FLITFEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	GPDMA2EN	GPDMA1EN
	Reset value	1	1		1				0			0	0	0	0	0		0	0		0				1							0	0	0
0x08C	RCC_AHB2ENR	SRAM3EN	SRAM2EN	Res.	Res.	Res.	Res.	Res.	ADC3EN ⁽²⁾	Res.	Res.	CCBEN ⁽²⁾	SAESEN	PKAEN	RNGEN	HASHEN	AESEN	Res.	Res.	Res.	DCMI_PSSIEN ⁽²⁾	DAC1EN	ADCEN	Res.	GPIOEN ⁽²⁾	GPIOHEN	GPIOGEN	GPIOFEN	GPIOEEN	GPIODEN	GPIOCEN	GPIOBEN	GPIOAEN	
	Reset value	1	1						0			0	0	0	0	0	0				0	0	0		0	0	0	0	0	0	0	0	0	0
0x094	RCC_AHB4ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OCTOSP1EN	Res.	Res.	Res.	FMCEN	Res.	Res.	Res.	SDMMC2EN ⁽²⁾	SDMMC1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value												0				0				0	0					0							
0x09C	RCC_APB1LENR	UART8EN ⁽²⁾	UART7EN ⁽²⁾	Res.	CECEN	USART11 ⁽²⁾	USART10EN ⁽²⁾	USART6EN	CRSEN	I3C1EN	I2C2EN	I2C1EN	UART5EN	UART4EN	USART3EN	USART2EN	COMPEN ⁽²⁾	SPI3EN	SPI2EN	Res.	Res.	Res.	Res.	Res.	TIM14EN ⁽²⁾	TIM13EN ⁽²⁾	TIM12EN	TIM7EN	TIM6EN	TIM5EN	TIM4EN	TIM3EN	TIM2EN	
	Reset value	0	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0						0	0	0	0	0	0	0	0	0	0
0x0A0	RCC_APB1HENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	UCPD1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANEN	Res.	Res.	Res.	Res.	LPTIM2EN	DTSEN	Res.	UART12EN ⁽²⁾	UART9EN ⁽²⁾	
	Reset value									0														0				0				0	0	0

Table 124. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0A4	RCC_APB2ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBN		SAI2EN ⁽²⁾	SAI1EN ⁽²⁾	SP16EN ⁽²⁾	SP14EN	TIM17EN ⁽²⁾	TIM16EN ⁽²⁾	TIM15EN	Res.	USART1EN	TIM8EN	SP11EN	TIM1EN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								0		0	0	0	0	0	0	0		0	0	0	0												
0x0A8	RCC_APB3ENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1EN ⁽²⁾	PLAY1APBEN ⁽²⁾	RTCAPBEN	VREFEN	Res.	Res.	Res.	Res.	LPTIM6EN ⁽²⁾	LPTIM5EN ⁽²⁾	LPTIM4EN ⁽²⁾	LPTIM3EN ⁽²⁾	LPTIM1EN	Res.	I3C2EN ⁽²⁾	I2C4EN ⁽²⁾	I2C3EN	LPUART1EN	SPI5EN ⁽²⁾			Res.	SBSEN	Res.	Res.
	Reset value									0	0	0	0					0	0	0	0	0		0	0	0	0	0				0		
0x0B0	RCC_AHB1LPENR	SRAM1LPEN	DCACHELPEN	ICACHELPEN	BKPRAMLLEN		Res.	Res.	TZSC1LPEN	Res.		ETHRXLPEN ⁽²⁾	ETHTXLPEN ⁽²⁾	ETHLPEN ⁽²⁾	ETHCKLPEN ⁽³⁾	RAMCFGLPEN	Res.	FMACLPEN ⁽²⁾	CORDICLPEN ⁽²⁾	Res.	CRCLPEN	Res.	Res.	Res.	FLITFLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	1	1	1	1				1			1	1	1	1	1		1	1		1				1								1	1
0x0B4	RCC_AHB2LPENR	SRAM3LPEN	SRAM2LPEN	Res.	Res.	Res.	Res.	Res.	ADC3LPEN ⁽²⁾	Res.	Res.	CCBLPEN ⁽²⁾	SAESLPEN	PKALPEN	RNGLPEN	HASHLPEN	AESLPEN	Res.	Res.	Res.	Res.	DCMI PSSILPEN ⁽²⁾	DAC1LPEN	ADCLPEN	Res.	GPIOLPEN ⁽²⁾	GPIOHLPEN	GPIOGLPEN	GPIOFLPEN	GPIOELPEN	GPIODLPEN	GPIOCLPEN	GPIOBLPEN	GPIOALPEN
	Reset value	1	1						1			1	1	1	1	1	1					1	1	1		1	1	1	1	1	1	1	1	1
0x0BC	RCC_AHB4LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OCTOSP11LPEN	Res.	Res.	Res.	FMCLPEN	Res.	Res.	Res.	SDMMC2LPEN ⁽²⁾	SDMMC1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value												1				1				1	1				1								
0x0C4	RCC_APB1LLPENR	UART8LPEN ⁽²⁾	UART7LPEN ⁽²⁾		CECLPEN	USART11LPEN ⁽²⁾	USART10LPEN ⁽²⁾	USART6LPEN	CRSLPEN	I3C1LPEN	I2C2LPEN	I2C1LPEN	UART5LPEN	UART4LPEN	USART3LPEN	USART2LPEN	COMPLPEN ⁽²⁾	SPI3LPEN	SPI2LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	1	1		1	1	1	1	1	1	1	1	1	1	1	1	1	1	1							1	1	1	1	1	1	1	1	1
0x0C8	RCC_APB1HLPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.		UCPD1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	FDCANLPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value									1														1										
0x0CC	RCC_APB2LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	USBLPEN	Res.	SAI2LPEN ⁽²⁾	SAI1LPEN ⁽²⁾	SP16LPEN ⁽²⁾	SP14LPEN	TIM17LPEN ⁽²⁾	TIM16LPEN ⁽²⁾	TIM15LPEN	USART11LPEN	TIM8LPEN	SP11LPEN	TIM1LPEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value								1		1	1	1	1	1	1	1	1	1	1	1	1												

Table 124. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x0D0	RCC_APB3LPENR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	PLAY1LPEN	Res.	RTCAPBLPEN	VREFLPEN	Res.	Res.	Res.	Res.	LPTIM6LPEN ⁽²⁾	LPTIM5LPEN ⁽²⁾	LPTIM4LPEN ⁽²⁾	LPTIM3LPEN ⁽²⁾	LPTIM1LPEN	Res.	I3C2LPEN ⁽⁴⁾	I2C4LPEN ⁽²⁾	I2C3LPEN	LPUART1LPEN	SPI5LPEN ⁽²⁾	Res.	Res.	SBSLPEN	Res.		
	Reset value									1		1	1					1	1	1	1	1		0	1	1	1	1	1			1		
0x0D8	RCC_CCIPR1	TIMCSEL	Res.	USART10SEL [2:0] ⁽⁵⁾			UART9SEL [2:0] ⁽⁵⁾			UART8SEL [2:0] ⁽⁵⁾			UART7SEL [2:0] ⁽⁵⁾			USART6SEL [2:0] ⁽⁵⁾			UART5SEL [2:0]			UART4SEL [2:0]			USART3SEL [2:0]			USART2SEL [2:0]			USART1SEL [2:0]			
	Reset value	0		0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0DC	RCC_CCIPR2	Res.	LPTIM6SEL [2:0] ⁽⁵⁾			Res.	LPTIM5SEL [2:0] ⁽⁵⁾			Res.	LPTIM4SEL [2:0] ⁽⁵⁾			Res.	LPTIM3SEL [2:0] ⁽⁵⁾			Res.	LPTIM2SEL [2:0]			Res.	LPTIM1SEL [2:0]			Res.	UART12SEL [2:0] ⁽⁵⁾			USART11SEL [2:0] ⁽⁵⁾				
	Reset value		0	0	0		0	0	0		0	0	0		0	0	0		0	0	0		0	0	0		0	0	0		0	0	0	
0x0E0	RCC_CCIPR3	PLAY1SEL [2:0] ⁽⁵⁾			Res.	Res.	LPUART1SEL [2:0]			PLAY1DIV SEL[2:0] ⁽⁵⁾			Res.	Res.	Res.	SPI6SEL [2:0] ⁽⁵⁾			SPI5SEL [2:0] ⁽⁵⁾			SPI4SEL [2:0]			SPI3SEL [2:0]			SPI2SEL [2:0]			SPI1SEL [2:0]			
	Reset value	0	0	0			0	0	0	0	0	0	0				0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0x0E4	RCC_CCIPR4	ETHPTPSEL [3:0] ⁽³⁾			I3C2SEL [1:0] ⁽⁵⁾			I3C1SEL [1:0]			I2C4SEL [1:0] ⁽⁵⁾			I2C3SEL [1:0]			I2C2SEL [1:0]			I2C1SEL [1:0]			Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0									0	0	0	0	0	0	0	0
0x0E8	RCC_CCIPR5	CKPERSEL [1:0]			Res.	Res.	Res.	Res.	Res.	Res.	ETHREFCLKSEL ⁽³⁾			SAI2SEL [2:0] ⁽⁵⁾			SAI1SEL [2:0] ⁽⁵⁾			Res.	Res.	Res.	ETHPTPCLKSEL[1:0] ⁽³⁾			FDCANSEL [1:0]			CECSEL [1:0]			RNGSEL [1:0]		
	Reset value	0	0								0			0	0	0	0	0					0	0	0	0	0	0	0	0	0	0	0	0
0x0F0	RCC_BDCR	Res.	Res.	Res.	Res.	LSIRDY	LSION	LSCOESEL	LSCOEN	Res.	Res.	Res.	Res.	Res.	Res.	Res.	VSWRST	RTCEN	Res.	Res.	Res.	Res.	Res.	Res.	RTCSEL [1:0]	LSEEXT ⁽⁶⁾	LSECSSD	LSECSSON	LSEDRV [1:0]	LSEBYP	LSEBYP	LSEBYP	LSEON	
	Reset value					0	0	0	0								0	0							0	0	0	0	0	0	0	0	0	0
0x0F4	RCC_RSR	LPWRSTF	WWDGRSTF	IWDGRSTF	SFTRSTF	BORRSTF	PINRSTF	Res.	Res.	RMVF	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	
	Reset value	0	0	0	0	1	1			0																								

Table 124. RCC register map and reset values (continued)

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x110	RCC_SECCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	CKPERSELSEC	RMVFSEC	HSI48SEC	Res.	PLL3SEC ⁽¹⁾	PLL2SEC	PLL1SEC	PRESCSEC	SYSCCLKSEC	LSESEC	LSISEC	CSISEC	HSESEC	HSISEC
	Reset value																			0	0	0		0	0	0	0	0	0	0	0	0	0
0x114	RCC_PRIVCFGR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	NSPRIV	SPRI
	Reset value																															0	0

1. Not Available on STM32H543/553xx devices. If not present, consider it as reserved, and keep it at reset value.
2. For the actual availability of this bit check the datasheet. If not present, consider it as reserved, and keep it at reset value.
3. Available only on STM32H543/553xx devices. If not present, consider it as reserved, and keep it at reset value.
4. The Reset value is 1 for STM32H523/533/543/553xx devices.
5. For the actual availability of these bits check the datasheet. If not present, consider them as reserved, and keep them at reset value.
6. The reset value is 1 for STM32H543/553xx devices.

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.

12 Clock recovery system (CRS)

12.1 CRS introduction

The clock recovery system (CRS) is an advanced digital controller acting on the internal fine-granularity trimmable RC oscillator HSI48. The CRS provides powerful means to evaluate the oscillator output frequency, based on comparison with a selectable synchronization signal. The CRS can perform automatic trimming adjustments based on the measured frequency error value, while keeping the possibility of a manual trimming.

The CRS is ideally suited to provide a precise clock to the USB peripheral. In this case, the synchronization signal can be derived from the start-of-frame (SOF) packet signalization on the USB bus, sent by a USB host at 1 ms intervals.

The synchronization signal can also be derived from alternative synchronization or generated by the user software.

12.2 CRS main features

- Selectable synchronization source with programmable prescaler and polarity (see [Section 12.4.2: CRS internal signals](#))
- Possibility to generate synchronization pulses by software
- Automatic oscillator trimming capability with no need of CPU action
- Manual control option for faster startup convergence
- 16-bit frequency error counter with automatic error value capture and reload
- Programmable limit for automatic frequency error value evaluation and status reporting
- Maskable interrupts/events:
 - Expected synchronization (ESYNC)
 - Synchronization OK (SYNCOK)
 - Synchronization warning (SYNCWARN)
 - Synchronization or trimming error (ERR)

12.3 CRS implementation

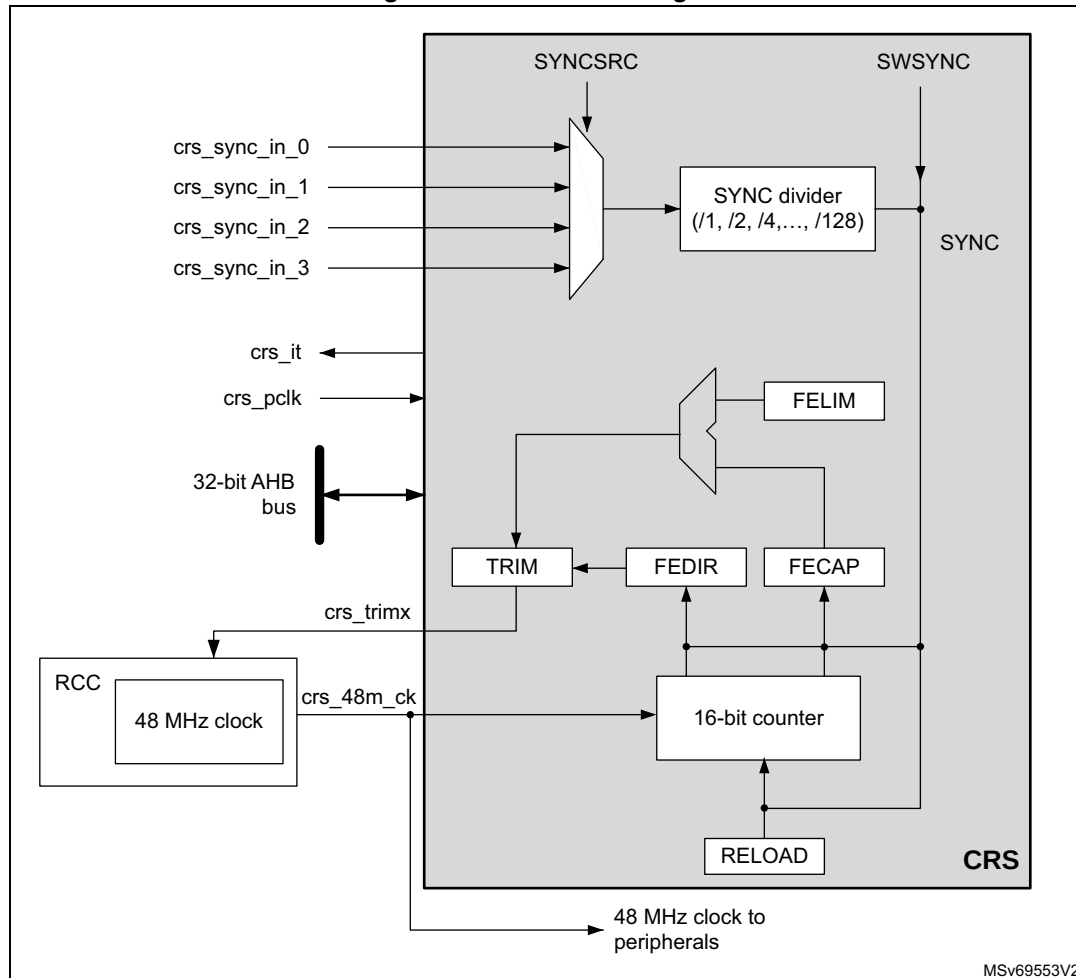
Table 125. CRS features

Feature	CRS
TRIM width	6 bits

12.4 CRS functional description

12.4.1 CRS block diagram

Figure 62. CRS block diagram



12.4.2 CRS internal signals

Below the list of CRS internal signals.

Table 126. CRS internal input/output signals

Internal signal name	Signal type	Description
crs_it	Digital output	CRS interrupt
crs_pclk	Digital input	AHB bus clock
crs_48m_ck	Digital input	HSI48 48 MHz clock
crs_trim[0:5]	Digital output	HSI48 oscillator smooth trimming value

Table 126. CRS internal input/output signals (continued)

Internal signal name	Signal type	Description
crs_sync_in_0	Digital input	SYNC signal source
crs_sync_in_1		
crs_sync_in_2		
crs_sync_in_3		

Table 127. CRS interconnection

Internal signal name	Description
crs_sync_in_0	GPIO selected as SYNC signal source
crs_sync_in_1	LSE selected as SYNC signal source
crs_sync_in_2	USB SOF selected as SYNC signal source (default)
crs_sync_in_3	Reserved

12.4.3 Synchronization input

The CRS synchronization source (crs_sync_in_x) can be selected through SYNCSRC[1:0] bitfield of CRS_CFGR register (refer to [Section 12.4.2: CRS internal signals](#) for the possible sources). For a better robustness of the crs_sync_in_x input, a simple digital filter (2 out of 3 majority votes, sampled by the 48 MHz clock) is implemented to filter out glitches. In addition, the source signal has a configurable polarity (selected through SYNCPOL bit of CRS_CFGR). The signal can then be divided by a programmable binary prescaler to obtain a synchronization signal in a suitable frequency range (usually around 1 kHz).

For more information on the CRS synchronization source configuration, refer to CRS_CFGR register.

It is also possible to generate a synchronization event by software, by setting the SWSYNC bit in the CRS_CR register.

For more information on the CRS synchronization source configuration, refer to [CRS configuration register \(CRS_CFGR\)](#).

It is also possible to generate a synchronization event by software, by setting the SWSYNC bit in the CRS_CR register.

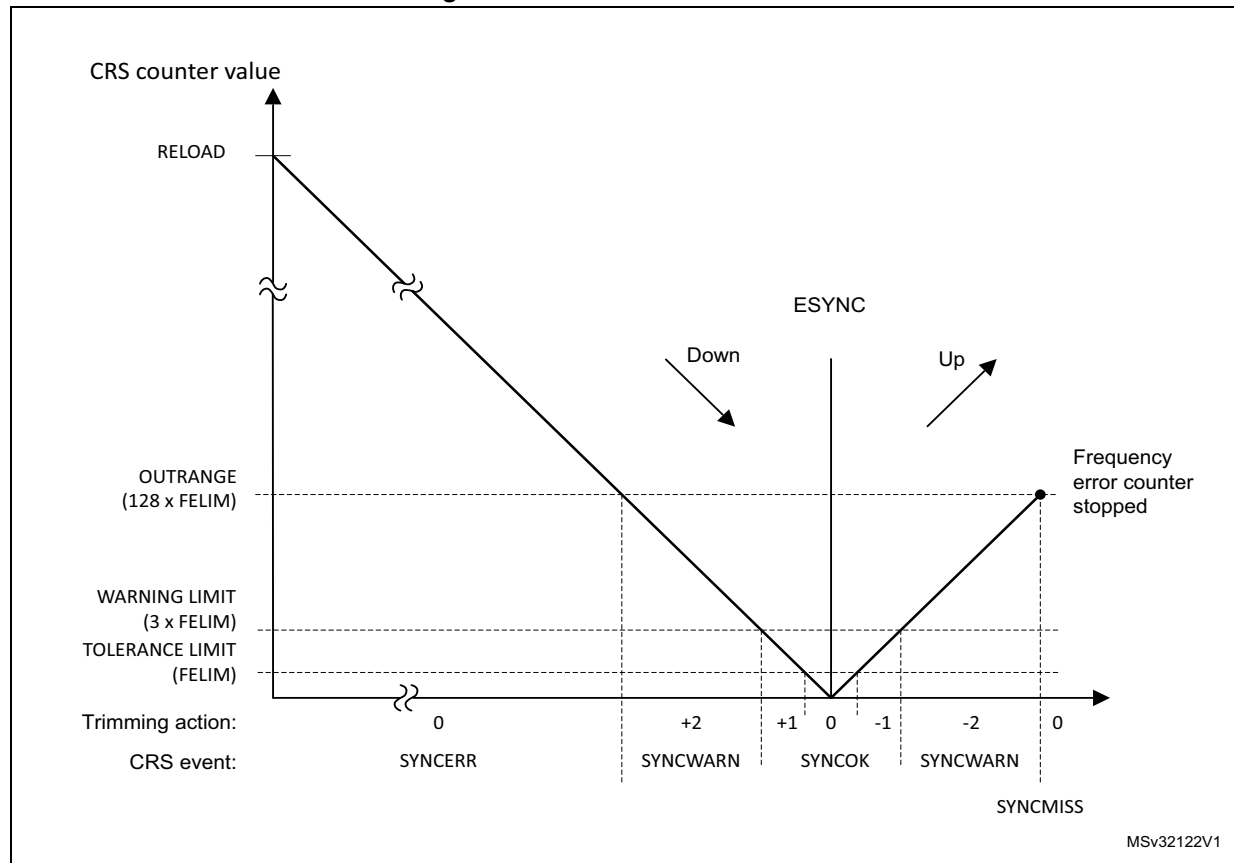
12.4.4 Frequency error measurement

The frequency error counter is a 16-bit down/up counter, reloaded with the RELOAD value on each SYNC event. It starts counting down until it reaches the zero value, where the ESYNC (expected synchronization) event is generated. Then it starts counting up to the OUTRANGE limit, where it eventually stops (if no SYNC event is received), and generates a SYNCMISS event. The OUTRANGE limit is defined as the frequency error limit (FELIM field of the CRS_CFGR register) multiplied by 128.

When the SYNC event is detected, the actual value of the frequency error counter and its counting direction are stored in the FECAP (frequency error capture) field and in the FEDIR (frequency error direction) bit of the CRS_ISR register. When the SYNC event is detected during the down-counting phase (before reaching the zero value), it means that the actual

frequency is lower than the target (the TRIM value must be incremented). When it is detected during the up-counting phase, it means that the actual frequency is higher (the TRIM value must be decremented).

Figure 63. CRS counter behavior



12.4.5 Frequency error evaluation and automatic trimming

The measured frequency error is evaluated by comparing its value with a set of limits:

- TOLERANCE LIMIT, given directly in the FELIM field of the CRS_CFGR register
- WARNING LIMIT, defined as $3 \times \text{FELIM}$ value
- OUTRANGE (error limit), defined as $128 \times \text{FELIM}$ value

The result of this comparison is used to generate the status indication and also to control the automatic trimming, which is enabled by setting the AUTOTRIMEN bit in the CRS_CR register:

- When the frequency error is below the tolerance limit, it means that the actual trimming value in the TRIM field is the optimal one, hence no trimming action is needed.
 - SYNCOK status indicated
 - TRIM value not changed in AUTOTRIM mode
- When the frequency error is below the warning limit but above or equal to the tolerance limit, it means that some trimming action is necessary but that adjustment by one trimming step is enough to reach the optimal TRIM value.
 - SYNCOK status indicated

- TRIM value adjusted by one trimming step in AUTOTRIM mode
- When the frequency error is above or equal to the warning limit but below the error limit, a stronger trimming action is necessary, and there is a risk that the optimal TRIM value is not reached for the next period.
 - SYNCWARN status indicated
 - TRIM value adjusted by two trimming steps in AUTOTRIM mode
- When the frequency error is above or equal to the error limit, the frequency is out of the trimming range. This can also happen when the SYNC input is not clean, or when some SYNC pulse is missing (for example when one USB SOF is corrupted).
 - SYNCERR or SYNCMISS status indicated
 - TRIM value not changed in AUTOTRIM mode

Note: *If the actual value of the TRIM field is close to its limits and the automatic trimming can force it to overflow or underflow, the TRIM value is set to the limit, and the TRIMOVF status is indicated.*

In AUTOTRIM mode (AUTOTRIMEN bit set in the CRS_CR register), the TRIM field of CRS_CR is adjusted by hardware and is read-only.

12.4.6 CRS initialization and configuration

RELOAD value

The RELOAD value must be selected according to the ratio between the target frequency and the frequency of the synchronization source after prescaling. This value is decreased by 1, to reach the expected synchronization on the zero value. The formula is the following:

$$\text{RELOAD} = (f_{\text{TARGET}} / f_{\text{SYNC}}) - 1$$

The reset value of the RELOAD field corresponds to a target frequency of 48 MHz and a synchronization signal frequency of 1 kHz (SOF signal from USB).

FELIM value

The selection of the FELIM value is closely coupled with the HSI48 oscillator characteristics and its typical trimming step size. The optimal value corresponds to half of the trimming step size, expressed as a number of oscillator clock ticks. The following formula can be used:

$$\text{FELIM} = (f_{\text{TARGET}} / f_{\text{SYNC}}) * \text{STEP}[\%] / 100\% / 2$$

The result must be always rounded up to the nearest integer value to obtain the best trimming response. If frequent trimming actions are not needed in the application, the hysteresis can be increased by slightly increasing the FELIM value.

The reset value of the FELIM field corresponds to $(f_{\text{TARGET}} / f_{\text{SYNC}}) = 48000$, and to a typical trimming step size of 0.14%.

Note: *The trimming step size depends upon the product, check the datasheet for accurate setting.*

Caution: There is no hardware protection from a wrong configuration of the RELOAD and FELIM fields, this can lead to an erratic trimming response. The expected operational mode requires proper setup of the RELOAD value (according to the synchronization source frequency), which is also greater than $128 * \text{FELIM}$ value (OUTRANGE limit).

12.5 CRS in low-power modes

Table 128. Effect of low-power modes on CRS

Mode	Description
Sleep	No effect. CRS interrupts cause the device to exit the Sleep mode.
Stop	CRS registers are frozen. The CRS stops operating until the Stop mode is exited and the HSI48 oscillator is restarted.
Standby	The peripheral is powered down and must be reinitialized after exiting Standby mode.
Shutdown	The peripheral is powered down and must be reinitialized after exiting Shutdown mode.

12.6 CRS interrupts

Table 129. Interrupt control bits

Interrupt event	Event flag	Enable control bit	Clear flag bit
Expected synchronization	ESYNCF	ESYNCIE	ESYNCC
Synchronization OK	SYNCOKF	SYNCOKIE	SYNCOKC
Synchronization warning	SYNCWARNF	SYNCWARNIE	SYNCWARNC
Synchronization or trimming error (TRIMOVF, SYNCMISS, SYNCERR)	ERRF	ERRIE	ERRC

12.7 CRS registers

Refer to [Section 1.2](#) for a list of abbreviations used in register descriptions.

The peripheral registers can be accessed only by words (32-bit).

12.7.1 CRS control register (CRS_CR)

Address offset: 0x00

Reset value: 0x0000 2000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	TRIM[5:0]						SW SYNC	AUTO TRIMEN	CEN	Res.	ESYNC IE	ERR IE	SYNC WARNIE	SYNC OKIE
		rw	rw	rw	rw	rw	rw	rt_w1	rw	rw		rw	rw	rw	rw

Bits 31:14 Reserved, must be kept at reset value.

Bits 13:8 TRIM[5:0]: HSI48 oscillator smooth trimming

These bits provide a user-programmable trimming value to the HSI48 oscillator. They can be programmed to adjust to variations in voltage and temperature that influence the oscillator frequency.

The default value of the HSI48 oscillator smooth trimming is 32, for a complete TRIM range of 64 steps.

The trimming step is specified in the product datasheet. A higher TRIM value corresponds to a higher output frequency.

When the AUTOTRIMEN bit is set, this field is controlled by hardware and is read-only.

Bit 7 SWSYNC: Generate software SYNC event

This bit is set by software in order to generate a software SYNC event. It is automatically cleared by hardware.

0: No action

1: A software SYNC event is generated.

Bit 6 AUTOTRIMEN: Automatic trimming enable

This bit enables the automatic hardware adjustment of TRIM bits according to the measured frequency error between two SYNC events. If this bit is set, the TRIM bits are read-only. The TRIM value can be adjusted by hardware by one or two steps at a time, depending on the measured frequency error value. Refer to [Section 12.4.5](#) for more details.

0: Automatic trimming disabled, TRIM bits can be adjusted by the user.

1: Automatic trimming enabled, TRIM bits are read-only and under hardware control.

Bit 5 CEN: Frequency error counter enable

This bit enables the oscillator clock for the frequency error counter.

0: Frequency error counter disabled

1: Frequency error counter enabled

When this bit is set, the CRS_CFGR register is write-protected and cannot be modified.

Bit 4 Reserved, must be kept at reset value.**Bit 3 ESYNCF:** Expected SYNC interrupt enable

0: Expected SYNC (ESYNCF) interrupt disabled

1: Expected SYNC (ESYNCF) interrupt enabled

Bit 2 ERRF: Synchronization or trimming error interrupt enable

0: Synchronization or trimming error (ERRF) interrupt disabled

1: Synchronization or trimming error (ERRF) interrupt enabled

Bit 1 SYNCWARNF: SYNC warning interrupt enable

0: SYNC warning (SYNCWARNF) interrupt disabled

1: SYNC warning (SYNCWARNF) interrupt enabled

Bit 0 SYNCOKF: SYNC event OK interrupt enable

0: SYNC event OK (SYNCOKF) interrupt disabled

1: SYNC event OK (SYNCOKF) interrupt enabled

12.7.2 CRS configuration register (CRS_CFGR)

This register can be written only when the frequency error counter is disabled (the CEN bit is cleared in CRS_CR). When the counter is enabled, this register is write-protected.

Address offset: 0x04

Reset value: 0x2022 BB7F

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
SYNCPOL	Res.	SYNCSRC[1:0]		Res.	SYNCDIV[2:0]			FELIM[7:0]							
rw		rw	rw		rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
RELOAD[15:0]															
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit 31 **SYNCPOL**: SYNC polarity selection

This bit is set and cleared by software to select the input polarity for the SYNC signal source.

0: SYNC active on rising edge (default)

1: SYNC active on falling edge

Bit 30 Reserved, must be kept at reset value.

Bits 29:28 **SYNCSRC[1:0]**: SYNC signal source selection

These bits are set and cleared by software to select the SYNC signal source (see [Section 12.4.2: CRS internal signals](#)).

00: crs_sync_in_0 selected as SYNC signal source

01: crs_sync_in_1 selected as SYNC signal source

10: crs_sync_in_2 selected as SYNC signal source

11: crs_sync_in_3 selected as SYNC signal source

Note: When using USB LPM (link power management) and the device is in Sleep mode, the periodic USB SOF is not generated by the host. No SYNC signal is therefore provided to the CRS to calibrate the 48 MHz clock on the run. To guarantee the required clock precision after waking up from Sleep mode, the LSE clock or the SYNC pin must be used as SYNC signal.

Bit 27 Reserved, must be kept at reset value.

Bits 26:24 **SYNCDIV[2:0]**: SYNC divider

These bits are set and cleared by software to control the division factor of the SYNC signal.

000: SYNC not divided (default)

001: SYNC divided by 2

010: SYNC divided by 4

011: SYNC divided by 8

100: SYNC divided by 16

101: SYNC divided by 32

110: SYNC divided by 64

111: SYNC divided by 128

Bits 23:16 **FELIM[7:0]**: Frequency error limit

FELIM contains the value to be used to evaluate the captured frequency error value latched in the FECAP[15:0] bits of the CRS_ISR register. Refer to [Section 12.4.5](#) for more details about FECAP evaluation.

Bits 15:0 **RELOAD[15:0]**: Counter reload value

RELOAD is the value to be loaded in the frequency error counter with each SYNC event.
Refer to [Section 12.4.4](#) for more details about counter behavior.

12.7.3 CRS interrupt and status register (CRS_ISR)

Address offset: 0x08

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
FECAP[15:0]															
r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	r
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
FEDIR	Res.	Res.	Res.	Res.	TRIM OVF	SYNC MISS	SYNC ERR	Res.	Res.	Res.	Res.	ESYNCF	ERRF	SYNC WARNF	SYNC OKF
r					r	r	r					r	r	r	r

Bits 31:16 **FECAP[15:0]**: Frequency error capture

FECAP is the frequency error counter value latched in the time of the last SYNC event.
Refer to [Section 12.4.5](#) for more details about FECAP usage.

Bit 15 **FEDIR**: Frequency error direction

FEDIR is the counting direction of the frequency error counter latched in the time of the last SYNC event. It shows whether the actual frequency is below or above the target.

0: Up-counting direction, the actual frequency is above the target

1: Down-counting direction, the actual frequency is below the target

Bits 14:11 Reserved, must be kept at reset value.

Bit 10 **TRIMOVF**: Trimming overflow or underflow

This flag is set by hardware when the automatic trimming tries to over- or under-flow the TRIM value. An interrupt is generated if the ERRIE bit is set in the CRS_CR register. It is cleared by software by setting the ERRC bit in the CRS_ICR register.

0: No trimming error signaled

1: Trimming error signaled

Bit 9 **SYNCMISS**: SYNC missed

This flag is set by hardware when the frequency error counter reaches value FELIM * 128 and no SYNC is detected, meaning either that a SYNC pulse was missed, or the frequency error is too big (internal frequency too high) to be compensated by adjusting the TRIM value, hence some other action must be taken. At this point, the frequency error counter is stopped (waiting for a next SYNC), and an interrupt is generated if the ERRIE bit is set in the CRS_CR register. It is cleared by software by setting the ERRC bit in the CRS_ICR register.

0: No SYNC missed error signaled

1: SYNC missed error signaled

Bit 8 **SYNCERR**: SYNC error

This flag is set by hardware when the SYNC pulse arrives before the ESYNC event and the measured frequency error is greater than or equal to $FELIM * 128$. This means that the frequency error is too big (internal frequency too low) to be compensated by adjusting the TRIM value, and that some other action has to be taken. An interrupt is generated if the ERRIE bit is set in the CRS_CR register. It is cleared by software by setting the ERRC bit in the CRS_ICR register.

0: No SYNC error signaled

1: SYNC error signaled

Bits 7:4 Reserved, must be kept at reset value.

Bit 3 **ESYNCF**: Expected SYNC flag

This flag is set by hardware when the frequency error counter reached a zero value. An interrupt is generated if the ESYNCCIE bit is set in the CRS_CR register. It is cleared by software by setting the ESYNCC bit in the CRS_ICR register.

0: No expected SYNC signaled

1: Expected SYNC signaled

Bit 2 **ERRF**: Error flag

This flag is set by hardware in case of any synchronization or trimming error. It is the logical OR of the TRIMOVF, SYNCMISS and SYNCERR bits. An interrupt is generated if the ERRIE bit is set in the CRS_CR register. It is cleared by software in reaction to setting the ERRC bit in the CRS_ICR register, which clears the TRIMOVF, SYNCMISS and SYNCERR bits.

0: No synchronization or trimming error signaled

1: Synchronization or trimming error signaled

Bit 1 **SYNCWARNF**: SYNC warning flag

This flag is set by hardware when the measured frequency error is greater than or equal to $FELIM * 3$, but smaller than $FELIM * 128$. This means that to compensate the frequency error, the TRIM value must be adjusted by two steps or more. An interrupt is generated if the SYNCWARNIE bit is set in the CRS_CR register. It is cleared by software by setting the SYNCWARNC bit in the CRS_ICR register.

0: No SYNC warning signaled

1: SYNC warning signaled

Bit 0 **SYNCOKF**: SYNC event OK flag

This flag is set by hardware when the measured frequency error is smaller than $FELIM * 3$. This means that either no adjustment of the TRIM value is needed or that an adjustment by one trimming step is enough to compensate the frequency error. An interrupt is generated if the SYNCOKIE bit is set in the CRS_CR register. It is cleared by software by setting the SYNCOKC bit in the CRS_ICR register.

0: No SYNC event OK signaled

1: SYNC event OK signaled

12.7.4 CRS interrupt flag clear register (CRS_ICR)

Address offset: 0x0C

Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ESYNCC	ERRC	SYNCWARNC	SYNCOKC
												rw	rw	rw	rw

Bits 31:4 Reserved, must be kept at reset value.

Bit 3 **ESYNCC**: Expected SYNC clear flag

Writing 1 to this bit clears the ESYNCF flag in the CRS_ISR register.

Bit 2 **ERRC**: Error clear flag

Writing 1 to this bit clears TRIMOVF, SYNCMISS, and SYNCERR bits and consequently also the ERRF flag in the CRS_ISR register.

Bit 1 **SYNCWARNC**: SYNC warning clear flag

Writing 1 to this bit clears the SYNCWARNF flag in the CRS_ISR register.

Bit 0 **SYNCOKC**: SYNC event OK clear flag

Writing 1 to this bit clears the SYNCOKF flag in the CRS_ISR register.

12.7.5 CRS register map

Table 130. CRS register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0x00	CRS_CR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	TRIM[5:0]					SWSYNC	AUTOTRIMEN	CEN	Res.	ESYNCE	ERRIE	SYNCWARNIE	SYNCOKIE		
	Reset value	0x0000 2000																																
0x04	CRS_CFGR	SYNCPOL	Res.	SYNCSRC[1:0]		Res.	SYNCDIV[2:0]		FELIM[7:0]							RELOAD[15:0]																		
	Reset value	0		1	0		0	0	0	0	0	1	0	0	0	1	0	1	0	1	0	1	1	1	0	1	1	1	1	1	1	1	1	1
0x08	CRS_ISR	FECAP[15:0]																FEDIR	Res.	Res.	Res.	Res.	TRIMOVF	SYNCMISS	SYNCERR	Res.	Res.	Res.	Res.	ESYNCF	ERRF	SYNCWARNF	SYNCOKF	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0					0	0	0					0	0	0	0
0x0C	CRS_ICR	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	ESYNCC	ERRC	SYNCWARNC	SYNCOKC
	Reset value																													0	0	0	0	

Refer to [Section 2.3: Memory organization](#) for the register boundary addresses.